

Hitachi Single-Chip Microcomputer

H8/520

HD6475208

HD6435208

User's Manual (Hardware)

Preface

The H8/520 is a high-performance single-chip microcomputer featuring a high-speed CPU with 16-bit internal data paths and a full complement of on-chip supporting modules. The H8/520 is an ideal microcontroller for a wide variety of medium-scale devices, including both office and industrial equipment and consumer products.

Its instruction set is designed for fast execution of programs coded in the high-level C language.

On-chip facilities include large RAM and ROM memories, numerous timers, serial I/O, an A/D converter, I/O ports, and other functions for compact implementation of high-performance application systems.

The H8/520 is available in both a ZTAT™ version* with on-chip PROM, ideal for the early stages of production or for products with frequently-changing specifications, and a masked-ROM version suitable for volume production.

This manual gives a hardware description of the H8/520. For details of the instruction set, refer to the *H8/500 Series Programming Manual*, which applies to all chips in the H8/500 family.

Note: * ZTAT (Zero Turn-Around Time) is a registered trademark of Hitachi, Ltd.

Contents

Section 1	Overview	1
1.1	Features	1
1.2	Block Diagram	4
1.3	Pin Arrangements and Functions	5
1.3.1	Pin Arrangement	5
1.3.2	Pin Functions.....	8
Section 2	MCU Operating Modes and Address Space.....	23
2.1	Overview	23
2.2	Mode Descriptions	24
2.3	Address Space Map.....	25
2.3.1	Page Segmentation	25
2.3.2	Page 0 Address Allocations.....	27
2.4	Mode Control Register (MDCR)	29
Section 3	CPU	31
3.1	Overview	31
3.1.1	Features	31
3.1.2	Address Space	32
3.1.3	Register Configuration	33
3.2	CPU Register Descriptions	34
3.2.1	General Registers	34
3.2.2	Control Registers.....	35
3.2.3	Initial Register Values	40
3.3	Data Formats.....	41
3.3.1	Data Formats in General Registers.....	42
3.3.2	Data Formats in Memory	43
3.4	Instructions.....	44
3.4.1	Basic Instruction Formats.....	44
3.4.2	Addressing Modes.....	45
3.4.3	Effective Address Calculation.....	47
3.5	Instruction Set	49
3.5.1	Overview	49
3.5.2	Data Transfer Instructions	51
3.5.3	Arithmetic Instructions.....	53
3.5.4	Logic Operations	54
3.5.5	Shift Operations.....	55
3.5.6	Bit Manipulations	56
3.5.7	Branching Instructions	57
3.5.8	System Control Instructions	59
3.5.9	Short-Format Instructions	62
3.6	Operating Modes.....	62
3.6.1	Minimum Mode.....	62
3.6.2	Maximum Mode.....	63
3.7	Basic Operational Timing	63
3.7.1	Overview	63
3.7.2	On-Chip Memory Access Cycle	64
3.7.3	Pin States during On-Chip Memory Access	65

3.7.4	Register Field Access Cycle (Addresses H'FF80 to H'FFFF).....	65
3.7.5	Pin States during Register Field Access (Addresses H'FF80 to H'FFFF).....	66
3.7.6	External Access Cycle.....	66
3.8	CPU States	67
3.8.1	Overview	67
3.8.2	Program Execution State.....	69
3.8.3	Exception-Handling State	70
3.8.4	Reset State.....	70
3.8.5	Power-Down State.....	70
Section 4 Exception Handling		71
4.1	Overview.....	71
4.1.1	Types of Exception Handling and Their Priority	71
4.1.2	Hardware Exception-Handling Sequence	72
4.1.3	Exception Sources and Vector Table.....	72
4.2	Reset.....	75
4.2.1	Overview	75
4.2.2	Reset Sequence.....	75
4.2.3	Stack Pointer Initialization	76
4.3	Address Error	79
4.3.1	Instruction Prefetch from Illegal Address	79
4.3.2	Word Data Access at Odd Address	79
4.3.3	Off-Chip Address Access in Single-Chip Mode	79
4.4	Trace.....	80
4.5	Interrupts.....	80
4.6	Invalid Instruction.....	82
4.7	Trap Instructions and Zero Divide	83
4.8	Cases in Which Exception Handling is Deferred	83
4.8.1	Instructions that Disable Interrupts	84
4.8.2	Disabling of Exceptions Immediately after a Reset.....	84
4.8.3	Disabling of Interrupts after a Data Transfer Cycle	85
4.9	Stack Status after Completion of Exception Handling	86
4.9.1	PC Value Pushed on Stack for Trace, Interrupts, Trap Instructions, and Zero Divide Exceptions.....	88
4.9.2	PC Value Pushed on Stack for Address Error and Invalid Instruction Exceptions.....	88
4.10	Notes on Use of the Stack.....	88
Section 5 Interrupt Controller.....		89
5.1	Overview.....	89
5.1.1	Features	89
5.1.2	Block Diagram	90
5.1.3	Register Configuration	91
5.2	Interrupt Types	91
5.2.1	External Interrupts.....	91
5.2.2	Internal Interrupts.....	93
5.2.3	Interrupt Vector Table.....	94
5.3	Register Descriptions	96
5.3.1	Interrupt Priority Registers A to D (IPRA to IPRD).....	96
5.3.2	NMI Control Register (NMICR)—H'FFFC.....	97
5.3.3	IRQ Control Register (IRQCR)—H'FFFD	97
5.4	Interrupt Handling Sequence.....	100

5.4.1	Interrupt-Handling Flow	100
5.4.2	Stack Status after Interrupt Exception-Handling Sequence	103
5.4.3	Timing of Interrupt Exception-Handling Sequence	104
5.5	Interrupts During Operation of the Data Transfer Controller	104
5.6	Interrupt Response Time	107
Section 6 Data Transfer Controller.....		109
6.1	Overview	109
6.1.1	Features	109
6.1.2	Block Diagram	109
6.1.3	Register Configuration	110
6.2	Register Descriptions	111
6.2.1	Data Transfer Mode Register (DTMR)	111
6.2.2	Data Transfer Source Address Register (DTSR)	112
6.2.3	Data Transfer Destination Register (DTDR).....	112
6.2.4	Data Transfer Count Register (DTCR)	112
6.2.5	Data Transfer Enable Registers A to D (DTEA to DTED).....	113
6.3	Data Transfer Operation.....	114
6.3.1	Data Transfer Cycle.....	114
6.3.2	DTC Vector Table.....	116
6.3.3	Location of Register Information in Memory	118
6.3.4	Length of Data Transfer Cycle	118
6.4	Procedure for Using the DTC	120
6.5	Example	121
Section 7 Wait-State Controller.....		125
7.1	Overview	125
7.1.1	Features	125
7.1.2	Block Diagram	126
7.1.3	Register Configuration	126
7.2	Wait-State Control Register	127
7.3	Operation in Each Wait Mode.....	128
7.3.1	Programmable Wait Mode.....	128
7.3.2	Pin Wait Mode.....	129
7.3.3	Pin Auto-Wait Mode	131
Section 8 Clock Pulse Generator		133
8.1	Overview	133
8.1.1	Block Diagram	133
8.2	Oscillator Circuit.....	133
8.3	System Clock Divider	136
Section 9 I/O Ports.....		137
9.1	Overview	137
9.2	Port 1.....	141
9.2.1	Overview	141
9.2.2	Port 1 Registers	143
9.2.3	Pin Functions in Each Mode	144
9.3	Port 2.....	152
9.3.1	Overview	152
9.3.2	Port 2 Registers	152

9.4	Port 3.....	155
9.4.1	Overview	155
9.4.2	Port 3 Registers	155
9.4.3	Pin Functions in Each Mode	157
9.4.4	Built-In MOS Pull-Up.....	158
9.5	Port 4.....	162
9.5.1	Overview	162
9.5.2	Port 4 Registers	163
9.5.3	Pin Functions in Each Mode	165
9.5.4	Built-in MOS Pull-Up	170
9.6	Port 5.....	171
9.6.1	Overview	171
9.6.2	Port 5 Registers	171
9.6.3	Pin Functions.....	173
9.7	Port 6.....	176
9.7.1	Overview	176
9.7.2	Port 6 Registers	177
9.8	Port 7.....	178
9.8.1	Overview	178
9.8.2	Port 7 Registers	180
9.8.3	Pin Functions.....	181
Section 10 16-Bit Free-Running Timers.....		189
10.1	Overview.....	189
10.1.1	Features	189
10.1.2	Block Diagram	190
10.1.3	Input and Output Pins.....	191
10.1.4	Register Configuration	192
10.2	Register Descriptions	193
10.2.1	Free-Running Counter (FRC)—H'FF92, H'FFA2.....	193
10.2.2	Output Compare Registers A and B (OCRA and OCRB)—H'FF94 and H'FF96, H'FFA4 and H'FFA6	193
10.2.3	Input Capture Register (ICR)—H'FF98, H'FFA8	194
10.2.4	Timer Control Register (TCR)—H'FF90, H'FFA0.....	195
10.2.5	Timer Control/Status Register (TCSR)—H'FF91, H'FFA1	197
10.3	CPU Interface.....	199
10.4	Operation.....	201
10.4.1	FRC Incrementation Timing	201
10.4.2	Output Compare Timing	202
10.4.3	Input Capture Timing	204
10.4.4	Setting of FRC Overflow Flag (OVF).....	206
10.5	CPU Interrupts and DTC Interrupts.....	206
10.6	Synchronization of Free-Running Timers 1 and 2.....	207
10.6.1	Synchronization after a Reset.....	207
10.6.2	Synchronization by Writing to FRCs	207
10.7	Sample Application.....	211
10.8	Application Notes	211

Section 11	8-Bit Timer.....	217
11.1	Overview.....	217
11.1.1	Features	217
11.1.2	Block Diagram	218
11.1.3	Input and Output Pins.....	219
11.1.4	Register Configuration	219
11.2	Register Descriptions	219
11.2.1	Timer Counter (TCNT)—H'FFD4	219
11.2.2	Time Constant Registers A and B (TCORA and TCORB)—H'FFD2 and H'FFD3.....	220
11.2.3	Timer Control Register (TCR)—H'FFD0	220
11.2.4	Timer Control/Status Register (TCSR)	222
11.3	Operation.....	224
11.3.1	TCNT Incrementation Timing.....	224
11.3.2	Compare Match Timing	226
11.3.3	External Reset of TCNT.....	227
11.3.4	Setting of TCNT Overflow Flag	228
11.4	CPU Interrupts and DTC Interrupts	229
11.5	Sample Application.....	230
11.6	Application Notes	231
Section 12	Watchdog Timer	237
12.1	Overview.....	237
12.1.1	Features	237
12.1.2	Block Diagram	238
12.1.3	Register Configuration	239
12.2	Register Descriptions	239
12.2.1	Timer Counter (TCNT)—H'FFED	239
12.2.2	Timer Control/Status Register (TCSR)—H'FFEC (Read), H'FFED (Write)	240
12.2.3	Reset Control/Status Register (RSTCSR)—H'FFFF (Read), H'FFFE (Write)	241
12.2.4	Notes on Register Access	242
12.3	Operation.....	244
12.3.1	Watchdog Timer Mode.....	244
12.3.2	Interval Timer Mode	246
12.3.3	Operation in Software Standby Mode.....	246
12.3.4	Setting of Overflow Flag.....	247
12.3.5	Setting of Watchdog Timer Reset (WRST) Bit.....	248
12.4	Application Notes	248
Section 13	Serial Communication Interface	251
13.1	Overview.....	251
13.1.1	Features	251
13.1.2	Block Diagram	252
13.1.3	Input and Output Pins.....	253
13.1.4	Register Configuration	253
13.2	Register Descriptions	254
13.2.1	Receive Shift Register (RSR).....	254
13.2.2	Receive Data Register (RDR)—H'FFDD (Channel 1), H'FFC5 (Channel 2).....	254
13.2.3	Transmit Shift Register (TSR)	254
13.2.4	Transmit Data Register (TDR)—H'FFDB (Channel 1), H'FFC3 (Channel 2).....	255
13.2.5	Serial Mode Register (SMR)—H'FFD8 (Channel 1), H'FFC0 (Channel 2)	255
13.2.6	Serial Control Register (SCR)—H'FFD4 (Channel 1), H'FFC4 (Channel 2)	255

13.2.7	Serial Status Register (SSR)—H'FFDC (Channel 1), H'FFC4 (Channel 2)	259
13.2.8	Bit Rate Register (BRR)—H'FFD9 (Channel 1), H'FFC1 (Channel 2)	261
13.3	Operation.....	267
13.3.1	Overview	267
13.3.2	Asynchronous Mode	268
13.3.3	Synchronous Mode.....	272
13.4	CPU Interrupts and DTC Interrupts	276
13.5	Application Notes	277
Section 14 A/D Converter		281
14.1	Overview.....	281
14.1.1	Features	281
14.1.2	Block Diagram	282
14.1.3	Input Pins.....	283
14.1.4	Register Configuration	284
14.2	Register Descriptions	284
14.2.1	A/D Data Registers (ADDR)—H'FFE0 to H'FFE7	284
14.2.2	A/D Control/Status Register (ADCSR)—H'FFE8	285
14.2.3	A/D Control Register (ADCR)—H'FFE9	287
14.3	CPU Interface.....	288
14.4	Operation.....	289
14.4.1	Single Mode	290
14.4.2	Scan Mode.....	293
14.4.3	Input Sampling Time and A/D Conversion Time	295
14.4.4	External Triggering of A/D Conversion.....	297
14.5	Interrupts and the Data Transfer Controller	297
Section 15 RAM.....		299
15.1	Overview.....	299
15.1.1	Block Diagram	299
15.1.2	Register Configuration	300
15.2	RAM Control Register (RAMCR).....	300
15.3	Operation.....	300
15.3.1	Expanded Modes (Modes 1, 2, 3 and 4).....	300
15.3.2	Single-Chip Mode (Mode 7)	301
Section 16 ROM.....		303
16.1	Overview.....	303
16.1.1	Block Diagram	303
16.2	PROM Mode.....	304
16.2.1	PROM Mode Setup	304
16.2.2	Socket Adapter Pin Arrangements and Memory Map	305
16.3	Programming.....	307
16.3.1	Writing and Verifying.....	307
16.3.2	Notes on Writing	310
16.3.3	Reliability of Written Data	311
16.3.4	Erasing of Data.....	312
16.4	Handling of Windowed Packages	312
Section 17 Power-Down State		313
17.1	Overview.....	313

17.2.1	Transition to Sleep Mode	314
17.2.2	Exit from Sleep Mode	314
17.3	Software Standby Mode.....	314
17.3.1	Transition to Software Standby Mode.....	314
17.3.2	Software Standby Control Register (SBYCR).....	315
17.3.3	Exit from Software Standby Mode.....	316
17.3.4	Sample Application of Software Standby Mode.....	316
17.3.5	Application Notes.....	317
17.4	Hardware Standby Mode	317
17.4.1	Transition to Hardware Standby Mode	317
17.4.2	Recovery from Hardware Standby Mode.....	318
17.4.3	Timing Sequence of Hardware Standby Mode	318
Section 18 Electrical Specifications.....		321
18.1	Absolute Maximum Ratings	321
18.2	Electrical Characteristics	321
18.2.1	DC Characteristics.....	321
18.2.2	AC Characteristics.....	324
18.2.3	A/D Converter Characteristics	327
18.3	MCU Operational Timing.....	328
18.3.1	Bus Timing.....	328
18.3.2	Control Signal Timing.....	330
18.3.3	Clock Oscillator Stabilization Timing.....	331
18.3.4	I/O Port Timing	332
18.3.5	16-Bit Free-Running Timer Timing	333
18.3.6	8-Bit Timer Timing	334
18.3.7	Serial Communication Interface Timing.....	335
18.3.8	A/D External Trigger Input Timing	335
Appendix A Instructions		337
A.1	Instruction Set	337
A.2	Instruction Codes	342
A.3	Operation Code Map.....	353
A.4	Instruction Execution Cycles	358
A.4.1	Calculation of Instruction Execution States	358
A.4.2	Tables of Instruction Execution Cycles.....	359
Appendix B Register Field		367
B.1	Register Addresses and Bit Names	367
B.2	Register Descriptions	372
Appendix C I/O Port Schematic Diagrams		405
C.1	Schematic Diagram of Port 1	405
C.2	Schematic Diagram of Port 2	410
C.3	Schematic Diagram of Port 3.....	411
C.4	Schematic Diagram of Port 4.....	412
C.5	Schematic Diagram of Port 5.....	414
C.6	Schematic Diagram of Port 6.....	421
C.7	Schematic Diagram of Port 7.....	422
Appendix D Memory Map		426

Appendix E Pin States 427

E.1 Port State of Each Pin State 427

E.2 Pin Status in the Reset State..... 431

Appendix F Package Dimensions..... 438

Section 1 Overview

1.1 Features

The H8/520 is an original Hitachi CMOS microcomputer unit (MCU) comprising a high-performance CPU core plus a full range of supporting functions—an entire system integrated onto a single chip.

The CPU features a highly orthogonal instruction set that permits addressing modes and data sizes to be specified independently in each instruction. An internal 16-bit architecture and 16-bit access to on-chip memory enhance the CPU's data-processing capability and provide the speed needed for realtime control applications. The address space can be expanded to perform high-volume data processing.

The on-chip supporting functions include RAM, ROM, timers, a serial communication interface (SCI), A/D converter, and I/O ports. An on-chip data transfer controller (DTC) provides an efficient way to transfer data in either direction between memory and I/O.

For the on-chip ROM, a choice is offered between masked ROM and programmable ROM (PROM). The PROM version can be programmed by the user with a general-purpose PROM writer.

Table 1-1 lists the main features of the H8/520 chip.

Table 1-1 Features

Feature	Description
CPU	General-register machine • Eight 16-bit general registers • Five 8-bit and two 16-bit control registers High speed • Maximum clock rate: 10 MHz (oscillator frequency: 20 MHz) Expanded operating modes supporting external memory • Minimum mode: up to 64-kbyte address space • Maximum mode: up to 1-Mbyte address space Highly orthogonal instruction set • Addressing modes and data size can be specified independently for each instruction Instructions can address registers or memory • Register-register operations • Register-memory operations Instruction set optimized for C language • Special short formats for frequently-used instructions and addressing modes
Memory	• 512-Byte high-speed RAM on-chip • 16-kbyte programmable or masked ROM on-chip
16-Bit free-running timer (2 channels)	Each channel provides: • 1 free-running counter (which can count external events) • 2 output-compare registers • 1 input capture register
8-Bit timer (1 channel)	• One 8-bit up-counter (which can count external events) • 2 time constant registers
Watchdog timer (1 channel)	• An overflow generates a reset • Can output an external reset signal • Can also be used as an interval timer

Table 1-1 Features (cont)

Feature	Description																								
Serial communication interface (SCI) (2 channels)	- Asynchronous or synchronous mode (selectable) - Full duplex: can send and receive simultaneously - Built-in baud rate generator																								
A/D converter	10-Bit resolution 4 (or 8*) channels, controllable in single mode or scan mode (selectable) - Sample-and-hold function - Can be externally triggered																								
I/O ports	46 input/output pins (five 8-bit ports, one 6-bit port) 4 (or 8*) input-only pins (one 4- or 8*-bit port)																								
Interrupt controller (INTC)	9 external interrupt pins (NMI, $\overline{\text{IRQ}}_0$ to $\overline{\text{IRQ}}_7$) 18 internal interrupts 8 priority levels																								
Data transfer controller (DTC)	Performs efficient, rapid, bidirectional data transfer between memory and I/O with minimal CPU programming.																								
Wait-state controller (WSC)	Can insert wait states in access to external memory or I/O																								
Operating modes	5 MCU operating modes - Expanded minimum modes, supporting up to 64 kbytes external memory with or without using on-chip ROM (Modes 1 and 2) - Expanded maximum modes, supporting up to 1 Mbyte external memory with or without using on-chip ROM (Modes 3 and 4) - Single-chip mode (Mode 7) 3 power-down modes - Sleep mode - Software standby mode - Hardware standby mode																								
Other features	Clock generator on-chip																								
Product line-up	<table><tr><th>Model Name</th><th>Package Options</th><th>ROM</th></tr><tr><td>HD6475208C</td><td>64-Pin windowed shrink DIP (DC-64S)</td><td>PROM</td></tr><tr><td>HD6475208P</td><td>64-Pin shrink DIP (DP-64S)</td><td></td></tr><tr><td>HD6475208CP</td><td>68-Pin PLCC (CP-68)</td><td></td></tr><tr><td>HD6475208F</td><td>64-Pin QFP (FP-64A)</td><td></td></tr><tr><td>HD6435208P</td><td>64-Pin shrink DIP (DP-64S)</td><td>Masked</td></tr><tr><td>HD6435208CP</td><td>68-Pin PLCC (CP-68)</td><td>ROM</td></tr><tr><td>HD6435208F</td><td>64-Pin QFP (FP-64A)</td><td></td></tr></table>	Model Name	Package Options	ROM	HD6475208C	64-Pin windowed shrink DIP (DC-64S)	PROM	HD6475208P	64-Pin shrink DIP (DP-64S)		HD6475208CP	68-Pin PLCC (CP-68)		HD6475208F	64-Pin QFP (FP-64A)		HD6435208P	64-Pin shrink DIP (DP-64S)	Masked	HD6435208CP	68-Pin PLCC (CP-68)	ROM	HD6435208F	64-Pin QFP (FP-64A)	
Model Name	Package Options	ROM																							
HD6475208C	64-Pin windowed shrink DIP (DC-64S)	PROM																							
HD6475208P	64-Pin shrink DIP (DP-64S)																								
HD6475208CP	68-Pin PLCC (CP-68)																								
HD6475208F	64-Pin QFP (FP-64A)																								
HD6435208P	64-Pin shrink DIP (DP-64S)	Masked																							
HD6435208CP	68-Pin PLCC (CP-68)	ROM																							
HD6435208F	64-Pin QFP (FP-64A)																								

Note: * CP- 68 package only.

1.2 Block Diagram

Figure 1-1 shows a block diagram of the H8/520 chip.

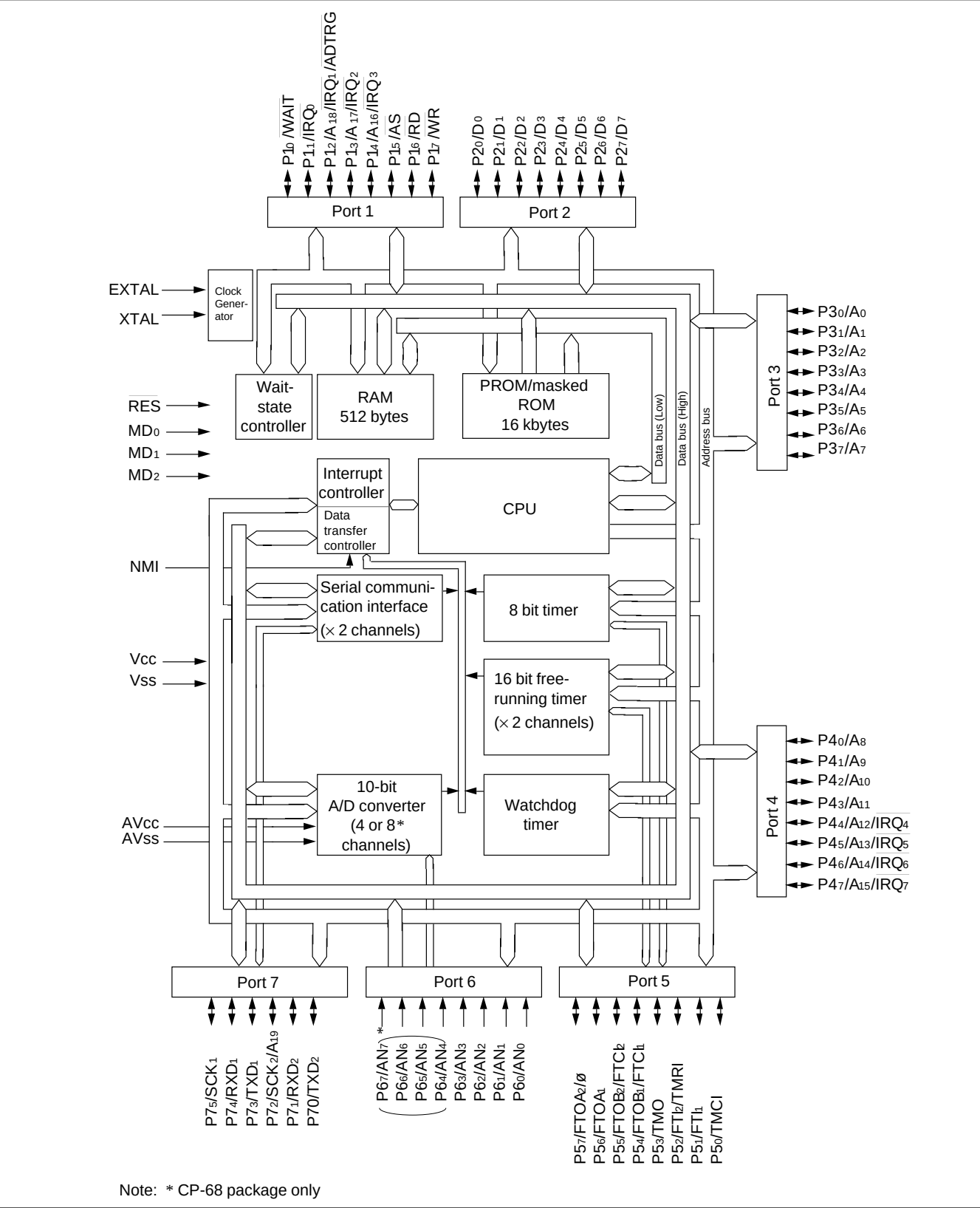


Figure 1-1 Block Diagram

1.3 Pin Arrangements and Functions

1.3.1 Pin Arrangement

Figure 1-2 shows the pin arrangement of the DC-64S and DP-64S packages. Figure 1-3 shows the pin arrangement of the FP-64A package. Figure 1-4 shows the pin arrangement of the CP-68 package.

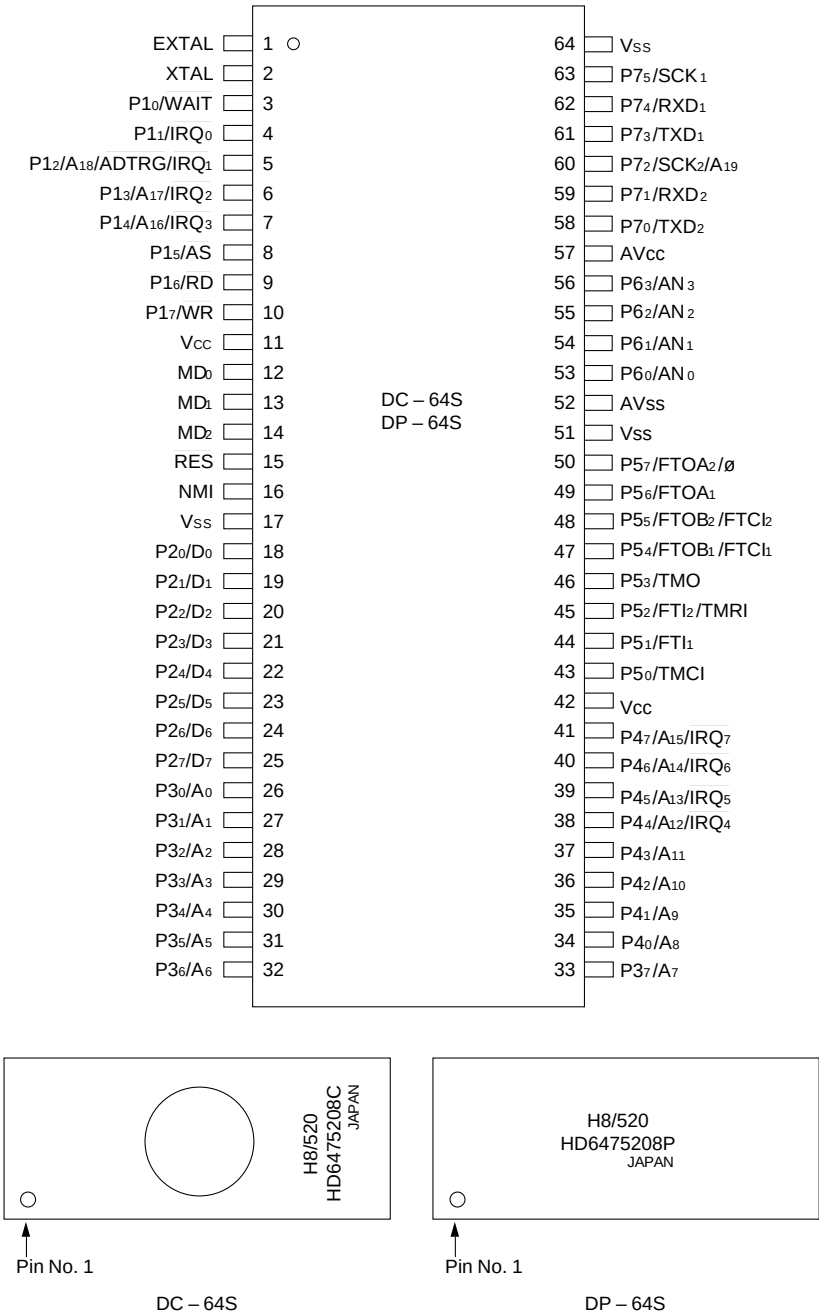
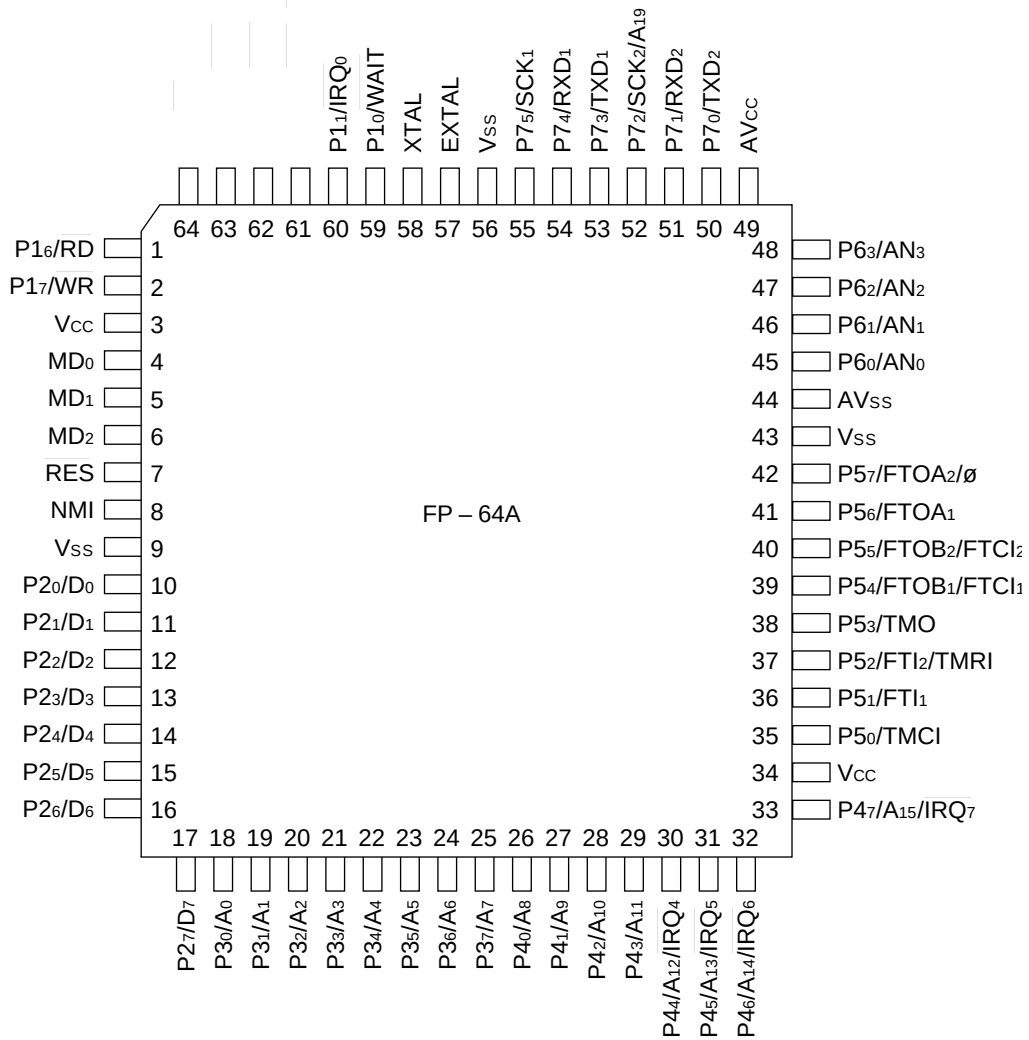


Figure 1-2 Pin Arrangement (DC-64S, DP-64S, Top View)



Pin No. 1 →

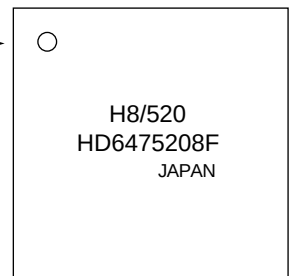
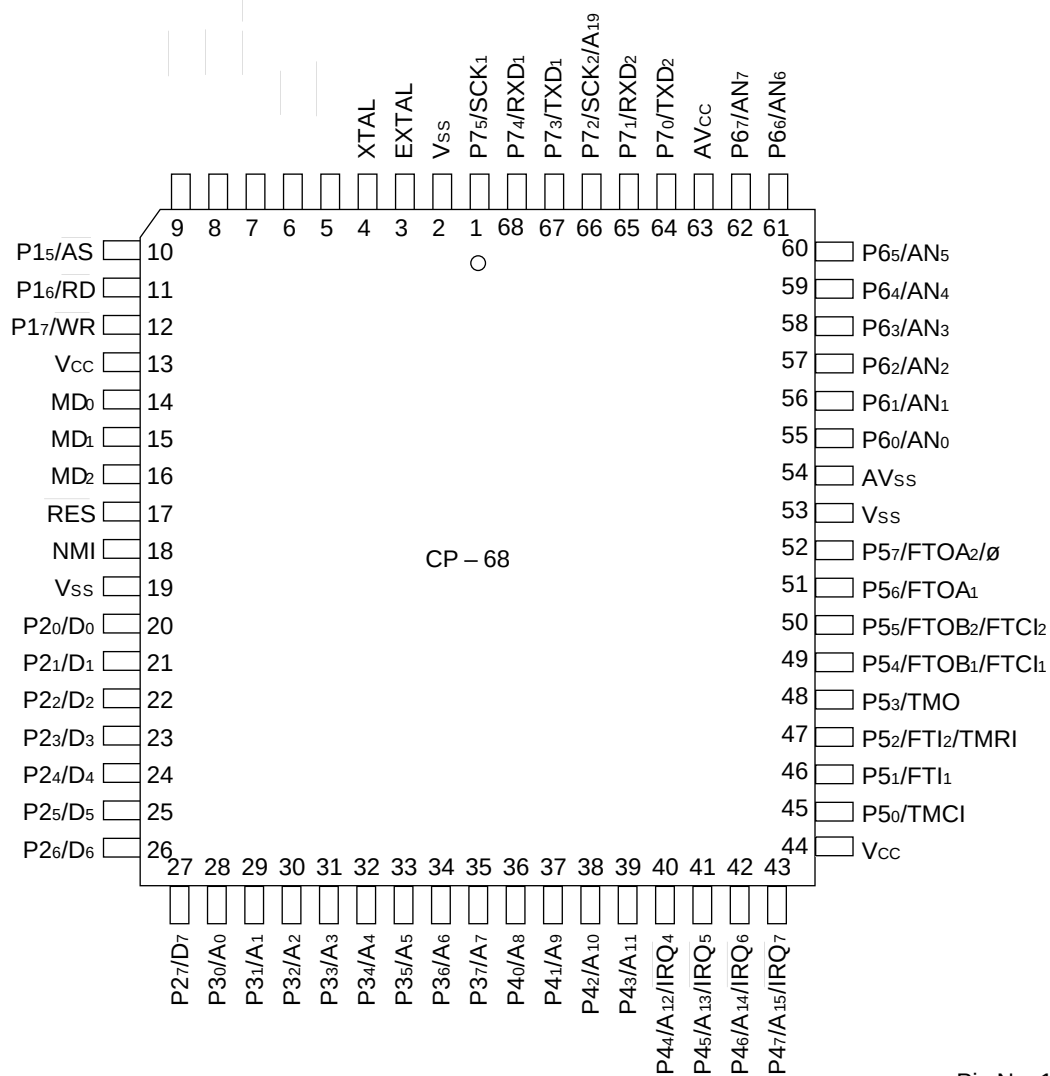


Figure 1-3 Pin Arrangement (FP-64A, Top View)



Pin No. 1



H8/520
HD6475208CP
JAPAN

Figure 1-4 Pin Arrangement (CP-68, Top View)

1.3.2 Pin Functions

Pin Arrangements in Each Operating Mode: Table 1-2 lists the arrangements of the pins of the DC-64S and DP-64S packages in each operating mode. Table 1-3 lists the arrangements for the FP-64A package. Table 1-4 lists the arrangements for the CP-68 package.

Table 1-2 Pin Arrangements in Each Operating Mode (DC-64S , DP-64S)

Pin Name						
Pin No.	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	Mode
	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
1	EXTAL	EXTAL	EXTAL	EXTAL	EXTAL	NC
2	XTAL	XTAL	XTAL	XTAL	XTAL	NC
3	P10 / $\overline{\text{WAIT}}$	P10 / $\overline{\text{WAIT}}$	P10 / $\overline{\text{WAIT}}$	P10 / $\overline{\text{WAIT}}$	P10	NC
4	P11 / $\overline{\text{IRQ}}_0$	P11 / $\overline{\text{IRQ}}_0$	P11 / $\overline{\text{IRQ}}_0$	P11 / $\overline{\text{IRQ}}_0$	P11 / $\overline{\text{IRQ}}_0$	NC
5	P12 / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	P12 / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	A18	P12 / A18 / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	P12 / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	NC
6	P13 / $\overline{\text{IRQ}}_2$	P13 / $\overline{\text{IRQ}}_2$	A17	P13 / A17 / $\overline{\text{IRQ}}_2$	P13 / $\overline{\text{IRQ}}_2$	NC
7	P14 / $\overline{\text{IRQ}}_3$	P14 / $\overline{\text{IRQ}}_3$	A16	P14 / A16 / $\overline{\text{IRQ}}_3$	P14 / $\overline{\text{IRQ}}_3$	NC
8	$\overline{\text{AS}}$	$\overline{\text{AS}}$	$\overline{\text{AS}}$	$\overline{\text{AS}}$	P15	NC
9	$\overline{\text{RD}}$	$\overline{\text{RD}}$	$\overline{\text{RD}}$	$\overline{\text{RD}}$	P16	NC
10	$\overline{\text{WR}}$	$\overline{\text{WR}}$	$\overline{\text{WR}}$	$\overline{\text{WR}}$	P17	NC
11	VCC	VCC	VCC	VCC	VCC	VCC
12	MD0	MD0	MD0	MD0	MD0	VCC
13	MD1	MD1	MD1	MD1	MD1	VSS
14	MD2	MD2	MD2	MD2	MD2	VCC
15	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	VPP
16	NMI	NMI	NMI	NMI	NMI	A9
17	VSS	VSS	VSS	VSS	VSS	VSS
18	D0	D0	D0	D0	P20	00
19	D1	D1	D1	D1	P21	01
20	D2	D2	D2	D2	P22	02
21	D3	D3	D3	D3	P23	03
22	D4	D4	D4	D4	P24	04

Notes: 1. For the PROM mode, see section 16, “ROM.”
2. Pins marked NC should be left unconnected.

Table 1-2 Pin Arrangements in Each Operating Mode (DC-64S, DP-64S) (cont)

Pin Name						
Pin	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	
	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
23	D5	D5	D5	D5	P25	O5
24	D6	D6	D6	D6	P26	O6
25	D7	D7	D7	D7	P27	O7
26	A0	A0	A0	A0	P30	A0
27	A1	A1	A1	A1	P31	A1
28	A2	A2	A2	A2	P32	A2
29	A3	A3	A3	A3	P33	A3
30	A4	A4	A4	A4	P34	A4
31	A5	A5	A5	A5	P35	A5
32	A6	A6	A6	A6	P36	A6
33	A7	A7	A7	A7	P37	A7
34	A8	P40 / A8	A8	P40 / A8	P40	A8
35	A9	P41 / A9	A9	P41 / A9	P41	OE
36	A10	P42 / A10	A10	P42 / A10	P42	A10
37	A11	P43 / A11	A11	P43 / A11	P43	A11
38	A12	P44 / A12 / $\overline{\text{IRQ}}_4$	A12	P44 / A12 / $\overline{\text{IRQ}}_4$	P44 / $\overline{\text{IRQ}}_4$	A12
39	A13	P45 / A13 / $\overline{\text{IRQ}}_5$	A13	P45 / A13 / $\overline{\text{IRQ}}_5$	P45 / $\overline{\text{IRQ}}_5$	A13
40	A14	P46 / A14 / $\overline{\text{IRQ}}_6$	A14	P46 / A14 / $\overline{\text{IRQ}}_6$	P46 / $\overline{\text{IRQ}}_6$	A14
41	A15	P47 / A15 / $\overline{\text{IRQ}}_7$	A15	P47 / A15 / $\overline{\text{IRQ}}_7$	P47 / $\overline{\text{IRQ}}_7$	CE
42	VCC	VCC	VCC	VCC	VCC	VCC
43	P50 / TMCI	P50 / TMCI	P50 / TMCI	P50 / TMCI	P50 / TMCI	VCC
44	P51 / FTI1	P51 / FTI1	P51 / FTI1	P51 / FTI1	P51 / FTI1	VCC
45	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	NC
46	P53 / TMO	P53 / TMO	P53 / TMO	P53 / TMO	P53 / TMO	NC
47	P54 / FTOB1 / FTCI1	P54 / FTOB1 / FTCI1	P54 / FTOB1 / FTCI1	P54 / FTOB1 / FTCI1	P54 / FTOB1 / FTCI1	NC

Notes: 1. For the PROM mode, see section 16, “ROM.”
2. Pins marked NC should be left unconnected.

Table 1-2 Pin Arrangements in Each Operating Mode (DC-64S, DP-64S) (cont)

Pin Name						
Pin	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	Mode
No.	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
48	P5 ₅ / FTOB ₂ / FTCl ₂	P5 ₅ / FTOB ₂ / FTCl ₂	P5 ₅ / FTOB ₂ / FTCl ₂	P5 ₅ / FTOB ₂ / FTCl ₂	P5 ₅ / FTOB ₂ / FTCl ₂	NC
49	P5 ₆ / FTOA ₁	P5 ₆ / FTOA ₁	P5 ₆ / FTOA ₁	P5 ₆ / FTOA ₁	P5 ₆ / FTOA ₁	NC
50	P5 ₇ / FTOA ₂ / ∅	P5 ₇ / FTOA ₂ / ∅	P5 ₇ / FTOA ₂ / ∅	P5 ₇ / FTOA ₂ / ∅	P5 ₇ / FTOA ₂ / ∅	NC
51	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}
52	AV _{SS}	AV _{SS}	AV _{SS}	AV _{SS}	AV _{SS}	V _{SS}
53	P6 ₀ / AN ₀	P6 ₀ / AN ₀	P6 ₀ / AN ₀	P6 ₀ / AN ₀	P6 ₀ / AN ₀	NC
54	P6 ₁ / AN ₁	P6 ₁ / AN ₁	P6 ₁ / AN ₁	P6 ₁ / AN ₁	P6 ₁ / AN ₁	NC
55	P6 ₂ / AN ₂	P6 ₂ / AN ₂	P6 ₂ / AN ₂	P6 ₂ / AN ₂	P6 ₂ / AN ₂	NC
56	P6 ₃ / AN ₃	P6 ₃ / AN ₃	P6 ₃ / AN ₃	P6 ₃ / AN ₃	P6 ₃ / AN ₃	NC
57	AV _{CC}	AV _{CC}	AV _{CC}	AV _{CC}	AV _{CC}	V _{CC}
58	P7 ₀ / TXD ₂	P7 ₀ / TXD ₂	P7 ₀ / TXD ₂	P7 ₀ / TXD ₂	P7 ₀ / TXD ₂	NC
59	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	NC
60	P7 ₂ / SCK ₂	P7 ₂ / SCK ₂	A ₁₉	P7 ₂ / SCK ₂ / A ₁₉	P7 ₂ / SCK ₂	NC
61	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	NC
62	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	NC
63	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	NC
64	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}

Notes: 1. For the PROM mode, see section 16, “ROM.”
2. Pins marked NC should be left unconnected.

Table 1-3 Pin Arrangements in Each Operating Mode (FP-64A)

Pin No.	Pin Name					
	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	Mode
	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
1	$\overline{\text{RD}}$	$\overline{\text{RD}}$	$\overline{\text{RD}}$	$\overline{\text{RD}}$	P1 ₆	NC
2	$\overline{\text{WR}}$	$\overline{\text{WR}}$	$\overline{\text{WR}}$	$\overline{\text{WR}}$	P1 ₇	NC
3	V _{CC}	V _{CC}	V _{CC}	V _{CC}	V _{CC}	V _{CC}
4	MD ₀	MD ₀	MD ₀	MD ₀	MD ₀	V _{CC}
5	MD ₁	MD ₁	MD ₁	MD ₁	MD ₁	V _{SS}
6	MD ₂	MD ₂	MD ₂	MD ₂	MD ₂	V _{CC}
7	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	V _{PP}
8	NMI	NMI	NMI	NMI	NMI	A ₉
9	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}
10	D ₀	D ₀	D ₀	D ₀	P2 ₀	O ₀
11	D ₁	D ₁	D ₁	D ₁	P2 ₁	O ₁
12	D ₂	D ₂	D ₂	D ₂	P2 ₂	O ₂
13	D ₃	D ₃	D ₃	D ₃	P2 ₃	O ₃
14	D ₄	D ₄	D ₄	D ₄	P2 ₄	O ₄
15	D ₅	D ₅	D ₅	D ₅	P2 ₅	O ₅
16	D ₆	D ₆	D ₆	D ₆	P2 ₆	O ₆
17	D ₇	D ₇	D ₇	D ₇	P2 ₇	O ₇
18	A ₀	P3 ₀ /A ₀	A ₀	P3 ₀ /A ₀	P3 ₀	A ₀
19	A ₁	P3 ₁ /A ₁	A ₁	P3 ₁ /A ₁	P3 ₁	A ₁
20	A ₂	P3 ₂ /A ₂	A ₂	P3 ₂ /A ₂	P3 ₂	A ₂
21	A ₃	P3 ₃ /A ₃	A ₃	P3 ₃ /A ₃	P3 ₃	A ₃
22	A ₄	P3 ₄ /A ₄	A ₄	P3 ₄ /A ₄	P3 ₄	A ₄
23	A ₅	P3 ₅ /A ₅	A ₅	P3 ₅ /A ₅	P3 ₅	A ₅
24	A ₆	P3 ₆ /A ₆	A ₆	P3 ₆ /A ₆	P3 ₆	A ₆
25	A ₇	P3 ₇ /A ₇	A ₇	P3 ₇ /A ₇	P3 ₇	A ₇
26	A ₈	P4 ₀ / A ₈	A ₈	P4 ₀ / A ₈	P4 ₀	A ₈
27	A ₉	P4 ₁ / A ₉	A ₉	P4 ₁ / A ₉	P4 ₁	$\overline{\text{OE}}$
28	A ₁₀	P4 ₂ / A ₁₀	A ₁₀	P4 ₂ / A ₁₀	P4 ₂	A ₁₀
29	A ₁₁	P4 ₃ / A ₁₁	A ₁₁	P4 ₃ / A ₁₁	P4 ₃	A ₁₁

Notes: 1. For the PROM mode, see section 16, “ROM.”
2. Pins marked NC should be left unconnected.

Table 1-3 Pin Arrangements in Each Operating Mode (FP-64A) (cont)

Pin No.	Pin Name					
	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	Mode
	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
30	A12	P44 / A12 / $\overline{\text{IRQ}}_4$	A12	P44 / A12 / $\overline{\text{IRQ}}_4$	P44 / $\overline{\text{IRQ}}_4$	A12
31	A13	P45 / A13 / $\overline{\text{IRQ}}_5$	A13	P45 / A13 / $\overline{\text{IRQ}}_5$	P45 / $\overline{\text{IRQ}}_5$	A13
32	A14	P46 / A14 / $\overline{\text{IRQ}}_6$	A14	P46 / A14 / $\overline{\text{IRQ}}_6$	P46 / $\overline{\text{IRQ}}_6$	A14
33	A15	P47 / A15 / $\overline{\text{IRQ}}_7$	A15	P47 / A15 / $\overline{\text{IRQ}}_7$	P47 / $\overline{\text{IRQ}}_7$	$\overline{\text{CE}}$
34	VCC	VCC	VCC	VCC	VCC	VCC
35	P50 / TMCI	P50 / TMCI	P50 / TMCI	P50 / TMCI	P50 / TMCI	VCC
36	P51 / FTI1	P51 / FTI1	P51 / FTI1	P51 / FTI1	P51 / FTI1	VCC
37	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	NC
38	P53 / TMO	P53 / TMO	P53 / TMO	P53 / TMO	P53 / TMO	NC
39	P54 / FTOB1 / FTCI1	P54 / FTOB1 / FTCI1	P54 / FTOB1 / FTCI1	P54 / FTOB1 / FTCI1	P54 / FTOB1 / FTCI1	NC
40	P55 / FTOB2 / FTCI2	P55 / FTOB2 / FTCI2	P55 / FTOB2 / FTCI2	P55 / FTOB2 / FTCI2	P55 / FTOB2 / FTCI2	NC
41	P56 / FTOA1	P56 / FTOA1	P56 / FTOA1	P56 / FTOA1	P56 / FTOA1	NC
42	P57 / FTOA2 / $\emptyset$	P57 / FTOA2 / $\emptyset$	P57 / FTOA2 / $\emptyset$	P57 / FTOA2 / $\emptyset$	P57 / FTOA2 / $\emptyset$	NC
43	VSS	VSS	VSS	VSS	VSS	VSS
44	AVSS	AVSS	AVSS	AVSS	AVSS	VSS
45	P60 / AN0	P60 / AN0	P60 / AN0	P60 / AN0	P60 / AN0	NC
46	P61 / AN1	P61 / AN1	P61 / AN1	P61 / AN1	P61 / AN1	NC
47	P62 / AN2	P62 / AN2	P62 / AN2	P62 / AN2	P62 / AN2	NC
48	P63 / AN3	P63 / AN3	P63 / AN3	P63 / AN3	P63 / AN3	NC
49	AVCC	AVCC	AVCC	AVCC	AVCC	VCC
50	P70 / TXD2	P70 / TXD2	P70 / TXD2	P70 / TXD2	P70 / TXD2	NC

Notes: 1. For the PROM mode, see section 16, "ROM."
2. Pins marked NC should be left unconnected.

Table 1-3 Pin Arrangements in Each Operating Mode (FP-64A) (cont)

Pin Name						
Pin No.	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	
	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
51	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	NC
52	P7 ₂ / SCK ₂	P7 ₂ / SCK ₂	A ₁₉	P7 ₂ / SCK ₂ / A ₁₉	P7 ₂ / SCK ₂	NC
53	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	NC
54	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	NC
55	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	NC
56	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}
57	EXTAL	EXTAL	EXTAL	EXTAL	EXTAL	NC
58	XTAL	XTAL	XTAL	XTAL	XTAL	NC
59	P1 ₀ / $\overline{\text{WAIT}}$	P1 ₀ / $\overline{\text{WAIT}}$	P1 ₀ / $\overline{\text{WAIT}}$	P1 ₀ / $\overline{\text{WAIT}}$	P1 ₀	NC
60	P1 ₁ / $\overline{\text{IRQ}}_0$	P1 ₁ / $\overline{\text{IRQ}}_0$	P1 ₁ / $\overline{\text{IRQ}}_0$	P1 ₁ / $\overline{\text{IRQ}}_0$	P1 ₁ / $\overline{\text{IRQ}}_0$	NC
61	P1 ₂ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	P1 ₂ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	A ₁₈	P1 ₂ / A ₁₈ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	P1 ₂ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	NC
62	P1 ₃ / $\overline{\text{IRQ}}_2$	P1 ₃ / $\overline{\text{IRQ}}_2$	A ₁₇	P1 ₃ / A ₁₇ / $\overline{\text{IRQ}}_2$	P1 ₃ / $\overline{\text{IRQ}}_2$	NC
63	P1 ₄ / $\overline{\text{IRQ}}_3$	P1 ₄ / $\overline{\text{IRQ}}_3$	A ₁₆	P1 ₄ / A ₁₆ / $\overline{\text{IRQ}}_3$	P1 ₄ / $\overline{\text{IRQ}}_3$	NC
64	$\overline{\text{AS}}$	$\overline{\text{AS}}$	$\overline{\text{AS}}$	$\overline{\text{AS}}$	P1 ₅	NC

Notes: 1. For the PROM mode, see section 16, “ROM.”
2. Pins marked NC should be left unconnected.

Table 1-4 Pin Arrangements in Each Operating Mode (CP-68)

Pin Name						
Pin No.	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	
	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
1	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	P7 ₅ / SCK ₁	NC
2	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}
3	EXTAL	EXTAL	EXTAL	EXTAL	EXTAL	NC
4	XTAL	XTAL	XTAL	XTAL	XTAL	NC
5	P1 ₀ / $\overline{\text{WAIT}}$	P1 ₀ / $\overline{\text{WAIT}}$	P1 ₀ / $\overline{\text{WAIT}}$	P1 ₀ / $\overline{\text{WAIT}}$	P1 ₀	NC
6	P1 ₁ / $\overline{\text{IRQ}}_0$	P1 ₁ / $\overline{\text{IRQ}}_0$	P1 ₁ / $\overline{\text{IRQ}}_0$	P1 ₁ / $\overline{\text{IRQ}}_0$	P1 ₁ / $\overline{\text{IRQ}}_0$	NC
7	P1 ₂ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	P1 ₂ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	A ₁₈	P1 ₂ / A ₁₈ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	P1 ₂ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	NC
8	P1 ₃ / $\overline{\text{IRQ}}_2$	P1 ₃ / $\overline{\text{IRQ}}_2$	A ₁₇	P1 ₃ / A ₁₇ / $\overline{\text{IRQ}}_2$	P1 ₃ / $\overline{\text{IRQ}}_2$	NC
9	P1 ₄ / $\overline{\text{IRQ}}_3$	P1 ₄ / $\overline{\text{IRQ}}_3$	A ₁₆	P1 ₄ / A ₁₆ / $\overline{\text{IRQ}}_3$	P1 ₄ / $\overline{\text{IRQ}}_3$	NC
10	$\overline{\text{AS}}$	$\overline{\text{AS}}$	$\overline{\text{AS}}$	$\overline{\text{AS}}$	P1 ₅	NC
11	$\overline{\text{RD}}$	$\overline{\text{RD}}$	$\overline{\text{RD}}$	$\overline{\text{RD}}$	P1 ₆	NC
12	$\overline{\text{WR}}$	$\overline{\text{WR}}$	$\overline{\text{WR}}$	$\overline{\text{WR}}$	P1 ₇	NC
13	V _{CC}	V _{CC}	V _{CC}	V _{CC}	V _{CC}	V _{CC}
14	MD ₀	MD ₀	MD ₀	MD ₀	MD ₀	V _{CC}
15	MD ₁	MD ₁	MD ₁	MD ₁	MD ₁	V _{SS}
16	MD ₂	MD ₂	MD ₂	MD ₂	MD ₂	V _{CC}
17	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	$\overline{\text{RES}}$	V _{PP}
18	NMI	NMI	NMI	NMI	NMI	A ₉
19	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}
20	D ₀	D ₀	D ₀	D ₀	P2 ₀	0 ₀
21	D ₁	D ₁	D ₁	D ₁	P2 ₁	0 ₁
22	D ₂	D ₂	D ₂	D ₂	P2 ₂	0 ₂
23	D ₃	D ₃	D ₃	D ₃	P2 ₃	0 ₃
24	D ₄	D ₄	D ₄	D ₄	P2 ₄	0 ₄

Notes: 1. For the PROM mode, see section 16, “ROM.”
2. Pins marked NC should be left unconnected.

Table 1-4 Pin Arrangements in Each Operating Mode (CP-68) (cont)

Pin Name						
Pin	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	
	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
25	D5	D5	D5	D5	P25	O5
26	D6	D6	D6	D6	P26	O6
27	D7	D7	D7	D7	P27	O7
28	A0	A0	A0	A0	P30	A0
29	A1	A1	A1	A1	P31	A1
30	A2	A2	A2	A2	P32	A2
31	A3	A3	A3	A3	P33	A3
32	A4	A4	A4	A4	P34	A4
33	A5	A5	A5	A5	P35	A5
34	A6	A6	A6	A6	P36	A6
35	A7	A7	A7	A7	P37	A7
36	A8	P40 / A8	A8	P40 / A8	P40	A8
37	A9	P41 / A9	A9	P41 / A9	P41	OE
38	A10	P42 / A10	A10	P42 / A10	P42	A10
39	A11	P43 / A11	A11	P43 / A11	P43	A11
40	A12	P44 / A12 / $\overline{\text{IRQ}}_4$	A12	P44 / A12 / $\overline{\text{IRQ}}_4$	P44 / $\overline{\text{IRQ}}_4$	A12
41	A13	P45 / A13 / $\overline{\text{IRQ}}_5$	A13	P45 / A13 / $\overline{\text{IRQ}}_5$	P45 / $\overline{\text{IRQ}}_5$	A13
42	A14	P46 / A14 / $\overline{\text{IRQ}}_6$	A14	P46 / A14 / $\overline{\text{IRQ}}_6$	P46 / $\overline{\text{IRQ}}_6$	A14
43	A15	P47 / A15 / $\overline{\text{IRQ}}_7$	A15	P47 / A15 / $\overline{\text{IRQ}}_7$	P47 / $\overline{\text{IRQ}}_7$	CE
44	VCC	VCC	VCC	VCC	VCC	VCC
45	P50 / TMCI	P50 / TMCI	P50 / TMCI	P50 / TMCI	P50 / TMCI	VCC
46	P51 / FTI1	P51 / FTI1	P51 / FTI1	P51 / FTI1	P51 / FTI1	VCC
47	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	P52 / FTI2 / TMRI	NC

Notes: 1. For the PROM mode, see section 16, “ROM.”
2. Pins marked NC should be left unconnected.

Table 1-4 Pin Arrangements in Each Operating Mode (CP-68) (cont)

Pin Name						
Pin No.	Expanded Minimum		Expanded Maximum		Single-Chip	PROM
	Modes		Modes		Mode	Mode
	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7	Mode
48	P5 ₃ / TMO	P5 ₃ / TMO	P5 ₃ / TMO	P5 ₃ / TMO	P5 ₃ / TMO	NC
49	P5 ₄ / FTOB ₁ / FTCl ₁	P5 ₄ / FTOB ₁ / FTCl ₁	P5 ₄ / FTOB ₁ / FTCl ₁	P5 ₄ / FTOB ₁ / FTCl ₁	P5 ₄ / FTOB ₁ / FTCl ₁	NC
50	P5 ₅ / FTOB ₂ / FTCl ₂	P5 ₅ / FTOB ₂ / FTCl ₂	P5 ₅ / FTOB ₂ / FTCl ₂	P5 ₅ / FTOB ₂ / FTCl ₂	P5 ₅ / FTOB ₂ / FTCl ₂	NC
51	P5 ₆ / FTOA ₁	P5 ₆ / FTOA ₁	P5 ₆ / FTOA ₁	P5 ₆ / FTOA ₁	P5 ₆ / FTOA ₁	NC
52	P5 ₇ / FTOA ₂ / ∅	P5 ₇ / FTOA ₂ / ∅	P5 ₇ / FTOA ₂ / ∅	P5 ₇ / FTOA ₂ / ∅	P5 ₇ / FTOA ₂ / ∅	NC
53	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}	V _{SS}
54	AV _{SS}	AV _{SS}	AV _{SS}	AV _{SS}	AV _{SS}	V _{SS}
55	P6 ₀ / AN ₀	P6 ₀ / AN ₀	P6 ₀ / AN ₀	P6 ₀ / AN ₀	P6 ₀ / AN ₀	NC
56	P6 ₁ / AN ₁	P6 ₁ / AN ₁	P6 ₁ / AN ₁	P6 ₁ / AN ₁	P6 ₁ / AN ₁	NC
57	P6 ₂ / AN ₂	P6 ₂ / AN ₂	P6 ₂ / AN ₂	P6 ₂ / AN ₂	P6 ₂ / AN ₂	NC
58	P6 ₃ / AN ₃	P6 ₃ / AN ₃	P6 ₃ / AN ₃	P6 ₃ / AN ₃	P6 ₃ / AN ₃	NC
59	P6 ₄ / AN ₄	P6 ₄ / AN ₄	P6 ₄ / AN ₄	P6 ₄ / AN ₄	P6 ₄ / AN ₄	NC
60	P6 ₅ / AN ₅	P6 ₅ / AN ₅	P6 ₅ / AN ₅	P6 ₅ / AN ₅	P6 ₅ / AN ₅	NC
61	P6 ₆ / AN ₆	P6 ₆ / AN ₆	P6 ₆ / AN ₆	P6 ₆ / AN ₆	P6 ₆ / AN ₆	NC
62	P6 ₇ / AN ₇	P6 ₇ / AN ₇	P6 ₇ / AN ₇	P6 ₇ / AN ₇	P6 ₇ / AN ₇	NC
63	AV _{CC}	AV _{CC}	AV _{CC}	AV _{CC}	AV _{CC}	V _{CC}
64	P7 ₀ / TXD ₂	P7 ₀ / TXD ₂	P7 ₀ / TXD ₂	P7 ₀ / TXD ₂	P7 ₀ / TXD ₂	NC
65	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	P7 ₁ / RXD ₂	NC
66	P7 ₂ / SCK ₂	P7 ₂ / SCK ₂	A ₁₉	P7 ₂ / SCK ₂ / A ₁₉	P7 ₂ / SCK ₂	NC
67	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	P7 ₃ / TXD ₁	NC
68	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	P7 ₄ / RXD ₁	NC

Notes: 1. For the PROM mode, see section 16, "ROM."
2. Pins marked NC should be left unconnected.

Pin Functions: Table 1-5 gives a concise description of the function of each pin.

Table 1-5 Pin Functions

		Pin No.			I/O	Name and Function
		DC-64S	FP-64A	CP-68		
Type	Symbol	DP-64S				
Power	Vcc	42, 11	34, 3	44, 13	I	Power: Connected to the power supply (+5 V). Connect both Vcc pins to the system power supply (+5 V). The chip will not operate if either pin is left unconnected.
	Vss	51, 17, 64	43, 9, 56	53, 19, 2	I	Ground: Connected to ground (0 V). Connect all Vss pins to the system power supply (0 V). The chip will not operate if either pin is left unconnected.
Clock	XTAL	2	58	4	O	Crystal: Connected to a crystal oscillator. The crystal frequency should be double the desired $\varnothing$ clock frequency. If an external clock is input at the EXTAL pin, input an inverted clock signal at the XTAL pin.
	EXTAL	1	57	3	I	External Crystal: Connected to a crystal oscillator or external clock. The frequency of the external clock should be double the desired $\varnothing$ clock frequency. See section 8.2, “Oscillator Circuit”, for examples of connections to a crystal and external clock.
	$\varnothing$	50	42	52	O	System Clock: Supplies the $\varnothing$ clock to peripheral devices.

Table 1-5 Pin Functions (cont)

		Pin No.				
		DC-64S	FP-64A	CP-68		
Type	Symbol	DP-64S			I/O	Name and Function
System control	$\overline{\text{RES}}$	15	7	17	I/O	Reset: A low input causes the H8/520 chip to reset. If the reset output enable bit (RSTOE) is set to 1, when the watchdog timer overflows, a low signal is output for 132 system clock cycles.
Address bus	$A_{19} - A_0$	60, 5 – 7 41 – 26	52, 61 – 63 33 – 18	66, 7 – 9 43 – 28	O	Address Bus: Address output pins.
Data bus	$D_7 - D_0$	25 – 18	17 – 10	27 – 20	I/O	Data Bus: 8-Bit bidirectional data bus.
Bus control	$\overline{\text{WAIT}}$	3	59	5	I	Wait: Requests the CPU to insert one or more T_w states when accessing an off-chip address.
	$\overline{\text{AS}}$	8	64	10	O	Address Strobe: Goes low to indicate that there is a valid address on the address bus.
	$\overline{\text{RD}}$	9	1	11	O	Read: Goes low to indicate that the CPU is reading an external address.
	$\overline{\text{WR}}$	10	2	12	O	Write: Goes low to indicate that the CPU is writing to an external address.

Table 1-5 Pin Functions (cont)

		Pin No.				
		DC-64S	FP-64A	CP-68		
Type	Symbol	DP-64S			I/O	Name and Function
Interrupt	NMI	16	8	18	I	NonMaskable Interrupt: Highest-priority interrupt request signal. The non-maskable interrupt control register (NMICR) determines whether the interrupt is requested on the rising or falling edge of the NMI input.
	$\overline{\text{IRQ}}_0$	4	60	6	I	Interrupt Request 0 to 7: Maskable interrupt request signals
	$\overline{\text{IRQ}}_1$	5	61	7		
	$\overline{\text{IRQ}}_2$	6	62	8		
	$\overline{\text{IRQ}}_3$	7	63	9		
	$\overline{\text{IRQ}}_4$	38	30	40		
	$\overline{\text{IRQ}}_5$	39	31	41		
	$\overline{\text{IRQ}}_6$	40	32	42		
	$\overline{\text{IRQ}}_7$	41	33	43		
Operating mode control	MD2	14	6	16	I	Mode: Input pins for setting the MCU operating mode according to the table below
	MD1	13	5	15		
	MD0	12	4	14		
MD2 MD1 MD0 Mode Description						
0 0 0 Mode 0 —						
0 0 1 Mode 1 Expanded minimum mode (ROM disabled)						
0 1 0 Mode 2 Expanded minimum mode (ROM enabled)						
0 1 1 Mode 3 Expanded maximum mode (ROM disabled)						
1 0 0 Mode 4 Expanded maximum mode (ROM enabled)						
1 0 1 Mode 5 —						
1 1 0 Mode 6 Hardware standby mode						
1 1 1 Mode 7 Single-chip mode						
The inputs at these pins are latched in mode select bits 2 to 0 (MDS2 – MDS0) of the mode control register (MDCR) on the rising edge of the $\overline{\text{RES}}$ signal.						

Table 1-5 Pin Functions (cont)

		Pin No.				
		DC-64S	FP-64A	CP-68		
Type	Symbol	DP-64S			I/O	Name and Function
16-bit free-running timer (FRT)	FTOA ₁	49	41	51	O	FRT Output Compare A (channels 1 and 2):
	FTOA ₂	50	42	52		Output pins for the output compare A function of free-running timer channels 1 and 2.
	FTOB ₁	47	39	49	O	FRT Output Compare B (channels 1 and 2):
	FTOB ₂	48	40	50		Output pins for the output compare B function of free-running timer channels 1 and 2.
	FTCl ₁	47	39	49	I	FRT Counter Clock Input (channels 1 and 2):
	FTCl ₂	48	40	50		External clock input pins for the free-running counters (FRCs) of free-running timer channels 1 and 2.
	FTI ₁	44	36	46	I	FRT Input Capture (channels 1 and 2): Input
	FTI ₂	45	37	47		capture pins for free-running timer channels 1 and 2.
8-bit timer	TMO	46	38	48	O	8-bit Timer Output: Compare-match output pin for the 8-bit timer.
	TMCI	43	35	45	I	8-bit Timer Clock Input: External clock input pin for the 8-bit timer counter.
	TMRI	45	37	47	I	8-bit Timer Counter Reset Input: A high input at this pin resets the 8-bit timer counter.

Table 1-5 Pin Functions (cont)

Type	Symbol	Pin No.			I/O	Name and Function
		DC-64S	FP-64A	CP-68		
Serial com- munication interface signals	TXD ₁	61	53	67	O	Transmit Data (channels 1 and 2): Data output pins for serial communication interface channels 1 and 2.
	TXD ₂	58	50	64		
	RXD ₁	62	54	68	I	Receive Data (channels 1 and 2): Data input pins for serial communication interface channels 1 and 2.
	RXD ₂	59	51	65		
	SCK ₁	63	55	1	I/O	Serial Clock (channels 1 and 2): Input/output pins for the serial interface clock.
	SCK ₂	60	52	66		
A/D converter	AN ₃ – AN ₀	56 – 53	48 – 45	58 – 55	I	Analog Input: Analog signal input pins.
	AN ₇ – AN ₄ *			62 – 59		
	AV _{CC}	57	49	63	I	Analog Reference Voltage: Reference voltage pin for the A/D converter.
	AV _{SS}	52	44	54	I	
	ADTRG	5	61	7	I	A/D External Trigger: External trigger input pin for the A/D converter.
Parallel I/O	P ₁₇ – P ₁₀	10 – 3	2 – 1, 64 - 59	12 – 5	I/O	Port 1: An 8-bit input/output port. The direction of each bit is determined by the port 1 data direction register (P1DDR).
	P ₂₇ – P ₂₀	25 – 18	17 – 10	27 – 20	I/O	
	P ₃₇ – P ₃₀	33 – 26	25 – 18	35 – 28	I/O	Port 3: An 8-bit input/output port. The direction of each bit is determined by the port 3 data direction register (P3DDR). These pins have built-in MOS input pull-ups. They can drive LED indicators.
	P ₄₇ – P ₄₀	41 – 34	33 – 26	43 – 36	I/O	

Note: * CP-68 only

Table 1-5 Pin Functions (cont)

Type	Symbol	Pin No.			I/O	Name and Function
		DC-64S	FP-64A	CP-68		
Parallel I/O (cont)	P57 – P50	50 – 43	42 – 35	52 – 45	I/O	Port 5: An 8-bit input/output port. The direction of each bit is determined by the port 5 data direction register (P5DDR). These pins have Schmitt inputs.
	P63 – P60	56 – 53	48 – 45	58 – 55	I	Port 6: A 4-bit (or 8-bit*) input port.
	P67 – P64*			62 – 59		
	P75 – P70	63 – 58	55 – 50	2 – 1, 68 – 64,	I/O	Port 7: A 6-bit input/output port. The direction of each bit is determined by the port 7 data direction register (P7DDR).

Note: * CP-68 package only

Section 2 MCU Operating Modes and Address Space

2.1 Overview

The H8/520 microcomputer unit (MCU) operates in five modes numbered 1, 2, 3, 4, and 7. The mode is selected by the inputs at the mode pins (MD2 to MD0) at the instant when the chip comes out of a reset. As indicated in table 2-1, the MCU mode determines the size of the address space, the usage of on-chip ROM, and the operating mode of the CPU. The MCU mode also affects the functions of I/O pins.

Table 2-1 Operating Modes

MD2	MD1	MD0	MCU Mode	Address Space	On-Chip RAM	On-Chip ROM	CPU Mode
0	0	0	Mode 0	—	—	—	—
0	0	1	Mode 1	Expanded minimum	Enabled*	Disabled	Minimum mode
0	1	0	Mode 2	Expanded minimum	Enabled*	Enabled	Minimum mode
0	1	1	Mode 3	Expanded maximum	Enabled*	Disabled	Maximum mode
1	0	0	Mode 4	Expanded maximum	Enabled*	Enabled	Maximum mode
1	0	1	Mode 5	—	—	—	—
1	1	0	Mode 6	Hardware standby mode	—	—	—
1	1	1	Mode 7	Single-chip only	Enabled*	Enabled	Minimum mode

Notation: 0: Low level
1: High level
—: Cannot be used

Note: * On-chip RAM can be disabled by RAME bit to 0 in RAM control register (RAMCR).

Modes 1 to 4 are referred to as “expanded” because they permit access to off-chip memory and peripheral addresses. The expanded minimum modes (modes 1 and 2) support a maximum address space of 64 kbytes. The expanded maximum modes (modes 3 and 4) support a maximum address space of 1 Mbyte.

Interrupt service is slightly slower in the expanded maximum modes than in the other modes because the CPU has to save its code page register.

The H8/520 cannot be set to modes 0 and 5. The mode pins should never be set to these values. The hardware standby mode (mode 6) is a power-down mode, not an operating mode. See section 17.4, “Hardware Standby Mode” for details.

2.2 Mode Descriptions

The five MCU modes are described below. For further information on the I/O pin functions in each mode, see section 9, “I/O ports.”

Mode 1 (Expanded Minimum Mode): Mode 1 supports a maximum 64-kbyte address space which does not include any on-chip ROM. Ports 1 to 4 are used for bus lines and bus control signals as follows:

Control signals: Port 1 (partly)
Data bus: Port 2
Address bus: Ports 3 and 4

Mode 2 (Expanded Minimum Mode): Mode 2 supports a maximum 64-kbyte address space of which the first 16 kbytes are in on-chip ROM. Ports 1 to 4 are used for bus lines and bus control signals as follows:

Control signals: Port 1 (partly)
Data bus: Port 2
Address bus: Ports 3 and 4

Note: In mode 2, port 4 is initially a general-purpose input port. Software must change the desired pins to output before using them for the address bus. See section 9.5, “Port 4” for details. The following instruction makes all pins of port 4 into output pins:

```
MOV.B  #H'FF, @H'FF85
```

Mode 3 (Expanded Maximum Mode): Mode 3 supports a maximum 1-Mbyte address space which does not include any on-chip ROM. Ports 1 to 4 and one pin in port 7 are used for bus lines and bus control signals as follows:

Control signals: Port 1 (partly)
Data bus: Port 2
Address bus: Ports 1 (partly), 3, 4, and 7 (partly)

Mode 4 (Expanded Maximum Mode): Mode 4 supports a maximum 1-Mbyte address space of which the first 16 kbytes are in on-chip ROM. Ports 1 to 4 and one pin in port 7 are used for bus lines and bus control signals as follows:

Control signals: Port 1 (partly)

Data bus: Port 2

Address bus: Ports 1 (partly), 3, 4, and 7 (partly)

Note: In mode 4, port 4, pins 2 to 4 of port 1, and pin 2 of port 7 are initially used for general-purpose input. Software must change the desired pins to output before using them for the address bus. See section 9, “I/O Ports” for details.

Mode 7 (Single-Chip Mode): In this mode all memory is on-chip, in 16 kbytes of ROM and 512 bytes of RAM. It is not possible to access off-chip addresses.

The single-chip mode provides the maximum number of ports. All the pins associated with the address and data buses in the expanded modes are available as general-purpose input/output ports in the single-chip mode.

2.3 Address Space Map

2.3.1 Page Segmentation

The H8/520’s address space is segmented into 64-kbyte pages. In the single-chip mode and expanded minimum modes there is just one page: page 0. In the expanded maximum modes there can be up to 16 pages. Figure 2-1 shows the address space in each mode and indicates which parts are on- and off-chip.

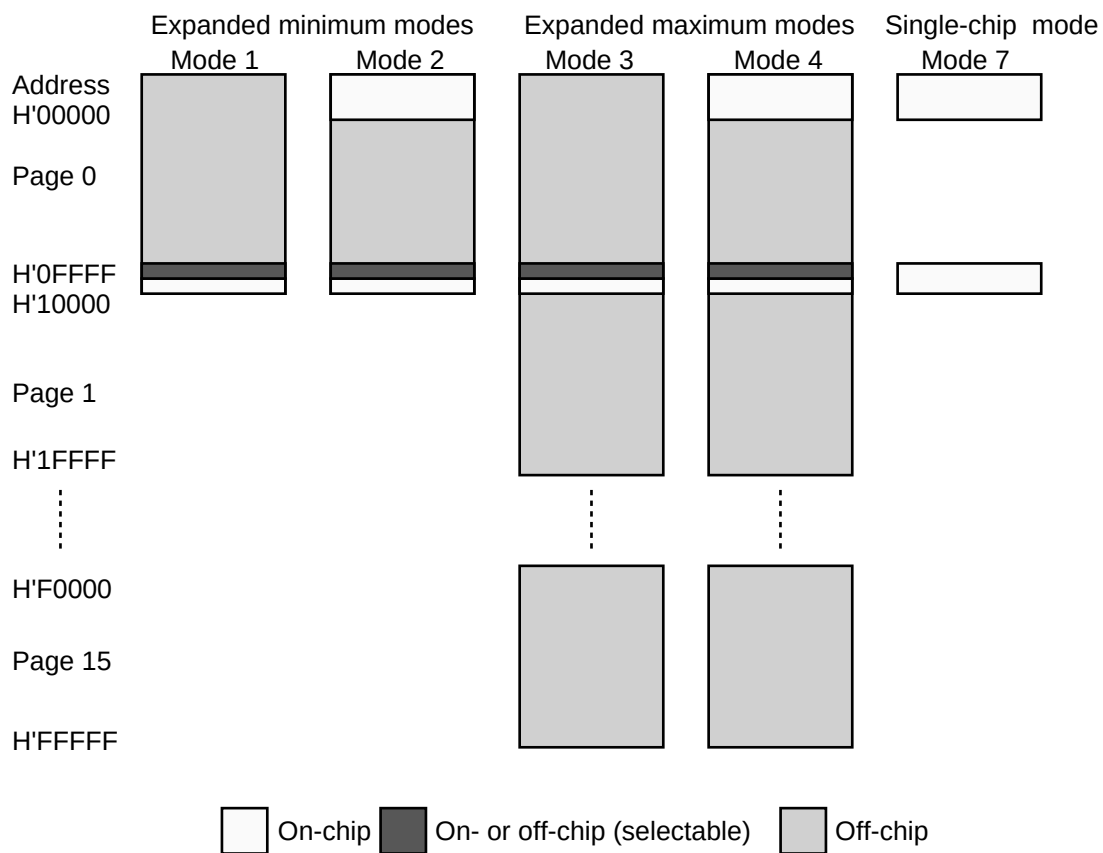


Figure 2-1 Address Space in Each Mode

2.3.2 Page 0 Address Allocations

The high and low address areas in page 0 are reserved for registers and vector tables.

Vector Tables: The low address contains the exception vector table and DTC vector table. The CPU accesses the exception vector table to obtain the addresses of user-coded exception-handling routines. The DTC vector table contains pointers to tables of register information used by the on-chip chip data transfer controller. The size of these tables depends on the CPU operating mode. Details are given in section 4.1.2, “Exception Sources and Vector Table,” section 5.2.3, “Interrupt Vector Table,” and section 6.3.2, “DTC Vector Table.”

In modes 2, 4, and 7 the vector tables are located in on-chip ROM. In modes 1 and 3 the vector tables are in external memory.

Register Field: The highest 128 addresses in page 0 (addresses H'FF80 to H'FFFF) belong to control, status, and data registers used by the I/O ports and on-chip supporting modules. Program code cannot be located at these addresses.

The CPU accesses addresses in this register field like other addresses in the address space. By reading and writing at these addresses the CPU controls the on-chip supporting modules and communicates via the I/O ports. A complete map of the register field is given in appendix B.

On-Chip RAM: One of the control registers in the register field is a RAM control register (RAMCR) containing a RAM enable bit (RAME) that enables or disables the 512-byte on-chip RAM. When this bit is set to 1 (its default value), addresses H'FD80 to H'FF7F are located on-chip. When this bit is cleared to 0, these addresses are located in external memory and the on-chip RAM is not used. See section 15, “RAM”, for further information.

The RAME bit is bit 7 at address H'FFF9.

Coding Example:

To enable on-chip RAM: `BSET.B #7, H'FFF9`

To disable on-chip RAM: `BCLR.B #7, H'FFF9`

Note: If on-chip RAM is disabled in the single-chip mode, access to addresses H'FD80 to H'FF7F causes an address error.

Figure 2-2 is a map of page 0 of the address space.

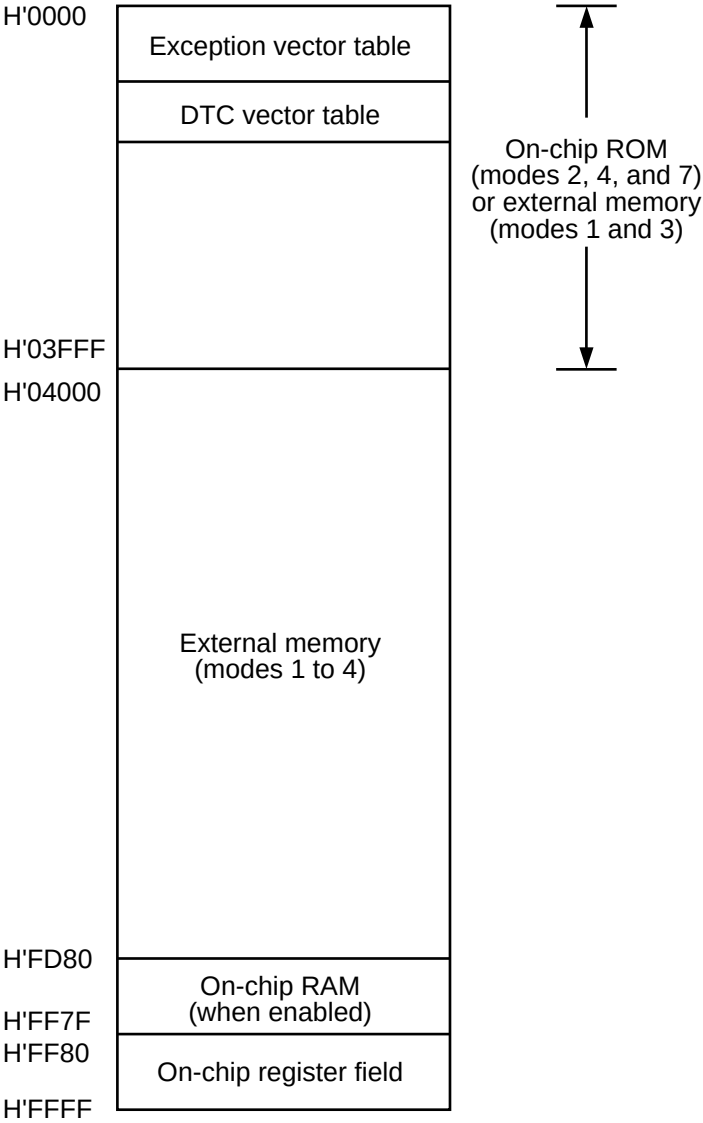


Figure 2-2 Map of Page 0

2.4 Mode Control Register (MDCR)

Another control register in the register field in page 0 is the mode control register (MDCR). The inputs at the mode pins are latched in this register on the rising edge of the signal. The mode control register can be read by the CPU, but not written. Table 2-2 lists the attributes of this register.

Table 2-2 Mode Control Register

Name	Abbreviation	Read/Write	Address
Mode control register	MDCR	Read only	H'FFFA

The bit configuration of this register is shown below.

Bit	7	6	5	4	3	2	1	0
	—	—	—	—	—	MDS2	MDS1	MDS0
Initial value	1	1	0	0	0	—*	—*	—*
Read/Write	—	—	—	—	—	R	R	R

Note: * Initialized according to MD₂ to MD₀.

Bits 7 and 6—Reserved: These bits cannot be modified and are always read as 1.

Bits 5 to 3—Reserved: These bits cannot be modified and are always read as 0.

Bits 2 to 0—Mode Select 2 to 0 (MDS2 to MDS0): These bits indicate the values of the mode pins (MD₂ to MD₀) latched on the rising edge of the $\overline{\text{RES}}$ signal. MDS2 corresponds to MD₂, MDS1 to MD₁, and MDS0 to MD₀. These bits can be read but not written.

Coding example: To test whether the MCU is operating in mode 1:

```
CMP:G.B #H'C1, @H'FFFA
```

The comparison is with H'C1 instead of H'01 because bits 7 and 6 are always read as 1.

Section 3 CPU

3.1 Overview

The H8/520 chip has the H8/500 Family CPU: a high-speed central processing unit designed for real-time control of a wide range of medium-scale office and industrial equipment. It features eight 16-bit general registers, internal 16-bit data paths, and an optimized instruction set.

Section 3 summarizes the CPU architecture and instruction set.

3.1.1 Features

The main features of the H8/500 CPU are listed below.

- General-register machine
 - Eight 16-bit general registers
 - Seven control registers (two 16-bit registers, five 8-bit registers)
- High speed: maximum 10-MHz clock
 - At 10 MHz a register-register add operation takes only 200 ns.
- Address space managed in 64-kbyte pages, expandable to 1 Mbyte*
 - Page registers make four pages available simultaneously: a code page, stack page, data page, and extended page.
- Two CPU operating modes:
 - Minimum mode: Maximum 64-kbyte address space
 - Maximum mode: Maximum 1-Mbyte address space*
- Highly orthogonal instruction set
 - Addressing modes and data sizes can be specified independently within each instruction.
- 1.5 addressing modes
 - Register-register and register-memory operations are supported.
- Optimized for efficient programming in C language
 - In addition to the general registers and orthogonal instruction set, the CPU has special short formats for frequently-used instructions and addressing modes.

Note: * The CPU Architecture supports up to 16 Mbytes of external memory, but the H8/520 chip has only enough address pins to address 1 Mbyte.

3.1.2 Address Space

The address space size depends on the operating mode.

The H8/520 MCU has five operating modes, which are selected by the input to the mode pins (MD2 to MD0) when the chip comes out of a reset. The CPU, however, has only two operating modes. The MCU operating mode determines the CPU operating mode, which in turn determines the maximum address space size as indicated in figure 3-1.

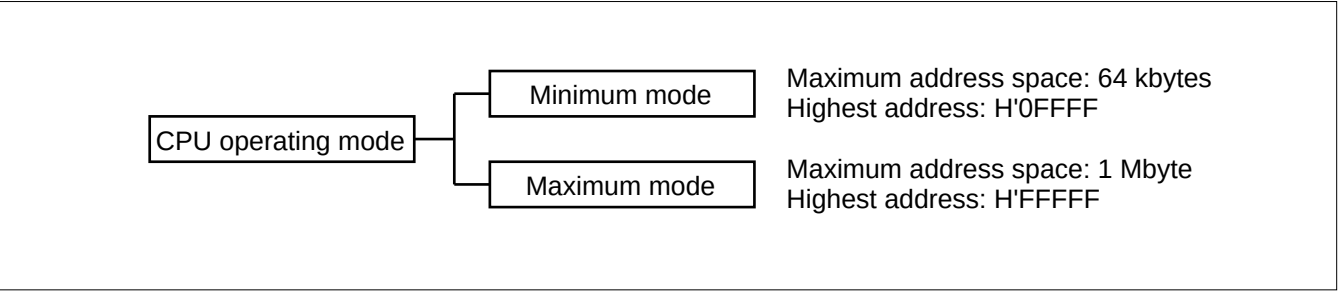


Figure 3-1 CPU Operating Modes

3.1.3 Register Configuration

Figure 3-2 shows the register structure of the CPU. There are two groups of registers: the general registers (Rn) and control registers (CR).

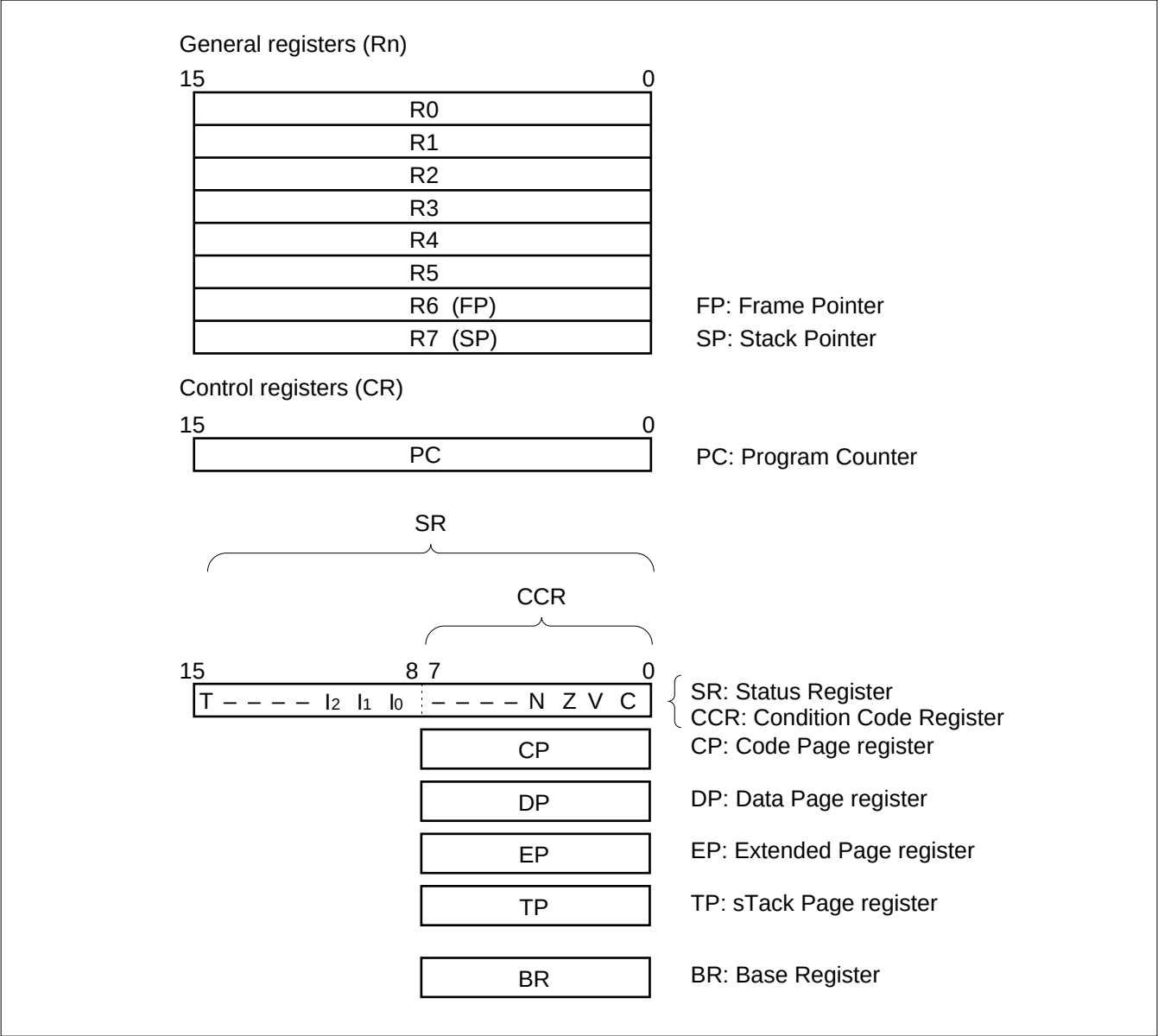


Figure 3-2 Registers in the CPU

3.2 CPU Register Descriptions

3.2.1 General Registers

All eight of the 16-bit general registers are functionally alike; there is no distinction between data registers and address registers. When these registers are accessed as data registers, either byte or word size can be selected.

R6 and R7, in addition to functioning as general registers, have special assignments.

R7 is the stack pointer, used implicitly in exception handling and subroutine calls. It can be designated by the name SP, which is synonymous with R7. As indicated in figure 3-3, it points to the top of the stack. It is also used implicitly by the LDM and STM instructions, which load and store multiple registers from and to the stack and pre-decrement or post-increment R7 accordingly.

R6 functions as a frame pointer (FP). The LINK and UNLK instructions use R6 implicitly to reserve or release a stack frame.

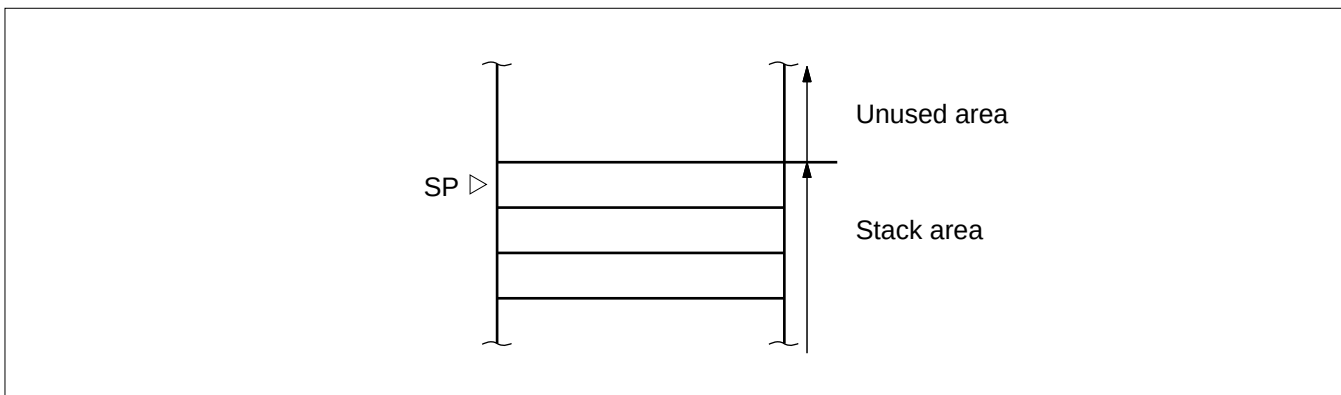


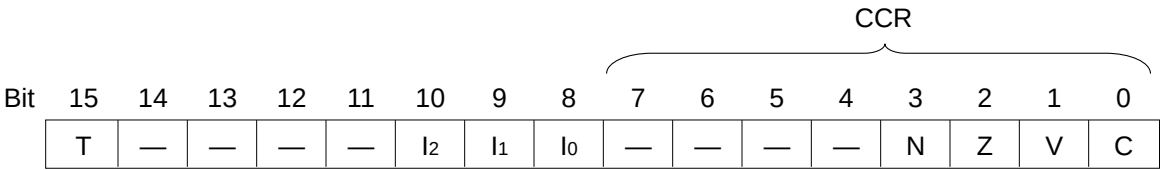
Figure 3-3 Stack Pointer

3.2.2 Control Registers

The CPU control registers include a 16-bit program counter (PC), a 16-bit status register (SR), four 8-bit page registers, and one 8-bit base register (BR).

Program Counter (PC): This 16-bit register indicates the address of the next instruction the CPU will execute.

Status Register (SR): This 16-bit register contains internal status information. The lower half of the status register is referred to as the condition code register (CCR): it can be accessed as a separate condition code byte.



Bit 15—Trace (T): When this bit is set to 1, the CPU operates in trace mode and generates a trace exception after every instruction. See section 4.4, “Trace”, for a description of the trace exception-handling sequence.

When the value of this bit is 0, instructions are executed in normal continuous sequence. This bit is cleared to 0 at a reset.

Bits 14 to 11—Reserved: These bits cannot be modified and are always read as 0.

Bits 10 to 8—Interrupt Mask (I₂, I₁, I₀): These bits indicate the interrupt request mask level (0 to 7). As shown in table 3-1, an interrupt request is not accepted unless it has a higher level than the value of the mask. A nonmaskable interrupt (NMI), which has level 8, is accepted at any mask level. After an interrupt is accepted, I₂, I₁, and I₀ are changed to the level of the interrupt. Table 3-2 indicates the values of the I bits after the interrupt is accepted.

A reset sets all three bits (I₂, I₁, and I₀) to 1, masking all interrupts except NMI.

Table 3-1 Interrupt Mask Levels

Priority	Mask	Mask Bits			Interrupts Accepted
	Level	I ₂	I ₁	I ₀	
High	7	1	1	1	NMI
	6	1	1	0	Level 7 and NMI
	5	1	0	1	Levels 6 to 7 and NMI
	4	1	0	0	Levels 5 to 7 and NMI
	3	0	1	1	Levels 4 to 7 and NMI
	2	0	1	0	Levels 3 to 7 and NMI
	1	0	0	1	Levels 2 to 7 and NMI
Low	0	0	0	0	Levels 1 to 7 and NMI

Table 3-2 Interrupt Mask Bits after an Interrupt is Accepted

Level of Interrupt Accepted	I ₂	I ₁	I ₀
NMI (8)	1	1	1
7	1	1	1
6	1	1	0
5	1	0	1
4	1	0	0
3	0	1	1
2	0	1	0
1	0	0	1

Bits 7 to 4—Reserved: These bits cannot be modified and are always read as 0.

Bit 3—Negative (N): This bit indicates the most significant bit (sign bit) of the result of an instruction.

Bit 2—Zero (Z): This bit is set to 1 to indicate a zero result and cleared to 0 to indicate a nonzero result.

Bit 1—Overflow (V): This bit is set to 1 when an arithmetic overflow occurs, and cleared to 0 at other times.

Bit 0—Carry (C): This bit is set to 1 when a carry or borrow occurs at the most significant bit, and is cleared to 0 (or left unchanged) at other times.

The specific changes that occur in the condition code bits when each instruction is executed are listed in appendix A.1 “Instruction Tables.” See the *H8/500 Series Programming Manual* for further details.

Page Registers: The code page register (CP), data page register (DP), extended page register (EP), and stack page register (TP) are 8-bit registers that are used only in the maximum mode. No use of their contents is made in the minimum mode.

In the maximum mode, the page registers combine with the program counter and general registers to generate 24-bit effective addresses as shown in figure 3-4, thereby expanding the program area, data area, and stack area.

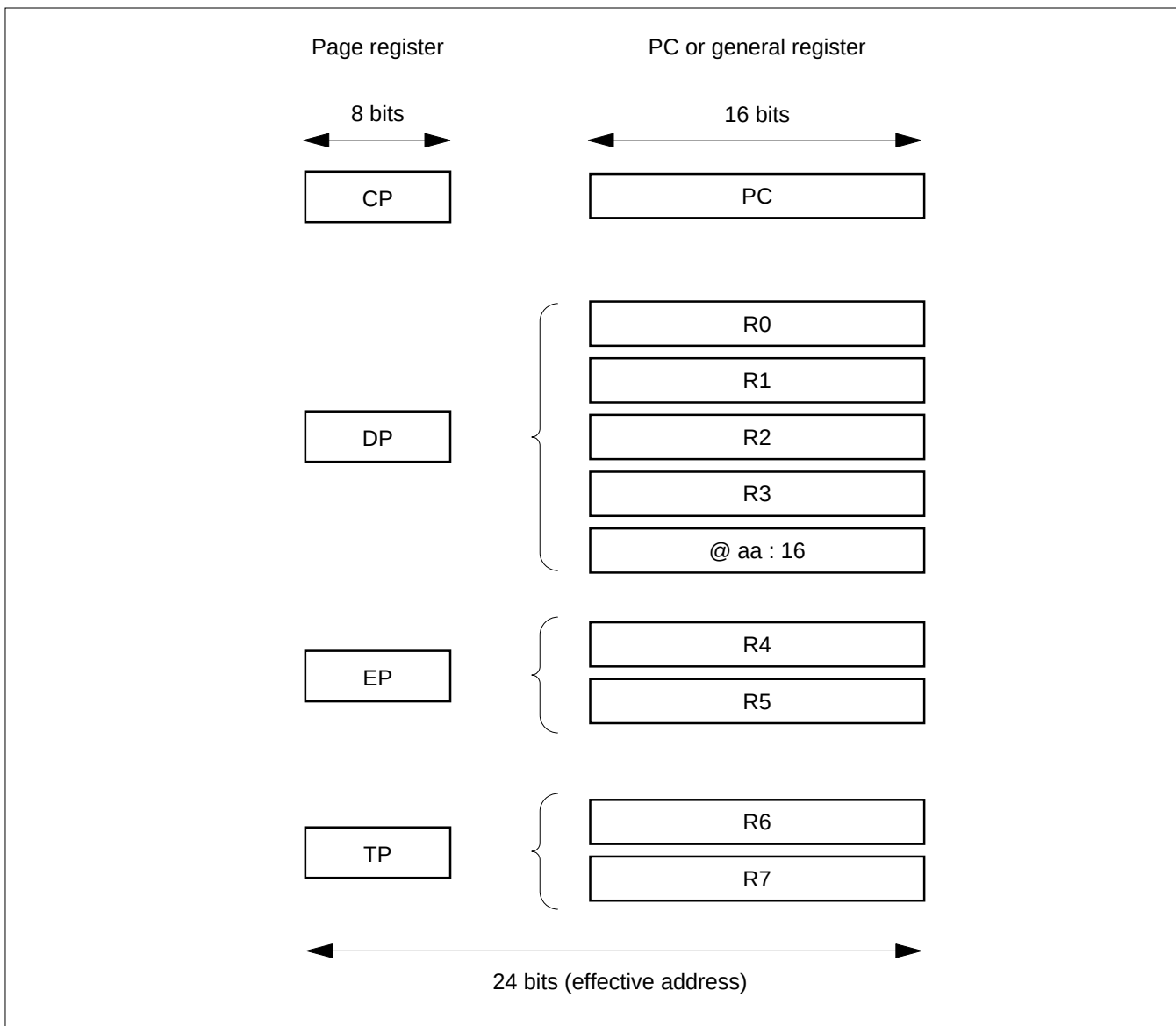


Figure 3-4 Combinations of Page Registers with Other Registers

Code Page Register (CP): The code page register and the program counter combine to generate a 24-bit program code address. The code page register contains the upper 8 bits of the address. In the maximum mode, the code page register is initialized at a reset to a value loaded from the vector table, and both the code page register and program counter are saved and restored in exception handling.

Data Page Register (DP): The data page register combines with general registers R0 to R3 to generate a 24-bit effective address. The data page register contains the upper 8 bits of the address. It is used to calculate effective addresses in the register indirect addressing mode using R0 to R3, and in the 16-bit absolute addressing mode (@aa:16), but not in the short absolute addressing mode (@aa:8).

The data page register is rewritten by the LDC instruction.

Extended Page Register (EP): The extended page register combines with general register R4 or R5 to generate a 24-bit operand address. The extended page register contains the upper 8 bits of the address. It is used to calculate effective addresses in the register indirect addressing mode using R4 or R5.

The extended page can be used as an additional data page.

Stack Page Register (TP): The stack page register combines with R6 (FP) or R7 (SP) to generate a 24-bit stack address. The stack page register contains the upper 8 bits of the address. It is used to calculate effective addresses in the register indirect addressing mode using R6 or R7, in exception handling, and in subroutine calls.

Base Register (BR): This 8-bit register stores the base address used in the short absolute addressing mode (@aa:8). In this addressing mode a 16-bit effective address in page 0 is generated by using the contents of the base register as the upper 8 bits and an address given in the instruction code as the lower 8 bits. See figure 3-5.

In the short absolute addressing mode the address is always located in page 0.

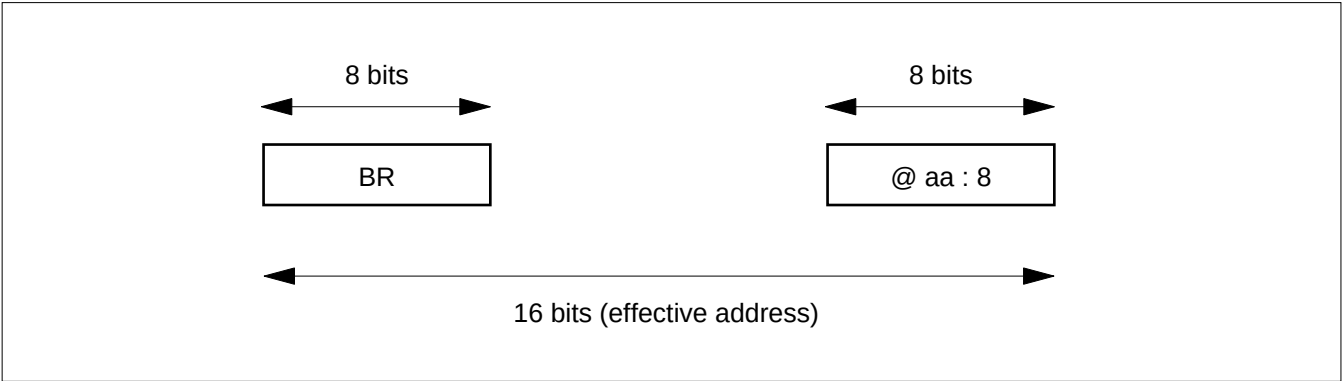


Figure 3-5 Short Absolute Addressing Mode and Base Register

3.2.3 Initial Register Values

When the CPU is reset, its internal registers are initialized as shown in table 3-3. Note that the stack pointer (R7) and base register (BR) are not initialized to fixed values. Also, of the page registers used in maximum mode, only the code page register (CP) is initialized; the other three page registers come out of the reset state with undetermined values.

Accordingly, in the minimum mode the first instruction executed after a reset should initialize the stack pointer. The base register must also be initialized before the short absolute addressing mode (@aa:8) is used.

In the maximum mode, the first instruction executed after a reset should initialize the stack page register (TP) and the next instruction should initialize the stack pointer. Later instructions should initialize the base register and the other page registers as necessary.

Table 3-3 Initial Values of Registers

Register	Initial Value	
	Minimum Mode	Maximum Mode
General registers		
150 R7 - R0	Undetermined	Undetermined
Control registers		
150 PC	Loaded from vector table	Loaded from vector table
SR		
CCR		
15870 T-----I2I1I0-----NZVC	H'070* (*: undetermined)	H'070* (*: undetermined)
70 CP	Undetermined	Loaded from vector table
70 DP	Undetermined	Undetermined
70 EP	Undetermined	Undetermined
70 TP	Undetermined	Undetermined
70 BR	Undetermined	Undetermined

3.3 Data Formats

The H8/500 can process 1-bit data, 4-bit BCD data, 8-bit (byte) data, 16-bit (word) data, and 32-bit (longword) data.

- Bit manipulation instructions operate on 1-bit data.
- Decimal arithmetic instructions operate on 4-bit BCD data.
- Almost all instructions operate on byte and word data.
- Multiply and divide instructions operate on longword data.

3.3.1 Data Formats in General Registers

Data of all the sizes above can be stored in general registers as shown in table 3-4.

Bit data locations are specified by bit number. Bit 15 is the most significant bit. Bit 0 is the least significant bit. BCD and byte data are stored in the lower 8 bits of a general register. Word data use all 16 bits of a general register. Longword data use two general registers: the upper 16 bits are stored in Rn (n must be an even number); the lower 16 bits are stored in Rn+1.

Operations performed on BCD data or byte data do not affect the upper 8 bits of the register.

Table 3-4 General Register Data Formats

Data Type	Register No.	Data Structure																																
1-Bit	Rn	150 <table><tr><td>15</td><td>14</td><td>13</td><td>12</td><td>11</td><td>10</td><td>9</td><td>8</td><td>7</td><td>6</td><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr></table>	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																			
BCD	Rn	1587430 <table><tr><td colspan="8">Don't-care</td><td colspan="4">Upper digit</td><td colspan="4">Lower digit</td></tr></table>	Don't-care								Upper digit				Lower digit																			
Don't-care								Upper digit				Lower digit																						
Byte	Rn	15870 <table><tr><td colspan="8">Don't-care</td><td colspan="8">MSBLSB</td></tr></table>	Don't-care								MSBLSB																							
Don't-care								MSBLSB																										
Word	Rn	150 <table><tr><td colspan="16">MSBLSB</td></tr></table>	MSBLSB																															
MSBLSB																																		
Longword	Rn*	3116 <table><tr><td colspan="16">MSBUpper 16 bits</td></tr><tr><td colspan="16">Lower 16 bitsLSB</td></tr></table>	MSBUpper 16 bits																Lower 16 bitsLSB															
MSBUpper 16 bits																																		
Lower 16 bitsLSB																																		
	Rn + 1*	150																																

Note: * For longword data, n must be even (0, 2, 4, or 6).

3.3.2 Data Formats in Memory

Table 3-5 indicates the data formats in memory.

Instructions that access bit data in memory have byte or word operands. The instruction specifies a bit number to indicate a specific bit in the operand.

Access to word data in memory must always begin at an even address. Access to word data starting at an odd address causes an address error. The upper 8 bits of word data are stored in address n (where n is an even number); the lower 8 bits are stored in address n+1.

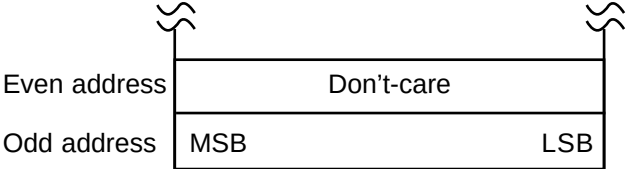
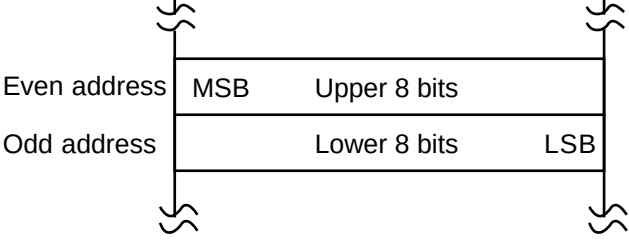
Table 3-5 Data Formats in Memory

Data Type	Data Format
1-bit (in byte operand data)	Address n 7 6 5 4 3 2 1 0
1-bit (in word operand data)	Even address 15 14 13 12 11 10 9 8 Odd address 7 6 5 4 3 2 1 0
Byte	Address n MSB LSB
Word	Even address MSB Upper 8 bits Odd address Lower 8 bits LSB

When the stack is accessed in exception processing (to save or restore the program counter, code page register, or status register), word access is always performed, regardless of the actual data size. Similarly, when the stack is accessed by an instruction using the pre-decrement or post-increment register indirect addressing mode specifying R7 (@-R7 or @R7+), which is the stack pointer, word access is performed regardless of the operand size specified in the instruction. An address error will therefore occur if the stack pointer indicates an odd address. Programs should be coded so that the stack pointer always indicates an even address.

Table 3-6 shows the data formats on the stack.

Table 3-6 Data Formats on the Stack

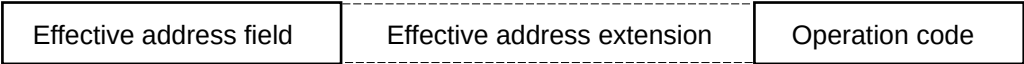
Data Type	Data Format
Byte data on stack	
Word data on stack	

3.4 Instructions

3.4.1 Basic Instruction Formats

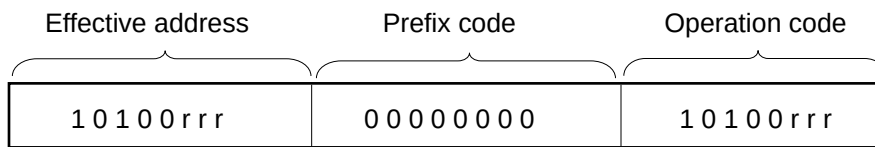
There are two basic CPU instruction formats: the general format and the special format.

General Format: This format consists of an effective address (EA) field, an effective address extension field, and an operation code (OP) field. The effective address is placed before the operation code because this results in faster execution of the instruction.

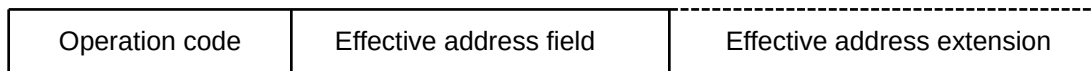


- **Effective address field:** One byte containing information used to calculate the effective address of an operand.
- **Effective address extension:** Zero to two bytes containing a displacement value, immediate data, or an absolute address. The size of the effective address extension is specified in the effective address field.
- **Operation code:** Defines the operation to be carried out on the operand located at the address calculated from the effective address information. Some instructions (DADD, DSUB) have an extended format in which the operand code is preceded by a one-byte prefix code.

- Example of prefix code in DADD instruction



Special Format: In this format the operation code comes first, followed by the effective address field and effective address extension. This format is used in branching instructions, system control instructions, and other instructions that can be executed faster if the operation is specified before the operand.



- Operation code: One or two bytes defining the operation to be performed by the instruction.
- Effective address field and effective address extension: Zero to three bytes containing information used to calculate an effective address.

3.4.2 Addressing Modes

The CPU supports 7 addressing modes: (1) register direct; (2) register indirect; (3) register indirect with displacement; (4) register indirect with pre-decrement or post-increment; (5) immediate; (6) absolute; and (7) PC-relative.

Due to the highly orthogonal nature of the instruction set, most instructions having operands can use any applicable addressing mode from (1) through (6). The PC-relative mode (7) is used by branching instructions.

In most instructions, the addressing mode is specified in the effective address field. The effective-address extension, if present, contains a displacement, immediate data, or an absolute address.

Table 3-7 indicates how the addressing mode is specified in the effective address field.

Table 3-7 Addressing Modes

No.	Addressing Mode	Mnemonic	EA Field	EA Extension
1	Register direct	Rn	1 0 1 0 Sz r r r *1 *2	None
2	Register indirect	@Rn	1 1 0 1 Sz r r r	None
3	Register indirect with displacement	@(d:8,Rn)	1 1 1 0 Sz r r r	Displacement (1 byte)
		@(d:16,Rn)	1 1 1 1 Sz r r r	Displacement (2 bytes)
4	Register indirect with pre-decrement	@-Rn	1 0 1 1 Sz r r r	None
	Register indirect with post-increment	@Rn+	1 1 0 0 Sz r r r	
5	Immediate	#xx:8	0 0 0 0 0 1 0 0	Immediate data (1 byte)
		#xx:16	0 0 0 0 1 1 0 0	Immediate data (2 bytes)
6	Absolute* ³	@aa:8	0 0 0 0 Sz 1 0 1	1-Byte absolute address (offset from BR)
		@aa:16	0 0 0 1 Sz 1 0 1	2-Byte absolute address
7	PC-relative	disp	No EA field. Addressing mode is specified in the operation code.	1- or 2-byte displacement

- Notes: 1. Sz: Specifies the operand size.
When Sz = 0: byte operand
When Sz = 1: word operand
2. r r r: Register number field, specifying a general register number.
0 0 0—R₀ 0 0 1—R₁ 0 1 0—R₂ 0 1 1—R₃
1 0 0—R₄ 1 0 1—R₅ 1 1 0—R₆ 1 1 1—R₇
3. The @aa:8 addressing mode is also referred to as the short absolute addressing mode.

3.4.3 Effective Address Calculation

Table 3-8 explains how the effective address is calculated in each addressing mode.

Table 3-8 Effective Address Calculation

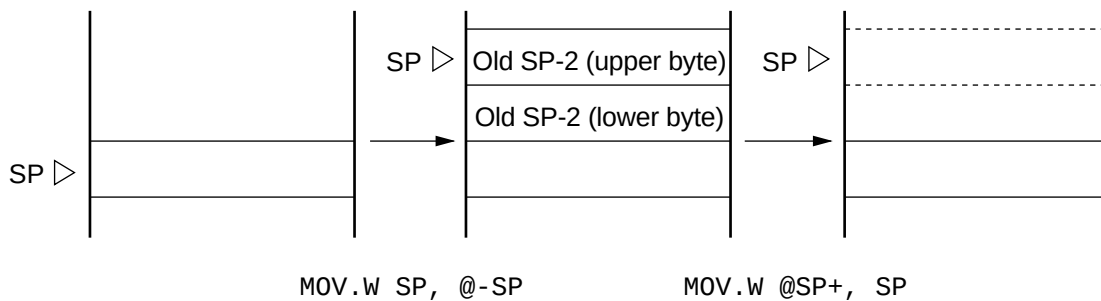
No.	Addressing Mode	Effective Address Calculation	Effective Address
1	Register direct Rn	—	Operand is contents of Rn
	1010Sz rrr		
2	Register indirect @Rn	—	23 15 0 DP Rn Or TP or EP
3	Register indirect with displacement @(d:8, Rn)	8 Bits 15 0 Rn 15 0 Displacement with sign extension +	23 15 0 DP Result Or TP or EP
	@(d:16, Rn) 1111Sz rrr	16 Bits 15 0 Rn 15 0 Displacement +	23 15 0 DP Result Or TP or EP
4	① Register indirect with pre-decrement @-Rn	15 0 Rn 1 or 2*3 -	23 15 0 DP*1 Result Or TP or EP*2
	② Register indirect with post-increment @Rn +	—	23 15 0 DP*1 Result Or TP or EP*2
	1100Sz rrr	Rn is incremented by +1 or +2 after instruction execution. *3, 4	

- Notes:
1. The page register is ignored in minimum mode.
 2. The page register used in addressing modes 2, 3, and 4 depends on the general register: DP for R0, R1, R2, or R3; EP for R4 or R5; TP for R6 or R7.
 3. Decrement by -1 for a byte operand, and by -2 for a word operand.
 4. The pre-decrement or post-increment is always ± 2 when R7 is specified, even if the operand is byte size.

Table 3-8 Effective Address Calculation (cont)

No.	Addressing Mode	Effective Address Calculation	Effective Address
5	Absolute address @aa:8	—	23150 H'00 BR EA extension data
	@aa:16	—	23150 DP EA extension data
6	Immediate #xx:8	—	Operand is 1-byte EA extension data
	#xx:16	—	Operand is 2-byte EA extension data
7	PC-relative disp:8 No EA code Specified in OP code	8 Bits 150 PC + 150 Displacement with sign extension	23150 CP Result
	disp:16 No EA code Specified in OP code	16 Bits 150 PC + 150 Displacement	23150 CP Result

Note: The drawing below shows what happens when the @-SP and @ SP+ addressing modes are used to save and restore the stack pointer.



3.5 Instruction Set

3.5.1 Overview

The main features of the CPU instruction set are:

- A general-register architecture.
- Orthogonality. Addressing modes and data sizes can be specified independently in each instruction.
- Register-register and register-memory operations are supported.
- Affinity for high-level languages, particularly C, with short formats for frequently-used instructions and addressing modes.

The CPU instruction set includes 61 (63)^{*1} types of instructions, listed by function in table 3-9.

Table 3-9 Instruction Classification

Function	Instructions	Types
Data transfer	MOV, LDM, STM, XCH, SWAP, (MOVTPE, MOVFPE) ^{*1}	5 (7) ^{*1}
Arithmetic operations	ADD, SUB, ADDS, SUBS, ADDX, SUBX, DADD, DSUB, MULXU, DIVXU, CMP, EXTS, EXTU, TST, NEG, CLR, TAS	17
Logic operations	AND, OR, XOR, NOT	4
Shift	SHAL, SHAR, SHLL, SHLR, ROTL, ROTR, ROTXL, ROTXR	8
Bit manipulation	BSET, BCLR, BTST, BNOT	4
Branch	Bcc ^{*2} , JMP, PJMP, BSR, JSR, PJSR, RTS, PRTD, PRTS, RTD, SCB (/F, /NE, /EQ)	11
System control	TRAPA, TRAP/VS, RTE, SLEEP, LDC, STC, ANDC, ORC, XORC, NOP, LINK, UNLK	12
Total		61 (63) ^{*1}

- Notes: 1. The H8/520 chip does not have an E clock output pin, so it does not support the MOVTPE and MOVFPE instructions. H8/520 software should not use these instructions.
2. Bcc is a conditional branch instruction in which cc represents a condition code.

Tables 3-10 to 3-16 give a concise summary of the instructions in each functional category. The MOV, ADD, and CMP instructions have special short formats, which are listed in table 3-17. For detailed descriptions of the instructions, refer to the *H8/500 Series Programming Manual*.

The notation used in tables 3-10 to 3-17 is defined below.

Operation	Notation
Rd	General register (destination)
Rs	General register (source)
Rn	General register
(EAd)	Destination operand
(EAs)	Source operand
CCR	Condition code register
N	N (negative) bit of CCR
Z	Z (zero) bit of CCR
V	V (overflow) bit of CCR
C	C (carry) bit of CCR
CR	Control register
PC	Program counter
CP	Code page register
SP	Stack pointer
FP	Frame pointer
#IMM	Immediate data
disp	Displacement
+	Addition
–	Subtraction
×	Multiplication
÷	Division
^	AND logical
∨	OR logical
⊕	Exclusive OR logical
→	Move
↔	Exchange
¬	Not

3.5.2 Data Transfer Instructions

Table 3-10 describes the seven data transfer instructions.

Table 3-10 Data Transfer Instructions

Instruction	Size*2	Function
Data transfer		
MOV		(EAs) → (EAd), #IMM → (EAd)
MOV : G	B/W	Moves data between two general registers, or between
MOV : E	B	a general register and memory, or moves immediate data
MOV : I	W	to a general register or memory.
MOV : F	B/W	
MOV : L	B/W	
MOV : S	B/W	
LDM	W	Stack → Rn (register list) Pops data from the stack to one or more registers.
STM	W	Rn (register list) → stack Pushes data from one or more registers onto the stack.
XCH	W	Rs ↔ Rd Exchanges data between two general registers.
SWAP	B	Rd (upper byte) ↔ Rd (lower byte) Exchanges the upper and lower bytes in a general register.
(MOVTPPE)*1	—	Not supported by the H8/520
(MOVFPPE)*1	—	Not supported by the H8/520

Notes: 1. The H8/520 does not have an E clock output pin, so it does not support the MOVTPPE and MOVFPPE instructions. H8/520 software should not use these instructions.

If the MOVTPPE and MOVFPPE instructions are used, the H8/520 executes them in the number of cycles indicated in figures A and B.

From 7 to 14 wait states (Tw) are automatically inserted between the T2 state and T3 state to synchronize the bus cycle with an internal E clock obtained by dividing the system clock (ø) by eight. Accordingly, the number of cycles taken by a MOVTPPE or MOVFPPE instruction varies. Note that no wait states (Tw) are inserted by the wait state controller.

2. B: Byte, W: Word

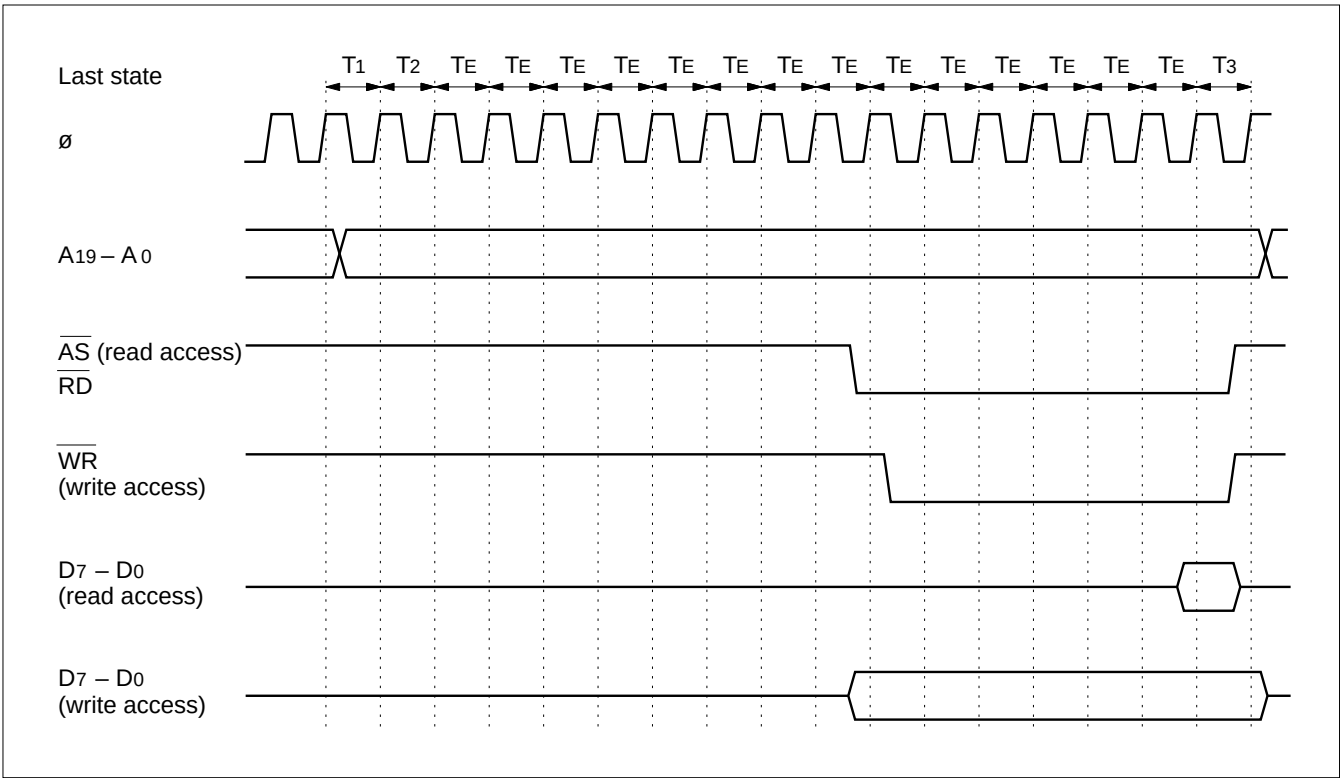


Figure A Execution Cycle Length of MOVTP and MOVFP Instructions in Expanded Modes (Maximum Number of Cycles)

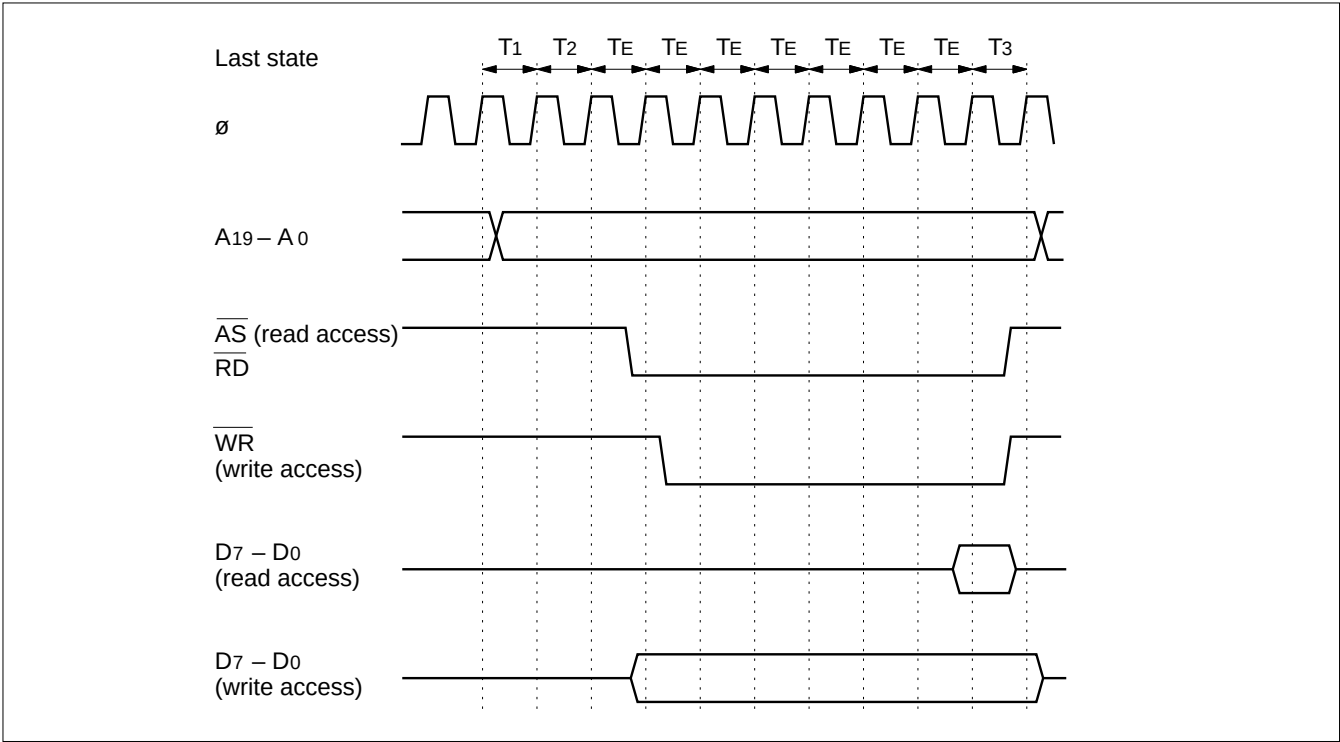


Figure B Execution Cycle Length of MOVTP and MOVFP Instructions in Expanded Modes (Minimum Number of Cycles)

3.5.3 Arithmetic Instructions

Table 3-11 describes the 17 arithmetic instructions.

Table 3-11 Arithmetic Instructions

Instruction		Size	Function
Arithmetic operations	ADD		$Rd \pm (EAs) \rightarrow Rd, (EAd) \pm \#IMM \rightarrow (EAd)$
	ADD : G	B/W	Performs addition or subtraction on data in a general register and data in another general register or memory, or on data in a general register or memory and immediate data.
	ADD : Q	B/W	
	SUB	B/W	
	ADDS	B/W	
	SUBS	B/W	
	ADDX	B/W	$Rd \pm (EAs) \pm C \rightarrow Rd$
	SUBX	B/W	Performs addition or subtraction with carry or borrow on data in a general register and data in another general register or memory, or on data in a general register and immediate data.
	DADD	B	$(Rd)_{10} \pm (Rs)_{10} \pm C \rightarrow (Rd)_{10}$
	DSUB	B	Performs decimal addition or subtraction on data in two general registers.
	MULXU	B/W	$Rn \times (EAs) \rightarrow Rd$ Performs 8-bit $\times$ 8-bit or 16-bit $\times$ 16-bit unsigned multiplication on data in a general register and data in another general register or memory, or on data in a general register and immediate data.
	DIVXU	B/W	$Rd \div (EAs) \rightarrow Rd$ Performs 16-bit $\div$ 8-bit or 32-bit $\div$ 16-bit unsigned division on data in a general register and data in another general register or memory, or on data in a general register and immediate data.
	CMP		$Rn - (EAs), (EAd) - \#IMM$
	CMP : G	B/W	Compares data in a general register with data in another general register or memory, or with immediate data, or compares data in memory with immediate data.
	CMP : E	B	
	CMP : I	W	

Table 3-11 Arithmetic Instructions (cont)

Instruction		Size	Function
Arithmetic operations	EXTS	B	$(\langle \text{bit } 7 \rangle \text{ of } \langle \text{Rd} \rangle) \rightarrow (\langle \text{bits } 15 \text{ to } 8 \rangle \text{ of } \langle \text{Rd} \rangle)$ Converts byte data in a general register to word data by extending the sign bit.
	EXTU	B	$0 \rightarrow (\langle \text{bits } 15 \text{ to } 8 \rangle \text{ of } \langle \text{Rd} \rangle)$ Converts byte data in a general register to word data by padding with zero bits.
	TST	B/W	$(\text{EAd}) - 0$ Compares general register or memory contents with 0.
	NEG	B/W	$0 - (\text{EAd}) \rightarrow (\text{EAd})$ Obtains the two's complement of general register or memory contents.
	CLR	B/W	$0 \rightarrow (\text{EAd})$ Clears general register or memory contents to 0.
	TAS	B	$(\text{EAd}) - 0, (1)_2 \rightarrow (\langle \text{bit } 7 \rangle \text{ of } \langle \text{EAd} \rangle)$ Tests general register or memory contents, then sets the most significant bit (bit 7) to 1.

3.5.4 Logic Operations

Table 3-12 lists the four instructions that perform logic operations.

Table 3-12 Logic Operation Instructions

Instruction		Size	Function
Logical operations	AND	B/W	$\text{Rd} \wedge (\text{EAs}) \rightarrow \text{Rd}$ Performs a logical AND operation on a general register and another general register, memory, or immediate data.
	OR	B/W	$\text{Rd} \vee (\text{EAs}) \rightarrow \text{Rd}$ Performs a logical OR operation on a general register and another general register, memory, or immediate data.
	XOR	B/W	$\text{Rd} \oplus (\text{EAs}) \rightarrow \text{Rd}$ Performs a logical exclusive OR operation on a general register and another general register, memory, or immediate data.
	NOT	B/W	$\neg (\text{EAd}) \rightarrow (\text{EAd})$ Obtains the one's complement of general register or memory contents.

3.5.5 Shift Operations

Table 3-13 lists the eight shift instructions.

Table 3-13 Shift Instructions

Instruction		Size	Function
Shift operations	SHAL	B/W	(EAd) shift → (EAd)
	SHAR	B/W	Performs an arithmetic shift operation on general register or memory contents.
	SHLL	B/W	(EAd) shift → (EAd)
	SHLR	B/W	Performs a logical shift operation on general register or memory contents.
	ROTL	B/W	(EAd) rotate → (EAd)
	ROTR	B/W	Rotates general register or memory contents.
	ROTXL	B/W	(EAd) rotate through carry → (EAd)
	ROTXR	B/W	Rotates general register or memory contents through the C (carry) bit.

3.5.6 Bit Manipulations

Table 3-14 describes the four bit-manipulation instructions

Table 3-14 Bit-Manipulation Instructions

Instruction		Size	Function
Bit manipulations	BSET	B/W	$\neg (<\text{bit-No.}> \text{ of } <\text{EAd}> \rightarrow Z, 1 \rightarrow (<\text{bit-No.}> \text{ of } <\text{EAd}>)$ Tests a specified bit in a general register or memory, then sets the bit to 1. The bit is specified by a bit number given in immediate data or a general register.
	BCLR	B/W	$\neg (<\text{bit-No.}> \text{ of } <\text{EAd}> \rightarrow Z, 0 \rightarrow (<\text{bit-No.}> \text{ of } <\text{EAd}>)$ Tests a specified bit in a general register or memory, then clears the bit to 0. The bit is specified by a bit number given in immediate data or a general register.
	BNOT	B/W	$\neg (<\text{bit-No.}> \text{ of } <\text{EAd}> \rightarrow Z, \rightarrow (<\text{bit-No.}> \text{ of } <\text{EAd}>)$ Tests a specified bit in a general register or memory, then inverts the bit. The bit is specified by a bit number given in immediate data or a general register.
	BTST	B/W	$\neg (<\text{bit-No.}> \text{ of } <\text{EAd}> \rightarrow Z$ Tests a specified bit in a general register or memory. The bit is specified by a bit number given in immediate data or a general register.

3.5.7 Branching Instructions

Table 3-15 describes the 11 branching instructions

Table 3-15 Branching Instructions

Instruction	Size	Function
Branch	Bcc	— Branches if condition cc is true.
Mnemonic	Description	Condition
BRA (BT)	Always (true)	True
BRN (BF)	Never (false)	False
BHI	High	$C \vee Z = 0$
BLS	Low or Same	$C \vee Z = 1$
BCC (BHS)	Carry Clear (High or Same)	$C = 0$
BCS (BLO)	Carry Set (Low)	$C = 1$
BNE	Not Equal	$Z = 0$
BEQ	Equal	$Z = 1$
BVC	Overflow Clear	$V = 0$
BVS	Overflow Set	$V = 1$
BPL	Plus	$N = 0$
BMI	Minus	$N = 1$
BGE	Greater or Equal	$N \oplus V = 0$
BLT	Less Than	$N \oplus V = 1$
BGT	Greater Than	$Z \vee (N \oplus V) = 0$
BLE	Less or Equal	$Z \vee (N \oplus V) = 1$
JMP	—	Branches unconditionally to a specified address in the same page.
PJMP	—	Branches unconditionally to a specified address in a specified page.
BSR	—	Branches to a subroutine at a specified address in the same page.
JSR	—	Branches to a subroutine at a specified address in the same page.
PJSR	—	Branches to a subroutine at a specified address in a specified page.
RTS	—	Returns from a subroutine in the same page.

Table 3-15 Branching Instructions (cont)

Instruction		Size	Function
Branch	PRTS	—	Returns from a subroutine in a different page.
	RTD	—	Returns from a subroutine in the same page and adjusts the stack pointer.
	PRTD	—	Returns from a subroutine in a different page and adjusts the stack pointer.
	SCB/F	—	Controls a loop using a loop counter and/or a specified termination condition.
	SCB/NE	—	
	SCB/EQ		

3.5.8 System Control Instructions

Table 3-16 describes the 12 system control instructions.

Table 3-16 System Control Instructions

Instruction		Size	Function
System control	TRAPA	—	Generates a trap exception with a specified vector number.
	TRAP/VS	—	Generates a trap exception if the V bit is set to 1 when the instruction is executed.
	RTE	—	Returns from an exception-handling routine.
	LINK	—	$FP \rightarrow @-SP$; $SP \rightarrow FP$; $SP + \#IMM \rightarrow SP$ Creates a stack frame.
	UNLK	—	$FP \rightarrow SP$; $@SP + \rightarrow FP$ Deallocates a stack frame created by the LINK instruction.
	SLEEP	—	Causes a transition to the power-down state.
	LDC	B/W*	$(EAs) \rightarrow CR$ Moves immediate data or general register or memory contents to a specified control register.
	STC	B/W*	$CR \rightarrow (EAd)$ Moves control register data to a specified general register or memory location.
	ANDC	B/W*	$CR \wedge \#IMM \rightarrow CR$ Logically ANDs a control register with immediate data.
	ORC	B/W*	$CR \vee \#IMM \rightarrow CR$ Logically ORs a control register with immediate data.
	XORC	B/W*	$CR \oplus \#IMM \rightarrow CR$ Logically exclusive-ORs a control register with immediate data.
	NOP	—	$PC + 1 \rightarrow PC$ No operation. Only increments the program counter.

Note: * The size depends on the control register.

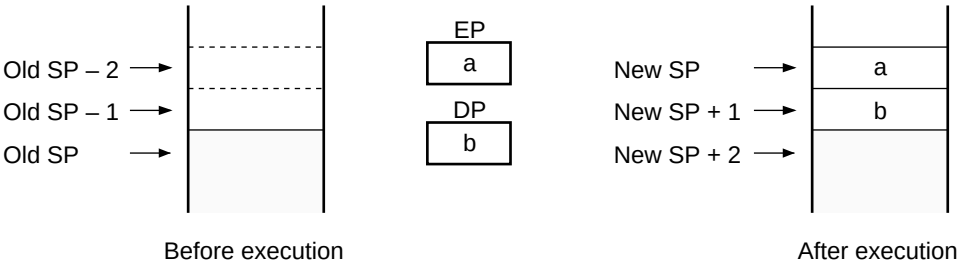
When using the LDC and STC instructions to stack and unstack the BR, CCR, TP, DP, and EP control registers in the H8/500 family, note the following point.

H8/500 hardware does not permit byte access to the stack. If the LDC.B or STC.B assembler mnemonic is coded with the @R7+ (@SP+) or @-R7 (@-SP) addressing mode, the stack-pointer addressing mode takes precedence and hardware automatically performs word access. Specifically, the LDC.B and STC.B instructions are executed as follows.

The following applies only to the stack-pointer addressing modes. In addressing modes that do not use the stack pointer, byte data access is performed as specified by the assembler mnemonic.

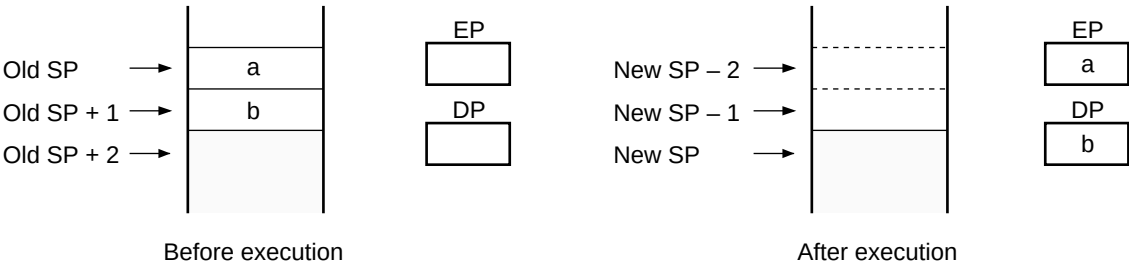
1. STC.B EP, @-SP

When word data access is applied to EP, both EP and DP are accessed. This instruction stores EP at address SP (old) - 2, and DP at address SP (old) - 1.



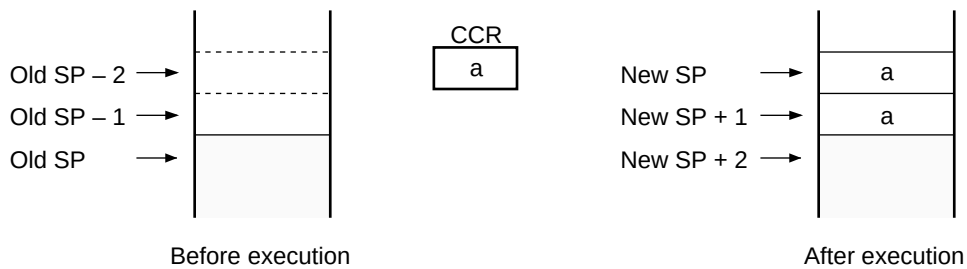
2. LDC.B @SP+, EP

When word data access is applied to EP, both EP and DP are accessed. This instruction loads EP from address SP (old), and DP from address SP (old) + 1, updating the DP value as well as the EP value.



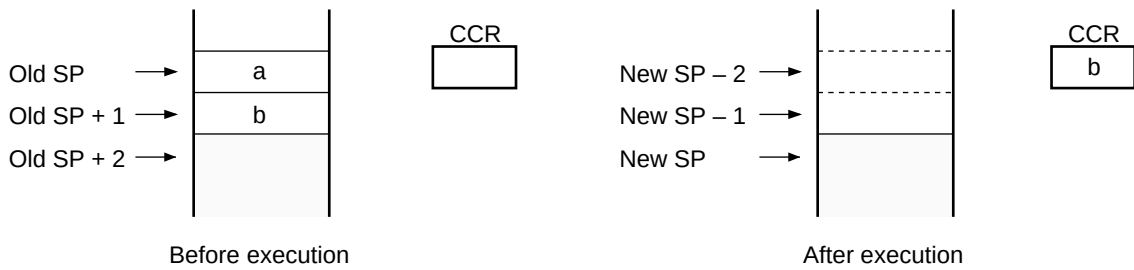
3. STC.B CCR, @-SP

When word data access is applied to CCR, only CCR is accessed. This instruction stores identical CCR contents at both address SP (old) – 2 and address SP (old) – 1.



4. LDC.B @SP+, CCR

When word data access is applied to CCR, only CCR is accessed. This instruction loads CCR from address SP (old) + 1. Note that the value in address SP (old) is not loaded.



BR, DP, and TP are accessed in the same way as CCR. When EP is specified, both EP and DP are accessed, but when CCR, BR, DP, or TP is specified, only the specified register is accessed.

3.5.9 Short-Format Instructions

The ADD, CMP, and MOV instructions have special short formats. Table 3-17 lists these short formats together with the equivalent general formats.

The short formats are a byte shorter than the corresponding general formats, and most of them execute one state faster.

Table 3-17 Short-Format Instructions and Equivalent General Formats

Short-Format Instruction	Length	Execution States*2	Equivalent General-Format Instruction	Length	Execution States*2
ADD:Q #xx, Rd*1	2	2	ADD:G #xx:8, Rd	3	3
CMP:E #xx:8, Rd	2	2	CMP:G.B #xx:8, Rd	3	3
CMP:I #xx:16, Rd	3	3	CMP:G.W #xx:16, Rd	4	4
MOV:E #xx:8, Rd	2	2	MOV:G.B #xx:8, Rd	3	3
MOV:I #xx:16, Rd	3	3	MOV:G.W #xx:16, Rd	4	4
MOV:L @aa:8, Rd	2	5	MOV:G @aa:8, Rd	3	5
MOV:S Rs, @aa:8	2	5	MOV:G Rs, @aa:8	3	5
MOV:F @(d:8, R6), Rd	2	5	MOV:G @(d:8, R6), Rd	3	5
MOV:F Rs, @(d:8, R6)	2	5	MOV:G Rs, @(d:8, R6)	3	5

Notes: 1. The ADD: Q instruction accepts other destination operands in addition to a general register, but the immediate data value (#xx) is limited to ± 1 or ± 2 .
2. Number of execution states for access to on-chip memory.

3.6 Operating Modes

The CPU operates in one of two modes: the minimum mode or the maximum mode. These modes are selected by the mode pins (MD2 to MD0).

3.6.1 Minimum Mode

The minimum mode supports a maximum address space of 64 kbytes. The page registers are ignored. Instructions that branch across page boundaries (PJMP, PJSR, PRTS, PRTD) are invalid.

3.6.2 Maximum Mode

In the maximum mode the page registers are valid, expanding the maximum address space to 1 Mbyte.

The address space is divided into 64-kbyte pages. The pages are separate; it is not possible to move continuously across a page boundary.

It is possible to move from one page to another with branching instructions (PJMP, PJSR, PRTS, PRD). The TRAPA instruction and instructions that branch to interrupt-handling routines can also jump across page boundaries. It is not necessary for a program to be contained in a single 64-kbyte page.

When data access crosses a page boundary, the program must rewrite the page register before it can access the data in the next page.

For further information on the operating modes, see section 2, “MCU Operating Modes and Address Space.”

3.7 Basic Operational Timing

3.7.1 Overview

The CPU operates on a system clock (ϕ) which is created by dividing the crystal oscillator frequency (f_{osc}) by two. One period of the system clock is referred to as a “state.” The CPU accesses memory in a cycle consisting of 2 or 3 states. The CPU uses different methods to access on-chip memory, the on-chip register field, and external devices.

Access to On-Chip Memory (RAM, ROM): For maximum speed, access to on-chip memory (RAM, ROM) is performed in two states, using a 16-bit-wide data bus.

Figure 3-6 shows the on-chip memory access cycle. Figure 3-7 indicates the pin states. The bus control signals output from the H8/520 chip go to the nonactive state during the access.

Access to On-Chip Register Field (Addresses H'FF80 to H'FFFF): The access cycle consists of three states. The data bus is 8 bits wide.

Figure 3-8 shows the on-chip supporting module access cycle. Figure 3-9 indicates the pin states.

Access to External Devices: The access cycle consists of three states. The data bus is 8 bits wide. Figure 3-10 (a) and (b) shows the external access cycle. Additional wait states (T_w) can be inserted by the wait-state controller (WSC).

3.7.2 On-Chip Memory Access Cycle

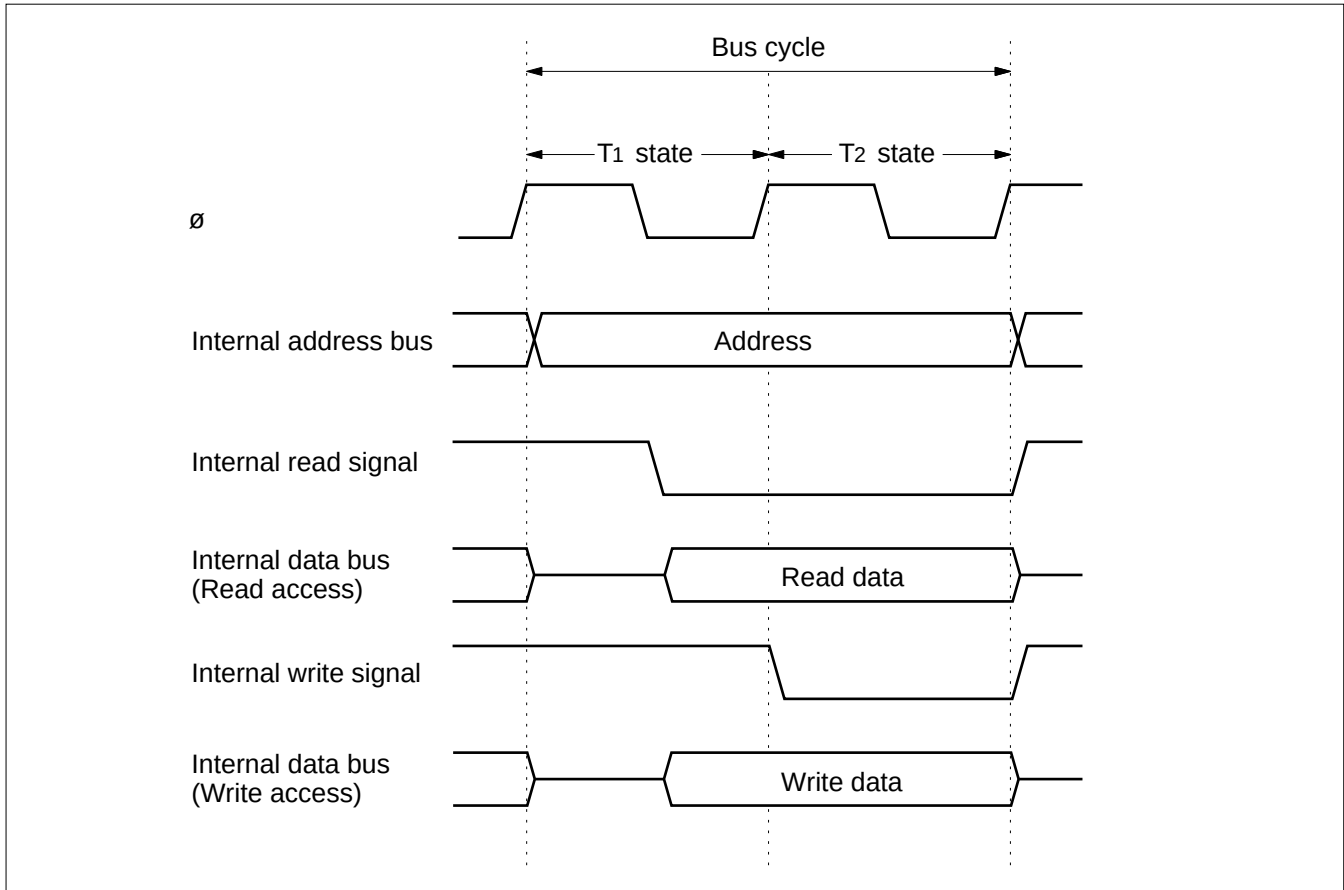


Figure 3-6 On-Chip Memory Access Timing

3.7.3 Pin States during On-Chip Memory Access

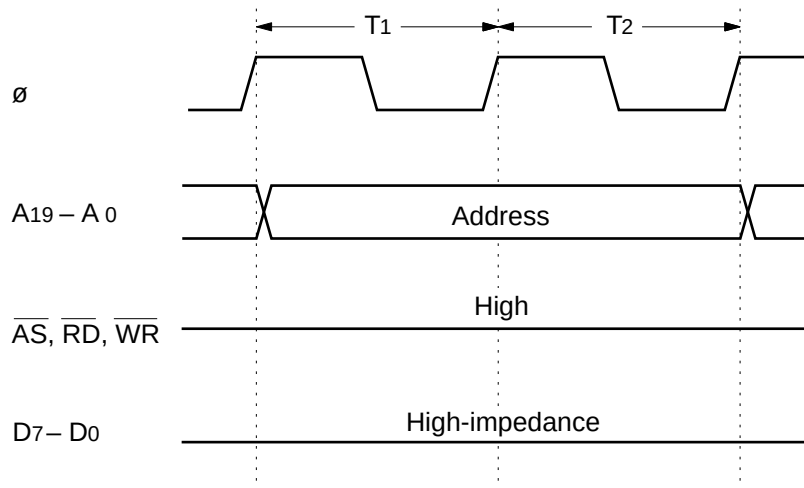


Figure 3-7 Pin States during Access to On-Chip Memory

3.7.4 Register Field Access Cycle (Addresses H'FF80 to H'FFFF)

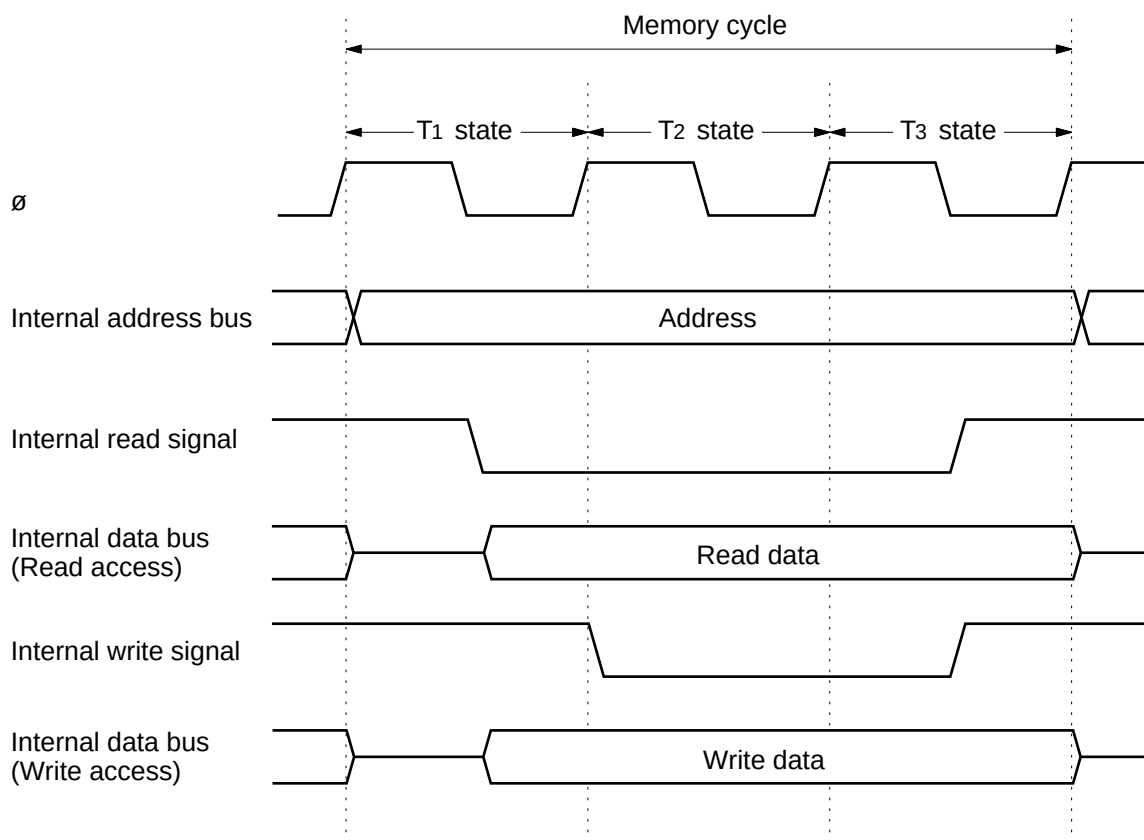


Figure 3-8 Register Field Access Timing

3.7.5 Pin States during Register Field Access (Addresses H'FF80 to H'FFFF)

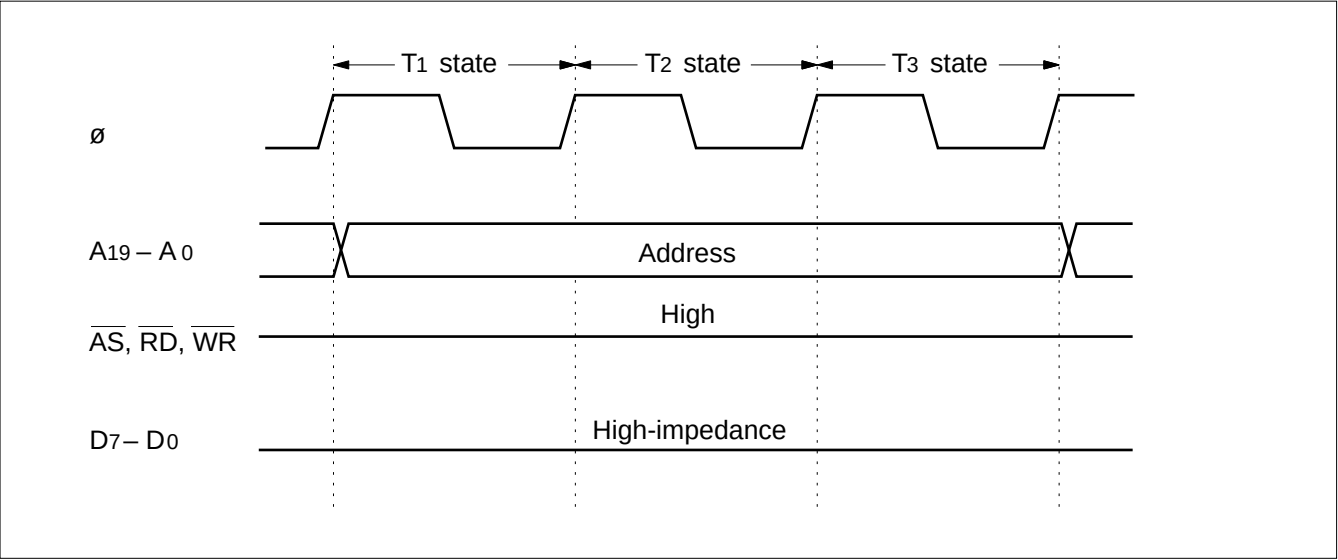


Figure 3-9 Pin States during Register Field Access

3.7.6 External Access Cycle

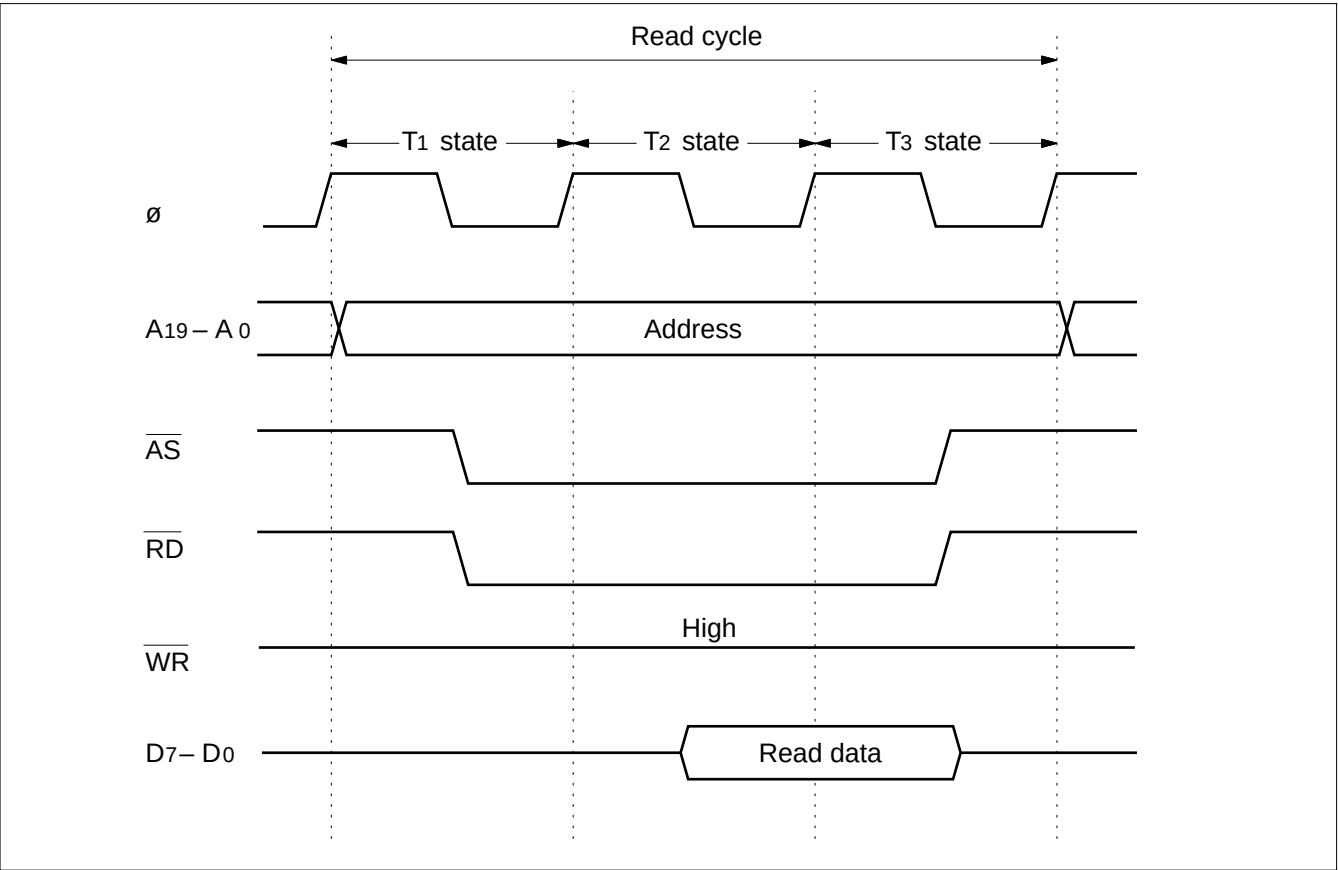


Figure 3-10 (a) External Access Cycle (Read Access)

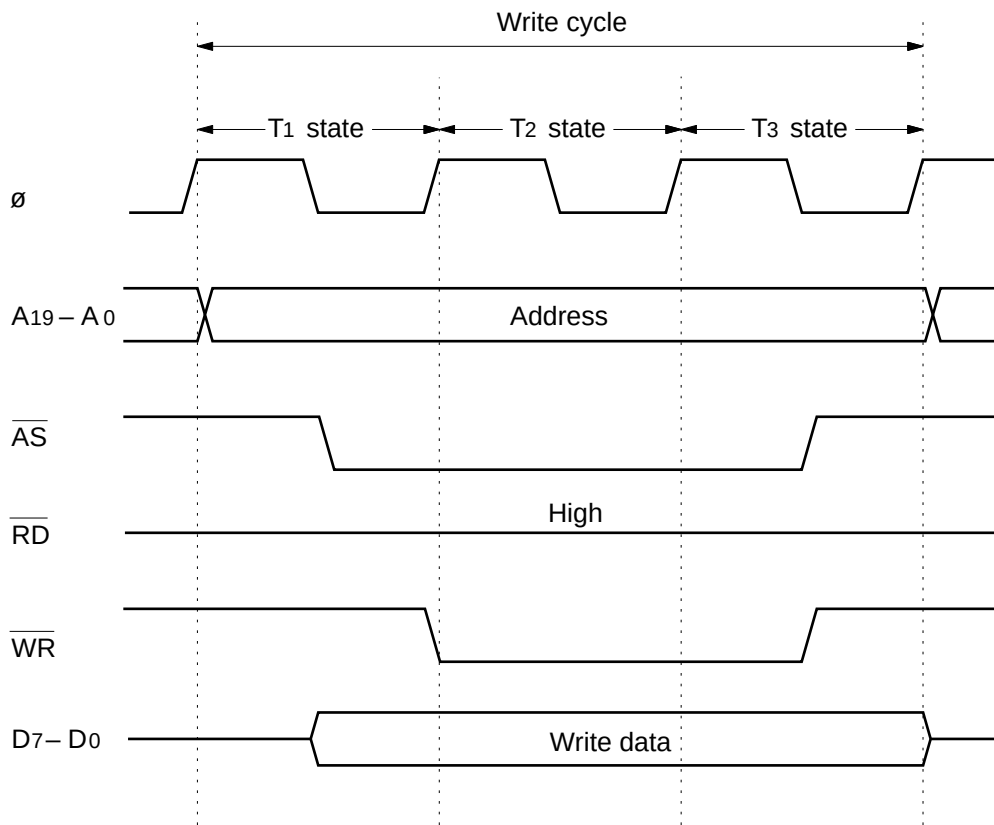


Figure 3-10 (b) External Access Cycle (Write Access)

3.8 CPU States

3.8.1 Overview

The CPU has four states: the program execution state, exception-handling state, reset state, and power-down state. The power-down state is further divided into the sleep mode, software standby mode, and hardware standby mode. Figure 3-11 summarizes these states, and figure 3-12 shows a map of the state transitions.

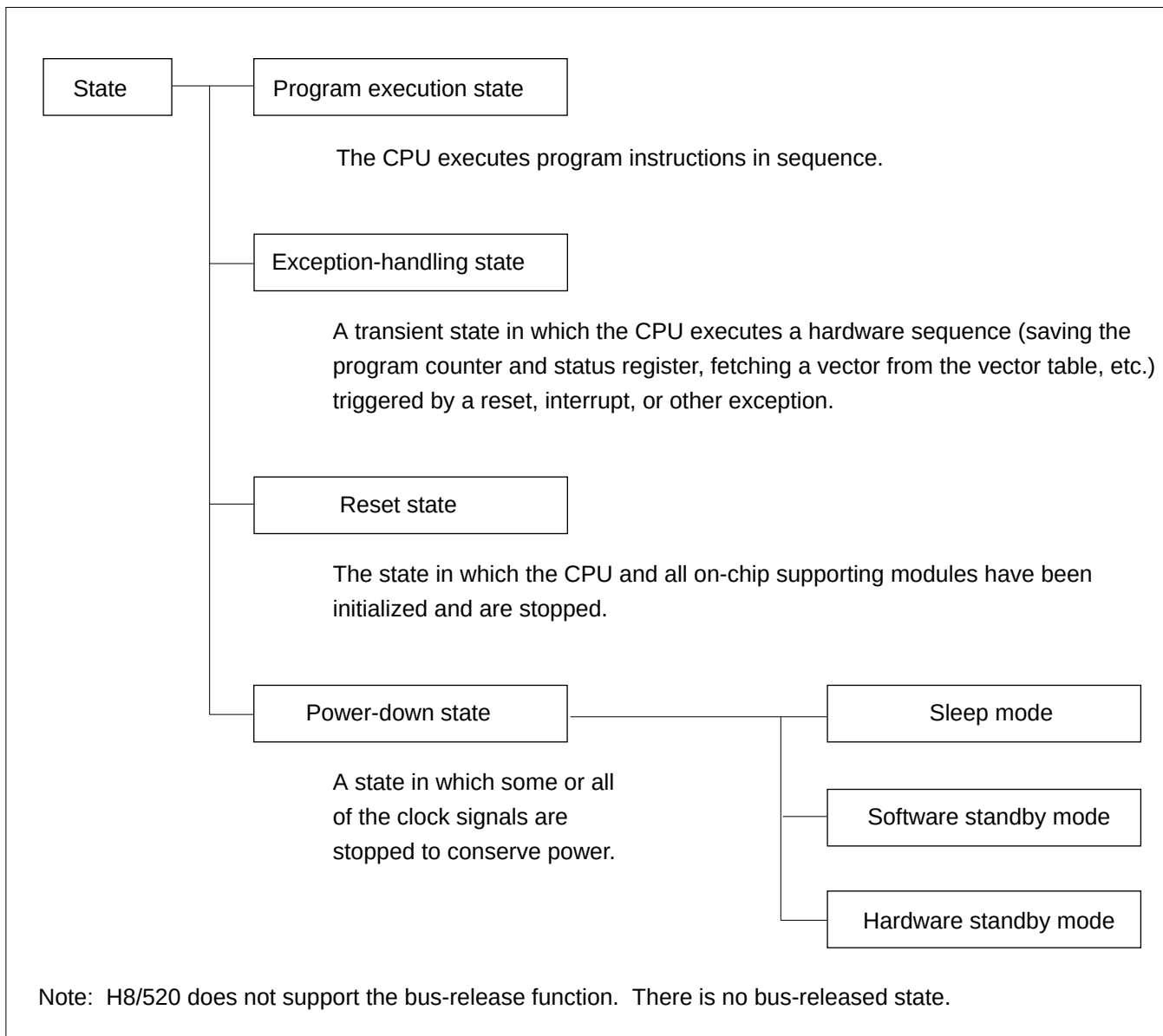
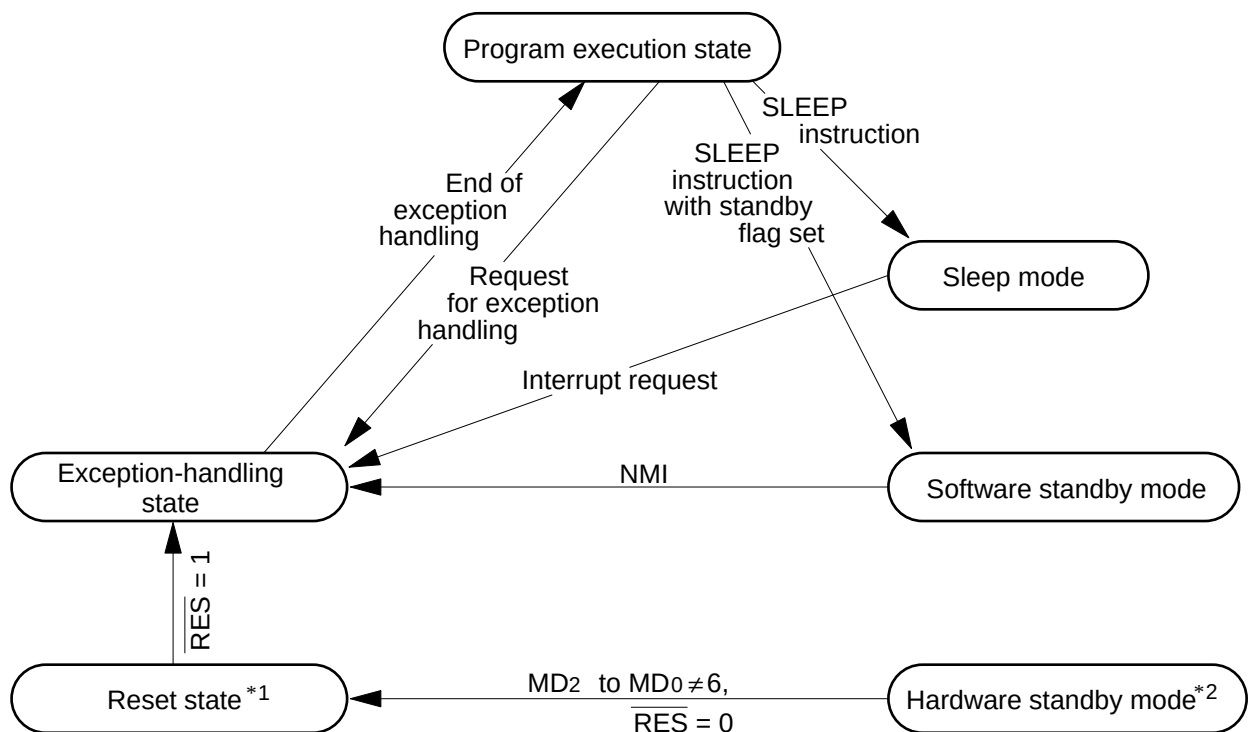


Figure 3-11 Operating States



- Notes: 1. From any state except the hardware standby mode, a transition to the reset state occurs whenever $\overline{\text{RES}}$ goes low or the watchdog timer requests a reset.
2. A transition to the hardware standby mode from any state occurs when the mode pins (MD2 to MD0) are set to 6.

Figure 3-12 State Transitions

3.8.2 Program Execution State

In this state the CPU executes program instructions in normal sequence.

3.8.3 Exception-Handling State

The exception-handling state is a transient state that occurs when the CPU alters the normal program flow due to an interrupt, trap instruction, address error, or other exception. In this state the CPU carries out a hardware-controlled sequence that prepares it to execute a user-coded exception-handling routine. In the hardware exception-handling sequence the CPU does the following:

1. Saves the program counter and status register (in minimum mode) or program counter, code page register, and status register (in maximum mode) to the stack.
2. Clears the T bit in the status register to 0.
3. Fetches the start address of the exception-handling routine from the exception vector table.
4. Branches to that address, returning to the program execution state.

See section 4, “Exception Handling”, for further information on the exception-handling state.

3.8.4 Reset State

In the reset state, the CPU and all on-chip supporting modules are initialized and placed in the stopped state. The CPU enters the reset state whenever the $\overline{\text{RES}}$ pin goes low, unless the CPU is currently in the hardware standby mode. It remains in the reset state until the $\overline{\text{RES}}$ pin goes high.

See section 4.2, “Reset”, for further information on the reset state.

3.8.5 Power-Down State

The power-down state comprises three modes: the sleep mode, software standby mode, and hardware standby mode.

See section 17, “Power-Down State”, for further information.

Section 4 Exception Handling

4.1 Overview

4.1.1 Types of Exception Handling and Their Priority

As indicated in table 4-1 (a) and (b), exception handling can be initiated by a reset, address error, trace, interrupt, or instruction. An instruction initiates exception handling if the instruction is an invalid instruction, a trap instruction, or a DIVXU instruction with zero divisor. Exception handling begins with a hardware exception-handling sequence which prepares for the execution of a user-coded software exception-handling routine.

There is a priority order among the different types of exceptions, as shown in table 4-1 (a). If two or more exceptions occur simultaneously, they are handled in their order of priority. An instruction exception cannot occur simultaneously with other types of exceptions.

Table 4-1 (a) Exceptions and Their Priority

Priority	Exception Type	Source	Detection Timing	Start of Exception-Handling Sequence
High	Reset	External, internal	$\overline{\text{RES}}$ Low-to-High transition	Immediately
	Address error	Internal	Instruction fetch or data read/write bus cycle	End of instruction execution
	Trace	Internal	End of instruction execution, if T = 1 in status register	End of instruction execution
	Interrupt	External, internal	End of instruction execution or end of exception-handling sequence	End of instruction execution
Low				

Table 4-1 (b) Instruction Exceptions

Exception Type	Start of Exception-Handling Sequence
Invalid instruction	Attempted execution of instruction with undefined code
Trap instruction	Started by execution of trap instruction
Zero divide	Attempted execution of DIVXU instruction with zero divisor

4.1.2 Hardware Exception-Handling Sequence

The hardware exception-handling sequence varies depending on the type of exception. When exception handling is initiated by an exception other than a reset, the CPU:

1. Saves the program counter and status register (in minimum mode) or program counter, code page register, and status register (in maximum mode) to the stack.
2. Clears the T bit in the status register to 0.
3. Fetches the start address of the exception-handling routine from the exception vector table.
4. Branches to that address.

For an interrupt, the CPU also alters the interrupt mask level in bits I₂ to I₀ of the status register.

For a reset, step 1 is omitted. See section 4.2, “Reset”, for the full reset sequence.

4.1.3 Exception Sources and Vector Table

The sources that initiate exception handling can be classified as shown in figure 4-1.

The starting addresses of the exception-handling routines for each source are contained in an exception vector table located in the low addresses of page 0. The vector addresses are listed in table 4-2. Note that there are different addresses for the minimum and maximum modes.

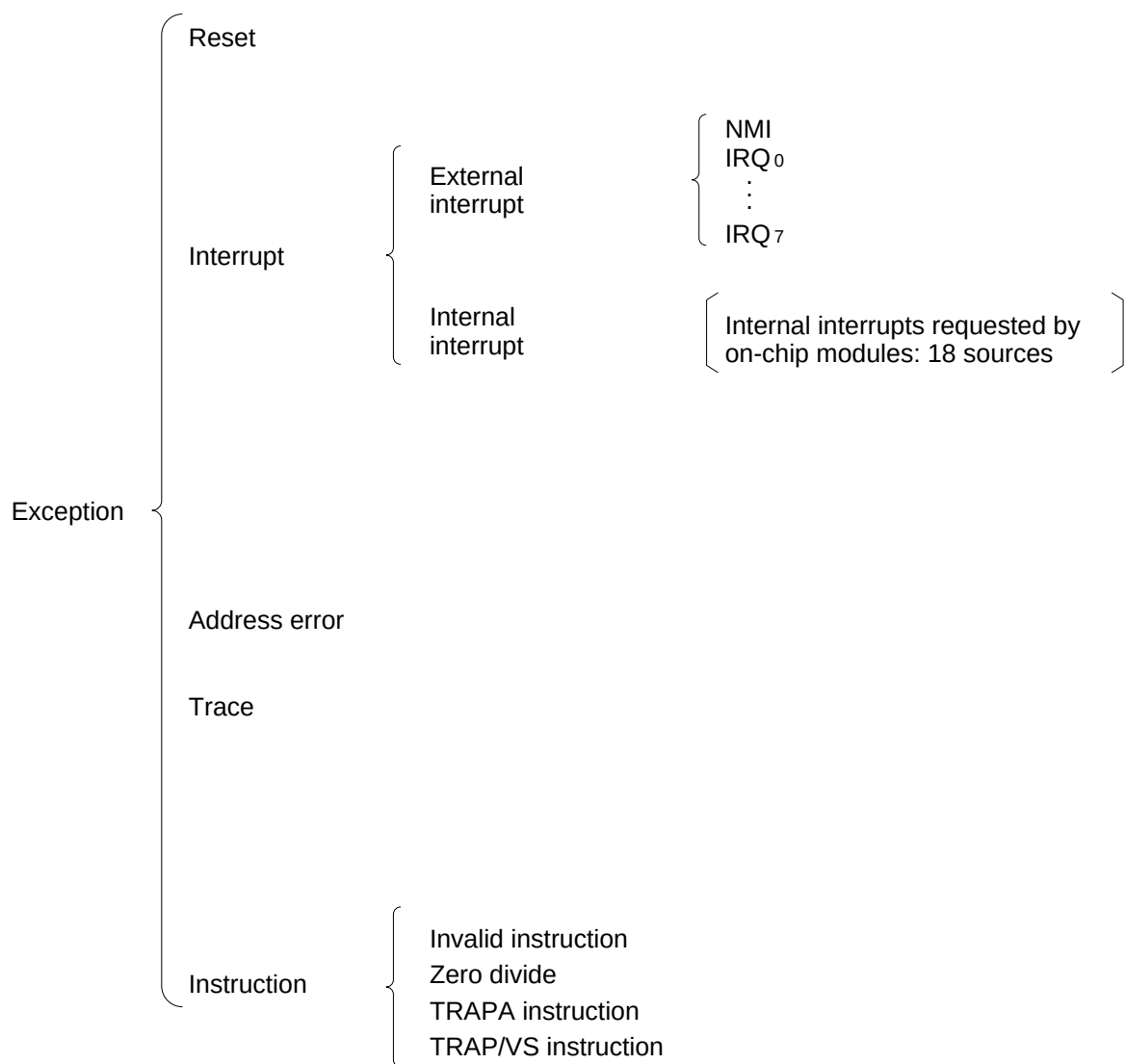


Figure 4-1 Sources Causing Exception Handling

Table 4-2 Exception Vector Table

Type of Exception	Vector Address	
	Minimum Mode	Maximum Mode
Reset (initialize PC)	H'0000 to H'0001	H'0000 to H'0003
— (Reserved for system)	H'0002 to H'0003	H'0004 to H'0007
Invalid instruction	H'0004 to H'0005	H'0008 to H'000B
DIVXU instruction (zero divide)	H'0006 to H'0007	H'000C to H'000F
TRAP/VS instruction	H'0008 to H'0009	H'0010 to H'0013
— (Reserved for system)	H'000A to H'000B	H'0014 to H'0017
	to	to
	H'000E to H'000F	H'001C to H'001F
Address error	H'0010 to H'0011	H'0020 to H'0023
Trace	H'0012 to H'0013	H'0024 to H'0027
— (Reserved for system)	H'0014 to H'0015	H'0028 to H'002B
Nonmaskable external interrupt (NMI)	H'0016 to H'0017	H'002C to H'002F
— (Reserved for system)	H'0018 to H'0019	H'0030 to H'0033
	to	to
	H'001E to H'001F	H'003C to H'003F
TRAPA instruction (16 vectors)	H'0020 to H'0021	H'0040 to H'0043
	to	to
	H'003E to H'003F	H'007C to H'007F
External interrupts IRQ ⁰ to IRQ ⁷	H'0040 to H'0041	H'0080 to H'0083
	to	to
	H'004E to H'004F	H'009C to H'009F
Internal interrupts	H'0050 to H'0051	H'00A0 to H'00A3
	to	to
	H'007E to H'007F	H'00FC to H'00FF

Notes: 1. The exception vector table is located at the beginning of page 0.
2. For details of the internal interrupt vectors, see table 5-2.

4.2 Reset

4.2.1 Overview

A reset has the highest exception-handling priority.

A reset can be generated by a low input at the $\overline{\text{RES}}$ pin or by a watchdog timer (WDT) overflow.

When the $\overline{\text{RES}}$ pin goes low, all current processing halts and the H8/520 chip enters the reset state. The internal status of the CPU and the contents of the registers of the on-chip supporting modules are initialized. When the $\overline{\text{RES}}$ pin returns from low to high, the hardware reset sequence described in the next section begins. To ensure that the H8/520 chip is reset correctly, the $\overline{\text{RES}}$ pin should be held low for at least 20 ms at power-up. To reset the H8/520 during operation, the $\overline{\text{RES}}$ pin should be held low for at least six system clock ($\emptyset$) cycles.

When the RSTOE bit (see below) is set to 1, the $\overline{\text{RES}}$ input must be held low for at least 520 system clock ($\emptyset$) cycles to reset the H8/520 chip.

When the watchdog timer operates in watchdog mode, if the watchdog timer counter (TCNT) overflows due to a program crash, for example, the watchdog timer generates an internal reset signal that resets the H8/520 chip. If in addition the reset output enable (RSTOE) bit in the reset control/status register (RSTCSR) is set to 1, a low output signal is generated at the $\overline{\text{RES}}$ pin for 132 system clock ($\emptyset$) cycles. This signal can be used to reset devices controlled by the H8/520.

See section 12, “Watchdog Timer”, for further information on the reset generated by the watchdog timer.

See appendix E, “Pin Status in the Reset State”, for the status of pins when a reset occurs.

4.2.2 Reset Sequence

When the $\overline{\text{RES}}$ pin returns to the high state after being held low for the necessary time, the hardware reset exception-handling sequence begins, during which:

1. The value at the mode pins (MD2 to MD0) is latched in bits MDS2 to MDS0 of the mode control register (MDCR).
2. In the status register (SR), the T bit is cleared to disable the trace mode, and the interrupt mask level (bits I2 to I0) is set to 7. A reset disables all interrupts.
3. The CPU loads the reset start address from the vector table into the program counter and begins executing the program at that address.

The contents of the vector table differs between minimum mode and maximum mode as indicated in figure 4-2. This affects step 3 as described below.

Minimum Mode: One word is copied from addresses H'0000 and H'0001 in the vector table to the program counter. Program execution then begins from the address in the program counter (PC).

Maximum Mode: Two words are read from addresses H'0000 to H'0003 in the vector table. The byte in address H'0000 is ignored. The byte in address H'0001 is copied to the code page register (CP). The contents of addresses H'0002 and H'0003 are copied to the program counter. Program execution starts from the address indicated by the code page register and program counter.

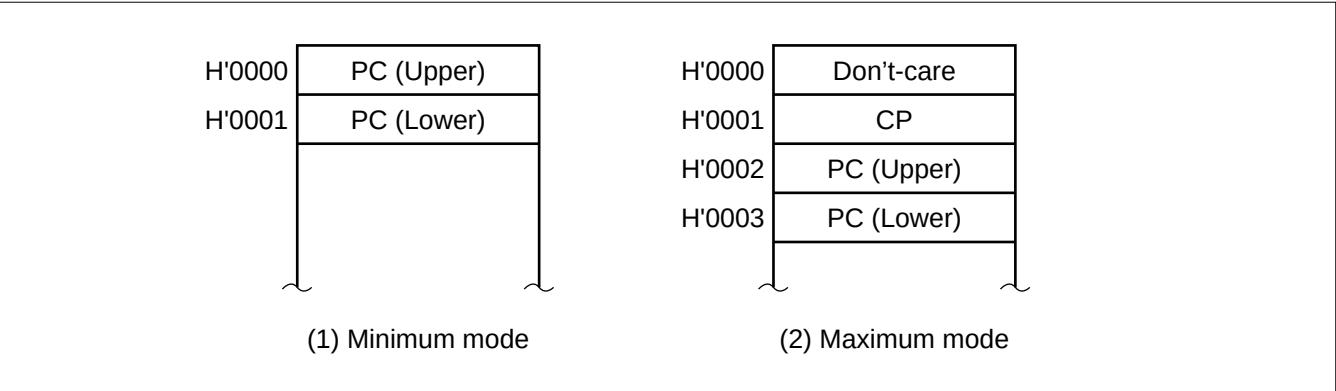


Figure 4-2 Reset Vector

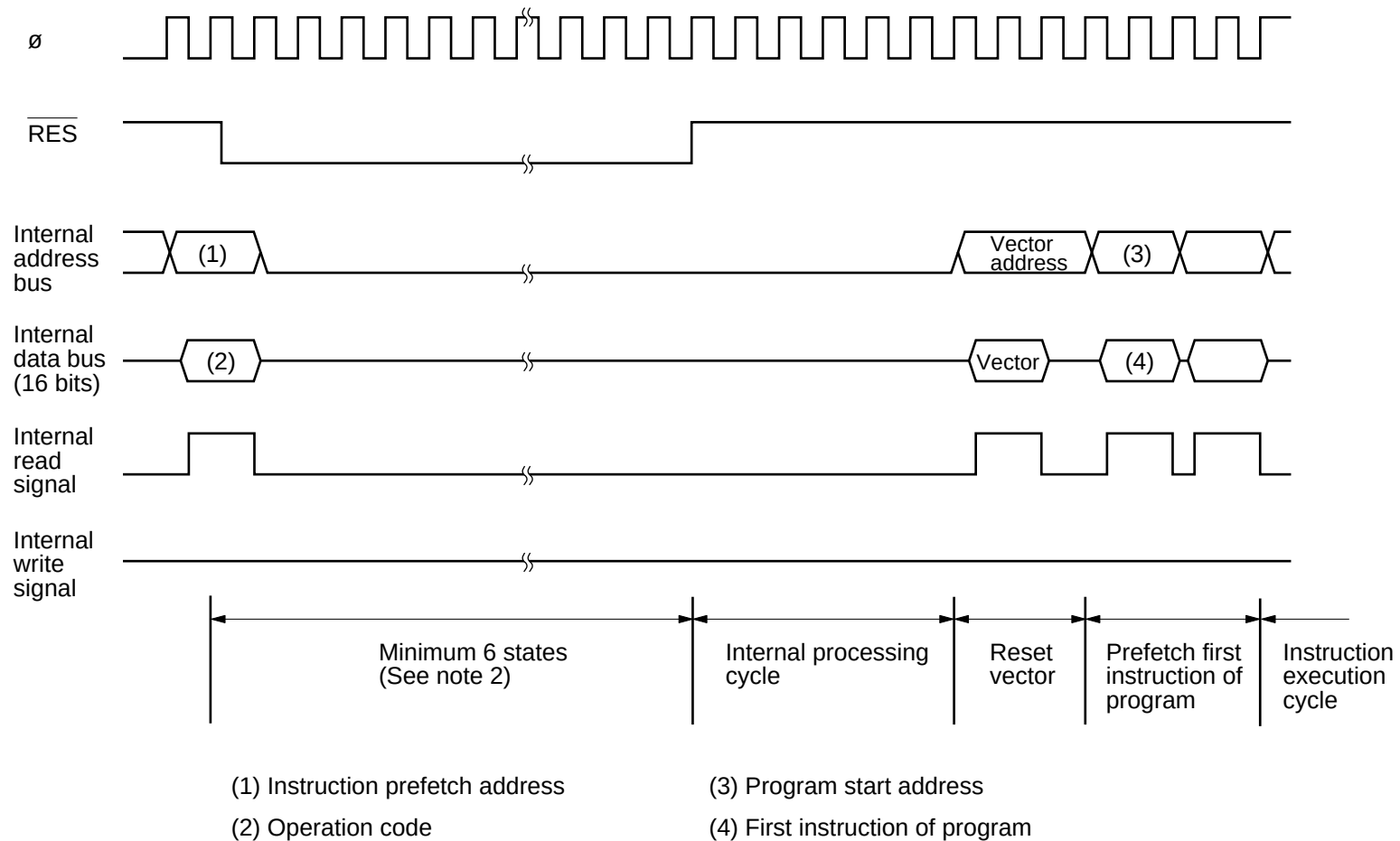
Figure 4-3 shows the timing of the reset sequence in minimum mode. Figure 4-4 shows the timing of the reset sequence in maximum mode.

4.2.3 Stack Pointer Initialization

The hardware reset sequence does not initialize the stack pointer, so this must be done by software. If an interrupt were to be accepted after a reset and before the stack pointer (SP) is initialized, the program counter and status register would not be saved correctly, causing a program crash. This danger can be avoided by coding the reset routine as explained next.

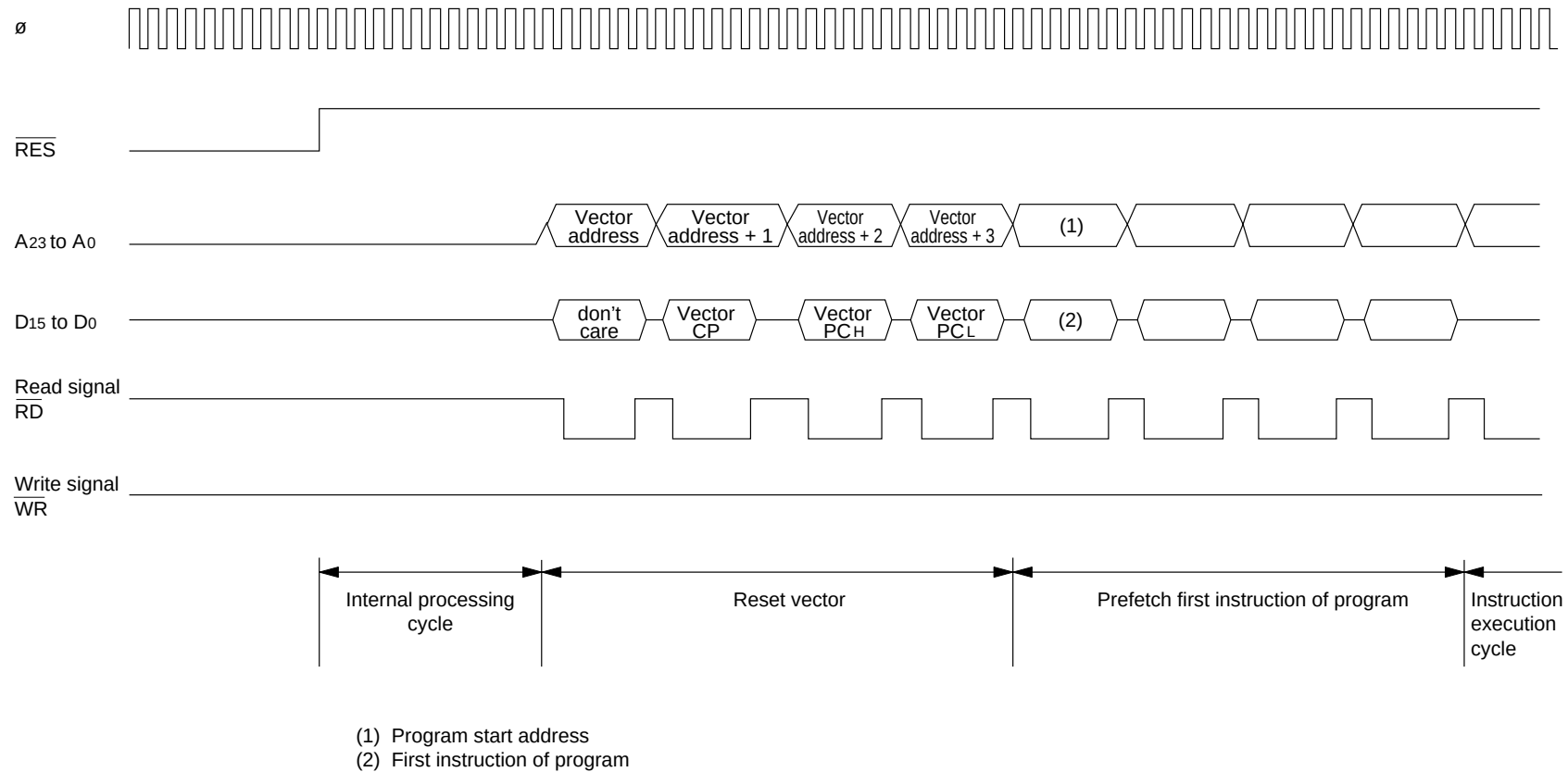
When the chip comes out of the reset state all interrupts, including NMI, are disabled, so the instruction at the reset start address is always executed. In the minimum mode, this instruction should initialize the stack pointer (SP). In the maximum mode, this instruction should be an LDC instruction initializing the stack page register (TP), and the next instruction should initialize the stack pointer. Execution of the LDC instruction disables interrupts again, ensuring that the stack pointer initializing instruction is executed.

Figure 4-3 Reset Sequence (Minimum Mode, On-Chip Memory)



Notes: 1. This timing chart applies to the minimum mode when the program and stack areas are both in on-chip memory and the program starts at an even address.
 2. Minimum 520 states when the RSTOE bit is set to 1.

Figure 4-4 Reset Sequence (Maximum Mode, External Memory)



Note: This diagram applies to maximum mode when the program area and vector table are both in external memory. After a reset, the wait-state controller inserts three wait states in each bus cycle.

4.3 Address Error

There are three causes of address errors:

- Instruction prefetch from illegal address
- Word data access at odd address
- Off-chip access in single-chip mode

An address error initiates the address error exception-handling sequence. This sequence clears the T bit of the status register to 0 to disable the trace mode, but does not affect the interrupt mask level in bits I2 to I0.

4.3.1 Instruction Prefetch from Illegal Address

An attempt to prefetch an instruction from the register field in memory addresses H'FF80 to H'FFFF causes an address error regardless of the MCU operating mode.

Handling of this address error begins when the prefetch cycle that caused the error has been completed and execution of the current instruction has also been completed. The program counter value pushed on the stack is the address of the instruction immediately following the last instruction executed. See section 4.9, “Stack Status after Completion of Exception Handling”, for a diagram of the stack.

Program code should not be located in addresses H'FF7D to H'FF7F. If the CPU executes an instruction in these addresses, it will attempt to prefetch the next instruction from the register field, causing an address error.

4.3.2 Word Data Access at Odd Address

If an attempt is made to access word data starting at an odd address, an address error occurs regardless of the MCU operating mode. The program counter value pushed on the stack in the handling of this error is the address of the next instruction after the instruction that attempted the illegal word access.

4.3.3 Off-Chip Address Access in Single-Chip Mode

In the single-chip mode there is no external memory, so in addition to the address errors described above, the following two types of address errors can occur.

Access to Addresses H'4000 to H'FD7F: These addresses exist neither in on-chip ROM or RAM nor in the on-chip register field, so an address error occurs if they are accessed for any purpose: for instruction prefetch, byte data access, or word data access.

Program code should not be located in the last three bytes of on-chip ROM (addresses H'3FFD to H'3FFF) in single-chip mode. If an instruction is located in these three bytes, the CPU will attempt to fetch the next instruction from addresses H'4000 to H'4002, causing an address error.

Access to Disabled RAM Area: The on-chip RAM area (H'FD80 to H'FF7F) can be disabled by clearing the RAME bit in the RAM control register (RAMCR). If any type of RAM access is attempted in this state in the single-chip mode, an address error occurs.

4.4 Trace

When the T bit of the status register is set to 1, the CPU operates in trace mode. A trace exception occurs at the completion of each instruction. The trace mode can be used to monitor program execution for debugging by a debugger.

In the trace exception sequence the T bit of the status register is cleared to 0 to disable the trace mode while the trace routine is executing. The interrupt mask level in bits I2 to I0 is not changed. Interrupts are accepted as usual during the trace routine.

In the status-register data saved on the stack, the T bit is set to 1. When the trace routine returns with the RTE instruction, the status register is popped from the stack and the trace mode resumes.

If an address error occurs during execution of the first instruction after the return from the trace routine, since the address error has higher priority, the address error exception-handling sequence is initiated, clearing the T bit in the status register to 0 and making it impossible to trace this instruction.

4.5 Interrupts

Interrupts can be requested from nine external sources (NMI and IRQ0 to IRQ7) and seven on-chip supporting modules: the 16-bit free-running timers (FRT1 and FRT2), the 8-bit timer, the serial communication interfaces (SCI1 and SCI2), the A/D converter, and the watchdog timer (WDT). The on-chip interrupt sources can request a total of eighteen different types of interrupts, each having its own interrupt vector. Figure 4-5 lists the interrupt sources and the number of different interrupts from each source.

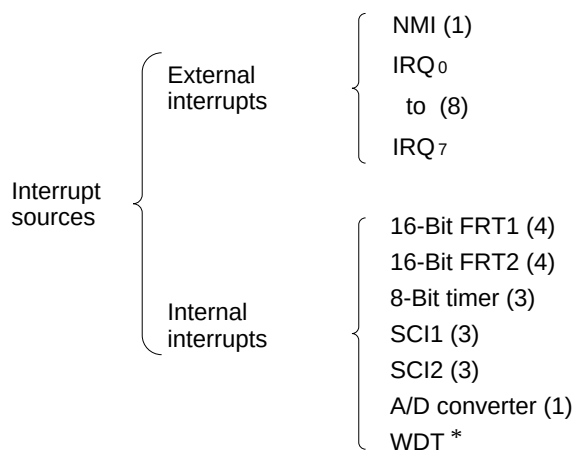
Each interrupt source has a priority. NMI interrupts have the highest priority, and are normally accepted unconditionally. The priorities of the other interrupt sources are set in interrupt priority registers A to D (IPRA to IPRD) in the register field at the high end of page 0 and can be changed by software. Priority levels range from 0 (low) to 7 (high), with NMI considered to be on level 8. IRQ1 to IRQ7 always have the same priority. The priority of IRQ0 can be set independently.

The on-chip interrupt controller decides whether an interrupt can be accepted by comparing its priority with the interrupt mask level, and determines the order in which to accept competing interrupt requests. Interrupts that are not accepted immediately remain pending until they can be accepted later.

When it accepts an interrupt, the interrupt controller also decides whether to have the interrupt handled by the CPU or the on-chip data transfer controller (DTC). This decision is controlled by bits set in data transfer enable registers A to D (DTEA to DTED) in the register field. The DTC is started if the corresponding DTE bit is set to 1; otherwise a CPU interrupt is generated. DTC interrupts provide an efficient way to send and receive blocks of data via the serial communication interface, or to transfer data between memory and I/O without detailed CPU programming. The CPU halts while the DTC is executing. DTC interrupts are described in section 6, “Data Transfer Controller”.

The hardware exception-handling sequence for a CPU interrupt clears the T bit in the status register to 0 and sets the interrupt mask level in bits I2 to I0 to the level of the interrupt it has accepted. This prevents the interrupt-handling routine from being interrupted except by a higher-level interrupt. The previous interrupt mask level is restored on the return from the interrupt-handling routine.

For further information on interrupts, see section 5, “Interrupt Controller”.



NMI: NonMaskable Interrupt

IRQ: Interrupt Request

FRT: Free-Running Timer

SCI: Serial Communication Interface

WDT: WatchDog Timer

Note: * Interrupts from the watchdog timer in the interval timer mode are handled as IRQ₀.

Figure 4-5 Interrupt Sources (and Number of Interrupt Types)

4.6 Invalid Instruction

An invalid instruction exception occurs if an attempt is made to execute an instruction with an undefined operation code or illegal addressing mode specification. The program counter value pushed on the stack is the value of the program counter when the invalid instruction code was detected.

In the invalid instruction exception-handling sequence the T bit of the status register is cleared to 0, but the interrupt mask level (I₂ to I₀) is not affected. If a normal interrupt is requested while a trap or zero-divide instruction is being executed, after the trap or zero-divide exception-handling sequence, the normal interrupt exception-handling sequence is carried out.

4.7 Trap Instructions and Zero Divide

A trap exception occurs when the TRAPA or TRAP/VS instruction is executed. A zero divide exception occurs if an attempt is made to execute a DIVXU instruction with a zero divisor.

In the exception-handling sequences for these exceptions the T bit of the status register is cleared to 0, but the interrupt mask level (I₂ to I₀) is not affected.

TRAPA Instruction: The TRAPA instruction always causes a trap exception. The TRAPA instruction includes a vector number from 0 to 15, allowing the user to provide up to sixteen different trap-handling routines.

TRAP/VS Instruction: When the TRAP/VS instruction is executed, a trap exception occurs if the overflow (V) bit in the condition code register is set to 1. If the V bit is cleared to 0, no exception occurs and the next instruction is executed.

DIVXU Instruction with Zero Divisor: An exception occurs if an attempt is made to divide by zero in a DIVXU instruction.

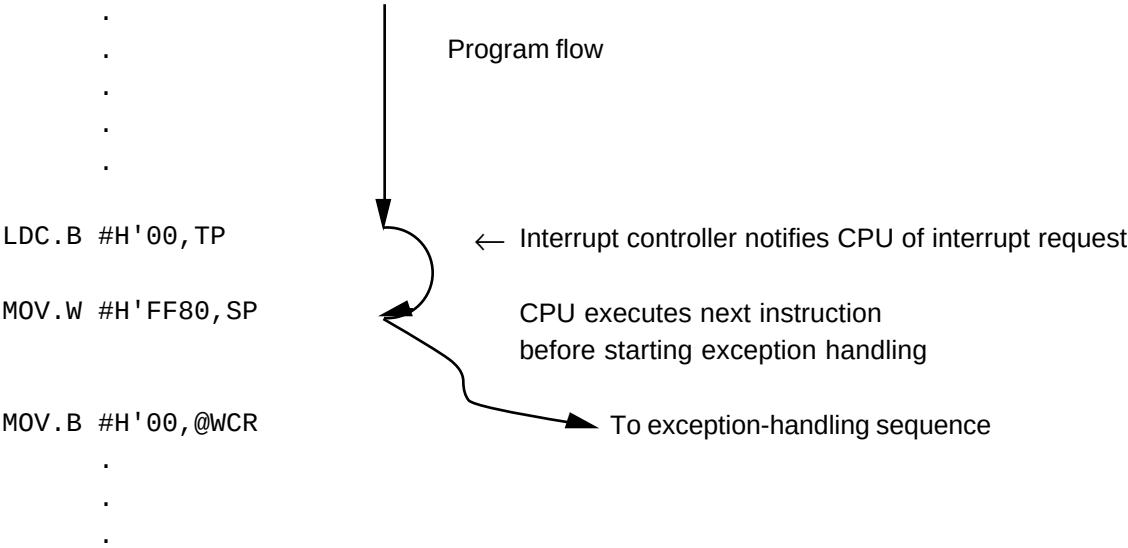
4.8 Cases in Which Exception Handling is Deferred

In the case described next, the address error exception, trace exception, external interrupt (NMI and IRQ₀ to IRQ₇) requests, and internal interrupt requests (18 types) are not accepted immediately but are deferred until after the next instruction has been executed.

4.8.1 Instructions that Disable Interrupts

Interrupts are disabled immediately after the execution of five instructions: XORC, ORC, ANDC, LDC, and RTE.

Suppose that an internal interrupt is requested and the interrupt controller, after checking the interrupt priority and interrupt mask level, notifies the CPU of the interrupt, but the CPU is currently executing one of the five instructions listed above. After executing this instruction the CPU always proceeds to the next instruction. (And if the next instruction is one of these five, the CPU also proceeds to the next instruction after that.) The exception-handling sequence starts after the next instruction that is not one of these five has been executed. The following is an example:



Note: When the LDC instruction alters the I bits in the status register (SR), the new I-bit values do not take effect until three states after the LDC instruction. If a program running in on-chip memory uses the LDC instruction to enable interrupts by modifying the I bits and the next instruction is a two-state instruction (NOP for example), interrupts will not be accepted after this next instruction; they will not be accepted until another instruction has been executed after that. The same applies to the ANDC, ORC, and XORC instructions.

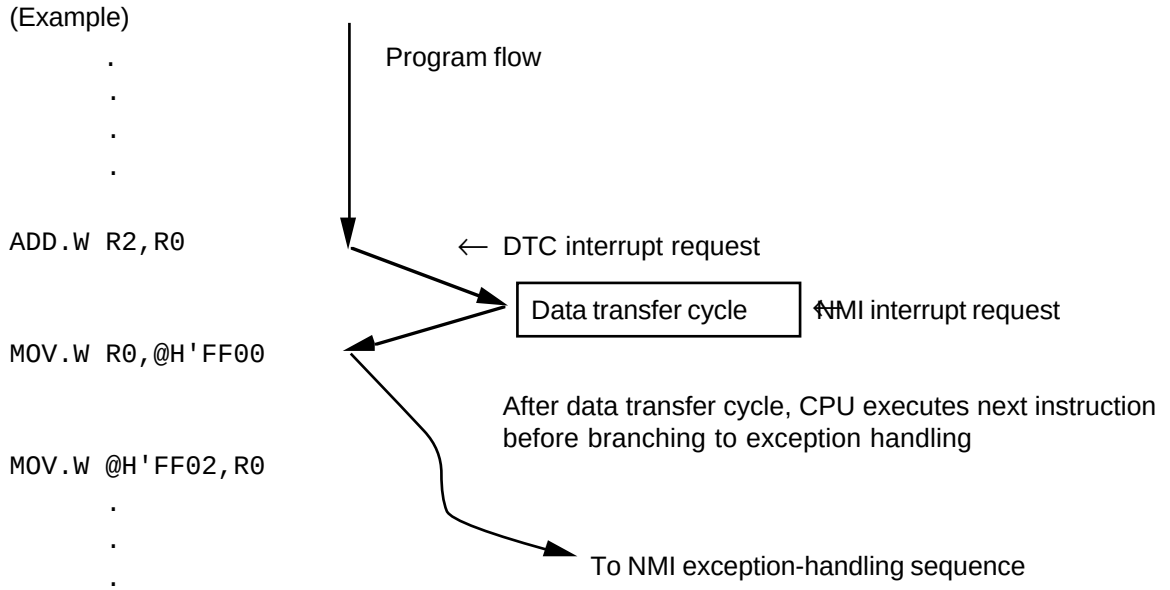
4.8.2 Disabling of Exceptions Immediately after a Reset

If an interrupt is accepted after a reset and before the stack pointer (SP) is initialized, the program counter and status register will not be saved correctly, leading to a program crash. To prevent this, when the chip comes out of the reset state all interrupts, including NMI, are disabled, so the first instruction of the reset routine is always executed. As noted earlier, in the minimum mode, this instruction should initialize the stack pointer (SP). In the maximum mode, the first instruction should be an LDC instruction that initializes the stack page register (TP); the next instruction should initialize the stack pointer.

4.8.3 Disabling of Interrupts after a Data Transfer Cycle

If an interrupt starts the data transfer controller and another interrupt is requested during the data transfer cycle, when the data transfer cycle ends, the CPU always executes the next instruction before handling the second interrupt.

Even if a nonmaskable interrupt (NMI) occurs during a data transfer cycle, it is not accepted until the next instruction has been executed. An example of this is shown below.



4.9 Stack Status after Completion of Exception Handling

The status of the stack after an exception-handling sequence is described below.

Table 4-3 shows the stack after completion of the exception-handling sequence for various types of exceptions in the minimum and maximum modes.

Table 4-3 Stack after Exception Handling Sequence

Exception	Minimum Mode	Maximum Mode
Trace	SP ▷ SR (upper byte) SR (lower byte)	TP:SP ▷ SR (upper byte) SR (lower byte)
Interrupt	Next instruction address (upper byte) Next instruction address (lower byte)	Don't-care Next instruction page (8 bits)
Trap		Next instruction address (upper byte) Next instruction address (lower byte)
Zero divide (DIVXU)		

Note: The RTE instruction returns to the next instruction after the instruction being executed when the exception occurred.

Table 4-3 Stack after Exception Handling Sequence (cont)

Exception	Minimum Mode	Maximum Mode
Invalid instruction	SP ▸ SR (upper byte) SR (lower byte) PC when error occurred (upper byte) PC when error occurred (lower byte)	TP:SP ▸ SR (upper byte) SR (lower byte) Don't-care CP when error occurred (8 bits) PC when error occurred (upper byte) PC when error occurred (lower byte)

Note: The program counter value pushed on the stack is not necessarily the address of the first byte of the invalid instruction.

Exception	Minimum Mode	Maximum Mode
Address error	SP ▸ SR (upper byte) SR (lower byte) PC when error occurred (upper byte) PC when error occurred (lower byte)	TP:SP ▸ SR (upper byte) SR (lower byte) Don't-care CP when error occurred (8 bits) PC when error occurred (upper byte) PC when error occurred (lower byte)

Note: The program counter value pushed on the stack is the address of the next instruction after the last instruction successfully executed.

4.9.1 PC Value Pushed on Stack for Trace, Interrupts, Trap Instructions, and Zero Divide Exceptions

The program counter value pushed on the stack for a trace, interrupt, trap, or zero divide exception is the address of the next instruction at the time when the interrupt or exception was accepted. The RTE instruction accordingly returns to the next instruction after the instruction executed before the exception-handling sequence.

4.9.2 PC Value Pushed on Stack for Address Error and Invalid Instruction Exceptions

The program counter value pushed on the stack for an address error or invalid instruction exception differs depending on the conditions when the exception occurred.

4.10 Notes on Use of the Stack

If the stack pointer is set to an odd address, an address error will occur when the stack is accessed during interrupt handling or for a subroutine call. The stack pointer should always point to an even address. To keep the stack pointer pointing to an even address, a program should use word data size when saving or restoring registers to and from the stack.

In the @-SP or @SP+ addressing mode, the CPU performs word access even if the instruction specifies byte size. (This is not true in the @-Rn and @Rn+ addressing modes when Rn is a register from R0 to R6.)

Section 5 Interrupt Controller

5.1 Overview

The interrupt controller decides which interrupts to accept, and how to deal with multiple interrupts and other exceptions. It also decides whether an interrupt should be served by the CPU or by the data transfer controller (DTC). This section explains the features of the interrupt controller, describes its internal structure and control registers, and details the handling of interrupts.

For detailed information on the data transfer controller, see section 6, “Data Transfer Controller”.

5.1.1 Features

The main features of the interrupt controller are as follows:

- Interrupt priorities are user-programmable.
User programs can set priority levels from 7 (high) to 0 (low) in four interrupt priority (IPR) registers for IRQ0, IRQ1 to IRQ7, and each of the on-chip supporting modules—for every interrupt, that is, except the nonmaskable interrupt (NMI). NMI has the highest priority level (8) and is normally always accepted. An interrupt with priority level 0 is always masked.
- Multiple interrupts on the same level are served in a default priority order.
Lower-priority interrupts remain pending until higher-priority interrupts have been handled.
- For most interrupts, software can select whether to have the interrupt served by the CPU or the on-chip data transfer controller (DTC).
User programs can make this selection by setting and clearing bits in four data transfer enable (DTE) registers. The data transfer controller can be started by any interrupts except NMI, IRQ4 to IRQ7, the error interrupt (ERI) from the on-chip serial communication interface, and the overflow interrupts (FOVI and OVI) from the on-chip timers.
- Software can select the NMI edge and can enable or disable $\overline{\text{IRQ}}_0$ to $\overline{\text{IRQ}}_7$.
The NMI control register (NMICR) determines whether a nonmaskable interrupt is triggered by the rising or falling edge of the NMI input signal. The IRQ control register (IRQCR) enables or disables $\overline{\text{IRQ}}_0$ to $\overline{\text{IRQ}}_7$.

5.1.2 Block Diagram

Figure 5-1 shows the block configuration of the interrupt controller.

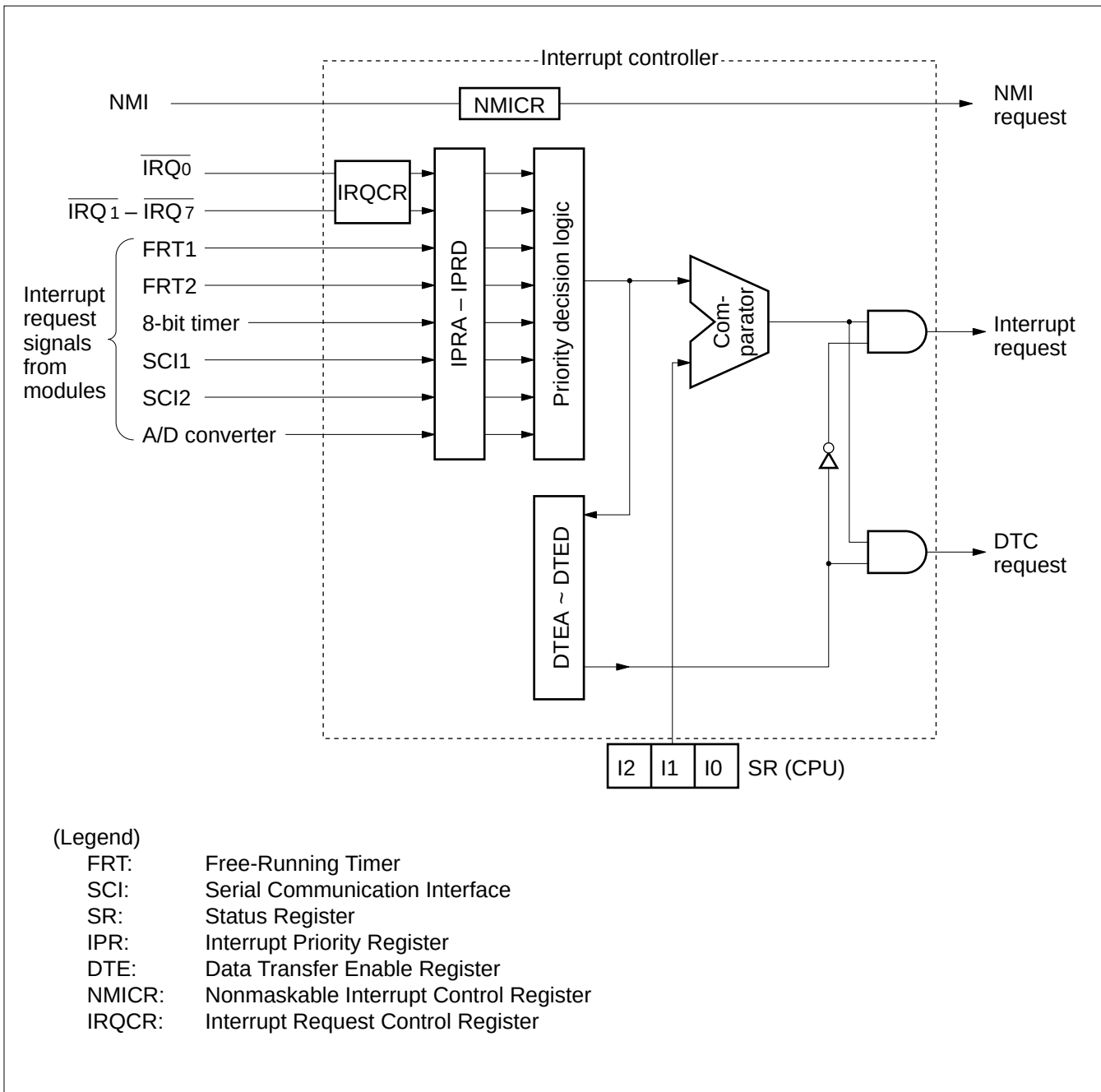


Figure 5-1 Interrupt Controller Block Diagram

5.1.3 Register Configuration

Table 5-1 lists the attributes of the registers used by the interrupt controller.

Table 5-1 Interrupt Controller Registers

Name		Abbreviation	Read/Write	Initial Value	Address
Interrupt priority register	A	IPRA	R/W	H'00	H'FFF0
	B	IPRB	R/W	H'00	H'FFF1
	C	IPRC	R/W	H'00	H'FFF2
	D	IPRD	R/W	H'00	H'FFF3
Data transfer enable register	A	DTEA	R/W	H'00	F'FFF4
	B	DTEB	R/W	H'00	H'FFF5
	C	DTEC	R/W	H'00	H'FFF6
	D	DTED	R/W	H'00	H'FFF7
NMI control register		NMICR	R/W	H'FE	H'FFFC
IRQ control register		IRQCR	R/W	H'00	H'FFFD

See section 6.2.5, “Data Transfer Enable Registers A to D”, for detailed information about DTEA to DTED.

5.2 Interrupt Types

There are 27 distinct types of interrupts: 9 external interrupts originating off-chip and 18 internal interrupts originating in the on-chip supporting modules.

5.2.1 External Interrupts

The nine external interrupts are NMI and IRQ0 to IRQ7.

NMI (Non Maskable Interrupt): This interrupt has the highest priority level (8) and cannot be masked. An NMI is generated by input to the NMI pin. The input at the NMI pin is edge-sensed. A user program can select whether to have the interrupt occur on the rising edge or falling edge of the NMI input by setting or clearing the nonmaskable interrupt edge bit (NMIEG) in the NMI control register (NMICR).

In the NMI exception-handling sequence, the T (Trace) bit in the CPU status register (SR) is cleared to 0, and the interrupt mask level in I2 to I0 is set to 7, masking all other interrupts. The interrupt controller holds the NMI request until the NMI exception-handling sequence begins, then clears the NMI request, so if another interrupt is requested at the NMI pin during the NMI exception-handling sequence, the NMI exception-handling sequence will be carried out again.

Coding Examples:

To select the rising edge of the NMI input: `BSET.B #0, @H'FFFC`

To select the falling edge of the NMI input: `BCLR.B #0, @H'FFFC`

IRQ0 (Interrupt Request 0): An IRQ0 interrupt can be requested by a low input to the $\overline{\text{IRQ0}}$ pin and/or a watchdog timer overflow. A low $\overline{\text{IRQ0}}$ input requests an IRQ0 interrupt if the interrupt request enable 0 bit (IRQ0 E) in the IRQ control register (IRQCR) is set to 1. The interrupt controller samples the level of the $\overline{\text{IRQ0}}$ pin directly, so this pin must be held low until the interrupt is accepted. Otherwise the request will be ignored.

A watchdog timer overflow requests an IRQ0 interrupt if the TME bit is set to 1 and the WT/ $\overline{\text{IT}}$ bit is cleared to 0 in the watchdog timer's control/status register. See section 12, "Watchdog Timer", for details of the watchdog timer.

The IRQ0 interrupt can be assigned any priority level from 7 to 0 by setting the corresponding value in the upper four bits of IPRA. If bit 4 of data transfer enable register A (DTEA) is set to 1, an IRQ0 interrupt starts the data transfer controller. Otherwise the interrupt is served by the CPU.

In the CPU interrupt-handling sequence for IRQ0 the T bit of the status register is cleared to 0, and the interrupt mask level is set to the value in the upper four bits of IPRA.

Coding Examples:

To enable IRQ0 to be requested by $\overline{\text{IRQ0}}$ input: `BSET.B #0, @H'FFFD`

To assign priority level 7 to IRQ0: `OR.B #70, @H'FFF0`

To have IRQ0 start the DTC: `BSET.B #4, @H'FFF4`

IRQ1 to IRQ7 (Interrupt Request 1 to 7): An IRQ1 to IRQ7 interrupt is requested by a high-to-low transition at the $\overline{\text{IRQ1}}$ to $\overline{\text{IRQ7}}$ pin. The IRQ1 to IRQ7 interrupt is enabled only when the interrupt request enable bit IRQ1E to IRQ7E in the IRQ control register is set to 1. The $\overline{\text{IRQ1}}$ to $\overline{\text{IRQ7}}$ input is latched in the interrupt controller and held until the interrupt request is accepted.

The IRQ₁ to IRQ₇ interrupts can be assigned any priority level from 7 (high) to 0 (low) by setting the corresponding value in the lower four bits of IPRA. These seven interrupts always have the same priority. They cannot be assigned priorities separately.

If bits 0 to 2 of data transfer enable register A (DTEA) are set to 1, IRQ₁ to IRQ₃ can start the data transfer controller. Otherwise the interrupt is served by the CPU. IRQ₄ to IRQ₇ cannot start the data transfer controller; they are always served by the CPU.

The interrupt controller holds IRQ₁ to IRQ₇ requests until the corresponding exception-handling sequence begins, then clears the request. Contention among IRQ₁ to IRQ₇ is resolved when the CPU accepts the interrupt by taking the interrupt with the highest priority first and holding lower-priority interrupts pending.

During the interrupt-handling routine, if the same external interrupt is requested again the request is held, but the exception-handling sequence is not carried out immediately because the interrupt is masked by bits I₂ to I₀ in the status register. On return from the interrupt-handling routine one more instruction is executed, then the pending exception-handling sequence is carried out.

In the CPU interrupt-handling sequence for IRQ₁ to IRQ₇, the T bit of the CPU status register is cleared to 0, and the interrupt mask level is set to the value in the lower four bits of IPRA.

Coding Examples:

To enable IRQ₁ to be requested by $\overline{\text{IRQ}}_1$ input:

```
BSET.B #1, @H'FFFD
```

To assign priority level 7 to IRQ₀ and level 5 to IRQ₁ to IRQ₇:

```
MOV.B #75, @H'FFF0
```

To have IRQ₁ start the DTC:

```
BSET.B #0, @H'FFF4
```

5.2.2 Internal Interrupts

Eighteen types of internal interrupts can be requested by the on-chip supporting modules. Each interrupt is separately vectored in the exception vector table, so it is not necessary for the user-coded interrupt handler routine to determine which type of interrupt has occurred.

Each of the internal interrupts can be enabled or disabled by setting or clearing an enable bit in the control register of the on-chip supporting module.

An interrupt priority level from 7 to 0 can be assigned to each on-chip supporting module by setting interrupt priority registers B to D. Within each module, different interrupts have a fixed priority order. For most of these interrupts, values set in data transfer enable registers B to D can select whether to have the interrupt served by the CPU or the data transfer controller.

In the CPU interrupt-handling sequence, the T bit of the CPU status register is cleared to 0, and the interrupt mask level in bits I2 to I0 is set to the value in the IPR.

5.2.3 Interrupt Vector Table

Table 5-2 lists the addresses of the exception vector table entries for each interrupt, and explains how their priority is determined. For the on-chip supporting modules, the priority level set in the interrupt priority register applies to the module as a whole: all interrupts from that module have the same priority level. A separate priority order is established among interrupts from the same module. If the same priority level is assigned to two or more modules and two interrupts are requested simultaneously from these modules, they are served in the priority order indicated in the rightmost column in table 5-2.

A reset clears the interrupt priority registers so that all interrupts except NMI start with priority level 0, meaning that they are unconditionally masked.

Table 5-2 Interrupts, Vectors, and Priorities

		Assignable Priority Levels (Initial Level)	IPR Bits	Priority within Module	Vector Table Entry Address		Priority among Interrupts on Same Level*
Interrupt					Minimum Mode	Maximum Mode	
NMI		8 (8)	—	—	H'0016 to H'0017	H'002C to H'002F	High ↑ Low
IRQ	IRQ ₀	7 to 0 (0)	IPRA Upper 4 bits	—	H'0040 to H'0041	H'0080 to H'0083	
	IRQ ₁	7 to 0	IPRA	6	H'0042 to H'0043	H'0084 to H'0087	
	IRQ ₂		Lower 4 bits	5	H'0044 to H'0045	H'0088 to H'008B	
	IRQ ₃			4	H'0046 to H'0047	H'008C to H'008F	
	IRQ ₄			3	H'0048 to H'0049	H'0090 to H'0093	
	IRQ ₅			2	H'004A to H'004B	H'0094 to H'0097	
	IRQ ₆			1	H'004C to H'004D	H'0098 to H'009B	
	IRQ ₇	(0)		0	H'004E to H'004F	H'009C to H'009F	
FRT1	ICI	7 to 0	IPRB	3	H'0050 to H'0051	H'00A0 to H'00A3	
	OCIA		Upper 4 bits	2	H'0052 to H'0053	H'00A4 to H'00A7	
	OCIB			1	H'0054 to H'0055	H'00A8 to H'00AB	
	FOVI	(0)		0	H'0056 to H'0057	H'00AC to H'00AF	
FRT2	ICI	7 to 0	IPRB	3	H'0058 to H'0059	H'00B0 to H'00B3	
	OCIA		Lower 4 bits	2	H'005A to H'005B	H'00B4 to H'00B7	
	OICB			1	H'005C to H'005D	H'00B8 to H'00BB	
	FOVI	(0)		0	H'005E to H'005F	H'00BC to H'00BF	
8-Bit timer	CMIA	7 to 0	IPRC	2	H'0060 to H'0061	H'00C0 to H'00C3	
	CMIB		Upper 4 bits	1	H'0062 to H'0063	H'00C4 to H'00C7	
	OVI	(0)		0	H'0064 to H'0065	H'00C8 to H'00CB	
SCI1	ERI	7 to 0	IPRC	2	H'0068 to H'0069	H'00D0 to H'00D3	
	RXI		Upper 4 bits	1	H'006A to H'006B	H'00D4 to H'00D7	
	TXI	(0)		0	H'006C to H'006D	H'00D8 to H'00DB	
SCI2	ERI	7 to 0	IPRD	2	H'0070 to H'0071	H'00E0 to H'00E3	
	RXI		Upper 4 bits	1	H'0072 to H'0073	H'00E4 to H'00E7	
	TXI	(0)		0	H'0074 to H'0075	H'00E8 to H'00EB	
A/D converter	ADI	7 to 0 (0)	IPRD Lower 4 bits	—	H'0078 to H'0079	H'00F0 to H'00F3	

Note: * If two or more interrupts are requested simultaneously, they are handled in order of priority level, as set in registers IPRA to IPRD. If they have the same priority level because they are requested from the same on-chip supporting module, they are handled in a fixed priority order within the module. If they are requested from different modules to which the same priority level is assigned, they are handled in the order indicated in the right-hand column.

5.3 Register Descriptions

5.3.1 Interrupt Priority Registers A to D (IPRA to IPRD)

IRQ₀, IRQ₁ to IRQ₇, and the on-chip supporting modules are each assigned three bits in one of the four interrupt priority registers (IPRA to IPRD). These bits specify a priority level from 7 (high) to 0 (low) for interrupts from the corresponding source. The drawing below shows the configuration of the interrupt priority registers. Table 5-3 lists their assignments to interrupt sources.

Bit	7	6	5	4	3	2	1	0
	—				—			
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R/W	R/W	R/W	R	R/W	R/W	R/W

Note: Bits 7 and 3 are reserved. They cannot be modified and are always read as 0.

Table 5-3 Assignment of Interrupt Priority Registers

Register	Interrupt Request Source	
	Bits 6 to 4	Bits 2 to 0
IPRA	IRQ ₀	IRQ ₁ to IRQ ₇
IPRB	FRT1	FRT2
IPRC	8-Bit timer	SCI1
IPRD	SCI2	A/D converter

As table 5-3 indicates, each interrupt priority register specifies priority levels for two interrupt sources. A user program can assign desired levels to these interrupt sources by writing 000 in bits 6 to 4 or bits 2 to 0 to set priority level 0, for example, or 111 to set priority level 7.

A reset clears registers IPRA to IPRD to H'00, so all interrupts except NMI are initially masked.

When the interrupt controller receives one or more interrupt requests, it selects the request with the highest priority and compares its priority level with the interrupt mask level set in bits I₂ to I₀ in the CPU status register. If the priority level is higher than the mask level, the interrupt controller passes the interrupt request to the CPU (or starts the data transfer controller). If the priority level is lower than the mask level, the interrupt controller leaves the interrupt request pending until the interrupt mask is altered to a lower level or the interrupt priority is raised. Similarly, if it receives two interrupt requests with the same priority level, the interrupt controller determines their priority as explained in table 5-2 and leaves the interrupt request with the lower priority pending.

The interrupt controller requires two system clock (ϕ) periods to determine the priority level of an interrupt. Accordingly, when an instruction modifies an instruction priority register, the new priority does not take effect until after the next instruction has been executed.

5.3.2 NMI Control Register (NMICR)—H'FFFC

Bit	7	6	5	4	3	2	1	0
	—	—	—	—	—	—	—	NMIEG
Initial value	1	1	1	1	1	1	1	0
Read/Write	—	—	—	—	—	—	—	R/W

The NMI control register (NMICR) is an 8-bit register that selects the edge of the NMI input signal which triggers a nonmaskable interrupt.

The NMICR is initialized to H'FF (falling edge) at a reset and in the hardware standby mode. It is not initialized in the software standby mode.

Bit 7 to 0—Reserved: These bits cannot be modified and are always read as 1.

Bit 0—Nonmaskable Interrupt Edge (NMIEG): This bit selects the valid edge of the NMI input signal.

Bit 0

NMIEG	Description
0	A nonmaskable interrupt is generated on the falling edge of the NMI input signal. (Initial value)
1	A nonmaskable interrupt is generated on the rising edge of the NMI input signal.

5.3.3 IRQ Control Register (IRQCR)—H'FFFD

Bit	7	6	5	4	3	2	1	0
	IRQ7E	IRQ6E	IRQ5E	IRQ4E	IRQ3E	IRQ2E	IRQ1E	IRQ0E
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The IRQ control register (IRQCR) enables or disables external interrupts on an individual basis. When an interrupt is enabled, the corresponding pin in port 1 or 4 can be used for interrupt request input. (The pin can also be read by the CPU as a port input pin.) The data direction bit in the port 1 or 4 data direction register must be cleared to 0 to designate the input mode.

The IRQCR is initialized to H'00 at a reset and in the hardware standby mode, disabling all interrupt requests. It is not initialized in the software standby mode.

Bit 7—Interrupt Request 7 Enable (IRQ7E): This bit determines whether a high-to-low transition at pin P47 is recognized as an $\overline{\text{IRQ}}_7$ interrupt request.

Bit 7

IRQ7E	Description
0	P47 is not used for $\overline{\text{IRQ}}_7$ input. (Initial value)
1	P47 can be used for $\overline{\text{IRQ}}_7$ input.*

Bit 6—Interrupt Request 6 Enable (IRQ6E): This bit determines whether a high-to-low transition at pin P46 is recognized as an $\overline{\text{IRQ}}_6$ interrupt request.

Bit 6

IRQ6E	Description
0	P46 is not used for $\overline{\text{IRQ}}_6$ input. (Initial value)
1	P46 can be used for $\overline{\text{IRQ}}_6$ input.*

Bit 5—Interrupt Request 5 Enable (IRQ5E): This bit determines whether a high-to-low transition at pin P45 is recognized as an $\overline{\text{IRQ}}_5$ interrupt request.

Bit 5

IRQ5E	Description
0	P45 is not used for $\overline{\text{IRQ}}_5$ input. (Initial value)
1	P45 can be used for $\overline{\text{IRQ}}_5$ input.*

Note: * In modes 1 and 3 these pins cannot be used for $\overline{\text{IRQ}}_7$ to $\overline{\text{IRQ}}_4$ input because they are occupied by bits 15 to 12 of the address bus.

Bit 4—Interrupt Request 4 Enable (IRQ4E): This bit determines whether a high-to-low transition at pin P44 is recognized as an $\overline{\text{IRQ}}_4$ interrupt request.

Bit 4

IRQ4E	Description
0	P44 is not used for $\overline{\text{IRQ}}_4$ input. (Initial value)
1	P44 can be used for $\overline{\text{IRQ}}_4$ input.*

Note: * In modes 1 and 3 these pins cannot be used for $\overline{\text{IRQ}}_7$ to $\overline{\text{IRQ}}_4$ input because they are occupied by bits 15 to 12 of the address bus.

Bit 3—Interrupt Request 3 Enable (IRQ3E): This bit determines whether a high-to-low transition at pin P14 is recognized as an $\overline{\text{IRQ}}_3$ interrupt request.

Bit 3

IRQ3E	Description
0	P14 is not used for $\overline{\text{IRQ}}_3$ input. (Initial value)
1	P14 can be used for $\overline{\text{IRQ}}_3$ input.*

Bit 2—Interrupt Request 2 Enable (IRQ2E): This bit determines whether a high-to-low transition at pin P13 is recognized as an $\overline{\text{IRQ}}_2$ interrupt request.

Bit 2

IRQ2E	Description
0	P13 is not used for $\overline{\text{IRQ}}_2$ input. (Initial value)
1	P13 can be used for $\overline{\text{IRQ}}_2$ input.*

Bit 1—Interrupt Request 1 Enable (IRQ1E): This bit determines whether a high-to-low transition at pin P12 is recognized as an $\overline{\text{IRQ}}_1$ interrupt request.

Bit 1

IRQ1E	Description
0	P12 is not used for $\overline{\text{IRQ}}_1$ input. (Initial value)
1	P12 can be used for $\overline{\text{IRQ}}_1$ input.*

Note: * In modes 3 these pins cannot be used for $\overline{\text{IRQ}}_3$ to $\overline{\text{IRQ}}_1$ input because they are occupied by the page address bus.

Bit 0—Interrupt Request 0 Enable (IRQ0E): This bit determines whether a low input at pin P11 is recognized as an $\overline{\text{IRQ}}_0$ interrupt request.

Bit 0

IRQ0E	Description	
0	P11 is not used for $\overline{\text{IRQ}}_0$ input.	(Initial value)
1	P11 can be used for $\overline{\text{IRQ}}_0$ input.	

5.4 Interrupt-Handling Sequence

5.4.1 Interrupt-Handling Flow

The interrupt-handling sequence follows the flowchart in figure 5-2, which also covers address-error and trace exceptions. Note that address error, trace exception, and NMI requests bypass the interrupt controller’s priority decision logic and are routed directly to the CPU.

1. Interrupt requests are generated by one or more on-chip supporting modules or external interrupt sources.
2. The interrupt controller checks the interrupt priorities set in the IPRA to IPRD and selects the interrupt with the highest priority. Interrupts with lower priorities remain pending. Among interrupts with the same priority level, the interrupt controller determines priority as explained in table 5-2.
3. The interrupt controller compares the priority level of the selected interrupt request with the mask level in the CPU status register (bits I2 to I0). If the priority level is equal to or less than the mask level, the interrupt request remains pending. If the priority level is higher than the mask level, the interrupt controller accepts the interrupt request and proceeds to the next step.
4. The interrupt controller checks the corresponding bit (if any) in the data transfer enable registers (DTEA to DTEB). If this bit is set to 1, the data transfer controller is started. Otherwise, the CPU interrupt exception-handling sequence is started. When the data transfer controller is started, the interrupt request is cleared (except for interrupt requests from the serial communication interface, which are cleared by writing to the TDR or reading the RDR).

If the data transfer enable bit is cleared to 0 (or is nonexistent), the sequence proceeds as follows. For the case in which the data transfer controller is started, see section 6, “Data Transfer Controller”.

5. After the CPU has finished executing the current instruction, the program counter and status register (in minimum mode) or program counter, code page register, and status register (in maximum mode) are saved to the stack, leaving the stack in the condition shown in figure 5-3 (a) or (b). The program counter value saved on the stack is the address of the next instruction to be executed.
6. The T (Trace) bit of the status register is cleared to 0, and the priority level of the interrupt is copied to bits I₂ to I₀, thus masking further interrupts unless they have a higher priority level. When an NMI is accepted, the interrupt mask level in bits I₂ to I₀ is set to 7.
7. The interrupt controller generates the vector address of the interrupt, and the entry at this address in the exception vector table is read to obtain the starting address of the user-coded interrupt handling routine.

In step 7, the same difference between the minimum and maximum modes exists as in the reset handling sequence. In the minimum mode, one word is copied from the vector table to the program counter, then the interrupt-handling routine starts executing from the address indicated in the program counter. In the maximum mode, two words are read. The lower byte of the first word is copied to the code page register. The second word is copied to the program counter. The interrupt-handling routine starts executing from the address indicated in the code page register and program counter.

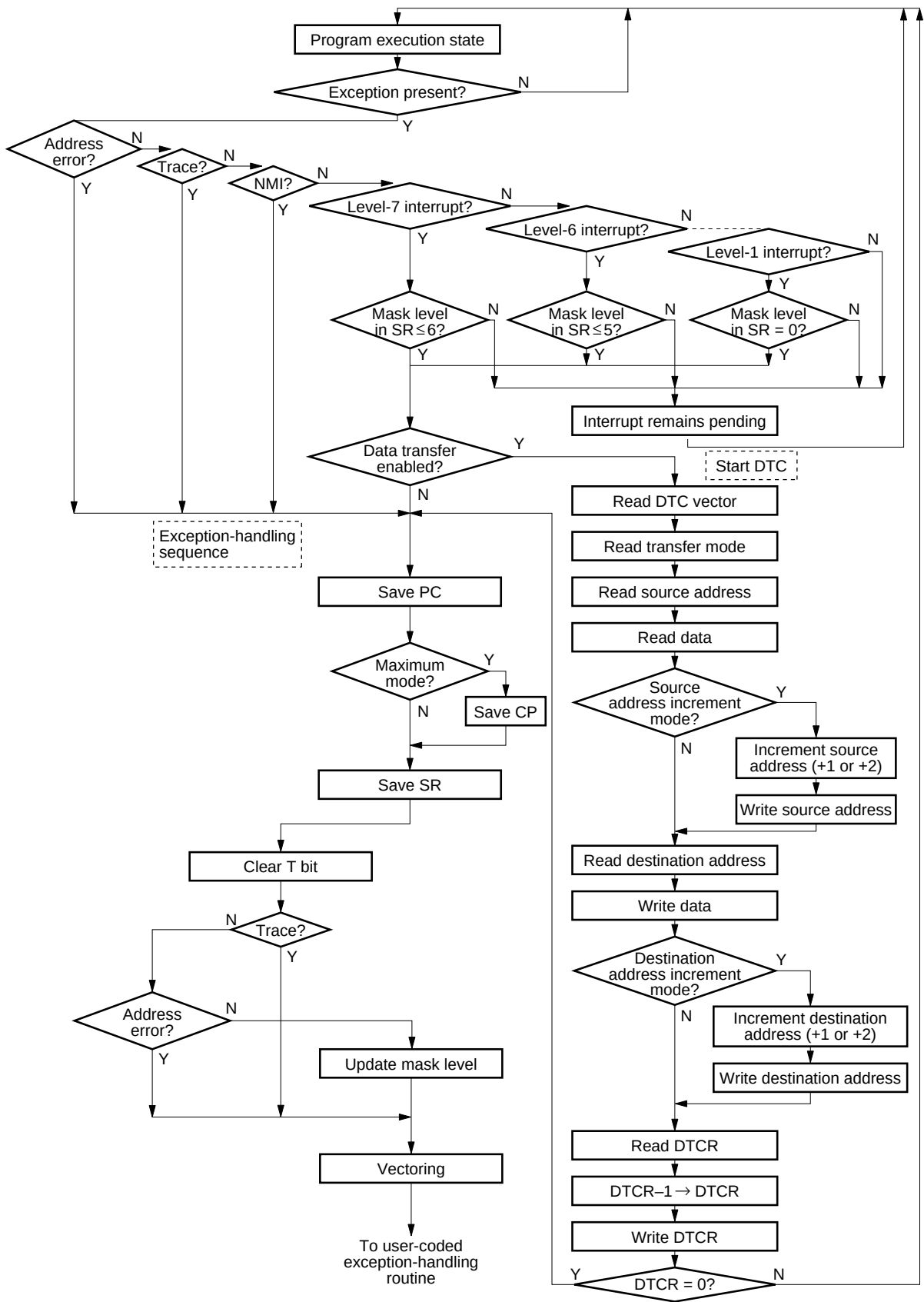


Figure 5-2 Interrupt Handling Flowchart

5.4.2 Stack Status after Interrupt Exception-Handling Sequence

Figure 5-3 (a) and (b) show the stack before and after the interrupt exception-handling sequence.

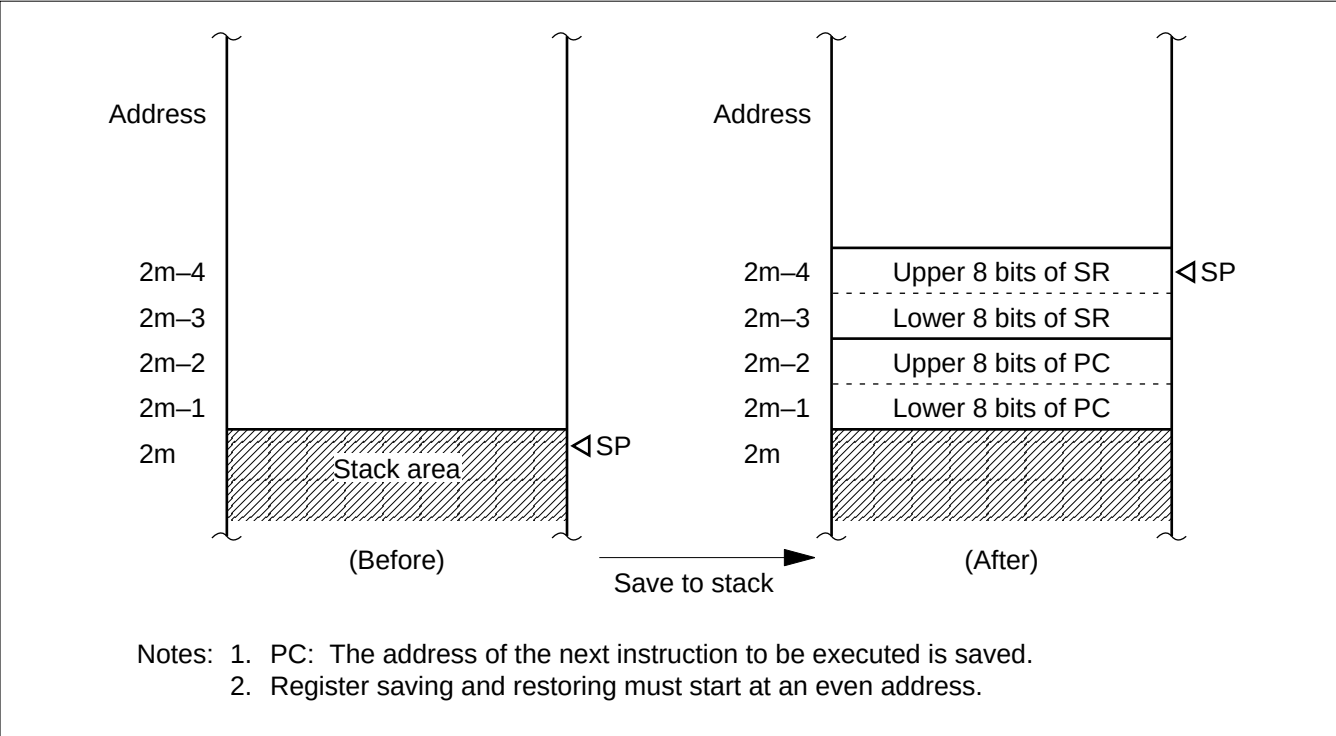


Figure 5-3 (a) Stack before and after Interrupt Exception-Handling (Minimum Mode)

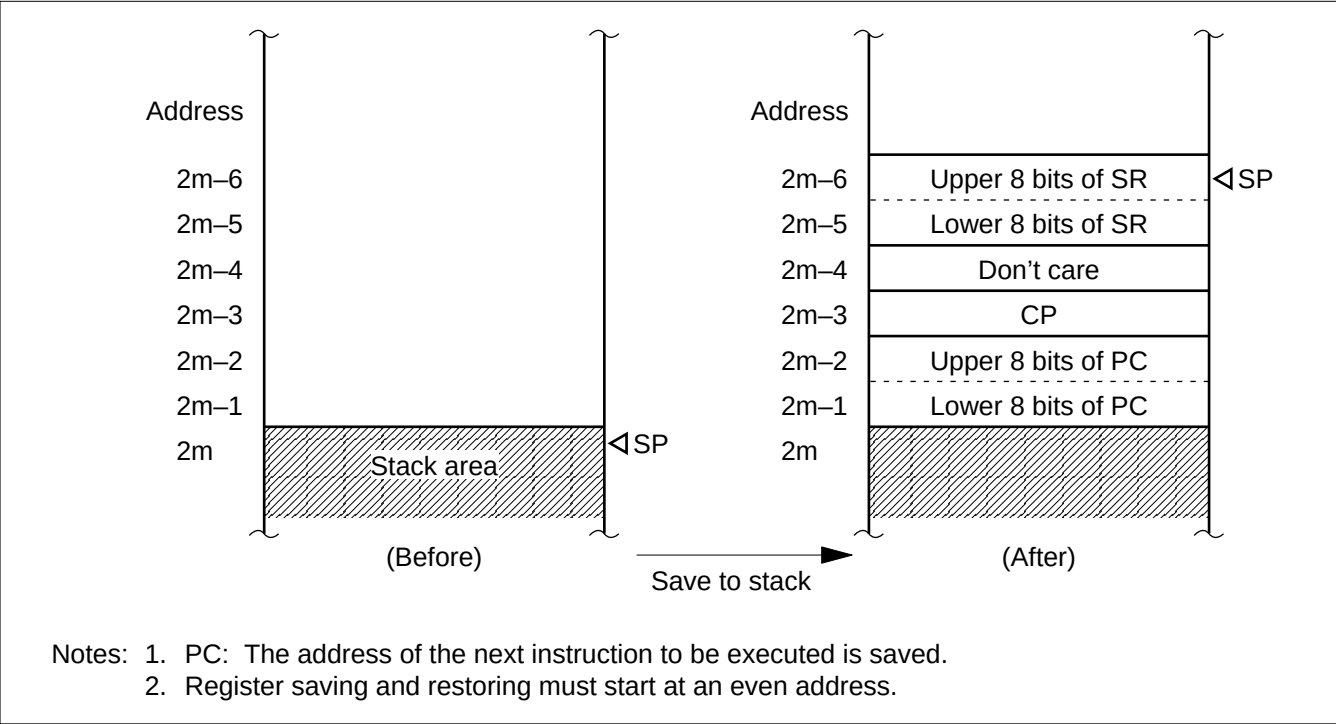


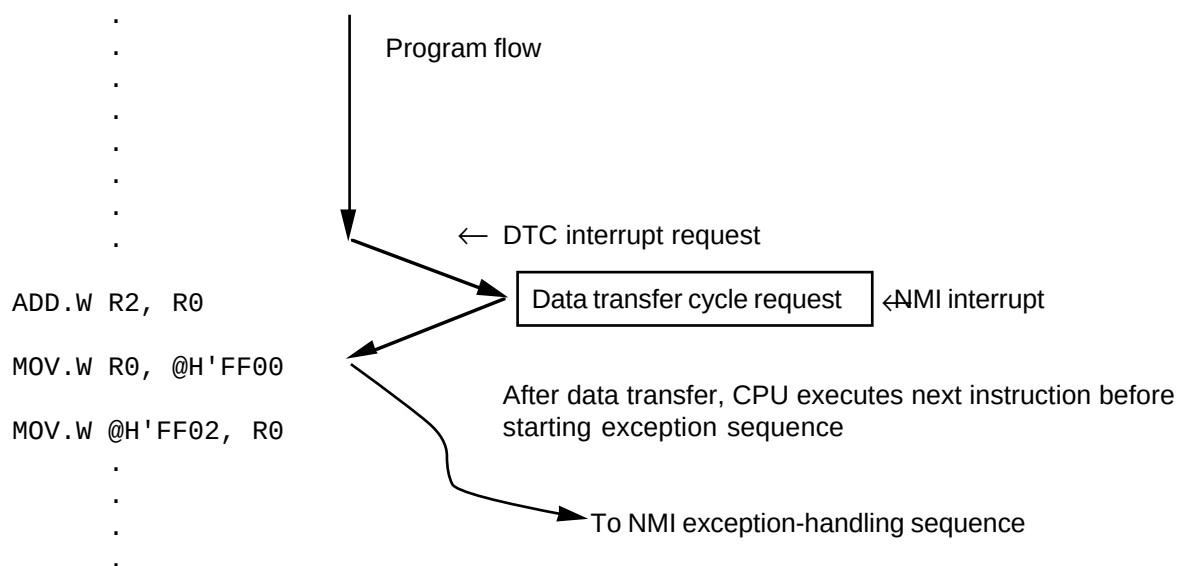
Figure 5-3 (b) Stack before and after Interrupt Exception-Handling (Maximum Mode)

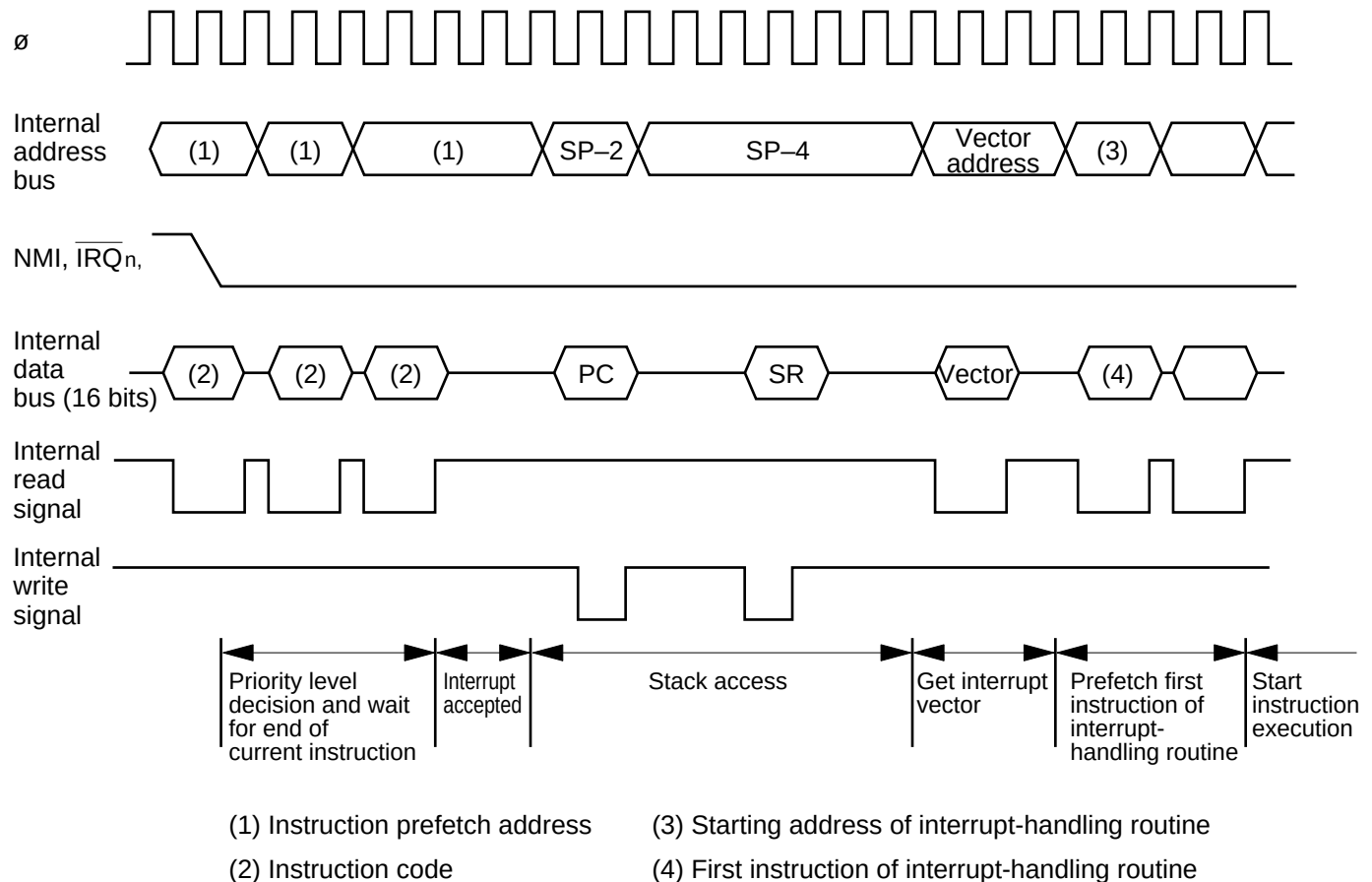
5.4.3 Timing of Interrupt Exception-Handling Sequence

Figure 5-4 shows the timing in minimum mode when the program area and stack are both in on-chip memory and the user-coded interrupt-handling routine starts at an even address. Figure 5-5 shows the timing in maximum mode when the program area and stack are both in external memory.

5.5 Interrupts During Operation of the Data Transfer Controller

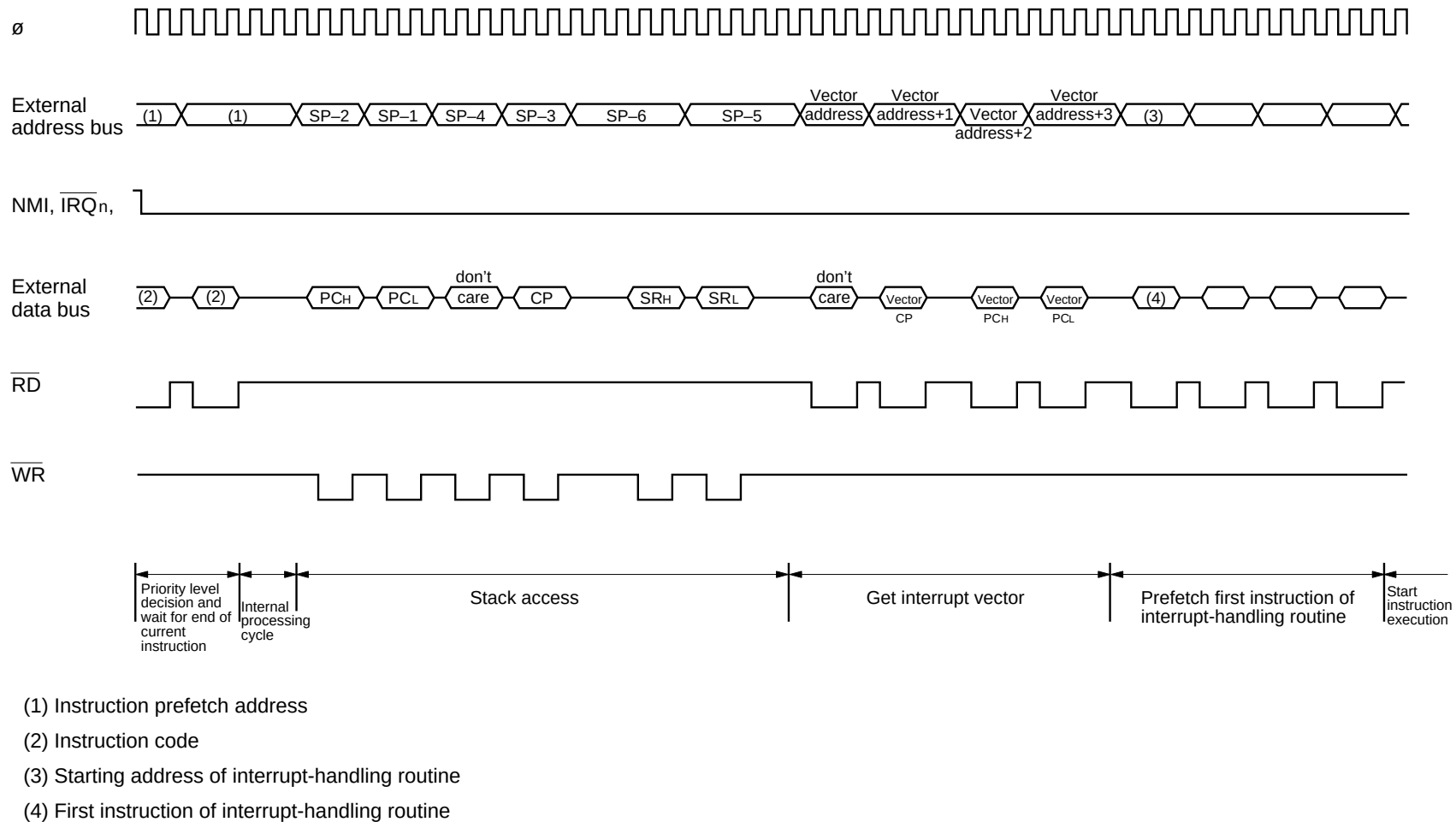
If an interrupt is requested during a DTC data transfer cycle, the interrupt is not accepted until the data transfer cycle has been completed and the next instruction has been executed. This is true even if the interrupt is an NMI. An example is shown below.





Note: This timing chart applies to the minimum mode when the program and stack areas are both in on-chip memory and the interrupt-handling routine starts at an even address.

Figure 5-4 Interrupt Sequence (Minimum Mode, On-Chip Memory)



Note: This timing chart applies to the maximum mode when the program and stack areas are both in external memory. Instruction execution is preceded by an interrupt vector fetch and 4-byte (4 bus cycles) instruction prefetch.

Figure 5-5 Interrupt Sequence (Maximum Mode, External Memory)

5.6 Interrupt Response Time

Table 5-4 indicates the number of states that may elapse between the generation of an interrupt request and the execution of the first instruction of the interrupt-handling routine, assuming that the interrupt is not masked and not preempted by a higher-priority interrupt. Since word access is performed to on-chip memory areas, fastest interrupt service can be obtained by placing the program in on-chip ROM and the stack in on-chip RAM.

Table 5-4 Number of States before Interrupt Service

No.	Reason for Wait		Number of States	
			Minimum Mode	Maximum Mode
1	Interrupt priority decision and comparison with mask level in CPU status register		2 states	
2	Maximum number of states to completion of current instruction	Instruction is in on-chip memory	x (x = 38 for LDM instruction specifying all registers)	
		Instruction is in external memory	y (y = 74 + 16m for LDM instruction specifying all registers)	
3	Number of states from saving of PC and SR (or PC, SR, and CP) until first instruction of interrupt-handling routine is prefetched.	Stack is in on-chip RAM	16	21
		Stack is in external memory	28 + 6m	41 + 10m
Total	Stack is in on-chip RAM	Instruction is in on-chip memory	18 + x (56)	23 + x (61)
		Instruction is in external memory	18 + y (92 + 16m)	23 + y (97 + 16m)
	Stack is in external RAM	Instruction is in on-chip memory	30 + 6m + x (68 + 6m)	43 + 10m + x (81 + 10m)
		Instruction is in external memory	30 + 6m + y (104 + 22m)	43 + 10m + y (117 + 26m)

Notes: m: Number of wait states inserted in external memory access.
Values in parentheses are for the LDM instruction specifying all registers.

Section 6 Data Transfer Controller

6.1 Overview

The H8/520 chip includes a data transfer controller (DTC) that can be started by designated interrupts to transfer data from a source address to a destination address located in page 0. These addresses include in particular the registers of the on-chip supporting modules and I/O ports. Typical uses of the DTC are to change the setting of a control register of an on-chip supporting module in response to an interrupt from that module, or to transfer data from memory to an I/O port or the serial communication interface. Once set up, the transfer is interrupt-driven, so it proceeds independently of program execution, although program execution temporarily stops while each byte or word is being transferred.

The data transfer functions of the DTC could also be performed by the CPU, but the DTC offers three advantages:

- It is faster.
- It requires less program coding.
- It has its own registers and does not require CPU registers to be used as pointers, etc.

6.1.1 Features

The main features of the DTC are listed below:

- The source address and destination address can be set anywhere in the 64-kbyte address space of page 0.
- The DTC can be programmed to transfer one byte or one word of data per interrupt.
- The DTC can be programmed to increment the source address and/or destination address after each byte or word is transferred.
- After transferring a designated number of bytes or words, the DTC generates a CPU interrupt with the vector of the interrupt source that started the DTC.
- This designated data transfer count can be set from 1 to 65,536 bytes or words.

6.1.2 Block Diagram

Figure 6-1 shows a block diagram of the DTC.

The four DTC control registers (DTMR, DTSR, DTDR, and DTCR) are invisible to the CPU, but corresponding information is kept in a register information table in memory. A separate table is maintained for each DTC interrupt type. When an interrupt requests DTC service, the DTC loads its

control registers from the table in memory, transfers the byte or word of data, and writes any altered register information back to memory.

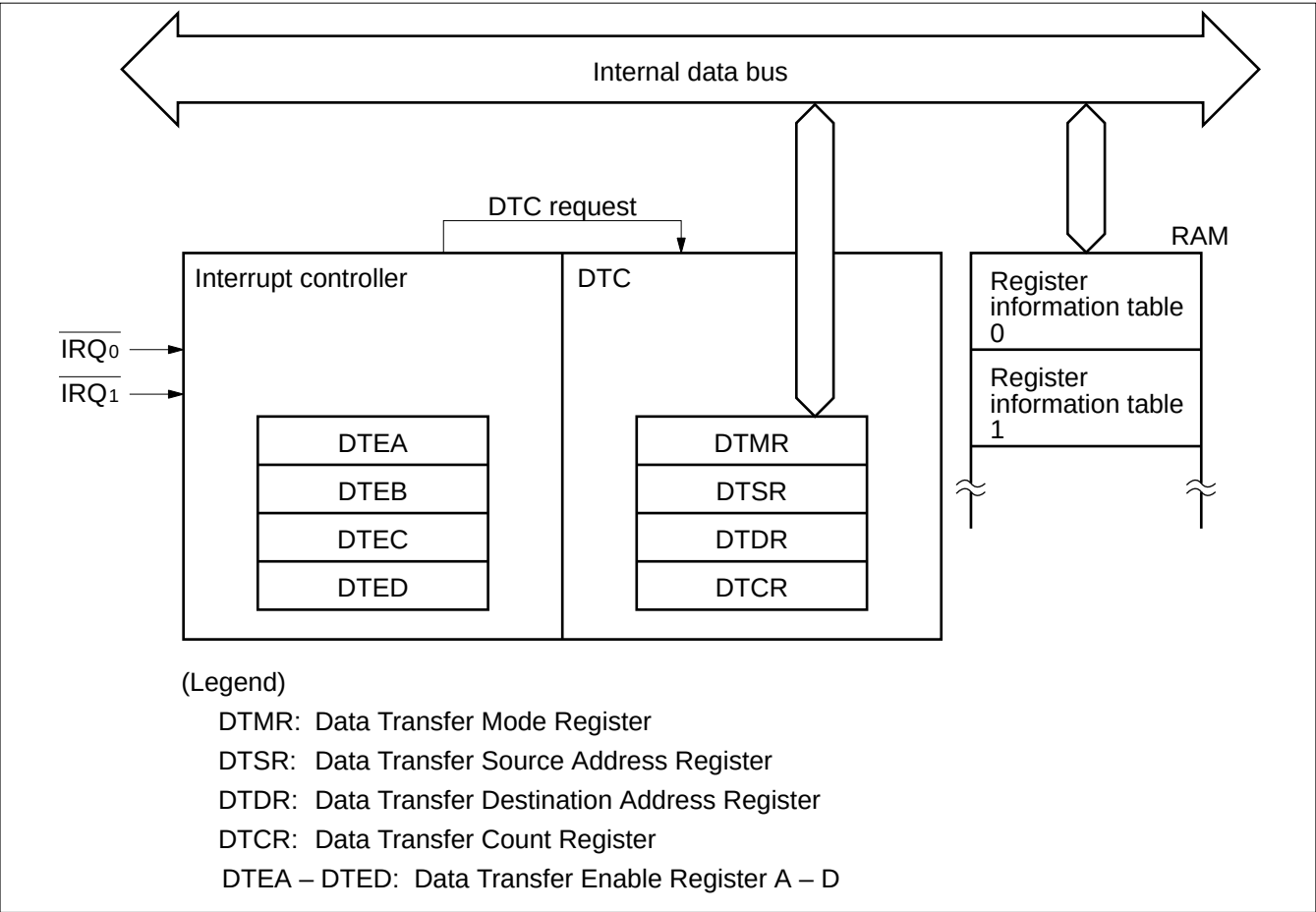


Figure 6-1 Block Diagram of Data Transfer Controller

6.1.3 Register Configuration

The four DTC control registers are listed in table 6-1. These registers are not located in the address space and cannot be written or read by the CPU. To set information in these registers, a program must write the information in a table in memory from which it will be loaded by the DTC.

Table 6-1 Internal Control Registers of the DTC

Name	Abbreviation	Read/Write
Data transfer mode register	DTMR	Disabled
Data transfer source address register	DTSR	Disabled
Data transfer destination address register	DTDR	Disabled
Data transfer count register	DTCR	Disabled

Starting of the DTC is controlled by the four data transfer enable registers, which are located in high addresses in page 0. Table 6-2 lists these registers.

Table 6-2 Data Transfer Enable Registers

Name		Abbreviation	Read/Write	Initial Value	Address
Data transfer enable register	A	DTEA	R/W	H'00	H'FFF4
	B	DTEB	R/W	H'00	H'FFF5
	C	DTEC	R/W	H'00	H'FFF6
	D	DTED	R/W	H'00	H'FFF7

6.2 Register Descriptions

6.2.1 Data Transfer Mode Register (DTMR)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Sz	SI	DI	—	—	—	—	—	—	—	—	—	—	—	—	—
Read/Write	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

The data transfer mode register is a 16-bit register, the first three bits of which designate the data size and specify whether to increment the source and destination addresses.

Bit 15—Sz (Size): This bit designates the size of the data transferred.

Bit 15

Sz	Description
0	Byte transfer
1	Word transfer* (two bytes at a time)

Note: * For word transfer, the source and designation addresses must be even addresses.

Bit 14—SI (Source Increment): This bit specifies whether to increment the source address.

Bit 14

SI	Description
0	Source address is not incremented.
1	1. If Sz = 0: Source address is incremented by +1 after each data transfer. 2. If Sz = 1: Source address is incremented by +2 after each data transfer.

Bit 13—DI (Destination Increment): This bit specifies whether to increment the destination address.

Bit 13

DI	Description
0	Destination address is not incremented.
1	1. If Sz = 0: Destination address is incremented by +1 after each data transfer. 2. If Sz = 1: Destination address is incremented by +2 after each data transfer.

Bits 12 to 0—Reserved Bits: These bits are reserved.

6.2.2 Data Transfer Source Address Register (DTSR)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read/Write	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

The data transfer source register is a 16-bit register that designates the data transfer source address. For word transfer this must be an even address. In the maximum mode, this address is implicitly located in page 0.

6.2.3 Data Transfer Destination Register (DTDR)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read/Write	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

The data transfer destination register is a 16-bit register that designates the data transfer destination address. For word transfer this must be an even address. In the maximum mode, this address is implicitly located in page 0.

6.2.4 Data Transfer Count Register (DTCR)

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Read/Write	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

The data transfer count register is a 16-bit register that counts the number of bytes or words of data remaining to be transferred. The initial count can be set from 1 to 65,536. A register value of 0 designates an initial count of 65,536.

The data transfer count register is decremented automatically after each byte or word is transferred. When its value reaches 0, indicating that the designated number of bytes or words have been transferred, a CPU interrupt is generated with the vector of the interrupt that requested the data transfer.

6.2.5 Data Transfer Enable Registers A to D (DTEA to DTED)

These four registers designate whether an interrupt starts the DTC. The bits in these registers are assigned to interrupts as indicated in table 6-3. No bits are assigned to the NMI, IRQ4, IRQ5, IRQ6, IRQ7, FOVI, OVI, and ERI interrupts, which cannot request data transfers.

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Table 6-3 Assignment of Data Transfer Enable Registers

Interrupt Source						Interrupt Source					
Register	Module	Bits 7 to 4				Module	Bits 3 to 0				
		7	6	5	4		3	2	1	0	
DTEA	$\overline{\text{IRQ}}_0$	—	—	—	IRQ₀	$\overline{\text{IRQ}}_3$ - $\overline{\text{IRQ}}_1$	—	IRQ₃	IRQ₂	IRQ₁	
DTEB	FRT1	—	OCIB	OCIA	ICI	FRT2	—	OCIB	OCIA	ICI	
DTEC	8-Bit timer	—	—	CMIB	CMIA	SCI1	—	TXI	RXI	—	
DTED	SCI2	—	TXI	RXI	—	A/D converter	—	—	—	ADI	

Note: Bits marked “—” should always be cleared to 0.

If the bit for a certain interrupt is set to 1, that interrupt is regarded as a request for DTC service. If the bit is cleared to 0, the interrupt is regarded as a CPU interrupt request.

Only the 16 interrupts indicated in table 6-3 can request DTC service. DTE bits not assigned to any interrupt (indicated by “—” in table 6-3) should be left cleared to 0.

Note on Timing of DTE Modifications: The interrupt controller requires two system clock (ϕ) periods to determine the priority level of an interrupt. Accordingly, when an instruction modifies a data transfer enable register, the new setting does not take effect until after the next instruction has been executed.

6.3 Data Transfer Operation

6.3.1 Data Transfer Cycle

When started by an interrupt, the DTC executes the following data transfer cycle:

1. From the DTC vector table, the DTC reads the address at which the register information table for that interrupt is located in memory.
2. The DTC loads the data transfer mode register and source address register from this table and reads the data (one byte or word) from the source address.
3. If so specified in the mode register, the DTC increments the source address register and writes the new source address back to the table in memory.
4. The DTC loads the data transfer destination address register and writes the byte or word of data to the destination address.
5. If so specified in the mode register, the DTC increments the destination address register and writes the new destination address back to the table in memory.
6. The DTC loads the data transfer count register from the table in memory, decrements the data count, and writes the new count back to memory.
7. If the data transfer count is now 0, the DTC generates a CPU interrupt. The interrupt vector is the vector of the interrupt type that started the DTC.

At an appropriate point during this procedure the DTC also clears the interrupt request by clearing the corresponding flag bit in the status register of the on-chip supporting module to 0. (For IRQ₁ to IRQ₃, the DTC clears an internal latch.)

But the DTC does not clear the data transfer enable bit in the data transfer enable register. This action, if necessary, must be taken by the user-coded interrupt-handling routine invoked at the end of the transfer.

The data transfer cycle is shown in a flowchart in figure 6-2.

For the steps from the occurrence of the interrupt up to the start of the data transfer cycle, see section 5.4.1, “Interrupt-Handling Flow”.

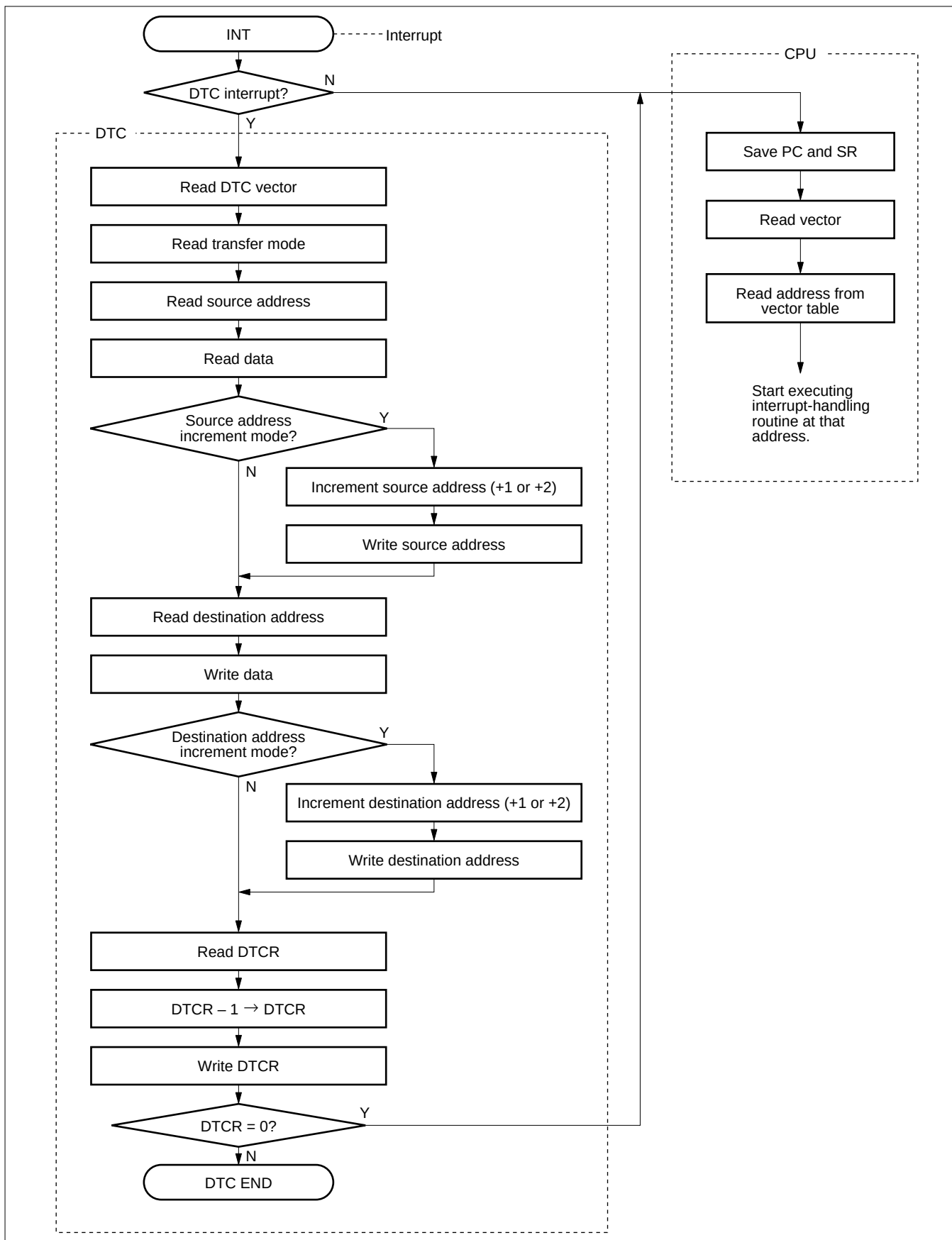
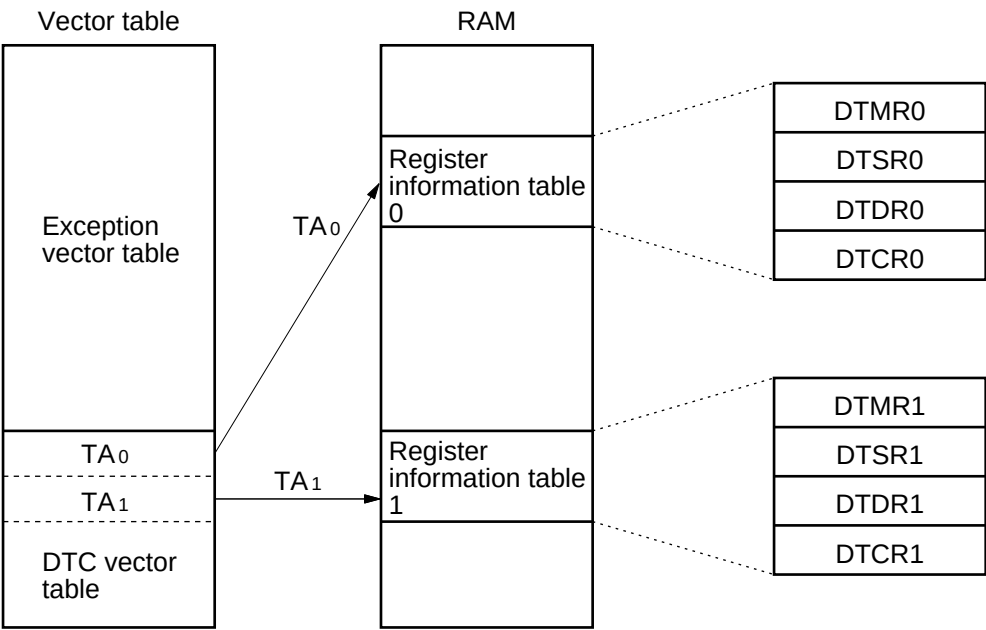


Figure 6-2 Flowchart of Data Transfer Cycle

6.3.2 DTC Vector Table

The DTC vector table is located immediately following the exception vector table at the beginning of page 0 in memory. For each interrupt that can request DTC service, the DTC vector table provides a pointer to an address in memory where the table of DTC control register information for that interrupt is stored. The register information tables can be placed in any available locations in page 0.



- Notes:
- 1. TA₀, TA₁, ...: Addresses of DTC register information tables in memory.
 - 2. The DTC register information tables should normally be located in RAM, but they may be located in ROM if it is not necessary to update the register information. Specifically, DTC register information can be located in ROM if neither the source nor the destination address is incremented and the desired number of data transfers is one (DTCR = 0) or infinite (DTCR > 0).

Figure 6-3 DTC Vector Table

In minimum mode, each entry in the DTC vector table consists of two bytes, pointing to an address in page 0. In maximum mode, for hardware reasons, each DTC vector table entry consists of four bytes but the first two bytes are ignored; the last two bytes point to an address which is implicitly assumed to be in page 0, regardless of the current page specifications.

Figure 6-4 shows one DTC vector table entry in minimum and maximum mode.

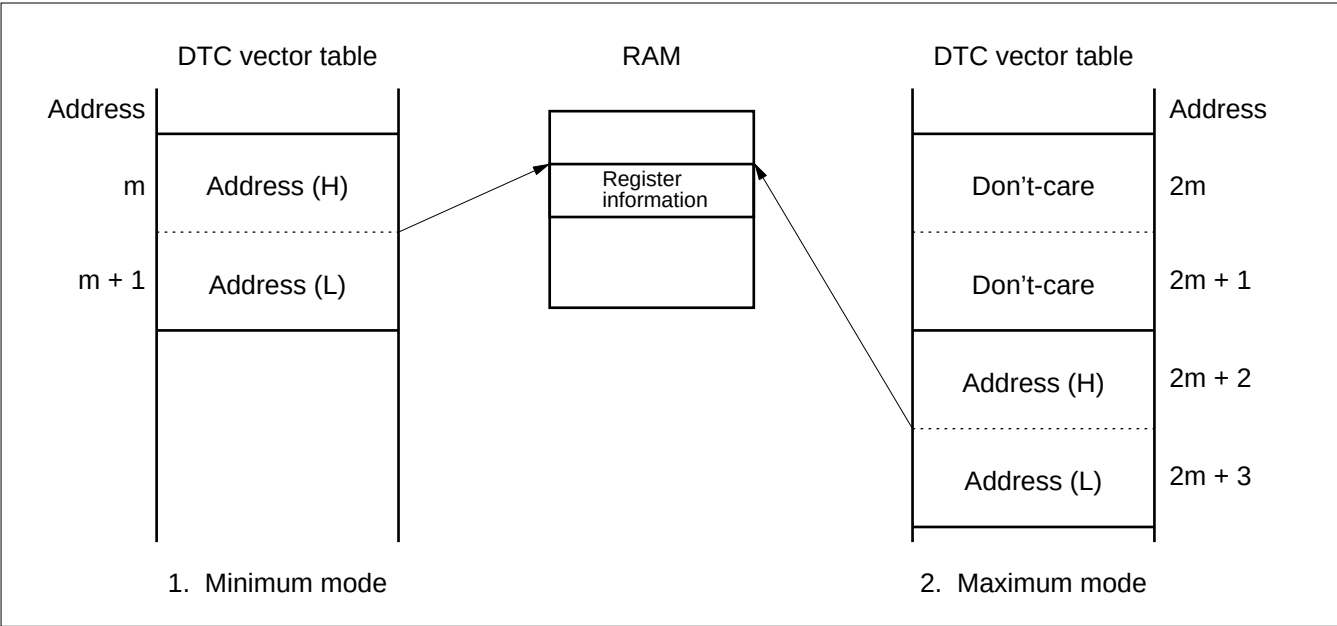


Figure 6-4 DTC Vector Table Entry

Table 6-4 lists the addresses of the entries in the DTC vector table for each interrupt.

Table 6-4 Addresses of DTC Vectors

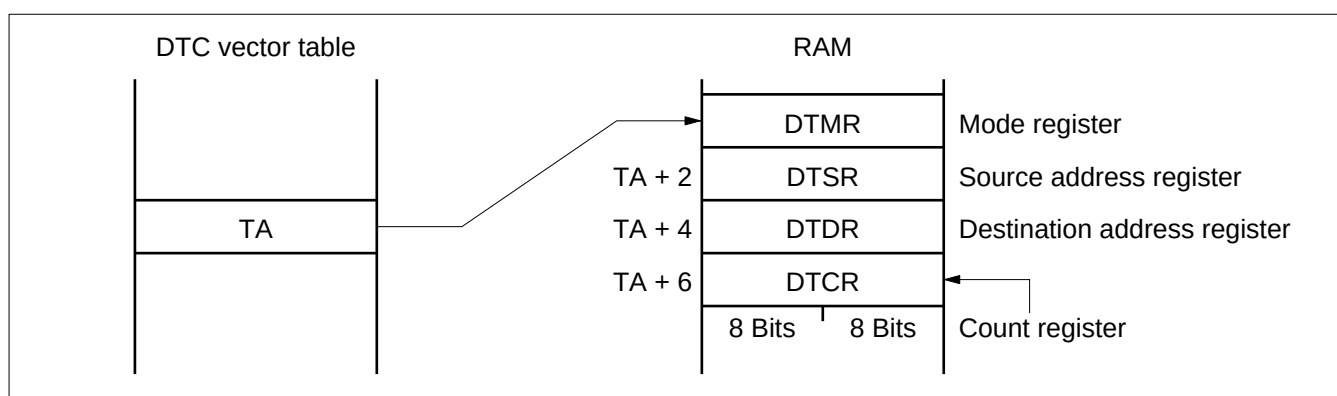
		Address of DTC Vector	
Interrupt		Minimum Mode	Maximum Mode
IRQ	IRQ ₀	H'0080 to H'0081	H'0100 to H'0103
	IRQ ₁	H'0082 to H'0083	H'0104 to H'0107
	IRQ ₂	H'0084 to H'0085	H'0108 to H'010B
	IRQ ₃	H'0086 to H'0087	H'010C to H'010F
FRT1	ICI	H'0090 to H'0091	H'0120 to H'0123
	OCIA	H'0092 to H'0093	H'0124 to H'0127
	OCIB	H'0094 to H'0095	H'0128 to H'012B
FRT2	ICI	H'0098 to H'0099	H'0130 to H'0133
	OCIA	H'009A to H'009B	H'0134 to H'0137
	OCIB	H'009C to H'009D	H'0138 to H'013B

Table 6-4 Addresses of DTC Vectors (cont)

		Address of DTC Vector	
		Minimum Mode	Maximum Mode
8-Bit timer	CMIA	H'00A0 to H'00A1	H'0140 to H'0143
	CMIB	H'00A2 to H'00A3	H'0144 to H'0147
Serial communication interface 1	RXI	H'00AA to H'00AB	H'0154 to H'0157
	TXI	H'00AC to H'00AD	H'0158 to H'015B
Serial communication interface 2	RXI	H'00B2 to H'00B3	H'0164 to H'0167
	TXI	H'00B4 to H'00B5	H'0168 to H'016B
A/D converter	ADI	H'00B8 to H'00B9	H'0170 to H'0173

6.3.3 Location of Register Information in Memory

For each interrupt, the DTC control register information is stored in four consecutive words in memory in the order shown in figure 6-5.

**Figure 6-5 Order of Register Information**

6.3.4 Length of Data Transfer Cycle

1. Register Information in On-Chip RAM

Table 6-5 lists the number of states required per data transfer, assuming that the DTC control register information is stored in on-chip RAM. This is the number of states required for loading and saving the DTC control registers and transferring one byte or word of data. Two cases are considered: a transfer between on-chip RAM and a register belonging to an I/O port or on-chip supporting module (i.e., a register in the register field from addresses H'FF80 to H'FFFF); and a transfer between such a register and external RAM.

Table 6-5 Number of States per Data Transfer

Increment Mode		On-Chip RAM ↔ Module or I/O Register		External RAM ↔ Module or I/O Register	
Source (SI)	Destination (DI)	Byte Transfer	Word Transfer	Byte Transfer	Word Transfer
0	0	31	34	32	38
0	1	33	36	34	40
1	0	33	36	34	40
1	1	35	38	36	42

Note: Numbers in the table are the number of states.

The values in table 6-5 are calculated from the formula:

$$N = 26 + 2 \times SI + 2 \times DI + M_S + M_D$$

Where M_S and M_D have the following meanings:

M_S : Number of states for reading source data

M_D : Number of states for writing destination data

The values of M_S and M_D depend on the data location as follows:

- Byte or word data in on-chip RAM: $\Rightarrow$ 2 states
- Byte data in external RAM or register field: $\Rightarrow$ 3 states
- Word data in external RAM or register field: $\Rightarrow$ 6 states

2. Register Information in External RAM

If the DTC control register information is stored in external RAM, $20 + 4 \times SI + 4 \times DI$ must be added to the values in table 6-5.

3. Interrupt Controller Wait

The values given above do not include the time between the occurrence of the interrupt request and the starting of the DTC. This time includes two states for the interrupt controller to check priority and a variable wait until the end of the current CPU instruction. At maximum, this time equals the sum of the values indicated for items No. 1 and 2 in table 6-6.

If the data transfer count is 0 at the end of a data transfer cycle, the number of states from the end of the data transfer cycle until the first instruction of the user-coded interrupt-handling routine is executed is the value given for item No. 3 in table 6-6.

Table 6-6 Number of States before Interrupt Service

No.	Reason for Wait		Number of States	
			Minimum Mode	Maximum Mode
1	Interrupt priority decision and comparison with mask level in CPU status register		2 states	
2	Maximum number of states to completion of current instruction	Instruction is in on-chip memory	x	(x = 38 for LDM instruction specifying all registers)
		Instruction is in external memory	y	(y = 74 + 16m for LDM instruction specifying all registers)
3	Number of states from saving of PC and SR (or PC, SR, and CP) until first instruction of interrupt-handling routine is prefetched.	Stack is in on-chip RAM	16	21
		Stack is in external memory	28 + 6m	41 + 10m

Note: m: Number of wait states inserted in external memory access.

6.4 Procedure for Using the DTC

A program that uses the DTC to transfer data must do the following:

1. Set the appropriate DTMR, DTSR, DTDR, and DTCR register information in the memory location indicated in the DTC vector table.
2. Set the data transfer enable bit of the pertinent interrupt to 1, and set the priority of the interrupt source (in the interrupt priority register) and the interrupt mask level (in the CPU status register) so that the interrupt can be accepted.
3. Set the interrupt enable bit in the control register for the interrupt source. (For IRQ₀ to IRQ₃, the control register is the IRQ control register.)

Following these preparations, the DTC will be started each time the interrupt occurs. When the number of bytes or words designated by the DTCR value have been transferred, after transferring the last byte or word, the DTC generates a CPU interrupt.

The user-coded interrupt-handling routine must take action to prepare for or disable further DTC data transfer: by readjusting the data transfer count, for example, or clearing the data transfer enable bit. If no action is taken, the next interrupt of the same type will start the DTC with an initial data transfer count of 65,536.

6.5 Example

Purpose: To receive 128 bytes of serial data via serial communication interface 1.

Conditions:

- Operating mode: Minimum mode
- Received data are to be stored in consecutive addresses starting at H'FE00.
- DTC control register information for the RXI interrupt is stored at addresses H'FD80 to H'FD87.
- Accordingly, the DTC vector table contains H'FD at address H'00AA and H'80 at address H'00AB.
- The desired interrupt mask level in the CPU status register is 4, and the desired SCI1 interrupt priority level is 5.

Procedure

1. The user program sets DTC control register information in addresses H'FD80 to H'FD87 as shown in table 6-7.

Table 6-7 DTC Control Register Information Set in RAM

Register	Description	Value Set
DTMR	Byte transfer	
	Source address fixed	H'2000
	Increment destination address	
DTSR	Address of SCI receive data register	H'FEDD
DTDR	Address H'FE00	H'FE00
DTCR	Number of bytes to be received: 128	H'0080

2. The program sets the RXI (SCI Receive Interrupt) bit in the data transfer enable register (bit 1 of register DTEC) to 1.
3. The program sets the interrupt mask in the CPU status register to 4, and the SCI1 interrupt priority in bits 2 to 0 of interrupt priority register IPRC to 5.

4. The program sets SCI1 to the appropriate receive mode, and sets the receive interrupt enable (RIE) bit in the serial control register (SCR) to 1 to enable receive interrupts.
5. Thereafter, each time the SCI1 receives one byte of data, it requests an RXI interrupt, which the interrupt controller directs toward the DTC. The DTC transfers the byte from SCI1's receive data register (RDR) into RAM, and clears the interrupt request before ending.
6. When 128 bytes have been transferred (DTCR = 0), the DTC generates a CPU interrupt. The interrupt source is SCI1. The interrupt type is RXI.
7. The user-coded RXI interrupt-handling routine processes the received data and disables further data transfer (by clearing the RIE bit, for example).

The diagram illustrates the DTC vector table and its mapping to RAM. On the left, the **DTC vector table** is shown with two entries:

Address	Value
H'00AA	H'FD
H'00AB	H'80

On the right, the **RAM** structure is shown, organized into several fields:

- Mode:** H'20 (at H'FD80), H'00 (at H'FD81)
- Source address:** H'FF (at H'FD82), H'DD (at H'FD83)
- Destination address:** H'FC (at H'FD84), H'00 (at H'FD85)
- Counter:** H'00 (at H'FD86), H'80 (at H'FD87)
- Receive data:** A series of 128 data points, from "Receive data 1" (at H'FE00) to "Receive data 128" (at H'FE7F).

A bracket on the right side of the RAM structure groups the fields from Mode to Counter. Below the RAM structure, a box labeled **SCI1** contains a sub-box labeled **RDR** (Receive Data Register). An arrow labeled "Transferred by DTC" points from the RDR to the "Receive data 1" field in the RAM.

Section 7 Wait-State Controller

7.1 Overview

To simplify interfacing to low-speed external devices, the H8/520 has an on-chip wait-state controller (WSC) that can insert wait states (T_w) to prolong bus cycles.

The wait-state function can be used in CPU and DTC access cycles to external addresses. It is not used in access to on-chip memory or registers. The T_w states are inserted between the T_2 state and T_3 state in the bus cycle. The number of wait states can be selected by a value set in the wait-state control register (WCR), or by holding the $\overline{\text{WAIT}}$ pin low for the required interval.

7.1.1 Features

The main features of the wait-state controller are as follows:

- Selection of three operating modes
Programmable wait mode, pin wait mode, or pin auto-wait mode
- 0, 1, 2, or 3 wait states can be inserted.
And in the pin wait mode, 4 or more states can be inserted by holding the $\overline{\text{WAIT}}$ pin low.

7.1.2 Block Diagram

Figure 7-1 shows a block diagram of the wait-state controller.

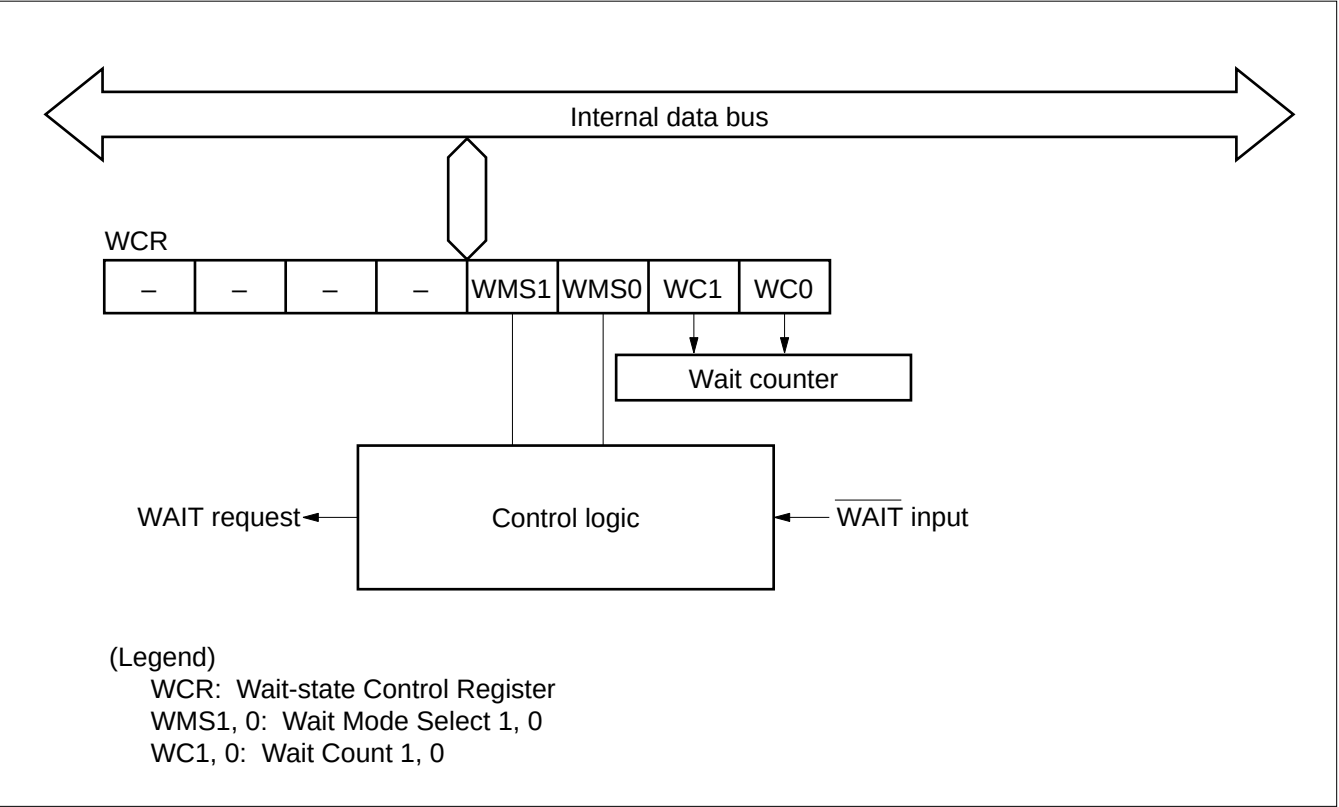


Figure 7-1 Block Diagram of Wait-State Controller

7.1.3 Register Configuration

The wait-state controller has one control register: the wait-state control register described in table 7-1.

Table 7-1 Register Configuration

Name	Abbreviation	Read/Write	Initial Value	Address
Wait-state control register	WCR	R/W	H'F3	H'FFF8

7.2 Wait-State Control Register

The wait-state control register (WCR) is an 8-bit register that specifies the wait mode and the number of wait states to be inserted. A reset initializes the WCR to specify the programmable wait mode with three wait states. The WCR is not initialized in the software standby mode.

Bit	7	6	5	4	3	2	1	0
	—	—	—	—	WMS1	WMS0	WC1	WC0
Initial value	1	1	1	1	0	0	1	1
Read/Write	—	—	—	—	R/W	R/W	R/W	R/W

Bits 7 to 4—Reserved: These bits cannot be modified and are always read as 1.

Bits 3 and 2—Wait Mode Select 1 and 0 (WMS1 and WMS0): These bits select the wait mode as shown below:

Bit 3 WMS1	Bit 2 WMS0	Description
0	0	Programmable wait mode (Initial value)
0	1	No wait states are inserted, regardless of the wait count.
1	0	Pin wait mode
1	1	Pin auto-wait mode

Bits 1 and 0—Wait Count (WC1 and WC0): These bits specify the number of wait states to be inserted.

Wait states are inserted only in bus cycles in which the CPU or DTC accesses an external address.

Bit 1 WC1	Bit 0 WC0	Description
0	0	No wait states are inserted, except in pin wait mode.
0	1	1 wait state is inserted.
1	0	2 wait states are inserted.
1	1	3 wait states are inserted. (Initial value)

7.3 Operation in Each Wait Mode

Table 7-2 summarizes the operation of the three wait modes.

Table 7-2 Wait Modes

Mode	$\overline{\text{WAIT}}$ Pin Function	Insertion Conditions	Number of Wait States Inserted
Programmable wait mode WMS1 = 0 WMS0 = 0	Disabled	Inserted on access to an off-chip address	0 to 3 wait states are inserted, as specified by bits WC0 and WC1.
Pin wait mode WMS1 = 1 WMS0 = 0	Enabled	Inserted on access to an off-chip address	0 to 3 wait states are inserted, as specified by bits WC0 and WC1, plus additional wait states while the $\overline{\text{WAIT}}$ pin is held low.
Pin auto-wait mode WMS1 = 1 WMS0 = 1	Enabled	Inserted on access to an off-chip address if the $\overline{\text{WAIT}}$ pin is low	0 to 3 wait states are inserted, as specified by bits WC0 and WC1.

7.3.1 Programmable Wait Mode

The programmable wait mode is selected when WMS1 = 0 and WMS0 = 0.

Whenever the CPU or DTC accesses an off-chip address, the number of wait states set in bits WC1 and WC0 are inserted. The $\overline{\text{WAIT}}$ pin is not used for wait control; it is available as an I/O pin (P10).

Figure 7-2 shows the timing of the operation in this mode when the wait count is 1 (WC1 = 0, WC0 = 1).

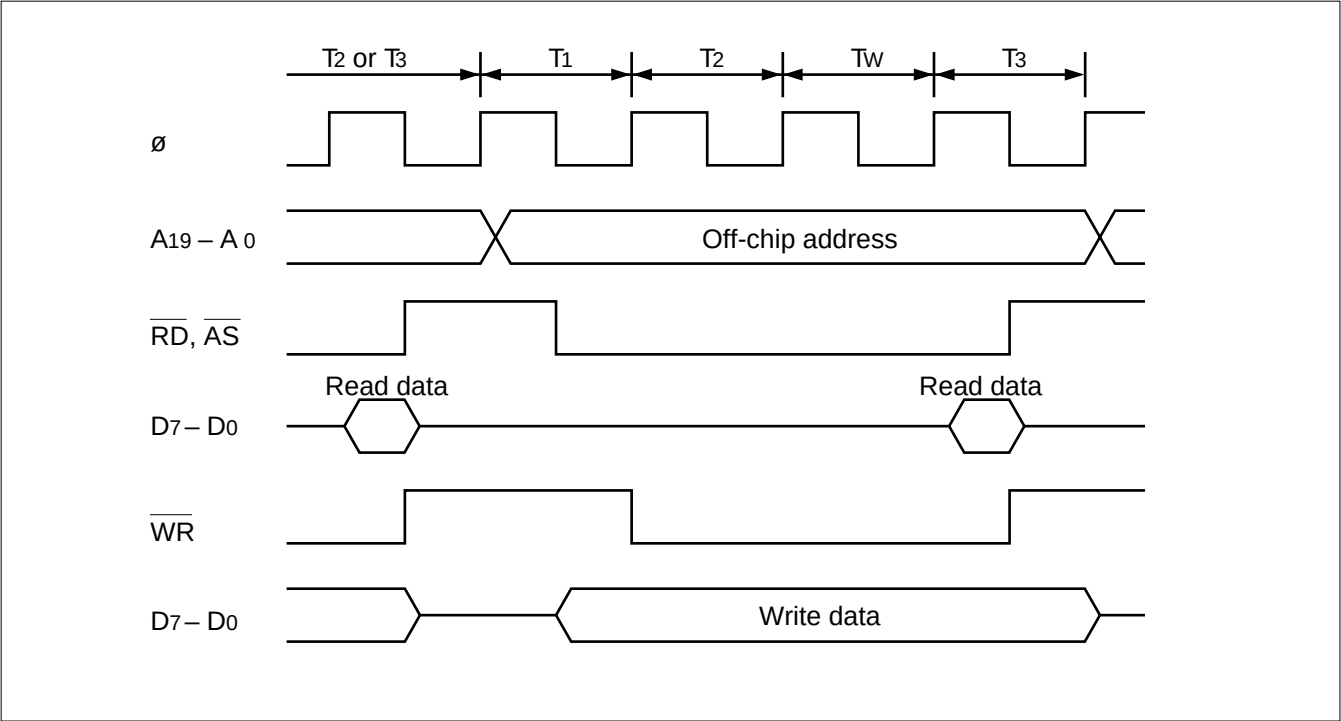


Figure 7-2 Programmable Wait Mode

7.3.2 Pin Wait Mode

The pin wait mode is selected when WMS1 = 1 and WMS0 = 0.

In this mode the $\overline{WAIT}$ function of the P10/ $\overline{WAIT}$ pin is used automatically.

The number of wait states indicated by bits WC1 and WC0 are inserted into any bus cycle in which the CPU or DTC accesses an off-chip address. In addition, wait states continue to be inserted as long as the $\overline{WAIT}$ pin is held low. In particular, if the wait count is 0 but the $\overline{WAIT}$ pin is low at the rising edge of the $\emptyset$ clock in the T_2 state, wait states are inserted until the $\overline{WAIT}$ pin goes high.

This mode is useful for inserting four or more wait states, or when different external devices require different numbers of wait states.

Figure 7-3 shows the timing of the operation in this mode when the wait count is 1 ($WC1 = 0$, $WC0 = 1$) and the $\overline{WAIT}$ pin is held low to insert one additional wait state.

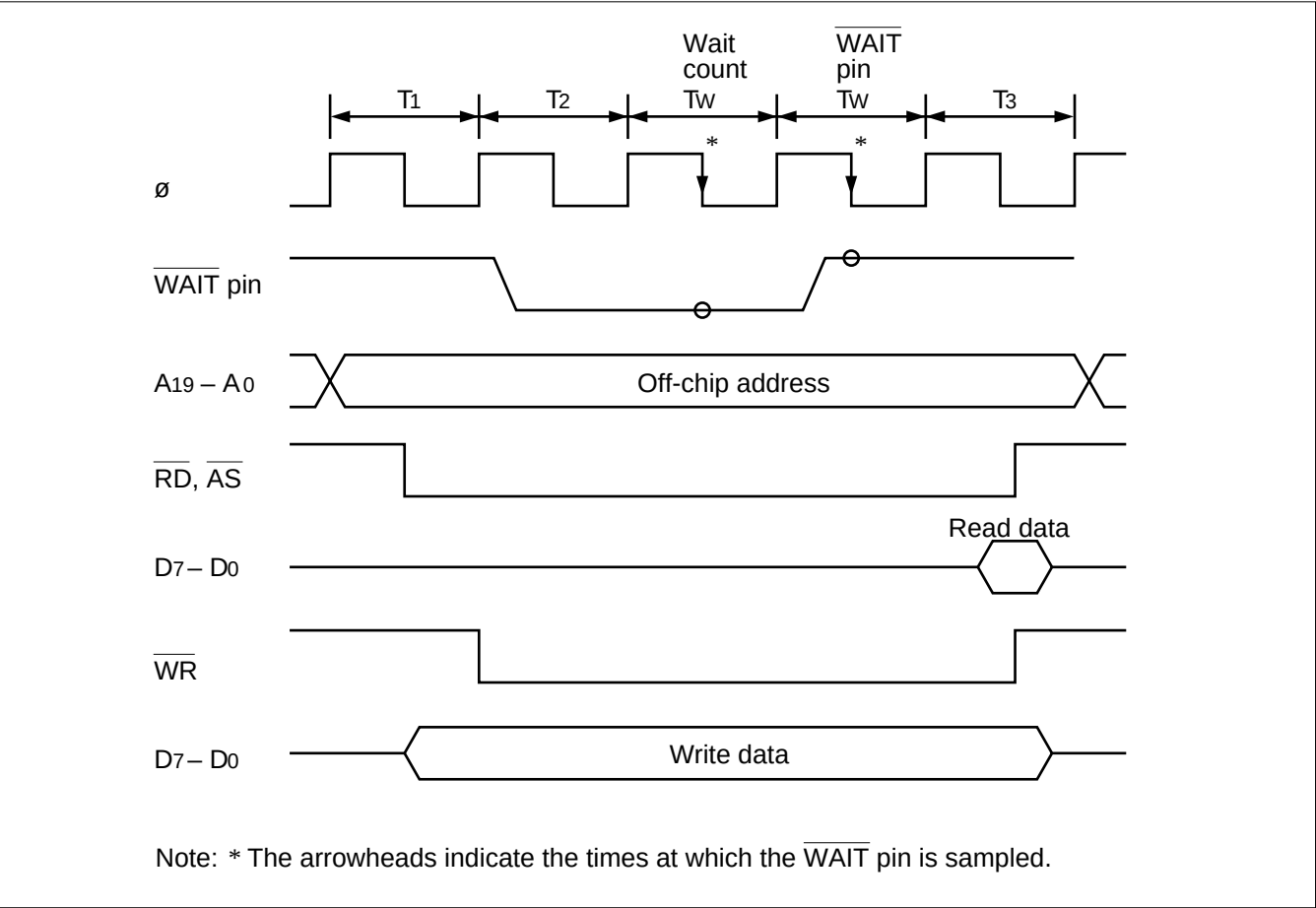


Figure 7-3 Pin Wait Mode

7.3.3 Pin Auto-Wait Mode

The pin auto-wait mode is selected when $WMS1 = 1$ and $WMS0 = 1$.

In this mode the $\overline{WAIT}$ function of the P10/ $\overline{WAIT}$ pin is used automatically.

In this mode, the number of wait states indicated by bits $WC1$ and $WC0$ are inserted, but only if there is a low input at the $\overline{WAIT}$ pin.

Figure 7-4 shows the timing of this operation when the wait count is 1.

In the pin auto-wait mode, the $\overline{WAIT}$ pin is sampled only once, on the falling edge of the ϕ clock in the T_2 state. If the $\overline{WAIT}$ pin is low at this time, the wait-state controller inserts the number of wait states indicated by bits $WC1$ and $WC0$. The $\overline{WAIT}$ pin is not sampled during the T_w and T_3 states, so no additional wait states are inserted even if the $\overline{WAIT}$ pin continues to be held low.

This mode offers a simple way to interface a low-speed device: the wait states can be inserted by routing the address strobe ($\overline{AS}$) signal to the $\overline{WAIT}$ pin and gating it with an address decode signal.

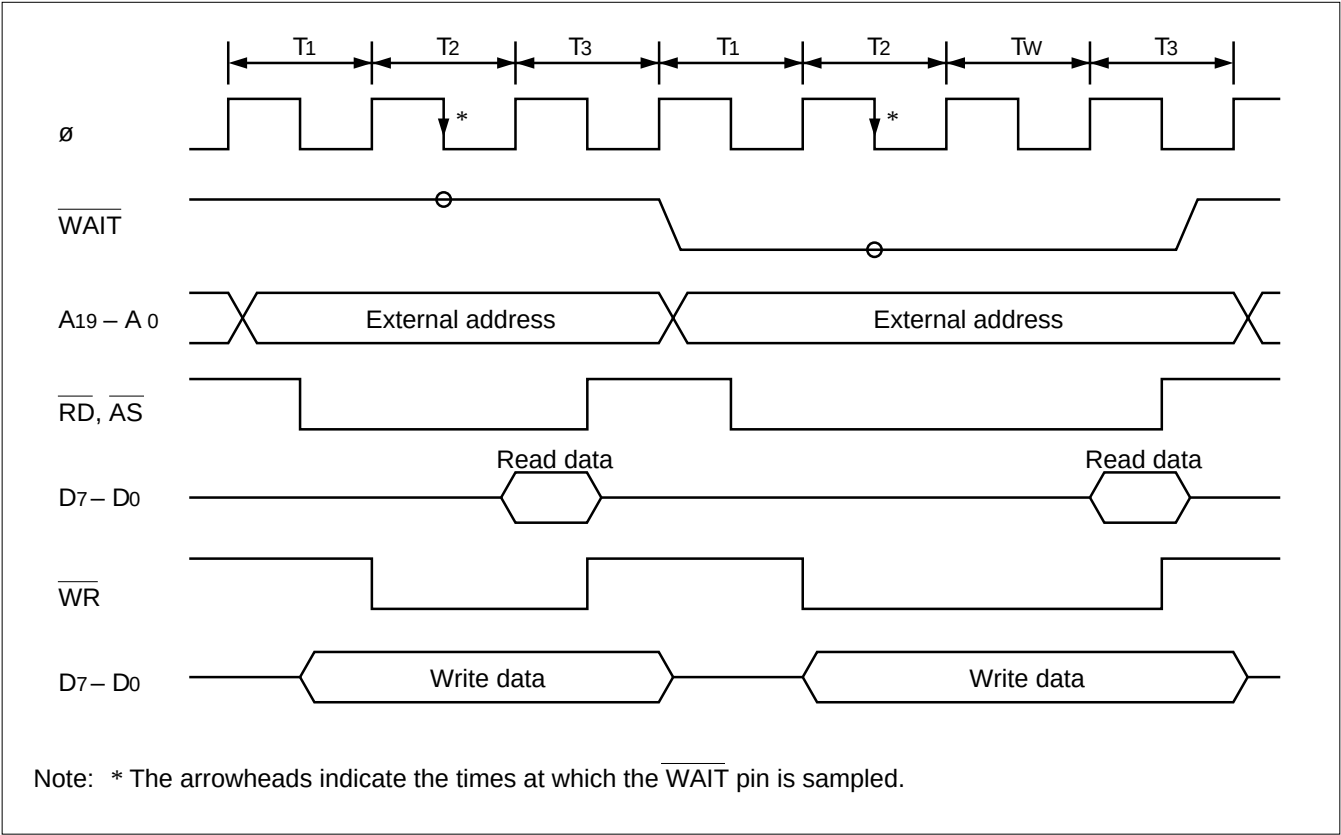


Figure 7-4 Pin Auto-Wait Mode

Section 8 Clock Pulse Generator

8.1 Overview

The H8/520 chip has a built-in clock pulse generator (CPG) consisting of an oscillator circuit, a system ($\emptyset$) clock divider, and a prescaler. The prescaler generates clock signals for the on-chip supporting modules.

8.1.1 Block Diagram

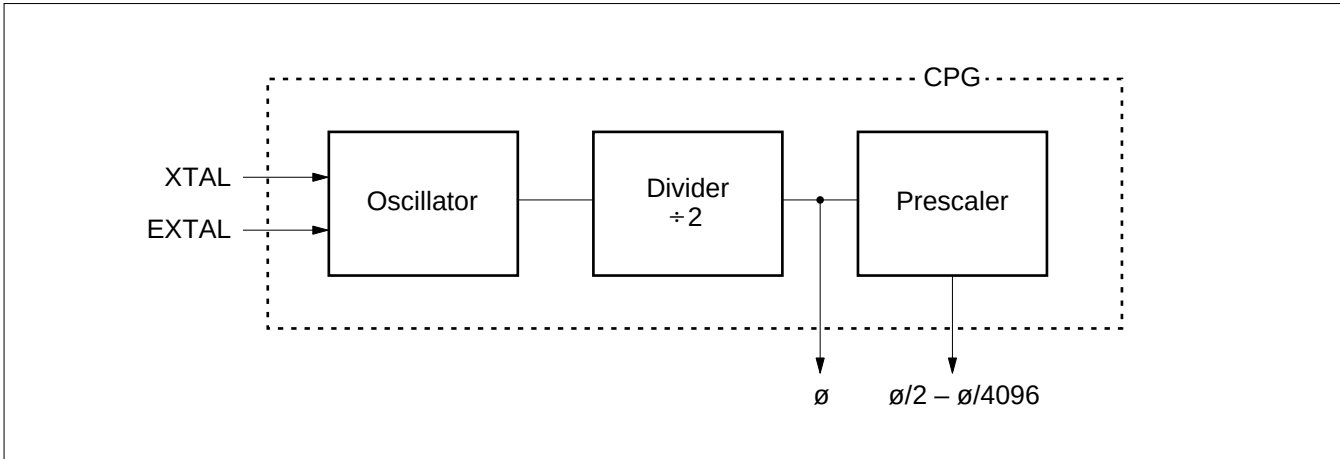


Figure 8-1 Block Diagram of Clock Pulse Generator

8.2 Oscillator Circuit

If an external crystal is connected across the EXTAL and XTAL pins, the on-chip oscillator circuit generates a clock signal for the system clock divider. Alternatively, an external clock signal can be applied directly.

1. Connecting an External Crystal

Circuit Configuration: An external crystal can be connected as in the example in figure 8-2. An AT-cut parallel resonating crystal should be used.

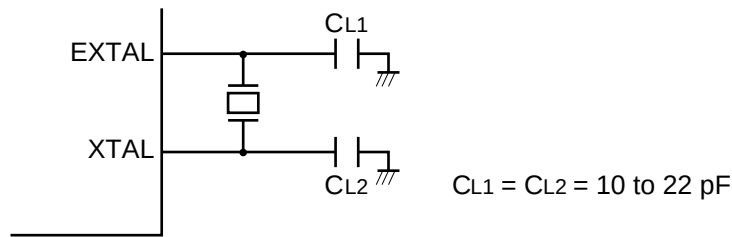


Figure 8-2 Connection of Crystal Oscillator (Example)

Crystal Oscillator: The external crystal should have the characteristics listed in table 8-1.

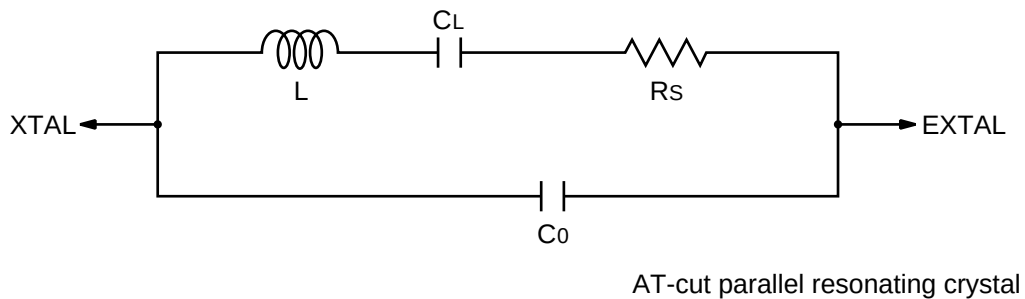


Figure 8-3 Crystal Oscillator Equivalent Circuit

Table 8-1 External Crystal Parameters

Frequency (MHz)	2	4	8	12	16	20
Rs max (Ω)	500	120	60	40	30	20
C0 (pF)	7 pF max					

Note on Board Design: When an external crystal is connected, other signal lines should be kept away from the crystal circuit to prevent induction from interfering with correct oscillation. See figure 8-4.

When the board is designed, the crystal and its load capacitors should be placed as close as possible to the XTAL and EXTAL pins.

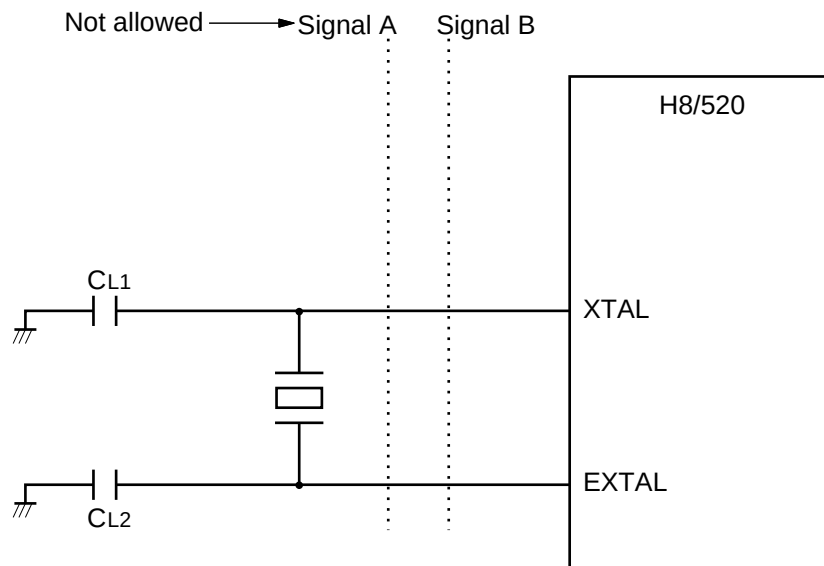


Figure 8-4 Notes on Board Design around External Crystal

2. Input of External Clock Signal

Circuit Configuration: An external clock signal can be input at the EXTAL pin as shown in the example in figure 8-5.

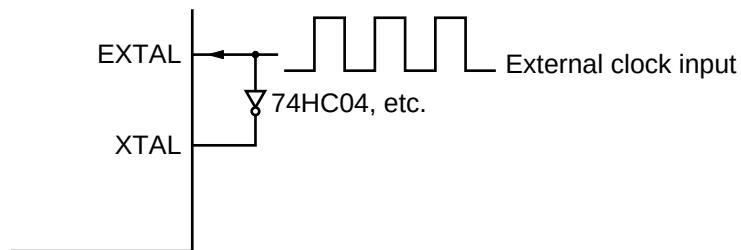


Figure 8-5 External Clock Input (Example)

Note: The masked ROM version can be driven by supplying an external clock signal to the EXTAL pin only, leaving the XTAL pin open. The PROM version can also be driven in this way, leaving the XTAL pin open, when the clock frequency is 16 MHz or less.

External Clock Input

Frequency	Double the system clock (ϕ) frequency
Duty factor	45% to 55%

Note on Connection: Invert the clock signal provided to the EXTAL pin, with a 74HC04 for example, and input the inverted clock signal to the XTAL pin.

8.3 System Clock Divider

The system clock divider divides the crystal oscillator or external clock frequency (f_{osc}) by 2 to create the ϕ clock.

Section 9 I/O Ports

9.1 Overview

The H8/520 has seven parallel I/O ports. Ports 1 to 5 are eight-bit input/output ports. Port 6 is a four-bit (or eight-bit*) input-only port. Port 7 is a six-bit input/output port. Table 9-1 summarizes the functions of each port.

Input and output are memory-mapped. The CPU views each port as a data register (DR) located in the register field at the high end of page 0 of the address space. Each port (except port 6) also has a data direction register (DDR) which determines which pins are used for input and which for output.

To read data from an I/O port, the CPU selects input in the data direction register and reads the data register. This causes the input logic level at the pin to be placed directly on the internal data bus. There is no intervening input latch.

To send data to an output port, the CPU selects output in the data direction register and writes the desired data in the data register, causing the data to be held in a latch. The latch output drives the pin through a buffer amplifier. If the CPU reads the data register of an output port, it obtains the data held in the latch rather than the actual level of the pin.

As table 9-1 indicates, all of the I/O port pins have dual functions. For example, pin 0 of port 1 can be used either as a general-purpose I/O pin (P10), or for input of the $\overline{\text{WAIT}}$ signal. The function of a pin is determined by the MCU operating mode, or by a value set in a control register.

Outputs from ports 1 to 4 can drive one TTL load and a 90-pF capacitive load. Outputs from ports 5 and 7 can drive one TTL load and a 30-pF capacitive load.

Outputs from ports 1 to 5 and 7 can also drive a Darlington transistor pair. Outputs from port 3 can drive a light-emitting diode (with 10-mA current sink). Ports 3 and 4 have built-in MOS pull-ups for each input. Port 5 has Schmitt inputs.

Schematic diagrams of the I/O port circuits are shown in appendix C.

Note: * CP-68 package only

Table 9-1 I/O Port Summary

Expanded Modes							Single-Chip Mode
Port	Description	Pins	Mode 1	Mode 2	Mode 3	Mode 4	Mode 7
Port 1	8-bit input/output port	P17/ $\overline{\text{WR}}$	$\overline{\text{AS}}$, $\overline{\text{RD}}$, and $\overline{\text{WR}}$ output				P17, P16, and P15 input/output
		P16/ $\overline{\text{RD}}$					
		P15/ $\overline{\text{AS}}$					
		P14/A16/ $\overline{\text{IRQ}}_3$	$\overline{\text{IRQ}}_3$ and $\overline{\text{IRQ}}_2$ input and		Page address	$\overline{\text{IRQ}}_3$ and $\overline{\text{IRQ}}_2$	$\overline{\text{IRQ}}_3$ and $\overline{\text{IRQ}}_2$
		P13/A17/ $\overline{\text{IRQ}}_2$	P14 and P13 input/output		output (A16, A17)	input, page address (A16, A17) output, and P14 and P13 input	input and P14 and P13 input/output
		P12/A18/ $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$	$\overline{\text{IRQ}}_1$ input, $\overline{\text{ADTRG}}$ input, and P12 input/output		Page address output (A18)	$\overline{\text{IRQ}}_1$ input, page address (A18) output, $\overline{\text{ADTRG}}$ input, and P12 input	$\overline{\text{IRQ}}_1$ input, $\overline{\text{ADTRG}}$ input, and P12 input/output
		P11/ $\overline{\text{IRQ}}_0$	$\overline{\text{IRQ}}_0$ input and P11 input/output				P10 input/output
		P10/ $\overline{\text{WAIT}}$	$\overline{\text{WAIT}}$ input and P10 input/output				
Port 2	8-bit input/output port	P27 – P20/ D7 – D0	Data bus (D7 to D0)				P27 to P20 input/output
Port 3	8-bit input/output port	P37 – P30/ A7 – A0	Low address bus output (A7 – A0)	Low address bus output (A7 – A0)	Low address bus output (A7 – A0)	Low address bus output (A7 – A0)	P37 to P30 input/output
	(Built-in MOS input pull-up. Can drive LEDs.)						

Table 9-1 I/O Port Summary (cont)

Port	Description	Pins	Expanded Modes				Single-Chip Mode
			Mode 1	Mode 2	Mode 3	Mode 4	Mode 7
Port 4	8-bit input/output port (Built-in MOS input pull-up)	P47/A15/ $\overline{\text{IRQ}}_7$	High	$\overline{\text{IRQ}}_7$ to	High	$\overline{\text{IRQ}}_7$ to	$\overline{\text{IRQ}}_7$ to
		P46/A14/ $\overline{\text{IRQ}}_6$	address	$\overline{\text{IRQ}}_4$ input,	address	$\overline{\text{IRQ}}_4$ input,	$\overline{\text{IRQ}}_4$ input
		P45/A13/ $\overline{\text{IRQ}}_5$	bus	high address	bus	high address	and P47 to P44
		A44/A12/ $\overline{\text{IRQ}}_4$	output	bus output	output	bus output	input/output
			(A15 to A8)	(A15 to A12), and P47 to P44 input	(A15 to A8)	(A15 to A12), and P47 to P44 input	
		P43/A11		High address		High address	P43 to P40
		P42/A10		bus output		bus output	input/output
		P41/A9		(A11 to A8),		(A11 to A8),	
		P40/A8		and P43 to P40 input		and P43 to P40 input	
Port 5	8-bit input/output port (Schmit trigger input)	P57/FTOA2/ $\emptyset$ P56/FTOA1 P55/FTOB2/ FTCl2 P54/FTOB1/ FTCl1 P53/TMO P52/FTI2/ TMRI P51/FTI1 P50/TMCI	General-purpose input/output pins (P57 to P50) also used for input and output by the 16-bit free-running timer module (FTOA2, FTOA1, FTOB1, FTCl2, FTCl1, FTI2, FTI1) and 8-bit timer module (TMO, TMRI, TMCI), and for output of the system clock ($\emptyset$).				

Table 9-1 I/O Port Summary (cont)

Port	Description	Pins	Expanded Modes				Single-Chip Mode
			Mode 1	Mode 2	Mode 3	Mode 4	Mode 7
Port 6	4-bit input port (8-bit input port)	P6 ₃ – P6 ₀ / AN ₃ – AN ₀	General-purpose input (P6 ₃ to P6 ₀) and analog input (AN ₀ to AN ₃)				
		(P6 ₇ – P6 ₀ / AN ₇ – AN ₀)*	[General-purpose input (P6 ₇ to P6 ₀) and analog input (AN ₇ to AN ₀)]*				
Port 7	6-bit input/output port	P7 ₅ /SCK ₁	General-purpose input/output pins (P7 ₅ to P7 ₃), also used for input and output by serial communication interface channel 1 (SCK ₁ , RXD ₁ , TXD ₁)				
		P7 ₄ /RXD ₁ P7 ₃ /TXD ₁					
		P7 ₂ /A ₁₉ / SCK ₂	SCK ₂ (serial communication interface channel 2 clock) input/output and P7 ₂ input/output		Page address output (A ₁₉)	SCK ₂ input/ output, page address output (A ₁₉), or P7 ₂ input/output	SCK ₂ input/ output or P7 ₂ input/output
		P7 ₁ /RXD ₂ P7 ₀ /TXD ₂	General-purpose input/output pins (P7 ₁ , P7 ₀), also used for input and output by serial communication interface channel 2 (RXD ₂ , TXD ₂)				

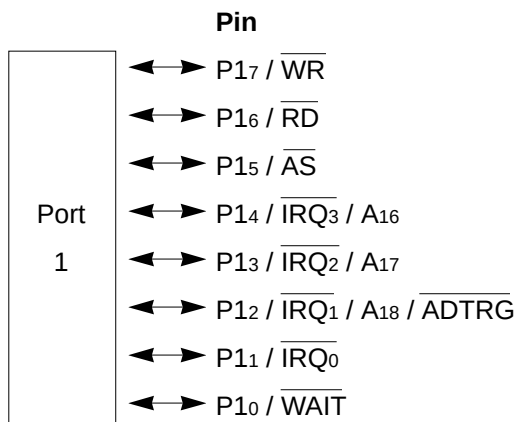
Note: * CP-68 package only.

9.2 Port 1

9.2.1 Overview

Port 1 is an 8-bit input/output port with the pin configuration shown in figure 9-1. The pin functions depend on the MCU operating mode. Some pins can perform two or three functions simultaneously.

Outputs from port 1 can drive one TTL load and a 90-pF capacitive load. They can also drive a Darlington transistor pair.



Modes 1 and 2

$\overline{WR}$ (output)

$\overline{RD}$ (output)

$\overline{AS}$ (output)

P14 (input/output) / $\overline{IRQ}_3$ (input)

P13 (input/output) / $\overline{IRQ}_2$ (input)

P12 (input/output) / $\overline{IRQ}_1$ (input) /

$\overline{ADTRG}$ (input)

P11 (input/output) / $\overline{IRQ}_0$ (input)

P10 (input/output) / $\overline{WAIT}$ (input)

Mode 4

$\overline{WR}$ (output)

$\overline{RD}$ (output)

$\overline{AS}$ (output)

P14 (input) / $\overline{IRQ}_3$ (input) /

A16 (output)

P13 (input) / $\overline{IRQ}_2$ (input) /

A17 (output)

P12 (input) / $\overline{IRQ}_1$ (input) /

A18 (output) / $\overline{ADTRG}$ (input)

P11 (input/output) / $\overline{IRQ}_0$ (input)

P10 (input/output) / $\overline{WAIT}$ (input)

Mode 3

$\overline{WR}$ (output)

$\overline{RD}$ (output)

$\overline{AS}$ (output)

A16 (output)

A17 (output)

A18 (output)

P11 (input/output) / $\overline{IRQ}_0$ (input)

P10 (input/output) / $\overline{WAIT}$ (input)

Single-Chip Mode

P17 (input/output)

P16 (input/output)

P15 (input/output)

P14 (input/output) / $\overline{IRQ}_3$ (input)

P13 (input/output) / $\overline{IRQ}_2$ (input)

P12 (input/output) / $\overline{IRQ}_1$ (input) /

$\overline{ADTRG}$ (input)

P11 (input/output) / $\overline{IRQ}_0$ (input)

P10 (input/output)

Figure 9-1 Pin Functions of Port 1

9.2.2 Port 1 Registers

Table 9-2 lists the registers of port 1.

Table 9-2 Port 1 Registers

Name	Abbreviation	Read/Write	Initial Value	Address
Port 1 data direction register	P1DDR	W	H'00*	H'FF80
Port 1 data register	P1DR	R/W	H'00	H'FF82

Note: * In single-chip mode.

1. Port 1 Data Direction Register (P1DDR)—H'FF80

Bit	7	6	5	4	3	2	1	0
	P17DDR	P16DDR	P15DDR	P14DDR	P13DDR	P12DDR	P11DDR	P10DDR
Initial value*	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

Note: * In single-chip mode

P1DDR is an 8-bit register that selects the direction of each pin in port 1. Details are given for each MCU operating mode below.

Modes 1 and 2 (Expanded Minimum Modes): Bits 7 to 5 of P1DDR are fixed at 1 and cannot be written. Pins P17 to P15 are used for output of bus control signals.

When bits 4 to 0 of P1DDR are set to 1, the corresponding pin of port 1 functions as an output pin. When these bits are cleared to 0, the corresponding pin functions as an input pin.

Mode 3 (Expanded Maximum Mode with On-Chip ROM Disabled): Bits 7 to 2 of P1DDR are fixed at 1 and cannot be written. Pins P17 to P15 are used for output of bus control signals. Pins P14 to P12 are used for page address output.

When bits 1 and 0 of P1DDR are set to 1, the corresponding pin of port 1 functions as an output pin. When these bits are cleared to 0, the corresponding pin functions as an input pin.

Mode 4 (Expanded Maximum Mode with On-Chip ROM Enabled): Bits 7 to 5 of P1DDR are fixed at 1 and cannot be written. Pins P17 to P15 are used for output of bus control signals.

When bits 4 to 2 of P1DDR are set to 1, pins P14 to P12 are used for page address output. When these bits are cleared to 0, the corresponding pin becomes available for general-purpose input.

When bits 1 and 0 of P1DDR are set to 1, pins P11 and P10 function as output pins. When these bits are cleared to 0, the corresponding pin functions as an input pin.

Mode 7 (Single-Chip Mode): A pin functions as an output pin if the corresponding bit in P1DDR is set to 1, and as an input pin if the bit is cleared to 0.

P1DDR can be written but not read. An attempt to read this register does not cause an error, but all bits are read as 1, regardless of their true values.

P1DDR is initialized to H'00 by a reset and in the hardware standby mode. P1DDR is not initialized in the software standby mode, so if a P1DDR bit is set to 1 when the chip enters the software standby mode, the corresponding pin continues to output the value in the port 1 data register.

2. Port 1 Data Register (P1DR)—H'FF82

Bit	7	6	5	4	3	2	1	0
	P17	P16	P15	P14	P13	P12	P11	P10
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P1DR is an 8-bit register containing output data for pins P17 to P10. When port 1 is read, output pins return the value in the P1DR latch, regardless of the actual level at the pin. Input pins return the level at the pin, not the value in the P1DR latch.

If any of the port 1 data direction bits are cleared to 0, selecting input, use only data transfer (MOV) instructions to write data in P1DR. Do not use arithmetic, logic, or bit manipulation instructions. These instructions read the input pins and may write unintended data in P1DR.

9.2.3 Pin Functions in Each Mode

The functions of port 1 depend on the MCU operating mode. Table 9-3 shows the pin functions in modes 1 and 2. Table 9-4 shows the pin functions in mode 3. Table 9-5 shows the pin functions in mode 4. Table 9-6 shows the pin functions in the single-chip mode.

Table 9-3 Port 1 Pin Functions in Modes 1 and 2

Pin	Functions															
WR	Output of WR signal.															
RD	Output of RD signal.															
AS	Output of AS signal.															
P14 / IRQ ₃	The function depends on the IRQ3E bit in the IRQ control register (IRQCR) and the P14DDR bit as follows: <table> <tr> <th>IRQ3E</th> <th colspan="2">0</th> <th colspan="2">1</th> </tr> <tr> <th>P14DDR</th> <th>0</th> <th>1</th> <th>0</th> <th>1</th> </tr> <tr> <th>Pin function</th> <td>P14 input</td> <td>P14 output</td> <td>IRQ₃ input and P14 input</td> <td>IRQ₃ input and P14 output</td> </tr> </table>	IRQ3E	0		1		P14DDR	0	1	0	1	Pin function	P14 input	P14 output	IRQ ₃ input and P14 input	IRQ ₃ input and P14 output
IRQ3E	0		1													
P14DDR	0	1	0	1												
Pin function	P14 input	P14 output	IRQ ₃ input and P14 input	IRQ ₃ input and P14 output												
P13 / IRQ ₂	The function depends on the IRQ2E bit and the P13DDR bit as follows: <table> <tr> <th>IRQ2E</th> <th colspan="2">0</th> <th colspan="2">1</th> </tr> <tr> <th>P13DDR</th> <th>0</th> <th>1</th> <th>0</th> <th>1</th> </tr> <tr> <th>Pin function</th> <td>P13 input</td> <td>P13 output</td> <td>IRQ₂ input and P13 input</td> <td>IRQ₂ input and P13 output</td> </tr> </table>	IRQ2E	0		1		P13DDR	0	1	0	1	Pin function	P13 input	P13 output	IRQ ₂ input and P13 input	IRQ ₂ input and P13 output
IRQ2E	0		1													
P13DDR	0	1	0	1												
Pin function	P13 input	P13 output	IRQ ₂ input and P13 input	IRQ ₂ input and P13 output												

Table 9-3 Port 1 Pin Functions in Modes 1 and 2 (cont)

Pin

Functions

P1₂ / $\overline{\text{IRQ}}_1$ / $\overline{\text{ADTRG}}$

The function depends on the IRQ1E bit, the $\text{P1}_2\text{DDR}$ bit, and the trigger enable bit (TRGE) in the A/D control register (ADCR) as follows:

TRGE	0			
IRQ1E	0		1	
P1 ₂ DDR	0	1	0	1
Pin function	P1 ₂ input	P1 ₂ output	$\overline{\text{IRQ}}_1$ input and P1 ₂ input	$\overline{\text{IRQ}}_1$ input and P1 ₂ output

TRGE	1			
IRQ1E	0		1	
P1 ₂ DDR	0	1	0	1
Pin function	$\overline{\text{ADTRG}}$ input and P1 ₂ input	$\overline{\text{ADTRG}}$ input and P1 ₂ output	$\overline{\text{ADTRG}}$ input, $\overline{\text{IRQ}}_1$ input, and P1 ₂ input	$\overline{\text{ADTRG}}$ input, $\overline{\text{IRQ}}_1$ input, and P1 ₂ output

P1₁ / $\overline{\text{IRQ}}_0$

The function depends on the IRQ0E bit and the $\text{P1}_1\text{DDR}$ bit as follows:

IRQ0E	0		1	
P1 ₁ DDR	0	1	0	1
Pin function	P1 ₁ input	P1 ₁ output	$\overline{\text{IRQ}}_0$ input and P1 ₁ input	$\overline{\text{IRQ}}_0$ input and P1 ₁ output

P1₀ / WAIT

The function depends on the wait mode select 1 bit (WMS1) of the wait-state control register (WCR) and the $\text{P1}_0\text{DDR}$ bit as follows:

WMS1	0		1	
P1 ₀ DDR	0	1	0	1
Pin function	P1 ₀ input	P1 ₀ output	WAIT input	

Table 9-4 Port 1 Pin Functions in Mode 3

Pin	Functions
WR	Output of WR signal.
RD	Output of RD signal.
AS	Output of AS signal.
A16	A16 output
A17	A17 output
A18	A18 output
P11 / $\overline{\text{IRQ}}_0$	The function depends on the IRQ0E bit and the P11DDR bit as follows:

IRQ0E	0		1	
P11DDR	0	1	0	1
Pin function	P11 input	P11 output	$\overline{\text{IRQ}}_0$ input and P11 input	$\overline{\text{IRQ}}_0$ input and P11 output

P10 / WAIT The function depends on the wait mode select 1 bit (WMS1) of the wait-state control register (WCR) and the P10DDR bit as follows:

WMS1	0		1	
P10DDR	0	1	0	1
Pin function	P10 input	P10 output	WAIT input	

Table 9-5 Port 1 Pin Functions in Mode 4

Pin	Functions
WR	Output of WR signal.
RD	Output of RD signal.
AS	Output of AS signal.
P14 / $\overline{\text{IRQ}}_3$ / A16	The function depends on the IRQ3E bit in the IRQ control register (IRQCR) and the P14DDR bit as follows:

IRQ3E	0		1	
P14DDR	0	1	0	1
Pin function	P14 input	A16 output	$\overline{\text{IRQ}}_3$ input and P14 input	A16 output

P13 / $\overline{\text{IRQ}}_2$ /
A17

The function depends on the IRQ2E bit and the P13DDR bit as follows:

IRQ2E	0		1	
P13DDR	0	1	0	1
Pin function	P13 input	A17 output	$\overline{\text{IRQ}}_2$ input and P13 input	A17 output

P12 / $\overline{\text{IRQ}}_1$ /
A18 /
ADTRG

The function depends on the IRQ1E bit, the P12DDR bit, and the trigger enable bit (TRGE) in the A/D control register (ADCR) as follows:

TRGE	0			
IRQ1E	0		1	
P12DDR	0	1	0	1
Pin function	P12 input	A18 output	$\overline{\text{IRQ}}_1$ input and P12 input	A18 output

TRGE	1			
IRQ1E	0		1	
P12DDR	0	1	0	1
Pin function	$\overline{\text{ADTRG}}$ input and P12 input	A18 output	$\overline{\text{ADTRG}}$ input, $\overline{\text{IRQ}}_1$ input, and P12 input	A18 output

Table 9-5 Port 1 Pin Functions in Mode 4 (cont)

Pin

Functions

P1₁ / IRQ₀

The function depends on the IRQ₀E bit and the P1₁DDR bit as follows:

IRQ ₀ E	0		1	
P1 ₁ DDR	0	1	0	1
Pin function	P1 ₁ input	P1 ₁ output	IRQ ₀ input and P1 ₁ input	IRQ ₀ input and P1 ₁ output

P1₀ / WAIT

The function depends on the wait mode select 1 bit (WMS1) of the wait-state control register (WCR) and the P1₀DDR bit as follows:

WMS1	0		1	
P1 ₀ DDR	0	1	0	1
Pin function	P1 ₀ input	P1 ₀ output	WAIT input	

Table 9-6 Port 1 Pin Functions in Single-Chip Mode**Pin Functions****P17**

P17DDR	0	1
Pin function	P17 input	P17 output

P16

P16DDR	0	1
Pin function	P16 input	P16 output

P15

P15DDR	0	1
Pin function	P15 input	P15 output

P14 / IRQ_3

The function depends on the IRQ3E bit in the IRQ control register (IRQCR) and the P14DDR bit as follows:

IRQ3E	0		1	
P14DDR	0	1	0	1
Pin function	P14 input	P14 output	IRQ_3 input and P14 input	IRQ_3 input and P14 output

P13 / IRQ_2

The function depends on the IRQ2E bit and the P13DDR bit as follows:

IRQ2E	0		1	
P13DDR	0	1	0	1
Pin function	P13 input	P13 output	IRQ_2 input and P13 input	IRQ_2 input and P13 output

Table 9-6 Port 1 Pin Functions in Single-Chip Mode (cont)**Pin Functions**

P1₂ / IRQ₁ / ADTRG The function depends on the IRQ1E bit, the P1₂DDR bit, and the trigger enable bit (TRGE) in the A/D control register (ADCR) as follows:

TRGE	0			
IRQ1E	0		1	
P1 ₂ DDR	0	1	0	1
Pin function	P1 ₂ input	P1 ₂ output	IRQ ₁ input and P1 ₂ input	IRQ ₁ input and P1 ₂ output

TRGE	1			
IRQ1E	0		1	
P1 ₂ DDR	0	1	0	1
Pin function	ADTRG input and P1 ₂ input	ADTRG input and P1 ₂ output	ADTRG input, IRQ ₁ input, and P1 ₂ input	ADTRG input, IRQ ₁ input, and P1 ₂ output

P1₁ / IRQ₀ The function depends on the IRQ0E bit and the P1₁DDR bit as follows:

IRQ0E	0		1	
P1 ₁ DDR	0	1	0	1
Pin function	P1 ₁ input	P1 ₁ output	IRQ ₀ input and P1 ₁ input	IRQ ₀ input and P1 ₁ output

P1₀

P1 ₀ DDR	0	1
Pin function	P1 ₀ input	P1 ₀ output

9.3 Port 2

9.3.1 Overview

Port 2 is an 8-bit input/output port with the pin configuration shown in figure 9-2. In the expanded modes it operates as the external data bus (D7 – D0). In the single-chip mode it operates as a general-purpose input/output port.

Outputs from port 2 can drive one TTL load and a 90-pF capacitive load. They can also drive a Darlington transistor pair.

	Pin	Expanded Modes	Single-Chip Mode
Port 2	↔ P27 / D7	D7 (input/output)	P27 (input/output)
	↔ P26 / D6	D6 (input/output)	P26 (input/output)
	↔ P25 / D5	D5 (input/output)	P25 (input/output)
	↔ P24 / D4	D4 (input/output)	P24 (input/output)
	↔ P23 / D3	D3 (input/output)	P23 (input/output)
	↔ P22 / D2	D2 (input/output)	P22 (input/output)
	↔ P21 / D1	D1 (input/output)	P21 (input/output)
	↔ P20 / D0	D0 (input/output)	P20 (input/output)

Figure 9-2 Pin Functions of Port 2

9.3.2 Port 2 Registers

Table 9-7 lists the registers of port 2.

Table 9-7 Port 2 Registers

Name	Abbreviation	Read/Write	Initial Value	Address
Port 2 data direction register	P2DDR	W	H'00	H'FF81
Port 2 data register	P2DR	R/W	H'00	H'FF83

1. Port 2 Data Direction Register (P2DDR)—H'FF81

Bit	7	6	5	4	3	2	1	0
	P27DDR	P26DDR	P25DDR	P24DDR	P23DDR	P22DDR	P21DDR	P20DDR
Initial value*	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

P2DDR is an 8-bit register that selects the direction of each pin in port 2.

Expanded Modes: P2DDR is not used.

Single-Chip Mode: A pin functions as an output pin if the corresponding bit in P2DDR is set to 1, and as an input pin if the bit is cleared to 0.

P2DDR can be written but not read. An attempt to read this register does not cause an error, but all bits are read as 1, regardless of their true values.

At a reset and in the hardware standby mode, P2DDR is initialized to H'00, making all eight pins input pins. P2DDR is not initialized in the software standby mode, so if a P2DDR bit is set to 1 when the chip enters the software standby mode, the corresponding pin continues to output the value in the port 2 data register.

2. Port 2 Data Register (P2DR)—H'FF83

Bit	7	6	5	4	3	2	1	0
	P2 ₇	P2 ₆	P2 ₅	P2 ₄	P2 ₃	P2 ₂	P2 ₁	P2 ₀
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P2DR is an 8-bit register containing output data for pins P2₇ to P2₀.

At a reset and in the hardware standby mode, P2DR is initialized to H'00.

When port 2 is read, output pins return the value in the P2DR latch, regardless of the actual level at the pin. Input pins return the level at the pin, not the value in the P2DR latch.

If any of the port 2 data direction bits are cleared to 0, selecting input, use only data transfer (MOV) instructions to write data in P2DR. Do not use arithmetic, logic, or bit manipulation instructions. These instructions read the input pins and may write unintended data in P2DR.

9.3.3 Pin Functions in Each Mode

Port 2 has different functions in the expanded modes (modes 1, 2, 3, 4) and the single-chip mode (mode 7). Separate descriptions are given below.

Pin Functions in Expanded Modes: In the expanded modes (modes 1, 2, 3, and 4), port 2 is automatically used as the data bus and P2DDR is ignored. Figure 9-3 shows the pin functions for the expanded modes.

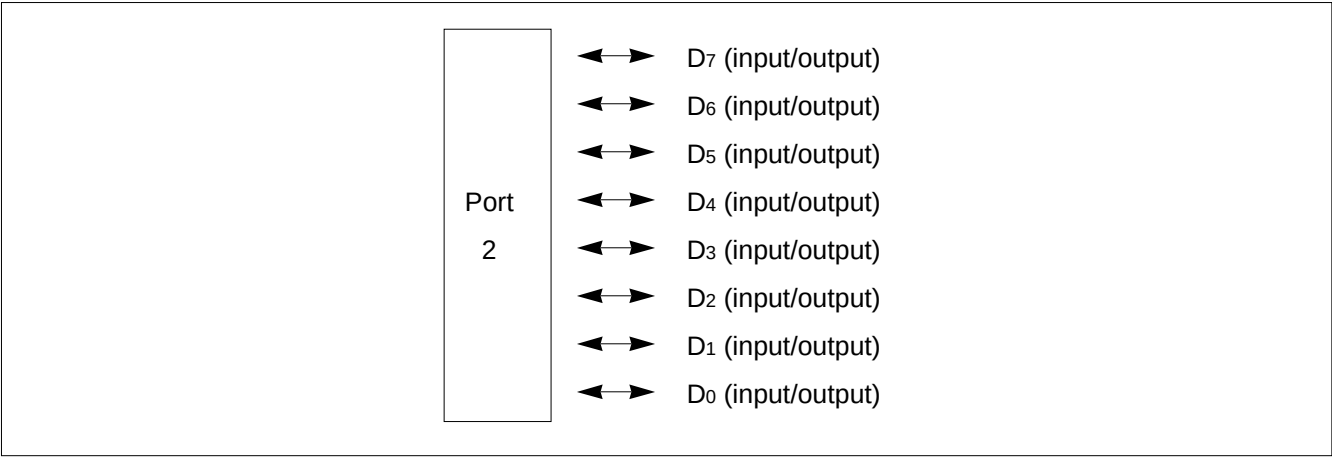


Figure 9-3 Port 2 Pin Functions in Expanded Modes

Pin Functions in Single-Chip Mode: In the single-chip mode (mode 7), each of the port 2 pins can be designated as an input pin or an output pin, as indicated in figure 9-4, by setting the corresponding bit in P2DDR to 1 for output or clearing it to 0 for input.

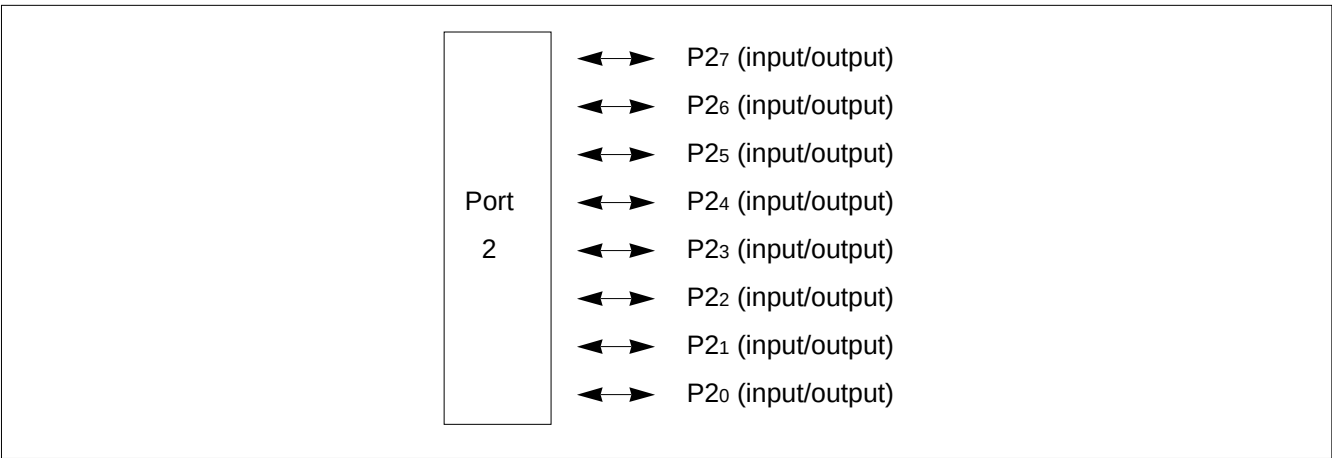


Figure 9-4 Port 2 Pin Functions in Single-Chip Mode

9.4 Port 3

9.4.1 Overview

Port 3 is an 8-bit input/output port with the pin configuration shown in figure 9-5. In the expanded modes it provides the low bits (A7 – A0) of the address bus. In the single-chip mode it operates as a general-purpose input/output port.

Port 3 has built-in MOS pull-ups that can be turned on or off under program control.

Outputs from port 3 can drive one TTL load and a 90-pF capacitive load. They can also drive a Darlington transistor pair or LED (with 10-mA current sink).

		Pin	Modes 1 to 4	Single-Chip Mode
Port 3	↔	P37 / A7	A7 (output)	P37 (input/output)
	↔	P36 / A6	A6 (output)	P36 (input/output)
	↔	P35 / A5	A5 (output)	P35 (input/output)
	↔	P34 / A4	A4 (output)	P34 (input/output)
	↔	P33 / A3	A3 (output)	P33 (input/output)
	↔	P32 / A2	A2 (output)	P32 (input/output)
	↔	P31 / A1	A1 (output)	P31 (input/output)
	↔	P30 / A0	A0 (output)	P30 (input/output)

Figure 9-5 Pin Functions of Port 3

9.4.2 Port 3 Registers

Table 9-8 lists the registers of port 3.

Table 9-8 Port 3 Registers

Name	Abbreviation	Read/Write	Initial Value	Address
Port 3 data direction register	P3DDR	W	H'00*	H'FF84
Port 3 data register	P3DR	R/W	H'00	H'FF86

Note: * Initialized to H'00 in modes 2, 4, and 7. Fixed at H'FF in modes 1 and 3.

1. Port 3 Data Direction Register (P3DDR)—H'FF84

Bit	7	6	5	4	3	2	1	0
	P3 ₇ DDR	P3 ₆ DDR	P3 ₅ DDR	P3 ₄ DDR	P3 ₃ DDR	P3 ₂ DDR	P3 ₁ DDR	P3 ₀ DDR
Initial value*	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

Note: * In mode 2, 4, and 7

P3DDR is an 8-bit register that selects the direction of each pin in port 3.

Modes 1, 2, 3, and 4: All bits of P3DDR are fixed at 1 and cannot be modified. Port 3 is used for address bus output.

Single-Chip Mode: A pin functions as an output pin if the corresponding bit in P3DDR is set to 1, and as an input pin if the bit is cleared to 0.

P3DDR can be written but not read. An attempt to read this register does not cause an error, but all bits are read as 1, regardless of their true values.

At a reset and in the hardware standby mode, P3DDR is initialized to H'00, making all eight pins input pins. P3DDR is not initialized in the software standby mode, so if a P3DDR bit is set to 1 when the chip enters the software standby mode, the corresponding pin continues to output the value in the port 3 data register.

2. Port 3 Data Register (P3DR)—H'FF86

Bit	7	6	5	4	3	2	1	0
	P3 ₇	P3 ₆	P3 ₅	P3 ₄	P3 ₃	P3 ₂	P3 ₁	P3 ₀
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P3DR is an 8-bit register containing output data for pins P37 to P30.

At a reset and in the hardware standby mode, P3DR is initialized to H'00.

When port 3 is read, output pins return the value in the P3DR latch, regardless of the actual level at the pin. Input pins return the level at the pin, not the value in the P3DR latch.

If any of the port 3 data direction bits are cleared to 0, selecting input, use only data transfer (MOV) instructions to write data in P3DR. Do not use arithmetic, logic, or bit manipulation instructions. These instructions read the input pins and may write unintended data in P3DR.

9.4.3 Pin Functions in Each Mode

Port 3 has different functions in the expanded modes (modes 1 to 4), and the single-chip mode (mode 7). Separate descriptions are given below.

Pin Functions in Modes 1 to 4: In the expanded modes, port 3 is used for output of the low bits (A7 – A0) of the address bus. P3DDR is automatically set for output. Figure 9-6 shows the pin functions for the expanded modes.

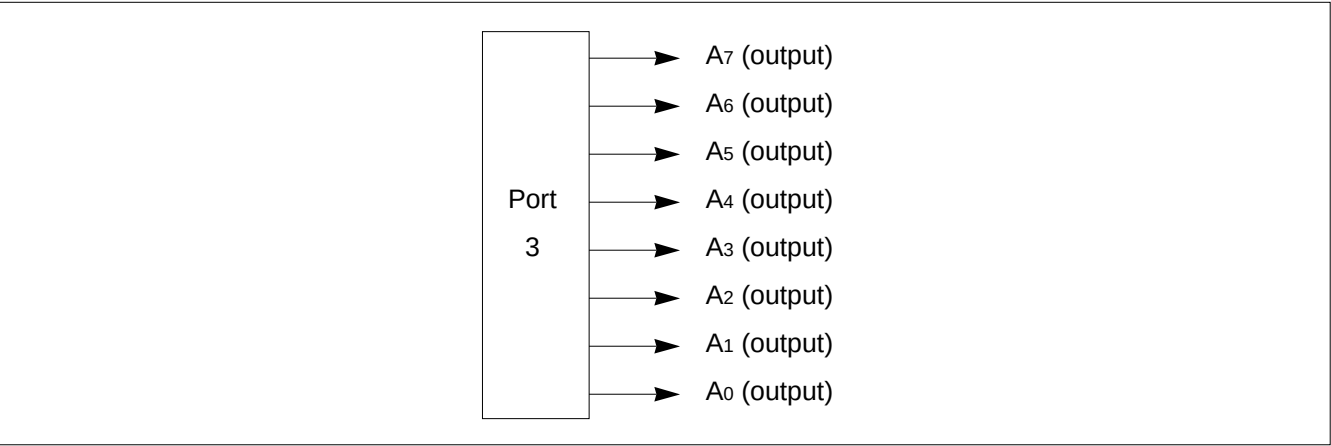


Figure 9-6 Port 3 Pin Functions in Modes 1 and 3

Pin Functions in Single-Chip Mode: In the single-chip mode (mode 7), each of the port 3 pins can be designated as an input pin or an output pin, as indicated in figure 9-7, by setting the corresponding bit in P3DDR to 1 for output or clearing it to 0 for input.

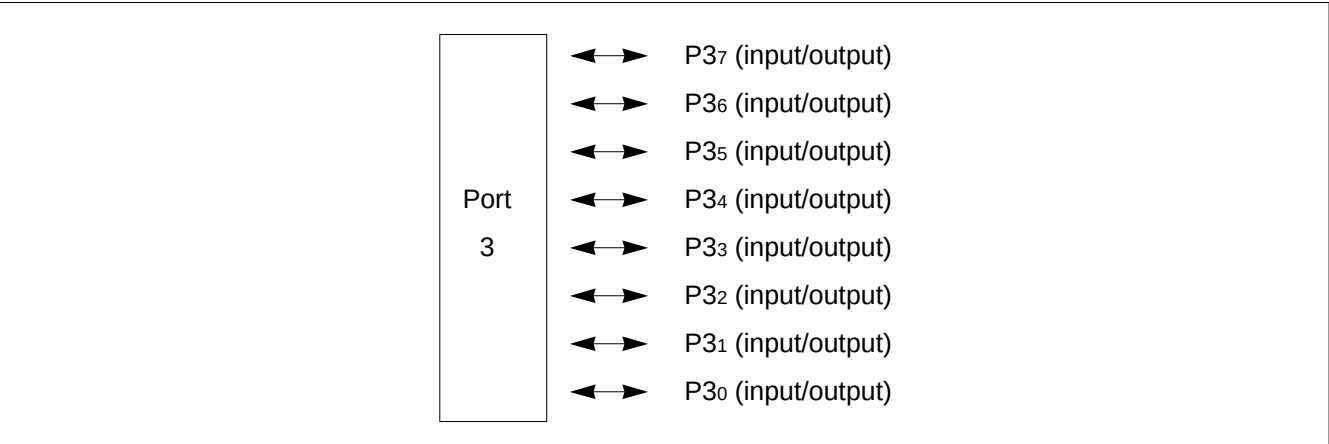


Figure 9-7 Port 3 Pin Functions in Single-Chip Mode

9.4.4 Built-in MOS Pull-Up

The MOS input pull-ups of port 3 are turned on by clearing the corresponding bit in P3DDR to 0 and writing a 1 in P3DR. These pull-ups are turned off at a reset and in the hardware standby mode. Table 9-9 indicates the status of the MOS pull-ups in various modes.

Table 9-9 Status of MOS Pull-Ups for Port 3

Mode	Reset	Hardware Standby Mode	Other Operating States*
1	OFF	OFF	OFF
2			
3			
4			
7			ON/OFF

Note: * Including the software standby mode.
Notation: OFF: The MOS pull-up is always off.
ON/OFF: The MOS pull-up is on when P3DDR = 0 and P3DR = 1, and off otherwise.

Note on Usage of MOS Pull-Ups: If a bit manipulation instruction (BSET, BCLR, or BNOT) is used to modify the port 3 data register, since the instruction rewrites the data register according to the levels of input pins, it may switch their built-in MOS pull-ups on or off unintentionally.

The same precaution applies to port 4.

Example (BSET Instruction): Suppose a BSET instruction is executed to set bit 0 in the port 3 data register (P3DR) under the following conditions.

P37: Input pin, low, MOS pull-up transistor on
P36: Input pin, high, MOS pull-up transistor off
P35 – P30: Output pins, low

The intended purpose of this BSET instruction is to switch the output level at P30 from low to high.

Before Execution of BSET Instruction

	P37	P36	P35	P34	P33	P32	P31	P30
Input/output	Input	Input	Output	Output	Output	Output	Output	Output
Pin state	Low	High	Low	Low	Low	Low	Low	Low
DDR	0	0	1	1	1	1	1	1
DR	1	0	0	0	0	0	0	0
Pull-up	On	Off	Off	Off	Off	Off	Off	Off

Execution of BSET Instruction

`BSET, B #0, @PORT3` ; set bit 0 in port 3 data register

After Execution of BSET Instruction

	P57	P56	P55	P54	P53	P52	P51	P50
Input/output	Input	Input	Output	Output	Output	Output	Output	Output
Pin state	Low	High	Low	Low	Low	Low	Low	High
DDR	0	0	1	1	1	1	1	1
DR	0	1	0	0	0	0	0	1
Pull-up	Off	On	Off	Off	Off	Off	Off	Off

Explanation: To execute the BSET instruction, the CPU begins by reading port 3. Since P37 and P36 are input pins, the CPU reads the level of these pins directly, not the value in the P3DR data register. It reads P37 as low (0) and P36 as high (1).

Since P35 to P30 are output pins, for these pins the CPU reads the value in the data register (0). The CPU therefore reads the value of port 3 as H'40, although the actual value in P3DR is H'80.

Next the CPU sets bit 0 of the read data to 1, changing the value to H'41.

Finally, the CPU writes this value (H'41) back to P3DR to complete the BSET instruction.

As a result, bit P30 is set to 1, switching pin P30 to high output. In addition, bits P37 and P36 are both modified, changing the on/off settings of the MOS pull-up transistors of pins P37 and P36.

Programming Solution: The switching of the pull-ups for P37 and P36 in this example can be avoided by reserving a one-byte work area in RAM, performing bit manipulations in the work area, then transferring the work area contents to the port 3 data register. RAM0 is a symbol for the user-selected address of the work area below.

Before Execution of BSET Instruction

MOV.B #80, R0	; put write data (H'80) for port 3 data register in R0
MOV.B R0, @RAM0	; transfer from R0 to work area (RAM0)
MOV.B R0, @PORT3	; transfer from R0 to port 3 data register

	P37	P36	P35	P34	P33	P32	P31	P30
Input/output	Input	Input	Output	Output	Output	Output	Output	Output
Pin state	Low	High	Low	Low	Low	Low	Low	Low
DDR	0	0	1	1	1	1	1	1
DR	1	0	0	0	0	0	0	0
Pull-up	On	Off	Off	Off	Off	Off	Off	Off
RAM0	1	0	0	0	0	0	0	0

Execution of BSET Instruction

BSET, B #0, @RAM0	; set bit 0 in work area (RAM0)
-------------------	---------------------------------

After Execution of BSET Instruction

MOV.B @RAM0, R0
MOV.B R0, @PORT3

; get value in work area (RAM0)

; write value to port 3 data register

	P3₇	P3₆	P3₅	P3₄	P3₃	P3₂	P3₁	P3₀
Input/output	Input	Input	Output	Output	Output	Output	Output	Output
Pin state	Low	High	Low	Low	Low	Low	Low	High
DDR	0	0	1	1	1	1	1	1
DR	1	0	0	0	0	0	0	1
Pull-up	On	Off	Off	Off	Off	Off	Off	Off
RAM0	1	0	0	0	0	0	0	0

9.5 Port 4

9.5.1 Overview

Port 4 is an eight-bit input/output port with the pin configuration shown in figure 9-8. In the expanded modes without on-chip ROM (modes 1 and 3), port 4 is used for output of bits A15 to A8 of the address bus. In the single-chip mode (mode 7) port 4 is a general-purpose input/output port which can also receive interrupt signals $\overline{\text{IRQ}}_7$ to $\overline{\text{IRQ}}_4$. In the expanded modes with on-chip ROM (modes 2 and 4), the pins of port 4 function either for output of bits A15 – A8 of the address bus, or for general-purpose input and/or output of $\overline{\text{IRQ}}_7$ to $\overline{\text{IRQ}}_4$.

Port 4 has built-in MOS pull-ups that can be turned on or off under program control.

Outputs from port 4 can drive one TTL load and a 90-pF capacitive load. They can also drive a Darlington transistor pair.

Pin		Modes 1 and 3	Modes 2 and 4
Port 4	↔ P47 / A15 / $\overline{\text{IRQ}}_7$	A15 (output)	P47 (input) / A15 (output) / $\overline{\text{IRQ}}_7$ (input)
	↔ P46 / A14 / $\overline{\text{IRQ}}_6$	A14 (output)	P46 (input) / A14 (output) / $\overline{\text{IRQ}}_6$ (input)
	↔ P45 / A13 / $\overline{\text{IRQ}}_5$	A13 (output)	P45 (input) / A13 (output) / $\overline{\text{IRQ}}_5$ (input)
	↔ P44 / A12 / $\overline{\text{IRQ}}_4$	A12 (output)	P44 (input) / A12 (output) / $\overline{\text{IRQ}}_4$ (input)
	↔ P43 / A11	A11 (output)	P43 (input) / A11 (output)
	↔ P42 / A10	A10 (output)	P42 (input) / A10 (output)
	↔ P41 / A9	A9 (output)	P41 (input) / A9 (output)
	↔ P40 / A8	A8 (output)	P40 (input) / A8 (output)
Single-Chip Mode			
P47 (input/output) / $\overline{\text{IRQ}}_7$ (input)			
P46 (input/output) / $\overline{\text{IRQ}}_6$ (input)			
P45 (input/output) / $\overline{\text{IRQ}}_5$ (input)			
P44 (input/output) / $\overline{\text{IRQ}}_4$ (input)			
P43 (input/output)			
P42 (input/output)			
P41 (input/output)			
P40 (input/output)			

Figure 9-8 Pin Functions of Port 4

9.5.2 Port 4 Registers

Table 9-10 lists the registers of port 4.

Table 9-10 Port 4 Registers

Name	Abbreviation	Read/Write	Initial Value	Address
Port 4 data direction register	P4DDR	W	H'00*	H'FF85
Port 4 data register	P4DR	R/W	H'00	H'FF87

Note: * Initialized to H'00 in modes 2, 4, and 7. Fixed at H'FF in modes 1 and 3.

1. Port 4 Data Direction Register (P4DDR)—H'FF85

Bit	7	6	5	4	3	2	1	0
	P4 ₇ DDR	P4 ₆ DDR	P4 ₅ DDR	P4 ₄ DDR	P4 ₃ DDR	P4 ₂ DDR	P4 ₁ DDR	P4 ₀ DDR
Initial value*	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

Note: * In modes 2, 4, and 7

P4DDR is an 8-bit register that selects the direction of each pin in port 4.

Expanded Modes Not Using On-Chip ROM (Modes 1 and 3): All bits of P4DDR are fixed at 1 and cannot be modified. Port 4 is used for address output.

Expanded Modes Using On-Chip ROM (Modes 2 and 4): If a bit in P4DDR is set to 1, the corresponding pin is used for address output. If a bit in P4DDR is cleared to 0, the pin is used for general-purpose input. P4DDR is initialized to H'00 at a reset and in the hardware standby mode.

Single-Chip Mode: A pin functions as an output pin if the corresponding bit in P4DDR is set to 1, and as an input pin if the bit is cleared to 0.

P4DDR can be written but not read. An attempt to read this register does not cause an error, but all bits are read as 1, regardless of their true values.

At a reset and in the hardware standby mode, P4DDR is initialized to H'00, making all eight pins input pins. P4DDR is not initialized in the software standby mode, so if a P4DDR bit is set to 1 when the chip enters the software standby mode, the corresponding pin continues to output the value in the port 4 data register.

2. Port 4 Data Register (P4DR)—H'FF87

Bit	7	6	5	4	3	2	1	0
	P4 ₇	P4 ₆	P4 ₅	P4 ₄	P4 ₃	P4 ₂	P4 ₁	P4 ₀
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P4DR is an 8-bit register containing output data for pins P4₇ to P4₀.

At a reset and in the hardware standby mode, P4DR is initialized to H'00.

When port 4 is read, output pins return the value in the P4DR latch, regardless of the actual level at the pin. Input pins return the level at the pin, not the value in the P4DR latch.

If any of the port 4 data direction bits are cleared to 0, selecting input, use only data transfer (MOV) instructions to write data in P4DR. Do not use arithmetic, logic, or bit manipulation instructions. These instructions read the input pins and may write unintended data in P4DR.

9.5.3 Pin Functions in Each Mode

Port 4 operates in one way in modes 1 and 3, in another way in modes 2 and 4, and in a third way in mode 7. Separate descriptions are given below.

Pin Functions in Modes 1 and 3: In modes 1 and 3 (expanded modes in which the on-chip ROM is not used), all bits of P4DDR are automatically set to 1 for output, and the pins of port 4 carry bits A15 – A8 of the address bus. Figure 9-9 shows the pin functions for modes 1 and 3.

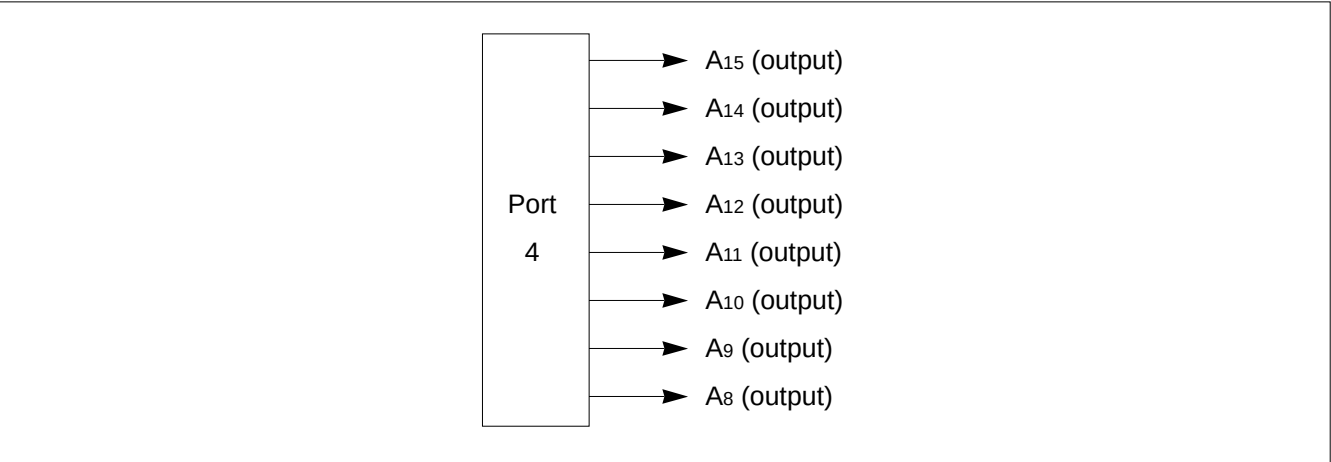


Figure 9-9 Port 4 Pin Functions in Modes 1 and 3

Pin Functions in Modes 2 and 4: Table 9-11 shows the usage of port 4 in modes 2 and 4.

Table 9-11 Port 4 Pin Functions in Modes 2 and 4**Pin Functions**

P4₇ / A₁₅ / $\overline{\text{IRQ}}_7$ The function depends on the IRQ7E bit and P4₇DDR bit as follows:

IRQ7E	0		1	
P4 ₇ DDR	0	1	0	1
Pin function	P4 ₇ input	A ₁₅ output	$\overline{\text{IRQ}}_7$ input and P4 ₇ input	A ₁₅ output

P4₆ / A₁₄ / $\overline{\text{IRQ}}_6$ The function depends on the IRQ6E bit and P4₆DDR bit as follows:

IRQ6E	0		1	
P4 ₆ DDR	0	1	0	1
Pin function	P4 ₆ input	A ₁₄ output	$\overline{\text{IRQ}}_6$ input and P4 ₆ input	A ₁₄ output

P4₅ / A₁₃ / $\overline{\text{IRQ}}_5$ The function depends on the IRQ5E bit and P4₅DDR bit as follows:

IRQ5E	0		1	
P4 ₅ DDR	0	1	0	1
Pin function	P4 ₅ input	A ₁₃ output	$\overline{\text{IRQ}}_5$ input and P4 ₅ input	A ₁₃ output

P4₄ / A₁₂ / $\overline{\text{IRQ}}_4$ The function depends on the IRQ4E bit and P4₄DDR bit as follows:

IRQ4E	0		1	
P4 ₄ DDR	0	1	0	1
Pin function	P4 ₄ input	A ₁₂ output	$\overline{\text{IRQ}}_4$ input and P4 ₄ input	A ₁₂ output

Table 9-11 Port 4 Pin Functions in Modes 2 and 4 (cont)

Pin	Functions		
P4 ₃ / A ₁₁	P4 ₃ DDR	0	1
	Pin function	P4 ₃ input	A ₁₁ output
P4 ₂ / A ₁₀	P4 ₂ DDR	0	1
	Pin function	P4 ₂ input	A ₁₀ output
P4 ₁ / A ₉	P4 ₁ DDR	0	1
	Pin function	P4 ₁ input	A ₉ output
P4 ₀ / A ₈	P4 ₀ DDR	0	1
	Pin function	P4 ₀ input	A ₈ output

Pin Functions in Single-Chip Mode: Table 9-12 shows the usage of port 4 in the single-chip mode.

Table 9-12 Port 4 Pin Functions in Single-Chip Mode

Pin	Functions			
P4 ₇ / $\overline{\text{IRQ}}_7$	The function depends on the IRQ7E bit and P4 ₇ DDR bit as follows:			
	IRQ7E	0		1
	P4 ₇ DDR	0	1	0 1
	Pin function	P4 ₇ input	P4 ₇ output	$\overline{\text{IRQ}}_7$ input and P4 ₇ input $\overline{\text{IRQ}}_7$ input and P4 ₇ output
P4 ₆ / $\overline{\text{IRQ}}_6$	The function depends on the IRQ6E bit and P4 ₆ DDR bit as follows:			
	IRQ6E	0		1
	P4 ₆ DDR	0	1	0 1
	Pin function	P4 ₆ input	P4 ₆ output	$\overline{\text{IRQ}}_6$ input and P4 ₆ input $\overline{\text{IRQ}}_6$ input and P4 ₆ output
P4 ₅ / $\overline{\text{IRQ}}_5$	The function depends on the IRQ5E bit and P4 ₅ DDR bit as follows:			
	IRQ5E	0		1
	P4 ₅ DDR	0	1	0 1
	Pin function	P4 ₅ input	P4 ₅ output	$\overline{\text{IRQ}}_5$ input and P4 ₅ input $\overline{\text{IRQ}}_5$ input and P4 ₅ output
P4 ₄ / $\overline{\text{IRQ}}_4$	The function depends on the IRQ4E bit and P4 ₄ DDR bit as follows:			
	IRQ4E	0		1
	P4 ₄ DDR	0	1	0 1
	Pin function	P4 ₄ input	P4 ₄ output	$\overline{\text{IRQ}}_4$ input and P4 ₄ input $\overline{\text{IRQ}}_4$ input and P4 ₄ output

Table 9-12 Port 4 Pin Functions in Single-Chip Mode (cont)

Pin	Functions		
P4 ₃	P4 ₃ DDR	0	1
	Pin function	P4 ₃ input	P4 ₃ output
P4 ₂	P4 ₂ DDR	0	1
	Pin function	P4 ₂ input	P4 ₂ output
P4 ₁	P4 ₁ DDR	0	1
	Pin function	P4 ₁ input	P4 ₁ output
P4 ₀	P4 ₀ DDR	0	1
	Pin function	P4 ₀ input	P4 ₀ output

9.5.4 Built-In MOS Pull-Up

The MOS input pull-ups of port 4 are turned on by clearing the corresponding bit in P4DDR to 0 and writing a 1 in P4DR. These pull-ups are turned off at a reset and in the hardware standby mode. Table 9-13 indicates the status of the MOS pull-ups in various modes.

Table 9-13 Status of MOS Pull-Ups for Port 4

Mode	Reset	Hardware Standby Mode	Other Operating States*
1	OFF	OFF	OFF
2			ON/OFF
3			OFF
4			ON/OFF
7			ON/OFF

Notes: * Including the software standby mode.

Notation: OFF: The MOS pull-up is always off.

ON/OFF: The MOS pull-up is on when P4DDR = 0 and P4DR = 1, and off otherwise.

Note on Usage of MOS Pull-Ups: See the note in section 9.4.4, “Built-in MOS Pull-up”.

9.6 Port 5

9.6.1 Overview

Port 5 is an eight-bit input/output port with the pin configuration shown in figure 9-10. Its pins also carry input and output signals for the free-running timers (FRT1 and FRT2) and 8-bit timer, and pin 7 can output the system clock (ø).

Port 5 has Schmitt inputs. Outputs from port 5 can drive one TTL load and a 30-pF capacitive load. They can also drive a Darlington transistor pair.

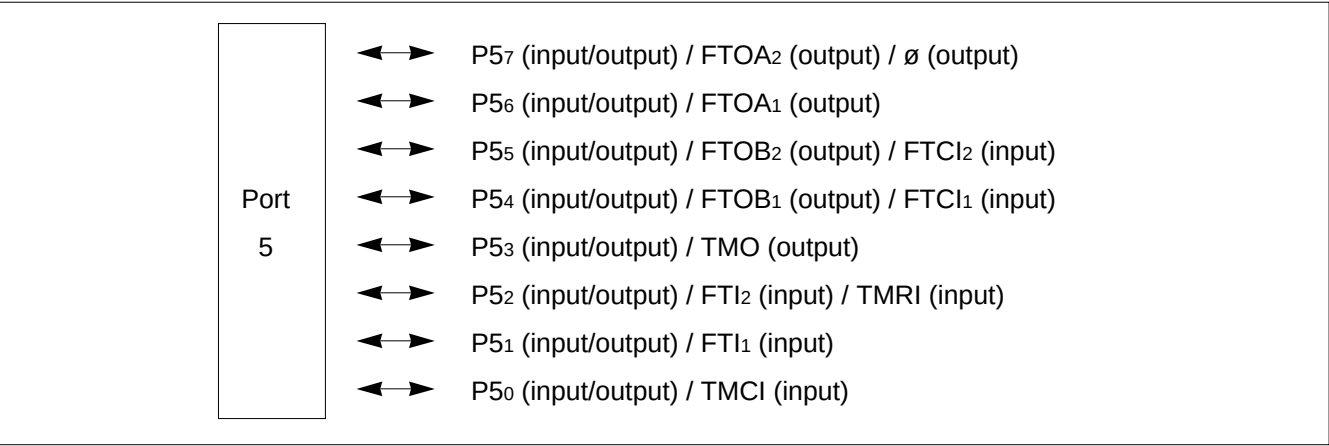


Figure 9-10 Pin Functions of Port 5

9.6.2 Port 5 Registers

Table 9-14 lists the registers of port 5.

Table 9-14 Port 5 Registers

Name	Abbreviation	Read/Write	Initial Value	Address
Port 5 data direction register	P5DDR	W	H'00	H'FF88
Port 5 data register	P5DR	R/W	H'00	H'FF8A

1. Port 5 Data Direction Register (P5DDR)—H'FF88

Bit	7	6	5	4	3	2	1	0
	P57DDR	P56DDR	P55DDR	P54DDR	P53DDR	P52DDR	P51DDR	P50DDR
Initial value	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

P5DDR is an 8-bit register that selects the direction of each pin in port 5. A pin functions as an output pin if the corresponding bit in P5DDR is set to 1, and as an input pin if the bit is cleared to 0.

P5DDR can be written but not read. An attempt to read this register does not cause an error, but all bits are read as 1, regardless of their true values.

At a reset and in the hardware standby mode, P5DDR is initialized to H'00, setting all pins for input. P5DDR is not initialized in the software standby mode, so if a P5DDR bit is set to 1 when the chip enters the software standby mode, the corresponding pin continues to output the value in the port 5 data register.

A transition to the software standby mode initializes the on-chip supporting modules, so any pins of port 5 that were being used by an on-chip timer when the transition occurs revert to general-purpose input or output, controlled by P5DDR and P5DR.

2. Port 5 Data Register (P5DR)—H'FF8A

Bit	7	6	5	4	3	2	1	0
	P57	P56	P55	P54	P53	P52	P51	P50
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P5DR is an 8-bit register containing output data for pins P57 to P50.

P5DR is initialized to H'00 by a reset and in the hardware standby mode.

When port 5 is read, output pins return the value in the P5DR latch, regardless of the actual level at the pin. Input pins return the level at the pin, not the value in the P5DR latch.

If any of the port 5 data direction bits are cleared to 0, selecting input, use only data transfer (MOV) instructions to write data in P5DR. Do not use arithmetic, logic, or bit manipulation instructions. These instructions read the input pins and may write unintended data in P5DR.

9.6.3 Pin Functions

The pin functions of port 5 are the same in all MCU operating modes. As figure 9-10 indicated, these pins are used for input and output of on-chip timer signals as well as for general-purpose input and output. For some pins, two or more functions can be enabled simultaneously.

Table 9-15 shows how the functions of the pins of port 5 are selected.

Table 9-15 Port 5 Pin Functions

Pin	Functions							
P5 ₇ / FTOA ₂ / $\emptyset$	The function depends on the output enable A bit (OEA) of the FRT1 timer control register (TCR), the P57DDR bit, and the system clock output enable bit ($\emptyset$ OE) in the port 7 data direction register, as follows:							
	1				0			
OEA	0		1		0		1	
P57DDR	0	1	0	1	0	1	0	1
Pin function	$\emptyset$ output				P5 ₇ input	P5 ₇ output	FTOA ₂ output	

Note: A reset initializes $\emptyset$ OE to 0 in mode 7 and to 1 in modes 1, 2, 3, and 4.

Table 9-15 Port 5 Pin Functions (cont)

Pin

Functions

P5₆ /
FTOA₂

The function depends on the output enable A bit (OEA) of the FRT2 timer control register (TCR) and on the P5₆DDR bit as follows:

OEB	0		1	
P5 ₆ DDR	0	1	0	1
Pin function	P5 ₆ input	P5 ₆ output	FTOA ₁ output	

P5₅ /
FTOB₂ /
FTCl₂

The function depends on the output enable B bit (OEB) of the FRT2 timer control register (TCR) and on the P5₅DDR bit as follows:

OEB	0		1	
P5 ₅ DDR	0	1	0	1
Pin function	P5 ₅ input	P5 ₅ output	FTOB ₂ output	
	FTCl ₂ input			

P5₄ /
FTOB₁ /
FTCl₁

The function depends on the output enable B bit (OEB) of the FRT1 timer control register (TCR) and on the P5₄DDR bit as follows:

OEB	0		1	
P5 ₄ DDR	0	1	0	1
Pin function	P5 ₄ input	P5 ₄ output	FTOB ₁ output	
	FTCl ₁ input			

Table 9-15 Port 5 Pin Functions (cont)

Pin

Functions

P5₃ / TMO

The function depends on output select bits 3 to 0 (OS3 to OS0) in the timer control/status register (TCSR) of the 8-bit timer, and on the P5₃DDR bit as follows:

OS3 to OS0	All three bits are 0		At least one bit is set to 1	
P5 ₃ DDR	0	1	0	1
Pin function	P5 ₃ input	P5 ₃ output	TMO output	

P5₂ / FTI₂ / TMRI

In addition to functioning for general-purpose input or output, this pin receives the input capture signal (FTI₂) for free-running timer 2 and the reset input (TMRI) for the 8-bit timer. TMRI input is enabled when the counter clear bits (CCLR1 and CCLR0) in the timer control register (TCR) are both set to 1.

P5 ₂ DDR	0	1
Pin function	P5 ₂ input	P5 ₂ output
	FTI ₂ and TMRI input	

P5₁ / FTI₁

P5 ₁ DDR	0	1
Pin function	P5 ₁ input	P5 ₁ output
	FTI ₁ input	

P5₀ / TMCI

In addition to functioning for general-purpose input or output, this pin can simultaneously be used for external clock input for the 8-bit timer, depending on clock select bits 2 to 0 (CKS2, CKS1, and CKS0) in the timer control register (TCR).

P5 ₀ DDR	0	1
Pin function	P5 ₀ input	P5 ₀ output
	TMCI input	

9.7 Port 6

9.7.1 Overview

Port 6 is a 4-bit input port that also receives inputs for the on-chip A/D converter. The pin functions are the same in all MCU operating modes, as shown in figure 9-11.

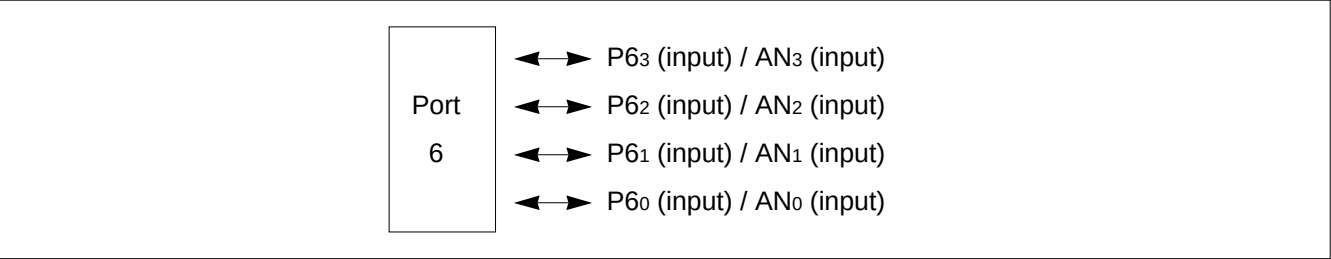


Figure 9-11 Pin Functions of Port 6

In the 68-pin CP-68 package, port 6 has eight pins for general-purpose input and analog input. Figure 9-12 shows the pin configuration of port 6 in the CP-68 package.

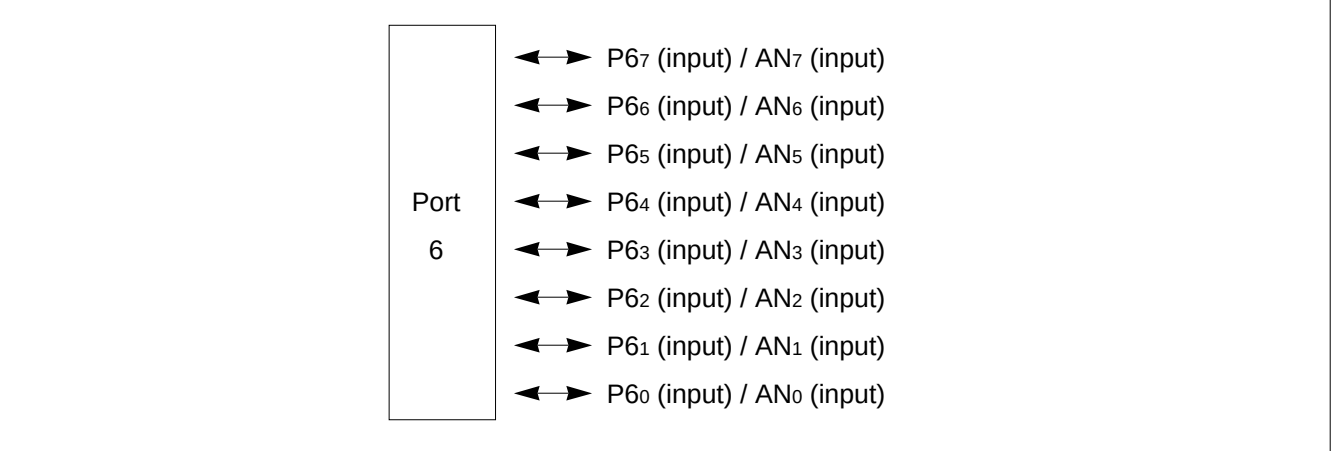


Figure 9-12 Pin Functions of Port 6 (CP-68 Package)

9.7.2 Port 6 Registers

Port 6 has only the data register described in table 9-16. Since it is exclusively an input port, there is no data direction register.

Table 9-16 Port 6 Registers

Name	Abbreviation	Read/Write	Address
Port 6 data register	P6DR	R	H'FF8B

1. Port 6 Data Register (P6DR)—H'FF8B

Bit	7	6	5	4	3	2	1	0
	P67	P66	P65	P64	P63	P62	P61	P60
Read/Write	R	R	R	R	R	R	R	R

Note: Bits 7 to 4 are valid in the CP-68 package only.

When the CPU reads P6DR it always reads the current status of each pin, except that during A/D conversion the pin currently being converted reads 1 regardless of the actual input voltage at that pin.

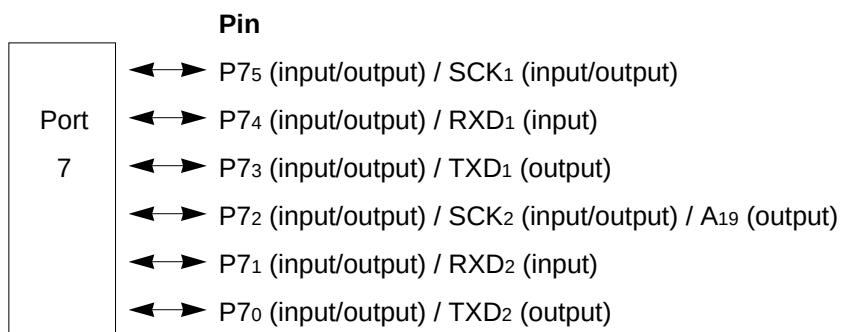
In a 64-pin package, the data read from the upper four bits are indeterminate.

9.8 Port 7

9.8.1 Overview

Port 7 is a 6-bit input/output port with the pin configuration shown in figure 9-13. In addition to general-purpose input and output, its pins are used for input and output by the on-chip serial communication interface (SCI). In the expanded maximum modes (modes 3 and 4), it also supplies bit A₁₉ of the page address bus.

Outputs from port 7 can drive one TTL load and a 30-pF capacitive load. They can also drive a Darlington transistor pair.



Modes 1, 2, and 7

P7₅ (input/output) / SCK₁ (input/output)
 P7₄ (input/output) / RXD₁ (input)
 P7₃ (input/output) / TXD₁ (output)
 P7₂ (input/output) / SCK₂ (input/output)
 P7₁ (input/output) / RXD₂ (input)
 P7₀ (input/output) / TXD₂ (output)

Mode 3

P7₅ (input/output) / SCK₁ (input/output)
 P7₄ (input/output) / RXD₁ (input)
 P7₃ (input/output) / TXD₁ (output)
 A₁₉ (output)
 P7₁ (input/output) / RXD₂ (input)
 P7₀ (input/output) / TXD₂ (output)

Mode 4

P7₅ (input/output) / SCK₁ (input/output)
 P7₄ (input/output) / RXD₁ (input)
 P7₃ (input/output) / TXD₁ (output)
 P7₂ (input/output) / SCK₂ (input/output) / A₁₉ (output)
 P7₁ (input/output) / RXD₂ (input)
 P7₀ (input/output) / TXD₂ (output)

Figure 9-13 Pin Functions of Port 7

9.8.2 Port 7 Registers

Table 9-17 lists the registers of port 7.

Table 9-17 Port 7 Registers

Name	Abbreviation	Read/Write	Initial Value	Address
Port 7 data direction register	P7DDR	W	H'40*	H'FF8C
Port 7 data register	P7DR	R/W	H'00	H'FF8E

Note: * Initialized to H'40 in modes 1, 2, 3, and 4, and to H'00 in mode 7.

1. Port 7 Data Direction Register (P7DDR)—H'FF8C

Bit	7	6	5	4	3	2	1	0
	—	øOE	P75DDR	P74DDR	P73DDR	P72DDR	P71DDR	P70DDR
Initial value	—	1/0	0	0	0	0	0	0
Read/Write	—	W	W	W	W	W	W	W

P7DDR is an 8-bit register that selects the direction of each pin in port 7. Bit 7 is reserved. Bit 6 selects whether the system clock (ø) is output at pin P57 in port 5.

The usage of P7DDR depends on the MCU operating mode as explained below.

Modes 1, 2, and 4: A pin functions as an output pin if the corresponding bit in P7DDR is set to 1, and as an input pin if the bit is cleared to 0.

P7DDR can be written but not read. An attempt to read this register does not cause an error, but all bits are read as 1, regardless of their true values.

At a reset and in the hardware standby mode, P7DDR is initialized to H'40, setting all pins to the input state. P7DDR is not initialized in the software standby mode, so if a P7DDR bit is set to 1 when the chip enters the software standby mode, the corresponding pin continues to output the value in the port 7 data register.

A transition to the software standby mode initializes the serial communication interface module, so any pins of port 7 that were being used for serial communication when the transition occurs revert to general-purpose input or output, controlled by P7DDR and P7DR.

Mode 3: Bit 2 is fixed at the value 1 and pin P7₂ is used for page address output.

Bits 5 to 3 and 1 to 0 can be set to 1 for output or cleared to 0 for input as in the other MCU modes.

Mode 7: In single-chip mode, P7DDR is initialized to H'00 at a reset and in the hardware standby mode.

2. Port 7 Data Register (P7DR)—H'FF8E

Bit	7	6	5	4	3	2	1	0
	—	—	P7 ₅	P7 ₄	P7 ₃	P7 ₂	P7 ₁	P7 ₀
Initial value	—	—	0	0	0	0	0	0
Read/Write	—	—	R/W	R/W	R/W	R/W	R/W	R/W

P7DR is an 8-bit register containing the data for pins P7₆ to P7₀. Bits 7 and 6 are reserved.

When port 7 is read, output pins return the value in the P7DR latch, regardless of the actual level at the pin. Input pins return the level at the pin, not the value in the P7DR latch.

If any of the port 7 data direction bits are cleared to 0, selecting input, use only data transfer (MOV) instructions to write data in P7DR. Do not use arithmetic, logic, or bit manipulation instructions. These instructions read the input pins and may write unintended data in P7DR.

9.8.3 Pin Functions

The pin functions of port 7 depend on the MCU operating mode. Table 9-18 shows how the functions are selected in modes 1, 2, and 7. Table 9-19 shows how they are selected in mode 3. Table 9-20 shows how they are selected in mode 4.

Table 9-18 Port 7 Pin Functions in Modes 1, 2, and 7

Pin	Functions							
P7 ₅ / SCK ₁	The function depends on the communication mode bit ($C/\bar{A}$) and the clock enable 1 and 2 bits (CKE1 and CKE0) of the serial control register (SCR) of SCI1 as follows:							
	$C/\bar{A}$				0			
	CKE1		0		1		1	
	CKE0		0	1	0	1	0	1
Pin function	P7 ₅ input or output*	SCI1 internal clock output	SCI1 external clock input		SCI1 internal clock output		SCI1 external clock input	

Note: * Input or output is selected by the P7₅DDR bit.

P7₄ / RXD₁

The function depends on the receive enable bit (RE) of the serial control register (SCR) of SCI1 and on the P7₄DDR bit as follows:

RE	0		1	
P7 ₄ DDR	0	1	0	1
Pin function	P7 ₄ input	P7 ₄ output	RXD ₁ input	

P7₃ / TXD₁

The function depends on the transmit enable bit (TE) of the serial control register (SCR) of SCI1 and on the P7₃DDR bit as follows:

TE	0		1	
P7 ₃ DDR	0	1	0	1
Pin function	P7 ₃ input	P7 ₃ output	TXD ₁ output	

Table 9-18 Port 7 Pin Functions in Modes 1, 2, and 7 (cont)

Pin	Functions							
P7 ₂ / SCK ₂	The function depends on the communication mode bit (C/A) and the clock enable 1 and 2 bits (CKE1 and CKE0) of the serial control register (SCR) of SCI2 as follows:							
	C/A	0				1		
	CKE1	0		1		0		1
	CKE0	0	1	0	1	0	1	0 1
Pin function	P7 ₂ input or output*	SCI2 internal clock output	SCI2 external clock input		SCI2 internal clock output		SCI2 external clock input	

Note: * Input or output is selected by the P7₂DDR bit.

P7₁ / RXD₂

The function depends on the receive enable bit (RE) of the serial control register (SCR) of SCI2 and on the P7₁DDR bit as follows:

RE	0		1	
P7 ₁ DDR	0	1	0	1
Pin function	P7 ₁ input	P7 ₁ output	RXD ₂ input	

P7₀ / TXD₂

The function depends on the transmit enable bit (TE) of the serial control register (SCR) of SCI2 and on the P7₀DDR bit as follows:

TE	0		1	
P7 ₀ DDR	0	1	0	1
Pin function	P7 ₀ input	P7 ₀ output	TXD ₂ output	

Table 9-19 Port 7 Pin Functions in Mode 3

Pin	Functions
P7 ₅ / SCK ₁	The function depends on the communication mode bit ($C/\bar{A}$) and the clock enable 1 and 2 bits (CKE1 and CKE0) of the serial control register (SCR) of SCI1 as follows:

$C/\bar{A}$	0				1			
CKE1	0		1		0		1	
CKE0	0	1	0	1	0	1	0	1
Pin function	P7 ₅ input or output*	SCI1 internal clock output	SCI1 external clock input		SCI1 internal clock output		SCI1 external clock input	

Note: * Input or output is selected by the P7₅DDR bit.

P7 ₄ / RXD ₁	The function depends on the receive enable bit (RE) of the serial control register (SCR) of SCI1 and on the P7 ₄ DDR bit as follows:
------------------------------------	---

RE	0		1	
P7 ₄ DDR	0	1	0	1
Pin function	P7 ₄ input		P7 ₄ output	RXD ₁ input

Table 9-19 Port 7 Pin Functions in Mode 3 (cont)

Pin	Functions															
P7 ₃ / TXD ₁	The function depends on the transmit enable bit (TE) of the serial control register (SCR) of SCI1 and on the P7₃DDR bit as follows: <table><tr><td>TE</td><td colspan="2">0</td><td colspan="2">1</td></tr><tr><td>P7₃DDR</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>Pin function</td><td>P7₃ input</td><td>P7₃ output</td><td colspan="2">TXD₁ output</td></tr></table>	TE	0		1		P7 ₃ DDR	0	1	0	1	Pin function	P7 ₃ input	P7 ₃ output	TXD ₁ output	
TE	0		1													
P7 ₃ DDR	0	1	0	1												
Pin function	P7 ₃ input	P7 ₃ output	TXD ₁ output													
A ₁₉	A ₁₉ page address output.															
P7 ₁ / RXD ₂	The function depends on the receive enable bit (RE) of the serial control register (SCR) of SCI2 and on the P7₁DDR bit as follows: <table><tr><td>RE</td><td colspan="2">0</td><td colspan="2">1</td></tr><tr><td>P7₁DDR</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>Pin function</td><td>P7₁ input</td><td>P7₁ output</td><td colspan="2">RXD₂ input</td></tr></table>	RE	0		1		P7 ₁ DDR	0	1	0	1	Pin function	P7 ₁ input	P7 ₁ output	RXD ₂ input	
RE	0		1													
P7 ₁ DDR	0	1	0	1												
Pin function	P7 ₁ input	P7 ₁ output	RXD ₂ input													
P7 ₀ / TXD ₂	The function depends on the transmit enable bit (TE) of the serial control register (SCR) of SCI2 and on the P7₀DDR bit as follows: <table><tr><td>TE</td><td colspan="2">0</td><td colspan="2">1</td></tr><tr><td>P7₀DDR</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>Pin function</td><td>P7₀ input</td><td>P7₀ output</td><td colspan="2">TXD₂ output</td></tr></table>	TE	0		1		P7 ₀ DDR	0	1	0	1	Pin function	P7 ₀ input	P7 ₀ output	TXD ₂ output	
TE	0		1													
P7 ₀ DDR	0	1	0	1												
Pin function	P7 ₀ input	P7 ₀ output	TXD ₂ output													

Table 9-20 Port 7 Pin Functions in Mode 4

Pin	Functions
P7 ₅ / SCK ₁	The function depends on the communication mode bit ($C/\bar{A}$) and the clock enable 1 and 2 bits (CKE1 and CKE0) of the serial control register (SCR) of SCI1 as follows:

$C/\bar{A}$	0				1			
CKE1	0		1		0		1	
CKE0	0	1	0	1	0	1	0	1
Pin function	P7 ₅ input or output*	SCI1 internal clock output	SCI1 external clock input		SCI1 internal clock output		SCI1 external clock input	

Note: * Input or output is selected by the P7₅DDR bit.

P7 ₄ / RXD ₁	The function depends on the receive enable bit (RE) of the serial control register (SCR) of SCI1 and on the P7 ₄ DDR bit as follows:
------------------------------------	---

RE	0		1	
P7 ₄ DDR	0	1	0	1
Pin function	P7 ₄ input		P7 ₄ output	RXD ₁ input

Table 9-20 Port 7 Pin Functions in Mode 4 (cont)

Pin

Functions

P7₃ / TXD₁

The function depends on the transmit enable bit (TE) of the serial control register (SCR) of SCI1 and on the P7₃DDR bit as follows:

TE	0		1	
P7 ₃ DDR	0	1	0	1
Pin function	P7 ₃ input		P7 ₃ output	
			TXD ₁ output	

P7₂ / A₁₉ / SCK₂

The function depends on the C/A, CKE1, and CKE0 bits of the serial control register (SCR) of SCI2 and on P7₂DDR as follows:

P7 ₂ DDR	0			
C/ $\bar{A}$	0			
CKE1	0		1	
CKE0	0	1	0	1
Pin function	P7 ₂ input	SCI2 internal clock output	SCI2 external clock input	

P7 ₂ DDR	0				1	
C/ $\bar{A}$	1				Don't care	
CKE1	0		1		Don't care	
CKE0	0	1	0	1	Don't care	
Pin function	SCI2 internal clock output		SCI2 external clock input		A ₁₉ output	

Table 9-20 Port 7 Pin Functions in Mode 4 (cont)

Pin	Functions															
P7 ₁ / RXD ₂	The function depends on the receive enable bit (RE) of the serial control register (SCR) of SCI2 and on the P7₁DDR bit as follows: <table><tr><td>RE</td><td colspan="2">0</td><td colspan="2">1</td></tr><tr><td>P7₁DDR</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>Pin function</td><td>P7₁ input</td><td>P7₁ output</td><td colspan="2">RXD₂ input</td></tr></table>	RE	0		1		P7 ₁ DDR	0	1	0	1	Pin function	P7 ₁ input	P7 ₁ output	RXD ₂ input	
RE	0		1													
P7 ₁ DDR	0	1	0	1												
Pin function	P7 ₁ input	P7 ₁ output	RXD ₂ input													
P7 ₀ / TXD ₂	The function depends on the transmit enable bit (TE) of the serial control register (SCR) of SCI2 and on the P7₀DDR bit as follows: <table><tr><td>TE</td><td colspan="2">0</td><td colspan="2">1</td></tr><tr><td>P7₀DDR</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>Pin function</td><td>P7₀ input</td><td>P7₀ output</td><td colspan="2">TXD₂ output</td></tr></table>	TE	0		1		P7 ₀ DDR	0	1	0	1	Pin function	P7 ₀ input	P7 ₀ output	TXD ₂ output	
TE	0		1													
P7 ₀ DDR	0	1	0	1												
Pin function	P7 ₀ input	P7 ₀ output	TXD ₂ output													

Section 10 16-Bit Free-Running Timers

10.1 Overview

The H8/520 has an on-chip 16-bit free-running timer (FRT) module with two independent channels (FRT1 and FRT2). Both channels are functionally identical.

Each channel has a 16-bit free-running counter that it uses as a time base. Applications of the FRT module include rectangular-wave output (up to two independent waveforms per channel), input pulse width measurement, and measurement of external clock periods.

10.1.1 Features

The features of the free-running timer module are listed below.

- Selection of four clock sources
The free-running counters can be driven by an internal clock source ($\emptyset/4$, $\emptyset/8$, or $\emptyset/32$), or an external clock input (enabling use as an external event counter).
- Two independent comparators
Each free-running timer channel can generate two independent waveforms.
- Input capture function
The current count can be captured on the rising or falling edge (selectable) of an input signal.
- Four types of interrupts
Compare-match A and B, input capture, and overflow interrupts can be requested independently. The compare-match and input capture interrupts can be served by the data transfer controller (DTC), enabling interrupt-driven data transfer with minimal CPU programming.
- Counter can be cleared under program control
The free-running counters can be cleared on compare-match A.

10.1.2 Block Diagram

Figure 10-1 shows a block diagram of one free-running timer channel.

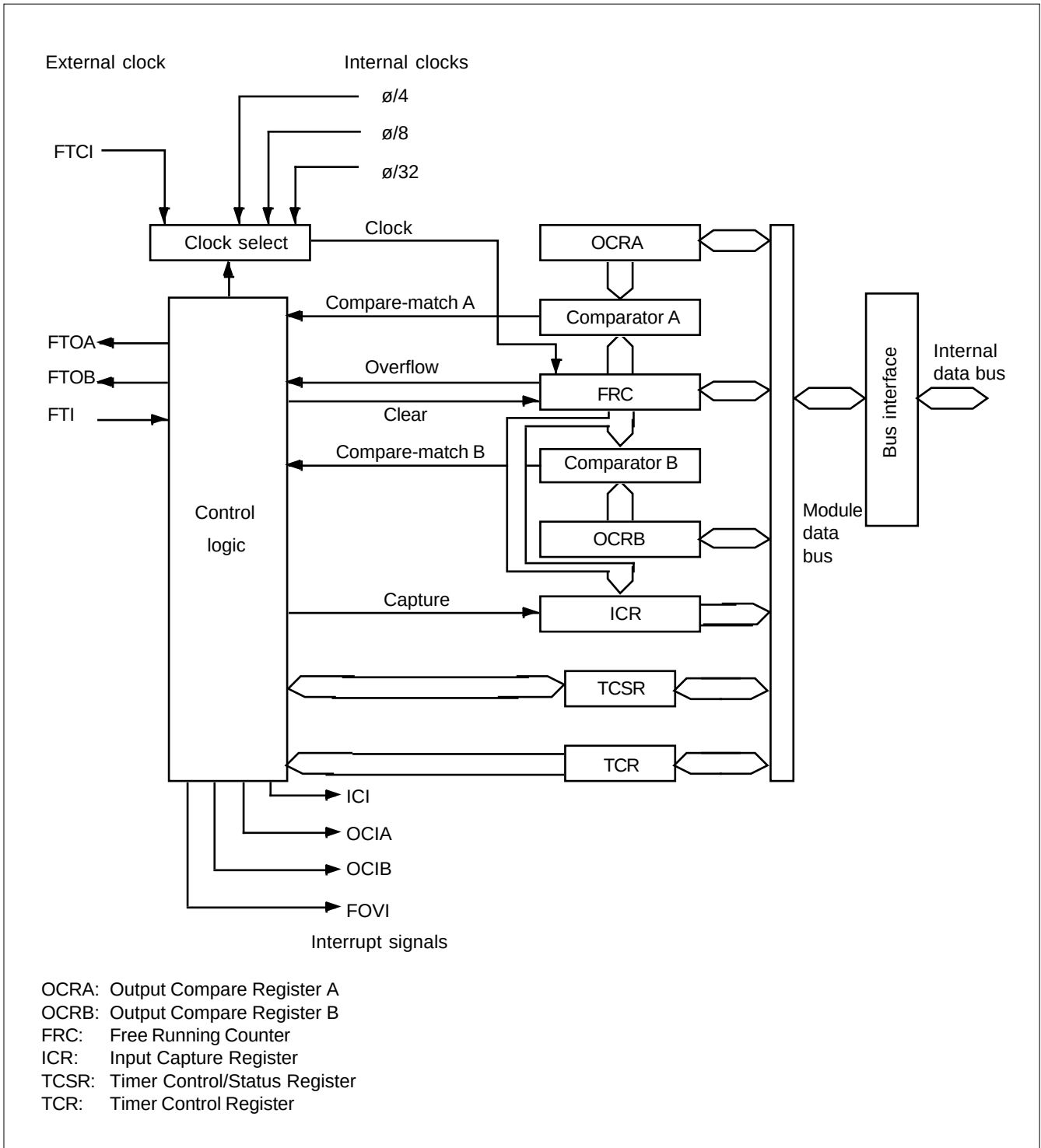


Figure 10-1 Block Diagram of 16-Bit Free-Running Timer

10.1.3 Input and Output Pins

Table 10-1 lists the input and output pins of the free-running timer module.

Table 10-1 Input and Output Pins of Free-Running Timer Module

Channel	Name	Abbreviation	I/O	Function
1	Output compare A	FTOA ₁	Output	Output controlled by comparator A of FRT1
	Output compare B or	FTOB ₁ /	Output /	Output controlled by comparator B of
	counter clock input	FTCl ₁	Input	FRT1, or input of external clock source for FRT1
	Input capture	FTI ₁	Input	Trigger for capturing current count of FRT1
2	Output compare A*	FTOA ₂	Output	Output controlled by comparator A of FRT2
	Output compare B or	FTOB ₂ /	Output /	Output controlled by comparator B of FRT2, or
	counter clock input	FTCl ₂	Input	input of external clock source for FRT2
	Input capture	FTI ₂	Input	Trigger for capturing current count of FRT2

Note: * When the $\emptyset$ OE bit in P7DDR is set to 1, this pin is used for system clock ($\emptyset$) output and cannot be used for FTOA₂.

10.1.4 Register Configuration

Table 10-2 lists the registers of each free-running timer channel.

Table 10-2 Register Configuration

Channel	Name	Abbreviation	R/W	Initial Value	Address
1	Timer control register	TCR	R/W	H'00	H'FF90
	Timer control/status register	TCSR	R/(W)*	H'00	H'FF91
	Free-running counter (High)	FRC (H)	R/W	H'00	H'FF92
	Free-running counter (Low)	FRC (L)	R/W	H'00	H'FF93
	Output compare register A (High)	OCRA (H)	R/W	H'FF	H'FF94
	Output compare register A (Low)	OCRA (L)	R/W	H'FF	H'FF95
	Output compare register B (High)	OCRB (H)	R/W	H'FF	H'FF96
	Output compare register B (Low)	OCRB (L)	R/W	H'FF	H'FF97
	Input capture register (High)	ICR (H)	R	H'00	H'FF98
	Input capture register (Low)	ICR (L)	R	H'00	H'FF99
2	Timer control register	TCR	R/W	H'00	H'FFA0
	Timer control/status register	TCSR	R/(W)*	H'00	H'FFA1
	Free-running counter (High)	FRC (H)	R/W	H'00	H'FFA2
	Free-running counter (Low)	FRC (L)	R/W	H'00	H'FFA3
	Output compare register A (High)	OCRA (H)	R/W	H'FF	H'FFA4
	Output compare register A (Low)	OCRA (L)	R/W	H'FF	H'FFA5
	Output compare register B (High)	OCRB (H)	R/W	H'FF	H'FFA6
	Output compare register B (Low)	OCRB (L)	R/W	H'FF	H'FFA7
	Input capture register (High)	ICR (H)	R	H'00	H'FFA8
	Input capture register (Low)	ICR (L)	R	H'00	H'FFA9

Note: * Software can write a 0 to clear bits 7 to 4, but cannot write a 1 in these bits.

10.2 Register Descriptions

10.2.1 Free-Running Counter (FRC)—H'FF92, H'FFA2

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Each FRC is a 16-bit readable/writable up-counter that increments on an internal pulse generated from a clock source. The clock source is selected by the clock select 1 and 0 bits (CKS1 and CKS0) of the timer control register (TCR).

The FRC can be cleared by compare-match A.

When the FRC overflows from H'FFFF to H'0000, the overflow flag (OVF) in the timer control/status register (TCSR) is set to 1.

Because the FRC is a 16-bit register, a temporary register (TEMP) is used when the FRC is written or read. See section 10.3, “CPU Interface”, for details.

The FRCs are initialized to H'0000 at a reset and in the standby modes.

10.2.2 Output Compare Registers A and B (OCRA and OCRB)—H'FF94 and H'FF96, H'FFA4 and H'FFA6

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

OCRA and OCRB are 16-bit readable/writable registers, the contents of which are continually compared with the value in the FRC. When a match is detected, the corresponding output compare flag (OCFA or OCFB) is set in the timer control/status register (TCSR).

In addition, if the output enable bit (OEA or OEB) in the timer control register (TCR) is set to 1, when the output compare register and FRC values match, the logic level selected by the output level bit (OLVLA or OLVLB) in the timer control status register (TCSR) is output at the output compare pin (FTOA or FTOB).

Because OCRA and OCRB are 16-bit registers, a temporary register (TEMP) is used when they are written. See section 10.3, “CPU Interface”, for details.

OCRA and OCRB are initialized to H'FFFF at a reset and in the standby modes.

10.2.3 Input Capture Register (ICR)—H'FF98, H'FFA8

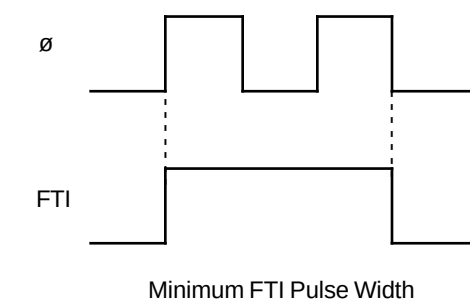
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R

The ICR is a 16-bit read-only register.

When the rising or falling edge of the signal at the input capture input pin is detected, the current value of the FRC is copied to the ICR. At the same time, the input capture flag (ICF) in the timer control/status register (TCSR) is set to 1. The input capture edge is selected by the input edge select bit (IEDG) in the TCSR.

Because the ICR is a 16-bit register, a temporary register (TEMP) is used when the ICR is written or read. See section 10.3, “CPU Interface”, for details.

To ensure input capture, the pulse width of the input capture signal should be at least 1.5 system clock periods (1.5 ϕ).



The ICR is initialized to H'0000 at a reset and in the standby modes.

Note: When input capture is detected, the FRC value is transferred to the ICR even if the input capture flag (ICF) is already set.

10.2.4 Timer Control Register (TCR)—H'FF90, H'FFA0

Bit	7	6	5	4	3	2	1	0
	ICIE	OCIEB	OCIEA	OVIE	OEB	OEA	CKS1	CKS0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The TCR is an 8-bit readable/writable register that selects the FRC clock source, enables the output compare signals, and enables interrupts.

The TCR is initialized to H'00 at a reset and in the standby modes.

Bit 7—Input Capture Interrupt Enable (ICIE): This bit selects whether to request an input capture interrupt (ICI) when the input capture flag (ICF) in the timer status/control register (TCSR) is set to 1.

Bit 7

ICIE	Description
0	The input capture interrupt request (ICI) is disabled. (Initial value)
1	The input capture interrupt request (ICI) is enabled.

Bit 6—Output Compare Interrupt Enable B (OCIEB): This bit selects whether to request output compare interrupt B (OCIB) when output compare flag B (OCFB) in the timer status/control register (TCSR) is set to 1.

Bit 6

OCIEB	Description
0	Output compare interrupt request B (OCIB) is disabled. (Initial value)
1	Output compare interrupt request B (OCIB) is enabled.

Bit 5—Output Compare Interrupt Enable A (OCIEA): This bit selects whether to request output compare interrupt A (OCIA) when output compare flag A (OCFA) in the timer status/control register (TCSR) is set to 1.

Bit 5

OCIEA	Description
0	Output compare interrupt request A (OCIA) is disabled. (Initial value)
1	Output compare interrupt request A (OCIA) is enabled.

Bit 4—Timer Overflow Interrupt Enable (OVIE): This bit selects whether to request a free-running timer overflow interrupt (FOVI) when the timer overflow flag (OVF) in the timer status/control register (TCSR) is set to 1.

Bit 4

OVIE	Description
0	The free-running timer overflow interrupt request (FOVI) is disabled. (Initial value)
1	The free-running timer overflow interrupt request (FOVI) is enabled.

Bit 3—Output Enable B (OEB): This bit selects whether to enable or disable output of the logic level selected by the OLVLB bit in the timer status/control register (TCSR) at the output compare B pin when the FRC and OCRB values match.

Bit 3

OEB	Description
0	Output compare B output is disabled. (Initial value)
1	Output compare B output is enabled.

Bit 2—Output Enable A (OEA): This bit selects whether to enable or disable output of the logic level selected by the OLVLA bit in the timer status/control register (TCSR) at the output compare A pin when the FRC and OCRA values match.

Bit 2

OEA	Description
0	Output compare A output is disabled. (Initial value)
1	Output compare A output is enabled.

Bits 1 and 0—Clock Select (CKS1 and CKS0): These bits select external clock input or one of three internal clock sources for the FRC. External clock pulses are counted on the rising edge.

Bit 1	Bit 0	Description
CKS1	CKS0	
0	0	Internal clock source ($\phi/4$) (Initial value)
0	1	Internal clock source ($\phi/8$)
1	0	Internal clock source ($\phi/32$)
1	1	External clock source (counted on the rising edge)*

Note: * Output enable B (bit 3) must be cleared to 0.

10.2.5 Timer Control/Status Register (TCSR)—H'FF91, H'FFA1

Bit	7	6	5	4	3	2	1	0
	ICF	OCFB	OCFA	OVF	OLVLB	OLVLA	IEDG	CCLRA
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/(W)*	R/(W)*	R/(W)*	R/(W)*	R/W	R/W	R/W	R/W

The TCSR is an 8-bit readable and partially writable* register that selects the input capture edge and output compare levels, and specifies whether to clear the counter on compare-match A. It also contains four status flags.

The TCSR is initialized to H'00 at a reset and in the standby modes.

Note: * Software can write a 0 in bits 7 to 4 to clear the flags, but cannot write a 1 in these bits.

Bit 7—Input Capture Flag (ICF): This status flag is set to 1 to indicate an input capture event. It signifies that the FRC value has been copied to the ICR.

Bit 7

ICF	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The CPU reads the ICF bit after the ICF bit has been set to 1, then writes a 0 in this bit. 2. The data transfer controller (DTC) serves an input capture interrupt .
1	This bit is set to 1 when an input capture signal causes the FRC value to be copied to the ICR.

Bit 6—Output Compare Flag B (OCFB): This status flag is set to 1 when the FRC value matches the OCRB value.

Bit 6

OCFB	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The CPU reads the OCFB bit after the OCFB bit has been set to 1, then writes a 0 in this bit. 2. The data transfer controller (DTC) serves output compare interrupt B.
1	This bit is set to 1 when FRC = OCRB.

Bit 5—Output Compare Flag A (OCFA): This status flag is set to 1 when the FRC value matches the OCRA value.

Bit 5

OCFA	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The CPU reads the OCFA bit after the OCFA bit has been set to 1, then writes a 0 in this bit. 2. The data transfer controller (DTC) serves output compare interrupt A.
1	This bit is set to 1 when FRC = OCRA.

Bit 4—Timer Overflow Flag (OVF): This status flag is set to 1 when the FRC overflows (changes from H'FFFF to H'0000).

Bit 4

OVF	Description
0	This bit is cleared from 1 to 0 when the CPU reads (Initial value) the OVF bit after the OVF bit has been set to 1, then writes a 0 in this bit.
1	This bit is set to 1 when FRC changes from H'FFFF to H'0000.

Bit 3—Output Level B (OLVLB): This bit selects the logic level to be output at the FTOB pin when the FRC and OCRB values match.

Bit 3

OLVLB	Description
0	A 0 logic level (low) is output for compare-match B. (Initial value)
1	A 1 logic level (high) is output for compare-match B.

Bit 2—Output Level A (OLVLA): This bit selects the logic level to be output at the FTOA pin when the FRC and OCRA values match.

Bit 2

OLVLA	Description
0	A 0 logic level (low) is output for compare-match A. (Initial value)
1	A 1 logic level (high) is output for compare-match A.

Bit 1—Input Edge Select (IEDG): This bit selects whether to capture the count on the rising or falling edge of the input capture signal.

Bit 1

IEDG	Description	
0	The FRC value is copied to the ICR on the falling edge of the input capture signal.	(Initial value)
1	The FRC value is copied to the ICR on the rising edge of the input capture signal.	

Bit 0—Counter Clear A (CCLRA): This bit selects whether to clear the FRC at compare-match A (when the FRC and OCRA values match).

Bit 0

CCLRA	Description	
0	The FRC is not cleared.	(Initial value)
1	The FRC is cleared at compare-match A.	

10.3 CPU Interface

The FRC, OCRA, OCRB, and ICR are 16-bit registers, but they are connected to an 8-bit data bus. When the CPU accesses these four registers, to ensure that both bytes are written or read simultaneously, the access is performed using an 8-bit temporary register (TEMP).

These registers are written and read as follows.

- **Register Write**

When the CPU writes to the upper byte, the upper byte of write data is placed in TEMP. Next, when the CPU writes to the lower byte, this byte of data is combined with the byte in TEMP and all 16 bits are written in the register simultaneously.

- **Register Read**

When the CPU reads the upper byte, the upper byte of data is sent to the CPU and the lower byte is placed in TEMP. When the CPU reads the lower byte, it receives the value in TEMP.

Programs that access these four registers should normally use word access. Equivalently, they may access first the upper byte, then the lower byte. Data will not be transferred correctly if the bytes are accessed in reverse order, or if only one byte is accessed.

The same considerations apply to access by the DTC.

Coding Examples

1. To write the contents of general register R0 to output compare register A in FRT1:

```
MOV.W R0, @H'FF94
```

2. To read the FRT2 input capture register contents into general register R0:

```
MOV.W @H'FFA8, R0
```

Figure 10-2 shows the data flow when the FRC is accessed. The other registers are accessed in the same way, except that when OCRA or OCRB is read, the upper and lower bytes are both transferred directly to the CPU without using the temporary register.

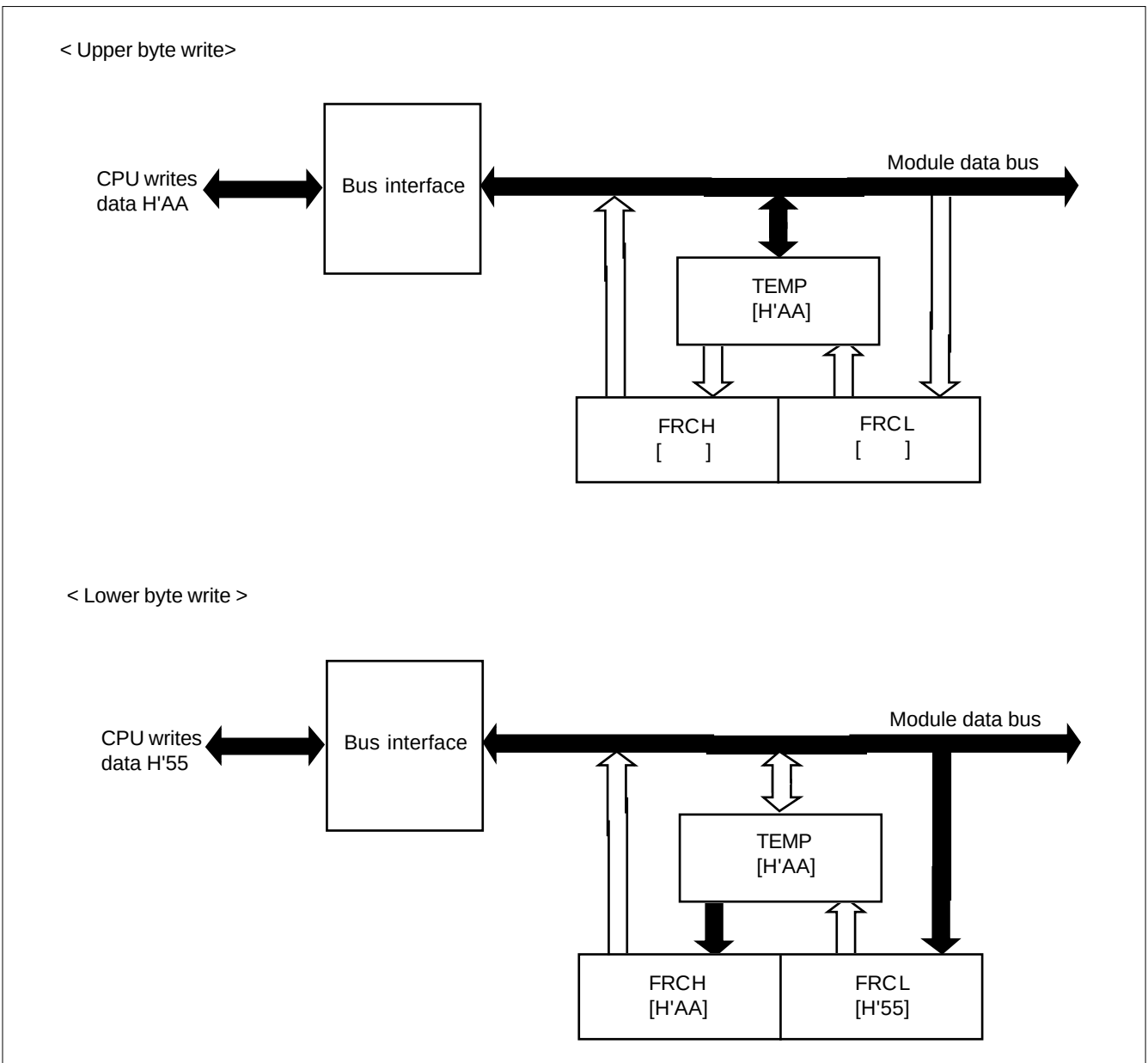
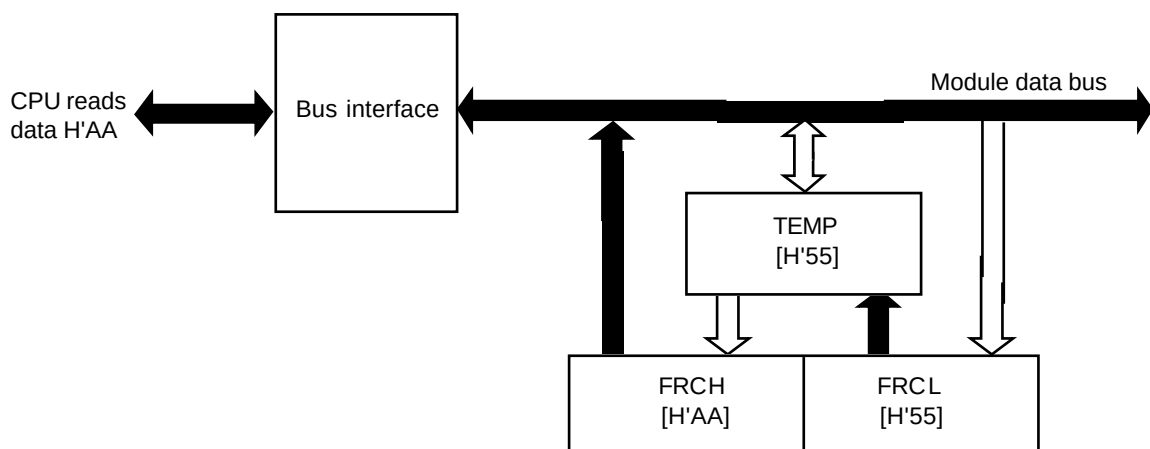


Figure 10-2 (a) Write Access to FRC (When CPU Writes H'AA55)

< Upper byte read >



< Lower byte read >

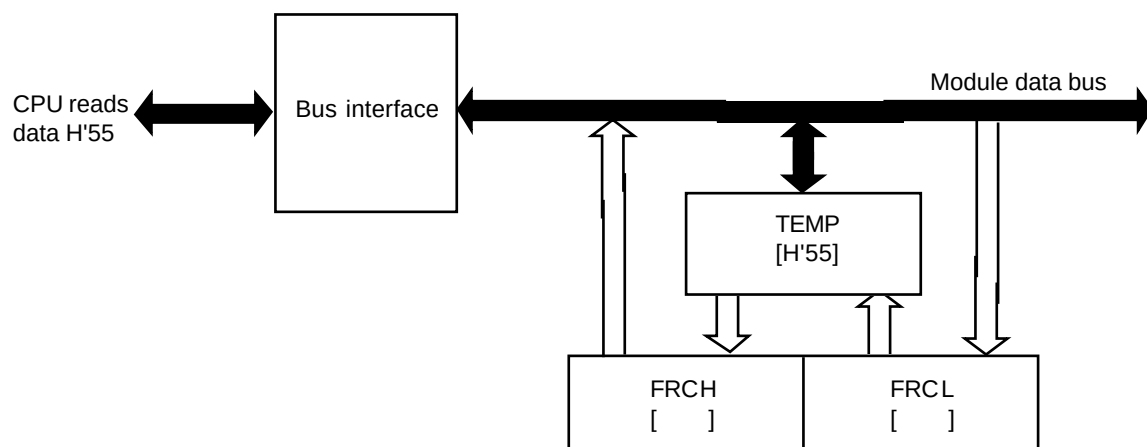


Figure 10-2 (b) Read Access to FRC (When FRC Contains H'AA55)

10.4 Operation

10.4.1 FRC Incrementation Timing

The FRC increments on a pulse generated once for each period of the selected (internal or external) clock source.

If external clock input is selected, the FRC increments on the rising edge of the clock signal. Figure 10-3 shows the increment timing.

The pulse width of the external clock signal must be at least 1.5 ϕ clock periods. The counter will not increment correctly if the pulse width is shorter than 1.5 ϕ clock periods.

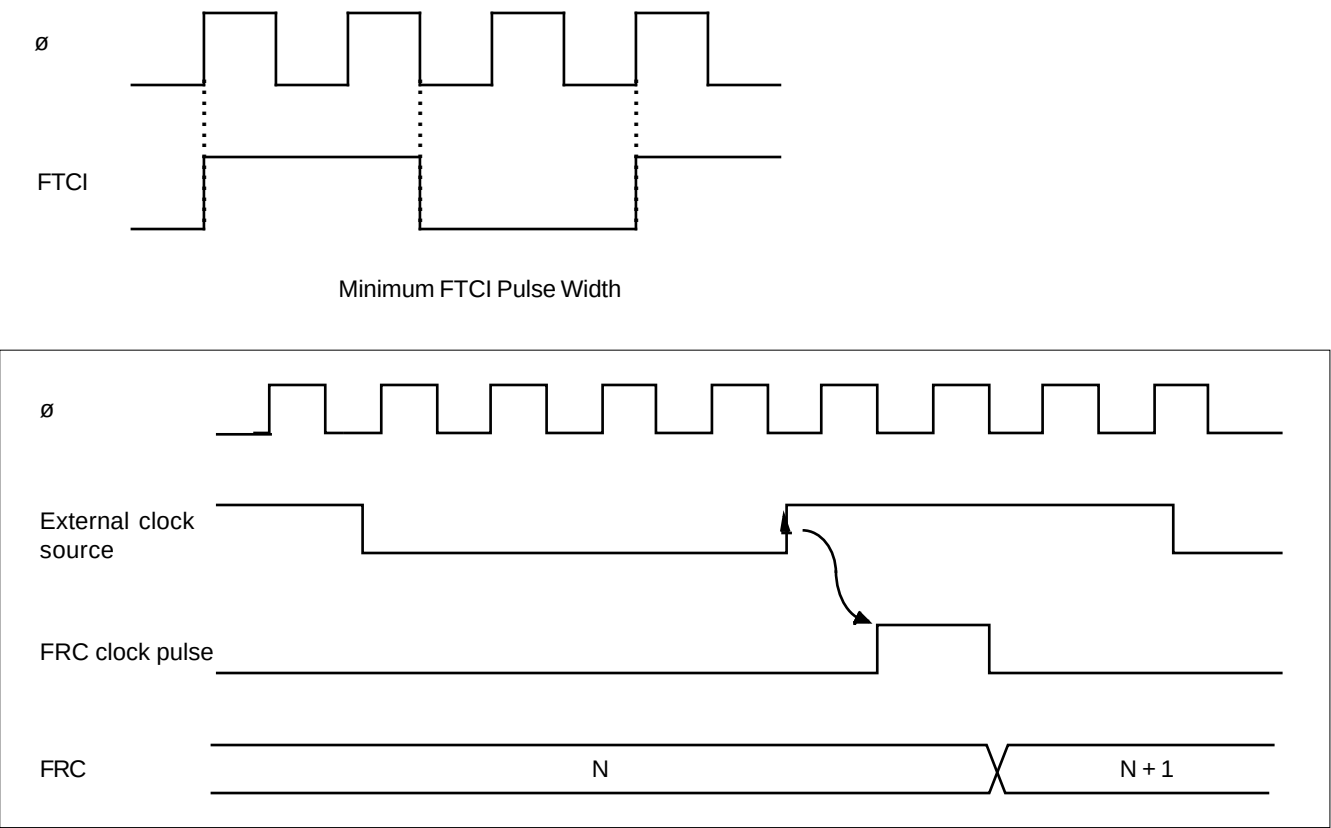


Figure 10-3 Increment Timing for External Clock Input

10.4.2 Output Compare Timing

Setting of Output Compare Flags A and B (OCFA and OCFB): The output compare flags are set to 1 by an internal compare-match signal generated when the FRC value matches the OCRA or OCRB value. This compare-match signal is generated at the last state in which the two values match, just as the FRC increments to a new value.

Accordingly, when the FRC and OCR values match, the compare-match signal is not generated until the next period of the clock source. Figure 10-4 shows the timing of the setting of the output compare flags.

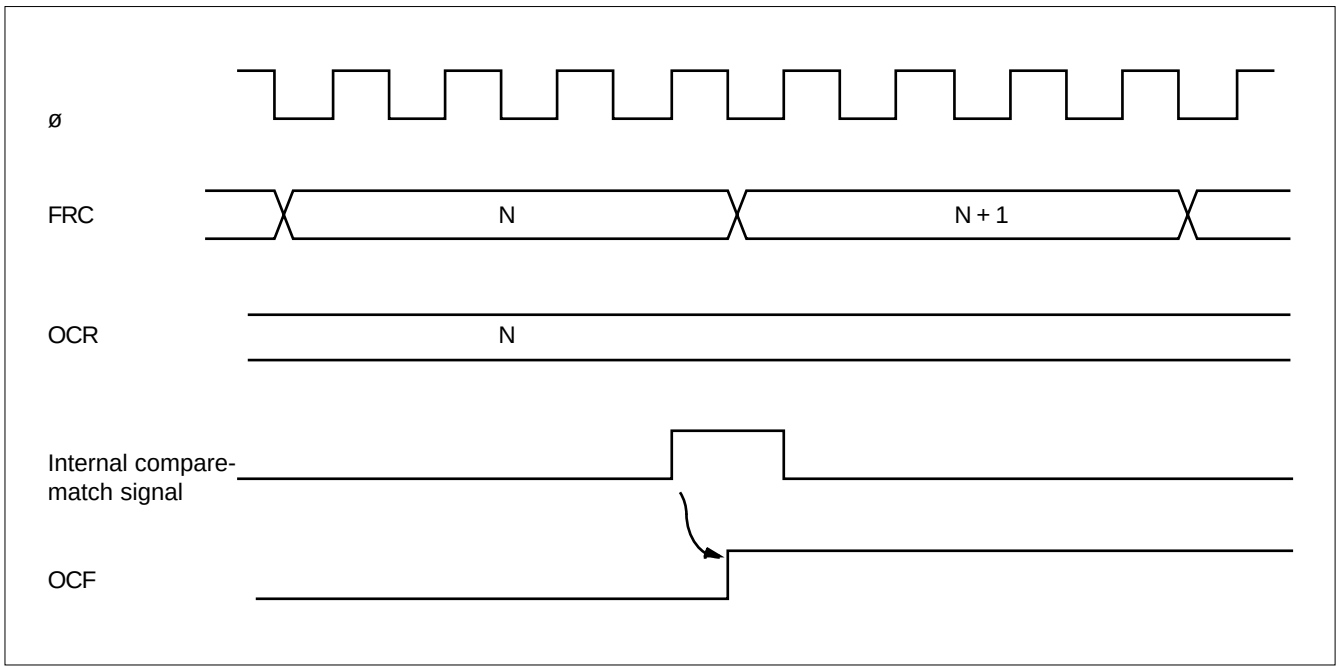


Figure 10-4 Setting of Output Compare Flags

Output Timing: When a compare-match occurs, the logic level selected by the output level bit (OLVLA or OLVLB) in the TCSR is output at the output compare pin (FTOA or FTOB). Figure 10-5 shows the timing of this operation for compare-match A.

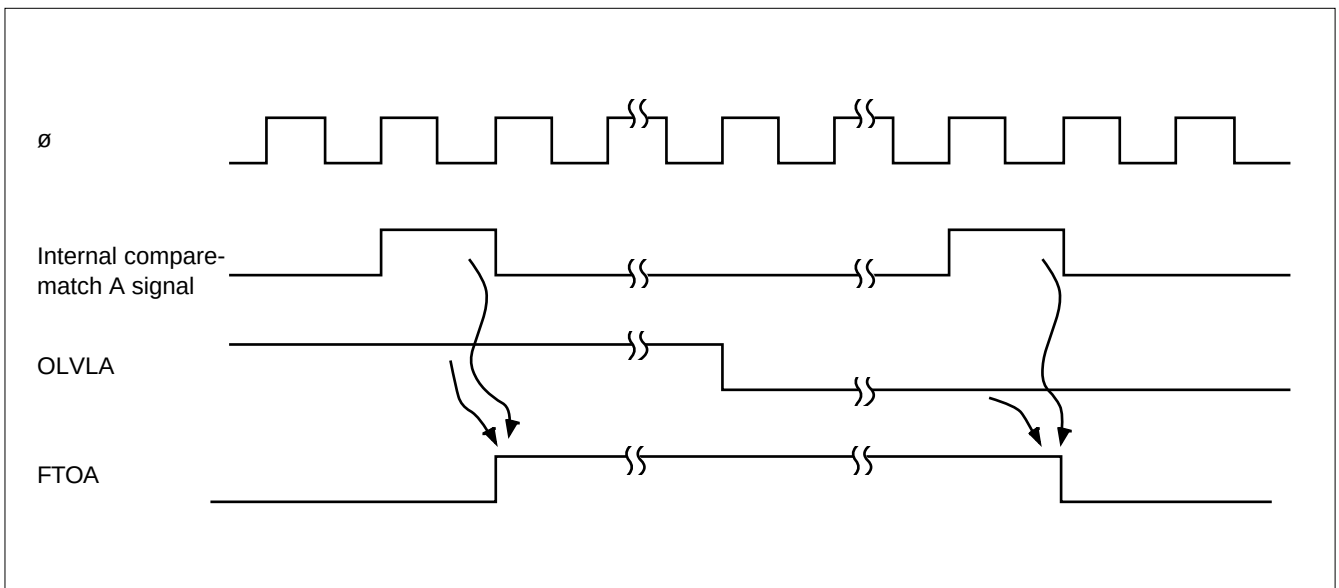


Figure 10-5 Timing of Output Compare A

FRC Clear Timing: If the CCLRA bit is set to 1, the FRC is cleared when compare-match A occurs. Figure 10-6 shows the timing of this operation.

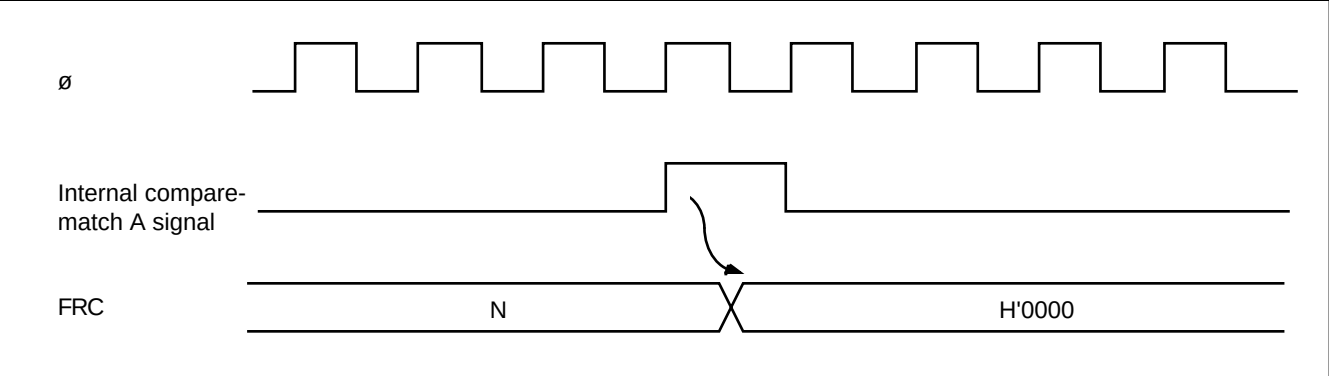


Figure 10-6 Clearing of FRC by Compare-Match A

10.4.3 Input Capture Timing

Input Capture Timing: An internal input capture signal is generated from the rising or falling edge of the input at the input capture pin (FTI), as selected by the IEDG bit in the TCSR. Figure 10-7 shows the usual input capture timing when the rising edge is selected (IEDG = 1).

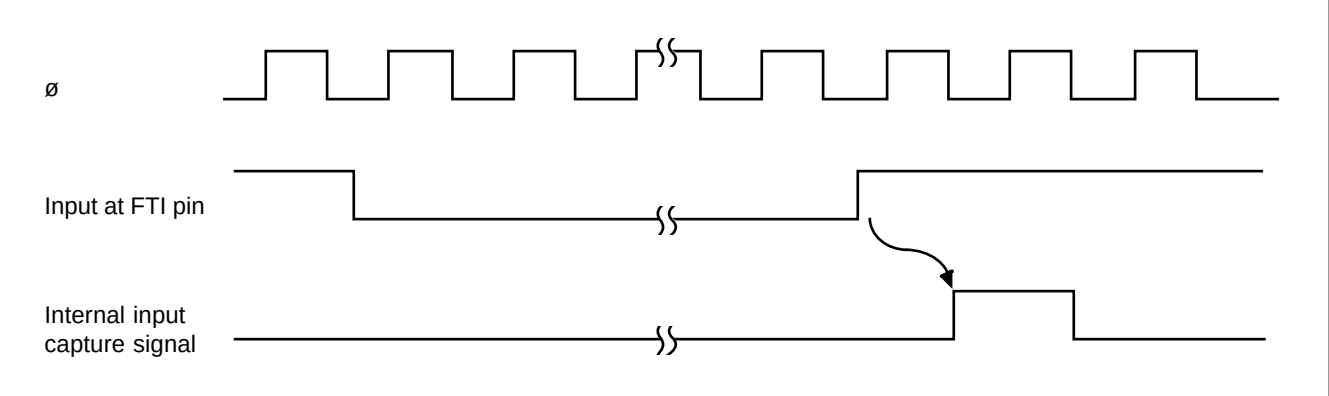


Figure 10-7 Input Capture Timing (Usual Case)

But if the upper byte of the ICR is being read when the input capture signal arrives, the internal input capture signal is delayed by one state. Figure 10-8 shows the timing for this case.

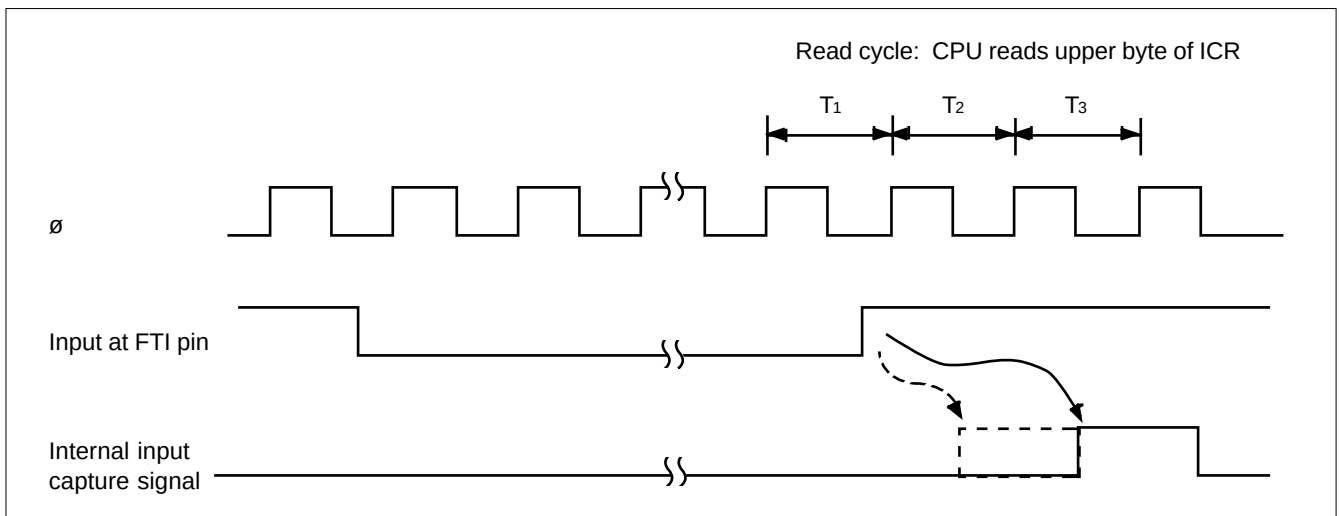


Figure 10-8 Input Capture Timing (1-State Delay)

Timing of Input Capture Flag (ICF) Setting: The input capture flag (ICF) is set to 1 by the internal input capture signal. Figure 10-9 shows the timing of this operation.

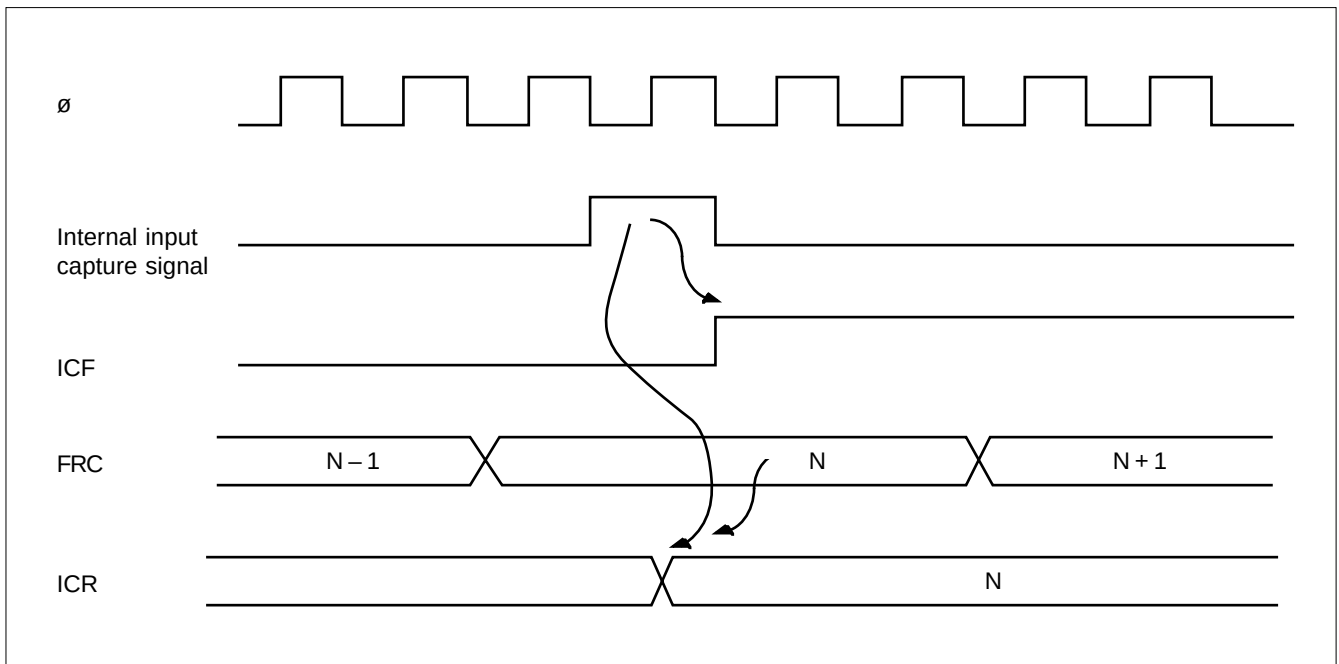


Figure 10-9 Setting of Input Capture Flag

10.4.4 Setting of FRC Overflow Flag (OVF)

The FRC overflow flag (OVF) is set to 1 when the FRC overflows (changes from H'FFFF to H'0000). Figure 10-10 shows the timing of this operation.

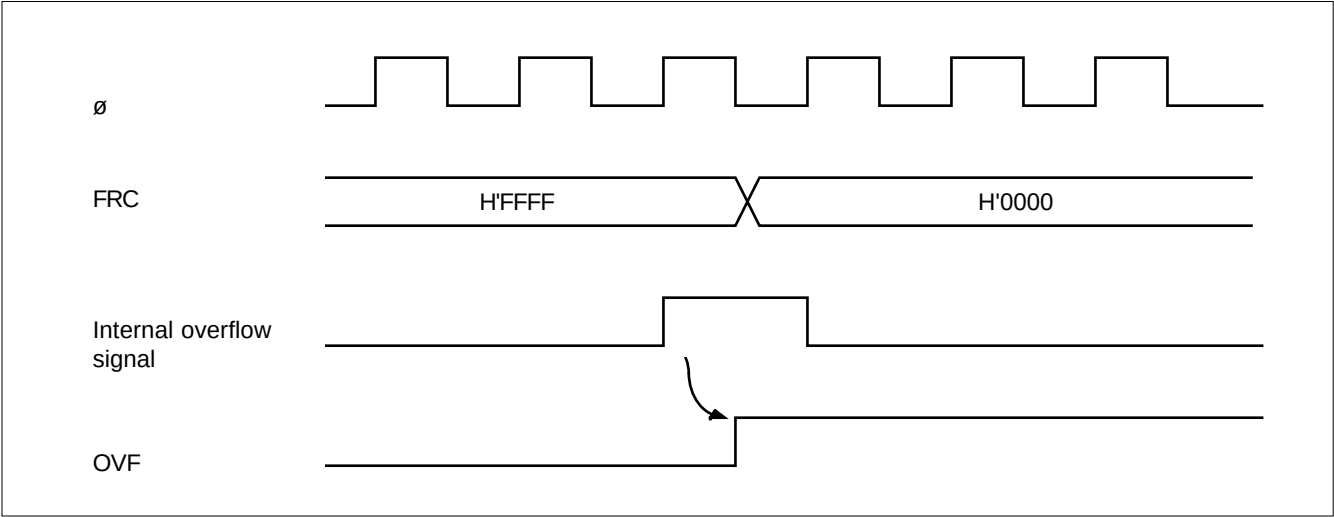


Figure 10-10 Setting of Overflow Flag (OVF)

10.5 CPU Interrupts and DTC Interrupts

Each free-running timer channel can request four types of interrupts: input capture (ICI), output compare A and B (OCIA and OCIB), and overflow (FOVI). Each interrupt is requested when the corresponding enable and flag bits are set. Independent signals are sent to the interrupt controller for each type of interrupt. Table 10-3 lists information about these interrupts.

Table 10-3 Free-Running Timer Interrupts

Interrupt	Description	DTC Service Available?	Priority
ICI	Requested when ICF is set	Yes	High ↑ Low
OCIA	Requested when OCFA is set	Yes	
OCIB	Requested when OCFB is set	Yes	
FOVI	Requested when OVF is set	No	Low

The ICI, OCIA, and OCIB interrupts can be directed to the data transfer controller (DTC) to have a data transfer performed in place of the usual interrupt-handling routine.

When the DTC serves one of these interrupts, it automatically clears the ICF, OCFA, or OCFB flag to 0. See section 6, “Data Transfer Controller”, for further information on the DTC.

10.6 Synchronization of Free-Running Timers 1 and 2

10.6.1 Synchronization after a Reset

The two free-running timer channels are synchronized at a reset and remain synchronized until one of the following conditions is satisfied:

- The clock source is changed.
- FRC contents are rewritten.
- An FRC is cleared.

After a reset, each free-running counter operates on the $\varnothing/4$ internal clock source.

10.6.2 Synchronization by Writing to FRCs

When synchronization between free-running timers 1 and 2 is lost, it can be restored by writing to the free-running counters.

Synchronization on Internal Clock Source: When an internal clock is selected, free-running timers 1 and 2 can be synchronized by writing data to their free-running counters as indicated in table 10-4.

Table 10-4 Synchronization by Writing to FRCs

Clock Source	Write Interval	Write Data	
$\varnothing/4$	4n (states)	m	(FRC1)
$\varnothing/8$	8n (states)	m + n	(FRC2)
$\varnothing/32$	32n (states)		

m, n: Arbitrary integers

After writing these data, synchronization can be checked by reading the two free-running counters at the same interval as the write interval. If the read data have the same relative difference as the write data, the free-running timers are synchronized.

Examples of programs for synchronizing the free-running timers are given next. Examples a, b, and c apply when the program is stored in on-chip memory. Examples d, e, and f apply when the program is stored in external memory which is accessed with zero wait states (T_w), assuming that there is no NMI input.

Example a: $\emptyset/4$ clock source, 12-state write interval ($n = 3$), on-chip memory

```

LA:   LDC.B   #H'FF, BR           ; Initialize base register for short-format instruction (MOV:S)
      LDC.W   #H'0700, SR         ; Raise interrupt mask level to 7
      MOV.W   #m, R1              ; Data for free-running timer 1
      MOV.W   #m+3, R2            ; Data for free-running timer 2 ( $m + n = m + 3$ )
      BSR     SET4                ; Call write routine
      .
      .
      .
      .ALIGN  2                  ; Align write instructions (MOV:S) at even address
SET4: MOV:S.W R1, @H'92:8         ; Write to FRC 1 (address H'FF92) 9 states
      BRN SET4:8                  ; 2-Byte dummy instruction      3 states
      MOV:S.W R2, @H'A2:8         ; Write to FRC2 (address H'FFA2)
      RTS

```

Total 12 states

Example b: $\emptyset/8$ clock source, 16-state write interval ($n = 2$), on-chip memory

```

LB:   LDC.B   #H'FF, BR           ; Initialize base register for short-format instruction (MOV:S)
      LDC.W   #H'0700, SR         ; Raise interrupt mask level to 7
      MOV.W   #m, R1              ; Data for free-running timer 1
      MOV.W   #m+2, R2            ; Data for free-running timer 2 ( $m + n = m + 2$ )
      BSR     SET8                ; Call write routine
      .
      .
      .
      .ALIGN  2                  ; Align write instructions (MOV:S) at even address
SET8: MOV:S.W R1, @H'92:8         ; 9 states
      BRN SET8:8                  ; 3 states
      XCH R1, R1                  ; 4 states
      MOV:S.W R2, @H'A2:8
      RTS

```

Total 16 states

Example c: $\emptyset/32$ clock source, 32-state write interval ($n = 1$), on-chip memory

```

LC:      LDC.B   #H'FF, BR
          LDC.W   #H'0700, SR
          MOV.W   #m, R1
          MOV.W   #m+1, R2
          BSR     SET32
          .
          .
          .
          .ALIGN  2                ; Align on even address
SET32:    MOV:S.W R1, @H'92:8      ; 2 bytes,    9 states
          BSR     WAIT:8          ; 2 bytes,    9 states
          MOV:S.W  R2, @H'A2:8
          RTS

          .ALIGN  2                ; Align on even address
WAIT:     NOP                    ; 2 states
          XCH     R1, R1          ; 4 states
          RTS                    ; 8 states
  
```

Total 32 states

Note: The stack is assumed to be in on-chip RAM.

Example d: $\emptyset/4$ clock source, 20-state write interval ($n = 5$), external memory

```

LD:      LDC.B   #H'FF, BR
          LDC.W   #H'0700, SR      ; Set interrupt mask level to 7
          CLR.B   @H'F8:8         ; Disable wait states
          MOV.W   #m, R1
          MOV.W   #m+5, R2
          MOV:S.W  R1, @H'92:8    ; 13 states
          BRN     LD:8            ; 2 bytes, 7 states
          MOV:S.W  R2, @H'A2:8
  
```

Total 20 states

Example e: $\emptyset/8$ clock source, 24-state write interval ($n = 3$), external memory

LE:	LDC.B	#H'FF, BR			
	LDC.W	#H'0700, SR			
	CLR.B	@H'F8:8			
	MOV.W	#m, R1			
	MOV.W	#m+3, R2			
	MOV:S.W	R1, @H'92:8	; 13 states		
	BRN	LE:8	; 2 bytes,		7 states
	NOP		; 1 byte,		4 states
	MOV:S.W	R2, @H'A2;8			

Example f: $\emptyset/32$ clock source, 32-state write interval ($n = 1$), external memory

LF:	LDC.B	#H'FF, BR		
	LDC.W	#H'0700, SR		
	CLR.B	@H'F8:8		
	MOV.W	#m, R1		
	MOV.W	#m+1, R2		
	MOV:S.W	R1, @H'92:8	; External memory, so	13 states
	XCH	R0, R0	;	8 states
	BRN	LF:8	; 2 bytes,	7 states
	NOP		;	4 states
	MOV:S.W	R2, @H'A2:8		

Synchronization on External Clock Source: When the external clock source is selected, the free-running timers can be synchronized by halting their external clock inputs, then writing identical values in their free-running counters.

10.7 Sample Application

In the example below, one free-running timer channel is used to generate two square-wave outputs with a 50% duty factor and arbitrary phase relationship. The programming is as follows:

1. The CCLRA bit in the TCSR is set to 1.
2. Each time a compare-match interrupt occurs, software inverts the corresponding output level bit in the TCSR.

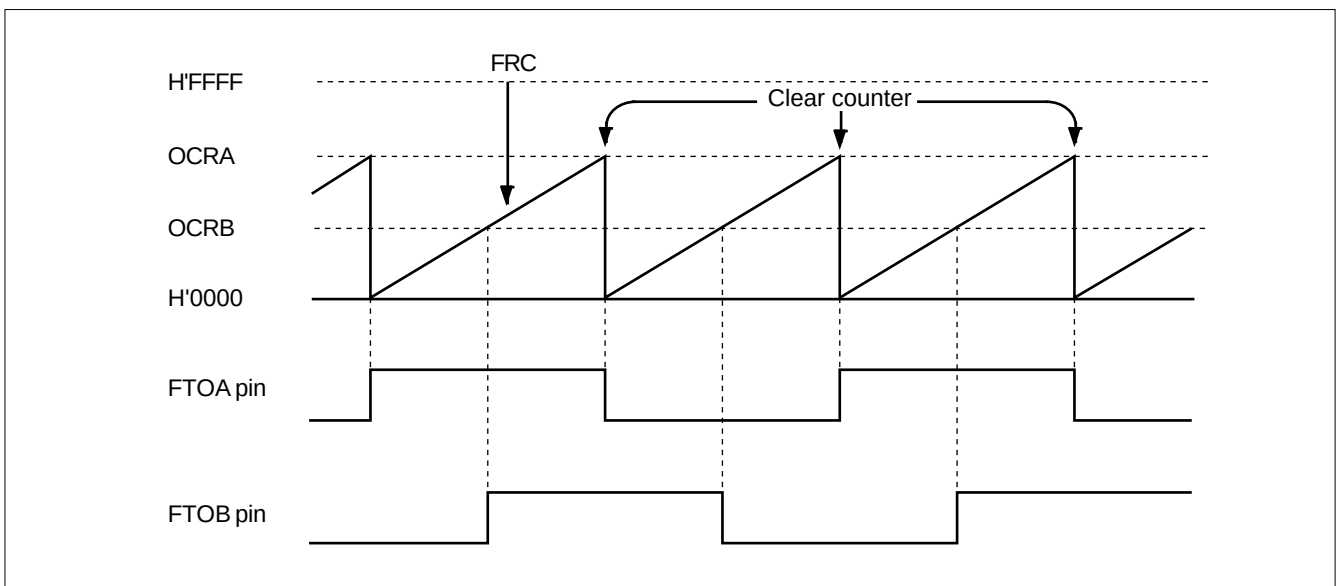


Figure 10-11 Square-Wave Output (Example)

10.8 Application Notes

Application programmers should note that the following types of contention can occur in the free-running timers.

Contention between FRC Write and Clear: If an internal counter clear signal is generated during the T3 state of a write cycle to the lower byte of a free-running counter, the clear signal takes priority and the write is not performed.

Figure 10-12 shows this type of contention.

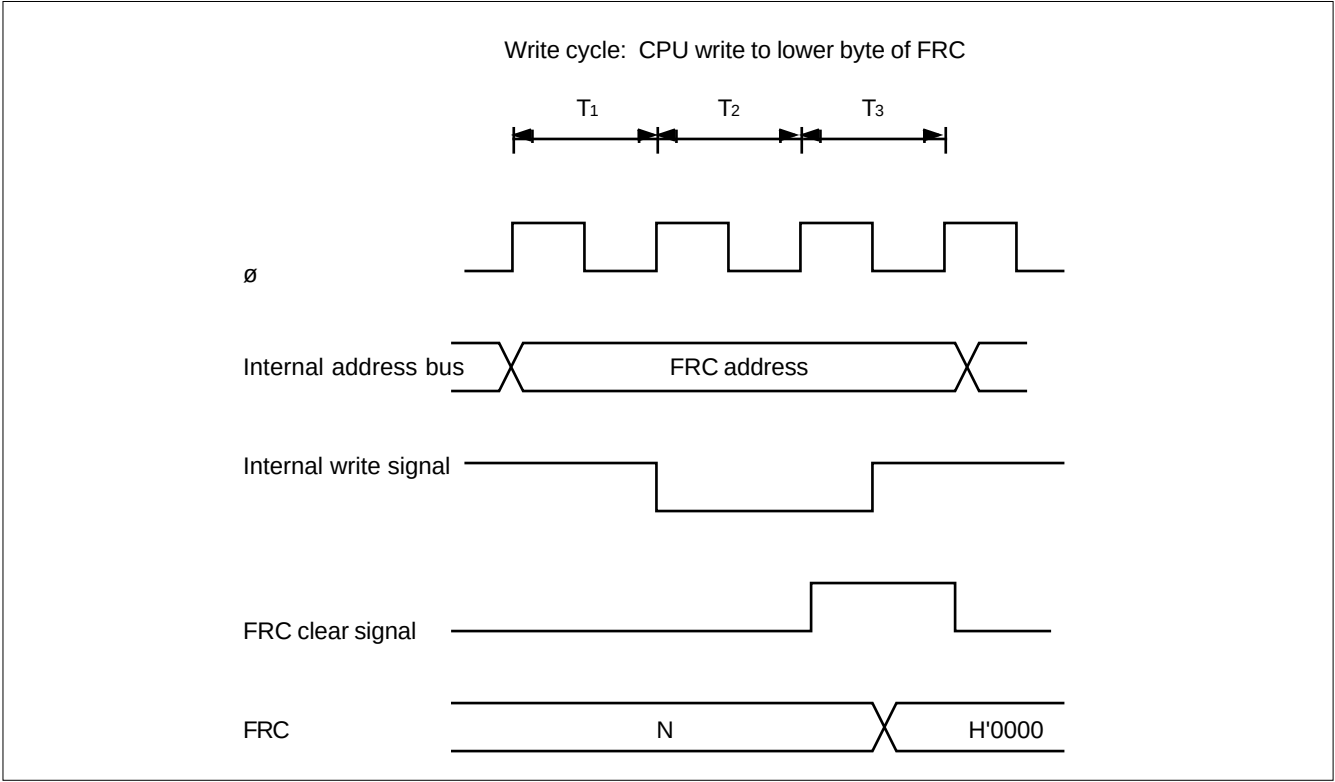


Figure 10-12 FRC Write-Clear Contention

Contention between FRC Write and Increment: If an FRC increment pulse is generated during the T₃ state of a write cycle to the lower byte of a free-running counter, the write takes priority and the FRC is not incremented.

Figure 10-13 shows this type of contention.

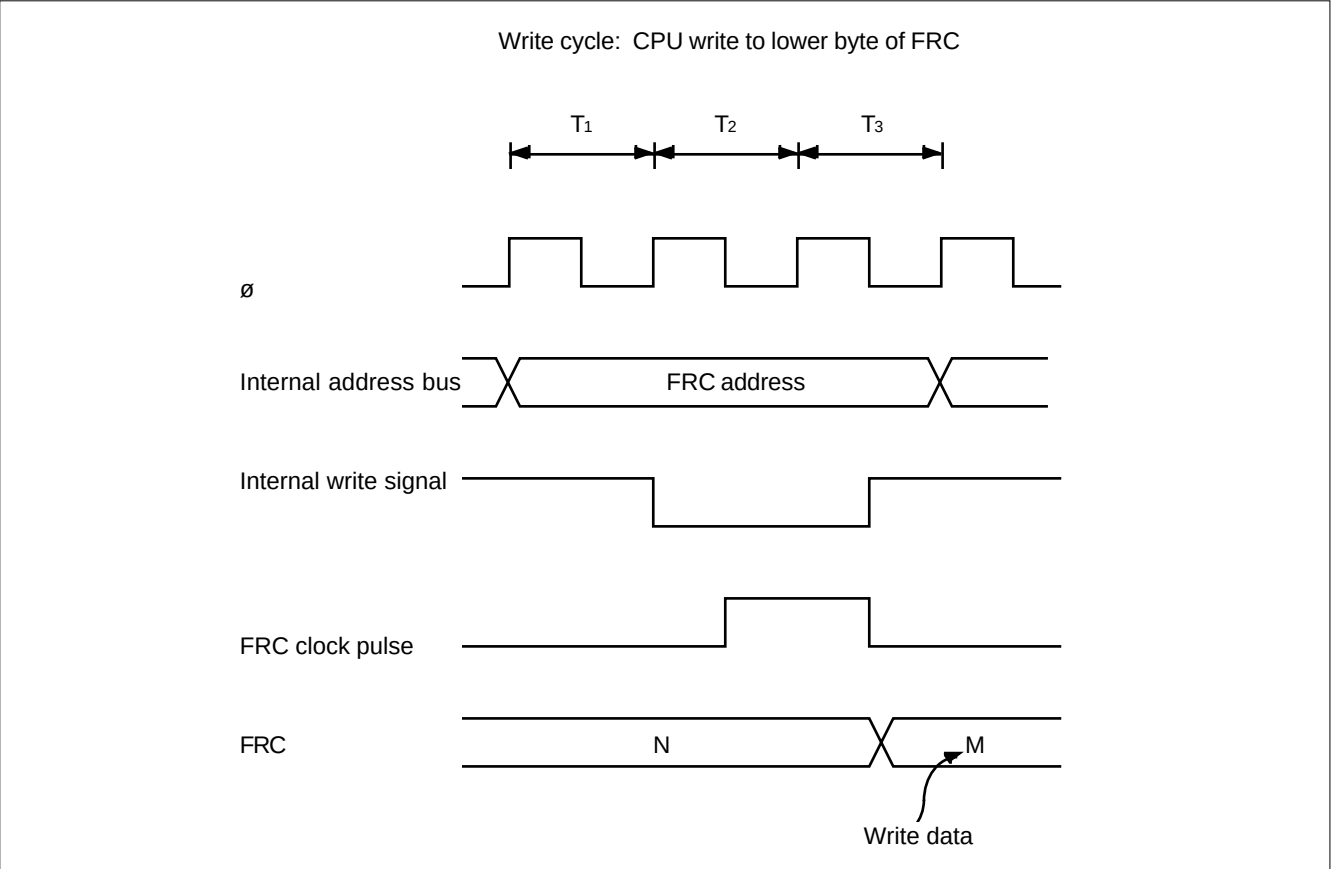


Figure 10-13 FRC Write-Increment Contention

Contention between OCR Write and Compare-Match: If a compare-match occurs during the T3 state of a write cycle to the lower byte of OCRA or OCRB, the write takes precedence and the compare-match signal is inhibited.

Figure 10-14 shows this type of contention.

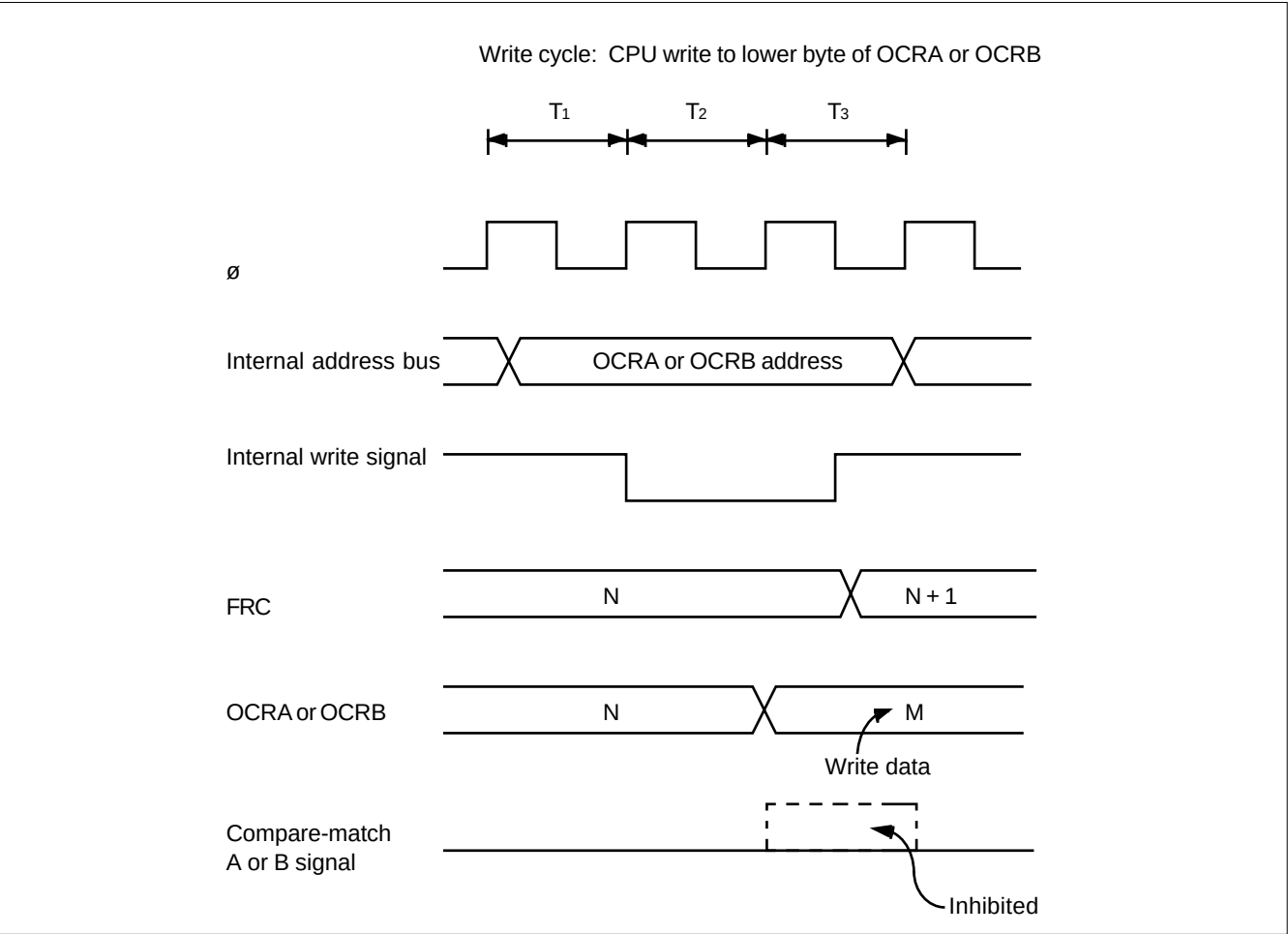


Figure 10-14 Contention between OCR Write and Compare-Match

Incrementation Caused by Changing of Internal Clock Source: When an internal clock source is changed, the changeover may cause the FRC to increment. This depends on the time at which the clock select bits (CKS1 and CKS0) are rewritten, as shown in table 10-5.

The pulse that increments the FRC is generated at the falling edge of the internal clock source. If clock sources are changed when the old source is high and the new source is low, as in case no. 3 in table 10-5, the changeover generates a falling edge that triggers the FRC increment pulse.

Switching between an internal and external clock source can also cause the FRC to increment.

Table 10-5 Effect of Changing Internal Clock Sources

No.	Description	Timing Chart
1	Low → Low: CKS1 and CKS0 are rewritten while both clock sources are low.	
2	Low → High: CKS1 and CKS0 are rewritten while old clock source is low and new clock source is high.	
3	High → Low: CKS1 and CKS0 are rewritten while old clock source is high and new clock source is low.	

Note: * The switching of clock sources is regarded as a falling edge that increments the FRC.

Table 10-5 Effect of Changing Internal Clock Sources (cont)

No.	Description	Timing Chart
4	High → High: CKS1 and CKS0 are rewritten while both clock sources are high.	The timing chart illustrates the sequence of events for a High → High clock source change. It consists of four horizontal signal lines: Old clock source: A square wave that is high during the transition period. New clock source: A square wave that is high during the transition period. FRC clock pulse: A narrow pulse that occurs during the high period of both clock sources. FRC: A signal divided into segments N, N+1, and N+2. A vertical line labeled "CKS rewrite" is positioned at the start of segment N+2.

Section 11 8-Bit Timer

11.1 Overview

The H8/520 chip includes a single 8-bit timer based on an 8-bit counter (TCNT). The timer has two time constant registers (TCORA and TCORB) that are constantly compared with the TCNT value to detect compare-match events. One application of the 8-bit timer is to generate a rectangular-wave output with an arbitrary duty factor.

11.1.1 Features

The features of the 8-bit timer are listed below.

- Selection of four clock sources
The counter can be driven by an internal clock signal ($\emptyset/8$, $\emptyset/64$, or $\emptyset/1024$) or an external clock input (enabling use as an external event counter).
- Selection of three ways to clear the counter
The counter can be cleared on compare-match A or B, or by an external reset signal.
- Timer output controlled by two compare-match signals
The single timer output (TMO) is controlled by two independent compare-match signals, enabling the timer to generate output waveforms with an arbitrary duty factor.
- Three types of interrupts
Compare-match A and B and overflow interrupts can be requested independently.
The compare match interrupts can be served by the data transfer controller (DTC), enabling interrupt-driven data transfer with minimal CPU programming.

11.1.2 Block Diagram

Figure 11-1 shows a block diagram of the 8-bit timer.

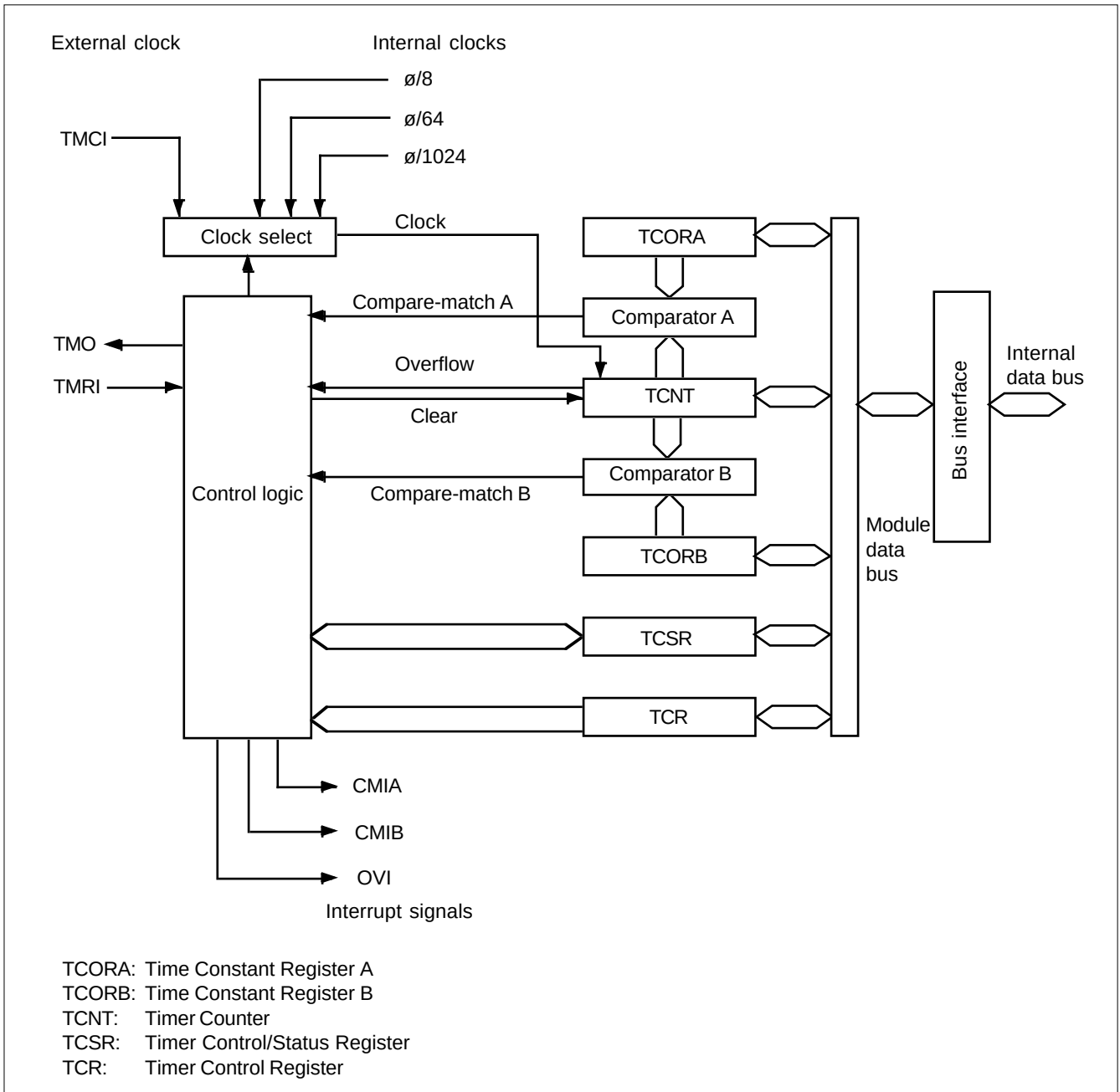


Figure 11-1 Block Diagram of 8-Bit Timer

11.1.3 Input and Output Pins

Table 11-1 lists the input and output pins of the 8-bit timer.

Table 11-1 Input and Output Pins of 8-Bit Timer

Name	Abbreviation	I/O	Function
Timer output	TMO	Output	Output controlled by compare-match
Timer clock input	TMCI	Input	External clock source for the counter
Timer reset input	TMRI	Input	External reset signal for the counter

11.1.4 Register Configuration

Table 11-2 lists the registers of the 8-bit timer.

Table 11-2 8-Bit Timer Registers

Name	Abbreviation	R/W	Initial Value	Address
Timer control register	TCR	R/W	H'00	H'FFD0
Timer control/status register	TCSR	R/(W)*	H'10	H'FFD1
Timer constant register A	TCORA	R/W	H'FF	H'FFD2
Timer constant register B	TCORB	R/W	H'FF	H'FFD3
Timer counter	TCNT	R/W	H'00	H'FFD4

Note: * Software can write a 0 to clear bits 7 to 5, but cannot write a 1 in these bits.

11.2 Register Descriptions

11.2.1 Timer Counter (TCNT)—H'FFD4

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The timer counter (TCNT) is an 8-bit up-counter that increments on a pulse generated from one of four clock sources. The clock source is selected by clock select bits 2 to 0 (CKS2 to CKS0) of the timer control register (TCR). The CPU can always read or write the timer counter.

The timer counter can be cleared by an external reset input or by an internal compare-match signal generated at a compare-match event. Clock clear bits 1 and 0 (CCLR1 and CCLR0) of the timer control register select the method of clearing.

When the timer counter overflows from H'FF to H'00, the overflow flag (OVF) in the timer control/status register (TCSR) is set to 1.

The timer counter is initialized to H'00 at a reset and in the standby modes.

11.2.2 Time Constant Registers A and B (TCORA and TCORB)—H'FFD2 and H'FFD3

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

TCORA and TCORB are 8-bit readable/writable registers. The timer count is continually compared with the constants written in these registers. When a match is detected, the corresponding compare-match flag (CMFA or CMFB) is set in the timer control/status register (TCSR).

The timer output signal (TMO) is controlled by these compare-match signals as specified by output select bits 3 to 0 (OS3 to OS0) in the timer status/control register (TCSR).

TCORA and TCORB are initialized to H'FF at a reset and in the standby modes.

11.2.3 Timer Control Register (TCR)—H'FFD0

Bit	7	6	5	4	3	2	1	0
	CMIEB	CMIEA	OVIE	CCLR1	CCLR0	CKS2	CKS1	CKS0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The TCR is an 8-bit readable/writable register that selects the clock source and the time at which the timer counter is cleared, and enables interrupts.

The TCR is initialized to H'00 at a reset and in the standby modes.

Bit 7—Compare-Match Interrupt Enable B (CMIEB): This bit selects whether to request compare-match interrupt B (CMIB) when compare-match flag B (CMFB) in the timer status/control register (TCSR) is set to 1.

Bit 7

CMIEB	Description
0	Compare-match interrupt request B (CMIB) is disabled. (Initial value)
1	Compare-match interrupt request B (CMIB) is enabled.

Bit 6—Compare-Match Interrupt Enable A (CMIEA): This bit selects whether to request compare-match interrupt A (CMIA) when compare-match flag A (CMFA) in the timer status/control register (TCSR) is set to 1.

Bit 6

CMIEA	Description
0	Compare-match interrupt request A (CMIA) is disabled. (Initial value)
1	Compare-match interrupt request A (CMIA) is enabled.

Bit 5—Timer Overflow Interrupt Enable (OVIE): This bit selects whether to request a timer overflow interrupt (OVI) when the overflow flag (OVF) in the timer status/control register (TCSR) is set to 1.

Bit 5

OVIE	Description
0	The timer overflow interrupt request (OVI) is disabled. (Initial value)
1	The timer overflow interrupt request (OVI) is enabled.

Bits 4 and 3—Counter Clear 1 and 0 (CCLR1 and CCLR0): These bits select how the timer counter is cleared: by compare-match A or B or by an external reset input.

Bit 4 CCLR1	Bit 3 CCLR0	Description
0	0	Not cleared. (Initial value)
0	1	Cleared on compare-match A.
1	0	Cleared on compare-match B.
1	1	Cleared on rising edge of external reset input signal.

Bits 2, 1, and 0—Clock Select (CKS2, CKS1, and CKS0): These bits select the internal or external clock source for the timer counter. For the external clock source they select whether to increment the count on the rising or falling edge of the clock input, or on both edges.

Bit 2	Bit 1	Bit 0	Description
CKS2	CKS1	CKS0	
0	0	0	No clock source (timer stopped). (Initial value)
0	0	1	Internal clock source ($\varnothing/8$).
0	1	0	Internal clock source ($\varnothing/64$).
0	1	1	Internal clock source ($\varnothing/1024$).
1	0	0	No clock source (timer stopped).
1	0	1	External clock source, counted on the rising edge.
1	1	0	External clock source, counted on the falling edge.
1	1	1	External clock source, counted on both the rising and falling edges.

11.2.4 Timer Control/Status Register (TCSR)

Bit	7	6	5	4	3	2	1	0
	CMFB	CMFA	OVF	—	OS3	OS2	OS1	OS0
Initial value	0	0	0	1	0	0	0	0
Read/Write	R/(W)*	R/(W)*	R/(W)*	—	R/W	R/W	R/W	R/W

The TCSR is an 8-bit readable and partially writable* register that indicates compare-match and over-flow status and selects the effect of compare-match events on the timer output signal (TMO).

The TCSR is initialized to H'10 at a reset and in the standby modes.

Note: * Software can write a 0 in bits 7 to 5 to clear the flags, but cannot write a 1 in these bits.

Bit 7—Compare-Match Flag B (CMFB): This status flag is set to 1 when the timer count matches the time constant set in TCORB.

Bit 7

CMFB	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The CPU reads the CMFB bit after the CMFB bit has been set to 1, then writes a 0 in this bit. 2. Compare-match interrupt B is served by the data transfer controller (DTC).
1	This bit is set to 1 when TCNT = TCORB.

Bit 6—Compare-Match Flag A (CMFA): This status flag is set to 1 when the timer count matches the time constant set in TCORA.

Bit 6

CMFA	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The CPU reads the CMFA bit after the CMFA bit has been set to 1, then writes a 0 in this bit. 2. Compare-match interrupt A is served by the data transfer controller (DTC).
1	This bit is set to 1 when TCNT = TCORA.

Bit 5—Timer Overflow Flag (OVF): This status flag is set to 1 when the timer count overflows (changes from H'FF to H'00).

Bit 5

OVF	Description
0	This bit is cleared from 1 to 0 when the CPU reads (Initial value) the OVF bit after the OVF bit has been set to 1, then writes a 0 in this bit.
1	This bit is set to 1 when TCNT changes from H'FF to H'00.

Bit 4—Reserved: This bit cannot be modified and is always read as 1.

Bits 3 to 0—Output Select 3 to 0 (OS3 to OS0): These bits specify the effect of compare-match events on the timer output signal (TMO). Bits OS3 and OS2 control the effect of compare-match B on the output level. Bits OS1 and OS0 control the effect of compare-match A on the output level.

When all four output select bits are cleared to 0 the TMO signal is not output.

After a reset, the TMO output is low (0) until the first compare-match event.

Bit 3	Bit 2	
OS3	OS2	Description
0	0	No change when compare-match B occurs. (Initial value)
0	1	Output changes to 0 when compare-match B occurs.
1	0	Output changes to 1 when compare-match B occurs.
1	1	Output inverts (toggles) when compare-match B occurs.

Bit 1	Bit 0	
OS1	OS0	Description
0	0	No change when compare-match A occurs. (Initial value)
0	1	Output changes to 0 when compare-match A occurs.
1	0	Output changes to 1 when compare-match A occurs.
1	1	Output inverts (toggles) when compare-match A occurs.

11.3 Operation

11.3.1 TCNT Incrementation Timing

The timer counter increments on a pulse generated once for each period of the selected (internal or external) clock source.

If external clock input (TMCI) is selected, the timer counter can increment on the rising edge, the falling edge, or both edges of the external clock signal.

The external clock pulse width must be at least 1.5 ϕ clock periods for incrementation on a single edge, and at least 2.5 ϕ clock periods for incrementation on both edges. The counter will not increment correctly if the pulse width is shorter than these values.

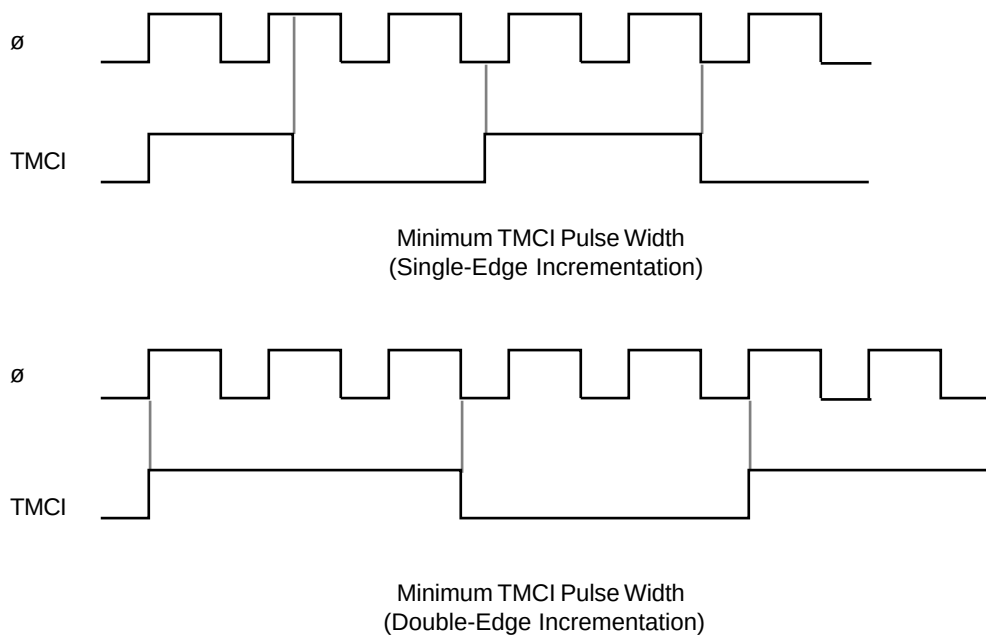


Figure 11-2 shows the timing of incrementation on both edges of an external clock signal.

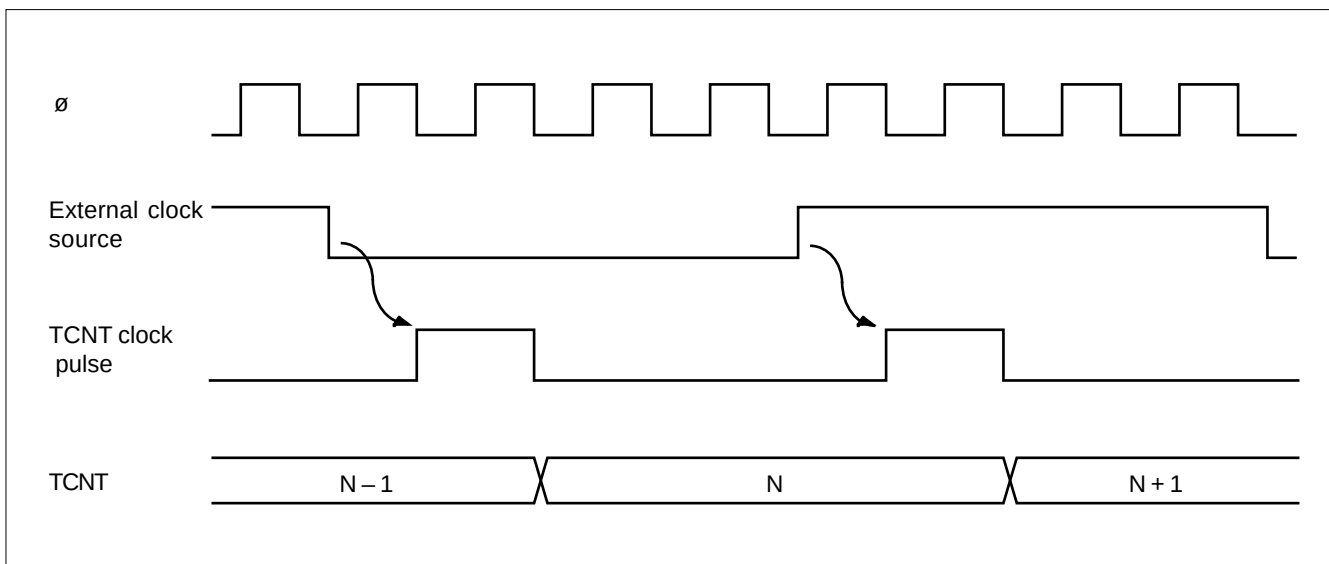


Figure 11-2 Count Timing for External Clock Input

11.3.2 Compare Match Timing

Setting of Compare-Match Flags A and B (CMFA and CMFB): The compare-match flags are set to 1 by an internal compare-match signal generated when the timer count matches the time constant in TCORA or TCORB. The compare-match signal is generated at the last state in which the match is true, just as the timer counter increments to a new value.

Accordingly, when the timer count matches one of the time constants, the compare-match signal is not generated until the next period of the clock source. Figure 11-3 shows the timing of the setting of the compare-match flags.

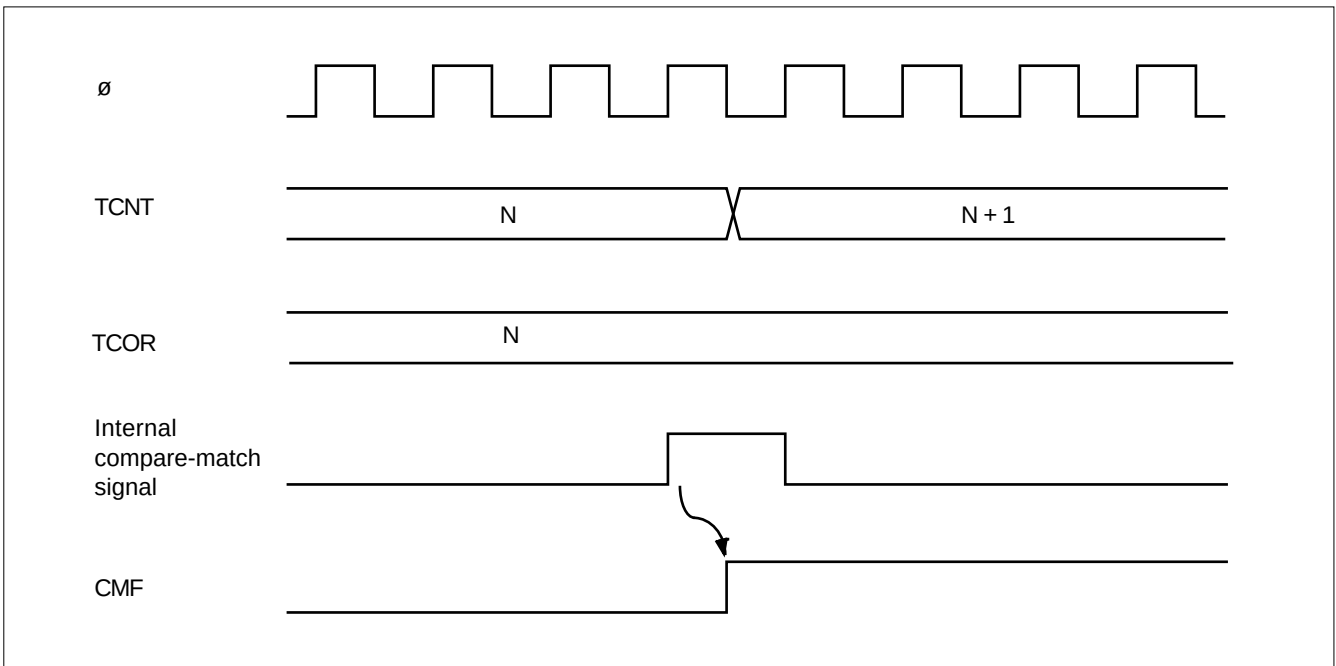


Figure 11-3 Setting of Compare-Match Flags

Output Timing: When a compare-match event occurs, the timer output (TMO) changes as specified by the output select bits (OS3 to OS0) in the TCSR. Depending on these bits, the output can remain the same, change to 0, change to 1, or toggle.

Figure 11-4 shows the timing when the output is set to toggle on compare-match A.

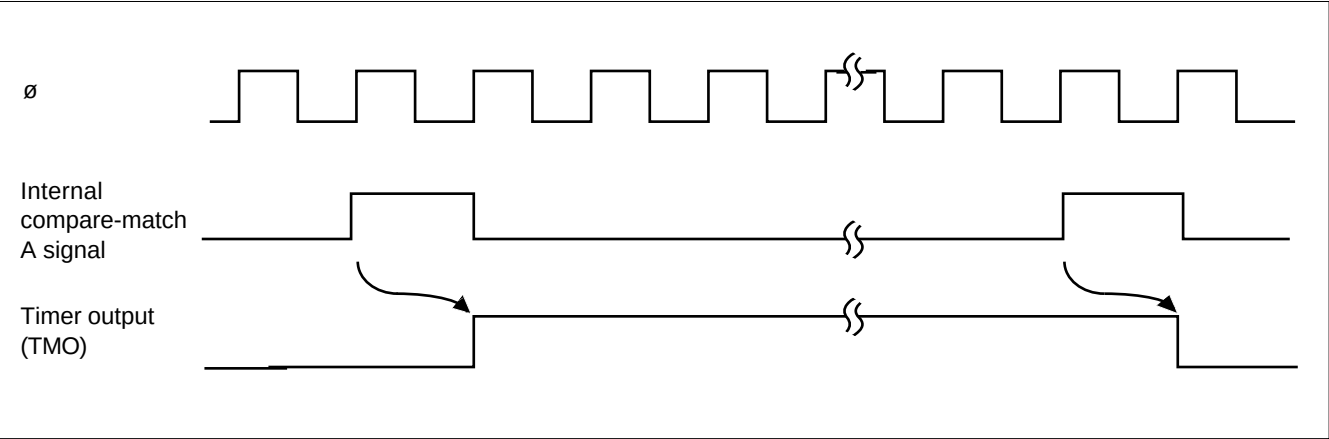


Figure 11-4 Timing of Timer Output

Timing of Compare-Match Clear

Depending on the CCLR1 and CCLR0 bits in the TCR, the timer counter can be cleared when compare-match A or B occurs. Figure 11-5 shows the timing of this operation.

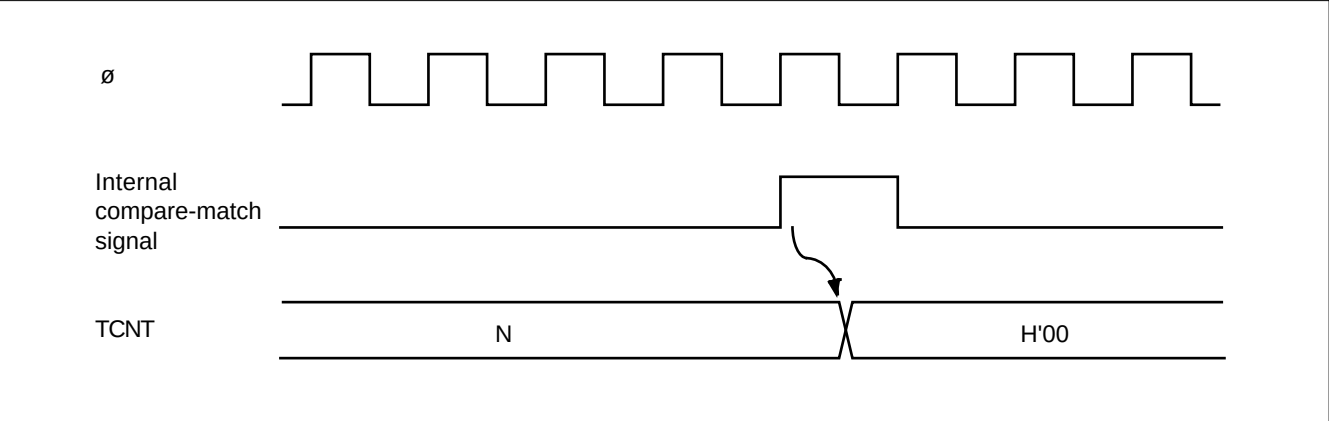


Figure 11-5 Timing of Compare-Match Clear

11.3.3 External Reset of TCNT

When the CCLR1 and CCLR0 bits in the TCR are both set to 1, the timer counter is cleared on the rising edge of an external reset input (TMRI). To ensure resetting, the TMRI pulse width must be at least 1.5 $\emptyset$ clock periods. Figure 11-6 shows the timing of this operation.

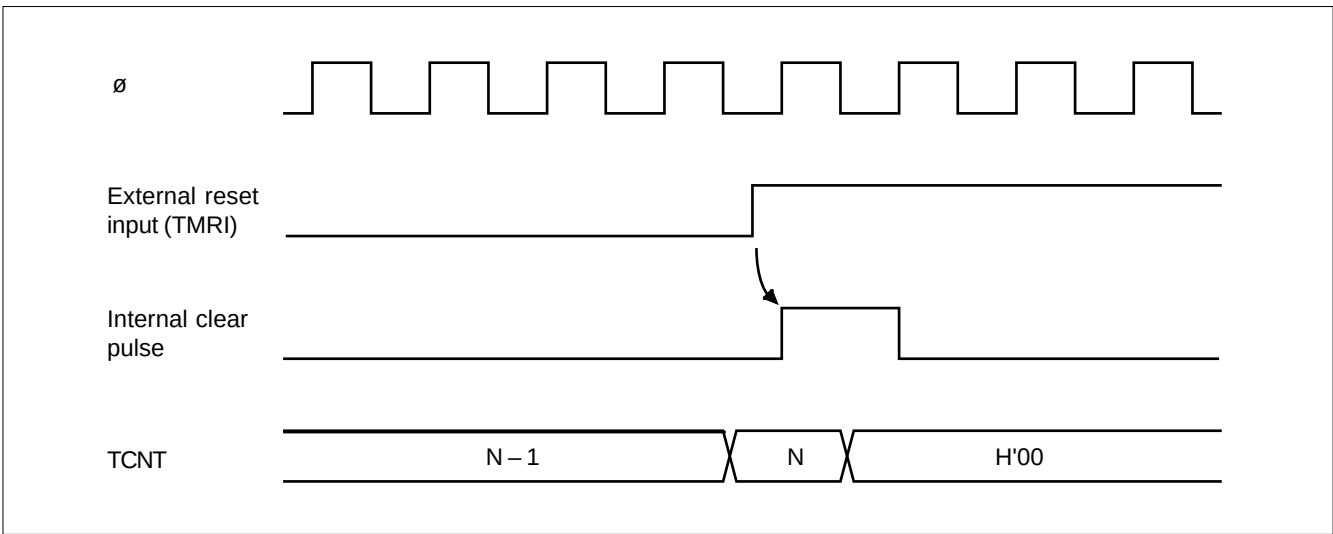


Figure 11-6 Timing of External Reset

11.3.4 Setting of TCNT Overflow Flag

The overflow flag (OVF) is set to 1 when the timer count overflows (changes from H'FF to H'00). Figure 11-7 shows the timing of this operation.

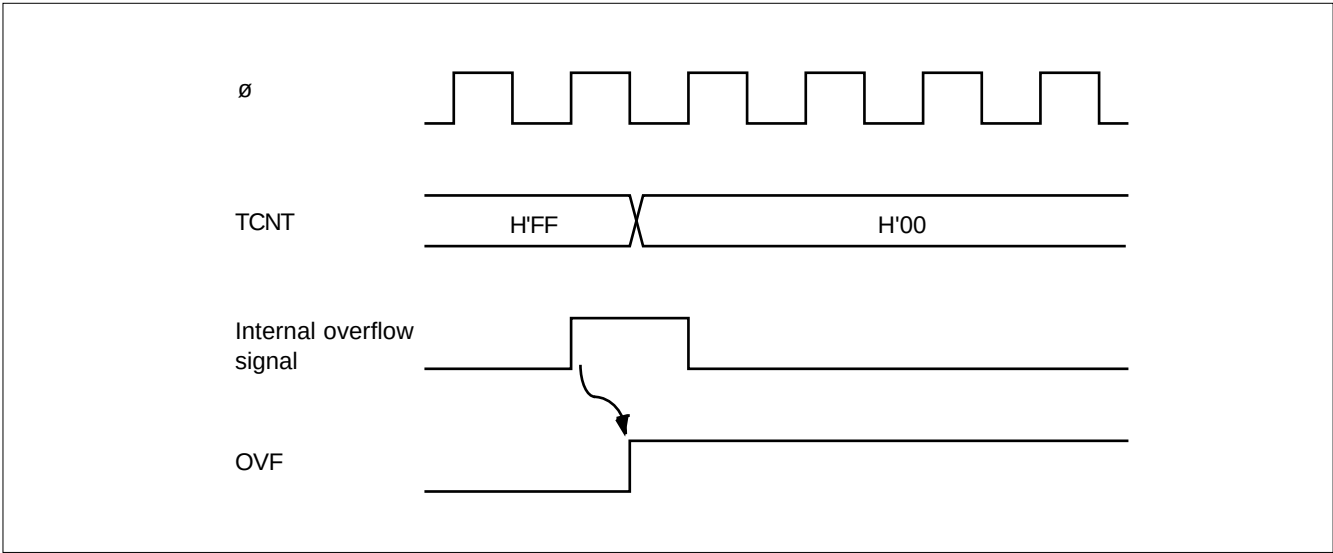


Figure 11-7 Setting of Overflow Flag (OVF)

11.4 CPU Interrupts and DTC Interrupts

The 8-bit timer can generate three types of interrupts: compare-match A and B (CMIA and CMIB), and overflow (OVI). Each interrupt is requested when the corresponding enable and flag bits are set in the TCR and TCSR. Independent signals are sent to the interrupt controller for each type of interrupt. Table 11-3 lists information about these interrupts.

Table 11-3 8-Bit Timer Interrupts

Interrupt	Description	DTC Service Available?	Priority
CMIA	Requested when CMFA is set	Yes	High
CMIB	Requested when CMFB is set	Yes	↑
OVI	Requested when OVF is set	No	Low

The CMIA and CMIB interrupts can be served by the data transfer controller (DTC) to have a data transfer performed.

When the DTC serves one of these interrupts, it automatically clears the CMFA or CMFB flag to 0. See section 6, “Data Transfer Controller”, for further information on the DTC.

11.5 Sample Application

In the example below, the 8-bit timer is used to generate a pulse output with a selected duty factor. The control bits are set as follows:

1. In the TCR, CCLR1 is cleared to 0 and CCLR0 is set to 1 so that the timer counter is cleared when its value matches the constant in TCORA.
2. In the TCSR, bits OS3 to OS0 are set to 0110, causing the output to change to 1 on compare-match A and to 0 on compare-match B.

With these settings, the 8-bit timer provides output of pulses at a rate determined by TCORA with a pulse width determined by TCORB. No software intervention is required.

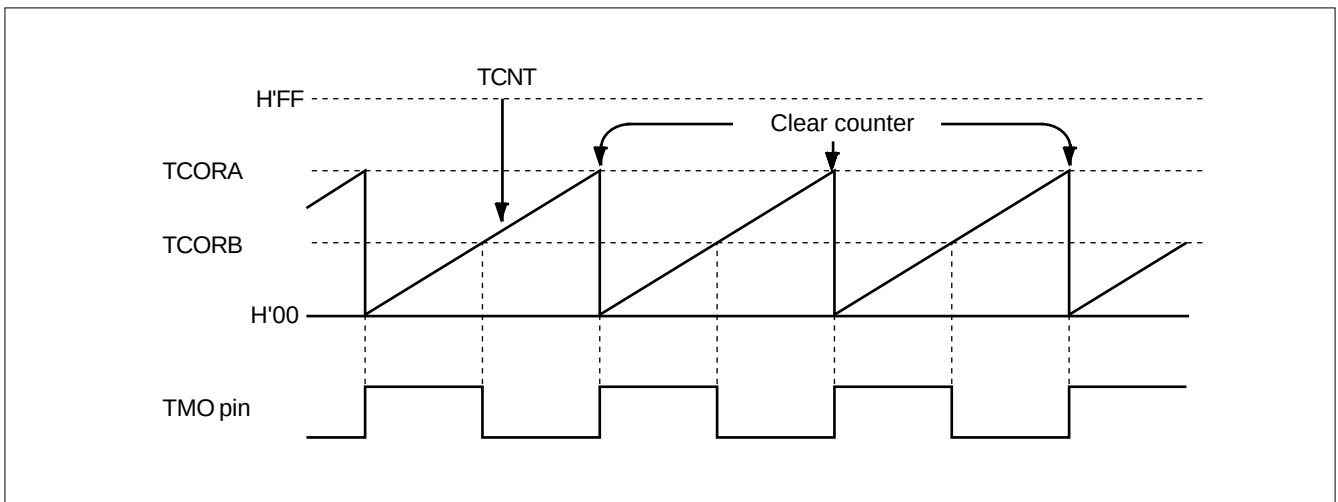


Figure 11-8 Example of Pulse Output

11.6 Application Notes

Application programmers should note that the following types of contention can occur in the 8-bit timer.

Contention between TCNT Write and Clear: If an internal counter clear signal is generated during the T3 state of a write cycle to the timer counter, the clear signal takes priority and the write is not performed.

Figure 11-9 shows this type of contention.

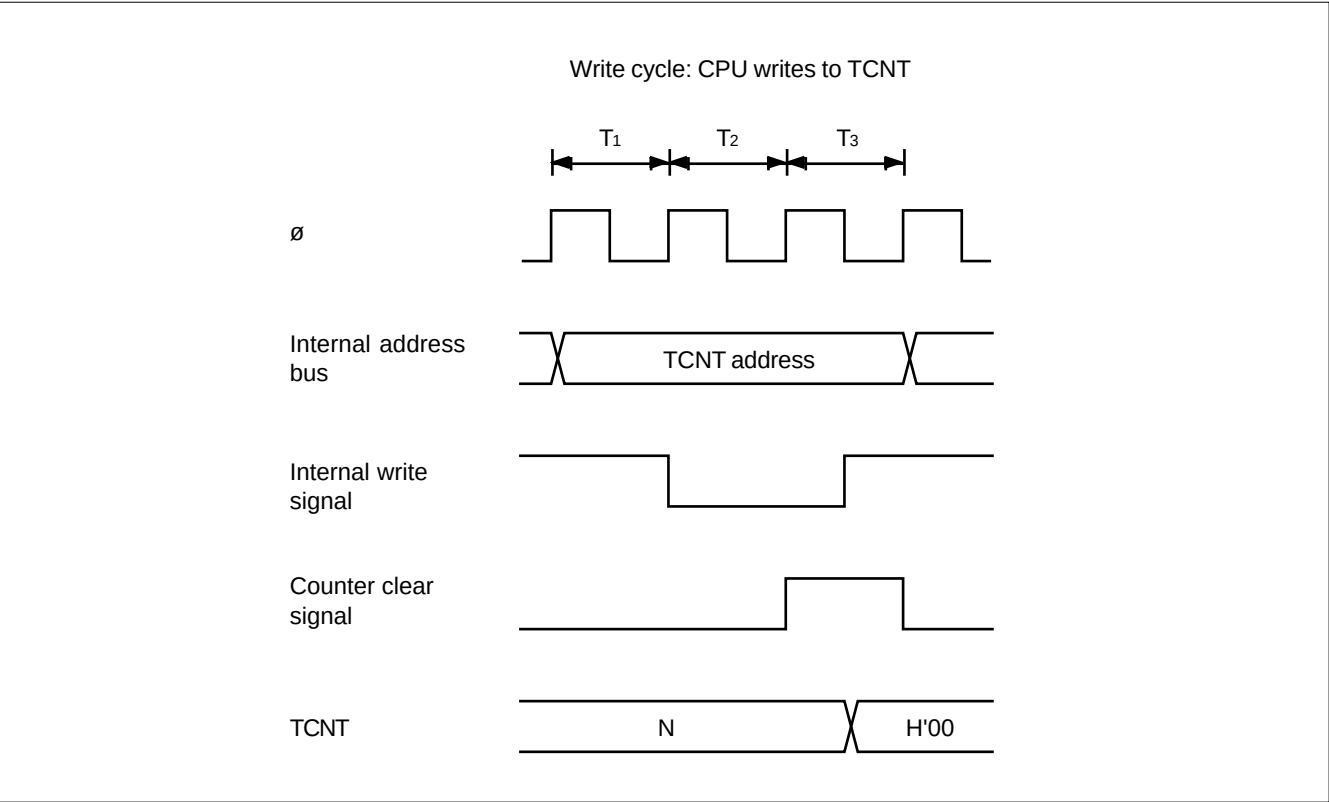


Figure 11-9 TCNT Write-Clear Contention

Contention between TCNT Write and Increment: If a timer counter increment pulse is generated during the T3 state of a write cycle to the timer counter, the write takes priority and the timer counter is not incremented.

Figure 11-10 shows this type of contention.

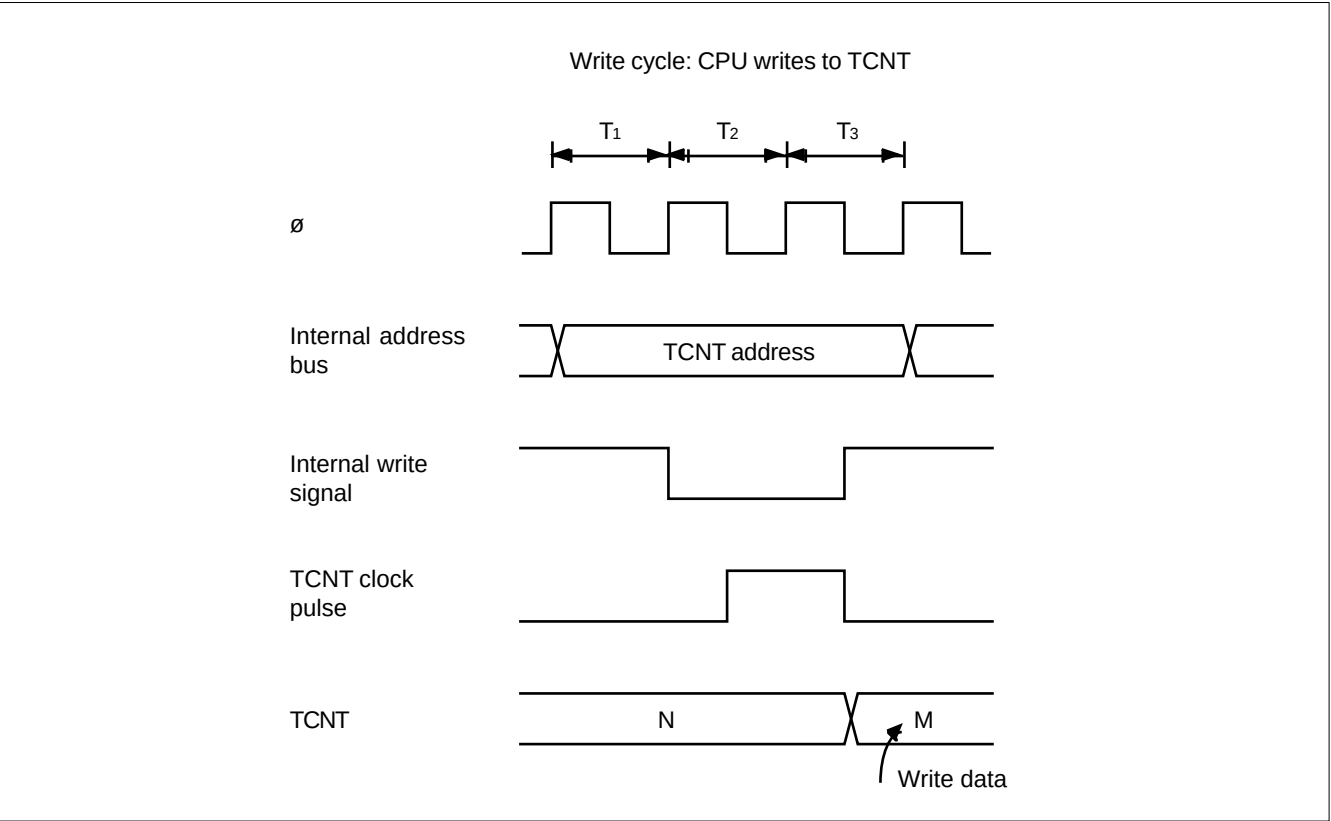


Figure 11-10 TCNT Write-Increment Contention

Contention between TCOR Write and Compare-Match: If a compare-match occurs during the T3 state of a write cycle to TCORA or TCORB, the write takes precedence and the compare-match signal is inhibited.

Figure 11-11 shows this type of contention.

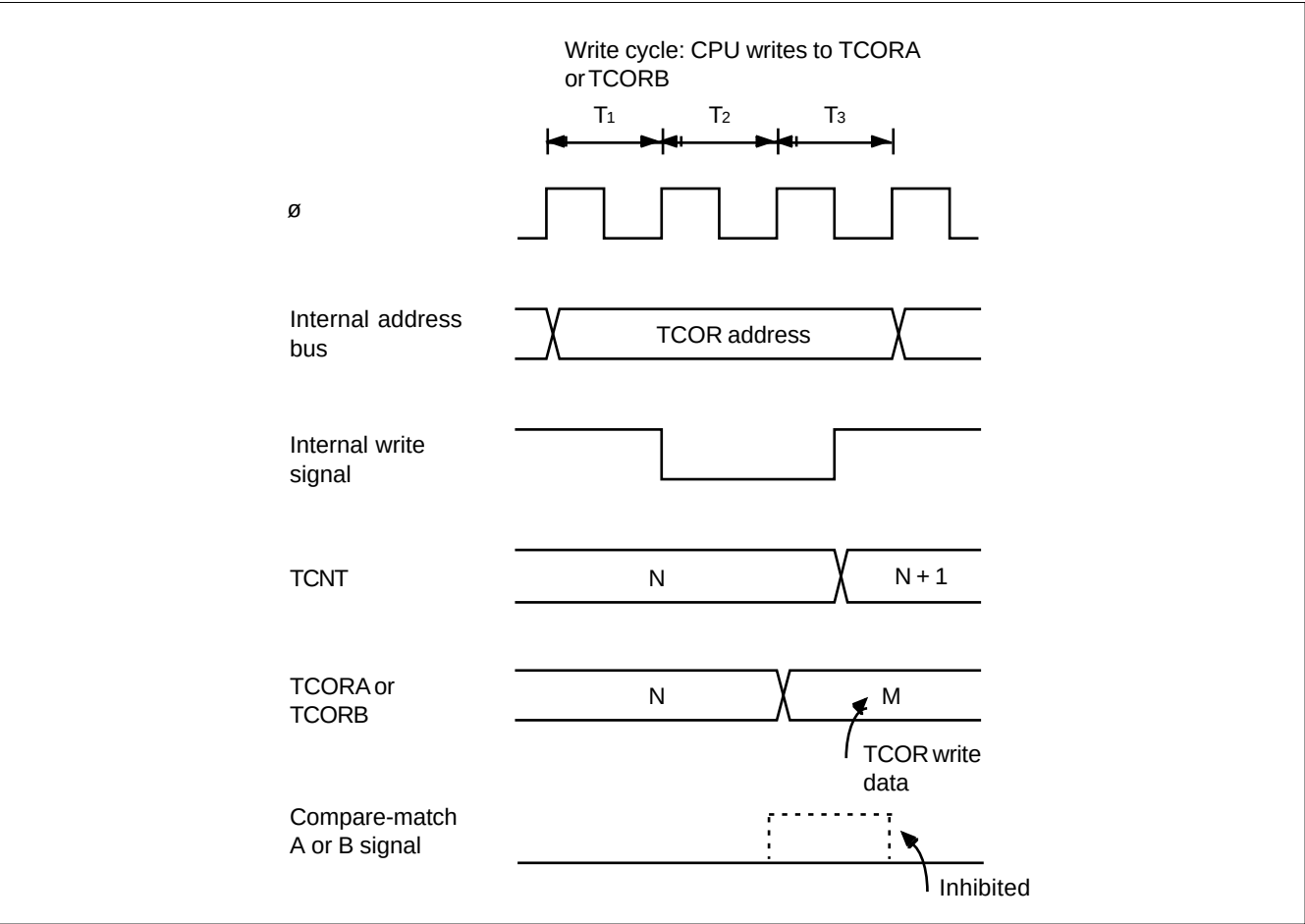


Figure 11-11 Contention between TCOR Write and Compare-Match

Contention between Compare-Match A and Compare-Match B: If identical time constants are written in TCORA and TCORB, causing compare-match A and B to occur simultaneously, any conflict between the output selections for compare-match A and B is resolved by following the priority order in table 11-4.

Table 11-4 Priority Order of Timer Output

Output Selection	Priority
Toggle	High
1 Output	↑ Low
0 Output	
No change	

Incrementation Caused by Changing of Internal Clock Source: When an internal clock source is changed, the changeover may cause the timer counter to increment. This depends on the time at which the clock select bits (CKS2 to CKS0) are rewritten, as shown in table 11-5.

The pulse that increments the timer counter is generated at the falling edge of the internal clock source signal. If clock sources are changed when the old source is high and the new source is low, as in case no. 3 in table 11-5, the changeover generates a falling edge that triggers the TCNT clock pulse and increments the timer counter.

Switching between an internal and external clock source can also cause the timer counter to increment.

Table 11-5 Effect of Changing Internal Clock Sources

No.	Description	Timing Chart
1	Low → Low*: CKS1 and CKS0 are rewritten while both clock sources are low.	Old clock source New clock source TCNT clock pulse TCNT CKS rewrite

Note: * Including a transition from low to the stopped state (CKS1 = 0, CKS0 = 0), or a transition from the stopped state to low.

Table 11-5 Effect of Changing Internal Clock Sources (cont)

No.	Description	Timing Chart
2	Low → High* ¹ : CKS1 and CKS0 are rewritten while old clock source is low and new clock source is high.	
3	High → Low* ² : CKS1 and CKS0 are rewritten while old clock source is high and new clock source is low.	

Notes: 1. Including a transition from the stopped state to high.
2. Including a transition from high to the stopped state.
3. The switching of clock sources is regarded as a falling edge that increments the TCNT.

Table 11-5 Effect of Changing Internal Clock Sources (cont)

No.	Description	Timing Chart
4	High → High: CKS1 and CKS0 are rewritten while both clock sources are high.	The timing chart illustrates the transition of the clock source from Old to New. The Old clock source is high during the first and third clock pulses. The New clock source is high during the second and fourth clock pulses. The TCNT counter increments from N to N+1 to N+2. A vertical line marks the CKS rewrite event, which occurs during the second clock pulse when both sources are high.

Section 12 Watchdog Timer

12.1 Overview

The H8/520 has an on-chip watchdog timer (WDT) module. This module can monitor system operation by generating a signal that resets the H8/520 chip if a system crash allows the timer count to overflow.

When this watchdog function is not needed, the WDT module can be used as an interval timer. In the interval timer mode, an IRQ₀ interrupt is requested at each counter overflow.

The WDT module is also used in recovering from the software standby mode.

12.1.1 Features

The basic features of the watchdog timer module are summarized as follows:

- Selection of eight clock sources
- Selection of two modes: watchdog timer mode and interval timer mode
- Counter overflow generates a reset signal or interrupt request
Reset signal in the watchdog timer mode; IRQ₀ request in the interval timer mode.
- External output of reset signal
Depending on a reset output enable bit, the reset signal can be output externally to reset devices controlled by the H8/520, as well as the H8/520 itself.

12.1.2 Block Diagram

Figure 12-1 is a block diagram of the watchdog timer.

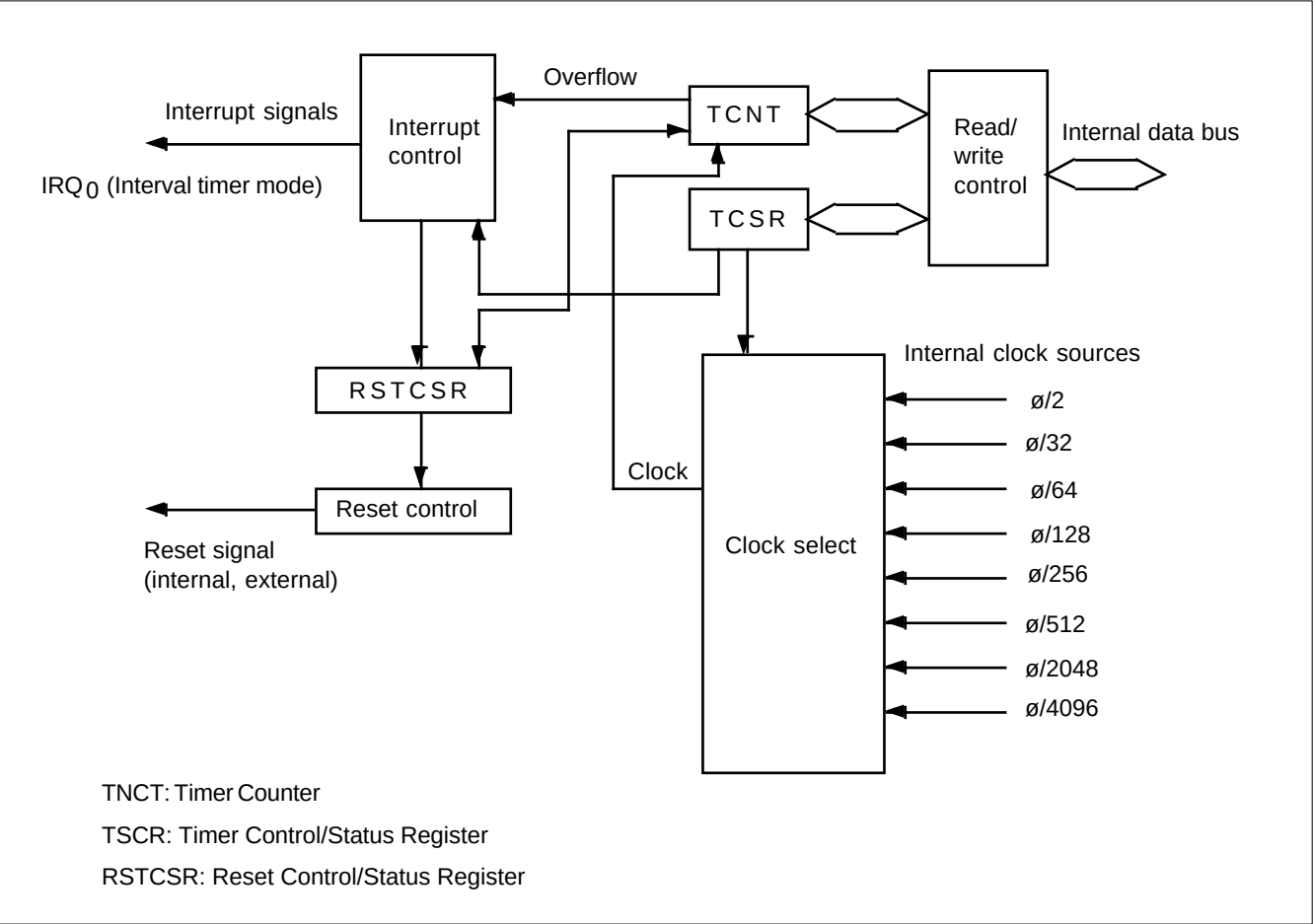


Figure 12-1 Block Diagram of Timer Counter

12.1.3 Register Configuration

Table 12-1 lists information on the watchdog timer registers.

Table 12-1 Register Configuration

Name	Abbreviation	R/W	Initial value	Addresses	
				Write	Read
Timer control/status register	TCSR	R/(W)*	H'18	H'FFEC	H'FFEC
Timer counter	TCNT	R/W	H'00	H'FFEC	H'FFED
Reset control/status register	RSTCSR	R/(W)*	H'3F	H'FFFE	H'FFFF

Note: * Software can write a 0 to clear bit 7, but cannot write a 1.

12.2 Register Descriptions

12.2.1 Timer Counter (TCNT)—H'FFED

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The watchdog timer counter (TCNT) is a readable/writable* 8-bit up-counter. When the timer enable bit (TME) in the timer control/status register (TCSR) is set to 1, the timer counter starts counting pulses of an internal clock source selected by clock select bits 2 to 0 (CKS2 to CKS0) in the TCSR. When the count overflows (changes from H'FF to H'00), a reset or interrupt signal is generated.

The watchdog timer counter is initialized to H'00 at a reset and when the TME bit is cleared to 0.

Note: * TCNT is write-protected by a password. See section 12.2.4, “Notes on Register Access”, for details.

12.2.2 Timer Control/Status Register (TCSR)—H'FFEC (Read), H'FFED (Write)

Bit	7	6	5	4	3	2	1	0
	OVF	WT/IT	TME	—	—	CKS2	CKS1	CKS0
Initial value	0	0	0	1	1	0	0	0
Read/Write	R/W*1	R/W	R/W	—	—	R/W	R/W	R/W

The watchdog timer control/status register (TCSR) is an 8-bit readable/writable*2 register that selects the timer mode and clock source and performs other functions.

Bits 7 to 5 are initialized to 0 at a reset and in the standby modes. Bits 2 to 0 are initialized to 0 at a reset, but retain their values in the software standby mode.

- Notes: 1. Software can write a 0 in bit 7 to clear the flag, but cannot set this bit to 1.
2. The TCSR is write-protected by a password. See section 12.2.4, “Notes on Register Access”, for details.

Bit 7—Overflow Flag (OVF): This bit indicates that the watchdog timer count has overflowed.

Bit 7	
OVF	Description
0	To clear this bit, the CPU must read this bit after it has been set to 1, (Initial value) then write a 0 in this bit.
1	This bit is set to 1 when TCNT changes from H'FF to H'00.*

Note: * The OVF bit is not set in the watchdog timer mode.

Bit 6—Timer Mode Select (WT/IT): This bit selects whether to operate in the watchdog timer mode or interval timer mode. If the watchdog timer mode is selected, a watchdog timer overflow resets the chip. If the interval timer mode is selected, a watchdog timer overflow generates an IRQ0 interrupt request.

Bit 6	
WT/IT	Description
0	Interval timer mode (IRQ0 request) (Initial value)
1	Watchdog timer mode (Reset)

Bit 5—Timer Enable (TME): This bit enables or disables the timer.

Bit 5	
TME	Description
0	TCNT is initialized to H'00 and stopped. (Initial value)
1	TCNT runs. A reset or interrupt is requested when the count overflows.

Bits 4 and 3—Reserved: These bits cannot be modified and are always read as 1.

Bits 2, 1, and 0—Clock Select (CKS2, CKS1, and CKS0): These bits select one of eight clock sources obtained by dividing the system clock (ϕ).

The overflow interval listed in the table below is the time from when the watchdog timer counter begins counting from H'00 until an overflow occurs.

In the interval timer mode, IRQ0 interrupts are requested at this interval.

Bit 2	Bit 1	Bit 0	Description	
CKS2	CKS1	CKS0	Clock Source	Overflow Interval (ϕ = 10 MHz)
0	0	0	$\phi/2$	51.2 μ s (Initial value)
0	0	1	$\phi/32$	819.2 μ s
0	1	0	$\phi/64$	1.6 ms
0	1	1	$\phi/128$	3.3 ms
1	0	0	$\phi/256$	6.6 ms
1	0	1	$\phi/512$	13.1 ms
1	1	0	$\phi/2048$	52.4 ms
1	1	1	$\phi/4096$	104.9 ms

12.2.3 Reset Control/Status Register (RSTCSR)—H'FFFF (Read), H'FFFE (Write)

Bit	7	6	5	4	3	2	1	0
	WRST	RSTOE	—	—	—	—	—	—
Initial value	0	0	1	1	1	1	1	1
Read/Write	R/(W)*1	R/W	—	—	—	—	—	—

The reset control/status register (RSTCSR) is an 8-bit readable/writable*2 register that indicates when a reset has been caused by a watchdog timer overflow, and controls external output of the reset signal.

Bit 6 is not initialized by the reset caused by the watchdog timer overflow. It is initialized, however, by a reset caused by input at the $\overline{\text{RES}}$ pin.

- Notes:
- 1. Software can write a 0 in bit 7 to clear the flag, but cannot set this bit to 1.
 - 2. The RSTCSR is write-protected by a password. See section 12.2.4, “Notes on Register Access”, for details.

Bit 7—Watchdog Timer Reset (WRST): This bit indicates that a reset signal has been generated by a watchdog timer overflow in the watchdog timer mode.

The reset signal generated by the overflow resets the entire H8/520 chip. In addition, if the reset output enable (RSTOE) bit is set to 1, a reset signal (low) is output at the $\overline{\text{RES}}$ pin to reset devices connected to the H8/520.

The WRST bit can be cleared by software by writing a 0. It is also cleared when a reset signal from an external device is received at the $\overline{\text{RES}}$ pin.

Bit 7

WRST	Description
0	This bit is cleared to 0 by a reset signal input from the $\overline{\text{RES}}$ pin, (Initial state) or when software writes a 0.
1	This bit is set to 1 when the watchdog timer overflows in the watchdog timer mode and an internal reset signal is generated.

Bit 6—Reset Output Enable (RSTOE): This bit selects whether to output a reset signal from the $\overline{\text{RST}}$ pin when the timer counter overflows in the watchdog timer mode.

Bit 6

RSTOE	Description
0	The reset signal generated by a watchdog timer overflow is not (Initial state) output to external devices.
1	The reset signal generated by a watchdog timer overflow is output to external devices.

Bits 5 to 0—Reserved: These bits cannot be modified and are always read as 1.

12.2.4 Notes on Register Access

The watchdog timer’s TCNT, TCSR, and RSTCSR registers differ from other registers in being more difficult to write. The procedures for writing and reading these registers are given below.

Writing to TCNT and TCSR: These registers must be written by word access. Programs cannot write to them by byte access. The word must contain the write data and a password.

The watchdog timer’s TCNT and TCSR registers both have the same write address. The write data must be contained in the lower byte of the word written at this address. The upper byte must contain H'5A (password for TCNT) or H'A5 (password for TCSR). See figure 12-2.

The result of the access depicted in figure 12-2 is to transfer the write data from the lower byte to the TCNT or TCSR.

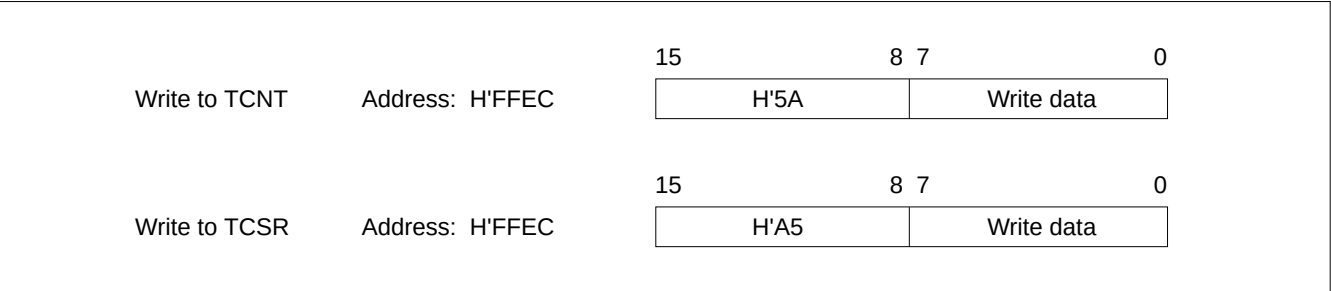


Figure 12-2 Writing to TCNT and TCSR

Coding Examples:

```
To clear TCNT to 00:      MOV.W #H'5A00, @H'FFEC
To write H'4F in TCSR:    MOV.W #H'A54F, @H'FFEC
```

Writing to RSTCSR: The RSTCSR must be written by moving word data to address H'FFFE. It cannot be written by byte access.

The upper byte of the word must contain a password. Separate passwords are used for clearing the WRST bit and for writing a 1 or 0 to the RSTOE bit.

To clear the WRST bit, the word written at address H'FFFE must contain the password H'A5 in the upper byte and the data H'00 in the lower byte. This clears the WRST bit to 0 without affecting other bits.

To set or clear the RSTOE bit, the word written at address H'FFFE must contain the password H'5A in the upper byte and the write data in the lower byte. This writes the desired data in the RSTOE bit without affecting other bits.

These write operations are illustrated in figure 12-3.

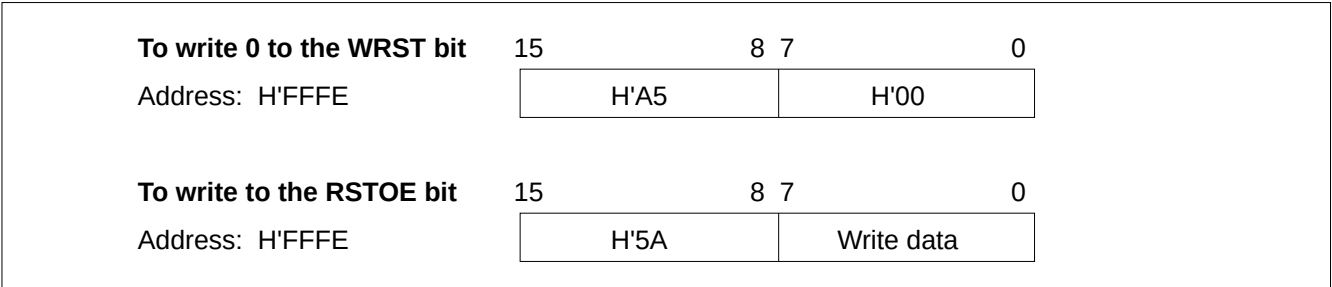


Figure 12-3 Writing to RSTCSR

Coding Examples:

```
To clear WRST to 0:  MOV.W #H'A500, @H'FFFE
To set RSTOE to 1:  MOV.W #H'5AFF, @H'FFFE
```

Reading TCNT, TCSR, and RSTCSR: The read addresses are H'FFEC for TCSR, H'FFED for TCNT, and H'FFFF for RSTCSR as indicated in table 12-2.

These three registers are read like other registers. Byte access instructions can be used.

Table 12-2 Read Addresses of TCNT and TCSR

Read Address	Register
H'FFEC	TCSR
H'FFED	TCNT
H'FFFF	RSTCSR

12.3 Operation

12.3.1 Watchdog Timer Mode

The watchdog timer function begins operating when software sets the WT/TT and TME bits to 1 in the TCSR. Thereafter, software should periodically rewrite the contents of the timer counter (normally by writing H'00) to prevent the count from overflowing. If a program crash allows the timer count to overflow, the watchdog timer generates a reset as shown in figure 12-4.

The reset signal from the watchdog timer can also be output from the RES pin to reset external devices. This reset output signal is a low pulse with a duration of 132 ø clock periods. The reset signal is output only if the RSTOE bit in the TCSR is set to 1.

The reset signal from the watchdog timer has the same vector as a reset generated by low input at the $\overline{\text{RES}}$ pin. Software should check the WRST bit in the RSTCSR to determine the source of the reset.

If a watchdog timer overflow occurs at the same time as a low input at the $\overline{\text{RES}}$ pin, priority is given to one type of reset or the other depending on the value of the RSTOE bit in the RSTCSR.

If the RSTOE bit is set to 1 when both types of reset occur simultaneously, the watchdog timer's reset signal takes precedence. The internal state of the H8/520 chip is reset, the RSTOE bit remains set to 1, the WRST bit is also set to 1, and the $\overline{\text{RES}}$ pin is held low for 132 ϕ clock periods. If at the end of 520 ϕ clock periods there is still an external low input to the $\overline{\text{RES}}$ pin, the external reset takes effect, clearing the WRST and RSTOE bits to 0. Note that if the external reset occurs before the watchdog timer overflows, it takes effect immediately and clears the RSTOE bit.

If the RSTOE bit is cleared to 0 when both types of reset occur simultaneously, the reset signal input from the $\overline{\text{RES}}$ pin takes precedence and the WRST bit is cleared to 0.

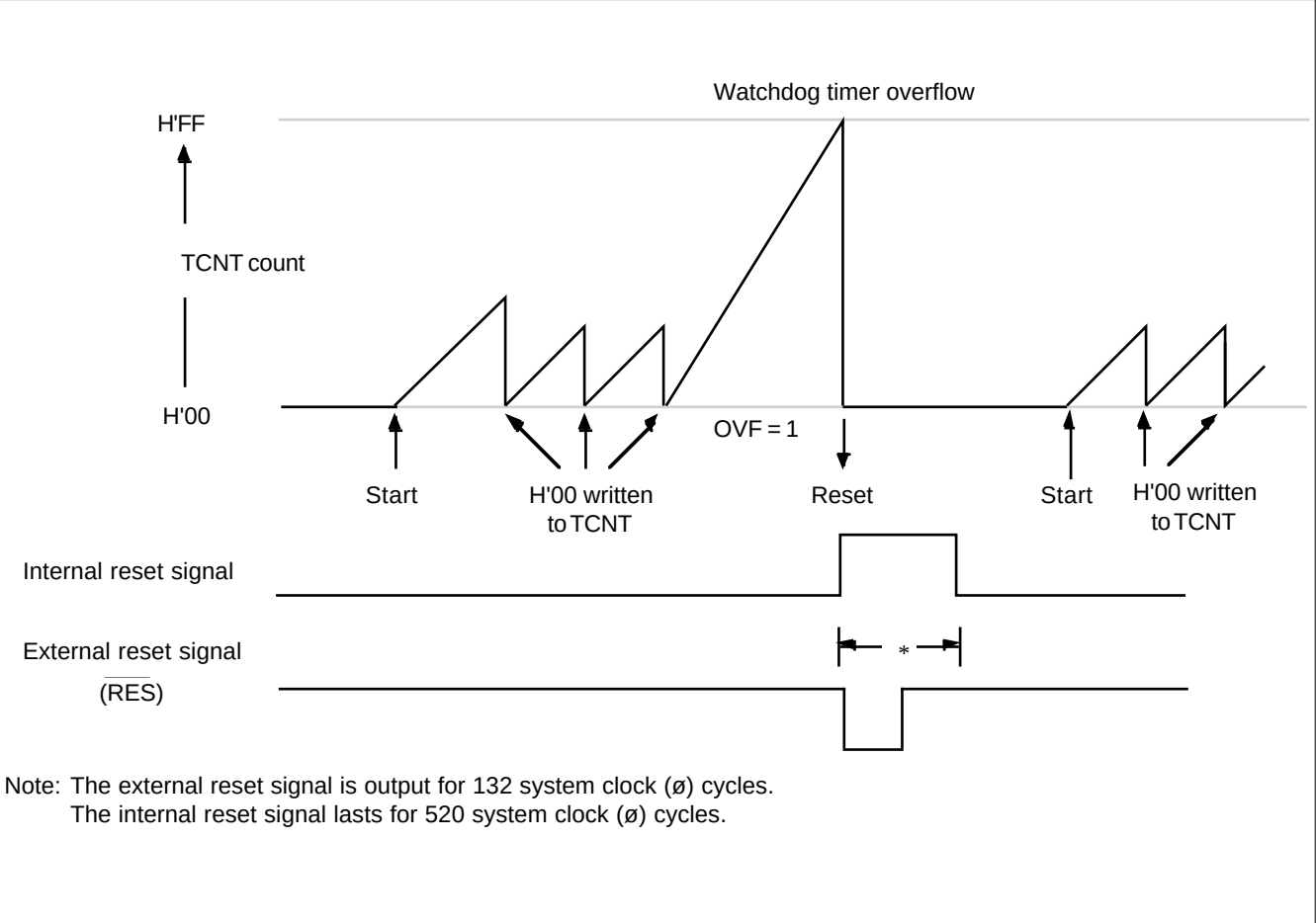


Figure 12-4 Operation in Watchdog Timer Mode

12.3.2 Interval Timer Mode

Interval timer operation begins when the $\overline{\text{WT/IT}}$ bit is cleared to 0 and the TME bit is set to 1.

In the interval timer mode, an IRQ₀ request is generated each time the timer count overflows. This function can be used to generate IRQ₀ requests at regular intervals. See figure 12-5.

IRQ₀ requests from the watchdog timer module have the same vector as IRQ₀ requests from the $\overline{\text{IRQ0}}$ pin, so the IRQ₀ interrupt-handling routine must check the OVF bit in the TCSR to determine the source of the interrupt.

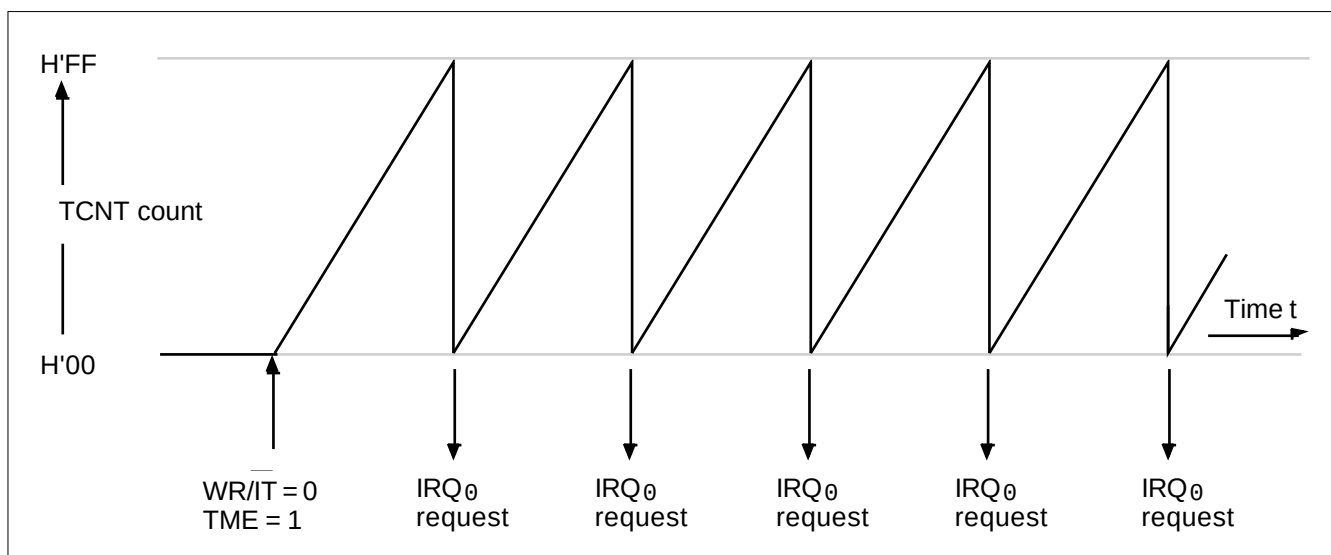


Figure 12-5 Operation in Interval Timer Mode

12.3.3 Operation in Software Standby Mode

The watchdog timer has a special function in the software standby mode. Specific watchdog timer settings are required when the software standby mode is used.

Before Transition to the Software Standby Mode: The TME bit must be cleared to 0 to stop the watchdog timer counter before a transition to the software standby mode. The chip cannot enter the software standby mode while the TME bit is set to 1. Before entering the software standby mode, software should also set the clock select bits (CKS2 to CKS0) to a value that makes the timer overflow interval equal to or greater than the settling time of the clock oscillator.

Recovery from the Software Standby Mode: Recovery from the software standby mode can be triggered by an NMI request. In this case the recovery proceeds as follows:

When an NMI request signal is received, the clock oscillator starts running and the watchdog timer starts counting at the rate selected by the clock select bits before the software standby mode was entered. When the count overflows from H'FF to H'00, the ϕ clock is presumed to be stable and usable, clock signals are supplied to all modules on the chip, and the NMI interrupt-handling routine starts executing.

12.3.4 Setting of Overflow Flag

The OVF bit is set to 1 when the timer count overflows in the interval timer mode. Simultaneously, the WDT module requests an IRQ0 interrupt. The timing is shown in figure 12-6.

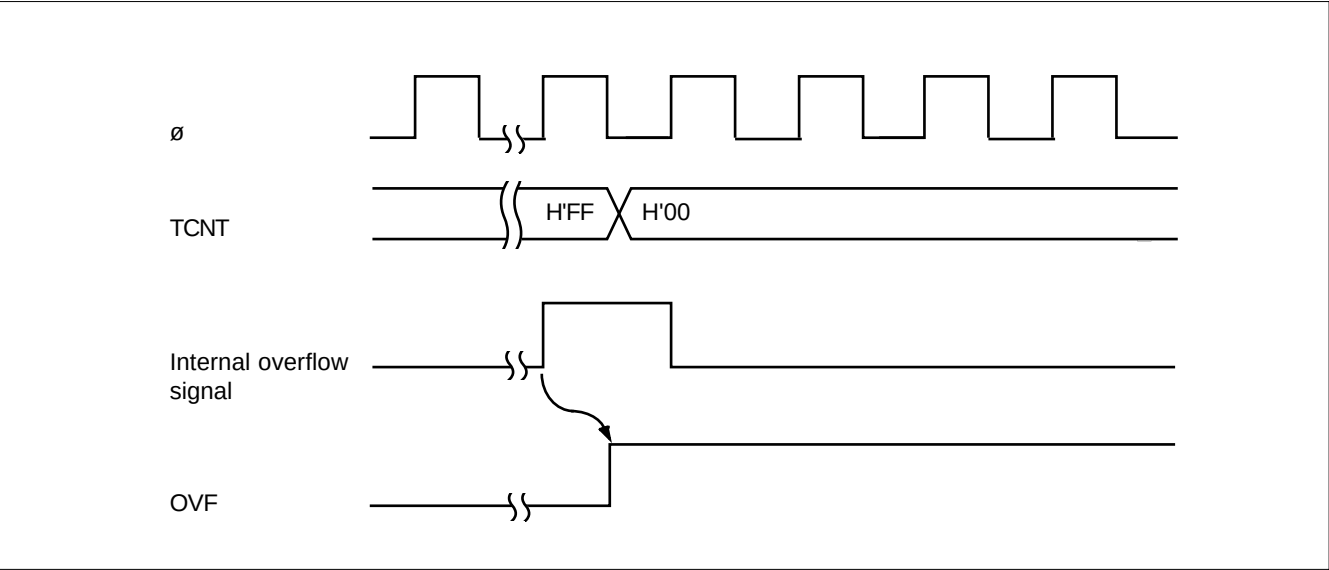


Figure 12-6 Setting of OVF Bit

12.3.5 Setting of Watchdog Timer Reset (WRST) Bit

The WRST bit is valid when $\overline{WT/IT} = 1$ and $TME = 1$.

The WRST bit is set to 1 when the timer count overflows. An internal reset signal is simultaneously generated for the entire H8/520 chip. The timing is shown in figure 12-7.

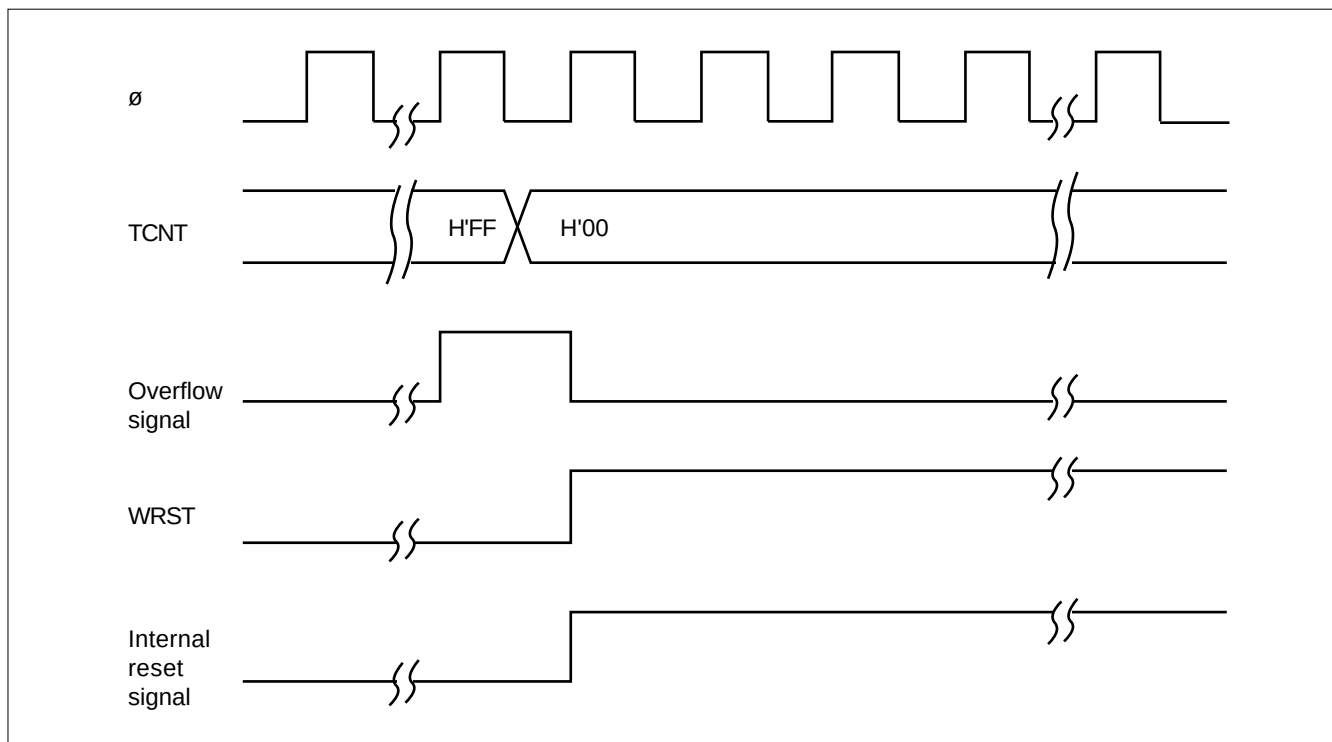


Figure 12-7 Setting of WRST Bit and Internal Reset Signal

12.4 Application Notes

Contention between TCNT Write and Increment: If a timer counter clock pulse is generated during the T3 state of a write cycle to the timer counter, the write takes priority and the timer counter is not incremented. See figure 12-8.

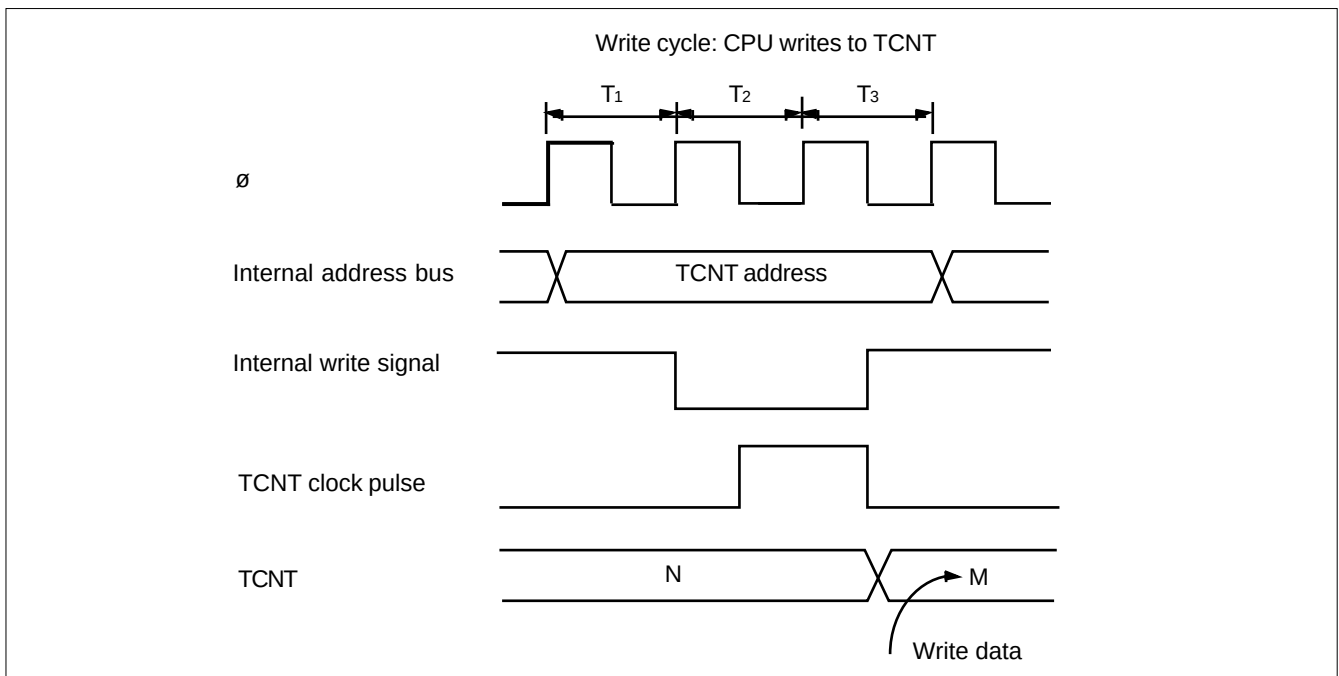


Figure 12-8 TCNT Write-Increment Contention

Changing the Clock Select Bits (CKS2 to CKS0): Software should stop the watchdog timer (by clearing the TME bit to 0) before changing the value of the clock select bits. If the clock select bits are modified while the watchdog timer is running, the timer count may be incremented incorrectly.

Section 13 Serial Communication Interface

13.1 Overview

The H8/520 chip includes a two-channel serial communication interface (SCI) for transferring serial data to and from other chips. The two channels are independent but are functionally identical.

Synchronous and asynchronous data transfer are supported on both channels.

13.1.1 Features

The features of the on-chip serial communication interface are as follows:

- Selection of asynchronous or synchronous mode
 - Asynchronous mode

The SCI can communicate with a UART (Universal Asynchronous Receiver/Transmitter), ACIA (Asynchronous Communication Interface Adapter), or other chip that employs standard asynchronous serial communication. Eight data formats are available.

 - Data length: 7 or 8 bits
 - Stop bit length: 1 or 2 bits
 - Parity: Even, odd, or none
 - Error detection: Parity, overrun, and framing errors
 - Synchronous mode

The SCI can communicate with chips able to synchronize data transfers with clock pulses.

 - Data length: 8 bits
 - Error detection: Overrun errors
- Full duplex communication

The transmitting and receiving sections are independent, so the SCI can transmit and receive simultaneously. Both the transmit and receive sections use double buffering, so continuous data transfer is possible in either direction.
- Built-in baud rate generator

Any specified baud rate can be generated.
- Internal or external clock source

The baud rate generator can operate on an internal clock source, or an external clock signal input at the SCK pin.
- Three interrupts

Transmit-end, receive-end, and receive-error interrupts are requested independently. The transmit-end and receive-end interrupts can be served by the on-chip data transfer controller (DTC), providing a convenient way to transfer data with minimal CPU programming.

13.1.2 Block Diagram

Figure 13-1 shows a block diagram of one serial communication interface channel.

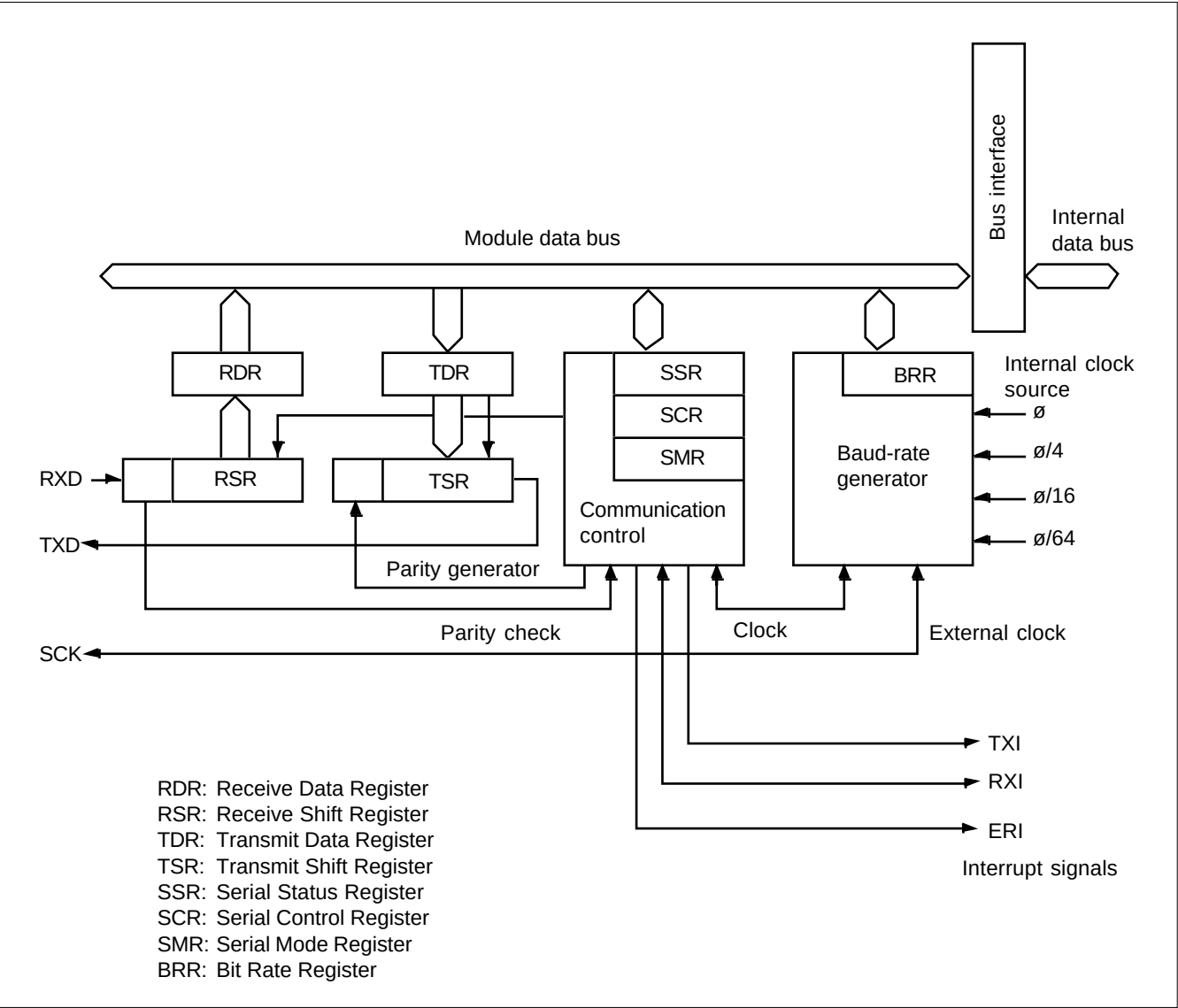


Figure 13-1 Block Diagram of Serial Communication Interface

13.1.3 Input and Output Pins

Table 13-1 lists the input and output pins used by the SCI module.

Table 13-1 SCI Input/Output Pins

Channel	Name	Abbreviation	I/O	Function
1	Serial clock	SCK ₁	Input/output	Serial clock input and output for channel 1
	Receive data	RXD ₁	Input	Receive data input for channel 1
	Transmit data	TXD ₁	Output	Transmit data output for channel 1
2	Serial clock	SCK ₂	Input/output	Serial clock input and output for channel 2
	Receive data	RXD ₂	Input	Receive data input for channel 2
	Transmit data	TXD ₂	Output	Transmit data output for channel 2

13.1.4 Register Configuration

Table 13-2 lists the SCI registers. These registers specify the communication mode (synchronous or asynchronous), data format, and bit rate, and control the transmit and receive sections.

Table 13-2 SCI Registers

Channel	Name	Abbreviation	R/W	Initial Value	Address
1	Receive shift register	RSR	—	—	—
	Receive data register	RDR	R	H'00	H'FFDD
	Transmit shift register	TSR	—	—	—
	Transmit data register	TDR	R/W	H'FF	H'FFDB
	Serial mode register	SMR	R/W	H'04	H'FFD8
	Serial control register	SCR	R/W	H'0C	H'FFDA
	Serial status register	SSR	R/(W)*	H'87	H'FFDC
	Bit rate register	BRR	R/W	H'FF	H'FFD9
2	Receive shift register	RSR	—	—	—
	Receive data register	RDR	R	H'00	H'FFC5
	Transmit shift register	TSR	—	—	—
	Transmit data register	TDR	R/W	H'FF	H'FFC3
	Serial mode register	SMR	R/W	H'04	H'FFC0
	Serial control register	SCR	R/W	H'0C	H'FFC2
	Serial status register	SSR	R/(W)*	H'87	H'FFC4
	Bit rate register	BRR	R/W	H'FF	H'FFC1

Note: * Software can write a 0 to clear the status flag bits, but cannot write a 1.

13.2 Register Descriptions

13.2.1 Receive Shift Register (RSR)

Bit	7	6	5	4	3	2	1	0
Read/Write	—	—	—	—	—	—	—	—

The RSR receives incoming data bits. When one character (one byte) has been received, it is transferred to the receive data register (RDR).

The CPU cannot read or write the RSR directly.

13.2.2 Receive Data Register (RDR)—H'FFDD (Channel 1), H'FFC5 (Channel 2)

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

The RDR stores received data. As each character is received, it is transferred from the RSR to the RDR, enabling the RSR to receive the next character. This double-buffering allows the SCI to receive data continuously.

The CPU can read but not write the RDR. The RDR is initialized to H'00 at a reset and in the standby modes.

13.2.3 Transmit Shift Register (TSR)

Bit	7	6	5	4	3	2	1	0
Read/Write	—	—	—	—	—	—	—	—

The TSR holds the character currently being transmitted. When transmission of this character is completed, the next character is moved from the transmit data register (TDR) to the TSR and transmission of that character begins. If the TDR does not contain valid data, the SCI stops transmitting.

The CPU cannot read or write the TSR directly.

13.2.4 Transmit Data Register (TDR)—H'FFDB (Channel 1), H'FFC3 (Channel 2)

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The TDR is an 8-bit readable/writable register that holds the next character to be transmitted. When the TSR becomes empty, the character written in the TDR is transferred to the TSR. Continuous data transmission is possible by writing the next byte in the TDR while the current byte is being transmitted from the TSR.

The TDR is initialized to H'FF at a reset and in the standby modes.

13.2.5 Serial Mode Register (SMR)—H'FFD8 (Channel 1), H'FFC0 (Channel 2)

Bit	7	6	5	4	3	2	1	0
	C/ $\overline{A}$	CHR	PE	O/ $\overline{E}$	STOP	—	CKS1	CKS0
Initial value	0	0	0	0	0	1	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	—	R/W	R/W

The SMR is an 8-bit readable/writable register that controls the communication format and selects the clock rate for the internal clock source. It is initialized to H'04 at a reset and in the standby modes.

Bit 7—Communication Mode (C/ $\overline{A}$): This bit selects the asynchronous or synchronous communication mode.

Bit 7	
C/ $\overline{A}$	Description
0	Asynchronous communication. (Initial value)
1	Communication is synchronized with the serial clock.

Bit 6—Character Length (CHR): This bit selects the character length in asynchronous mode. It is ignored in synchronous mode.

Bit 6

CHR	Description	
0	8 bits per character.	(Initial value)
1	7 bits per character.	

Bit 5—Parity Enable (PE): This bit selects whether to add a parity bit in asynchronous mode. It is ignored in synchronous mode.

Bit 5

PE	Description	
0	Transmit: No parity bit is added. Receive: Parity is not checked.	(Initial value)
1	Transmit: A parity bit is added. Receive: Parity is checked.	

Bit 4—Parity Mode (O/E): In asynchronous mode, when parity is enabled (PE = 1), this bit selects even or odd parity.

Even parity means that a parity bit is added to the data bits for each character to make the total number of 1s even. Odd parity means that the total number of 1s is made odd.

This bit is ignored when PE = 0 and in the synchronous mode.

Bit 4

O/E	Description	
0	Even parity.	(Initial value)
1	Odd parity.	

Bit 3—Stop Bit Length (STOP): This bit selects the number of stop bits. It is ignored in the synchronous mode.

Bit 3

STOP	Description	
0	1 stop bit.	(Initial value)
1	2 stop bits.	

Bit 2—Reserved: This bit cannot be modified and is always read as 1.

Bits 1 and 0—Clock Select 1 and 0 (CKS1 and CKS0): These bits select the internal clock source when the baud rate generator is clocked from within the H8/520 chip.

Bit 1	Bit 0	Description
CKS1	CKS0	
0	0	∅ clock (Initial value)
0	1	∅/4 clock
1	0	∅/16 clock
1	1	∅/64 clock

13.2.6 Serial Control Register (SCR)—H'FFDA (Channel 1), H'FFC2 (Channel 2)

Bit	7	6	5	4	3	2	1	0
	TIE	RIE	TE	RE	—	—	CKE1	CKE0
Initial value	0	0	0	0	1	1	0	0
Read/Write	R/W	R/W	R/W	R/W	—	—	R/W	R/W

The SCR is an 8-bit readable/writable register that enables or disables various SCI functions. It is initialized to H'0C at a reset and in the standby modes.

Bit 7—Transmit Interrupt Enable (RIE): This bit enables or disables the transmit-end interrupt (TXI) request when the transmit data register empty (TDRE) bit in the serial status register (SSR) is set to 1.

Bit 7	
TIE	Description
0	The transmit-end interrupt request (TXI) is disabled. (Initial value)
1	The transmit-end interrupt request (TXI) is enabled.

Bit 6—Receive Interrupt Enable (RIE): This bit enables or disables the receive-end interrupt (RXI) request when the receive data register full (RDRF) bit in the serial status register (SSR) is set to 1. It also enables and disables the receive-error interrupt (ERI) request.

Bit 6	
RIE	Description
0	The receive-end interrupt (RXI) and receive-error interrupt (ERI) requests are disabled. (Initial value)
1	The receive-end interrupt (RXI) and receive-error interrupt (ERI) requests are enabled.

Bit 5—Transmit Enable (TE): This bit enables or disables the transmit function. When the transmit function is enabled, the TXD pin is automatically used for output. When the transmit function is disabled, the TXD pin can be used as a general-purpose I/O port.

Bit 5

TE	Description
0	The transmit function is disabled. The TXD pin can be used as a general-purpose I/O port. (Initial value)
1	The transmit function is enabled. The TXD pin is used for output.

Bit 4—Receive Enable (RE): This bit enables or disables the receive function. When the receive function is enabled, the RXD pin is automatically used for input. When the receive function is disabled, the RXD pin is available as a general-purpose I/O port.

Bit 4

RE	Description
0	The receive function is disabled. The RXD pin can be used as a general-purpose I/O port. (Initial value)
1	The receive function is enabled. The RXD pin is used for input.

Bits 3 and 2—Reserved: These bits cannot be modified and are always read as 1.

Bit 1—Clock Enable 1 (CKE1): This bit selects the SCI clock source: either the internal baud rate generator or an external clock signal input at the SCK pin. When the external clock source is selected, the SCK pin is automatically used for input of the external clock signal.

Bit 1

CKE1	Description
0	Internal clock source. (Initial value)
1	External clock source. (The SCK pin is used for input.)

Bit 0—Clock Enable 0 (CKE0): When an internal clock source is used in synchronous mode, this bit enables or disables serial clock output at the SCK pin.

This bit is ignored when the external clock is selected, or when the asynchronous mode is selected.

For further information on the communication format and clock source selection, see tables 13-5 and 13-6 in section 13.3, “Operation”.

Bit 0

CKE0	Description
0	The SCK pin is not used by the SCI (and is available as a general-purpose I/O port). (Initial value)
1	The SCK pin is used for serial clock output.

13.2.7 Serial Status Register (SSR)—H'FFDC (Channel 1), H'FFC4 (Channel 2)

Bit	7	6	5	4	3	2	1	0
	TDRE	RDRF	ORER	FER	PER	—	—	—
Initial value	1	0	0	0	0	1	1	1
Read/Write	R/(W)*	R/(W)*	R/(W)*	R/(W)*	R/(W)*	R	R	R

Note: * Software can write a 0 to clear the flags, but cannot write a 1 in these bits.

The SSR is an 8-bit register that indicates transmit and receive status. It is initialized to H'87 at a reset and in the standby modes.

Bit 7—Transmit Data Register Empty (TDRE): This bit indicates when the TDR contents have been transferred to the TSR and the next character can safely be written in the TDR.

Bit 7

TDRE	Description
0	This bit is cleared from 1 to 0 when: 1. The CPU reads the TDRE bit after the TDRE bit has been set to 1, then writes a 0 in this bit. 2. The data transfer controller (DTC) writes data in the TDR.
1	This bit is set to 1 at the following times: (Initial value) 1. The chip is reset or enters a standby mode. 2. When TDR contents are transferred to the TSR. 3. When TDRE = 0 and the TE bit is cleared to 0.

Bit 6—Receive Data Register Full (RDRF): This bit indicates when one character has been received and transferred to the RDR.

Bit 6

RDRF	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The CPU reads the RDRF bit after the RDRF bit has been set to 1, then writes a 0 in this bit. 2. The data transfer controller (DTC) reads the RDR. 3. The chip is reset or enters a standby mode.
1	This bit is set to 1 when one character is received without error and transferred from the RSR to the RDR.

Bit 5—Overrun Error (ORER): This bit indicates an overrun error during reception.

Bit 5

ORER	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The CPU reads the ORER bit after the ORER bit has been set to 1, then writes a 0 in this bit. 2. The chip is reset or enters a standby mode.
1	This bit is set to 1 if reception of the next character ends while the receive data register is still full (RDRF = 1).

Bit 4—Framing Error (FER): This bit indicates a framing error during data reception in the asynchronous mode. It has no meaning in the synchronous mode.

Bit 4

FER	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The CPU reads the FER bit after the FER bit has been set to 1, then writes a 0 in this bit. 2. The chip is reset or enters a standby mode.
1	This bit is set to 1 if a framing error occurs (stop bit = 0).

Bit 3—Parity Error (PER): This bit indicates a parity error during data reception in the asynchronous mode, when a communication format with parity bits is used.

This bit has no meaning in the synchronous mode, or when a communication format without parity bits is used.

Bit 3	
PER	Description
0	This bit is cleared from 1 to 0 when: (Initial value) - The CPU reads the PER bit after the PER bit has been set to 1, then writes a 0 in this bit. - The chip is reset or enters a standby mode.
1	This bit is set to 1 when a parity error occurs (the parity of the received data does not match the parity selected by the O/ $\bar{E}$ bit in the SMR).

Bits 2 to 0—Reserved: These bits cannot be modified and are always read as 1.

13.2.8 Bit Rate Register (BRR)—H'FFD9 (Channel 1), H'FFC1 (Channel 2)

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The BRR is an 8-bit register that, together with the CKS1 and CKS0 bits in the SMR, determines the baud rate output by the baud rate generator.

The BRR is initialized to H'FF (the slowest rate) at a reset and in the standby modes.

Tables 13-3 and 13-5 show examples of BRR (N) and CKS (n) settings for commonly used bit rates. Different values can be set for each SCI channel. Table 13-4 indicates the maximum bit rates for various crystal oscillator frequencies in the asynchronous mode.

Table 13-3 Examples of BRR Settings in Asynchronous Mode (1)

Bit Rate	XTAL Frequency (MHz)											
	2			2.4576			4			4.194304		
	n	N	Error (%)	n	N	Error (%)	n	N	Error (%)	n	N	Error (%)
110	1	70	+0.03	1	86	+0.31	1	141	+0.03	1	148	−0.04
150	0	207	+0.16	0	255	0	1	103	+0.16	1	108	+0.21
300	0	103	+0.16	0	127	0	0	207	+0.16	0	217	+0.21
600	0	51	+0.16	0	63	0	0	103	+0.16	0	108	+0.21
1200	0	25	+0.16	0	31	0	0	51	+0.16	0	54	−0.70
2400	0	12	+0.16	0	15	0	0	25	+0.16	0	26	+1.14
4800	—	—	—	0	7	0	0	12	+0.16	0	13	−2.48
9600	—	—	—	0	3	0	—	—	—	—	—	—
19200	—	—	—	0	1	0	—	—	—	—	—	—
31250	—	—	—	—	—	—	0	1	0	—	—	—
38400	—	—	—	0	0	0	—	—	—	—	—	—

Table 13-3 Examples of BRR Settings in Asynchronous Mode (2)

Bit Rate	XTAL Frequency (MHz)											
	4.9152			6			7.3728			8		
	n	N	Error (%)	n	N	Error (%)	n	N	Error (%)	n	N	Error (%)
110	1	174	−0.26	2	52	+0.50	2	64	+0.70	2	70	+0.03
150	1	127	0	1	155	+0.16	1	191	0	1	207	+0.16
300	0	255	0	1	77	+0.16	1	95	0	1	103	+0.16
600	0	127	0	0	155	+0.16	0	191	0	0	207	+0.16
1200	0	63	0	0	77	+0.16	0	95	0	0	103	+0.16
2400	0	31	0	0	38	+0.16	0	47	0	0	51	+0.16
4800	0	15	0	0	19	−2.34	0	23	0	0	25	+0.16
9600	0	7	0	—	—	—	0	11	0	0	12	+0.16
19200	0	3	0	—	—	—	0	5	0	—	—	—
31250	—	—	—	0	2	0	—	—	—	0	3	0
38400	0	1	0	—	—	—	0	2	0	—	—	—

Table 13-3 Examples of BRR Settings in Asynchronous Mode (3)

Bit Rate	XTAL Frequency (MHz)											
	9.8304			10			12			12.288		
	n	N	Error (%)	n	N	Error (%)	n	N	Error (%)	n	N	Error (%)
110	2	86	+0.31	2	88	−0.25	2	106	−0.44	2	108	+0.88
150	1	255	0	2	64	+0.16	2	77	0	2	79	0
300	1	127	0	1	129	+0.16	1	155	0	1	159	0
600	0	255	0	1	64	+0.16	1	77	0	1	79	0
1200	0	127	0	0	129	+0.16	0	155	+0.16	0	159	0
2400	0	63	0	0	64	+0.16	0	77	+0.16	0	79	0
4800	0	31	0	0	32	−1.36	0	38	+0.16	0	39	0
9600	0	15	0	0	15	+1.73	0	19	−2.34	0	19	0
19200	0	7	0	0	7	+1.73	—	—	—	0	9	0
31250	0	4	−1.70	0	4	0	0	5	0	0	5	+2.40
38400	0	3	0	0	3	+1.73	—	—	—	0	4	0

Table 13-3 Examples of BRR Settings in Asynchronous Mode (4)

Bit Rate	XTAL Frequency (MHz)											
	14.7456			16			19.6608			20		
	n	N	Error (%)	n	N	Error (%)	n	N	Error (%)	n	N	Error (%)
110	2	130	−0.07	2	141	+0.03	2	174	−0.26	3	43	+0.88
150	2	95	0	2	103	+0.16	2	127	0	2	129	+0.16
300	1	191	0	1	207	+0.16	1	255	0	2	64	+0.16
600	1	95	0	1	103	+0.16	1	127	0	1	129	+0.16
1200	0	191	0	0	207	+0.16	0	255	0	1	64	+0.16
2400	0	95	0	0	103	+0.16	0	127	0	0	129	+0.16
4800	0	47	0	0	51	+0.16	0	63	0	0	64	+0.16
9600	0	23	0	0	25	+0.16	0	31	0	0	32	−1.36
19200	0	11	0	0	12	+0.16	0	15	0	0	15	+1.73
31250	—	—	—	0	7	0	0	9	−1.70	0	9	0
38400	0	5	0	—	—	—	0	7	0	0	7	+1.73
307200	—	—	—	—	—	—	0	0	0	—	—	—
312500	—	—	—	—	—	—	—	—	—	0	0	0

Note: If possible, the error should be within 1%.

$$B = OSC \times 10^6 / [64 \times 2^{2n} \times (N + 1)]$$

- B: Bit rate (bits/s)
- N: BRR value (0 ≤ N ≤ 255)
- OSC: Crystal oscillator frequency in MHz
- n: Internal clock source (0, 1, 2, or 3)

The meaning of n is given by the table below:

n	CKS1	CKS0	Clock
0	0	0	∅
1	0	1	∅/4
2	1	0	∅/16
3	1	1	∅/64

Table 13-4 Maximum Bit Rate for Various Crystal Oscillator Frequencies (In Asynchronous Mode)

XTAL (MHz)	Maximum Bit Rate (bits/s)	CKS and BRR	
		n	N
2	31250	0	0
2.4576	38400	0	0
4	62500	0	0
4.194304	65536	0	0
4.9152	76800	0	0
6	93750	0	0
7.3728	115200	0	0
8	125000	0	0
9.8304	153600	0	0
10	156250	0	0
12	187500	0	0
12.288	192000	0	0
14.7456	230400	0	0
16	250000	0	0
19.6608	307200	0	0
20	312500	0	0

Table 13-5 Examples of BRR Settings in Synchronous Mode

Bit	XTAL Frequency (MHz)											
	2		4		8		10		16		20	
Rate	n	N	n	N	n	N	n	N	n	N	n	N
100	—	—	—	—	—	—	—	—	—	—	—	—
250	1	249	2	124	2	249	—	—	3	124	—	—
500	1	124	1	249	2	124	—	—	2	249	—	—
1 k	0	249	1	124	1	249	—	—	2	124	—	—
2.5 k	0	99	0	199	1	99	1	124	1	199	1	249
5 k	0	49	0	99	0	199	0	249	1	99	1	124
10 k	0	24	0	49	0	99	0	124	0	199	0	249
25 k	0	9	0	19	0	39	0	49	0	79	0	99
50 k	0	4	0	9	0	19	0	24	0	39	0	49
100 k	—	—	0	4	0	9	—	—	0	19	0	24
250 k	0	0*	0	1	0	3	0	4	0	7	0	9
500 k			0	0*	0	1	—	—	0	3	0	4
1 M					0	0*	—	—	0	1	—	—
2.5 M											0	0*

Notes: Blank: No setting is available.

—: A setting is available, but the bit rate is inaccurate.

*: Continuous transfer is not possible.

$$B = OSC / [8 \times 2^{2n} \times (N + 1)]$$

B: Bit rate (bits/s)

N: BRR value ($0 \leq N \leq 255$)

OSC: Crystal oscillator frequency in MHz

n: Internal clock source (0, 1, 2, or 3)

The meaning of n is given by the table below:

n	CKS1	CKS0	Clock
0	0	0	Ø
1	0	1	Ø/4
2	1	0	Ø/16
3	1	1	Ø/64

13.3 Operation

13.3.1 Overview

The SCI supports serial data transfer in both asynchronous and synchronous modes.

The communication format depends on settings in the SMR as indicated in table 13-6. The clock source and usage of the SCK pin depend on settings in the SMR and SCR as indicated in table 13-7.

Table 13-6 Communication Formats Used by SCI

SMR				Mode	Format	Parity	Stop Bit
C/A	CHR	PE	STOP				Length
0	0	0	0	Asynchronous	8-Bit data	None	1
			1				2
		1	0			Yes	1
			1				2
	1	0	0		7-Bit data	None	1
			1				2
		1	0			Yes	1
			1				2
1	—	—	—	Synchronous	8-Bit data	—	—

Table 13-7 SCI Clock Source Selection

SMR	SCR		Clock	
C/A	CKE1	CKE0	Source	SCK Pin
0 (Async mode)	0	0	Internal	I/O port*
		1		Clock output at same frequency as bit rate
	1	0	External	Clock input at 16 times the bit rate frequency
		1		
1 (Sync mode)	0	0	Internal	Serial clock output
		1		
	1	0	External	Serial clock input
		1		

Note: * Not used by the SCI.

Transmitting and receiving operations in the two modes are described next.

13.3.2 Asynchronous Mode

In asynchronous mode, each character is individually synchronized by framing it with a start bit and stop bit.

Full duplex data transfer is possible because the SCI has independent transmit and receive sections. Double buffering in both sections enables the SCI to be programmed for continuous data transfer.

Figure 13-2 shows the general format of one character sent or received in the asynchronous mode. The communication channel is normally held in the mark state (high). Character transmission or reception starts with a transition to the space state (low).

The first bit transmitted or received is the start bit (low). It is followed by the data bits, in which the least significant bit (LSB) comes first. The data bits are followed by the parity bit, if present, then the stop bit or bits (high) confirming the end of the frame.

In receiving, the SCI synchronizes on the falling edge of the start bit, and samples each bit at the center of the bit (at the 8th cycle of the internal serial clock, which runs at 16 times the bit rate).

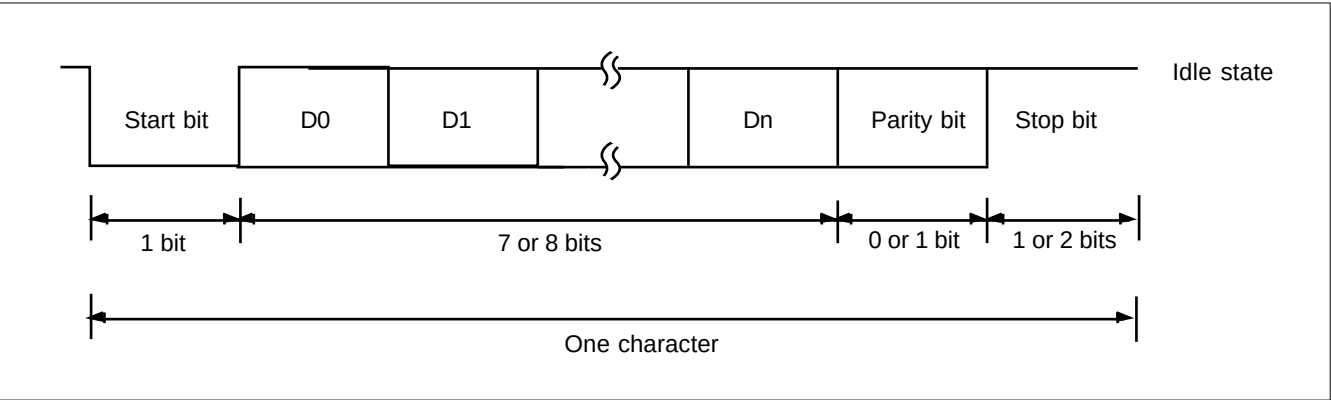


Figure 13-2 Data Format in Asynchronous Mode

Data Format: Table 13-8 lists the data formats that can be sent and received in asynchronous mode. Eight formats can be selected by bits in the SMR.

Table 13-8 Data Formats in Asynchronous Mode

SMR Bits			Data Format				
CHR	PE	STOP					
0	0	0	START	8-Bit data	STOP		
0	0	1	START	8-Bit data	STOP	STOP	
0	1	0	START	8-Bit data	P	STOP	
0	1	1	START	8-Bit data	P	STOP	STOP
1	0	0	START	7-Bit data	STOP		
1	0	1	START	7-Bit data	STOP	STOP	
1	1	0	START	7-Bit data	P	STOP	
1	1	1	START	7-Bit data	P	STOP	STOP

Note: START: Start bit
STOP: Stop bit
P: Parity bit

Clock: In the asynchronous mode it is possible to select either an internal clock created by the on-chip baud rate generator, or an external clock input at the SCK pin. Refer to table 13-7.

If an external clock is input at the SCK pin, its frequency should be 16 times the desired bit rate.

If the internal clock provided by the on-chip baud rate generator is selected and the SCK pin is used for clock output, the output clock frequency is equal to the bit rate, and the clock pulse rises at the center of the transmit data bits. Figure 13-3 shows the phase relationship between the output clock and transmit data.

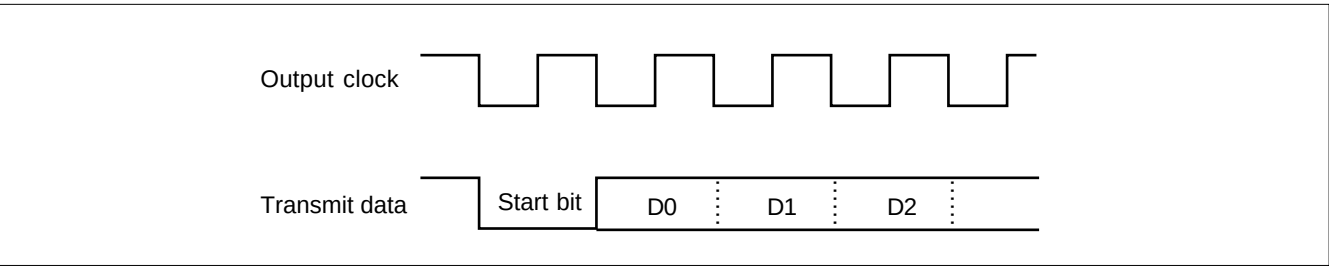


Figure 13-3 Phase Relationship Between Clock Output and Transmit Data

SCI Initialization: Before data can be transmitted or received, the SCI must be initialized by software. To initialize the SCI, software must clear the TE and RE bits to 0, then execute the following procedure.

1. Set the desired communication format in the SMR.
2. Write the value corresponding to the desired bit rate in the BRR. (This step is not necessary if an external clock is used.)
3. Select the clock and enable desired interrupts in the SCR.
4. Set the TE and/or RE bit in the SCR to 1.

The TE and RE bits must both be cleared to 0 whenever the operating mode or data format is changed.

After changing the operating mode or data format, before setting the TE and RE bits to 1 software must wait for at least 1 bit transfer time at the selected communication speed, to make sure the SCI is initialized. If an external clock is used, the clock must not be stopped.

When clearing the TDRE bit during data transmission, to assure transfer of the correct data, do not clear the TDRE bit until after writing data in the TDR. Similarly, in receiving data, do not clear the RDRF bit until after reading data from the RDR.

Data Transmission: The procedure for transmitting data in the asynchronous mode is as follows.

1. Set up the desired transmitting conditions in the SMR, SCR, and BRR.
2. Set the TE bit in the SCR to 1.
The TXD pin will automatically be switched to output and one frame* of all 1s will be transmitted, after which the SCI is ready to transmit data.
Note: * A frame is the data for one character, including the start bit and stop bit(s).
3. Check that the TDRE bit is set to 1, then write the first byte of transmit data in the TDR. Next clear the TDRE bit to 0.
4. The first byte of transmit data is transferred from the TDR to the TSR and sent in the designated format as follows.
 - a. Start bit (one 0 bit).
 - b. Transmit data (seven or eight bits, starting from bit 0)

- c. Parity bit (odd or even parity bit, or no parity bit)
 - d. Stop bit (one or two consecutive 1 bits)
5. Transfer of the transmit data from the TDR to the TSR makes the TDR empty, so the TDRE bit is set to 1.
- If the TIE bit is set to 1, a transmit-end interrupt (TXI) is requested.
- When the transmit function is enabled but the TDR is empty (TDRE = 1), the output at the TXD pin is held at 1 until the TDRE bit is cleared to 0.

Data Reception: The procedure for receiving data in the asynchronous mode is as follows.

1. Set up the desired receiving conditions in the SMR, SCR, and BRR.
2. Set the RE bit in the SCR to 1.
The RXD pin will automatically be switched to input and the SCI is ready to receive data.
3. The SCI synchronizes with the incoming data by detecting the start bit, and places the received bits in the RSR. At the end of the data, the SCI checks that the stop bit is 1.
If the stop bit length is 2 bits, the SCI checks that both bits are 1.
4. When a complete frame has been received, the SCI transfers the received data to the RDR so that it can be read. If the character length is 7 bits, the most significant bit of the RDR is cleared to 0.
At the same time, the SCI sets the RDRF bit in the SSR to 1. If the RIE bit is set to 1, a receive-end interrupt (RXI) is requested.
5. The RDRF bit is cleared to 0 when the CPU reads the SSR, then writes a 0 in the RDRF bit, or when the RDR is read by the data transfer controller (DTC). The RDR is then ready to receive the next character from the RSR.

When a frame is not received correctly, a receive error occurs. There are three types of receive errors, listed in table 13-9.

If a receive error occurs, the RDRF bit in the SSR is not set to 1. The corresponding error flag is set to 1 instead. If the RIE bit in the SCR is set to 1, a receive-error interrupt (ERI) is requested.

When a framing or parity error occurs, the RSR contents are transferred to the RDR. If an overrun error occurs, however, the RSR contents are not transferred to the RDR.

If multiple receive errors occur simultaneously, all the corresponding error flags are set to 1. See section 13.5, “Application Notes”.

To clear a receive-error flag (ORER, FER, or PER), software must read the SSR, then write a 0 in the flag bit.

Table 13-9 Receive Errors

Name	Abbreviation	Description
Overrun error	ORER	Reception of the next frame ends while the RDRF bit is still set to 1. The RSR contents are not transferred to the RDR.
Framing error	FER	A stop bit is 0. The RSR contents are transferred to the RDR.
Parity error	PER	The parity of a frame does not match the value selected by the bit in the SMR. The RSR contents are transferred to the RDR.

13.3.3 Synchronous Mode

The synchronous mode is suited for high-speed, continuous data transfer. Each bit of data is synchronized with a serial clock pulse.

Continuous data transfer is enabled by the double buffering employed in both the transmit and receive sections of the SCI. Full duplex communication (with the same clock) is possible because the transmit and receive sections are independent.

Data Format: Figure 13-4 shows the communication format used in the synchronous mode. The data length is 8 bits for both the transmit and receive directions. The least significant bit (LSB) is sent and received first. Each bit of transmit data is output from the falling edge of the serial clock pulse to the next falling edge. Received bits are latched on the rising edge of the serial clock pulse.

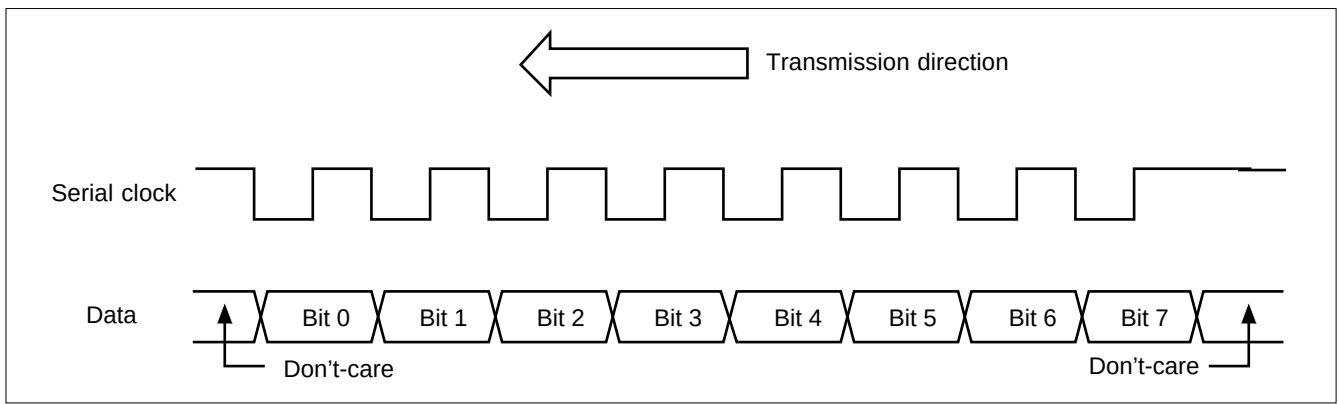


Figure 13-4 Data Format in Synchronous Mode

Clock: Either the internal serial clock created by the on-chip baud rate generator or an external clock input at the SCK pin can be selected in the synchronous mode. See table 13-7 for details.

SCI Initialization: Before data can be transmitted or received, the SCI must be initialized by software. To initialize the SCI, software must clear the TE and RE bits to 0 to disable both the transmit and receive functions, then execute the following procedure.

1. Write the value corresponding to the desired bit rate in the BRR. (This step is not necessary if an external clock is used.)
2. Select the clock and enable desired interrupts in the SCR.
3. Select the synchronous mode in the SMR.
4. Set the TE and/or RE bit in the SCR to 1.

Note: The input/output status of the SCK pin depends on the $\overline{C/A}$ bit in the SMR and the CKE0 and CKE1 bits in the SCR. (See table 13-7.) To prevent incorrect output from the SCK pin, set the SCR before the SMR.

The TE and RE bits must both be cleared to 0 whenever the operating mode or data format is changed. After changing the operating mode or data format, before setting the TE and RE bits to 1 software must wait for at least 1 bit transfer time at the selected communication speed, to make sure the SCI is initialized.

When clearing the TDRE bit during data transmission, to assure correct data transfer, do not clear the TDRE bit until after writing data in the TDR. Similarly, in receiving data, do not clear the RDRF bit until after reading data from the RDR.

Data Transmission: The procedure for transmitting data in the synchronous mode is as follows.

1. Set up the desired transmitting conditions in the SMR, BRR, and SCR.
2. Set the TE bit in the SCR to 1.
The TXD pin will automatically be switched to output, after which the SCI is ready to transmit data.
3. Check that the TDRE bit in the SSR is set to 1, then write the first byte of transmit data in the TDR. Next clear the TDRE bit to 0.
4. The first byte of transmit data is transferred from the TDR to the TSR and sent, each bit synchronized with a clock pulse. Bit 0 is sent first.
Transfer of the transmit data from the TDR to the TSR makes the TDR empty, so the TDRE bit is set to 1. If the TIE bit is set to 1, a transmit-end interrupt (TXI) is requested.

The TDR and TSR function as a double buffer. Continuous data transmission can be achieved by writing the next transmit data in the TDR and clearing the TDRE bit to 0 while the SCI is transmitting the current data from the TSR.

If an internal clock source is selected, after transferring the transmit data from the TDR to the TSR, while transmitting the data from the TSR the SCI also outputs a serial clock signal at the SCK pin. When all data bits in the TSR have been transmitted, if the TDR is empty (TDRE = 1), serial clock output is suspended until the next data byte is written in the TDR and the TDRE bit is cleared to 0. During this interval the TXD pin is held at the value of the last bit transmitted.

If the external clock source is selected, data transmission is synchronized with the clock signal input at the SCK pin. When all data bits in the TSR have been transmitted, if the TDR is empty (TDRE = 1) but external clock pulses continue to arrive, the TXD pin outputs a string of bits equal to the last bit transmitted.

Data Reception: The procedure for receiving data in the synchronous mode is as follows.

1. Set up the desired receiving conditions in the SMR, BRR, and SCR.

2. Set the RE bit in the SCR to 1.

The RXD pin will automatically be switched to input and the SCI is ready to receive data.

3. Incoming data bits are latched in the RSR on eight clock pulses.

When 8 bits of data have been received, the SCI sets the RDRF bit in the SSR to 1. If the RIE bit is set to 1, a receive-end interrupt (RXI) is requested.

4. The SCI transfers the received data byte to the RDR so that it can be read.

The RDRF bit is cleared when the program reads the RDRF bit in the SSR, then writes a 0 in the RDRF bit, or when the data transfer controller (DTC) reads the RDR.

The RDR and RSR function as a double buffer. Data can be received continuously by reading each byte of data from the RDR and clearing the RDRF bit to 0 before the last bit of the next byte is received.

In general, an external clock source should be used for receiving data.

If an internal clock source is selected, the SCI starts receiving data as soon as the RE bit is set to 1. The serial clock is also output at the SCK pin. The SCI continues receiving until the RE bit is cleared to 0.

If the last bit of the next data byte is received while the RDRF bit is still set to 1, an overrun error occurs and the ORER bit is set to 1. If the RIE bit is set to 1, a receive-error interrupt (ERI) is requested. The data received in the RSR are not transferred to the RDR when an overrun error occurs.

After an overrun error, reception of the next data is enabled when the ORER bit is cleared to 0.

Simultaneous Transmit and Receive: The procedure for transmitting and receiving simultaneously in the synchronous mode is as follows:

1. Set up the desired communication conditions in the SMR, BRR, and SCR.

2. Set the TE and RE bits in the SCR to 1.

The TXD and RXD pins are automatically switched to output and input, respectively, and the SCI is ready to transmit and receive data.

3. Data transmitting and receiving start when the TDRE bit in the SSR is cleared to 0.

4. Data are sent and received in synchronization with eight clock pulses.
5. First, the transmit data are transferred from the TDR to the TSR. This makes the TDR empty, so the TDRE bit is set to 1. If the TIE bit is set to 1, a transmit-end interrupt (TXI) is requested. If continuous data transmission is desired, the CPU must read the TDRE bit in the SSR, write the next transmit data in the TDR, then clear the TDRE bit to 0. Alternatively, the DTC can write the next transmit data in the TDR, in which case the TDRE bit is cleared automatically. If the TDRE bit is not cleared to 0 by the time the SCI finishes sending the current byte from the TSR, the TXD pin continues to output the last bit in the TSR.
6. In the receiving section, when 8 bits of data have been received they are transferred from the RSR to the RDR, and the RDRF bit in the SSR is set to 1. If the RIE bit is set to 1, a receive-end interrupt (RXI) is requested.
7. To clear the RDRF bit software must read the RDRF bit in the SSR, then write a 0 in the RDRF bit. Alternatively, the DTC can read the RDR, in which case the RDRF bit is cleared automatically. For continuous data reception, the RDRF bit must be cleared to 0 before the last bit of the next byte of data is received.

If the last bit of the next data byte is received while the RDRF bit is still set to 1, an overrun error occurs and the ORER bit is set to 1. If the RIE bit is set to 1, a receive-error interrupt (ERI) is requested. The data received in the RSR are not transferred to the RDR when an overrun error occurs.

After an overrun error, reception of the next data is enabled when the ORER bit is cleared to 0.

An overrun error does not affect the transmit section of the SCI, which continues to transmit normally.

13.4 CPU Interrupts and DTC Interrupts

The SCI can request three types of interrupts: transmit-end (TXI), receive-end (RXI), and receive-error (ERI). Interrupt requests are enabled or disabled by the TIE and RIE bits in the SCR.

Independent signals are sent to the interrupt controller for each type of interrupt. The transmit-end and receive-end interrupt request signals are obtained from the TDRE and RDRF flags. The receive-error interrupt request signal is the logical OR of the three error flags: overrun error (ORER), framing error (FER), and parity error (PER). Table 13-10 lists information about these interrupts.

Table 13-10 SCI Interrupts

Interrupt	Description	DTC Service Available?	Priority
ERI	Receive-error interrupt, requested when ORER, FER, or PER is set.	No	High ↑ Low
RXI	Receive-end interrupt, requested when RDRF is set.	Yes	
TXI	Transmit-end interrupt, requested when TDRE is set.	Yes	

The TXI and RXI interrupts can be served by the data transfer controller (DTC) to have a data transfer performed. When the DTC serves one of these interrupts, it clears the TDRE or RDRF bit to 0 under the following conditions, which differ between the two bits.

When invoked by a TXI request, if the DTC writes to the TDR, it automatically clears the TDRE bit to 0. When invoked by an RXI request, if the DTC reads from the RDR, it automatically clears the RDRF bit to 0.

See section 6, “Data Transfer Controller”, for further information on the DTC.

13.5 Application Notes

Application programmers should note the following features of the SCI.

TDR Write: The TDRE bit in the SSR is simply a flag that indicates that the TDR contents have been transferred to the TSR. The TDR contents can be rewritten regardless of the TDRE value. If a new byte is written in the TDR while the TDRE bit is 0, before the old TDR contents have been moved into the TSR, the old byte will be lost. Normally, software should check that the TDRE bit is set to 1 before writing to the TDR.

Multiple Receive Errors: Table 13-11 lists the values of flag bits in the SSR when multiple receive errors occur, and indicates whether the RSR contents are transferred to the RDR.

Table 13-11 SSR Bit States and Data Transfer When Multiple Receive Errors Occur

Receive Error	SSR Bits				RSR to RDR*2
	RDRF	ORER	FER	PER	
Overflow error	1*1	1	0	0	No
Framing error	0	0	1	0	Yes
Parity error	0	0	0	1	Yes
Overflow + framing errors	1*1	1	1	0	No
Overflow + parity errors	1*1	1	0	1	No
Framing + parity errors	0	0	1	1	Yes
Overflow + framing + parity errors	1*1	1	1	1	No

Notes: 1. Set to 1 before the overflow error occurs.
 2. Yes: The RSR contents are transferred to the RDR.
 No: The RSR contents are not transferred to the RDR.

Line Break Detection: When the RXD pin receives a continuous stream of 0s in the asynchronous mode (line-break state), a framing error occurs because the SCI detects a 0 stop bit. The value H'00 is transferred from the RSR to the RDR. Software can detect the line-break state as a framing error accompanied by H'00 data in the RDR.

The SCI continues to receive data, so if the FER bit is cleared to 0 another framing error will occur.

Sampling Timing and Receive Margin in Asynchronous Mode: The serial clock used by the SCI in asynchronous mode runs at 16 times the bit rate. The falling edge of the start bit is detected by sampling the RXD input on the falling edge of this clock. After the start bit is detected, each bit of receive data in the frame (including the start bit, parity bit, and stop bit or bits) is sampled on the rising edge of the serial clock pulse at the center of the bit. See figure 13-5.

It follows that the receive margin can be calculated as in equation (1).

When the absolute frequency deviation of the clock signal is 0 and the clock duty factor is 0.5, data can theoretically be received with distortion up to the margin given by equation (2). This is a theoretical limit, however. In practice, system designers should allow a margin of 20% to 30%.

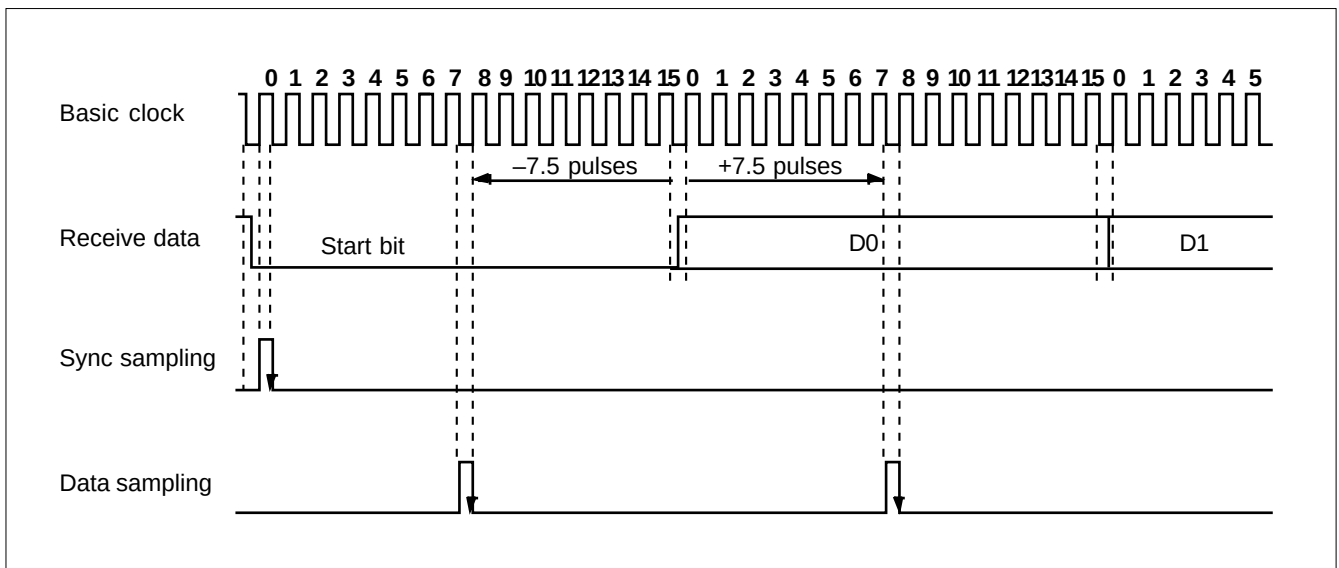


Figure 13-5 Sampling Timing (Asynchronous Mode)

$$M = \{ (0.5 - 1/2N) - (D - 0.5)/N - (L - 0.5)F \} \times 100 \text{ [\%]} \quad (1)$$

M: Receive margin

N: Ratio of serial clock to bit rate ($N = 16$)

D: Duty cycle of high or low clock pulses, whichever is longer (0.5 to 1.0)

L: Frame length (9 to 12)

F: Absolute value of clock frequency deviation

When $D = 0.5$ and $F = 0$:

$$M = (0.5 - 1/2 \times 16) \times 100 \text{ [\%]} = 46.875\% \quad (2)$$

Section 14 A/D Converter

14.1 Overview

The H8/520 chip includes an analog-to-digital converter module which can be programmed for input of analog signals on up to four (or eight*) channels. A/D conversion is performed by the successive approximations method with 10-bit resolution.

14.1.1 Features

The features of the on-chip A/D module are as follows:

- Four (or eight*) analog input channels
- External trigger
A/D conversion can be started by an external trigger input.
- Sample and hold circuit
- 10-Bit resolution
- Rapid conversion
Conversion time is 13.8 μ s per channel (at $\phi = 10$ MHz)
- Single and scan modes
 - Single mode: A/D conversion is performed once.
 - Scan mode: A/D conversion is performed in a repeated cycle on one to four channels.
- Four 16-bit data registers
These registers store A/D conversion results for up to four channels.
- A CPU interrupt (ADI) can be requested at the completion of each A/D conversion cycle.
This interrupt can also be served by the on-chip data transfer controller (DTC), providing a convenient way to move results into memory.

Note: * CP-68 package only

14.1.2 Block Diagram

Figure 14-1 shows a block diagram of A/D converter.

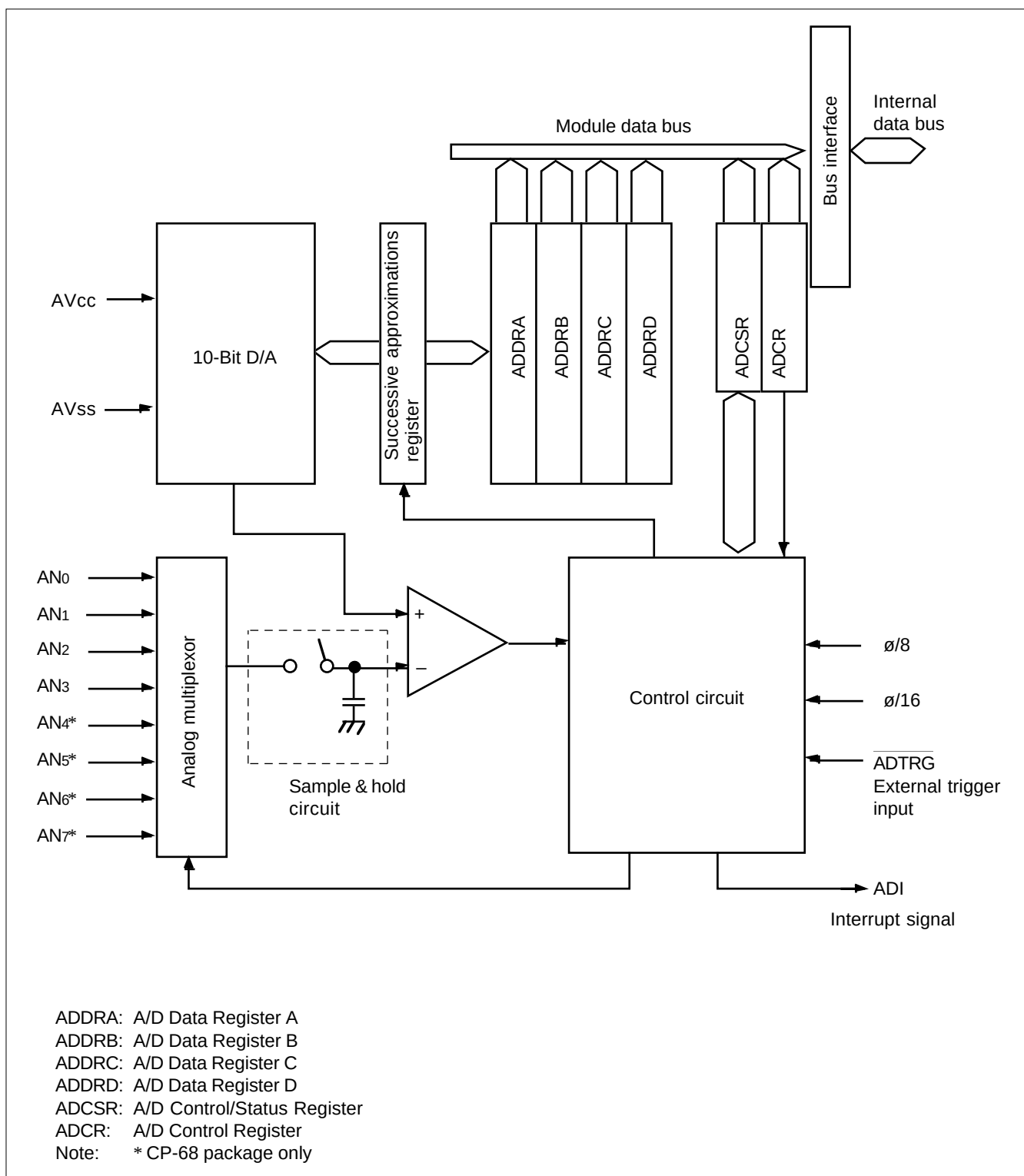


Figure 14-1 Block Diagram of A/D Converter

14.1.3 Input Pins

Table 14-1 lists the input pins used by the A/D converter module.

The eight analog input pins provided in the CP-68 package are divided into two groups, consisting of analog inputs 0 to 3 (AN₀ to AN₃) and analog inputs 4 to 7 (AN₄ to AN₇), respectively.

Table 14-1 A/D Input Pins

Name	Abbreviation	I/O	Function
Analog supply voltage	AV _{CC}	Input	Power supply and reference voltage for the analog circuits.
Analog ground	AV _{SS}	Input	Ground and reference voltage for the analog circuits.
Analog input 0	AN ₀	Input	Analog input pins, group 0
Analog input 1	AN ₁	Input	
Analog input 2	AN ₂	Input	
Analog input 3	AN ₃	Input	
Analog input 4	AN ₄	Input	Analog input pins, group 1* ¹
Analog input 5	AN ₅	Input	
Analog input 6	AN ₆	Input	
Analog input 7	AN ₇	Input	
A/D external trigger input	<u>ADTRG</u>	Input	External trigger for starting A/D conversion* ²

Notes: 1. CP-68 package only.

2. Not available in MCU mode 3 because this pin is used for the page address bus (A₁₈).

14.1.4 Register Configuration

Table 14-2 lists the registers of the A/D converter module.

Table 14-2 A/D Registers

Name	Abbreviation	R/W	Initial Value	Address
A/D data register A (High)	ADDRA (H)	R	H'00	H'FFE0
A/D data register A (Low)	ADDRA (L)	R	H'00	H'FFE1
A/D data register B (High)	ADDRB (H)	R	H'00	H'FFE2
A/D data register B (Low)	ADDRB (L)	R	H'00	H'FFE3
A/D data register C (High)	ADDRC (H)	R	H'00	H'FFE4
A/D data register C (Low)	ADDRC (L)	R	H'00	H'FFE5
A/D data register D (High)	ADDRD (H)	R	H'00	H'FFE6
A/D data register D (Low)	ADDRD (L)	R	H'00	H'FFE7
A/D control/status register	ADCSR	R/(W)*	H'00	H'FFE8
A/D control register	ADCR	R/W	H'7F	H'FFE9

Note: * Software can write a 0 to clear the status flag in bit 7 but cannot write a 1.

14.2 Register Descriptions

14.2.1 A/D Data Registers (ADDR)—H'FFE0 to H'FFE7

Bit	7	6	5	4	3	2	1	0
ADDRn H	AD ₉	AD ₈	AD ₇	AD ₆	AD ₅	AD ₄	AD ₃	AD ₂
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

(n = A to D)

Bit	7	6	5	4	3	2	1	0
ADDRn L	AD ₁	AD ₀	—	—	—	—	—	—
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

(n = A to D)

The four A/D data registers (ADDRA to ADDRD) are 16-bit read-only registers that store the results of A/D conversion.

Each result consist of 10 bits. The first 8 bits are stored in the upper byte of the data register corresponding to the selected channel. The last two bits are stored in the lower data register byte. The data registers are assigned to analog input channels as indicated in table 14-3.

The A/D data registers are always readable by the CPU. The upper byte can be read directly. The lower byte is read via a temporary register. See section 14-3, “CPU Interface”, for details.

The unused bits (bits 5 to 0) of the lower data register byte are always read as 0.

The A/D data registers are initialized to H'0000 at a reset and in the standby modes.

Table 14-3 Assignment of Data Registers to Analog Input Channels

Analog Input Channel		
Group 0	Group 1 *	A/D Data Register
AN0	AN4	ADDRA
AN1	AN5	ADDRB
AN2	AN6	ADDRC
AN3	AN7	ADDRD

Note: * CP-68 package only.

14.2.2 A/D Control/Status Register (ADCSR)—H'FFE8

Bit	7	6	5	4	3	2	1	0
	ADF	ADIE	ADST	SCAN	CKS	CH2	CH1	CH0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/(W)*	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Note: * Software can write a 0 in bit 7 to clear the flag, but cannot write a 1 in this bit.

The A/D control/status register (ADCSR) is an 8-bit readable/writable register that controls the operation of the A/D converter module.

The ADCSR is initialized to H'00 at a reset and in the standby modes.

Bit 7—A/D End Flag (ADF): This status flag indicates the end of one cycle of A/D conversion.

Bit 7

ADF	Description
0	This bit is cleared from 1 to 0 when: (Initial value) 1. The chip is reset or placed in a standby mode. 2. The CPU reads the ADF bit after the ADF bit is set to 1, then writes a 0 in this bit. 3. An A/D interrupt is served by the data transfer controller (DTC).
1	This bit is set to 1 at the following times: 1. Single mode: when one A/D conversion is completed. 2. Scan mode: when inputs on all selected channels have been converted.

Bit 6—A/D Interrupt Enable (ADI): This bit selects whether to request an A/D interrupt (ADI) when A/D conversion is completed.

Bit 6

ADIE	Description
0	The A/D interrupt request (ADI) is disabled. (Initial value)
1	The A/D interrupt request (ADI) is enabled.

Bit 5—A/D Start (ADST): The A/D converter operates while this bit is set to 1. In the single mode, this bit is automatically cleared to 0 at the end of each A/D conversion.

Bit 5

ADST	Description
0	A/D conversion is halted. (Initial value)
1	1. Single mode: One A/D conversion is performed. The ADST bit is automatically cleared to 0 at the end of the conversion. 2. Scan mode: A/D conversion starts and continues cyclically on the selected channels until the ADST bit is cleared to 0.

Bit 4—Scan Mode (SCAN): This bit selects the scan mode or single mode of operation.

See section 14.4, “Operation”, for descriptions of these modes.

The mode should be changed only when the ADST bit is cleared to 0.

Bit 4

SCAN	Description
0	Single mode (Initial value)
1	Scan mode

Bit 3—Clock Select (CKS): This bit controls the A/D conversion time.

The conversion time should be changed only when the ADST bit is cleared to 0.

Bit 3

CKS	Description		
0	Conversion time = 274 states	(maximum)	(Initial value)
1	Conversion time = 138 states	(maximum)	

Bits 2 to 0—Channel Select 2 to 0 (CH2 to CH0): These bits and the SCAN bit combine to select one or more analog input channels.

The channel selection should be changed only when the ADST bit is cleared to 0.

Group Select	Channel Select		Selected Channels	
	CH1	CH0	Single Mode	Scan Mode
0	0	0	AN ₀	AN ₀
	0	1	AN ₁	AN ₀ and AN ₁
	1	0	AN ₂	AN ₀ to AN ₂
	1	1	AN ₃	AN ₀ to AN ₃
1	0	0	AN ₄ *	AN ₄ *
	0	1	AN ₅ *	AN ₄ and AN ₅ *
	1	0	AN ₆ *	AN ₄ to AN ₆ *
	1	1	AN ₇ *	AN ₄ to AN ₇ *

Note: * CP-68 package only

14.2.3 A/D Control Register (ADCR)—H'FFE9

Bit	7	6	5	4	3	2	1	0
	TRGE	—	—	—	—	—	—	—
Initial value	0	1	1	1	1	1	1	1
Read/Write	R/W	—	—	—	—	—	—	—

The A/D control register (ADCR) is an 8-bit readable/writable register that enables or disables the A/D external trigger signal.

The ADCR is initialized to H'7F at a reset and in the standby modes.

Bit 7—Trigger Enable (TRGE): This bit enables the $\overline{\text{ADTRG}}$ (A/D external trigger) signal. When enabled, a high-to-low transition of $\overline{\text{ADTRG}}$ sets the ADST bit, starting A/D conversion.

Bit 7

TRGE	Description
0	A/D external trigger is disabled. $\overline{\text{ADTRG}}$ does not set the ADST bit. (Initial value)
1	A/D external trigger is enabled. A high-to-low transition of $\overline{\text{ADTRG}}$ sets the ADST bit.

Bits 6 to 0—Reserved: These bits cannot be modified and are always read as 1.

14.3 CPU Interface

The A/D data registers (ADDRA to ADDR D) are 16-bit registers, but they are accessed via an 8-bit module data bus. Accordingly, the upper byte of each register can be read directly, but the lower byte is accessed through an 8-bit temporary register (TEMP).

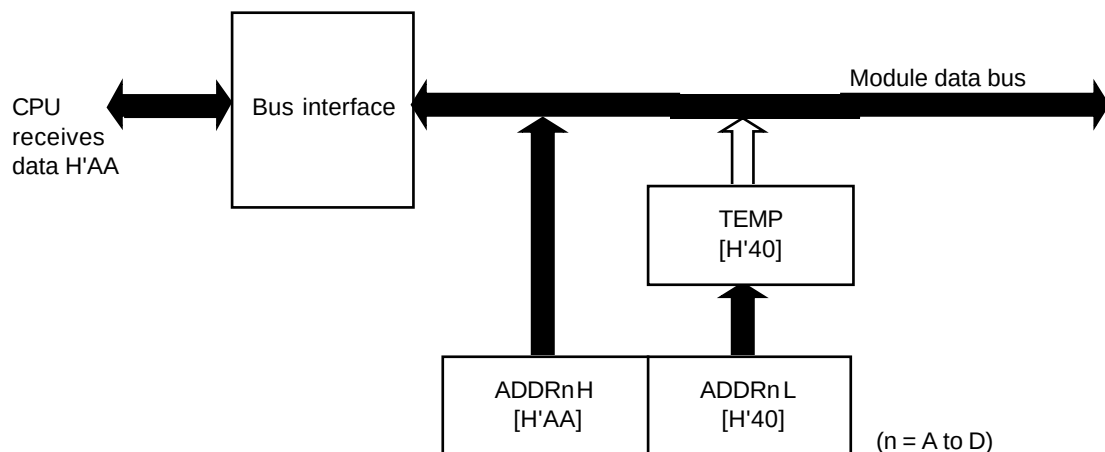
When the CPU or DTC reads the upper byte of an A/D data register, at the same time as the upper byte is placed on the internal data bus, the lower byte is transferred to TEMP. When the lower byte is accessed, the value in TEMP is placed on the internal data bus.

A program that requires all 10 bits of an A/D result should perform word access, or should read first the upper byte, then the lower byte of the A/D data register. Either way, it is assured of obtaining consistent data. Consistent data are not assured if the program reads the lower byte first.

A program that requires only 8-bit A/D accuracy should perform byte access to the upper byte of the A/D data register. The value in TEMP can be left unread.

Figure 14-2 shows the data flow when the CPU (or DTC) reads an A/D data register.

< Upper byte read >



< Lower byte read >

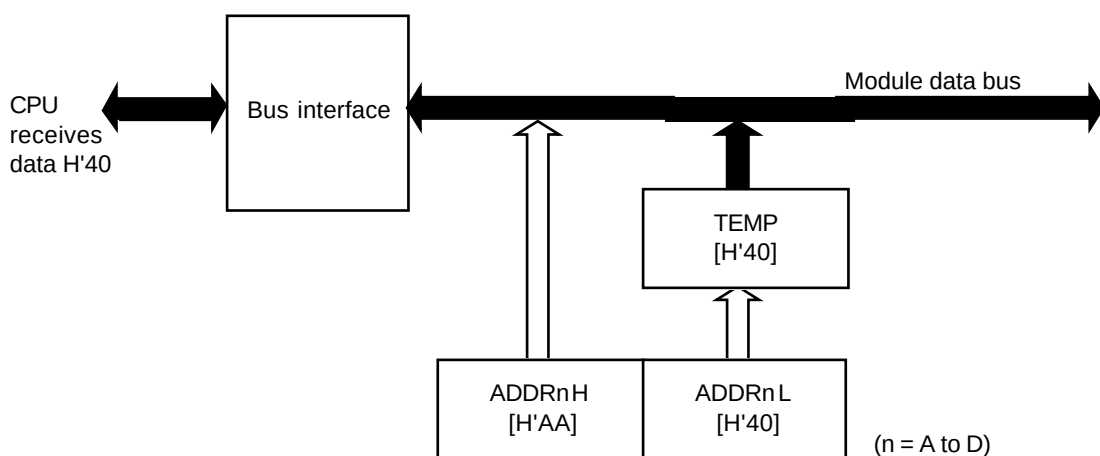


Figure 14-2 Read Access to A/D Data Register (When Register Contains H'AA40)

14.4 Operation

The A/D converter performs 10 successive approximations to obtain a result ranging from H'0000 (corresponding to AVss) to H'FFC0 (corresponding to AVcc). Only the first 10 bits of the result are significant.

The response of the A/D converter is shown below. H'FFC0 corresponds to voltages of approximately 0.999AV_{CC} and above.

The A/D converter module can be programmed to operate in single mode or scan mode as explained below.

14.4.1 Single Mode

The single mode is suitable for obtaining a single data value from a single channel. A/D conversion starts when the ADST bit is set to 1 by software or external trigger input. During the conversion process the ADST bit remains set to 1. When conversion is completed, the ADST bit is automatically cleared to 0.

When the conversion is completed, the ADF bit is set to 1. If the interrupt enable bit (ADIE) is also set to 1, an A/D conversion end interrupt (ADI) is requested, so that the converted data can be processed by an interrupt-handling routine. Alternatively, the interrupt can be served by the data transfer controller (DTC).

When an A/D interrupt is served by the DTC, the DTC automatically clears the ADF bit to 0. When an A/D interrupt is served by the CPU, however, the ADF bit remains set until the CPU reads the ADCSR, then writes a 0 in the ADF bit.

Before selecting the single mode, clock, and analog input channel, software should clear the ADST bit to 0 to make sure the A/D converter is stopped. Changing the mode, clock, or channel selection while A/D conversion is in progress can lead to conversion errors.

The following example explains the A/D conversion process in single mode when channel 1 (AN₁) is selected and external triggering is not used. Figure 14-3 shows the corresponding timing chart.

1. Software clears the ADST bit to 0, then selects the single mode (SCAN = 0) and channel 1 (CH2 to CH0 = 001), enables the A/D interrupt request (ADIE = 1), and sets the ADST bit to 1 to start A/D conversion.

Coding Example: (when using the slow clock, CKS = 0)

```
BCLR #7, @H'FFE9
```

```
BCLR #5, @H'FFE8
```

```
MOV.B #H'61, @H'FFE8
```

2. The A/D converter samples the AN1 input and converts the voltage level to a digital value. At the end of the conversion process the A/D converter transfers the result to register ADDR_B, sets the ADF bit to 1, clears the ADST bit to 0, and halts.
3. ADF = 1 and ADIE = 1, so an A/D interrupt is requested.
4. The user-coded A/D interrupt-handling routine is started.
5. The interrupt-handling routine reads the ADCSR value, then writes a 0 in the ADF bit to clear this bit to 0. The reading and writing can be done with a single BCLR #7, @H'FFE8 instruction.
6. The interrupt-handling routine reads and processes the A/D conversion result.
7. The routine ends.

Steps 2 to 7 can now be repeated by setting the ADST bit to 1 again.

If the ADI bit in data transfer enable register D (bit 0 at address H'FFF7) is set to 1, the interrupt is served by the data transfer controller (DTC). Steps 4 to 7 then change as follows.

- 4'. The DTC is started.
- 5'. The DTC automatically clears the ADF bit to 0.
- 6'. The DTC transfers the A/D conversion result from ADDR_B to a specified destination address.
- 7'. The DTC ends.

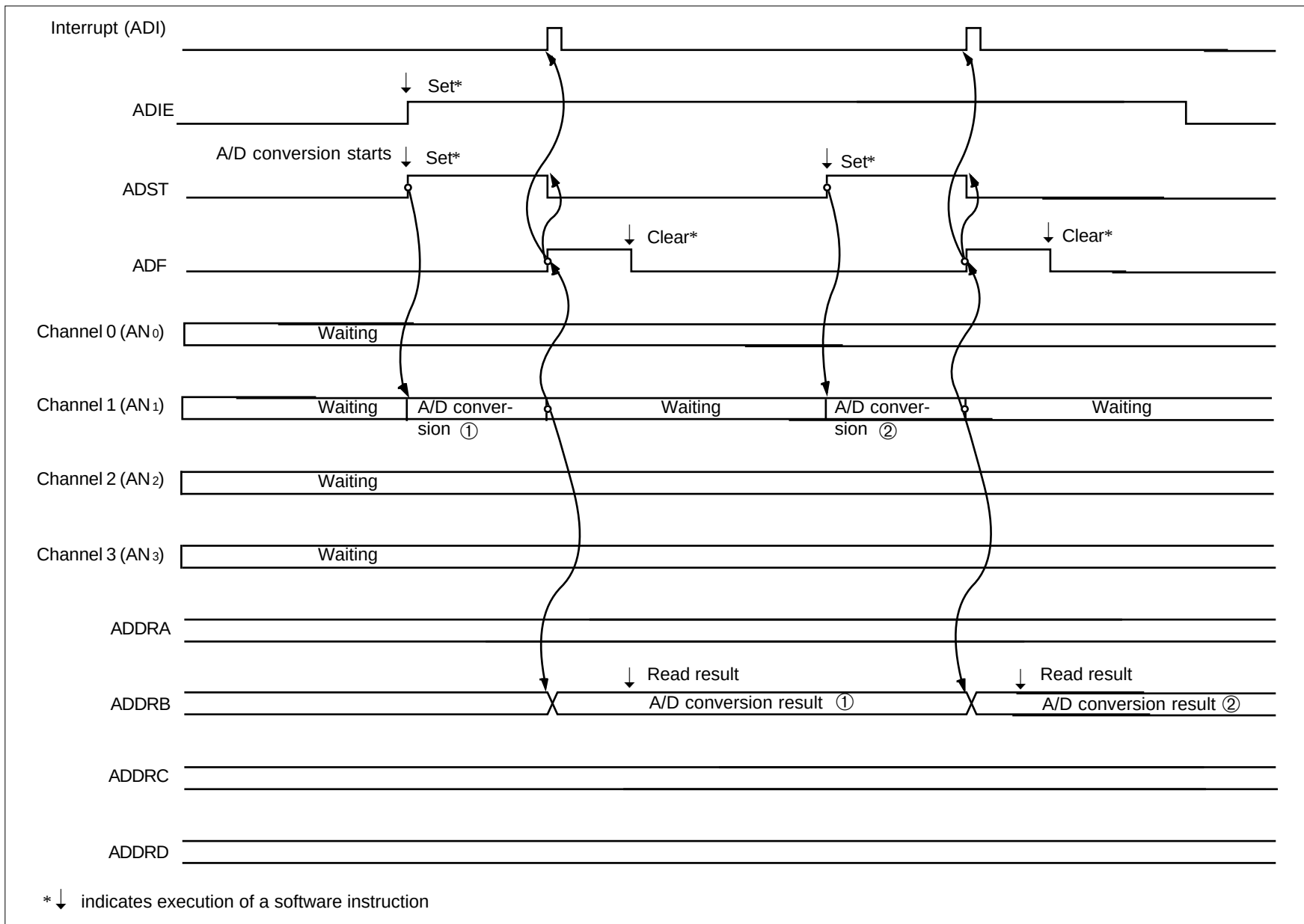


Figure 14-3 A/D Operation in Single Mode (When Channel 1 is Selected)

14.4.2 Scan Mode

The scan mode can be used to monitor analog inputs on one or more channels. When the ADST bit is set to 1 by software or by external trigger input, A/D conversion starts from the first channel (AN₀) in the scan group.*

If the scan group includes more than one channel (i.e., if bit CH1 or CH0 is set), conversion of the next channel begins as soon as conversion of the first channel ends.

Conversion of the selected channels continues cyclically until the ADST bit is cleared to 0. The conversion results are placed in the data registers corresponding to the selected channels.

Before selecting the scan mode, clock, and analog input channels, software should clear the ADST bit to 0 to make sure the A/D converter is stopped. Changing the mode, clock, or channel selection while A/D conversion is in progress can lead to conversion errors.

The following example explains the A/D conversion process when three channels in group 0 are selected (AN₀, AN₁, and AN₂) and external triggering is not used. Figure 14-4 shows the timing.

1. Software clears the ADST bit to 0, then selects the scan mode (SCAN = 1), scan group 0 (CH2 = 0), and analog input channels AN₀ to AN₂ (CH1 = 1, CH0 = 0) and sets the ADST bit to 1 to start A/D conversion.

Coding Example: (with slow clock and ADI interrupt enabled)

```
BCLR #7, @H'FFE9
```

```
BCLR #5, @H'FFE8
```

```
MOV.B #H'72, @FFE8
```

2. The A/D converter samples the input at AN₀, converts the voltage level to a digital value, and transfers the result to register ADDRA.
3. Next the A/D converter samples and converts AN₁ and transfers the result to ADDR_B. Then it samples and converts AN₂ and transfers the result to ADDR_C.
4. After all selected channels (AN₀ to AN₂) have been converted, the AD converter sets the ADF bit to 1. If the ADIE bit is set to 1, an A/D interrupt (ADI) is requested. Then the A/D converter begins converting AN₀ again.
5. Steps 2 to 4 are repeated cyclically as long as the ADST bit remains set to 1.

To stop the A/D converter, software must clear the ADST bit to 0. The data currently undergoing conversion when the ADST bit is cleared are ignored. The A/D data registers retain the last completed conversion results.

Regardless of which channel is being converted when the ADST bit is cleared to 0, when the ADST bit is set to 1 again, conversion begins from the the first selected channel (AN₀ or AN₄).

Note: * In the CP-68 package, the first channel is AN₀ if CH2 = 0, and AN₄ if CH2 = 1.

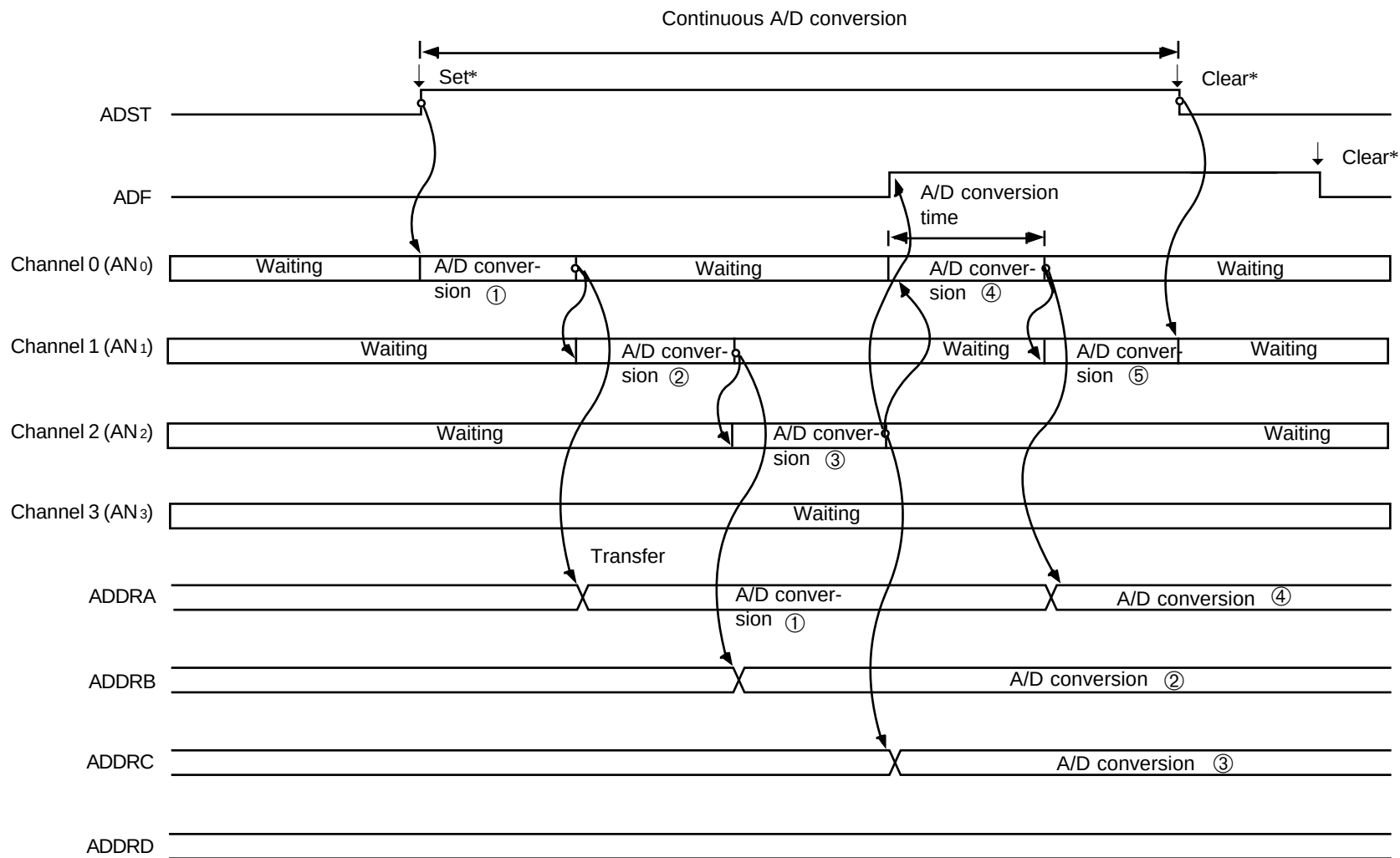


Figure 14-4 A/D Operation in Scan Mode (When Channels 0 to 2 are Selected)

14.4.3 Input Sampling Time and A/D Conversion Time

The A/D converter includes a built-in sample-and-hold circuit. Sampling of the input starts at a time t_D after the ADST bit is set to 1. The sampling process lasts for a time t_{SPL} . The actual A/D conversion begins after sampling is completed. Figure 14-5 shows the timing of these steps, and table 15-4 lists the total conversion times (t_{CONV}) for the single mode.

The total conversion time includes t_D and t_{SPL} . The purpose of t_D is to synchronize the ADCSR write time with the A/D conversion process, so the length of t_D is variable. The total conversion time therefore varies within the minimum to maximum ranges indicated in table 14-4.

In the scan mode, the ranges given in table 14-4 apply to the first conversion. The length of the second and subsequent conversion processes is fixed at 256 states (when $CKS = 0$) or 128 states (when $CKS = 1$).

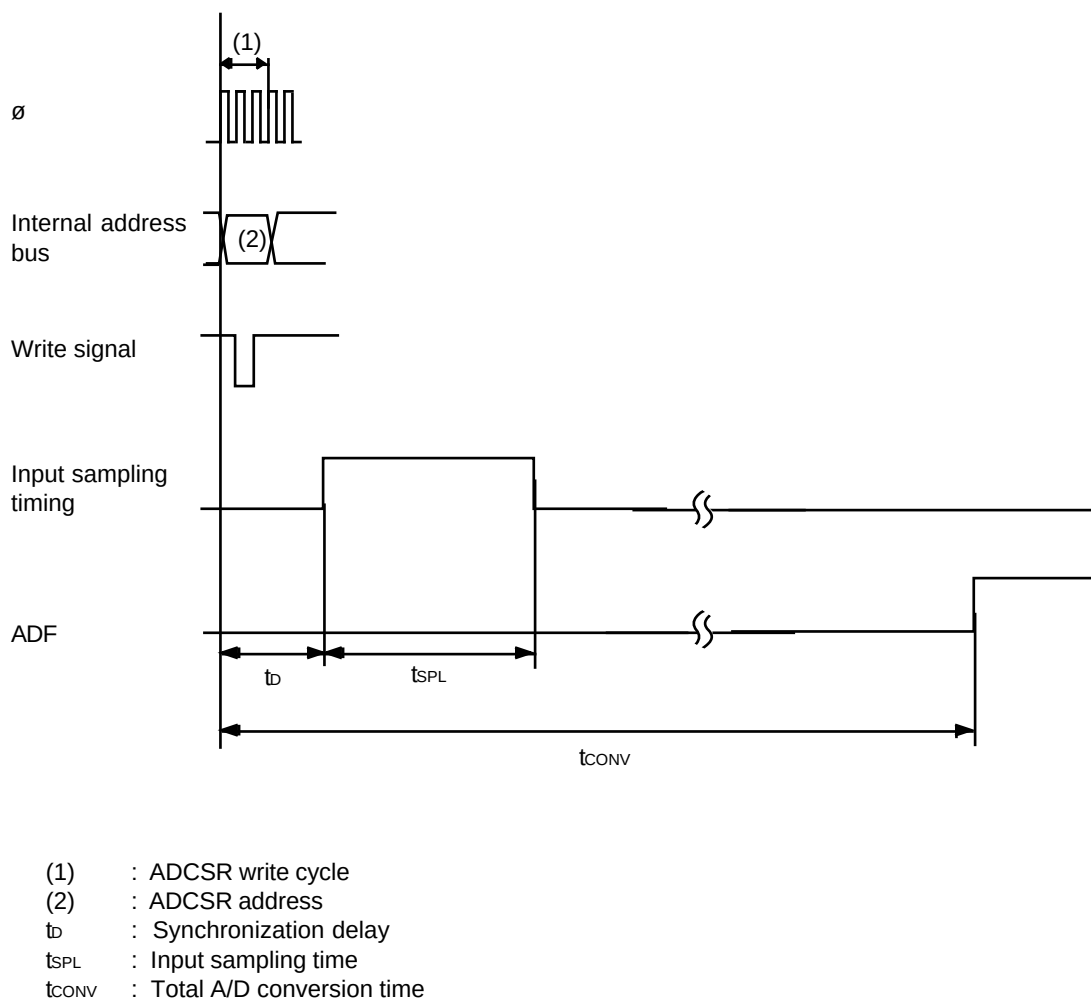


Figure 14-5 A/D Conversion Timing

Table 14-4 A/D Conversion Time (Single Mode)

Item	Symbol	CKS = 0			CKS = 1		
		min	typ	max	min	typ	max
Synchronization delay	t_D	18	—	33	10	—	17
Input sampling time	t_{SPL}	—	63	—	—	31	—
Total A/D conversion time	t_{CONV}	259	—	274	131	—	138

Note: Values in the table are numbers of states.

14.4.4 External Triggering of A/D Conversion

The A/D conversion process can be started by an external trigger input.

External trigger input is enabled at the $\overline{\text{ADTRG}}$ pin when the TRGE bit in the ADCR is set to 1. 1.0 ϕ clock cycles after the $\overline{\text{ADTRG}}$ input is sampled, the ADST bit in the ADCSR is set to 1 and A/D conversion commences.

The timing of external triggering is shown in figure 14-6.

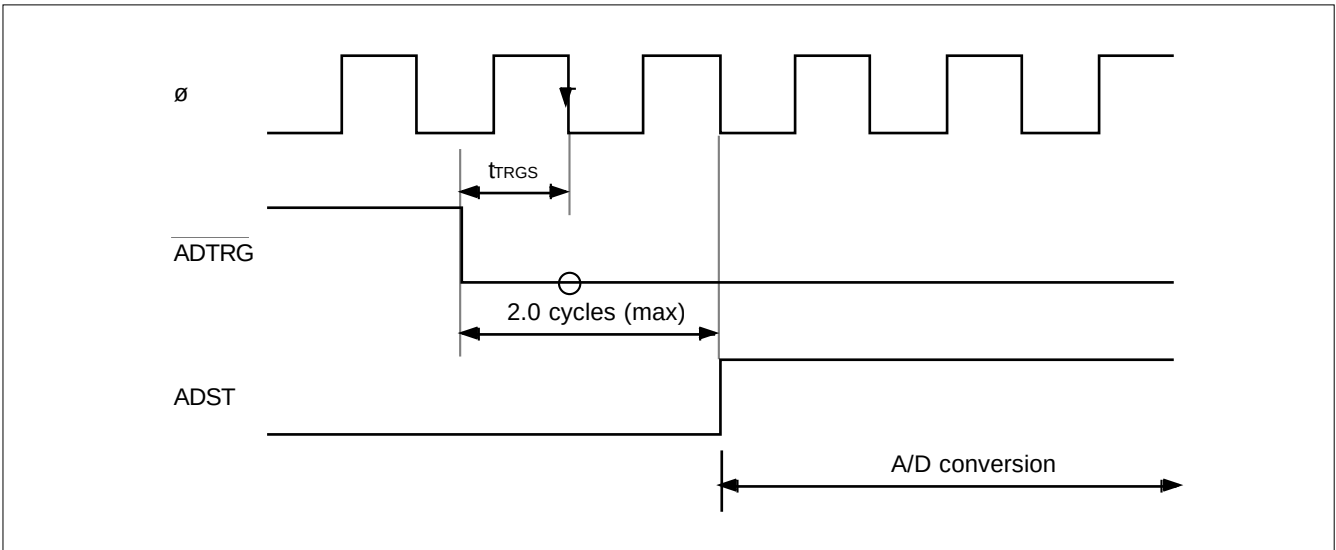


Figure 14-6 Timing of Setting of ADST Bit

14.5 Interrupts and the Data Transfer Controller

The ADI interrupt request is enabled or disabled by the ADIE bit in the ADCSR.

When the ADI bit in data transfer enable register DTED (bit 0 at address H'FFF7) is set to 1, the ADI interrupt is served by the data transfer controller. The DTC can be used to transfer A/D results to a buffer in memory, or to an I/O port. The DTC automatically clears the ADF bit to 0.

Note: In scan mode, the DTC can transfer data for only one channel per interrupt, even if two or more channels are selected.

Section 15 RAM

15.1 Overview

The H8/520 includes 512 bytes of on-chip static RAM, connected to the CPU by a 16-bit data bus. Both byte and word access to the on-chip RAM are performed in two states, enabling rapid data transfer and instruction execution.

The on-chip RAM is assigned to addresses H'FD80 to H'FF7F in the chip's address space. A RAM control register (RAMCR) can enable or disable the on-chip RAM, permitting these addresses to be allocated to external memory instead, if so desired.

15.1.1 Block Diagram

Figure 15-1 shows a block diagram of the on-chip RAM.

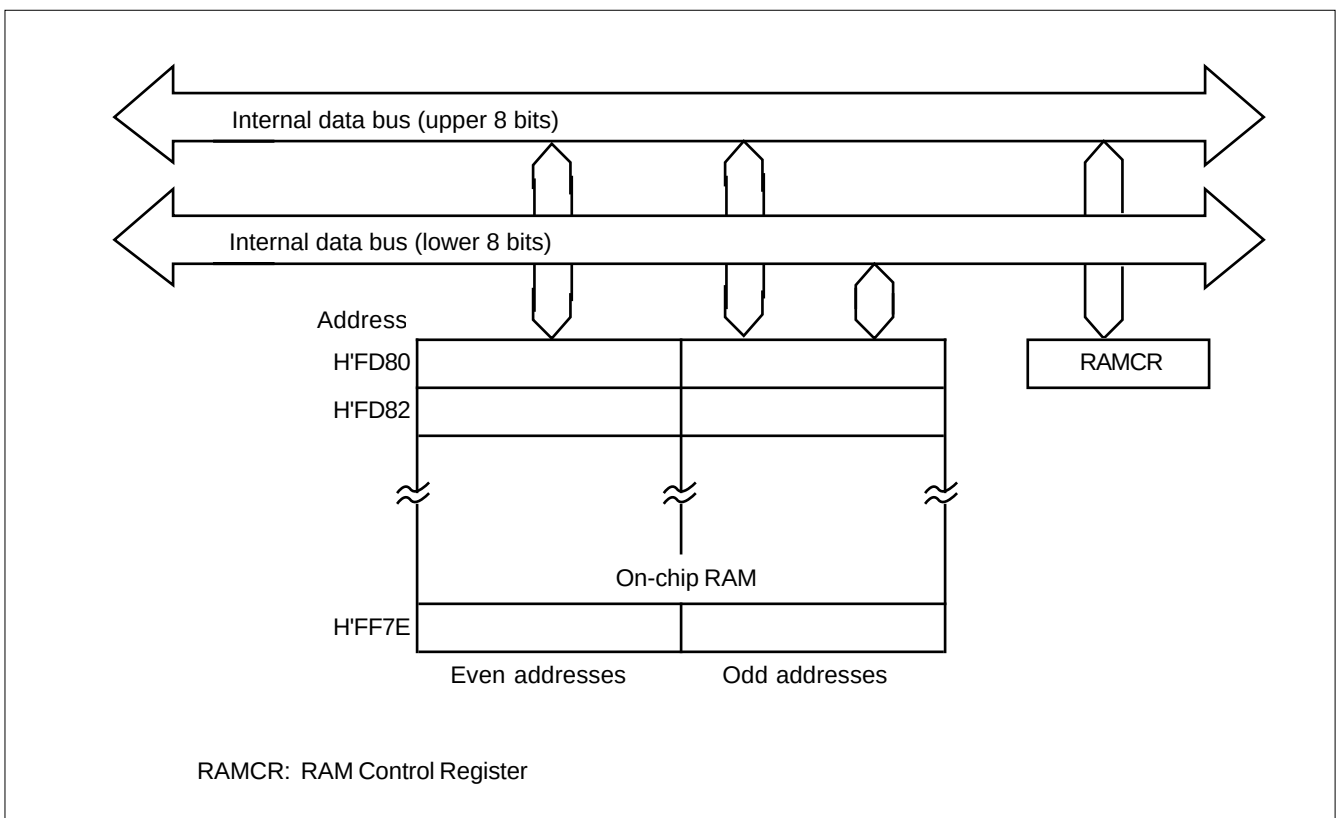


Figure 15-1 Block Diagram of On-Chip RAM

15.1.2 Register Configuration

The on-chip RAM is controlled by the register described in table 15-1.

Table 15-1 RAM Control Register

Name	Abbreviation	R/W	Initial Value	Address
RAM control register	RAMCR	R/W	H'FF	H'FFF9

15.2 RAM Control Register (RAMCR)

Bit	7	6	5	4	3	2	1	0
	RAME	—	—	—	—	—	—	—
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	—	—	—	—	—	—	—

The RAM control register (RAMCR) is an 8-bit register that enables or disables the on-chip RAM.

Bit 7—RAM Enable (RAME): This bit enables or disables the on-chip RAM.

The RAME bit is initialized by a reset. It is not initialized in the software standby mode.

Bit 7	
RAME	Description
0	On-chip RAM is disabled.
1	On-chip RAM is enabled. (Initial value)

Bits 6 to 0—Reserved: These bits cannot be modified and are always read as 1.

15.3 Operation

15.3.1 Expanded Modes (Modes 1, 2, 3, and 4)

If the RAME bit is set to 1, accesses to addresses H'FD80 to H'FF7F are directed to the on-chip RAM. If the RAME bit is cleared to 0, accesses to addresses H'FD80 to H'FF7F are directed to the external data bus.

15.3.2 Single-Chip Mode (Mode 7)

If the RAME bit is set to 1, accesses to addresses H'FD80 to H'FF7F are directed to the on-chip RAM. If the RAME bit is cleared to 0, access of any type (instruction fetch or data read or write) to addresses H'FD80 to H'FF7F causes an address error and initiates the CPU's exception-handling sequence.

Section 16 ROM

16.1 Overview

The H8/520 includes 16 kbytes of high-speed on-chip ROM. The on-chip ROM is connected to the CPU via a 16-bit data bus and is accessed in two states.

Users wishing to program the chip themselves can request electrically programmable ROM (PROM). The PROM version of the H8/520 has a PROM mode in which the chip can be programmed with a standard, external PROM writer. The chip is also available with masked ROM.

The on-chip ROM is enabled or disabled depending on the MCU operating mode, which is determined by the inputs at the mode pins when the chip comes out of the reset state. See table 16-1.

Table 16-1 ROM Usage in Each MCU Mode

Mode	Mode Pins			ROM
	MD ₂	MD ₁	MD ₀	
Mode 1 (expanded minimum mode)	0	0	1	Disabled (external addresses)
Mode 2 (expanded minimum mode)	0	1	0	Enabled
Mode 3 (expanded maximum mode)	0	1	1	Disabled (external addresses)
Mode 4 (expanded maximum mode)	1	0	0	Enabled
Mode 7 (single-chip mode)	1	1	1	Enabled

16.1.1 Block Diagram

Figure 16-1 shows the block diagram of the on-chip ROM.

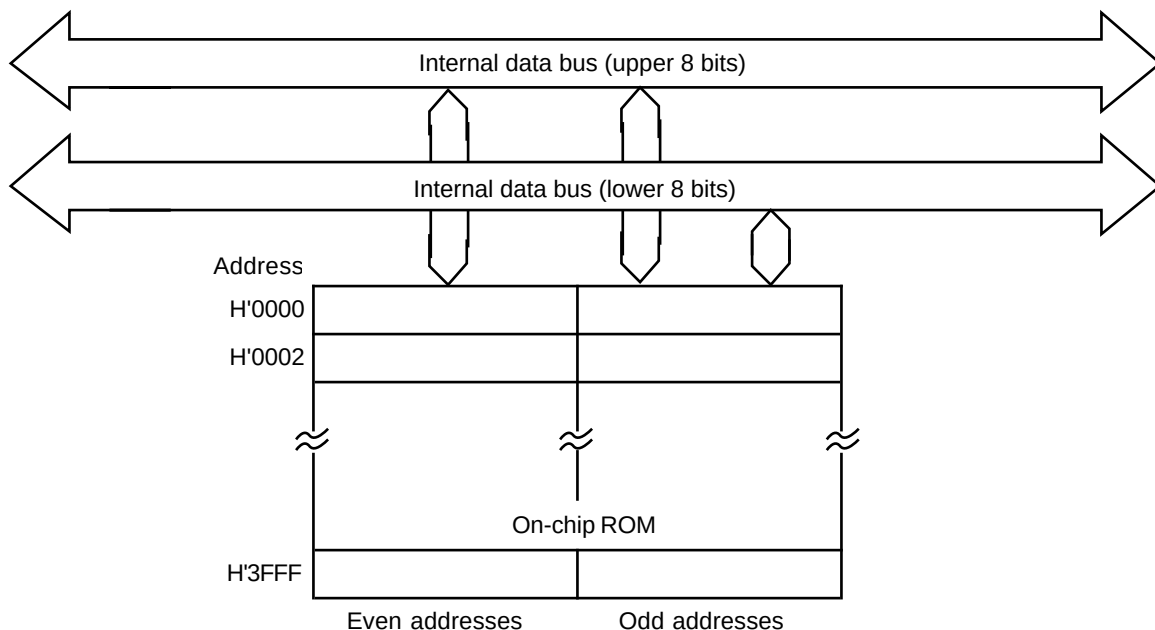


Figure 16-1 Block Diagram of On-Chip ROM

16.2 PROM Mode

16.2.1 PROM Mode Setup

The PROM version of the H8/520 has a PROM mode in which the usual microcomputer functions are halted to allow the on-chip PROM to be programmed. The programming method is the same as for the HN27C256.

To select the PROM mode, apply the signal inputs listed in table 16-2 to the mode pins (MD2 to MD0) and pins P51 and P50.

Table 16-2 Selection of PROM Mode

Pin	Input
MD ₁	Low
MD ₂ and MD ₀	High
P5 ₁ and P5 ₀	High

16.2.2 Socket Adapter Pin Arrangements and Memory Map

The H8/520 can be programmed with a general-purpose PROM writer by attaching a socket adapter as listed in table 16-3. The socket adapter depends on the type of package. Figure 16-2 shows the socket adapter pin arrangements by giving the correspondence between H8/520 pins and HN27C256 pin functions. Figure 16-3 is a memory map.

Table 16-3 Socket Adapter

Package	Socket Adapter
64-Pin windowed shrink DIP (DC-64S)	HS528ESS01H
64-Pin shrink DIP (DP-64S)	
64-Pin QFP (FP-64A)	HS528ESH01H
68-Pin PLCC (CP-68)	HS528ESC01H

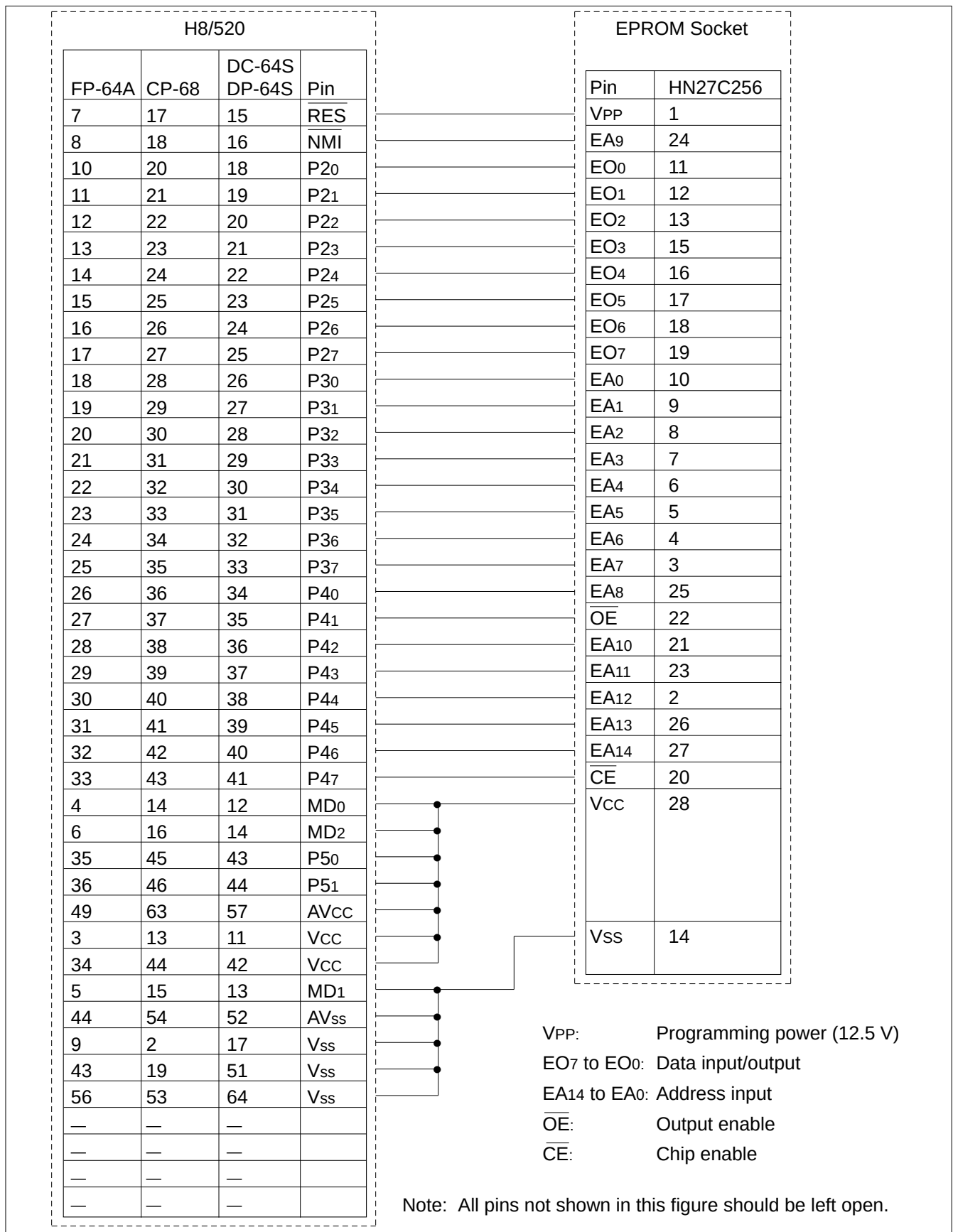


Figure 16-2 Socket Adapter Pin Arrangements

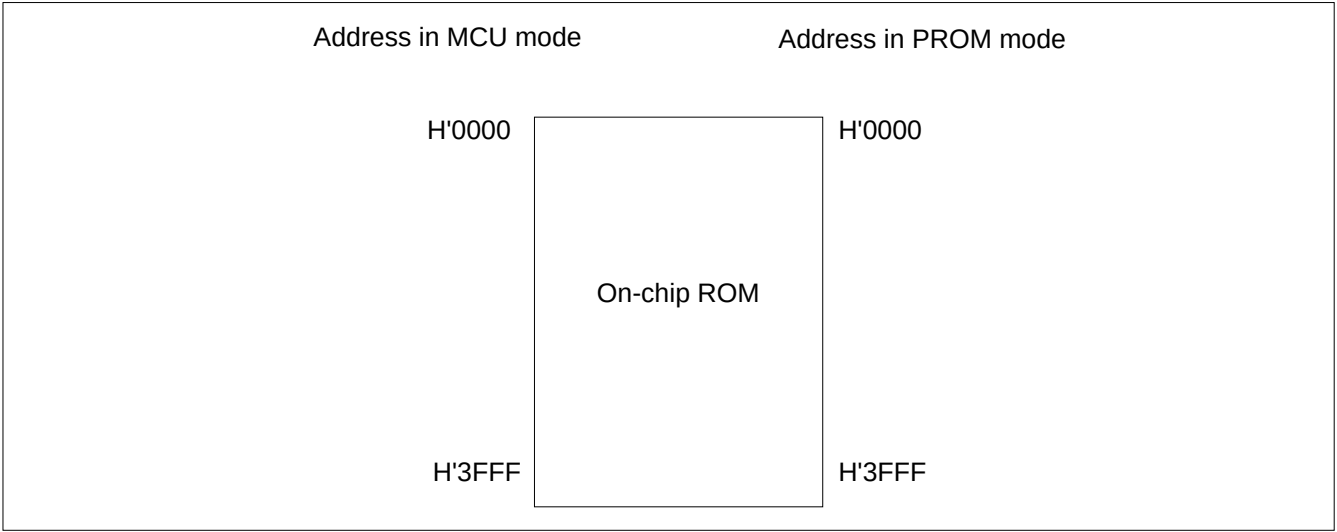


Figure 16-3 Memory Map in PROM Mode

16.3 Programming

The write, verify, and inhibited sub-modes of the PROM mode are selected as shown in table 16-4.

Table 16-4 Selection of Sub-Modes in PROM Mode

Mode	Pins					
	$\overline{\text{CE}}$	$\overline{\text{OE}}$	V _{PP}	V _{CC}	0 ₇ to 0 ₀	A ₁₄ to A ₀
Write	Low	High	V _{PP}	V _{CC}	Data input	Address input
Verify	High	Low	V _{PP}	V _{CC}	Data output	Address input
Programming inhibited	High	High	V _{PP}	V _{CC}	High-impedance	Address input
Read	Low	Low	V _{PP}	V _{CC}	Data output	Address input

Note: The V_{PP} and V_{CC} pins must be held at the V_{PP} and V_{CC} voltage levels.

The H8/520 PROM uses the same, standard read/write specifications as the HN27C256 and HN27256.

16.3.1 Writing and Verifying

An efficient, high-speed programming procedure can be used to write and verify PROM data. This procedure writes data quickly without subjecting the chip to voltage stress and without sacrificing data reliability. It leaves the data H'FF written in unused addresses.

Figure 16-4 shows the basic high-speed programming flowchart.

Tables 16-5 and 16-6 list the electrical characteristics of the chip in the PROM mode. Figure 16-5 shows a write/verify timing chart.

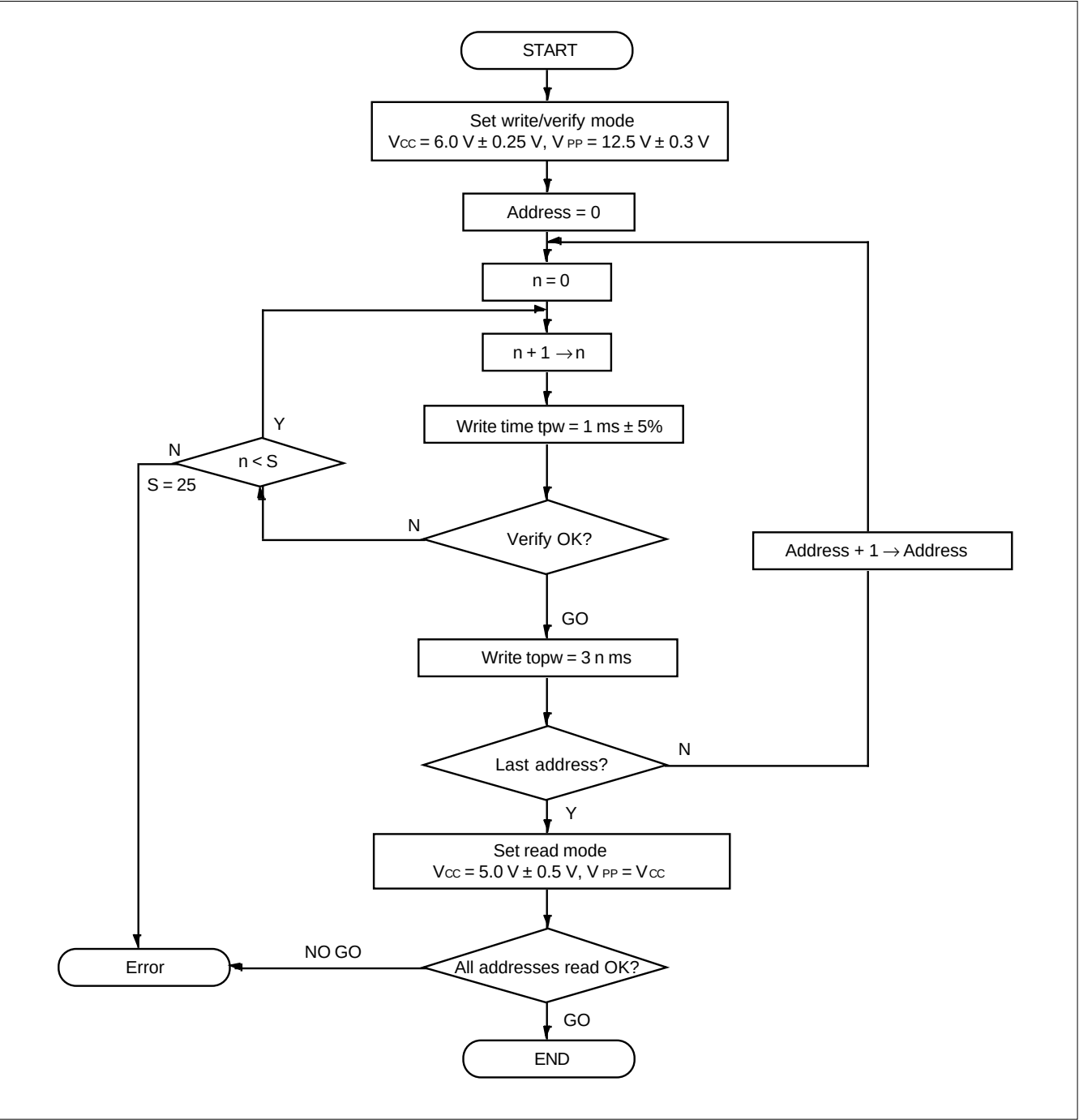


Figure 16-4 High-Speed Programming Flowchart

Table 16-5 DC Characteristics**(When $V_{CC} = 6.0\text{ V} \pm 0.25\text{ V}$, $V_{PP} = 12.5\text{ V} \pm 0.3\text{ V}$, $V_{SS} = 0\text{ V}$, $T_a = 25^\circ\text{C} \pm 5^\circ\text{C}$)**

Item	Symbol	min	typ	max	Unit	Measurement
						Conditions
Input high voltage	O_7 to O_0 , A_{14} to A_0 , $\overline{OE}$, $\overline{CE}$	V_{IH}	2.2	—	$V_{CC} + 0.3$	V
Input low voltage	O_7 to O_0 , A_{14} to A_0 , $\overline{OE}$, $\overline{CE}$	V_{IL}	−0.3	—	0.8	V
Output high voltage	O_7 to O_0	V_{OH}	2.4	—	—	V $I_{OH} = -200\text{ }\mu\text{A}$
Output low voltage	O_7 to O_0	V_{OL}	—	—	0.45	V $I_{OL} = 1.6\text{ mA}$
Input leakage current	O_7 to O_0 , A_{14} to A_0 , $\overline{OE}$, $\overline{CE}$	$ I_{LI} $	—	—	2	μA $V_{in} = 5.25\text{ V}/0.5\text{ V}$
V_{CC} current		I_{CC}	—	—	50	mA
V_{PP} current		I_{PP}	—	—	40	mA

Table 16-6 AC Characteristics**(When $V_{CC} = 6.0\text{ V} \pm 0.25\text{ V}$, $V_{PP} = 12.5\text{ V} \pm 0.3\text{ V}$, $V_{SS} = 0\text{ V}$, $T_a = 25^\circ\text{C} \pm 5^\circ\text{C}$)**

Item	Symbol	min	typ	max	Unit	Measurement
						Conditions
Address setup time	t_{AS}	2	—	—	μs	See figure 16-5*
$\overline{OE}$ setup time	t_{OES}	2	—	—	μs	
Data setup time	t_{DS}	2	—	—	μs	
Address hold time	t_{AH}	0	—	—	μs	
Data hold time	t_{DH}	2	—	—	μs	
Data output disable time	t_{DF}	—	—	130	ns	
V_{PP} setup time	t_{VPS}	2	—	—	μs	
Program pulse width	t_{PW}	0.95	1.0	1.05	ms	
$\overline{OE}$ pulse width for overwrite-programming	t_{OPW}	2.85	—	78.75	ms	
V_{CC} setup time	t_{VCS}	2	—	—	μs	
Data output delay time	t_{OE}	0	—	500	ns	

Note: * Input pulse level: 0.8 V to 2.2 V

Input rise/fall time $\leq 20\text{ ns}$

Timing reference levels: input—1.0 V, 2.0 V; output—0.8 V, 2.0 V

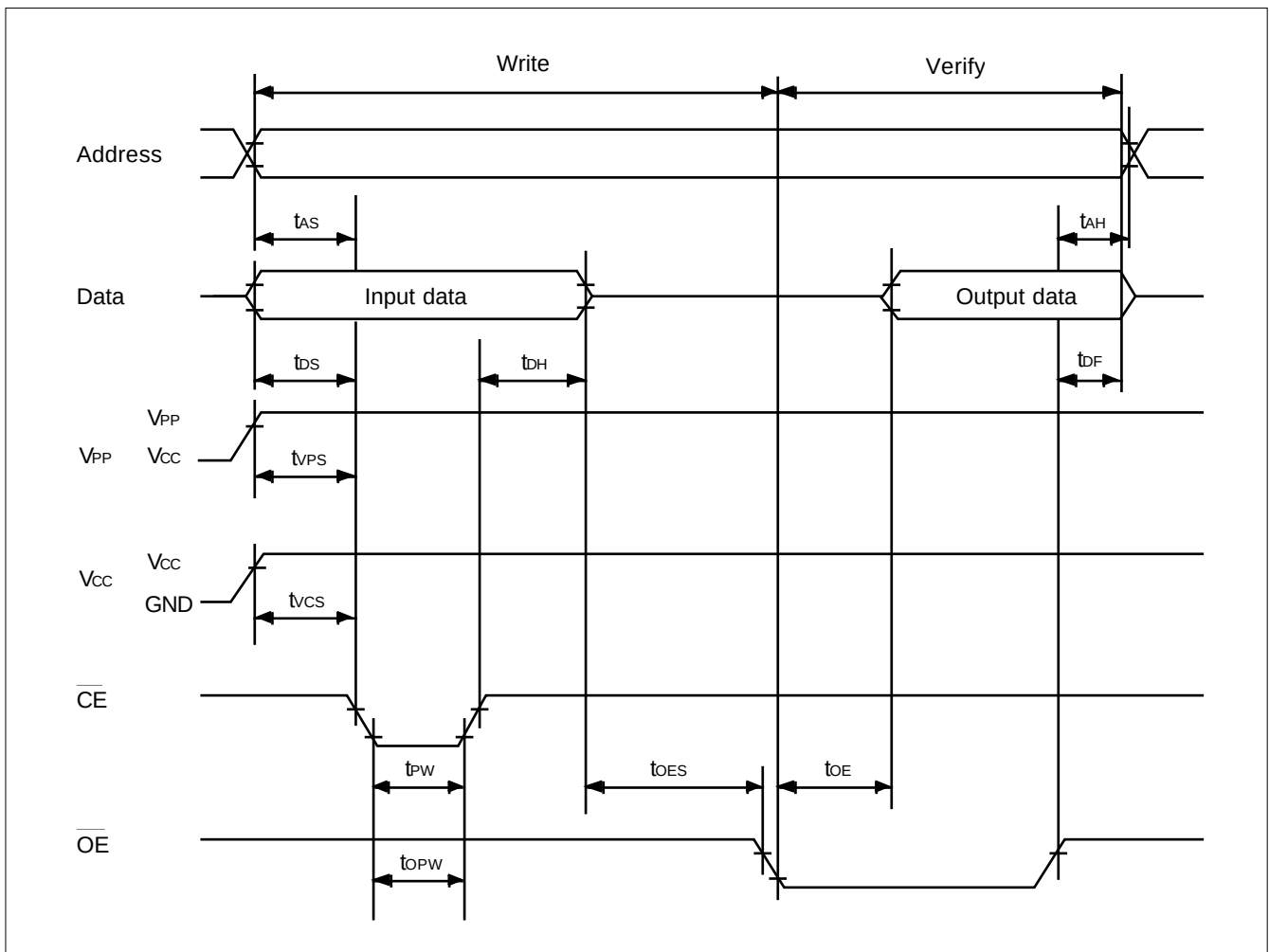


Figure 16-5 PROM Write/Verify Timing

16.3.2 Notes on Writing

1. Write with the specified voltages and timing. The programming voltage (V_{PP}) in the PROM mode is 12.5 V.

Caution: Applied voltages in excess of the specified values can permanently destroy the chip. Be particularly careful about the PROM writer's overshoot characteristics.

If the PROM writer is set to Intel specifications or Hitachi HN27256 or HN27C256 specifications, V_{PP} will be 12.5 V.

2. Before writing data, check that the socket adapter and chip are correctly mounted in the PROM writer. Overcurrent damage to the chip can result if the index marks on the PROM writer, socket adapter, and chip are not correctly aligned.

3. Don't touch the socket adapter or chip while writing. Touching either of these can cause contact faults and write errors.

16.3.3 Reliability of Written Data

An effective way to assure the data holding characteristics of the programmed chips is to bake them at 150°C, then screen them for data errors. This procedure quickly eliminates chips with PROM memory cells prone to early failure.

Figure 16-6 shows the recommended screening procedure.

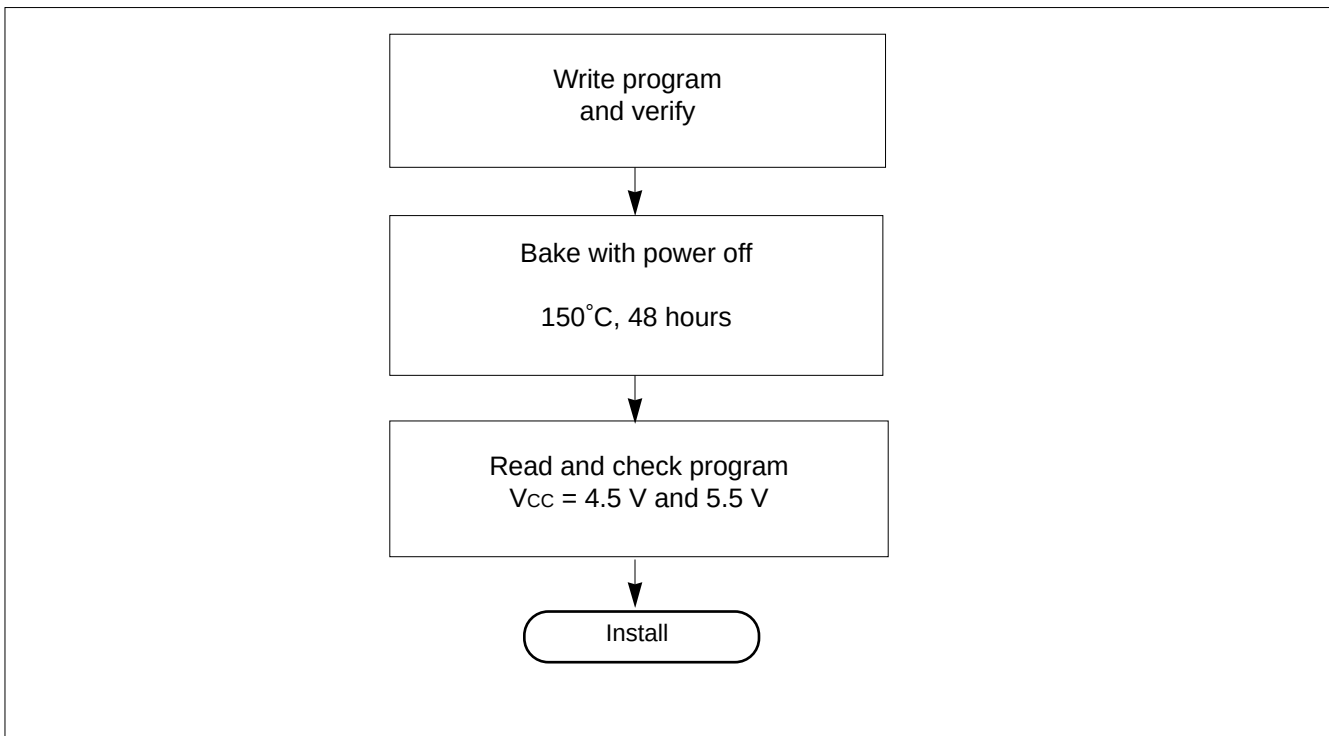


Figure 16-6 Recommended Screening Procedure

If a series of write errors occurs while the same PROM writer is in use, stop programming and check the PROM writer and socket adapter for defects, using a microcomputer with a windowed package and on-chip EPROM.

Please inform Hitachi of any abnormal conditions noted during programming or in screening of program data after high-temperature baking.

16.3.4 Erasing of Data

The windowed package enables data to be erased by illuminating the window with ultraviolet light. Table 16-7 lists the erasing conditions.

Table 16-7 Erasing Conditions

Item	Value
Ultraviolet wavelength	2537 Å
Minimum illumination	15 W·s/cm ²

The conditions in table 16-7 can be satisfied by placing a 12000 μW/cm² ultraviolet lamp 2 or 3 centimeters directly above the chip and leaving it on for about 20 minutes.

16.4 Handling of Windowed Packages

Glass Erasing Window: Rubbing the glass erasing window of a windowed package with a plastic material or touching it with an electrically charged object can create a static charge on the window surface which may cause the chip to malfunction.

If the erasing window becomes charged, the charge can be neutralized by a short exposure to ultraviolet light. This returns the chip to its normal condition, but it also reduces the charge stored in the floating gates of the PROM, so it is recommended that the chip be reprogrammed afterward.

Accumulation of static charge on the window surface can be prevented by the following precautions:

1. When handling the package, ground yourself. Don't wear gloves. Avoid other possible sources of static charge.
2. Avoid friction between the glass window and plastic or other materials that tend to accumulate static charge.
3. Be careful when using cooling sprays, since they may have a slight ion content.
4. Cover the window with an ultraviolet-shield label, preferably a label including a conductive material. Besides protecting the PROM contents from ultraviolet light, the label protects the chip by distributing static charge uniformly.

Handling after Programming: Fluorescent light and sunlight contain small amounts of ultraviolet, so prolonged exposure to these types of light can cause programmed data to invert. In addition, exposure to any type of intense light can induce photoelectric effects that may lead to chip malfunction. It is recommended that after programming the chip, cover the erasing window with a light-proof label (such as an ultraviolet-shield label).

Section 17 Power-Down State

17.1 Overview

The H8/520 has a power-down state that greatly reduces power consumption by stopping the CPU functions. The power-down state includes three modes:

1. Sleep mode: software-triggered mode in which the CPU halts but the rest of the chip remains active.
2. Software standby mode: software-triggered mode in which the entire chip is inactive.
3. Hardware standby mode: hardware-triggered mode in which the entire chip is inactive.

The sleep mode and software standby mode are entered from the program execution state by executing the SLEEP instruction under the conditions given in table 17-1. The hardware standby mode is entered from any other state by setting mode 6 at the mode pins (MD2 to MD0).

Table 17-1 lists the conditions for entering and leaving the power-down modes. It also indicates the status of the CPU, on-chip supporting modules, etc. in each power-down mode.

Table 17-1 Power-Down State

Mode	Entering Procedure	Clock	CPU	CPU Registers	Peripheral Functions	RAM	I/O Ports	Exiting Methods
Sleep mode	Execute SLEEP instruction	Run	Halt	Held	Run	Held	Held	• Interrupt • $\overline{\text{RES}}$ low • Mode 6
Software standby mode	Set SSBY bit in SBYCR to 1, then execute SLEEP instruction*	Halt	Halt	Held	Halt and initialized	Held	Held	• NMI • $\overline{\text{RES}}$ low • Mode 6
Hardware standby mode	Set mode pins to mode 6	Halt	Halt	Not held	Halt and initialized	Held	High impedance state	• Mode 1,2,3, 4, or 7 then $\overline{\text{RES}}$ low → high

Notes: * The watchdog timer must also be stopped.
 SBYCR: Software standby control register
 SSBY: Software standby bit

17.2 Sleep Mode

17.2.1 Transition to Sleep Mode

Execution of the SLEEP instruction causes a transition from the program execution state to the sleep mode. After executing the SLEEP instruction, the CPU halts, but the contents of its internal registers remain unchanged. The functions of the on-chip supporting modules do not stop in the sleep mode.

17.2.2 Exit from Sleep Mode

The chip wakes up from the sleep mode when it receives an internal or external interrupt request or a low input at the $\overline{\text{RES}}$ pin, or when mode 6 is set at the mode pins.

Wake-Up by Interrupt: An interrupt releases the sleep mode and starts either the CPU's interrupt-handling sequence or the data transfer controller (DTC).

If the interrupt is served by the DTC, after the data transfer is completed the CPU executes the instruction following the SLEEP instruction, unless the count in the data transfer count register (DTCR) is 0.

If an interrupt on a level equal to or less than the mask level in the CPU's status register (SR) is requested, the interrupt is left pending and the sleep mode continues. Also, if an interrupt from an on-chip supporting module is disabled by the corresponding enable/disable bit in the module's control register, the interrupt cannot be requested, so it cannot wake the chip up.

Wake-Up by $\overline{\text{RES}}$ pin: When the $\overline{\text{RES}}$ pin goes low, the chip exits from the sleep mode to the reset state.

Wake-Up by Mode 6: When the mode pins are set to mode 6, the chip exits from the sleep mode to the hardware standby mode.

17.3 Software Standby Mode

17.3.1 Transition to Software Standby Mode

A program enters the software standby mode by setting the standby bit (SSBY) in the software standby control register (SBYCR) to 1, then executing the SLEEP instruction. Table 17-2 lists the attributes of the software standby control register.

Table 17-2 Software Standby Control Register

Name	Abbreviation	R/W	Initial Value	Address
Software standby control register	SBYCR	R/W	H'7F	H'FFFB

In the software standby mode, the CPU, clock, and the on-chip supporting module functions all stop, reducing power consumption to an extremely low level. The on-chip supporting modules and their registers are reset to their initial state, but as long as a minimum necessary voltage supply is maintained (at least 2 V), the contents of the CPU registers and on-chip RAM remain unchanged. The I/O ports also remain in their current states.

17.3.2 Software Standby Control Register (SBYCR)

Bit	7	6	5	4	3	2	1	0
	SSBY	—	—	—	—	—	—	—
Initial value	0	1	1	1	1	1	1	1
Read/Write	R/W	—	—	—	—	—	—	—

The software standby control register (SBYCR) is an 8-bit register that controls the action of the SLEEP instruction.

Bit 7—Software Standby (SSBY): This bit enables or disables the transition to the software standby mode.

Bit 7

SSBY	Description
0	The SLEEP instruction causes a transition to the sleep mode. (Initial value)
1	The SLEEP instruction causes a transition to the software standby mode.

The watchdog timer must be stopped before the chip can enter the software standby mode. To stop the watchdog timer, clear the timer enable bit (TME) in the watchdog timer’s timer control/status register (TCSR) to 0. The SSBY bit cannot be set to 1 while the TME bit is set to 1.

When the chip is recovered from the software standby mode by a nonmaskable interrupt (NMI), the SSBY bit is automatically cleared to 0. It is also cleared to 0 by a reset or transition to the hardware standby mode.

Bits 6 to 0—Reserved: These bits cannot be modified and are always read as 1.

17.3.3 Exit from Software Standby Mode

The chip can be brought out of the software standby mode by an input at the NMI pin, $\overline{\text{RES}}$ pin, or mode pins.

Recovery by NMI Pin: When an NMI request signal is received, the clock oscillator begins operating but clock pulses are supplied only to the watchdog timer (WDT). The watchdog timer begins counting from H'00 at the rate determined by the clock select bits (CKS2 to CKS0) in its timer status/control register (TCSR). This rate should be set slow enough to allow the clock oscillator to stabilize before the count reaches H'FF. When the count overflows from H'FF to H'00, clock pulses are supplied to the whole chip, the software standby mode ends, and execution of the NMI interrupt-handling sequence begins.

The clock select bits (CKS2 to CKS0) should be set as follows.

Crystal Oscillator: Set CKS2 to CKS0 to a value that makes the watchdog timer interval equal to or greater than 10 ms, which is the clock stabilization time.

External Clock Input: CKS2 to CKS0 can be set to any value. The minimum value (CKS2 = CKS1 = CKS0 = 0) is recommended.

Recovery by $\overline{\text{RES}}$ Pin: When the $\overline{\text{RES}}$ pin goes low, the clock oscillator starts. Next, when the $\overline{\text{RES}}$ pin goes high, the CPU begins executing the reset sequence.

When the chip recovers from the software standby mode by a reset, clock pulses are supplied to the entire chip at once. Be sure to hold the $\overline{\text{RES}}$ pin low long enough for the clock to stabilize.

Recovery by Mode 6: When the mode pins are set to mode 6, the chip exits from the software standby mode to the hardware standby mode.

17.3.4 Sample Application of Software Standby Mode

In this example the chip enters the software standby mode on the falling edge of the NMI input and recovers from the software standby mode on the rising edge of NMI. Figure 17-1 shows a timing chart of the transitions.

The nonmaskable interrupt edge bit (NMIEG) in the NMI control register (NMICR) is originally cleared to 0, selecting the falling edge as the NMI trigger. After accepting an NMI interrupt in this

condition, software changes the NMIEG bit to 1, sets the SSBY bit to 1, and executes the SLEEP instruction to enter the software standby mode. The chip recovers from the software standby mode on the next rising edge at the NMI pin.

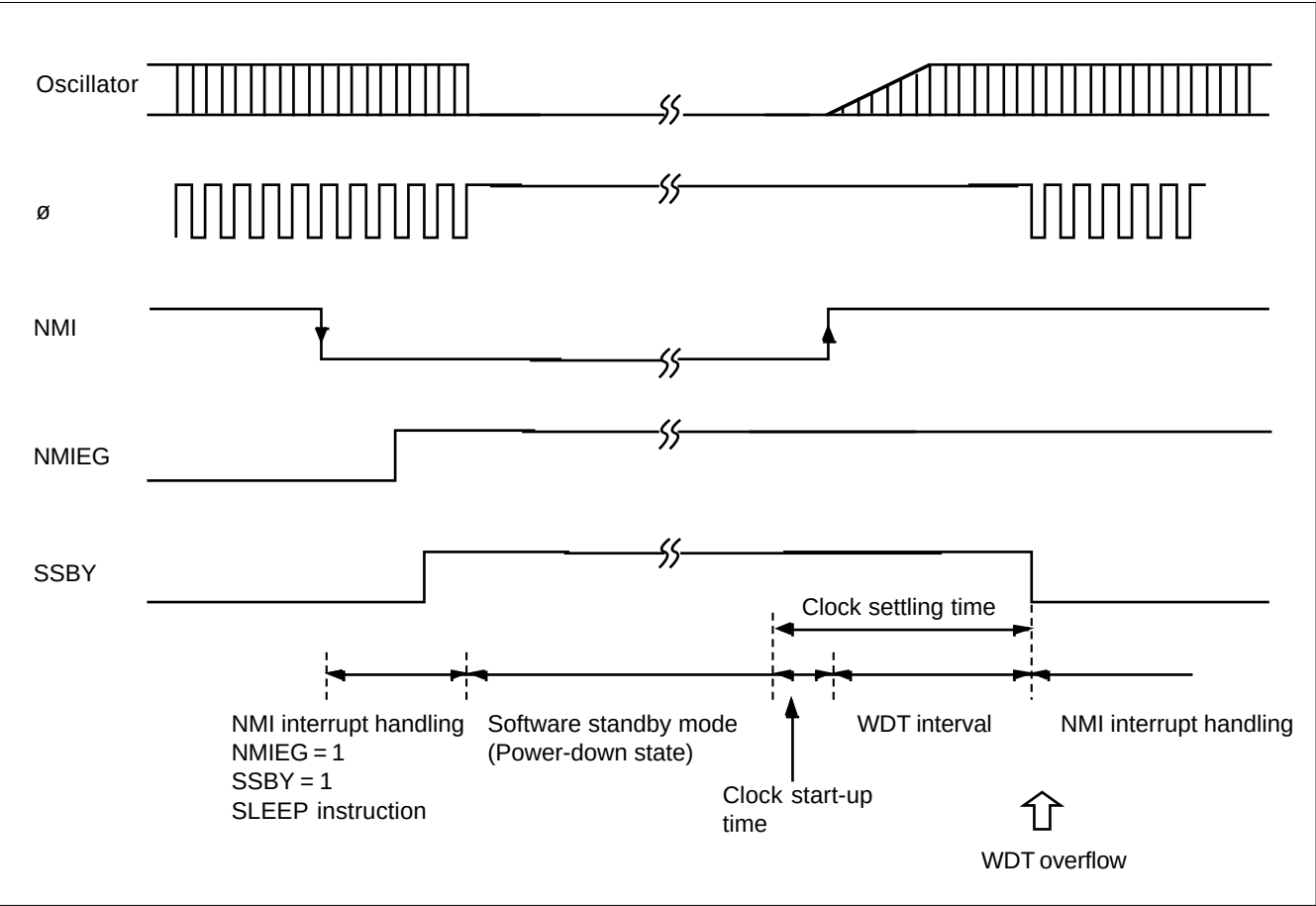


Figure 17-1 NMI Timing of Software Standby Mode (Application Example)

17.3.5 Application Notes

The I/O ports remain in their current states in the software standby mode. If a port is in the high output state, the output current is not reduced in the software standby mode.

17.4 Hardware Standby Mode

17.4.1 Transition to Hardware Standby Mode

Regardless of its current state, the chip enters the hardware standby mode whenever the mode pins are set to mode 6 (MD2 and MD1 high, MD0 low).

The hardware standby mode reduces power consumption drastically by halting the CPU, stopping all the functions of the on-chip supporting modules, and placing I/O ports in the high-impedance state.

The registers of the on-chip supporting modules are reset to their initial values. Only the on-chip RAM is held unchanged, provided the minimum necessary voltage supply is maintained (at least 2 V).*

- Notes:
1. The RAME bit in the RAM control register should be cleared to 0 before the mode pins are set to mode 6, to disable the on-chip RAM during the hardware standby mode.
 2. Do not change the inputs at the mode pins (MD2, MD1, MD0) during hardware standby mode. Be particularly careful not to let all three mode inputs go low, since that would place the chip in PROM mode, causing increased current dissipation.

17.4.2 Recovery from Hardware Standby Mode

Recovery from the hardware standby mode requires inputs at both the mode and RES pins.

When the mode pins are set to mode 1, 2, 3, 4, or 7, the clock oscillator begins running. The $\overline{\text{RES}}$ pin should be low at this time and should be held low long enough for the clock to stabilize. When the $\overline{\text{RES}}$ pin changes from low to high, the reset sequence is executed and the chip returns to the program execution state.

17.4.3 Timing Sequence of Hardware Standby Mode

Figure 17-2 shows the usual sequence for entering and leaving the hardware standby mode.

First the $\overline{\text{RES}}$ pin goes low, placing the chip in the reset state. Then the mode pins are set to mode 6, placing the chip in the hardware standby mode and stopping the clock. In the recovery sequence first the mode pins are set to mode 1, 2, 3, 4, or 7; then after the clock stabilizes, the $\overline{\text{RES}}$ pin is returned to the high level.

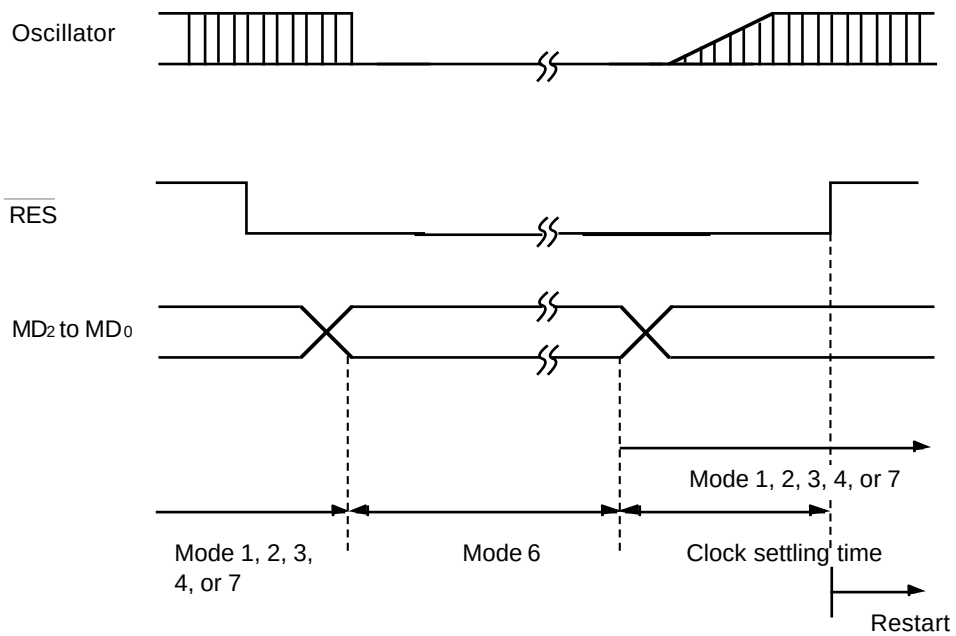


Figure 17-2 Hardware Standby Sequence

Section 18 Electrical Specifications

18.1 Absolute Maximum Ratings

Table 18-1 lists the absolute maximum ratings.

Table 18-1 Absolute Maximum Ratings

Item	Symbol	Rating	Unit
Supply voltage	V _{CC}	−0.3 to +0.7	V
Programming voltage	V _{PP}	−0.3 to +13.5	V
Input voltage (except port 6)	V _{in}	−0.3 to V _{CC} + 0.3	V
Input voltage (port 6)	V _{in}	−0.3 to AV _{CC} + 0.3	V
Analog supply voltage	AV _{CC}	−0.3 to +7.0	V
Analog input voltage	VA _N	−0.3 to AV _{CC} + 0.3	V
Operating temperature	T _{opr}	Regular specifications: −20 to +75	°C
		Wide-range specifications: −40 to +85	°C
Storage temperature	T _{stg}	−55 to +125	°C

Note: Permanent damage to the chip may result if the absolute maximum ratings shown in table 18-1 are exceeded.

18.2 Electrical Characteristics

18.2.1 DC Characteristics

Table 18-2 lists the DC characteristics.

Table 18-2 DC Characteristics

Conditions: $V_{CC} = AV_{CC} = 5.0\text{ V} \pm 10\%^{*1}$, $V_{SS} = AV_{SS} = 0\text{ V}$,

$T_a = -20\text{ to }75^\circ\text{C}$ (Regular specifications)

$T_a = -40\text{ to }85^\circ\text{C}$ (Wide-range specifications)

Item		Symbol	min	typ	max	Unit	Test
							Conditions
Input high voltage	RES, MD ₂ , MD ₁ , MD ₀	V _{IH}	$V_{CC} - 0.7$	—	$V_{CC} + 0.3$	V	
	EXTAL		$V_{CC} \times 0.7$	—	$V_{CC} + 0.3$	V	
	Port 6		2.2	—	$AV_{CC} + 0.3$	V	
	Other input pins (except port 5)		2.2	—	$V_{CC} + 0.3$	V	
Input low voltage	RES, MD ₂ , MD ₁ , MD ₀	V _{IL}	-0.3	—	0.5	V	
	Other input pins (except port 5)		-0.3	—	0.8		
Schmitt trigger input voltage	Port 5	VT ⁻	1.0	—	2.5	V	
		VT ⁺	2.0	—	3.5	V	
		VT ⁺ - VT ⁻	0.4	—	—	V	
Input leakage current	RES	I _{in}	—	—	10.0	μA	V _{in} = 0.5 to
	NMI, MD ₂ , MD ₁ , MD ₀ ,		—	—	1.0	μA	V _{CC} - 0.5 V
	Port 6		—	—	1.0	μA	V _{in} = 0.5 to AV _{CC} - 0.5 V
Leakage current in 3-state (off state)	Port 7, ports 5 to 1	I _{TSI}	—	—	1.0	μA	V _{in} = 0.5 to V _{CC} - 0.5 V
Input pull-up MOS current	Ports 3 and 4	-I _P	50	—	200	μA	V _{in} = 0 V
Output high voltage	All output pins	V _{OH}	$V_{CC} - 0.5$	—	—	V	I _{OH} = -200 μA
			3.5	—	—	V	I _{OH} = -1 mA
Output low voltage	All output pins (except RES)	V _{OL}	—	—	0.4	V	I _{OL} = 1.6 mA
	Port 3		—	—	1.0	V	I _{OL} = 8 mA
			—	—	1.2	V	I _{OL} = 10 mA
	RES		—	—	0.4	V	I _{OL} = 2.6 mA
Input capacitance	RES	C _{in}	—	—	60	pF	V _{in} = 0 V
	NMI		—	—	30	pF	f = 1 MHz
	All input pins except RES		—	—	15	pF	T _a = 25°C

Table 18-2 DC Characteristics (cont)

		Symbol					Test
Item			min	typ	max	Unit	Conditions
Current dissipation*	Normal operation	I _{CC}	—	20	30	mA	f = 6 MHz
			—	25	40	mA	f = 8 MHz
			—	30	50	mA	f = 10 MHz
			—	12	20	mA	f = 6 MHz
	Sleep mode		—	16	25	mA	f = 8 MHz
			—	20	30	mA	f = 10 MHz
	Standby		—	0.01	5.0	μA	
Analog supply current	During A/D conversion	A _{I_{CC}}	—	0.6	2.0	mA	
	While waiting		—	0.01	5.0	μA	
RAM standby voltage		V _{RAM}	2.0	—	—	V	

Note: AV_{CC} must be connected to a power supply even when the A/D converter is not used.

* Current dissipation values assume that V_{IH} min = V_{CC} – 0.5 V, V_{IL} max = 0.5 V, all output pins are in the no-load state, and all MOS input pull-ups are off.

Table 18-3 Allowable Output Current Sink Values

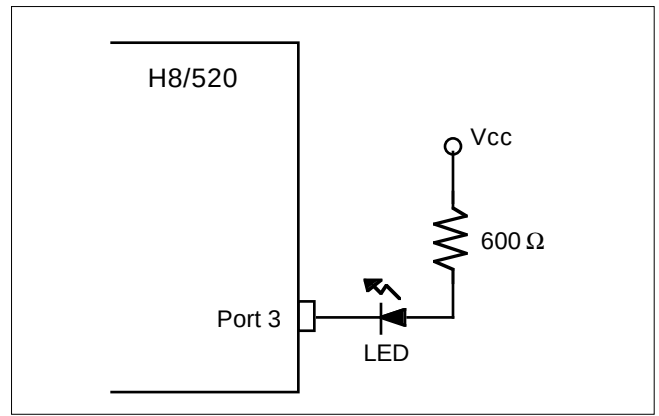
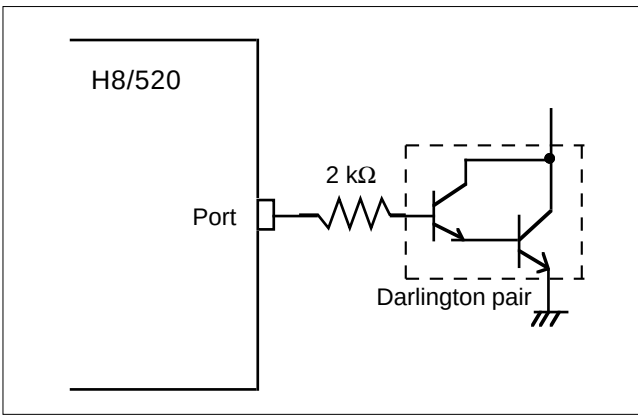
Conditions: V_{CC} = AV_{CC} = 5.0 V ± 10%, V_{SS} = AV_{SS} = 0 V,

T_a = –20 to 75°C (Regular specifications)

T_a = –40 to 85°C (Wide-range specifications)

Item		Symbol	min	typ	max	Unit
Allowable output low current sink (per pin)	Port 3	I _{OL}	—	—	10	mA
	RES		—	—	2.6	mA
	Other output pins		—	—	2.0	mA
Allowable output low current sink (total)	Port 3, total of 8 pins	Σ I _{OL}	—	—	40	mA
	Total of all other output pins		—	—	80	mA
Allowable output high current sink (per pin)	All output pins	–I _{OH}	—	—	2.0	mA
Allowable output high current sink (total)	Total of all output pins	Σ –I _{OH}	—	—	40	mA

Note: To avoid degrading the reliability of the chip, be careful not to exceed the output current sink values in table 18-3. In particular, when driving a Darlington transistor pair or LED directly, be sure to insert a current-limiting resistor in the output path. See figures 18-1 and 18-2.



Darlington Transistor Pair

Figure 18-1 Example of Circuit for Driving a

18.2.2 AC Characteristics

The AC characteristics of the H8/520 chip are listed in three tables. Bus timing parameters are given in table 18-4, control signal timing parameters in table 18-5, and timing parameters of the on-chip supporting modules in table 18-6. See figure 18-3 for the output load circuit.

Table 18-4 Bus Timing

Conditions: $V_{CC} = 5.0 \text{ V} \pm 10\%$, $\phi = 0.5$ to 10 MHz, $V_{SS} = 0 \text{ V}$
 $T_a = -20$ to 75°C (Regular specifications)
 $T_a = -40$ to 85°C (Wide-range specifications)

Item	Symbol	6 MHz		8 MHz		10 MHz		Unit	Test Conditions
		min	max	min	max	min	max		
Clock cycle time	t_{cyc}	166.7	2000	125	2000	100	2000	ns	See figure 18-4
Clock pulse width low	t_{CL}	65	—	45	—	35	—	ns	
Clock pulse width high	t_{CH}	65	—	45	—	35	—	ns	
Clock rise time	t_{Cr}	—	15	—	15	—	15	ns	
Clock fall time	t_{Cf}	—	15	—	15	—	15	ns	
Address delay time	t_{AD}	—	70	—	60	—	55	ns	See figure 18-4
Address hold time	t_{AH}	30	—	25	—	20	—	ns	
$\overline{RD}$ delay time 1	t_{RDD1}	—	70	—	60	—	40	ns	
$\overline{RD}$ delay time 2	t_{RDD2}	—	70	—	60	—	50	ns	
$\overline{WR}$ delay time 1	t_{WRD1}	—	70	—	60	—	50	ns	
$\overline{WR}$ delay time 2	t_{WRD2}	—	70	—	60	—	50	ns	
Write data strobe pulse width	t_{DSWW}	200	—	150	—	120	—	ns	
Address setup time 1	t_{AS1}	25	—	20	—	15	—	ns	
Address setup time 2	t_{AS2}	105	—	80	—	65	—	ns	
Read data setup time	t_{RDS}	60	—	50	—	40	—	ns	

Table 18-4 Bus Timing (cont)

Item	Symbol	6 MHz		8 MHz		10 MHz		Unit	Test Conditions
		min	max	min	max	min	max		
Read data hold time	tRDH	0	—	0	—	0	—	ns	See figure 18-4
Read data access time	tACC	—	280	—	190	—	160	ns	
Write data delay time	tWDD	—	70	—	60	—	60	ns	
Write data setup time	tWDS	30	—	15	—	10	—	ns	
Write data hold time	tWDH	30	—	25	—	20	—	ns	See figure 18-5
Wait setup time	tWTS	40	—	40	—	40	—	ns	
Wait hold time	tWTH	10	—	10	—	10	—	ns	

Table 18-5 Control Signal Timing

Conditions: $V_{CC} = 5.0\text{ V} \pm 10\%$, $\phi = 0.5$ to 10 MHz, $V_{SS} = 0\text{ V}$

$T_a = -20$ to 75°C (Regular specifications)

$T_a = -40$ to 85°C (Wide-range specifications)

Item	Symbol	6 MHz		8 MHz		10 MHz		Unit	Test Conditions
		min	max	min	max	min	max		
RES setup time	tRESS	200	—	200	—	200	—	ns	See figure 18-6
RES pulse width 1*	tRESW1	6.0	—	6.0	—	6.0	—	t _{cyc}	
RES pulse width 2*	tRESW2	520	—	520	—	520	—	t _{cyc}	See figure 18-7
RES output delay time	tRESO	—	100	—	100	—	100	ns	
RES output pulse width	tRESOW	132	—	132	—	132	—	t _{cyc}	See figure 18-6
Mode programming setup time	tMDS	4.0	—	4.0	—	4.0	—	t _{cyc}	
NMI setup time	tNMIS	150	—	150	—	150	—	ns	See figure 18-8
NMI hold time	tNMIH	10	—	10	—	10	—	ns	
IRQ ₀ setup time	tIRQ0S	50	—	50	—	50	—	ns	See figure 18-9
IRQ ₁ to IRQ ₇ setup time	tIRQ1S	50	—	50	—	50	—	ns	
IRQ ₁ to IRQ ₇ hold time	tIRQ1H	10	—	10	—	10	—	ns	See figure 17-1
NMI pulse width (for recovery from software standby mode)	tNMIW	200	—	200	—	200	—	ns	
A/D trigger setup time	tTRGS	50	—	50	—	50	—	ns	See figure 18-18
A/D trigger hold time	tTRGH	10	—	10	—	10	—	ns	
Crystal oscillator settling time (reset)	tOSC1	20	—	20	—	20	—	ms	See figure 17-1
Crystal oscillator settling time (software standby)	tOSC2	10	—	10	—	10	—	ms	

Note: * tRESW2 applies when the RSTOE bit in the reset control/status register (RSTCR) is set to 1. tRESW1 applies when RSTOE is cleared to 0. tRESW1 also applies at power-up.

Table 18-6 Timing Conditions of On-Chip Supporting Modules

Conditions: $V_{CC} = 5.0\text{ V} \pm 10\%$, $\phi = 0.5$ to 10 MHz , $V_{SS} = 0\text{ V}$
 $T_a = -20$ to 75°C (Regular specifications)
 $T_a = -40$ to 85°C (Wide-range specifications)

Item	Symbol	6 MHz		8 MHz		10 MHz		Test	
		min	max	min	max	min	max	Unit	Conditions
FRT	Timer output delay time	t_{FTOD}	— 100	— 100	— 100	— 100	— 100	ns	See figure 18-11
	Timer input setup time	t_{FTIS}	50 —	50 —	50 —	50 —	50 —	ns	
	Timer clock input setup time	t_{FTCS}	50 —	50 —	50 —	50 —	50 —	ns	See figure 18-12
	Timer clock pulse width	t_{FTCW}	1.5 —	1.5 —	1.5 —	1.5 —	1.5 —	t_{cyc}	
TMR	Timer output delay time	t_{TMOD}	— 100	— 100	— 100	— 100	— 100	ns	See figure 18-13
	Timer clock input setup time	t_{TMCS}	50 —	50 —	50 —	50 —	50 —	ns	See figure 18-14
	Timer clock pulse width	t_{TMCW}	1.5 —	1.5 —	1.5 —	1.5 —	1.5 —	t_{cyc}	
	Timer reset input setup time	t_{TMRS}	50 —	50 —	50 —	50 —	50 —	ns	See figure 18-15
SCI	Input clock cycle	(Async) t_{Scyc}	2 —	2 —	2 —	2 —	2 —	t_{cyc}	See figure 18-16
		(Sync)	4 —	4 —	4 —	4 —	4 —	t_{cyc}	
	Input clock pulse width	t_{SCKW}	0.4 0.6	0.4 0.6	0.4 0.6	0.4 0.6	0.4 0.6	t_{Scyc}	
	Transmit data delay time	(Sync) t_{TXD}	— 100	— 100	— 100	— 100	— 100	ns	
	Receive data setup time	(Sync) t_{RXS}	100 —	100 —	100 —	100 —	100 —	ns	
	Receive data hold time	(Sync) t_{RXH}	— 100	— 100	— 100	— 100	— 100	ns	
Ports	Output data delay time	t_{PWD}	— 100	— 100	— 100	— 100	— 100	ns	See figure 18-10
	Input data setup time	t_{PRS}	50 —	50 —	50 —	50 —	50 —	ns	
	Input data hold time	t_{PRH}	50 —	50 —	50 —	50 —	50 —	ns	

• **Measurement Conditions for AC Characteristics**

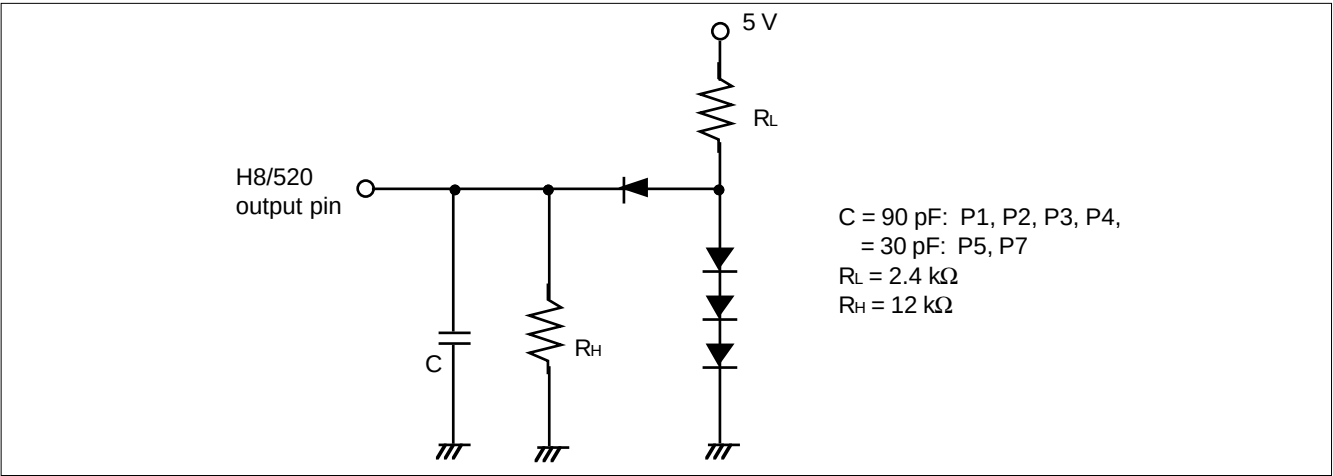


Figure 18-3 Output Load Circuit

18.2.3 A/D Converter Characteristics

Table 18-7 lists the characteristics of the on-chip A/D converter.

Table 18-7 (1) A/D Converter Characteristics

Conditions: $V_{CC} = AV_{CC} = 5.0\text{ V} \pm 10\%$, $V_{SS} = AV_{SS} = 0\text{ V}$,
 $T_a = -40\text{ to }85^{\circ}\text{C}$ (Wide-range specifications)

Item	6 MHz			8 MHz			10 MHz			Unit
	min	typ	max	min	typ	max	min	typ	max	
Resolution	10	10	10	10	10	10	10	10	10	Bits
Conversion time	—	—	23.0	—	—	17.25	—	—	13.8	μs
Analog input capacitance	—	—	20	—	—	20	—	—	20	pF
Allowable signal-source impedance	—	—	10	—	—	10	—	—	10	k Ω
Nonlinearity error	—	—	± 2.0	—	—	± 2.0	—	—	± 2.0	LSB
Offset error	—	—	± 2.0	—	—	± 2.0	—	—	± 2.0	LSB
Full-scale error	—	—	± 2.0	—	—	± 2.0	—	—	± 2.0	LSB
Quantizing error	—	—	± 0.5	—	—	± 0.5	—	—	± 0.5	LSB
Absolute accuracy	—	—	± 2.5	—	—	± 2.5	—	—	± 2.5	LSB

Table 18-7 (2) A/D Converter Characteristics

Conditions: $V_{CC} = AV_{CC} = 5.0\text{ V} \pm 10\%$, $V_{SS} = AV_{SS} = 0\text{ V}$,
 $T_a = -20\text{ to }75^{\circ}\text{C}$ (Regular specifications)

Item	6 MHz			8 MHz			10 MHz			Unit
	min	typ	max	min	typ	max	min	typ	max	
Resolution	10	10	10	10	10	10	10	10	10	Bits
Conversion time	—	—	23.0	—	—	17.25	—	—	13.8	μs
Analog input capacitance	—	—	20	—	—	20	—	—	20	pF
Allowable signal-source impedance	—	—	10	—	—	10	—	—	10	k Ω
Nonlinearity error	—	—	± 3.5	—	—	± 3.5	—	—	± 3.5	LSB
Offset error	—	—	± 3.5	—	—	± 3.5	—	—	± 3.5	LSB
Full-scale error	—	—	± 3.5	—	—	± 3.5	—	—	± 3.5	LSB
Quantizing error	—	—	± 0.5	—	—	± 0.5	—	—	± 0.5	LSB
Absolute accuracy	—	—	± 4.0	—	—	± 4.0	—	—	± 4.0	LSB

18.3 MCU Operational Timing

This section provides the following timing charts:

18.3.1 Bus timing	Figures 18-4 and 18-5
18.3.2 Control Signal Timing	Figures 18-6 to 18-8
18.3.3 Clock Timing	Figure 18-9
18.3.4 I/O Port Timing	Figure 18-10
18.3.5 16-Bit Free-Running Timer Timing	Figures 18-11 and 18-12
18.3.6 8-Bit Timer Timing	Figures 18-13 to 18-15
18.3.7 SCI Timing	Figures 18-16 and 18-17

18.3.1 Bus Timing

1. Basic Bus Cycle (without Wait States) in Expanded Modes

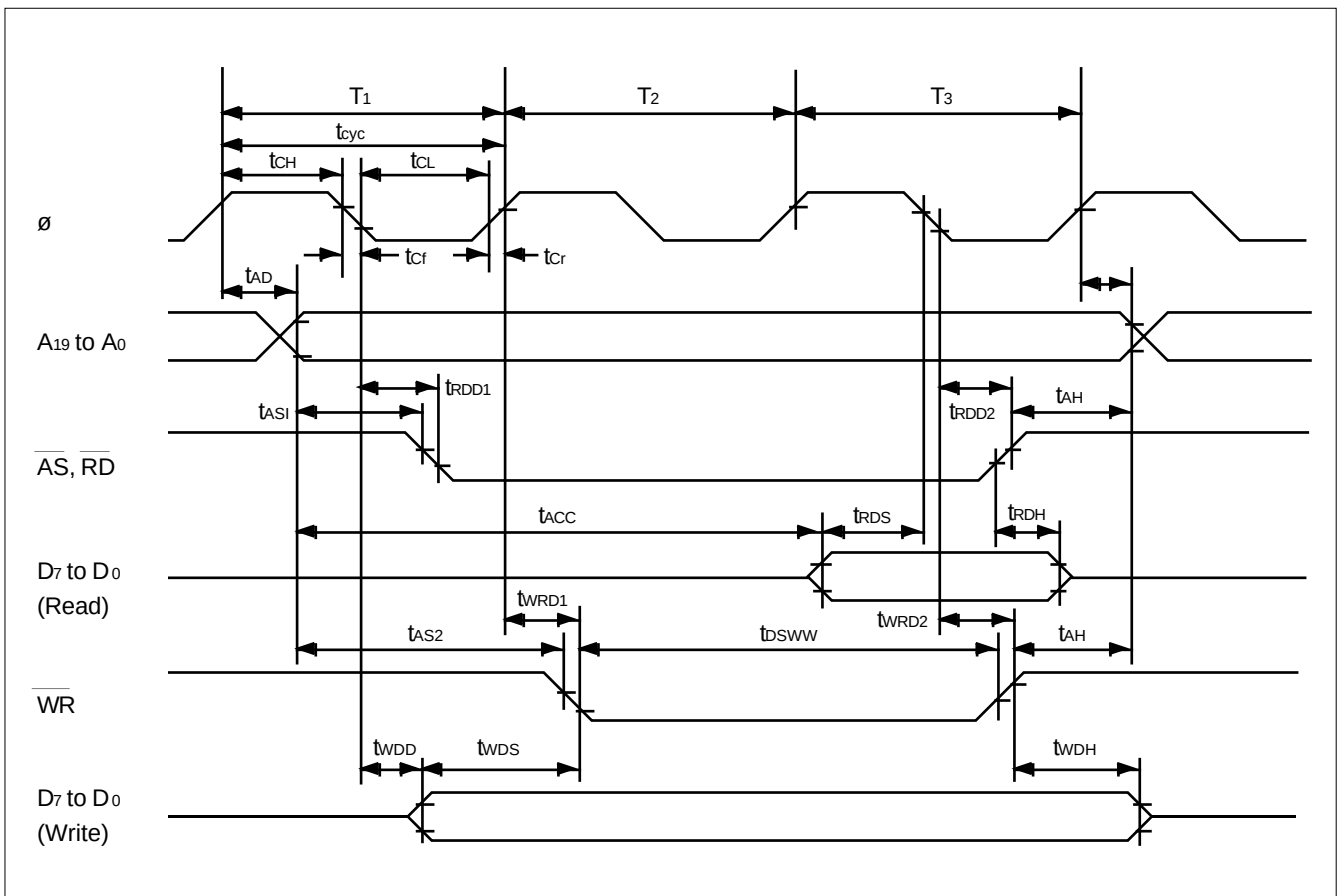


Figure 18-4 Basic Bus Cycle (without Wait States) in Expanded Modes

2. Basic Bus Cycle (with 1 Wait State) in Expanded Modes

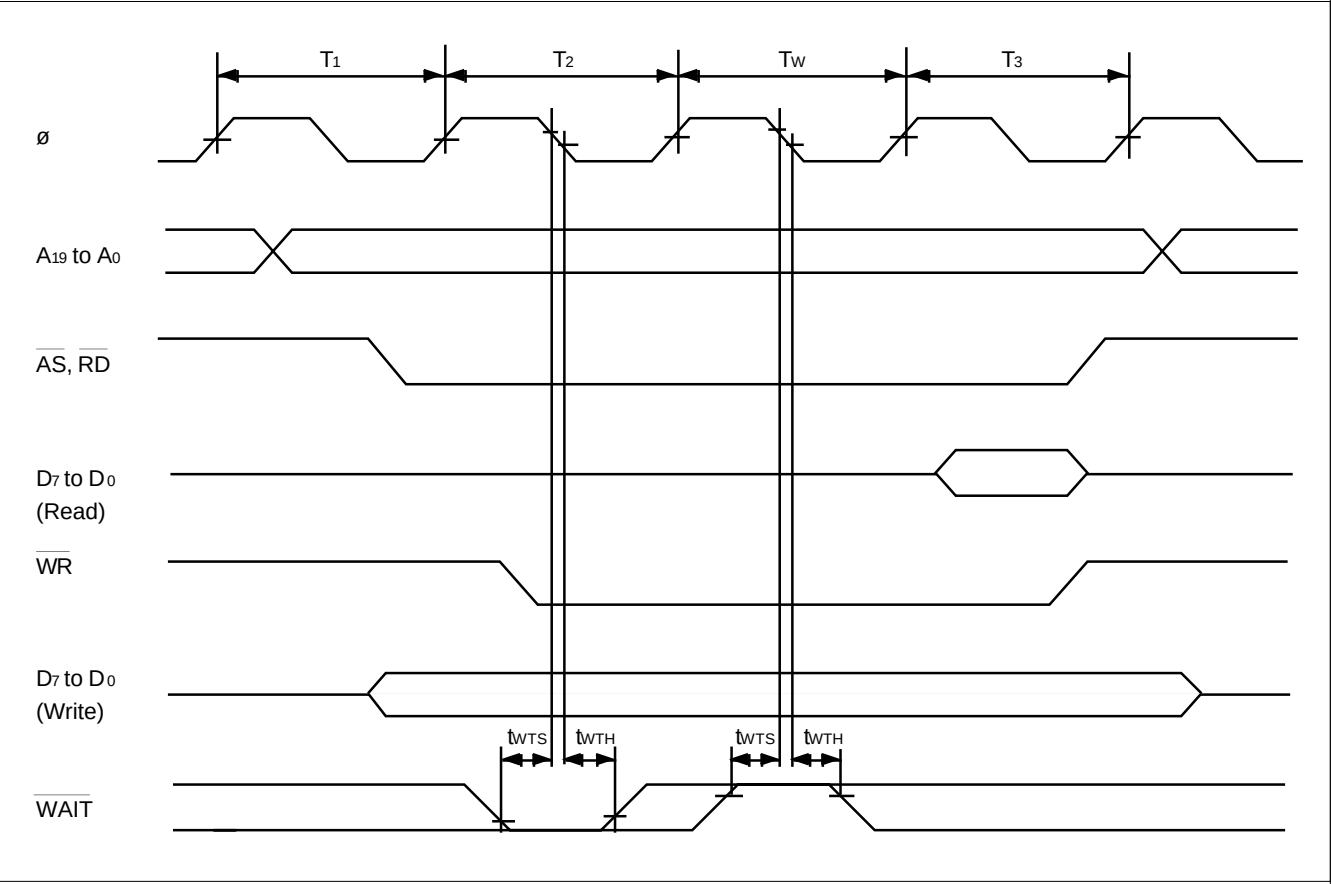


Figure 18-5 Basic Bus Cycle (with 1 Wait State) in Expanded Modes

18.3.2 Control Signal Timing

1. Reset Input Timing

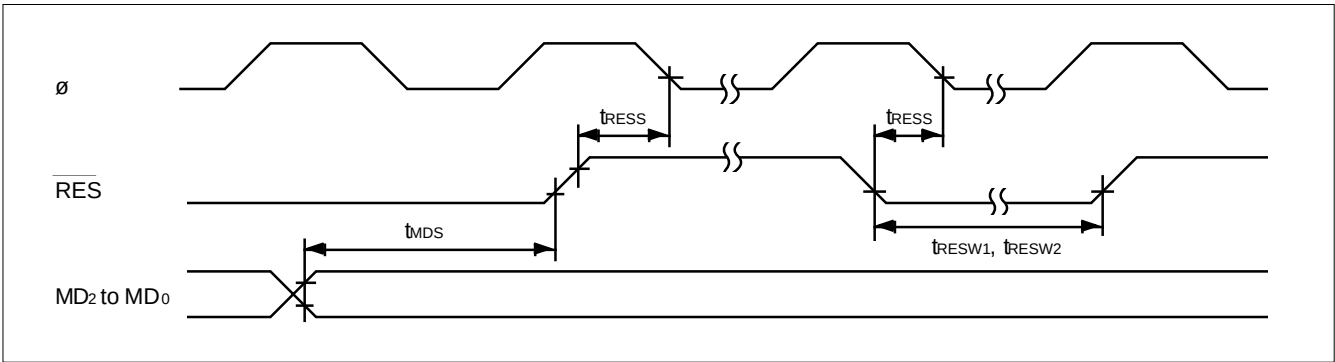


Figure 18-6 Reset Input Timing

2. Reset Output Timing

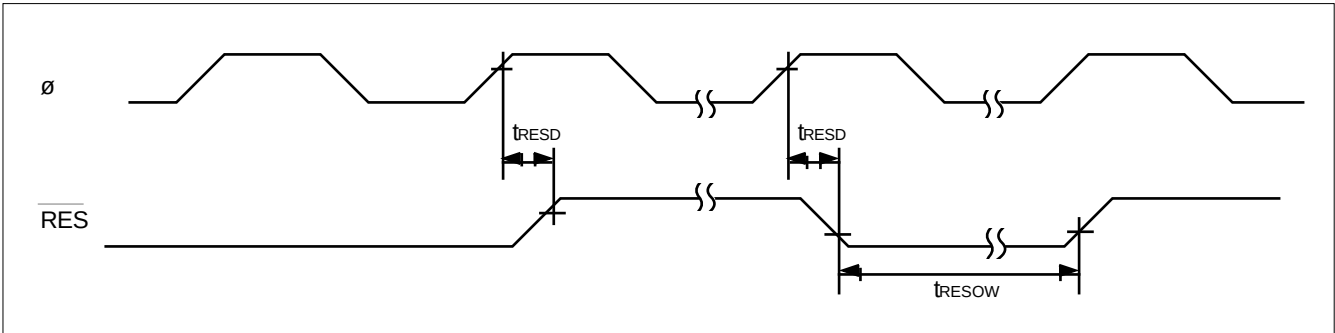


Figure 18-7 Reset Output Timing

3. Interrupt Input Timing

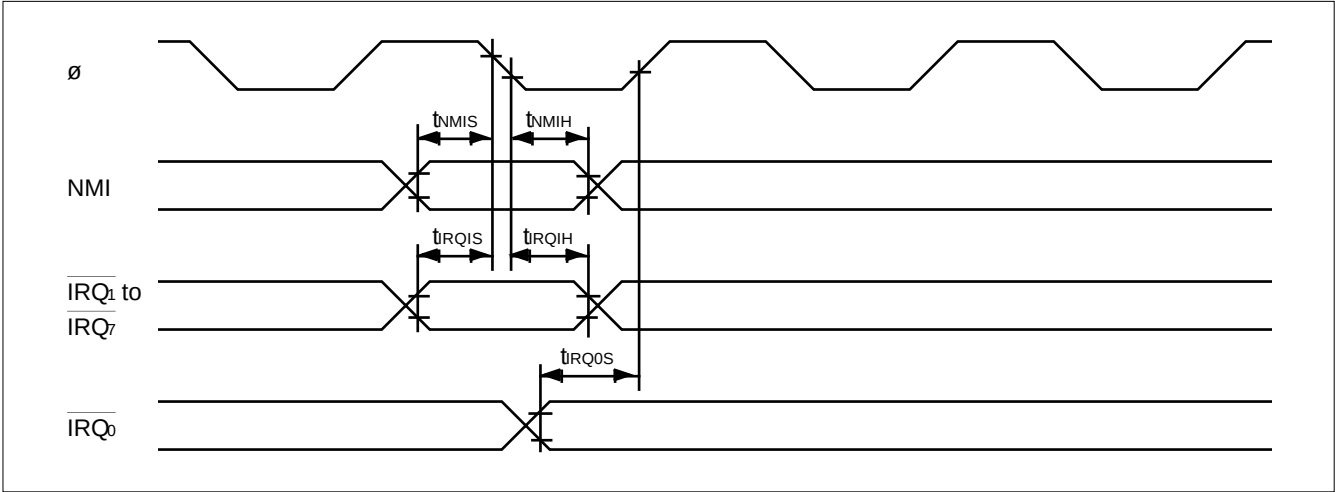


Figure 18-8 Interrupt Input Timing

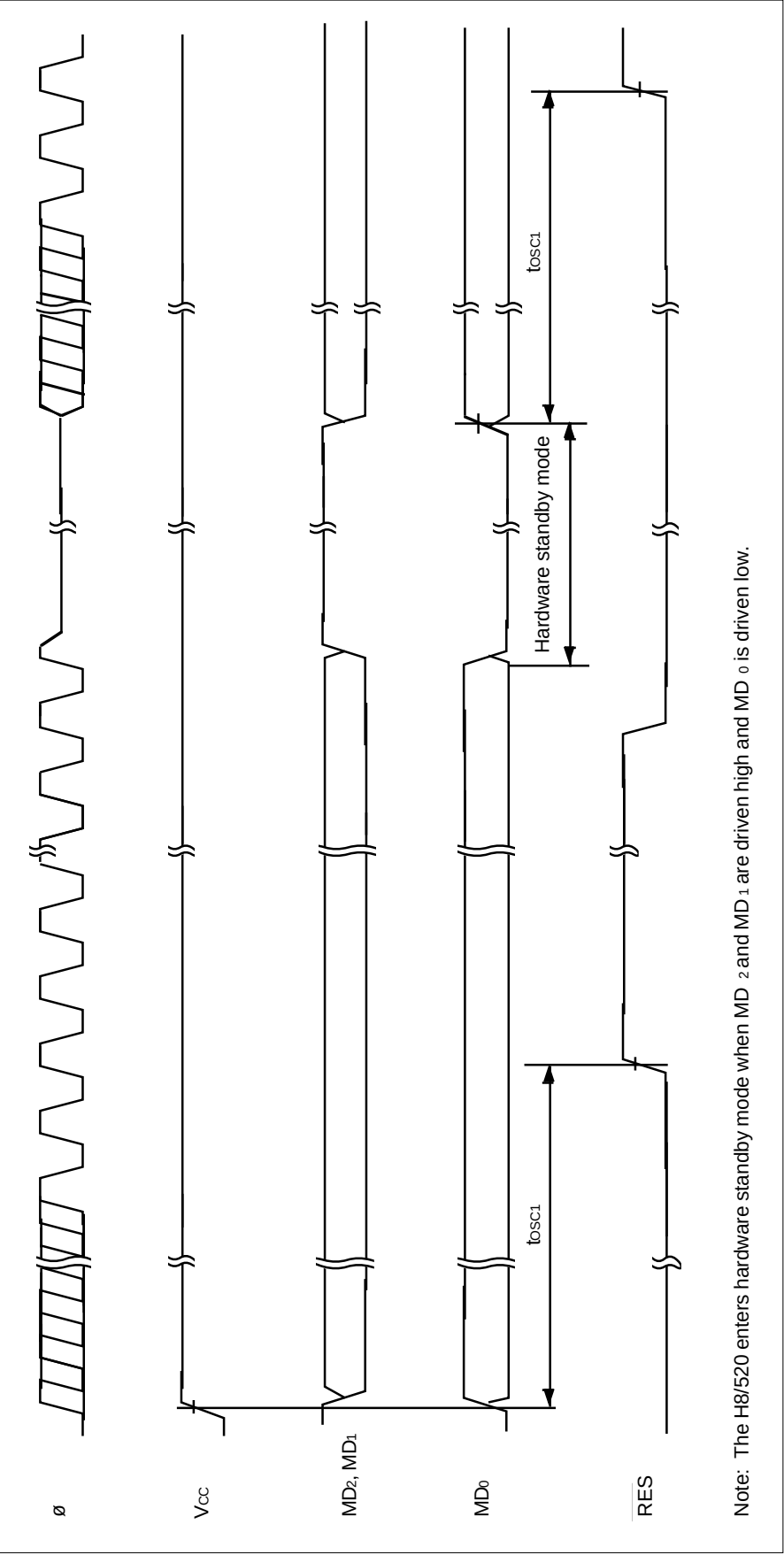


Figure 18-9 Clock Oscillator Stabilization

18.3.4 I/O Port Timing

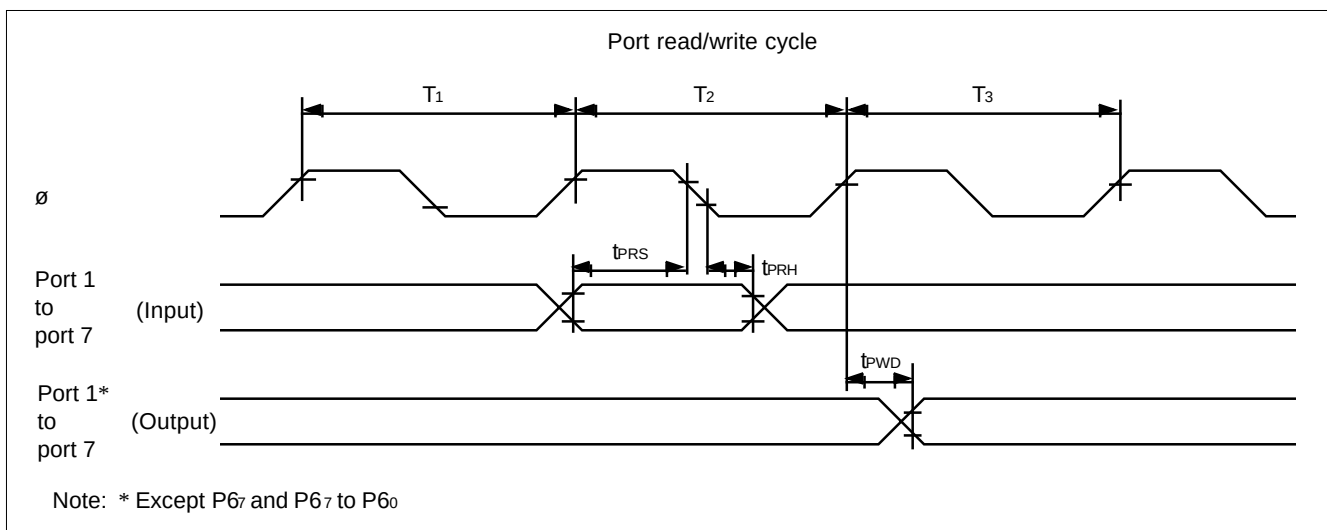


Figure 18-10 I/O Port Input/Output Timing

18.3.5 16-Bit Free-Running Timer Timing

1. Free-Running Timer Input/Output Timing

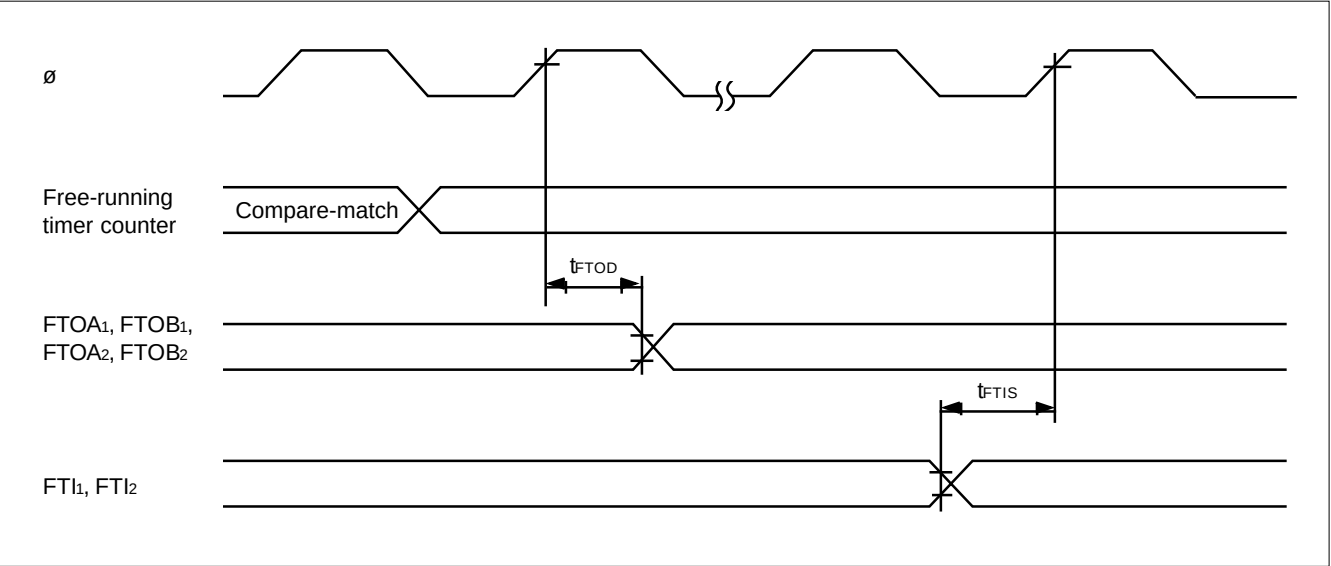


Figure 18-11 Free-Running Timer Input/Output Timing

2. External Clock Input Timing for Free-Running Timers

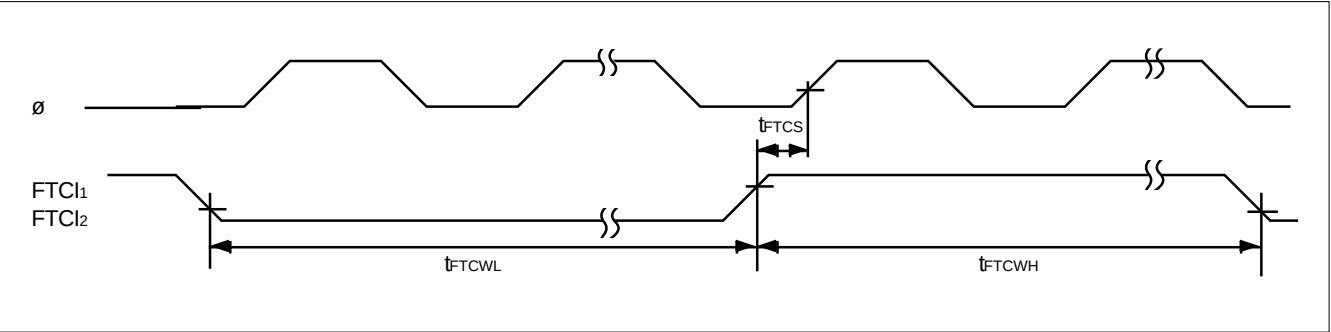


Figure 18-12 External Clock Input Timing for Free-Running Timers

18.3.6 8-Bit Timer Timing

1. 8-Bit Timer Output Timing

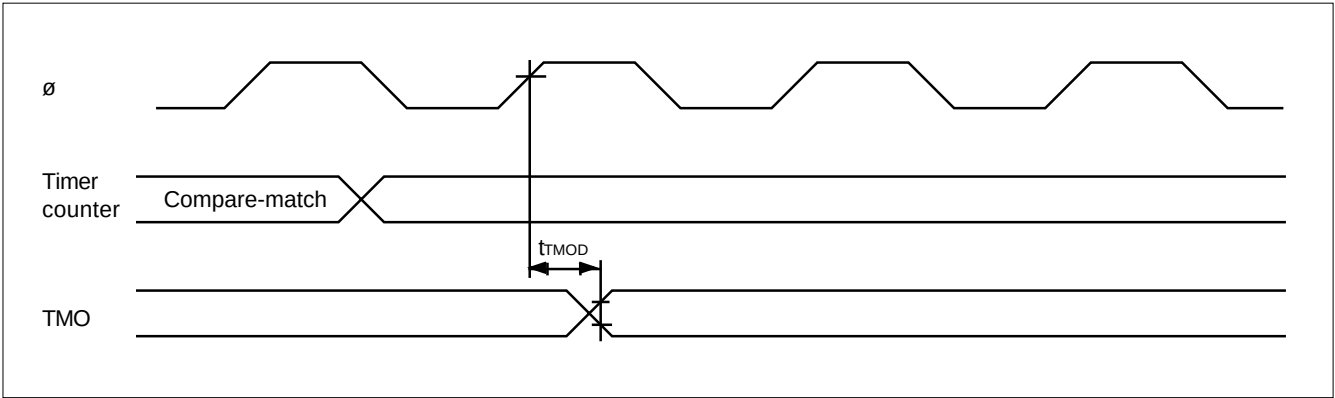


Figure 18-13 8-Bit Timer Output Timing

2. 8-Bit Timer Clock Input Timing

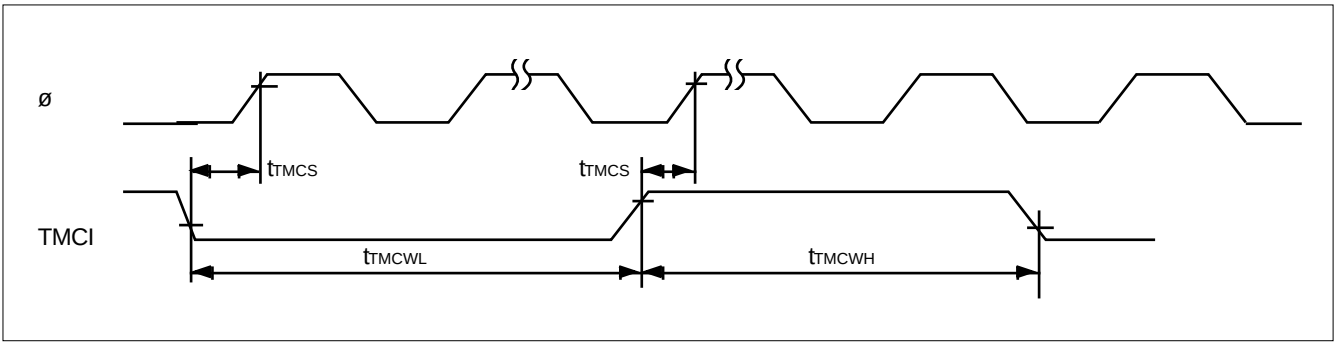


Figure 18-14 8-Bit Timer Clock Input Timing

3. 8-Bit Timer Reset Input Timing

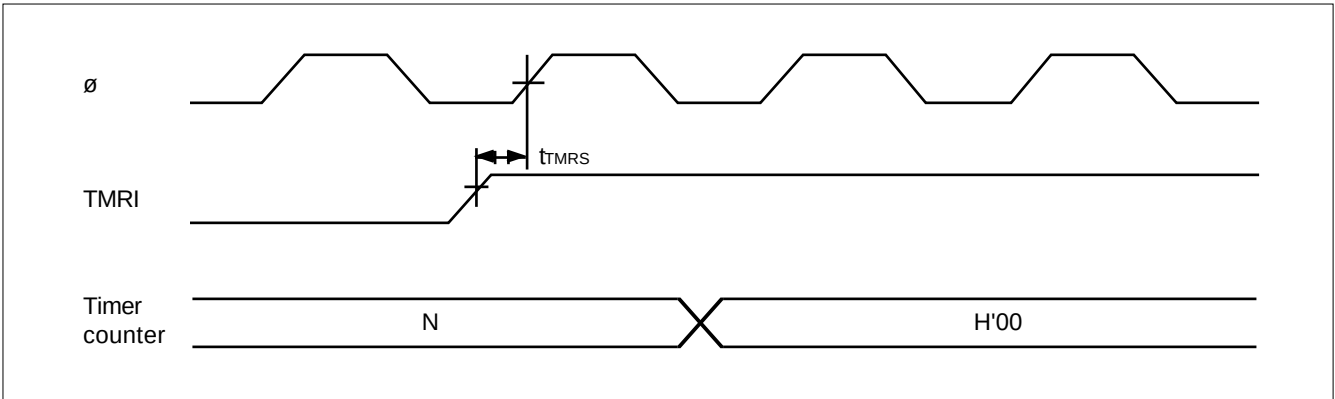


Figure 18-15 8-Bit Timer Reset Input Timing

18.3.7 Serial Communication Interface Timing

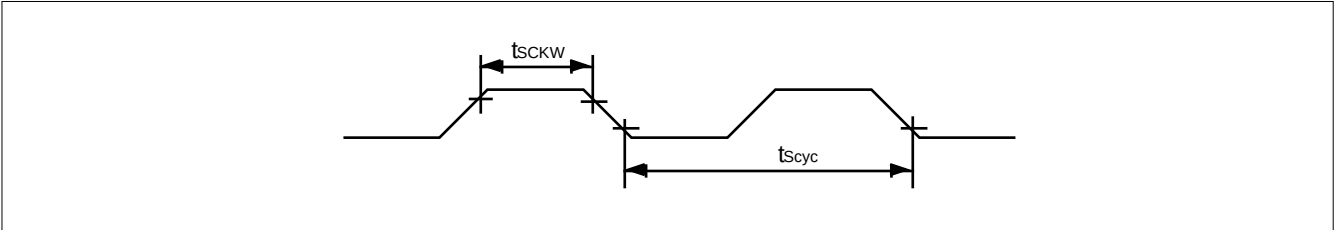


Figure 18-16 SCI Input Clock Timing

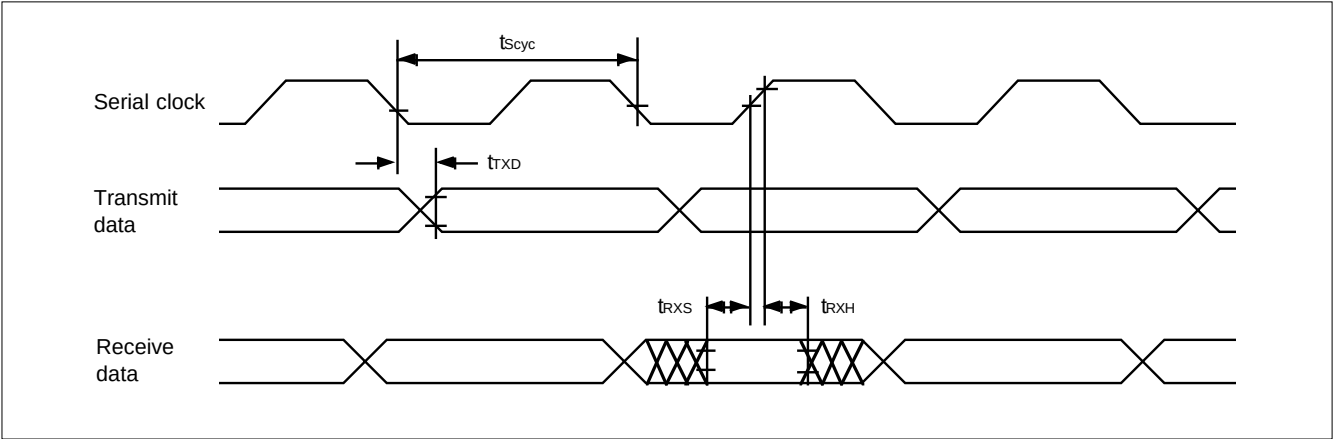


Figure 18-17 SCI Input/Output Timing (Synchronous Mode)

18.3.8 A/D External Trigger Input Timing

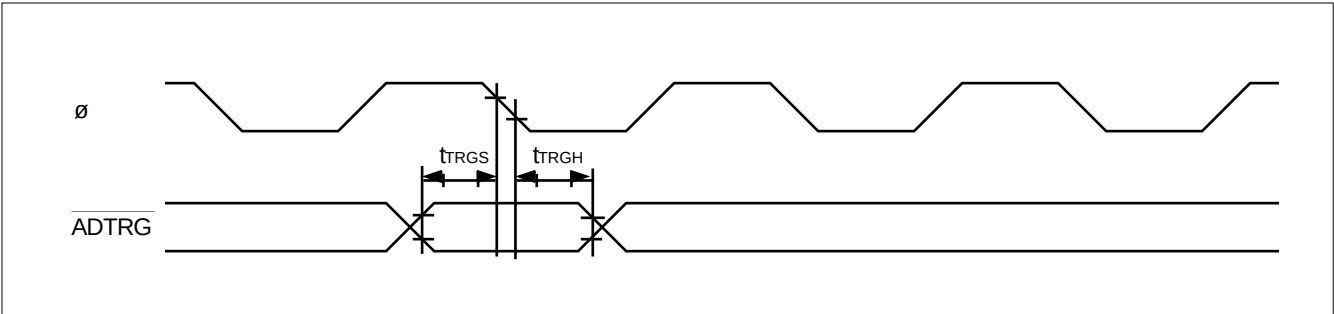


Figure 18-18 A/D External Trigger Input Timing

Appendix A Instructions

A.1 Instruction Set

Operation Notation

Rd	General register (destination operand)
Rs	General register (source operand)
Rn	General register
(EAd)	Destination operand
(EAs)	Source operand
CCR	Condition code register
N	N (Negative) flag in CCR
Z	Z (Zero) flag in CCR
V	V (Overflow) flag in CCR
C	C (Carry) flag in CCR
CR	Control register
PC	Program counter
CP	Code page register
SP	Stack pointer

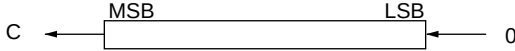
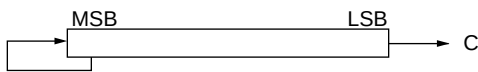
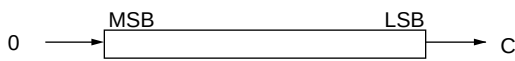
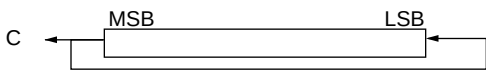
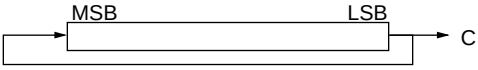
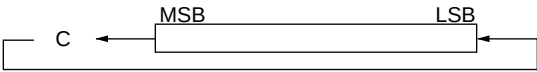
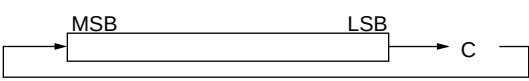
FP	Frame pointer
#IMM	Immediate data
disp	Displacement
+	Add
−	Subtract
×	Multiply
÷	Divide
^	Logical AND
∨	Logical OR
⊕	Logical exclusive OR
→	Move
↔	Swap
¬	Logical NOT

Condition Code Symbols

↑	Changed after instruction execution
0	Cleared to zero
1	Set to 1
—	Value before operation is retained
Δ	Varies depending on conditions

Instruction set

Mnemonic	Operation	Size	CCR Bit			
		B/W	N	Z	V	C
Data transfer	MOV: G (EAs) → Rd Rs → (EAd) #IMM → (EAd)	B/W	↑	↑	0	—
	MOV: E #IMM → Rd (short format)	B	↑	↑	0	—
	MOV: F @ (d: 8, FP) → Rd Rs → @ (d: 8, FP) (short format)	B/W	↑	↑	0	—
	MOV: I #IMM → Rd (short format)	W	↑	↑	0	—
	MOV: L (@aa: 8) → Rd (short format)	B/W	↑	↑	0	—
	MOV: S Rs → (@aa: 8) (short format)	B/W	↑	↑	0	—
	LDM @ SP + → Rn (register list)	W	—	—	—	—
	STM Rn (register list) → @ – SP	W	—	—	—	—
	XCH Rs ↔ Rd	W	—	—	—	—
	SWAP Rd (upper byte) ↔ Rd (lower byte)	B	↑	↑	0	—
	(MOVTPE) Cannot be used in the H8/520					
	(MOVFPPE) Cannot be used in the H8/520					
Arithmetic operations	ADD: G Rd + (EAs) → Rd	B/W	↑	↑	↑	↑
	ADD: Q (EAd) + #IMM → (EAd) (#IMM = ±1, ±2) (short format)	B/W	↑	↑	↑	↑
	ADDS Rd + (EAs) → Rd (Rd is always word size)	B/W	—	—	—	—
	ADDX Rd + (EAs) + C → Rd	B/W	↑	↑	↑	↑
	DADD (Rd) ₁₀ + (Rs) ₁₀ + C → (Rd) ₁₀	B	—	↑	—	↑
	SUB Rd – (EAs) → Rd	B/W	↑	↑	↑	↑
	SUBS Rd – (EAs) → Rd	B/W	—	—	—	—
	SUBX Rd – (EAs) – C → Rd	B/W	↑	↑	↑	↑
	DSUB (Rd) ₁₀ – (Rs) ₁₀ – C → (Rd) ₁₀	B	—	↑	—	↑
	MULXU Rd × (EAs) → Rd 8 × 8 (Unsigned) 16 × 16	B/W	↑	↑	0	0
	DIVXU Rd ÷ (EAs) → Rd 16 ÷ 8 (Unsigned) 32 ÷ 16	B/W	↑	↑	↑	0
	CMP: G Rd – (EAs), Set CCR (EAd) – #IMM, Set CCR	B/W	↑	↑	↑	↑
	CMP: E Rd – #IMM, Set CCR (short format)	B	↑	↑	↑	↑
	CMP: I Rd – #IMM, Set CCR (short format)	W	↑	↑	↑	↑

			Size	CCR Bit			
Mnemonic		Operation	B/W	N	Z	V	C
Arithmetic operations	EXTS	(< Bit 7 > of < Rd >) → (< Bit 15 to 8 > of < Rd >)	B	↑	↑	0	0
	EXTU	0 → (< Bit 15 to 8 > of < Rd >)	B	0	↑	0	0
	TST	(EAd) – 0, Set CCR	B/W	↑	↑	0	0
	NEG	0 – (EAd) → (EAd)	B/W	↑	↑	0	↑
	CLR	0 → (EAd)	B/W	0	1	0	0
	TAS	(EAd) – 0, Set CCR (1) ₂ → (< Bit 7 > of < EAd >)	B	↑	↑	0	0
Shift operations	SHAL		B/W	↑	↑	↑	↑
	SHAR		B/W	↑	↑	0	↑
	SHLL		B/W	↑	↑	0	↑
	SHLR		B/W	0	↑	0	↑
	ROTL		B/W	↑	↑	0	↑
	ROTR		B/W	↑	↑	0	↑
	ROTXL		B/W	↑	↑	0	↑
	ROTXR		B/W	↑	↑	0	↑
Logic operations	AND	$Rd \wedge (EAs) \rightarrow Rd$	B/W	↑	↑	0	—
	OR	$Rd \vee (EAs) \rightarrow Rd$	B/W	↑	↑	0	—
	XOR	$Rd \oplus (EAs) \rightarrow Rd$	B/W	↑	↑	0	—
	NOT	$\neg (EAd) \rightarrow (EAd)$	B/W	↑	↑	0	—
Bit manipulations	BSET	$\neg (< \text{Bit number} > \text{ of } < EAd >) \rightarrow Z$ $1 \rightarrow (< \text{Bit number} > \text{ of } < Rn >)$	B/W	—	↑	—	—
	BCLR	$\neg (< \text{Bit number} > \text{ of } < EAd >) \rightarrow Z$ $0 \rightarrow (< \text{Bit number} > \text{ of } < Rn >)$	B/W	—	↑	—	—
	BTST	$\neg (< \text{Bit number} > \text{ of } < EAd >) \rightarrow Z$	B/W	—	↑	—	—
	BNOT	$\neg (< \text{Bit number} > \text{ of } < EAd >) \rightarrow Z$ → (< Bit number > of < Rn >)	B/W	—	↑	—	—

Mnemonic	Operation	Size	CCR Bit			
		B/W	N	Z	V	C
Branching instructions	Bcc	—	—	—	—	—
	If condition is true then PC + disp → PC else next;					
	Mnemonic	Description	Condition			
	BRA (BT)	Always (True)	True			
	BRN (BF)	Never (False)	False			
	BHI	High	$C \vee Z = 0$			
	BLS	Low or Same	$C \vee Z = 0$			
	Bcc (BHS)	Carry Clear (High or Same)	$C = 0$			
	BCS (BLO)	Carry Set (LOW)	$C = 1$			
	BNE	Not Equal	$Z = 0$			
	BEQ	Equal	$Z = 1$			
	BVC	oVerflow Clear	$V = 0$			
	BVS	oVerflow Set	$V = 1$			
	BPL	PLus	$N = 0$			
	BMI	MInus	$N = 1$			
	BGE	Greater or Equal	$N \oplus V = 0$			
	BLT	Less Than	$N \oplus V = 1$			
	BGT	Greater Than	$Z \vee (N \oplus V) = 0$			
	BLE	Less or Equal	$Z \vee (N \oplus V) = 1$			
JMP	Effective address → PC	—	—	—	—	—
PJMP	Effective address → CP, PC	—	—	—	—	—
BSR	PC → @ – SP	—	—	—	—	—
	PC + disp → PC					
JSR	PC → @ – SP	—	—	—	—	—
	Effective address → PC					
PJSR	PC → @ – SP	—	—	—	—	—
	CP → @ – SP					
	Effective address → CP, PC					
RTS	@ SP + → PC	—	—	—	—	—
PRTS	@ SP + → CP	—	—	—	—	—
	@ SP + → PC					
RTD	@ SP + → PC	—	—	—	—	—
	SP + #IMM → SP					
PRTD	@ SP + → CP	—	—	—	—	—
	@ SP + → PC					
	SP + #IMM → SP					
SCB	If condition is true then next;	—	—	—	—	—
SCB/F	else Rn – 1 → Rn;					
SCB/NE	If Rn = –1 then next;					
SCB/EQ	else PC + disp → PC;					
	Mnemonic	Description	Condition			
	SCB/F		False			
	SCB/NE	Not Equal	$Z = 0$			
	SCB/EQ	Equal	$Z = 1$			

Mnemonic		Operation	Size	CCR Bit			
			B/W	N	Z	V	C
System control	TRAPA	PC $\rightarrow$ @ – SP (If MAX MODE CP $\rightarrow$ @ – SP) SR $\rightarrow$ @ – SP (If MAX MODE < vector > $\rightarrow$ CP) < vector > $\rightarrow$ PC	—	—	—	—	—
	TRAP/VS	If V bit = 1 then TRAP else next;	—	—	—	—	—
	RTE	@ SP + $\rightarrow$ SR (If MAX MODE @ SP + $\rightarrow$ CP) @ SP + $\rightarrow$ PC	—	$\updownarrow$	$\updownarrow$	$\updownarrow$	$\updownarrow$
	LINK	FP (R6) $\rightarrow$ @ – SP SP $\rightarrow$ FP (R6) SP + #IMM $\rightarrow$ SP	—	—	—	—	—
	UNLK	FP (R6) $\rightarrow$ SP @SP + $\rightarrow$ FP	—	—	—	—	—
	SLEEP	Normal running mode $\rightarrow$ power-down state	—	—	—	—	—
	LDC	(EAs) $\rightarrow$ CR	B/W*	Δ	Δ	Δ	Δ
	STC	CR $\rightarrow$ (EAd)	B/W*	—	—	—	—
	ANDC	CR $\wedge$ #IMM $\rightarrow$ CR	B/W*	Δ	Δ	Δ	Δ
	ORC	CR $\vee$ #IMM $\rightarrow$ CR	B/W*	Δ	Δ	Δ	Δ
	XORC	CR $\oplus$ #IMM $\rightarrow$ CR	B/W*	Δ	Δ	Δ	Δ
	NOP	PC + 1 $\rightarrow$ PC	—	—	—	—	—

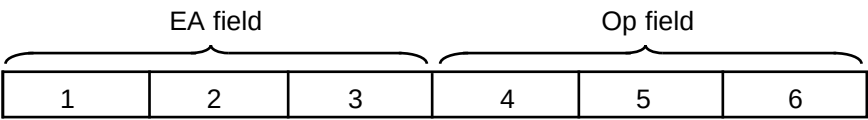
Note: * Depends on the CR.

A.2 Instruction Codes

Table A-1 shows the machine-language coding of each instruction.

- **How to read table A-1 (a) to (d).**

The general operand format consists of an effective address (EA) field and operation-code (OP) field specified in the following order:



Bytes 2, 3, 5, and 6 are not present in all instructions.

Instruction		Instruction										Operation code (OP)			Operation code (EA)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
																		1			2			3																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
																		Rn			@Rn			@{d:8, Rn}			@{d:16, Rn}			@-Rn			@Rn+			@aa:8			@aa:16			#xx:8			#xx:16																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					
												4			5			6																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																

- **r r r**: General register number field

r r r	Sz = 0 (Byte)				Sz = 1 (Word)	
	15	8	7	0	15	0
0 0 0	Not used		R0		R0	
0 0 1	Not used		R1		R1	
0 1 0	Not used		R2		R2	
0 1 1	Not used		R3		R3	
1 0 0	Not used		R4		R4	
1 0 1	Not used		R5		R5	
1 1 0	Not used		R6		R6	
1 1 1	Not used		R7		R7	

- **c c c**: Control register number field

c c c	Sz = 0 (Byte)				Sz = 1 (Word)	
0 0 0	(Not allowed*)				15	0
	15	8	7	0	SR	
0 0 1	Not used		CCR		(Not allowed)	
0 1 0	(Not allowed)				(Not allowed)	
0 1 1	Not used		BR		(Not allowed)	
1 0 0	Not used		EP		(Not allowed)	
1 0 1	Not used		DP		(Not allowed)	
1 1 0	(Not allowed)				(Not allowed)	
1 1 1	Not used		TP		(Not allowed)	

Note: * “Disallowed” means that this combination of bits must not be specified. Specifying a disallowed combination may cause abnormal results.

- Register list: A byte in which bits indicate general registers as follows.

Bit	7	6	5	4	3	2	1	0
	R7	R6	R5	R4	R3	R2	R1	R0

- #VEC: Four bits designating a vector number from 0 to 15. The vector numbers correspond to addresses of entries in the exception vector table as follows:

Vector Address			Vector Address		
#VEC	Minimum Mode	Maximum Mode	#VEC	Minimum Mode	Maximum Mode
0	H'0020 – H'0021	H'0040 – H'0043	8	H'0030 – H'0031	H'0060 – H'0063
1	H'0022 – H'0023	H'0044 – H'0047	9	H'0032 – H'0033	H'0064 – H'0067
2	H'0024 – H'0025	H'0048 – H'004B	A	H'0034 – H'0035	H'0068 – H'006B
3	H'0026 – H'0027	H'004C – H'004F	B	H'0036 – H'0037	H'006C – H'006F
4	H'0028 – H'0029	H'0050 – H'0053	C	H'0038 – H'0039	H'0070 – H'0073
5	H'002A – H'002B	H'0054 – H'0057	D	H'003A – H'003B	H'0074 – H'0077
6	H'002C – H'002D	H'0058 – H'005B	E	H'003C – H'003D	H'0078 – H'007B
7	H'002E – H'002F	H'005C – H'005F	F	H'003E – H'003F	H'007C – H'007F

- Examples of machine-language coding

Example 1: ADD:G.B @R0,R1

	EA Field	OP Field	Notes
Table A-1 (1)	1 1 0 1 Sz r r r	0 0 1 0 0 r r r	Machine code for ADD:G.B @Rs, Rd
Machine code	1 1 0 1 0 0 0 0	0 0 1 0 0 0 0 1	Sz = 0 (byte) Rs = R0, Rd = R1
	H'D021		

Example 2: ADD:G.W @H'11:8,R1

	EA Field		OP Field	Notes
Table A-1 (1)	0 0 0 0 Sz 1 0 1	0 0 0 1 0 0 0 1	0 0 1 0 0 r r r	Machine code for ADD:G.W @aa:8, Rd
Machine code	0 0 0 0 1 1 0 1	0 0 0 1 0 0 0 1	0 0 1 0 0 0 0 1	Sz = 1 (word) aa = H'11, Rd = R1
	H'0D1121			

Table A-1 (a) Machine Language Coding [General Format]

Instruction		Operation code (EA)										Operation code (OP)		
		Address- ing mode	1			2			3			4	5	6
			Rn	@Rn	@(d:8, Rn)	@(d:16, Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8	#xx:16		
Data transfer instruction	MOV:G.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		100d0rrr		
	MOV:G.W <EAs>,Rd	2	2	3	4	2	2	3	4	4		100d0rrr		
	MOV:G.B Rs,<EAd>		2	3	4	2	2	3	4	3		100d0rrr		
	MOV:G.W Rs,<EAd>		2	3	4	2	2	3	4	4		100d0rrr		
	MOV:G.B #xx:8,<EAd>		3	4	5	3	3	4	5			00000110	data	
	MOV:G.W #xx:16,<EAd>		4	5	6	4	4	5	6			00000111	data (H)	data (L)
	LDM.W @SP+,<register list>						2					00000010	register list	
	STM.W <register list>,@-SP						2					00010010	register list	
	XCH.W Rs,Rd	2										10010rrr		
	SWAP.B Rd	2										00010000		
Arithmetic operation instruction	(MOVTPE.B Rs,<EAd>)* ¹		3	4	5	3	3	4	5			00000000	10010rrr	
	(MOVFPPE.B <EAs>,Rd)* ¹		3	4	5	3	3	4	5			00000000	10000rrr	
	ADD:G.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		00100rrr		
	ADD:G.W <EAs>,Rd	2	2	3	4	2	2	3	4	4		00100rrr		
	ADD:Q.B #1,<EAd>* ²	2	2	3	4	2	2	3	4			00001000		
	ADD:Q.W #1,<EAd>* ²	2	2	3	4	2	2	3	4			00001000		
	ADD:Q.B #2,<EAd>* ²	2	2	3	4	2	2	3	4			00001001		
	ADD:Q.W #2,<EAd>* ²	2	2	3	4	2	2	3	4			00001001		
	ADD:Q.B #-1,<EAd>* ²	2	2	3	4	2	2	3	4			00001100		
	ADD:Q.W #-1,<EAd>* ²	2	2	3	4	2	2	3	4			00001100		
	ADD:Q.B #-2,<EAd>* ²	2	2	3	4	2	2	3	4			00001101		
	ADD:Q.W #-2,<EAd>* ²	2	2	3	4	2	2	3	4			00001101		
	ADDS.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		00101rrr		
	ADDS.W <EAs>,Rd	2	2	3	4	2	2	3	4	4		00101rrr		
	ADDX.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		10100rrr		
	ADDX.W <EAs>,Rd	2	2	3	4	2	2	3	4	4		10100rrr		

Notes: 1. Cannot be used in the H8/520.
2. Short format instruction.

Table A-1 (a) Machine Language Coding [General Format] (cont)

Instruction		Operation code (EA)										Operation code (OP)		
		Addressing mode		1		2		3						
		Rn	@Rn	@(d:8, Rn)	@(d:16, Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8	#xx:16	4	5	6
Arithmetic operation instruction	DADD.B Rs,Rd	3										00000000	10100rdrdrd	
	SUB.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		00110rrrr		
	SUB.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	00110rrrr		
	SUBS.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		00111rrrr		
	SUBS.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	00111rrrr		
	SUBX.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		10110rrrr		
	SUBX.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	10110rrrr		
	DSUB.B Rs,Rd	3										00000000	10110rdrdrd	
	MULXU.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		10101rrrr		
	MULXU.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	10101rrrr		
	DIVXU.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		10111rrrr		
	DIVXU.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	10111rrrr		
	CMP:G.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		01110rrrr		
	CMP:G.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	01110rrrr		
	CMP:G.B #xx,<EAd>		3	4	5	3	3	4	5			00000100	data	
	CMP:G.W #xx,<EAd>		4	5	6	4	4	5	6			00000101	data (H)	data (L)
	EXTS.B Rd	2										00010001		
	EXTU.B Rd	2										00010010		
	TST.B <EAd>	2	2	3	4	2	2	3	4			00010110		
	TST.W <EAd>	2	2	3	4	2	2	3	4			00010110		
	NEG.B <EAd>	2	2	3	4	2	2	3	4			00010100		
	NEG.W <EAd>	2	2	3	4	2	2	3	4			00010100		
	CLR.B <EAd>	2	2	3	4	2	2	3	4			00010011		
	CLR.W <EAd>	2	2	3	4	2	2	3	4			00010011		
TAS.B <EAd>	2	2	3	4	2	2	3	4			00010111			

Table A-1 (a) Machine Language Coding [General Format] (cont)

Instruction		Address- ing mode	Operation code (EA)								Operation code (OP)			
			1		2		3				4	5	6	
			Rn	@Rn	@(d:8, Rn)	@(d:16, Rn)	@-Rn	@Rn+	@aa:8	@aa:16				#xx:8
Shift instruction	SHAL.B <EAd>	2	2	3	4	2	2	3	4			00011000		
	SHAL.W <EAd>	2	2	3	4	2	2	3	4			00011000		
	SHAR.B <EAd>	2	2	3	4	2	2	3	4			00011001		
	SHAR.W <EAd>	2	2	3	4	2	2	3	4			00011001		
	SHLL.B <EAd>	2	2	3	4	2	2	3	4			00011010		
	SHLL.W <EAd>	2	2	3	4	2	2	3	4			00011010		
	SHLR.B <EAd>	2	2	3	4	2	2	3	4			00011011		
	SHLR.W <EAd>	2	2	3	4	2	2	3	4			00011011		
	ROTL.B <EAd>	2	2	3	4	2	2	3	4			00011100		
	ROTL.W <EAd>	2	2	3	4	2	2	3	4			00011100		
	ROTR.B <EAd>	2	2	3	4	2	2	3	4			00011101		
	ROTR.W <EAd>	2	2	3	4	2	2	3	4			00011101		
	ROTXL.B <EAd>	2	2	3	4	2	2	3	4			00011110		
	ROTXL.W <EAd>	2	2	3	4	2	2	3	4			00011110		
	ROTXR.B <EAd>	2	2	3	4	2	2	3	4			00011111		
	ROTXR.W <EAd>	2	2	3	4	2	2	3	4			00011111		
Logic operation instruction	AND.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		01010rrr		
	AND.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	01010rrr		
	OR.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		01000rrr		
	OR.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	01000rrr		
	XOR.B <EAs>,Rd	2	2	3	4	2	2	3	4	3		01100rrr		
	XOR.W <EAs>,Rd	2	2	3	4	2	2	3	4		4	01100rrr		
	NOT.B <EAd>	2	2	3	4	2	2	3	4			00010101		
	NOT.W <EAd>	2	2	3	4	2	2	3	4			00010101		

Table A-1 (a) Machine Language Coding [General Format] (cont)

Instruction		Address- ing mode										Operation code (EA)			Operation code (OP)								
												1						2			3		
												Rn	@Rn	@(d:8, Rn)				@(d:16, Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8
Bit manipulation instruction	BSET.B #xx,<EAd>	2	2	3	4	2	2	3	4				1 1 0 0 (data)										
	BSET.W #xx,<EAd>	2	2	3	4	2	2	3	4				1 1 0 0 (data)										
	BSET.B Rs,<EAd>	2	2	3	4	2	2	3	4				0 1 0 0 1 r r r										
	BSET.W Rs,<EAd>	2	2	3	4	2	2	3	4				0 1 0 0 1 r r r										
	BCLR.B #xx,<EAd>	2	2	3	4	2	2	3	4				1 1 0 1 (data)										
	BCLR.W #xx,<EAd>	2	2	3	4	2	2	3	4				1 1 0 1 (data)										
	BCLR.B Rs,<EAd>	2	2	3	4	2	2	3	4				0 1 0 1 1 r r r										
	BCLR.W Rs,<EAd>	2	2	3	4	2	2	3	4				0 1 0 1 1 r r r										
	BTST.B #xx,<EAd>	2	2	3	4	2	2	3	4				1 1 1 1 (data)										
	BTST.W #xx,<EAd>	2	2	3	4	2	2	3	4				1 1 1 1 (data)										
	BTST.B Rs,<EAd>	2	2	3	4	2	2	3	4				0 1 1 1 1 r r r										
	BTST.W Rs,<EAd>	2	2	3	4	2	2	3	4				0 1 1 1 1 r r r										
	BNOT.B #xx,<EAd>	2	2	3	4	2	2	3	4				1 1 1 0 (data)										
	BNOT.W #xx,<EAd>	2	2	3	4	2	2	3	4				1 1 1 0 (data)										
	BNOT.B Rs,<EAd>	2	2	3	4	2	2	3	4				0 1 1 0 1 r r r										
	BNOT.W Rs,<EAd>	2	2	3	4	2	2	3	4				0 1 1 0 1 r r r										
System control instruction	LDC.B <EAs>,CR	2	2	3	4	2	2	3	4	3			1 0 0 0 1 c c c										
	LDC.W<EAs>,CR	2	2	3	4	2	2	3	4		4		1 0 0 0 1 c c c										
	STC.B CR,<EAd>	2	2	3	4	2	2	3	4				1 0 0 1 1 c c c										
	STC.W CR,<EAd>	2	2	3	4	2	2	3	4				1 0 0 1 1 c c c										
	ANDC.B #xx:8, CR										3		0 1 0 1 1 c c c										
	ANDC.W #xx:16, CR											4	0 1 0 1 1 c c c										
	ORC.B #xx:8, CR										3		0 1 0 0 1 c c c										
	ORC.W #xx:16, CR											4	0 1 0 0 1 c c c										
	XORC.B #xx:8, CR										3		0 1 1 0 1 c c c										
	XORC.W #xx:16, CR											4	0 1 1 0 1 c c c										

Table A-1 (b) Machine Language Coding [Special Format: Short Format]

Instruction	Byte	Operation Code			
		1	2	3	4
MOV:E #xx:8,Rd	2	0 1 0 1 0 r r r	data		
MOV:I #xx:16,Rd	3	0 1 0 1 1 r r r	data (H)	data (L)	
MOV:L.B @aa:8,Rd	2	0 1 1 0 0 r r r	address (L)		
MOV:L.W @aa:8,Rd	2	0 1 1 0 1 r r r	address (L)		
MOV:S.B Rs,@aa:8	2	0 1 1 1 0 r r r	address (L)		
MOV:S.W Rs,@aa:8	2	0 1 1 1 1 r r r	address (L)		
MOV:F.B @(d:8,R6),Rd	2	1 0 0 0 0 r r r	disp		
MOV:F.W @(d:8,R6),Rd	2	1 0 0 0 1 r r r	disp		
MOV:F.B Rs,@(d:8,R6)	2	1 0 0 1 0 r r r	disp		
MOV:F.W Rs,@(d:8,R6)	2	1 0 0 1 1 r r r	disp		
CMP:E #xx:8,Rd	2	0 1 0 0 0 r r r	data		
CMP:I #xx:16,Rd	3	0 1 0 0 1 r r r	data (H)	data (L)	

Table A-1 (c) Machine Language Coding [Special Format: Branch Instructions]

Instruction		Byte	Operation Code			
			1	2	3	4
Bcc d:8	BRA (BT)	2	00100000	disp		
	BRN (BF)		00100001	disp		
	BHI		00100010	disp		
	BLS		00100011	disp		
	BCC (BHS)		00100100	disp		
	BCS (BLO)		00100101	disp		
	BNE		00100110	disp		
	BEQ		00100111	disp		
	BVC		00101000	disp		
	BVS		00101001	disp		
	BPL		00101010	disp		
	BMI		00101011	disp		
	BGE		00101100	disp		
	BLT		00101101	disp		
	BGT		00101110	disp		
	BLE		00101111	disp		
Bcc d:16	BRA (BT)	3	00110000	disp (H)	disp (L)	
	BRN (BF)		00110001	disp (H)	disp (L)	
	BHI		00110010	disp (H)	disp (L)	
	BLS		00110011	disp (H)	disp (L)	
	BCC (BHS)		00110100	disp (H)	disp (L)	
	BCS (BLO)		00110101	disp (H)	disp (L)	
	BNE		00110110	disp (H)	disp (L)	
	BEQ		00110111	disp (H)	disp (L)	
	BVC		00111000	disp (H)	disp (L)	
	BVS		00111001	disp (H)	disp (L)	
	BPL		00111010	disp (H)	disp (L)	
	BMI		00111011	disp (H)	disp (L)	
	BGE		00111100	disp (H)	disp (L)	
	BLT		00111101	disp (H)	disp (L)	
	BGT		00111110	disp (H)	disp (L)	
	BLE		00111111	disp (H)	disp (L)	
JMP @Rn		2	00010001	11010rrr		
JMP @aa:16		3	00010000	address (H)	address (L)	

Table A-1 (c) Machine Language Coding [Special Format: Branch Instructions] (cont)

Instruction	Byte	Operation Code			
		1	2	3	4
JMP @(d:8,Rn)	3	00010001	11100rrr	disp	
JMP @(d:16,Rn)	4	00010001	11110rrr	disp (H)	disp (L)
BSR d:8	2	00001110	disp		
BSR d:16	3	00011110	disp(H)	disp (L)	
JSR @Rn	2	00010001	11011rrr		
JSR @aa:16	3	00011000	address(H)	address (L)	
JSR @(d:8,Rn)	3	00010001	11101rrr	disp	
JSR @(d:16,Rn)	4	00010001	11111rrr	disp (H)	disp (L)
RTS	1	00011001			
RTD #xx:8	2	00010100	data		
RTD #xx:16	3	00011100	data (H)	data (L)	
SCB/cc Rn, disp	SCB/F	3	00000001	10111rrr	disp
	SCB/NE		00000110	10111rrr	disp
	SCB/EQ		00000111	10111rrr	disp
PJMP @aa:24	4	00010011	page	address (H)	address (L)
PJMP @Rn	2	00010001	11000rrr		
PJSR @aa:24	4	00000011	page	address (H)	address (L)
PJSR @Rn	2	00010001	11001rrr		
PRTS	2	00010001	00011001		
PRTD #xx:8	3	00010001	00010100	data	
PRTD #xx:16	4	00010001	00011100	data (H)	data (L)

Table A-1 (d) Machine Language Coding [Special Format: System Control Instructions]

Instruction	Byte	Operation Code			
		1	2	3	4
TRAPA #xx	2	00001000	0001#VEC		
TRAP/VS	1	00001001			
RTE	1	00001010			
LINK FP,#xx:8	2	00010111	data		
LINK FP,#xx:16	3	00011111	data (H)	data (L)	
UNLK FP	1	00001111			
SLEEP	1	00011010			
NOP	1	00000000			

A.4 Instruction Execution Cycles

Tables A-7 (1) through (6) list the number of cycles required by the CPU to execute each instruction in each addressing mode.

The meaning of the symbols in the tables is explained below. The values of I, J, and K are used to calculate the number of execution cycles when off-chip memory is accessed for an instruction fetch or operand read/write. The formulas for these calculations are given next.

A.4.1 Calculation of Instruction Execution States

Instruction Fetch	Operand Read/Write	Number of States
On-chip memory* ¹	On-chip memory or general register	(Value given in table A-7) + (Value in table A-8)
	On-chip supporting module	Byte (Value in table A-7) + (Value in table A-8) + I
	or off-chip memory* ²	Word (Value in table A-7) + (Value in table A-8) + 2 I
Off-chip memory* ²	On-chip memory or general register	(Value given in table A-7) + 2(J + K)
	On-chip supporting module	Byte (Value in table A-7) + I + 2(J + K)
	or off-chip memory* ²	Word (Value in table A-7) + 2(I + J + K)

- Notes: 1. When the instruction is fetched from on-chip memory (ROM or RAM), the number of execution states varies by 1 or 2 depending of whether the instruction is stored at an even or odd address. This difference must be noted when software is used for timing, and in other cases in which the exact number of states is important.
2. If wait states are inserted in access to external memory, add the necessary number of cycles.

A.4.2 Tables of Instruction Execution Cycles

Tables A-7 (1) through (6) should be read as shown below:

J + K: Number of instruction fetch cycles.

I: Total number of bytes written and read when operand is in memory.

			Addressing Mode									
Instruction	I	J \ K	Rn	@Rn	@(d:8,Rn)	@(d:16,Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8	#xx:16
			1	1	2	3	1	1	2	3	2	3
ADD.B	1	1	2	5	5	6	5	6	5	6	3	
ADD.W	2	1	2	5	5	6	5	6	5	6		4
ADD:Q.B	2	1	2	7	7	8	7	8	7	8		
ADD:Q.W	4	1	2	7	7	8	7	8	7	8		
DADD		2	4									

Shading in the I column means the operand cannot be in memory.

Shading indicates addressing modes that cannot be used with this instruction.

- **Examples of Calculation of Number of States Required for Execution**

(Example 1) Instruction fetch from on-chip memory

Operand	Start	Assembler Notation			Table A-7 + Table A-8	Number of States
Read/Write	Addr.	Address	Code	Mnemonic		
On-chip memory	Even	H'0100	H'D821	ADD:G.W @R0, R1	5 + 1	6
or general register	Odd	H'0101	H'D821	ADD:G.W @R0, R1	5 + 0	5

(Example 2) Instruction fetch from on-chip memory

Operand	Start	Assembler Notation			Table A-7 +	Number of States
Read/Write	Addr.	Address	Code	Mnemonic	Table A-8 + 2I	
On-chip supporting	Even	H'FC00	H'11D8	JSR @R0	9 + 0 + 2 × 2	13
module or external	Odd	H'FC01	H'11D8	JSR @R0	9 + 1 + 2 × 2	14
memory (word)						

(Example 3) Instruction fetch from external memory

Operand	Assembler Notation			Table A-7 + 2(J + K)	Number of States
Read/Write	Address	Code	Mnemonic		
On-chip memory or general register	H'9002	H'D821	ADD:G.W @R0, R1	5 + 2 × (1 + 1)	9

Table A-7 Instruction Execution Cycles (1)

			Addressing Mode									
			Rn	@Rn	@(d:8,Rn)	@(d:16,Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8	#xx:16
Instruction	I	J K	1	1	2	3	1	1	2	3	2	3
ADD:G.B <EAs>, Rd	1	1	2	5	5	6	5	6	5	6	3	
ADD:G.W <EAs>, Rd	2	1	2	5	5	6	5	6	5	6		4
ADD:Q.B #xx, <EAd>	2	1	2	7	7	8	7	8	7	8		
ADD:Q.W #xx, <EAd>	4	1	2	7	7	8	7	8	7	8		
ADDS.B <EAs>, Rd	1	1	3	5	5	6	5	6	5	6	3	
ADDS.W <EAs>, Rd	2	1	3	5	5	6	5	6	5	6		4
ADDX.B <EAs>, Rd	1	1	2	5	5	6	5	6	5	6	3	
ADDX.W <EAs>, Rd	2	1	2	5	5	6	5	6	5	6		4
AND.B <EAs>, Rd	1	1	2	5	5	6	5	6	5	6	3	
AND.W <EAs>, Rd	2	1	2	5	5	6	5	6	5	6		4
ANDC #xx, CR		1									5	9
BCLR.B #xx, <EAd>*	2	1	4	7	7	8	7	8	7	8		
BCLR.W #xx, <EAd>*	4	1	4	7	7	8	7	8	7	8		
BNOT.B #xx, <EAd>*	2	1	4	7	7	8	7	8	7	8		
BNOT.W #xx, <EAd>*	4	1	4	7	7	8	7	8	7	8		
BSET.B #xx, <EAd>*	2	1	4	7	7	8	7	8	7	8		
BSET.W #xx, <EAd>*	4	1	4	7	7	8	7	8	7	8		
BTST.B #xx, <EAd>*	1	1	3	5	5	6	5	6	5	6		
BTST.W #xx, <EAd>*	2	1	3	5	5	6	5	6	5	6		
CLR.B <EAd>	1	1	2	5	5	6	5	6	5	6		
CLR.W <EAd>	2	1	2	5	5	6	5	6	5	6		
CMP:G.B <EAs>, Rd	1	1	2	5	5	6	5	6	5	6	3	
CMP:G.W <EAs>, Rd	2	1	2	5	5	6	5	6	5	6		4
CMP:G.B #xx:8,<EA>	1	2		6	6	7	6	7	6	7		
CMP:G.B #xx:16,<EA>	2	3		7	7	8	7	8	7	8		

Note: * Rs can be specified in the source operand.

Table A-7 Instruction Execution Cycles (2)

				Addressing Mode									
				Rn	@Rn	@(d:8,Rn)	@(d:16,Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8	#xx:16
Instruction	I	J	K	1	1	2	3	1	1	2	3	2	3
CMP:E #xx:8,Rd		0										2	
CMP:I #xx:16,Rd		0											3
DADD Rs, Rd		2		4									
DIVXU.B <EAs>, Rd	1	1		20	23	23	24	23	24	23	24	21	
DIVXU.W <EAs>, Rd	2	1		26	29	29	30	29	30	29	30		28
DSUB Rs, Rd		2		4									
EXTS Rd		1		3									
EXTU Rd		1		3									
LDC.B <EAs>, CR	1	1		3	6	6	7	6	7	6	7	4	
LDC.W <EAs>, CR	2	1		4	7	7	8	7	8	7	8		6
MOV:G.B	1	1		2	5	5	6	5	6	5	6	3	
MOV:G.W	2	1		2	5	5	6	5	6	5	6		4
MOV:G.B #xx:8,<EAd>	1	2			7	7	8	7	8	7	8		
MOV:G.W #xx:16,<EAd>	2	3			8	8	9	8	9	8	9		
MOV:E #xx:8,Rd		0										2	
MOV:I #xx:16,Rd		0											3
MOV:L.B @aa:8,Rd	1	0								5			
MOV:L.W @aa:8,Rd	2	0								5			
MOV:S.B Rs,@aa:8	1	0								5			
MOV:S.W Rs,@aa:8	2	0								5			
MOV:F.B @(d:8, R6), Rd	1	0				5							
MOV:F.W @(d:8, R6), Rd	2	0				5							
MOV:F.B Rs, @(d:8, R6)	1	0				5							
MOV:F.W Rs, @(d:8, R6)	2	0				5							

Table A-7 Instruction Execution Cycles (3)

			Addressing Mode									
Instruction	I	J K	Rn	@Rn	@(d:8,Rn)	@(d:16,Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8	#xx:16
			1	1	2	3	1	1	2	3	2	3
(MOVFPPE <EAs>, Rd)*	0	2		13 20	13 20	14 21	13 20	14 21	13 20	14 21		
(MOVTPPE Rs, <EAd>)*	0	2		13 20	13 20	14 21	13 20	14 21	13 20	14 21		
MULXU.B <EAs>, Rd	1	1	16	19	19	20	19	20	19	20	18	
MULXU.W <EAs>, Rd	2	1	23	25	25	26	25	26	25	26		25
NEG.B <EAd>	2	1	2	7	7	8	7	8	7	8		
NEG.W <EAd>	4	1	2	7	7	8	7	8	7	8		
NOT.B <EAd>	2	1	2	7	7	8	7	8	7	8		
NOT.W <EAd>	4	1	2	7	7	8	7	8	7	8		
OR.B <EAs>, Rd	1	1	2	5	5	6	5	6	5	6	3	
OR.W <EAs>, Rd	2	1	2	5	5	6	5	6	5	6		4
ORC #xx, CR		1									5	9
ROTL.B <EAd>	2	1	2	7	7	8	7	8	7	8		
ROTL.W <EAd>	4	1	2	7	7	8	7	8	7	8		
ROTR.B <EAd>	2	1	2	7	7	8	7	8	7	8		
ROTR.W <EAd>	4	1	2	7	7	8	7	8	7	8		
ROTXL.B <EAd>	2	1	2	7	7	8	7	8	7	8		
ROTXL.W <EAd>	4	1	2	7	7	8	7	8	7	8		
ROTXR.B <EAd>	2	1	2	7	7	8	7	8	7	8		
ROTXR.W <EAd>	4	1	2	7	7	8	7	8	7	8		
SHAL.B <EAd>	2	1	2	7	7	8	7	8	7	8		
SHAL.W <EAd>	4	1	2	7	7	8	7	8	7	8		
SHAR.B <EAd>	2	1	2	7	7	8	7	8	7	8		
SHAR.W <EAd>	4	1	2	7	7	8	7	8	7	8		
SHLL.B <EAd>	2	1	2	7	7	8	7	8	7	8		
SHLL.W <EAd>	4	1	2	7	7	8	7	8	7	8		

Note: * MOVFPPE and MOVTPPE are executed synchronous with the E-clock, so the number of execution states will change depending on timing of the execution.

Table A-7 Instruction Execution Cycles (4)

Instruction	I	J	K	Addressing Mode									
				Rn	@Rn	@(d:8,Rn)	@(d:16,Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8	#xx:16
SHLR.B <EAd>	2	1		2	7	7	8	7	8	7	8		
SHLR.W <EAd>	4	1		2	7	7	8	7	8	7	8		
STC.B CR, <EAd>	1	1		4	7	7	8	7	8	7	8		
STC.W CR, <EAd>	2	1		4	7	7	8	7	8	7	8		
SUB.B <EAs>, Rd	1	1		2	5	5	6	5	6	5	6	3	
SUB.W <EAs>, Rd	2	1		2	5	5	6	5	6	5	6		4
SUBS.B <EAs>, Rd	1	1		3	5	5	6	5	6	5	6	3	
SUBS.W <EAs>, Rd	2	1		3	5	5	6	5	6	5	6		4
SUBX.B <EAs>, Rd	1	1		2	5	5	6	5	6	5	6	3	
SUBX.W <EAs>, Rd	2	1		2	5	5	6	5	6	5	6		4
SWAP Rd		1		3									
TAS <EAd>	2	1		4	7	7	8	7	8	7	8		
TST.B <EAd>	1	1		2	5	5	6	5	6	5	6		
TST.W <EAd>	2	1		2	5	5	6	5	6	5	6		
XCH Rs, Rd		1		4									
XOR.B <EAs>, Rd	1	1		2	5	6	5	5	6	5	6	3	
XOR.W <EAs>, Rd	2	1		2	5	6	5	5	6	5	6		4
XORC #xx, CR		1										5	9

* ↙

DIVXU.B Zero divide, minimum mode	6/7	1	20	23	23	24	23	24	23	24	21	
DIVXU.B Zero divide, maximum mode	10/11	1	25	28	28	29	28	29	28	29	21	
DIVXU.W Zero divide, minimum mode	6/8	1	20	23	23	24	23	24	23	24		27
DIVXU.W Zero divide, maximum mode	10/12	1	25	28	28	29	28	29	28	29		27
DIVXU.B Overflow	1	1	8	11	11	12	11	12	11	12	9	
DIVXU.W Overflow	2	1	8	11	11	12	11	12	11	12		10

* ↗ For register and immediate operands



For memory operand

Table A-7 Instruction Execution Cycles (5)

Instruction	Condition	Execution Cycle	I	J + K
Bcc d:ε	Condition false, branch not taken	3		2
	Condition true, branch taken	7		5
Bcc d:1ε	Condition false, branch not taken	3		3
	Condition true, branch taken	7		6
BSR	d:8	9	2	4
	d:16	9	2	5
JMP	@aa:1ε	7		5
	@Rn	6		5
	@(d:8, Rn	7		5
	@(d:16, Rn	8		6
JSR	@aa:1ε	9	2	5
	@Rn	9	2	5
	@(d:8, Rn	9	2	5
	@(d:16, Rn	10	2	6
LDM		6 + 4n*	2n	2
LINK	#xx:ε	6	2	2
	#xx:1ε	7	2	3
NOP		2		1
RTD	#xx:ε	9	2	4
	#xx:1ε	9	2	5
RTE	Minimum mode	13	4	4
	Maximum mode	15	6	4
RTS		8	2	4
SCB	Condition false, branch not taken	3		3
	Count = -1, branch not taken	4		3
	Other than the above, branch taken	8		6
SLEEP	Cycles preceding transition to power-down mode	2		0
STM		6 + 3n*	2n	2
TRAPA	Minimum mode	17	6	4
	Maximum mode	22	10	4
TRAP/Vε	V = 0, branch not taken	3		1
	V = 1, branch taken, minimum mode	18	6	4
	V = 1, branch taken, maximum mode	23	10	4

Note: * n is the number of registers specified in the register list.

Table A-7 Instruction Execution Cycles (6)

Instruction	Condition	Execution Cycle	I	J + K
UNLK		5	2	1
PJMP	@aa:24	9		6
	@Rn	8		5
PJSR	@aa:24	15	4	6
	@Rn	13	4	5
PRTS		12	4	5
PRTD	#xx:8	13	4	5
	#xx:16	13	4	6

Table A-8 (a) Adjusted Value (Branch instructions)

Instruction	Address	Adjusted Value
BSR, JMP, JSR, RTS, RTD, RTE	even	0
TRAPA, PJMP, PJSR, PRTS, PRTD	odd	1
Bcc, SCB, TRAP/VS (When branches)	even	0
	odd	1

Table A-8 (b) Adjusted Value (Other instructions by addressing modes)

Instruction	Start address	Rn	@Rn	@(d:8,Rn)	@(d:16,Rn)	@-Rn	@Rn+	@aa:8	@aa:16	#xx:8	#xx:16
MOV.B #xx:8, <EA>	even		1	1	1	1	1	1	1		
	odd		1	1	1	1	1	1	1		
MOV.W #xx:16, <EA>	even		2	0	2	2	2	0	2		
	odd		0	2	0	0	0	2	0		
Instructions other than above	even	0	1	0	1	1	1	0	1	0	0
	odd	0	0	1	0	0	0	1	0	0	0

Appendix B Register Field

B.1 Register Addresses and Bit Names

Addr. (last byte)	Register Name	Bit Names								Module
		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
H'80	P1DDR	P17DDR	P16DDR	P15DDR	P14DDR	P13DDR	P12DDR	P11DDR	P10DDR	Port 1
H'81	P2DDR	P27DDR	P26DDR	P25DDR	P24DDR	P23DDR	P22DDR	P21DDR	P20DDR	Port 2
H'82	P1DR	P17	P16	P15	P14	P13	P12	P11	P10	Port 1
H'83	P2DR	P27	P26	P25	P24	P23	P22	P21	P20	Port 2
H'84	P3DDR	P37DDR	P36DDR	P35DDR	P34DDR	P33DDR	P32DDR	P31DDR	P30DDR	Port 3
H'85	P4DDR	P47DDR	P46DDR	P45DDR	P44DDR	P43DDR	P42DDR	P41DDR	P40DDR	Port 4
H'86	P3DR	P37	P36	P35	P34	P33	P32	P31	P30	Port 3
H'87	P4DR	P47	P46	P45	P44	P43	P42	P41	P40	Port 4
H'88	P5DDR	P57DDR	P56DDR	P55DDR	P54DDR	P53DDR	P52DDR	P51DDR	P50DDR	Port 5
H'89	—	—	—	—	—	—	—	—	—	—
H'8A	P5DR	P57	P56	P55	P54	P53	P52	P51	P50	Port 5
H'8B	P6DR	P67*	P66*	P65*	P64*	P63	P62	P61	P60	Port 6
H'8C	P7DDR	—	ØOE	P75DDR	P74DDR	P73DDR	P72DDR	P71DDR	P70DDR	Port 7
H'8D	—	—	—	—	—	—	—	—	—	—
H'8E	P7DR	—	—	P75	P74	P73	P72	P71	P70	Port 7
H'8F	—	—	—	—	—	—	—	—	—	—
H'90	TCR	ICIE	OCIEB	OCIEA	OVIE	OEB	OEA	CKS1	CKS0	FRT1
H'91	TCSR	ICF	OCFB	OCFA	OVF	OLVLB	OLVLA	IEDG	CCLRA	
H'92	FRC (H)									
H'93	FRC (L)									
H'94	OCRA (H)									
H'95	OCRA (L)									
H'96	OCRB (H)									
H'97	OCRB (L)									
H'98	ICR (H)									
H'99	ICR (L)									
H'9A	—	—	—	—	—	—	—	—	—	
H'9B	—	—	—	—	—	—	—	—	—	
H'9C	—	—	—	—	—	—	—	—	—	
H'9D	—	—	—	—	—	—	—	—	—	
H'9E	—	—	—	—	—	—	—	—	—	
H'9F	—	—	—	—	—	—	—	—	—	

(Continued on next page)

Notes: FRT1: 16-bit Free-Running Timer channel 1

* CP-68 package only.

(Continued from preceding page)

Addr. (last byte)	Register Name	Bit Names								Module
		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
H'A0	TCR	ICIE	OCIEB	OCIEA	OVIE	OEB	OEA	CKS1	CKS0	FRT2
H'A1	TCSR	ICF	OCFB	OCFA	OVF	OLVLB	OLVLA	IEDG	CCLRA	
H'A2	FRC (H)									
H'A3	FRC (L)									
H'A4	OCRA (H)									
H'A5	OCRA (L)									
H'A6	OCRB (H)									
H'A7	OCRB (L)									
H'A8	ICR (H)									
H'A9	ICR (L)									
H'AA	—	—	—	—	—	—	—	—	—	
H'AB	—	—	—	—	—	—	—	—	—	
H'AC	—	—	—	—	—	—	—	—	—	
H'AD	—	—	—	—	—	—	—	—	—	
H'AE	—	—	—	—	—	—	—	—	—	
H'AF	—	—	—	—	—	—	—	—	—	
H'B0	—	—	—	—	—	—	—	—	—	—
H'B1	—	—	—	—	—	—	—	—	—	
H'B2	—	—	—	—	—	—	—	—	—	
H'B3	—	—	—	—	—	—	—	—	—	
H'B4	—	—	—	—	—	—	—	—	—	
H'B5	—	—	—	—	—	—	—	—	—	
H'B6	—	—	—	—	—	—	—	—	—	
H'B7	—	—	—	—	—	—	—	—	—	
H'B8	—	—	—	—	—	—	—	—	—	
H'B9	—	—	—	—	—	—	—	—	—	
H'BA	—	—	—	—	—	—	—	—	—	
H'BB	—	—	—	—	—	—	—	—	—	
H'BC	—	—	—	—	—	—	—	—	—	
H'BD	—	—	—	—	—	—	—	—	—	
H'BE	—	—	—	—	—	—	—	—	—	
H'BF	—	—	—	—	—	—	—	—	—	

(Continued on next page)

Note: FRT2: 16-bit Free-Running Timer channel 2

Addr. (last byte)	Register Name	Bit Names								Module
		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
H'C0	SMR	C/ $\bar{A}$	CHR	PE	O/ $\bar{E}$	STOP	—	CKS1	CKS0	SCI2
H'C1	BRR									
H'C2	SCR	TIE	RIE	TE	RE	—	—	CKE1	CKE0	
H'C3	TDR	—	—	—	—	—	—	—	—	
H'C4	SSR	TDRE	RDRF	ORER	FER	PER	—	—	—	
H'C5	RDR									
H'C6	—	—	—	—	—	—	—	—	—	
H'C7	—	—	—	—	—	—	—	—	—	
H'C8	—	—	—	—	—	—	—	—	—	—
H'C9	—	—	—	—	—	—	—	—	—	
H'CA	—	—	—	—	—	—	—	—	—	
H'CB	—	—	—	—	—	—	—	—	—	
H'CC	—	—	—	—	—	—	—	—	—	
H'CD	—	—	—	—	—	—	—	—	—	
H'CE	—	—	—	—	—	—	—	—	—	
H'CF	—	—	—	—	—	—	—	—	—	
H'D0	TCR	CMIEB	CMIEA	OVIE	CCLR1	CCLR0	CKS2	CKS1	CKS0	TMR
H'D1	TCSR	CMFB	CMFA	OVF	—	OS3	OS2	OS1	OS0	
H'D2	TCORA									
H'D3	TCORB									
H'D4	TCNT									
H'D5	—	—	—	—	—	—	—	—	—	
H'D6	—	—	—	—	—	—	—	—	—	
H'D7	—	—	—	—	—	—	—	—	—	
H'D8	SMR	C/ $\bar{A}$	CHR	PE	O/ $\bar{E}$	STOP	—	CKS1	CKS0	SCI1
H'D9	BRR									
H'DA	SCR	TIE	RIE	TE	RE	—	—	CKE1	CKE0	
H'DB	TDR									
H'DC	SSR	TDRE	RDRF	ORER	FER	PER	—	—	—	
H'DD	RDR									
H'DE	—	—	—	—	—	—	—	—	—	
H'DF	—	—	—	—	—	—	—	—	—	

(Continued on next page)

Notes: TMR: 8-Bit Timer

SCI1: Serial Communication Interface channel 1

SCI2: Serial Communication Interface channel 2

(Continued from preceding page)

Addr. (last byte)	Register Name	Bit Names								Module
		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
H'E0	ADDRA (H)	AD ₉	AD ₈	AD ₇	AD ₆	AD ₅	AD ₄	AD ₃	AD ₂	A/D
H'E1	ADDRA (L)	AD ₁	AD ₀	—	—	—	—	—	—	
H'E2	ADDRB (H)	AD ₉	AD ₈	AD ₇	AD ₆	AD ₅	AD ₄	AD ₃	AD ₂	
H'E3	ADDRB (L)	AD ₁	AD ₀	—	—	—	—	—	—	
H'E4	ADDRC (H)	AD ₉	AD ₈	AD ₇	AD ₆	AD ₅	AD ₄	AD ₃	AD ₂	
H'E5	ADDRC (L)	AD ₁	AD ₀	—	—	—	—	—	—	
H'E6	ADDRD (H)	AD ₉	AD ₈	AD ₇	AD ₆	AD ₅	AD ₄	AD ₃	AD ₂	
H'E7	ADDRD (L)	AD ₁	AD ₀	—	—	—	—	—	—	
H'E8	ADCSR	ADF	ADIE	ADST	SCAN	CKS	CH2	CH1	CH0	
H'E9	ADCR	TRGE	—	—	—	—	—	—	—	
H'EA	—	—	—	—	—	—	—	—	—	
H'EB	—	—	—	—	—	—	—	—	—	
H'EC	Password ----- TCSR*	OVF	WT/IT	TME	—	—	CKS2	CKS1	CKS0	WDT
H'ED	TCSR/TCNT* ----- TCNT*									
H'EE	—	—	—	—	—	—	—	—	—	—
H'EF	—	—	—	—	—	—	—	—	—	

(Continued on next page)

Notes: A/D: Analog-to-Digital converter

WDT: Watchdog Timer

* Read addresses are shown. Write addresses of both TCSR and TCNT are H'FFED, preceded by a password at H'FFEC. See section 13.2.3, "Notes on Register Access", for details.

Addr. (last byte)	Register Name	Bit Names								
		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Module
H'F0	IPRA	—	IRQ ₀ priority			—	IRQ ₁ - IRQ ₇ priority			INTC
H'F1	IPRB	—	FRT1 interrupt priority			—	FRT2 interrupt priority			
H'F2	IPRC	—	TMR interrupt priority			—	SCI1 interrupt priority			
H'F3	IPRD	—	SCI2 interrupt priority			—	A/D interrupt priority			
H'F4	DTEA	—	—	—	IRQ ₀	—	IRQ ₃	IRQ ₂	IRQ ₁	
H'F5	DTEB	—	OCIB ₁	OCIA ₁	ICI	—	OCIB	OCIA	ICI	
H'F6	DTEC	—	—	CMIB	CMIA	—	TXI	RXI	—	
H'F7	DTED	—	TXI	RXI	—	—	—	—	ADI	
H'F8	WCR	—	—	—	—	WMS1	WMS0	WC1	WC0	WSC
H'F9	RAMCR	RAME	—	—	—	—	—	—	—	RAM
H'FA	MDCR	—	—	—	—	—	MDS2	MDS1	MDS0	
H'FB	SBYCR	SSBY	—	—	—	—	—	—	—	
H'FC	NMICR	—	—	—	—	—	—	—	NMIEG	INTC
H'FD	IRQCR	IRQ7E	IRQ6E	IRQ5E	IRQ4E	IRQ3E	IRQ2E	IRQ1E	IRQ0E	WDT
H'FE	Password*									
H'FF	RSTCSR	WRST	RSTOE	—	—	—	—	—	—	

Notes: INTC: Interrupt Controller

WSC: Wait State Controller

WDT: Watchdog Timer

* The password is required for write access to RSTCSR. See section 12.2.3 for details.

B.2 Register Descriptions

- How to Read the Register Descriptions

WCR—Wait-State Control Register

Acronym of the register

Register name

Bit numbers

Initial bit values

Initial value

Read/Write

Type of access permitted

RRead only

WWrite only

R/WBoth read and write

H'FFF8

Address to which the register is mapped

WSC

Name of the on-chip supporting module

7

6

5

4

3

2

1

0

—

—

—

—

WMS1

WMS0

WC1

WC0

1

1

1

1

0

0

1

1

—

—

—

—

R/W

R/W

R/W

R/W

Names of the bits. Dashes (—) indicate reserved bits.

Wait Count 1 and 0

00

No wait states (Tw) are inserted.

01

1 wait state is inserted.

10

2 wait states are inserted.

11

3 wait states are inserted.

Full name of the bit

Wait Mode Select 1 and 0

00

Programmable wait mode

01

No wait states are inserted, regardless of the wait count.

10

Pin wait mode

11

Pin auto-wait mode

Functions of the bit settings

P1DDR—Port 1 Data Direction Register

H'FF80

Port 1

Bit	7	6	5	4	3	2	1	0
	P17DDR	P16DDR	P15DDR	P14DDR	P13DDR	P12DDR	P11DDR	P10DDR
Initial value	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

Port 1 Input/Output Selection

0	Input port
1	Output port

P1DR—Port 1 Data Register

H'FF82

Port 1

Bit	7	6	5	4	3	2	1	0
	P17	P16	P15	P14	P13	P12	P11	P10
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P2DDR—Port 2 Data Direction Register

H'FF81

Port 2

Bit	7	6	5	4	3	2	1	0
	P27DDR	P26DDR	P25DDR	P24DDR	P23DDR	P22DDR	P21DDR	P20DDR
Initial value	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

Port 2 Input/Output Selection

0	Input port
1	Output port

P2DR—Port 2 Data Register				H'FF83		Port 2		
Bit	7	6	5	4	3	2	1	0
	P2 ₇	P2 ₆	P2 ₅	P2 ₄	P2 ₃	P2 ₂	P2 ₁	P2 ₀
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P3DDR—Port 3 Data Direction Register					H'FF84		Port 3	
Bit	7	6	5	4	3	2	1	0
	P3 ₇ DDR	P3 ₆ DDR	P3 ₅ DDR	P3 ₄ DDR	P3 ₃ DDR	P3 ₂ DDR	P3 ₁ DDR	P3 ₀ DDR
Initial value	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

Port 3 Input/Output Selection

0	Input port
1	Output port

P3DR—Port 3 Data Register				H'FF86		Port 3		
Bit	7	6	5	4	3	2	1	0
	P3 ₇	P3 ₆	P3 ₅	P3 ₄	P3 ₃	P3 ₂	P3 ₁	P3 ₀
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P4DDR—Port 4 Data Direction Register				H'FF85		Port 4		
Bit	7	6	5	4	3	2	1	0
	P4 ₇ DDR	P4 ₆ DDR	P4 ₅ DDR	P4 ₄ DDR	P4 ₃ DDR	P4 ₂ DDR	P4 ₁ DDR	P4 ₀ DDR
Initial value	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

Port 4 Input/Output Selection

0	Input port
1	Output port

P4DR—Port 4 Data Register

H'FF87

Port 4

Bit	7	6	5	4	3	2	1	0
	P4 ₇	P4 ₆	P4 ₅	P4 ₄	P4 ₃	P4 ₂	P4 ₁	P4 ₀
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

P5DDR—Port 5 Data Direction Register

H'FF88

Port 5

Bit	7	6	5	4	3	2	1	0
	P5 ₇ DDR	P5 ₆ DDR	P5 ₅ DDR	P5 ₄ DDR	P5 ₃ DDR	P5 ₂ DDR	P5 ₁ DDR	P5 ₀ DDR
Initial value	0	0	0	0	0	0	0	0
Read/Write	W	W	W	W	W	W	W	W

Port 5 Input/Output Selection

0	Input port
1	Output port

P5DR—Port 5 Data Register

H'FF8A

Port 5

Bit	7	6	5	4	3	2	1	0
	P5 ₇	P5 ₆	P5 ₅	P5 ₄	P5 ₃	P5 ₂	P5 ₁	P5 ₀
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit	7	6	5	4	3	2	1	0
	ICIE	OCIEB	OCIEA	OVIE	OEB	OEA	CKS1	CKS0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Clock Select

0	0	Internal clock source: $\varnothing/4$
0	1	Internal clock source: $\varnothing/8$
1	0	Internal clock source: $\varnothing/32$
1	1	External clock source: counted on rising edge

Output Enable A

0	Compare-A output is disabled.
1	Compare-A output is enabled.

Output Enable B

0	Compare-B output is disabled.
1	Compare-B output is enabled.

Timer Overflow Interrupt Enable

0	Overflow interrupt request is disabled.
1	Overflow interrupt request is enabled.

Output Compare Interrupt Enable A

0	Compare-match A interrupt request is disabled.
1	Compare-match A interrupt request is enabled.

Output Compare Interrupt Enable B

0	Compare-match B interrupt request is disabled.
1	Compare-match B interrupt request is enabled.

Input Capture Interrupt Enable

0	Input capture interrupt request is disabled.
1	Input capture interrupt request is enabled.

Bit	7	6	5	4	3	2	1	0
	ICF	OCFB	OCFA	OVF	OLVLB	OLVLA	IEDG	CCLRA
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/(W)*	R/(W)*	R/(W)*	R/(W)*	R/W	R/W	R/W	R/W

Counter Clear A	
0	FRC count is not cleared.
1	FRC count is cleared by compare-match A.

Input Edge Select	
0	Count is captured on falling edge of input capture signal (FTI).
1	Count is captured on rising edge of input capture signal.

Output Level A	
0	Compare-match A causes 0 output.
1	Compare-match A causes 1 output.

Output Level B	
0	Compare-match B causes 0 output.
1	Compare-match B causes 1 output.

Timer Overflow	
0	Cleared from 1 to 0 when CPU reads OVF = 1, then writes 0 in OVF.
1	Set to 1 when FRC changes from H'FFFF to H'0000.

Output Compare Flag A	
0	Cleared from 1 to 0 when: 1. CPU reads OCFA = 1, then writes 0 in OCFA. 2. OCIA interrupt is served by DTC.
1	Set to 1 when FRC = OCRA.

Output Compare Flag B	
0	Cleared from 1 to 0 when: 1. CPU reads OCFB = 1, then writes 0 in OCFB. 2. OCIB interrupt is served by DTC.
1	Set to 1 when FRC = OCRB.

Input Capture Flag	
0	Cleared from 1 to 0 when: 1. CPU reads ICF = 1, then writes 0 in ICF. 2. ICI interrupt is served by DTC.
1	Set to 1 when input capture signal is received and FRC count is copied to ICR.

Note: * Only writing of a 0 to clear the flag is enabled.

FRC (H and L)—Free-Running Counter

H'FF92, H'FF93

FRT1

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Count value

OCRA (H and L)—Output Compare Register A

H'FF94, H'FF95

FRT1

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Continually compared with FRC. OCFA is set to 1 when OCRA = FRC.

OCRB (H and L)—Output Compare Register B

H'FF96, H'FF97

FRT1

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Continually compared with FRC. OCFB is set to 1 when OCRB = FRC.

ICR (H and L)—Input Capture Register

H'FF98, H'FF99

FRT1

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

Contains FRC count captured when external input capture signal changes.

TCR—Timer Control Register**H'FFA0****FRT2**

Bit	7	6	5	4	3	2	1	0
	ICIE	OCIEB	OCIEA	OVIE	OEB	OEA	CKS1	CKS0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Note: Bit functions are the same as for FRT1.

TCSR—Timer Control/Status Register**H'FFA1****FRT2**

Bit	7	6	5	4	3	2	1	0
	ICF	OCFB	OCFA	OVF	OLVLB	OLVLA	IEDG	CCLRA
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/(W)*	R/(W)*	R/(W)*	R/(W)*	R/W	R/W	R/W	R/W

Note: Bit functions are the same as for FRT1.

* Only writing of a 0 to clear the flag is enabled.

FRC (H and L)—Free-Running Counter**H'FFA2, H'FFA3****FRT2**

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Note: Bit functions are the same as for FRT1.

OCRA (H and L)—Output Compare Register A

H'FFA4, H'FFA5

FRT2

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Note: Bit functions are the same as for FRT1.

OCRB (H and L)—Output Compare Register B

H'FFA6, H'FFA7

FRT2

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Note: Bit functions are the same as for FRT1.

ICR (H and L)—Input Capture Register

H'FFA8, H'FFA9

FRT2

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

Note: Bit functions are the same as for FRT1.

Bit	7	6	5	4	3	2	1	0
	CMIEB	CMIEA	OVIE	CCLR1	CCLR0	CKS2	CKS1	CKS0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Clock Select

0	0	0	No clock source; timer stops.
0	0	1	Internal clock source: $\varnothing/8$.
0	1	0	Internal clock source: $\varnothing/64$.
0	1	1	Internal clock source: $\varnothing/1024$.
1	0	0	No clock source; timer stops.
1	0	1	External clock source, counted on rising edge.
1	1	0	External clock source, counted on falling edge.
1	1	1	External clock source, counted on both rising and falling edges.

Counter Clear

0	0	Counter is not cleared.
0	1	Cleared by compare-match A.
1	0	Cleared by compare-match B.
1	1	Cleared on rising edge of external reset input.

Timer Overflow Interrupt Enable

0	Overflow interrupt request is disabled.
1	Overflow interrupt request is enabled.

Compare-Match Interrupt Enable A

0	Compare-match A interrupt request is disabled.
1	Compare-match A interrupt request is enabled.

Compare-Match Interrupt Enable B

0	Compare-match B interrupt request is disabled.
1	Compare-match B interrupt request is enabled.

Bit	7	6	5	4	3	2	1	0
	CMFB	CMFA	OVF	—	OS3*2	OS2*2	OS1*2	OS0*2
Initial value	0	0	0	1	0	0	0	0
Read/Write	R/(W)*1	R/(W)*1	R/(W)*1	—	R/W	R/W	R/W	R/W

Output Select

0	0	No change on compare-match A.
0	1	Output 0 on compare-match A.
1	0	Output 1 on compare-match A.
1	1	Invert (toggle) output on compare-match A.

Output Select

0	0	No change on compare-match B.
0	1	Output 0 on compare-match B.
1	0	Output 1 on compare-match B.
1	1	Invert (toggle) output on compare-match B.

Timer Overflow Flag

0	Cleared from 1 to 0 when CPU reads OVF = 1, then writes 0 in OVF.
1	Set to 1 when TCNT changes from H'FF to H'00.

Compare-Match Flag A

0	Cleared from 1 to 0 when: 1. CPU reads CMFA = 1, then writes 0 in CMFA. 2. CMIA interrupt is served by the DTC.
1	Set to 1 when TCNT = TCORA.

Compare-Match Flag B

0	Cleared from 1 to 0 when: 1. CPU reads CMFB = 1, then writes 0 in CMFB. 2. CMIB interrupt is served by the DTC.
1	Set to 1 when TCNT = CMFB.

Notes: 1. Only writing of 0 to clear the flag is enabled.
2. When all four bits (OS3 to OS0) are cleared to 0, output is disabled.

TCORA—Time Constant Register A				H'FFD2		TMR		
Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The CMFA bit is set to 1 when TCORA = TCNT.

TCORB—Time Constant Register B				H'FFD3		TMR		
Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

The CMFB bit is set to 1 when TCORB = TCNT.

TCNT—Timer Counter				H'FFD4		TMR		
Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Count value

Bit	7	6	5	4	3	2	1	0
	C/ $\overline{A}$	CHR	PE	O/ $\overline{E}$	STOP	—	CKS1	CKS0
Initial value	0	0	0	0	0	1	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	—	R/W	R/W

Clock Select

0	0	ø clock
0	1	ø/4 clock
1	0	ø/16 clock
1	1	ø/64 clock

Stop Bit Length

0	One stop bit
1	Two stop bits

Parity Mode

0	Even parity
1	Odd parity

Parity Enable

0	Transmit: No parity bit added. Receive: Parity bit not checked.
1	Transmit: Parity bit added. Receive: Parity bit checked.

Character Length

0	8-Bit data length
1	7-Bit data length

Communication Mode

0	Asynchronous
1	Synchronous

BRR—Bit Rate Register

H'FFD9

SCI1

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Constant that determines the baud rate

SCR—Serial Control Register

H'FFDA

SCI1

Bit	7	6	5	4	3	2	1	0
	TIE	RIE	TE	RE	—	—	CKE1	CKE0
Initial value	0	0	0	0	1	1	0	0
Read/Write	R/W	R/W	R/W	R/W	—	—	R/W	R/W

Receive Enable

Transmit Enable

Receive Interrupt Enable

Transmit Interrupt Enable

Clock Enable 0

Clock Enable 1

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Transmit data

Bit	7	6	5	4	3	2	1	0
	TDRE	RDRF	ORER	FER	PER	—	—	—
Initial value	1	0	0	0	0	1	1	1
Read/Write	R/(W)*	R/(W)*	R/(W)*	R/(W)*	R/(W)*	—	—	—

Parity Error

0	Cleared from 1 to 0 when: 1. CPU reads PER = 1, then writes 0 in PER. 2. The chip is reset or enters a standby mode.
1	Set to 1 when a parity error occurs (parity of receive data does not match parity selected by O/E bit in SMR).

Framing Error

0	Cleared from 1 to 0 when: 1. CPU reads FER = 1, then writes 0 in FER. 2. The chip is reset or enters a standby mode.
1	Set to 1 when a framing error occurs (stop bit is 0).

Overrun Error

0	Cleared from 1 to 0 when: 1. CPU reads ORER = 1, then writes 0 in ORER. 2. The chip is reset or enters a standby mode.
1	Set to 1 when an overrun error occurs (next data is completely received while RDRF bit is set to 1).

Receive Data Register Full

0	Cleared from 1 to 0 when: 1. CPU reads RDRF = 1, then writes 0 in RDRF. 2. RDR is read by the DTC. 3. The chip is reset or enters a standby mode.
1	Set to 1 when one character is received normally and transferred from RSR to RDR.

Transmit Data Register Empty

0	Cleared from 1 to 0 when: 1. CPU reads TDRE = 1, then writes 0 in TDRE. 2. The DTC writes data in TDR.
1	Set to 1 when: 1. The chip is reset or enters a standby mode. 2. Data is transferred from TDR to TSR. 3. TE is cleared to 0 when TDRE = 0.

Note: * Only writing of 0 to clear the flag is enabled.

RDR—Receive Data Register

H'FFDD

SCI1

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

Receive data

SMR—Serial Mode Register

H'FFC0

SCI2

Bit	7	6	5	4	3	2	1	0
	C/ $\overline{A}$	CHR	PE	O/E	STOP	—	CKS1	CKS0
Initial value	0	0	0	0	0	1	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	—	R/W	R/W

Note: Bit functions are the same as for SCI1.

BRR—Bit Rate Register

H'FFC1

SCI2

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Note: Bit functions are the same as for SCI1.

SCR—Serial Control Register**H'FFC2****SCI2**

Bit	7	6	5	4	3	2	1	0
	TIE	RIE	TE	RE	—	—	CKE1	CKE0
Initial value	0	0	0	0	1	1	0	0
Read/Write	R/W	R/W	R/W	R/W	—	—	R/W	R/W

Note: Bit functions are the same as for SCI1.

TDR—Transmit Data Register**H'FFC3****SCI2**

Bit	7	6	5	4	3	2	1	0
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Note: Bit functions are the same as for SCI1.

SSR—Serial Status Register**H'FFC4****SCI2**

Bit	7	6	5	4	3	2	1	0
TDRE	RDRF	ORER	FER	PER	—	—	—	
Initial value	0	0	0	0	0	1	1	1
Read/Write	R/(W)*	R/(W)*	R/(W)*	R/(W)*	R/(W)*	—	—	—

Note: Bit functions are the same as for SCI1.

* Only writing of 0 to clear the flag is enabled.

RDR—Receive Data Register**H'FFC5****SCI2**

Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

Note: Bit functions are the same as for SCI1.

ADDRn (H)—A/D Data Register n (High**H'FFE0, H'FFE2, H'FFE4, H'FFE6****(n = A, B, C, D)****A/D**

Bit	7	6	5	4	3	2	1	0
	AD ₉	AD ₈	AD ₇	AD ₆	AD ₅	AD ₄	AD ₃	AD ₂
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

Upper 8 bits of 10-bit A/D conversion result

ADDRn (L)—A/D Data Register n (Low)**H'FFE1, H'FFE3, H'FFE5, H'FFE7****(n = A, B, C, D)****A/D**

Bit	7	6	5	4	3	2	1	0
	AD ₁	AD ₀	—	—	—	—	—	—
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R	R	R	R	R	R	R

Lower 2 bits of 10-bit A/D conversion result

Bit	7	6	5	4	3	2	1	0
	ADF	ADIE	ADST	SCAN	CKS	CH2	CH1	CH0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/(W)*1	R/W	R/W	R/W	R/W	R/W	R/W	R/W
						Channel Select		
						CH2	CH1	CH0
						0	0	0
								Single mode
								Scan mode
						0	0	AN0
						0	1	AN1
						1	0	AN2
						1	1	AN3
						1	0	AN4*2
						0	1	AN5*2
						1	0	AN6*2
						1	1	AN7*2
						Clock Select		
						0	Conversion time = 274 states (max)	
						1	Conversion time = 138 states (max)	
						Scan Mode		
						0	Single mode	
						1	Scan mode	
						A/D Start		
						0	A/D conversion is halted.	
						1	1. Single mode: One A/D conversion is performed, then this bit is automatically cleared to 0. 2. Scan mode: A/D conversion starts and continues cyclically on all selected channels until 0 is written in this bit.	
						A/D Interrupt Enable		
						0	The A/D interrupt request (ADI) is disabled.	
						1	The A/D interrupt request (ADI) is enabled.	
						A/D End Flag		
						0	Cleared from 1 to 0 when: 1. The chip is reset or enters a standby mode. 2. CPU reads ADF = 1, then writes 0 in ADF. 3. An A/D interrupt is served by the DTC.	
						1	Set to 1 at the following times: 1. Single mode: at the completion of A/D conversion 2. Scan mode: when all selected channels have been converted.	

Notes: 1. Only writing of 0 to clear the flag is enabled.
2. CP-68 package only.

Bit	7	6	5	4	3	2	1	0
	TRGE	—	—	—	—	—	—	—
Initial value	0	1	1	1	1	1	1	1
Read/Write	R/W	—	—	—	—	—	—	—
	Trigger Enable							
	0	The A/D external trigger is disabled.						
	1	The A/D external trigger is enabled. A/D conversion starts on the falling edge of ADTRG.						

TCSR—Timer Status/Control Register

H'FFEC*1, H'FFED*2

WDT

Bit	7	6	5	4	3	2	1	0
	OVF	WT/IT	TME	—	—	CKS2	CKS1	CKS0
Initial value	0	0	0	1	1	0	0	0
Read/Write	R/(W)*3	R/W	R/W	—	—	R/W	R/W	R/W

Clock Select

0	0	0	ø/2 (51.2 µs)*4
0	0	1	ø/32 (819.2 µs)
0	1	0	ø/64 (1.6 ms)
0	1	1	ø/128 (3.3 ms)
1	0	0	ø/256 (6.6 ms)
1	0	1	ø/512 (13.1 ms)
1	1	0	ø/2048 (52.4 ms)
1	1	1	ø/4096 (104.9 ms)

Timer Enable

0	Timer is disabled. • TCNT is initialized to H'00 and stopped.
1	Timer is enabled. • TCNT starts incrementing. • Reset or interrupt request is enabled.

Timer Mode Select

0	Interval timer mode (IRQø interrupt request)
1	Watchdog timer mode (reset output)

Overflow Flag

0	Cleared from 1 to 0 when CPU reads OVF = 1, then writes 0 in OVF.
1	Set to 1 when TCNT changes from H'FF to H'00.

Notes: 1. Write address
2. Read address
3. Only writing of 0 to clear the flag is enabled.
4. Times in parentheses are the times for TCNT to increment from H'00 to H'FF and change to H'00 again when ø = 10 MHz.

TCNT—Timer Counter				H'FFED		WDT		
Bit	7	6	5	4	3	2	1	0
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Count value

RSTCSR—Reset Status/Control Register

H'FFFF*2

WDT

Bit	7	6	5	4	3	2	1	0
	WRST	RSTOE	—	—	—	—	—	—
Initial value	0	0	1	1	1	1	1	1
Read/Write	R/(W)*1	R/W	—	—	—	—	—	—

Reset Output Enable

0	The reset signal is not output externally.
1	The reset signal is output externally.

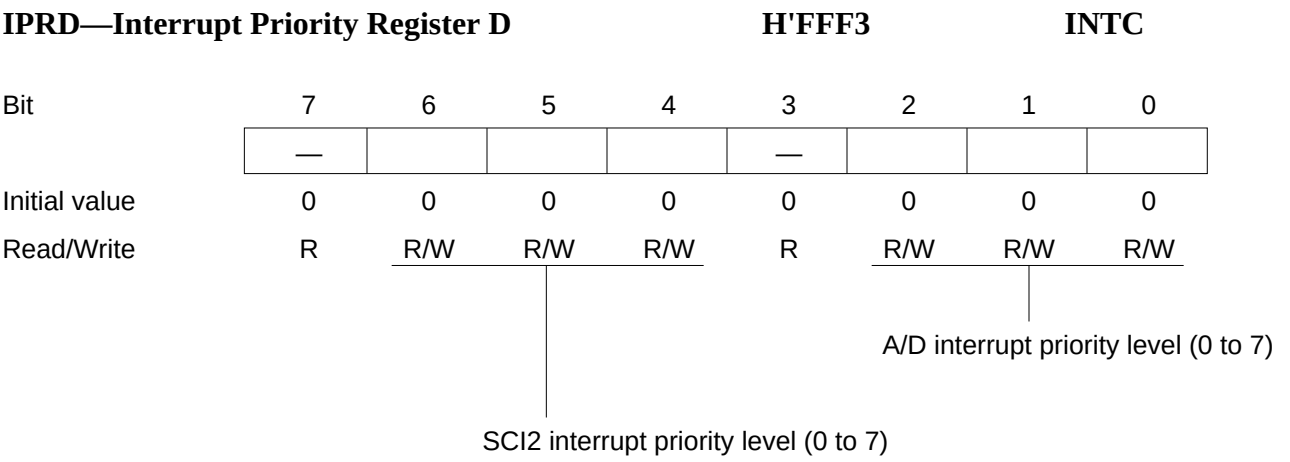
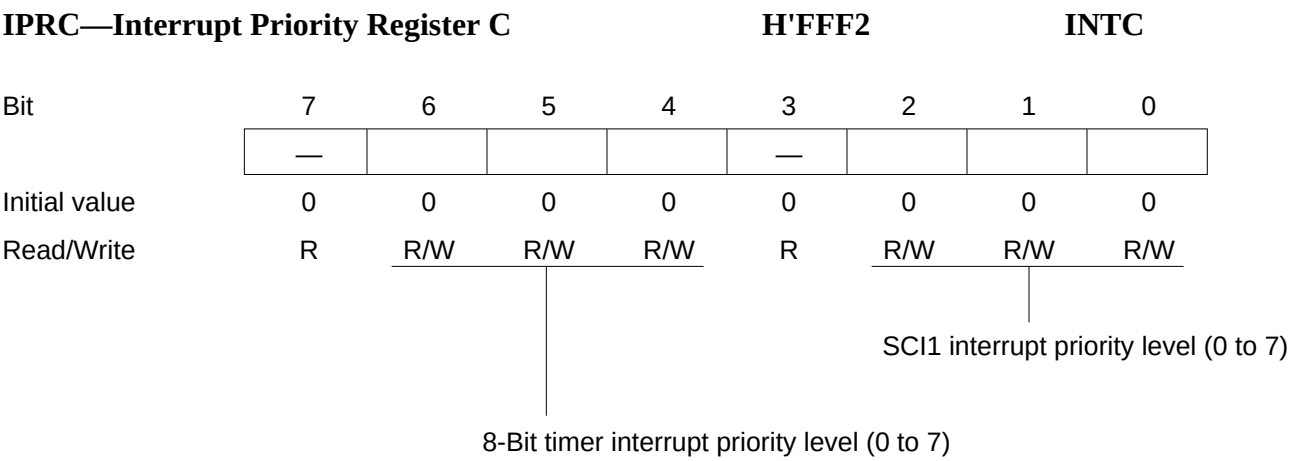
Watchdog Timer Reset

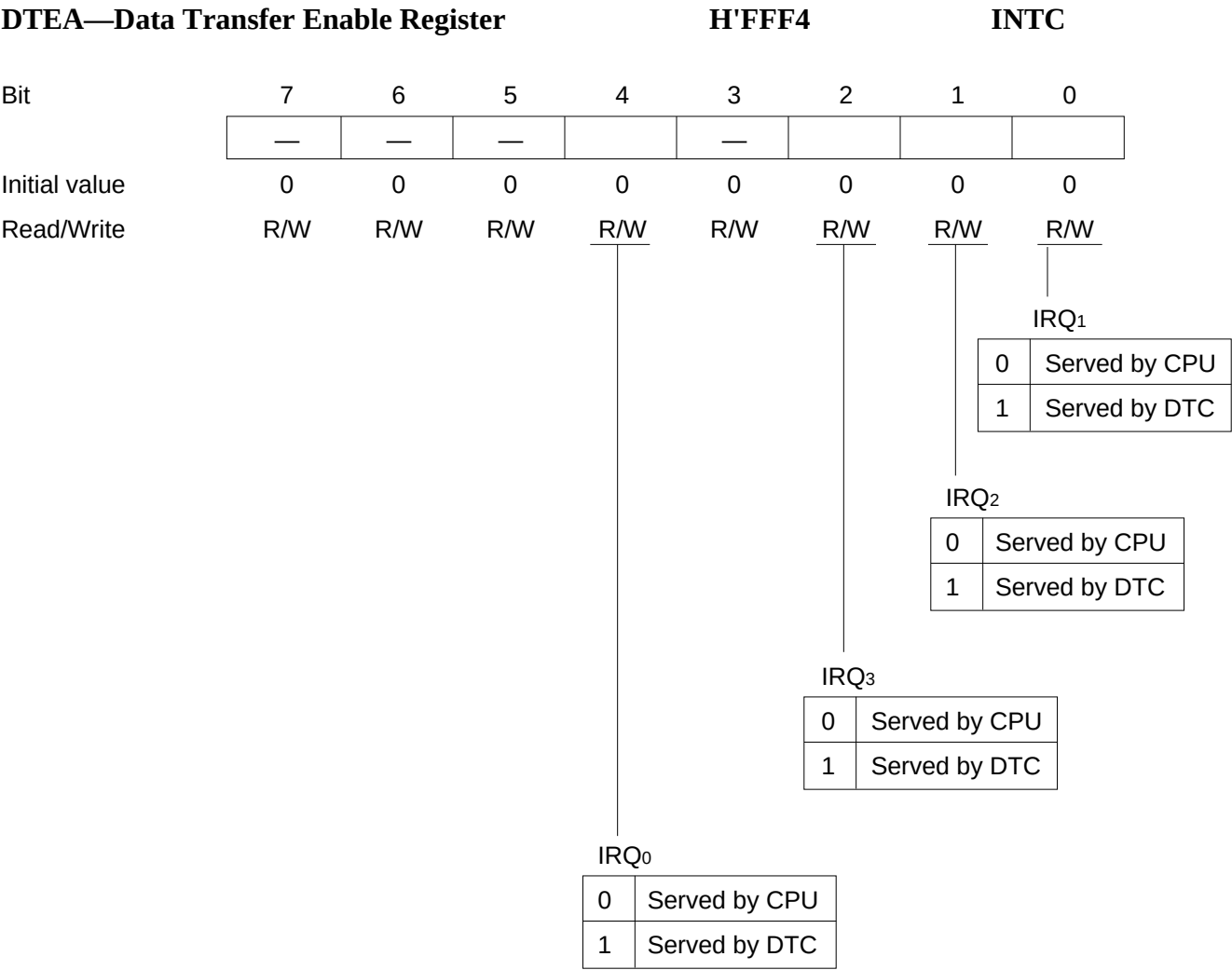
0	Cleared from 1 to 0 by software, or by a low input at the $\overline{\text{RES}}$ pin.
1	Set to 1 when TCNT overflows and a reset signal is generated.

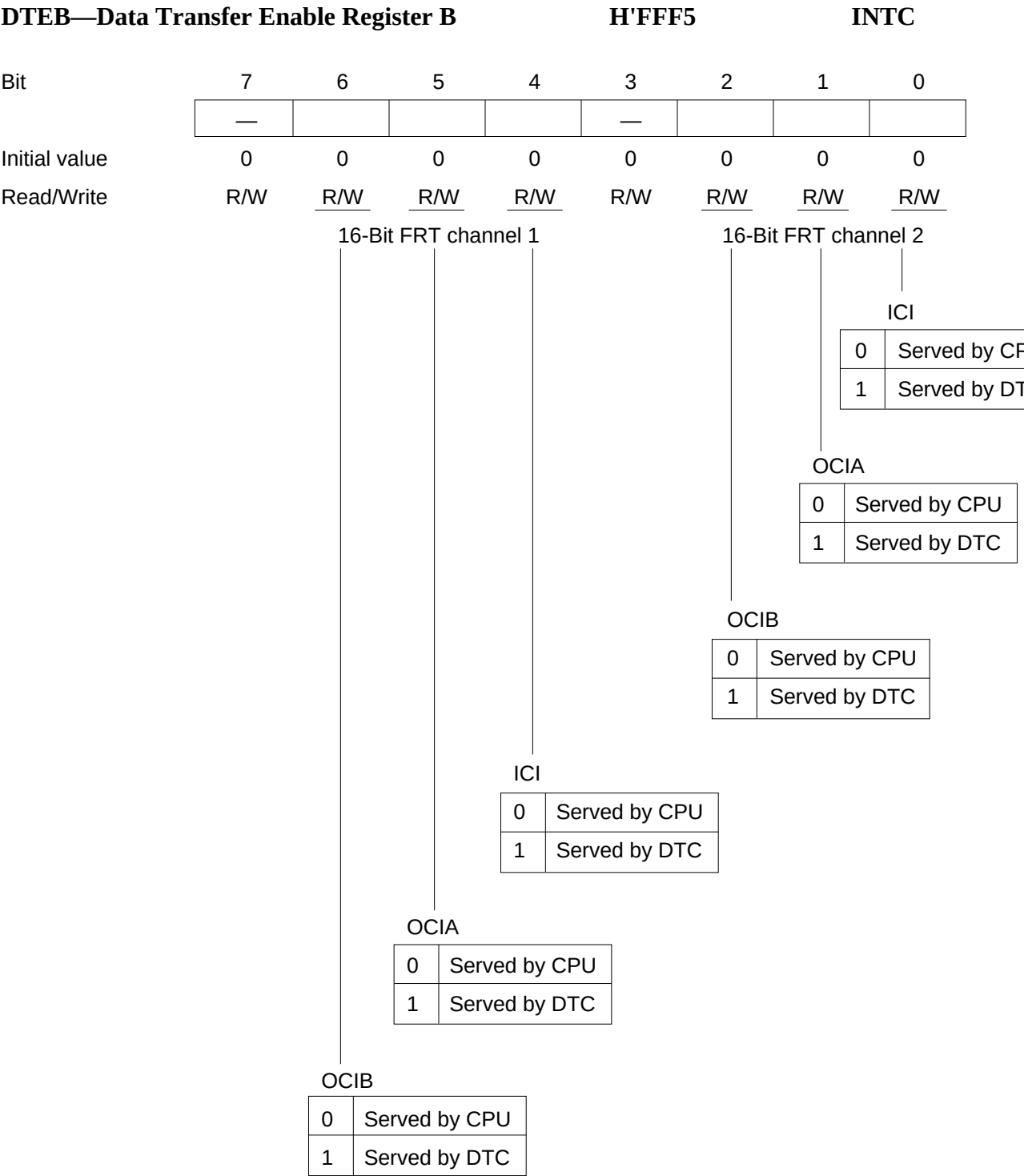
- Notes: 1. Software can write a 0 in bit 7 to clear the flag but cannot write a 1.
2. Read address: H'FFFF
Write address: H'FFFE

IPRA—Interrupt Priority Register A					H'FFF0		INTC	
Bit	7	6	5	4	3	2	1	0
	—				—			
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R/W	R/W	R/W	R	R/W	R/W	R/W
IRQ ₀ interrupt priority level (0 to 7)					IRQ ₁ to IRQ ₇ interrupt priority level (0 to 7)			

IPRB—Interrupt Priority Register B					H'FFF1		INTC	
Bit	7	6	5	4	3	2	1	0
	—				—			
Initial value	0	0	0	0	0	0	0	0
Read/Write	R	R/W	R/W	R/W	R	R/W	R/W	R/W
16-Bit FRT1 interrupt priority level (0 to 7)					16-Bit FRT2 interrupt priority level (0 to 7)			







Bit	7	6	5	4	3	2	1	0
	—	—			—			—
Initial value	0	0	0	0	0	0	0	0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

8-Bit timer

SCI1

RXI

0	Served by CPU
1	Served by DTC

TXI

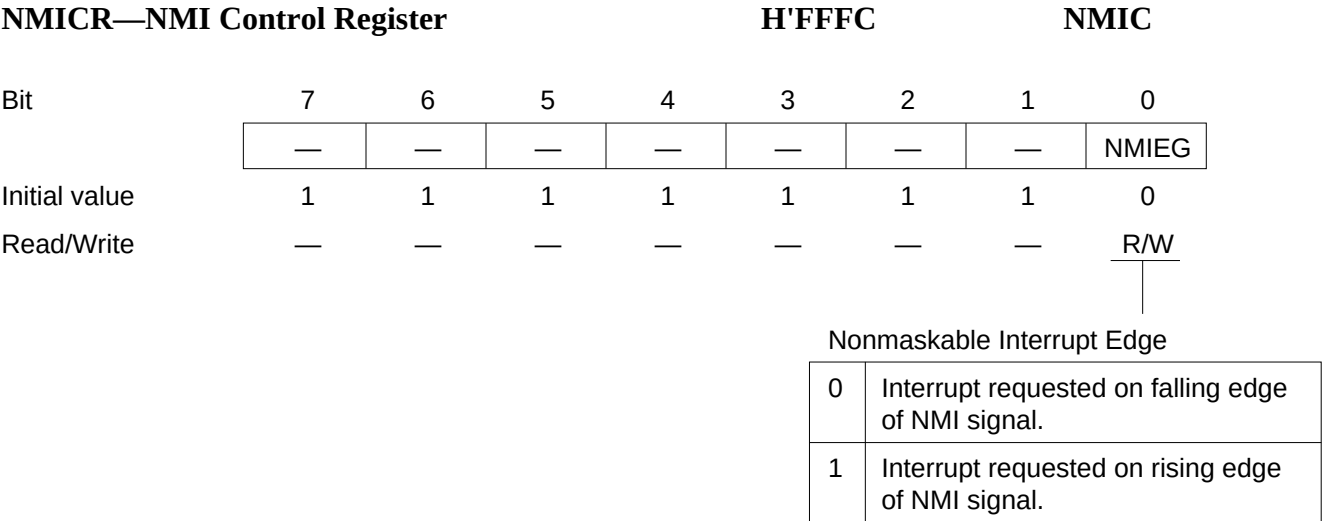
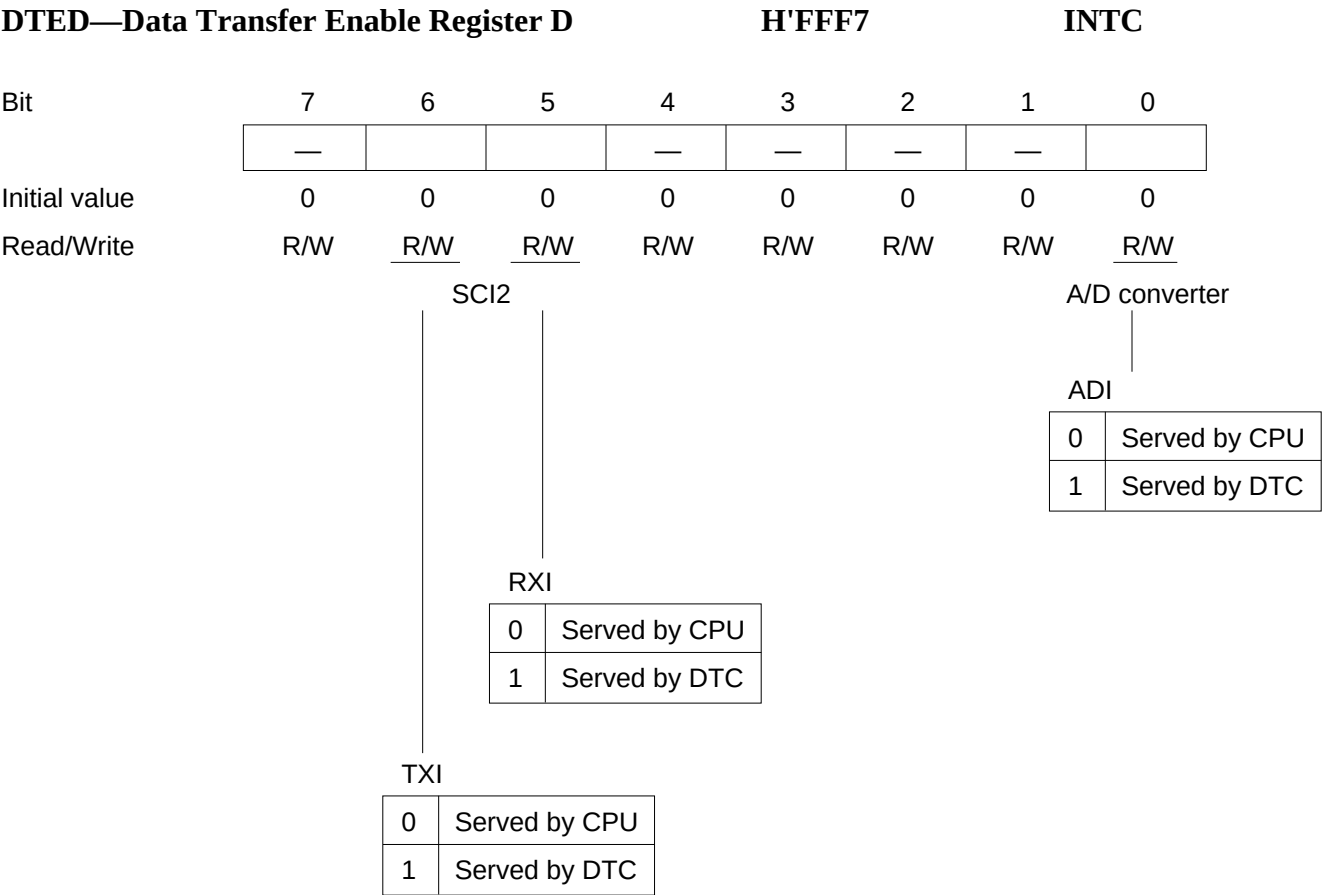
0	Served by CPU
1	Served by DTC

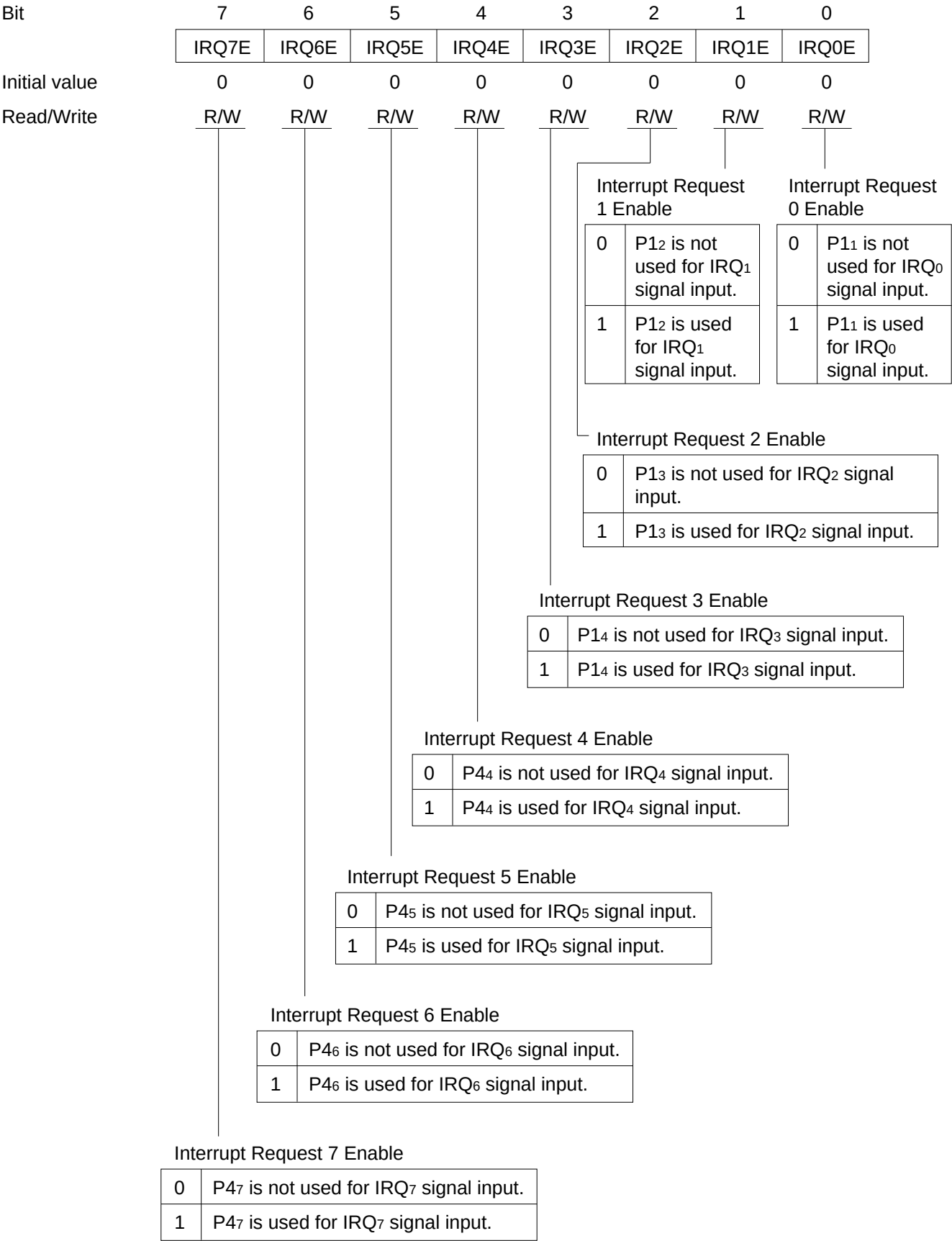
CMIA

0	Served by CPU
1	Served by DTC

CMIB

0	Served by CPU
1	Served by DTC





WCR—Wait-State Control Register

H'FFF8

WSC

Bit

7	6	5	4	3	2	1	0
—	—	—	—	WMS1	WMS0	WC1	WC0

Initial value

1	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

Read/Write

—	—	—	—	R/W	R/W	R/W	R/W
---	---	---	---	-----	-----	-----	-----

Wait Count 1 and 0

0	0	No wait states (Tw) are inserted
0	1	1 wait state is inserted.
1	0	2 wait states are inserted.
1	1	3 wait states are inserted.

Wait Mode Select 1 and 0

0	0	Programmable wait mode
0	1	No wait states are inserted, regardless of the wait count.
1	0	Pin wait mode
1	1	Pin auto-wait mode

RAMCR—RAM Control Register				H'FFF9		RAM		
Bit	7	6	5	4	3	2	1	0
	RAME	—	—	—	—	—	—	—
Initial value	1	1	1	1	1	1	1	1
Read/Write	R/W	—	—	—	—	—	—	—

RAM Enable

0	On-chip RAM is disabled.
1	On-chip RAM is enabled.

MDCR—Mode Control Register

H'FFFA

Bit	7	6	5	4	3	2	1	0
	—	—	—	—	—	MDS2	MDS1	MDS0
Initial value	1	1	0	0	0	—*	—*	—*
Read/Write	—	—	—	—	—	R	R	R

Mode Select

Value input at mode pins

Note: * Initialized according to the inputs at pins MD₂, MD₁, and MD₀.

SBYCR—Software Standby Control Register

H'FFFB

Bit	7	6	5	4	3	2	1	0
	SSBY	—	—	—	—	—	—	—
Initial value	0	1	1	1	1	1	1	1
Read/Write	R/W	—	—	—	—	—	—	—

Software Standby

0	SLEEP instruction causes transition to sleep mode.
1	SLEEP instruction causes transition to software standby mode.

Appendix C I/O Port Schematic Diagrams

C.1 Schematic Diagrams of Port 1

Figure C-1 (a) to (g) gives a schematic view of the port 1 input/output circuits.

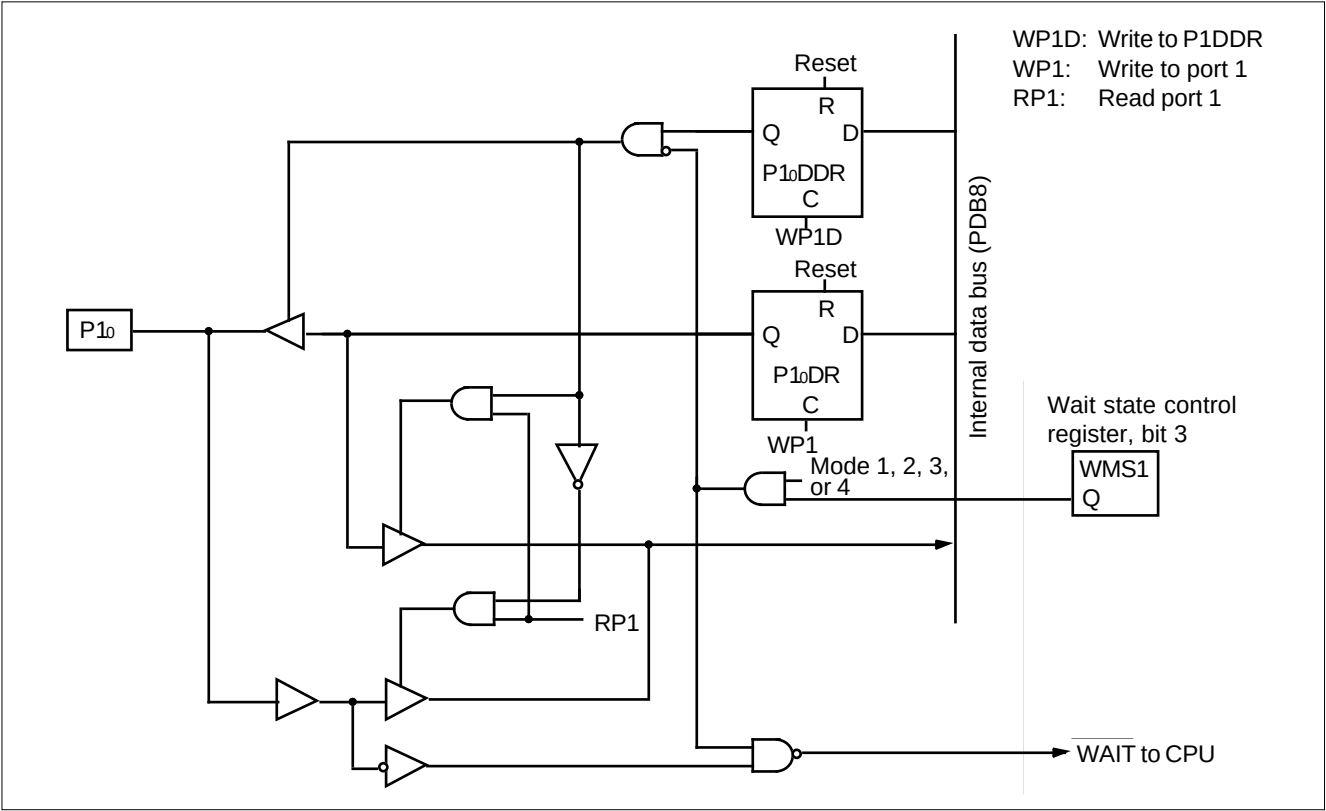


Figure C-1 (a) Schematic Diagram of Port 1, Pin P10

Table C-1 (a) Data Read from Port 1 (Pin P10)

Mode	Setting	Port Read Data
Mode 1, 2, 3, or 4	WMS1 = 1	Pin value
	WMS1 = 0	DDR = 0 Pin value
		DDR = 1 Value written in DR
Mode 7	DDR = 0	Pin value
	DDR = 1	Value written in DR

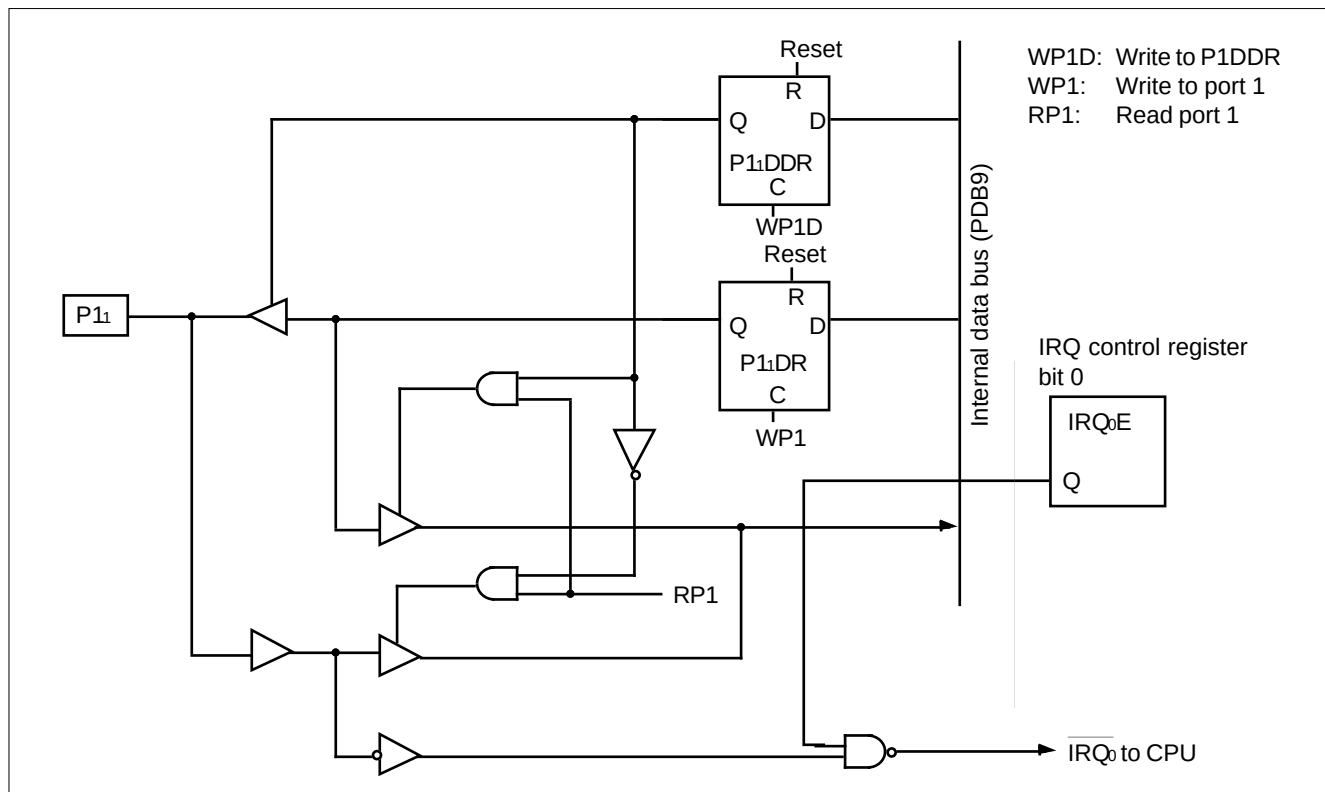


Figure C-1 (b) Schematic Diagram of Port 1, Pin P11

Table C-1 (b) Data Read from Port 1 (Pin P11)

Setting	Port Read Data
DDR = 0	Pin value
DDR = 1	Value written in DR

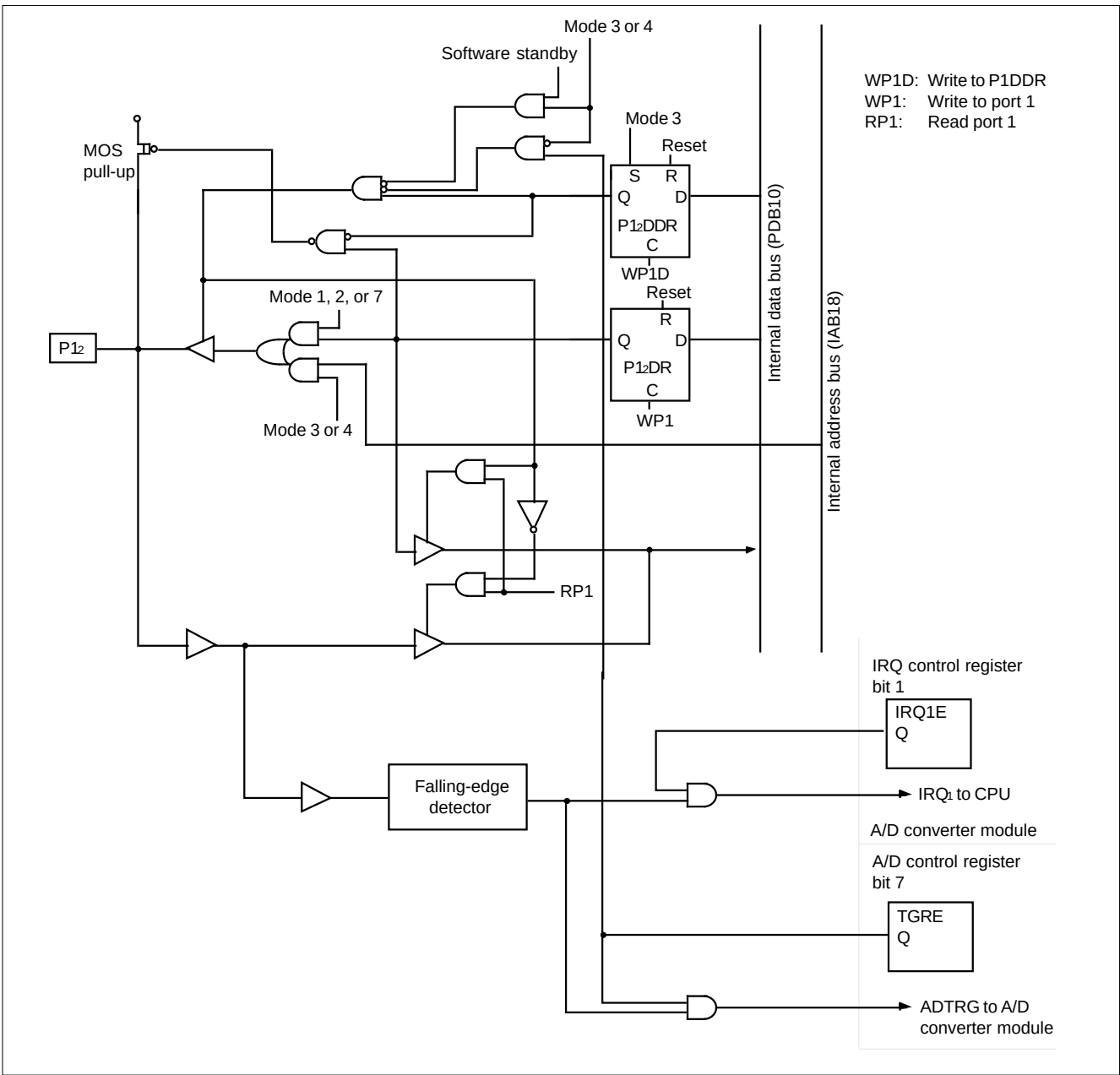


Figure C-1 (c) Schematic Diagram of Port 1, Pin P12

Table C-1 (c) Data Read from Port 1 (Pin P12)

Mode	Setting		Port Read Data
Mode 1, 2, or 7	TRGE = 1		Value written in DR
	TRGE = 0	DDR = 0	Pin value
		DDR = 1	Value written in DR
Mode 3			Value written in DR
Mode 4	DDR = 0		Pin value
	DDR = 1		Value written in DR

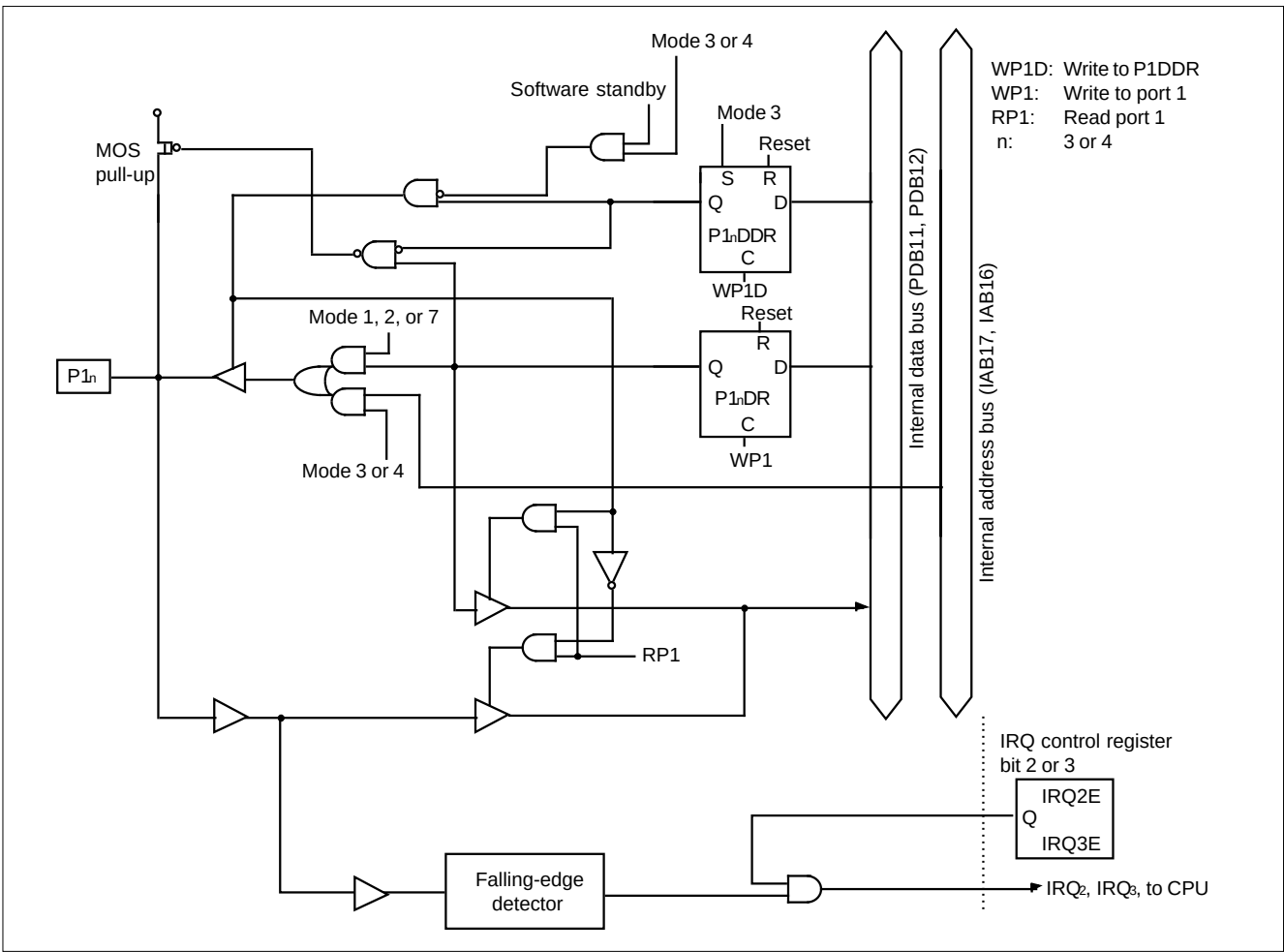


Figure C-1 (d) Schematic Diagram of Port 1, Pins P13 and P14

Table C-1 (d) Data Read from Port 1 (Pins P13 and P14)

Mode	Setting	Port Read Data
Mode 3		Value written in DR
Mode 1, 2, 4, or 7	DDR = 0	Pin value
	DDR = 1	Value written in DR

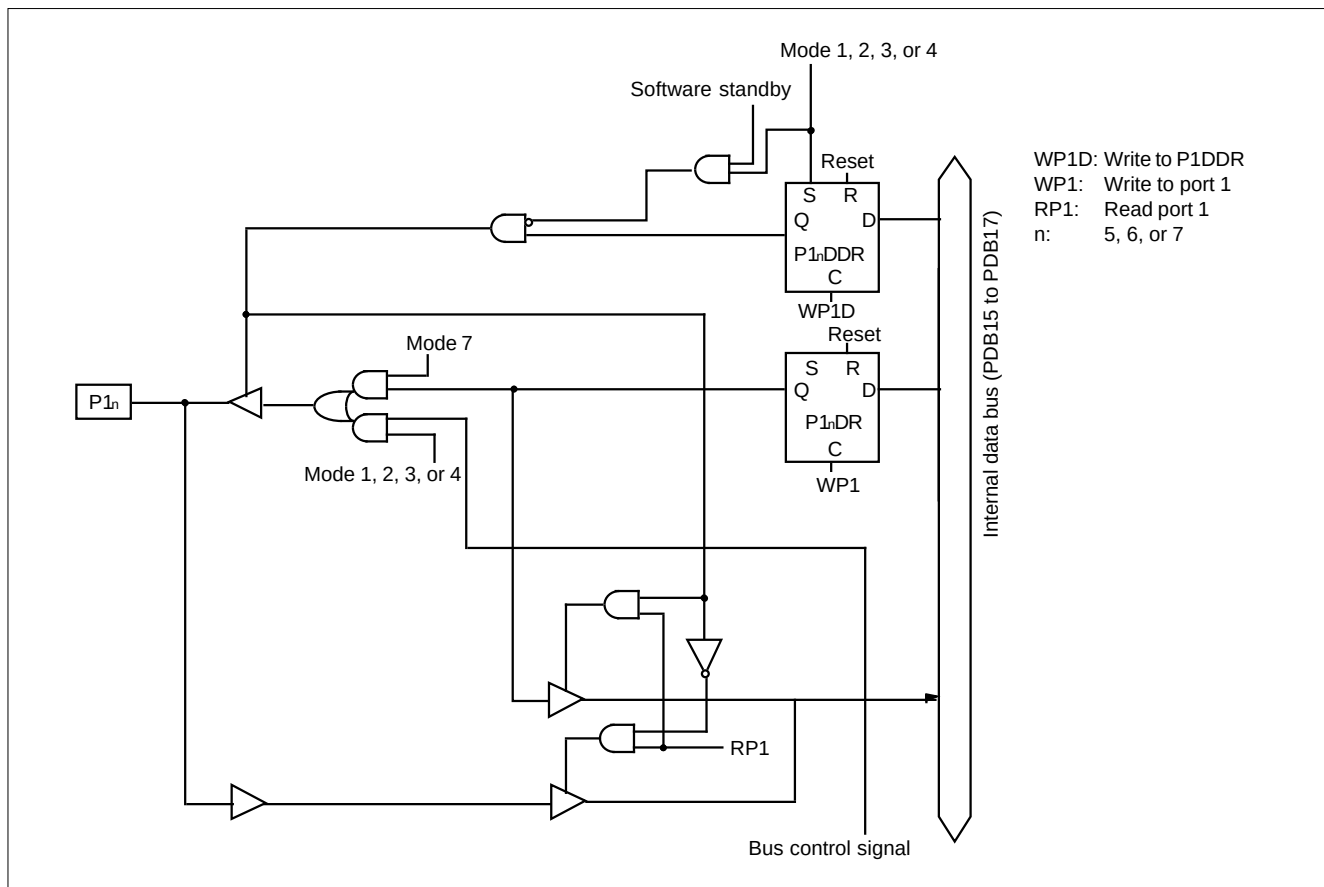


Figure C-1 (e) Schematic Diagram of Port 1, Pins P15, P16, and P17

Table C-1 (e) Data Read from Port 1 (Pins P15, P16, and P17)

Mode	Setting	Port Read Data
Mode 1, 2, 3, or 4		Value written in DR
Mode 7	DDR = 0	Pin value
	DDR = 1	Value written in DR

C.2 Schematic Diagram of Port 2

Figure C-2 gives a schematic view of the port 2 input/output circuits.

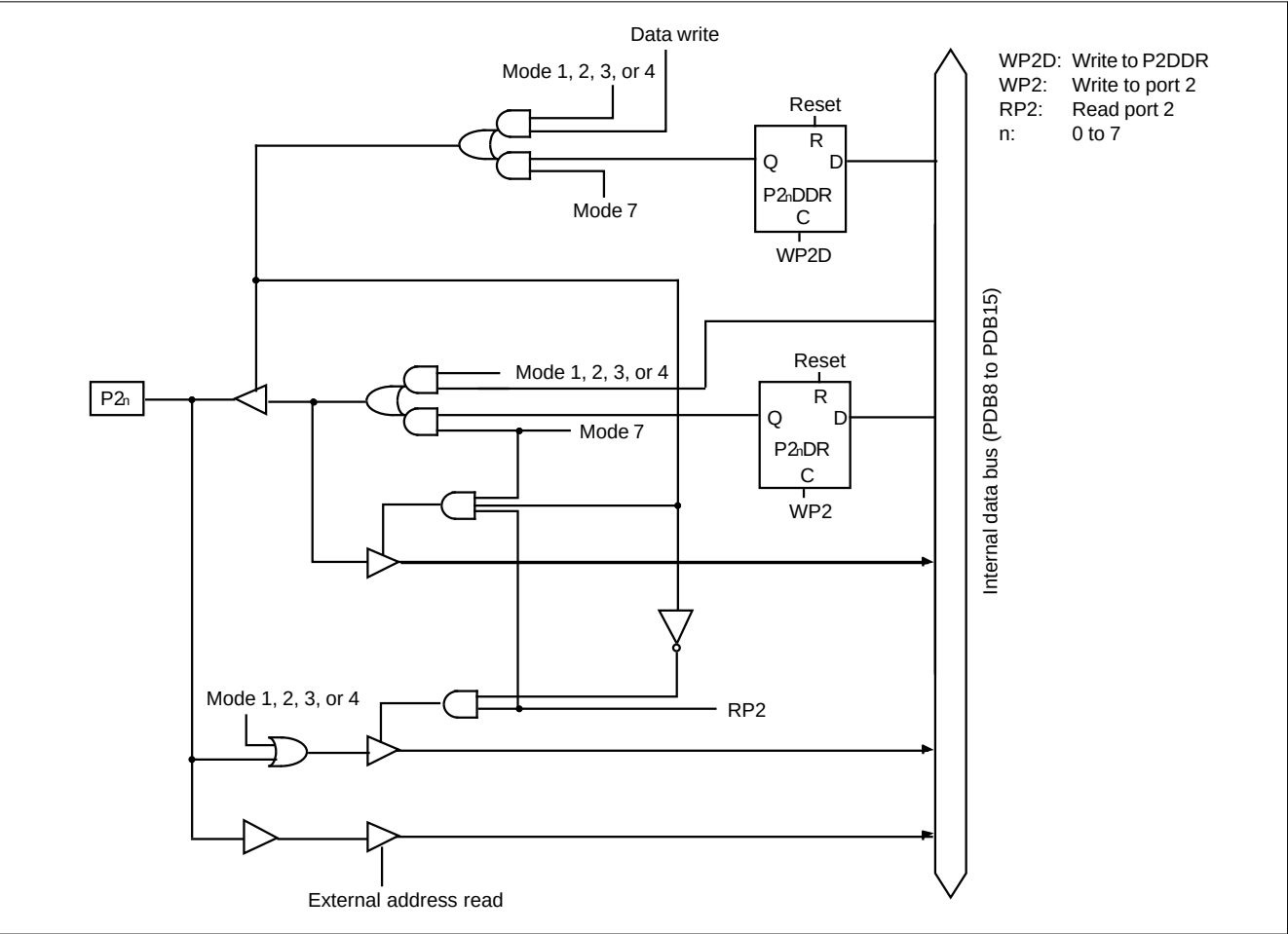


Figure C-2 Schematic Diagram of Port 2

Table C-2 Data Read from Port 2

Mode	Setting	Port Read Data
Mode 1, 2, 3, or 4		Always read as 1
Mode 7	DDR = 0	Pin value
	DDR = 1	Value written in DR

C.3 Schematic Diagram of Port 3

Figure C-3 gives a schematic view of the port 3 input/output circuits.

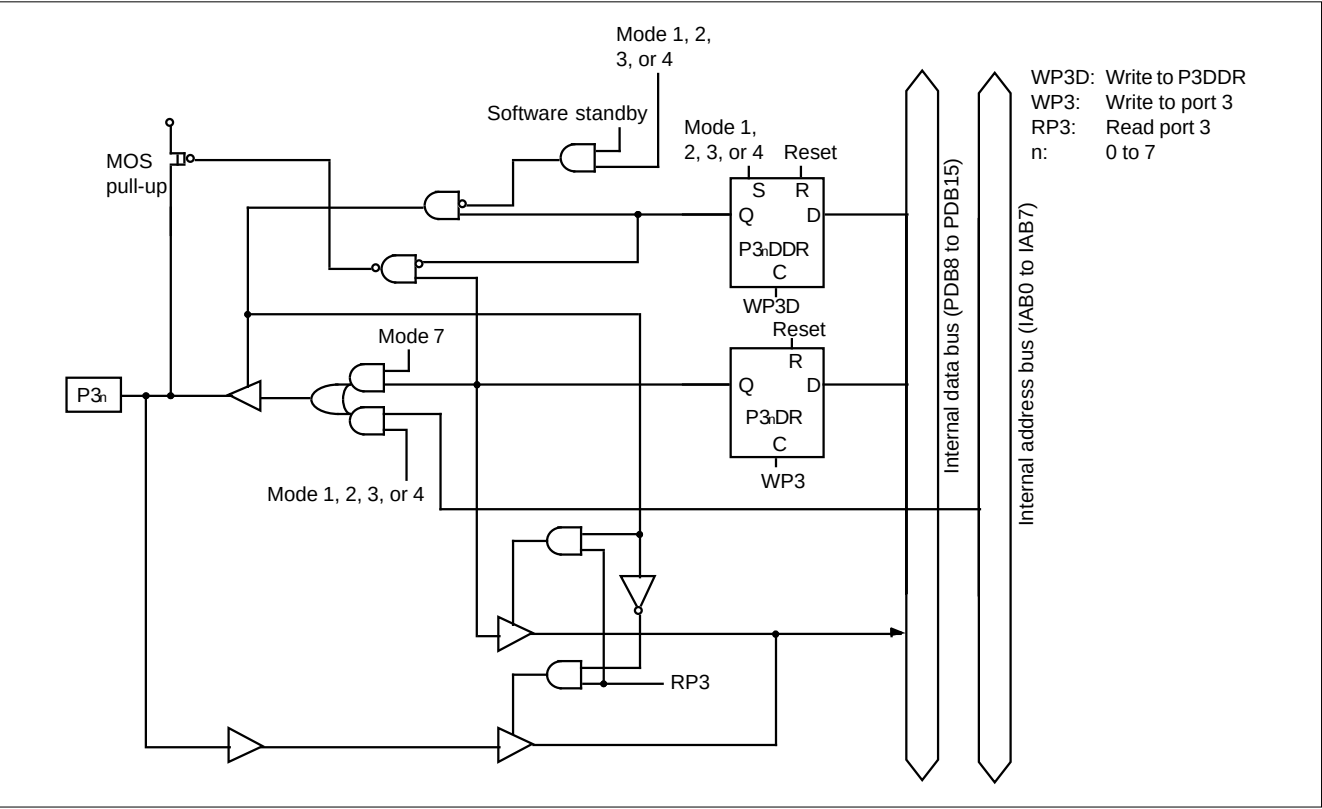


Figure C-3 Schematic Diagram of Port 3

Table C-3 Data Read from Port 3

Mode	Setting	Port Read Data
Mode 1, 2, 3, or 4		Value written in DR
Mode 7	DDR = 0	Pin value
	DDR = 1	Value written in DR

C.4 Schematic Diagram of Port 4

Figure C-4 (a) and (b) gives a schematic view of the port 4 input/output circuits.

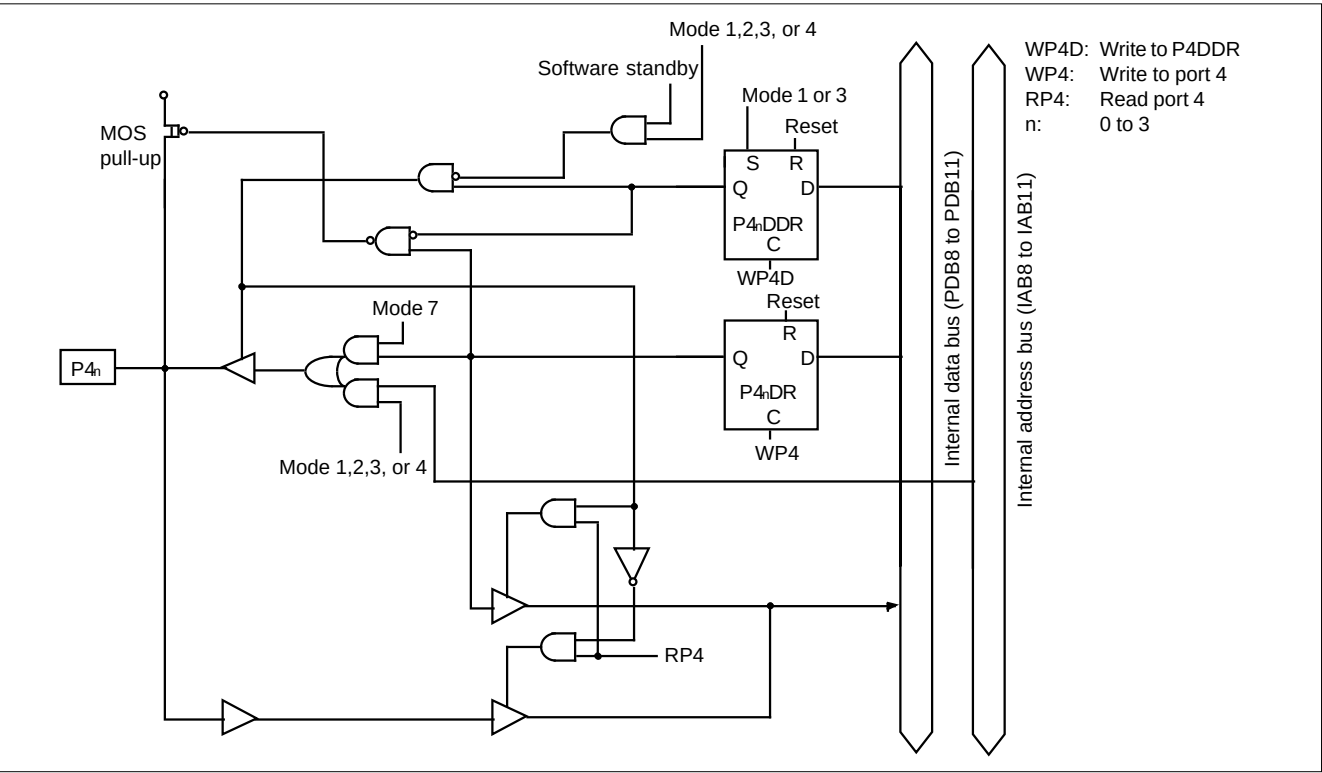


Figure C-4 (a) Schematic Diagram of Port 4, Pins 40 to 43

Table C-4 (a) Data Read from Port 4 (Pins 40 to 43)

Mode	Setting	Port Read Data
Mode 1 or 3		Value written in DR
Mode 2, 4, or 7	DDR = 0	Pin value
	DDR = 1	Value written in DR

C.5 Schematic Diagram of Port 5

Figure C-5 (a) to (g) gives a schematic view of the port 5 input/output circuits.

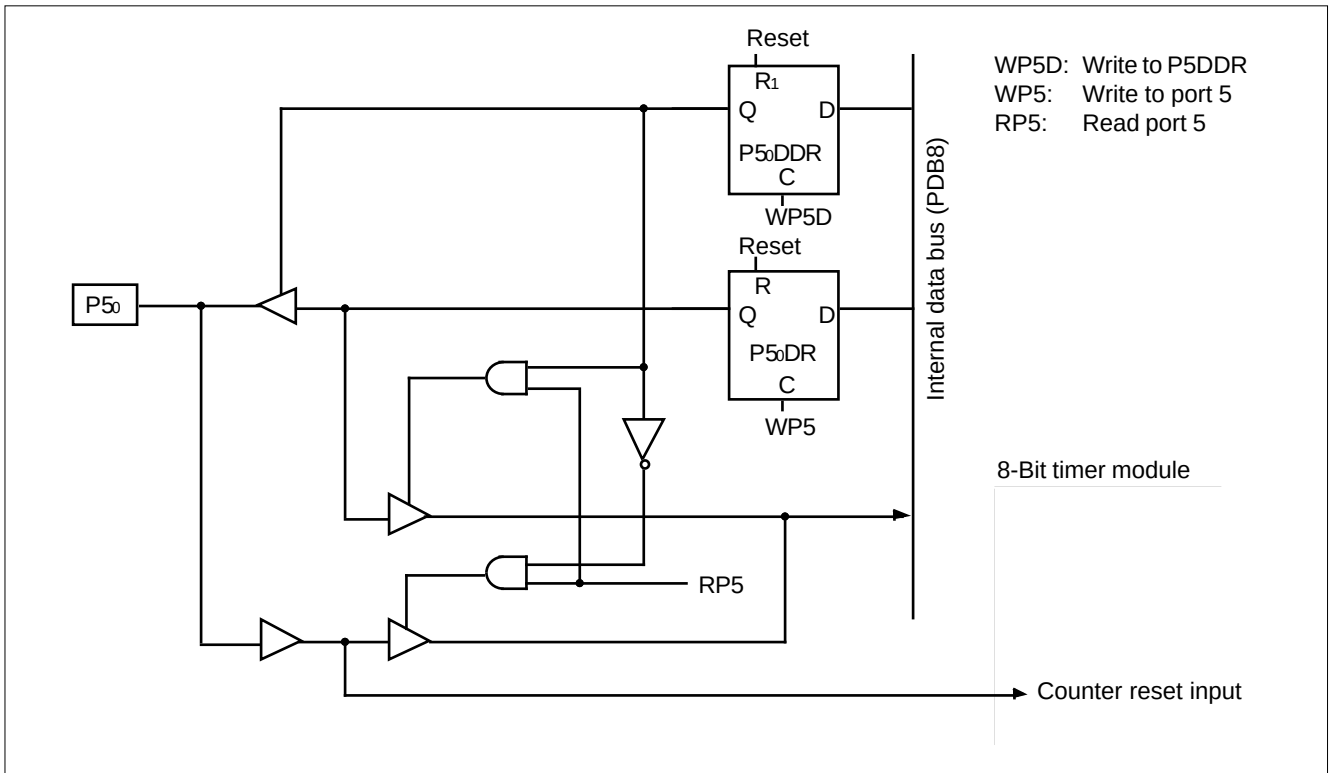


Figure C-5 (a) Schematic Diagram of Port 5, Pin P50

Table C-5 (a) Data Read from Port 5 (Pin P50)

Setting	Port Read Data
DDR = 0	Pin value
DDR = 1	Value written in DR

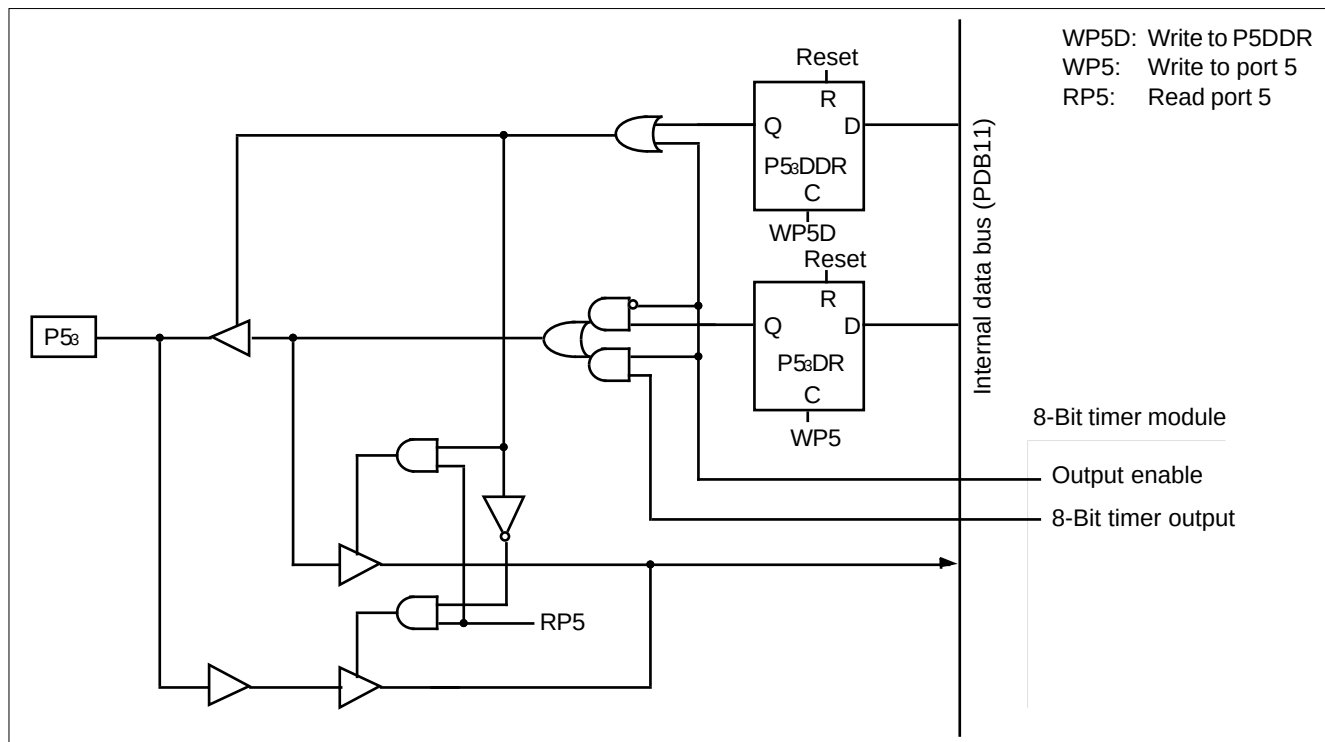


Figure C-5 (d) Schematic Diagram of Port 5, Pin P53

Table C-5 (d) Data Read from Port 5 (Pin P53)

Mode	Setting	Port Read Data
8-Bit timer output enabled		8-Bit timer output
8-Bit timer output disabled	DDR = 0	Pin value
	DDR = 1	Value written in DR

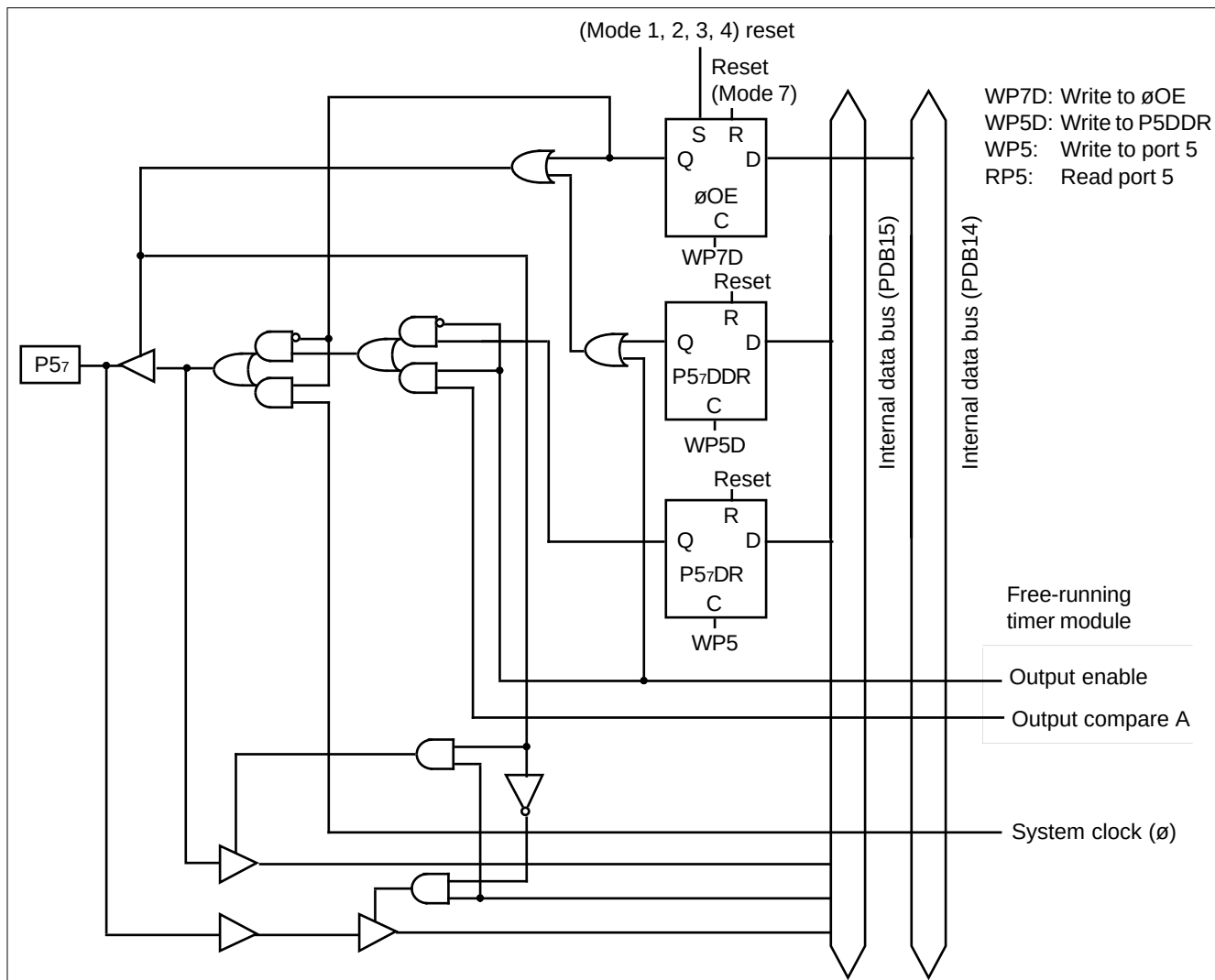


Figure C-5 (g) Schematic Diagram of Port 5, Pin P57

Table C-5 (g) Data Read from Port 5 (Pin 57)

Setting		Port Read Data
$\emptyset$ OE = 1		System clock value ($\emptyset$)
$\emptyset$ OE = 0	Output compare A enabled	Free-running timer output (Output compare A)
	Output compare A disabled	DDR = 0 Pin value
		DDR = 1 Value written in DR

C.6 Schematic Diagram of Port 6

Figure C-6 gives a schematic view of the port 6 input circuits.

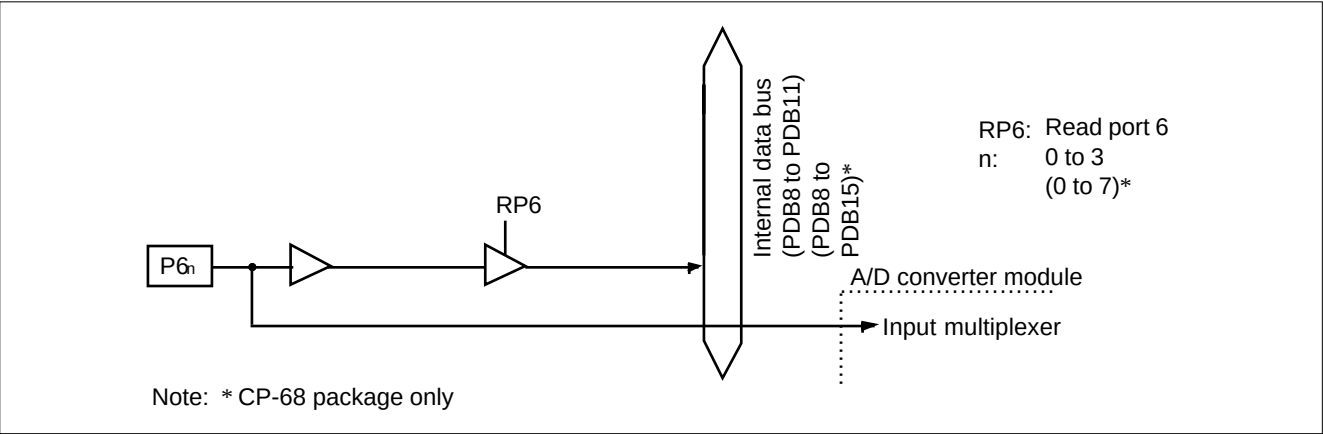


Figure C-6 Schematic Diagram of Port 6

C.7 Schematic Diagrams of Port 7

Figure C-7 (a) to (d) gives a schematic view of the port 7 input/output circuits.

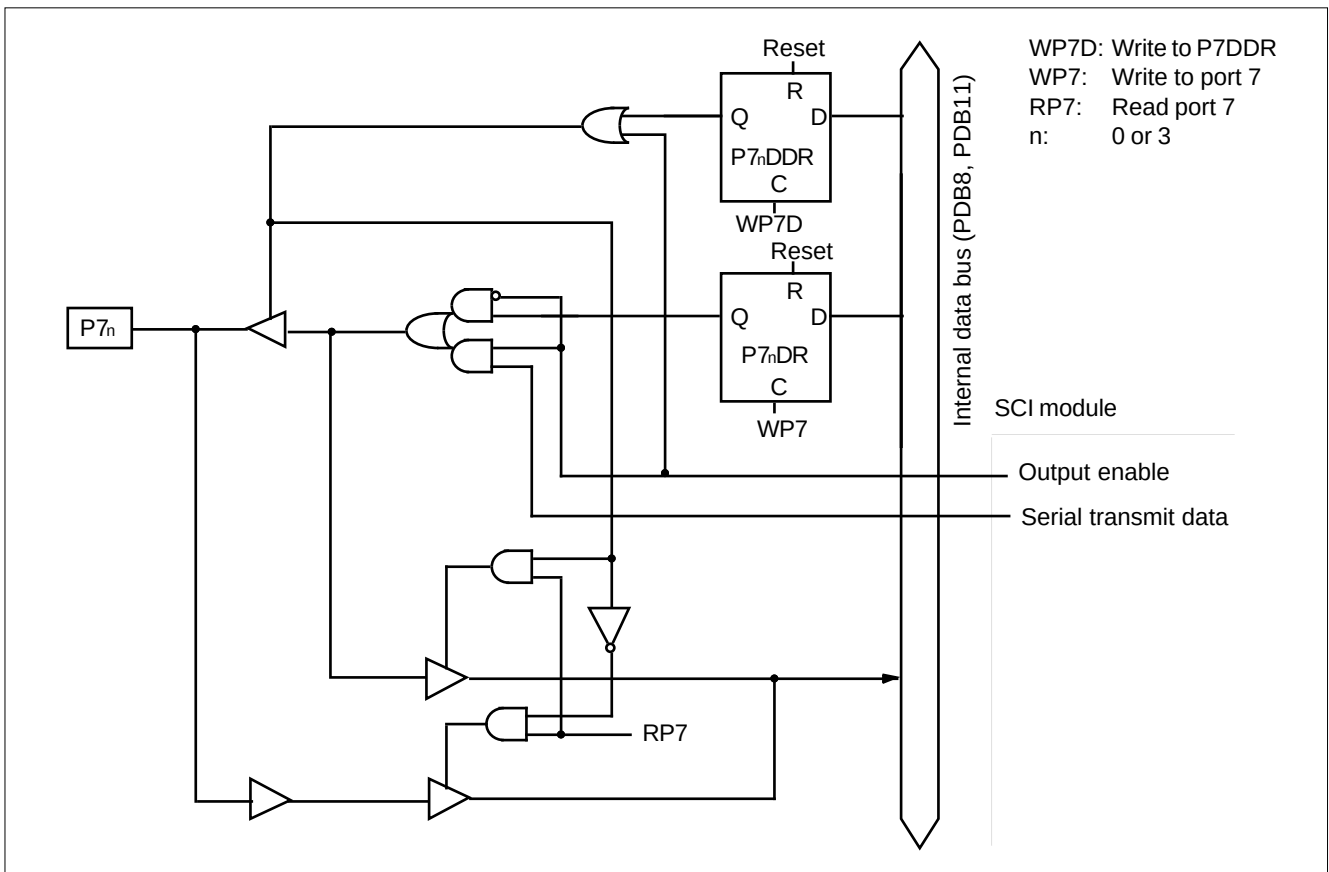


Figure C-7 (a) Schematic Diagram of Port 7, Pins P70 and P73

Table C-7 (a) Data Read from Port 7 (Pins P70 and P73)

Setting		Port Read Data
Serial output enabled		Serial transmit data
Serial output disabled	DDR = 0	Pin value
	DDR = 1	Value written in DR

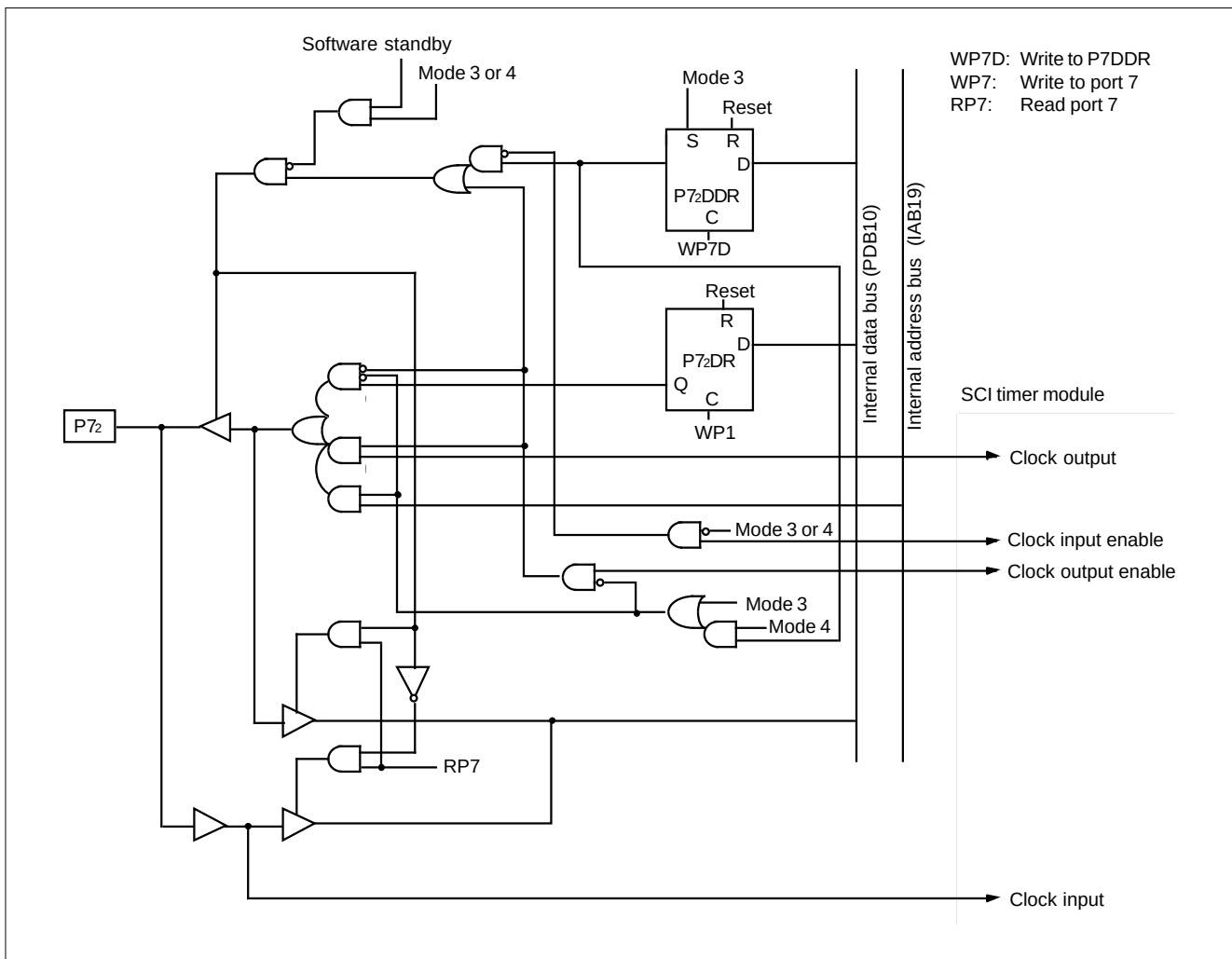


Figure C-7 (c) Schematic Diagram of Port 7, Pin P72

Table C-7 (c) Data Read from Port 7 (Pin P72)

Mode	Setting	Port Read Data
Mode 3		IAB19 value
Mode 4	DDR = 1	IAB19 value
Mode 1, 2, or 7, or	Serial clock input enabled	Pin value
Mode 4 with DDR = 0	Serial clock output enabled	Output clock value
Mode 4 with DDR = 0	Serial clock input and output disabled	Pin value
Mode 1, 2, or 7	Serial clock input	DDR = 0 Pin value
	and output disabled	DDR = 1 Value written in DR

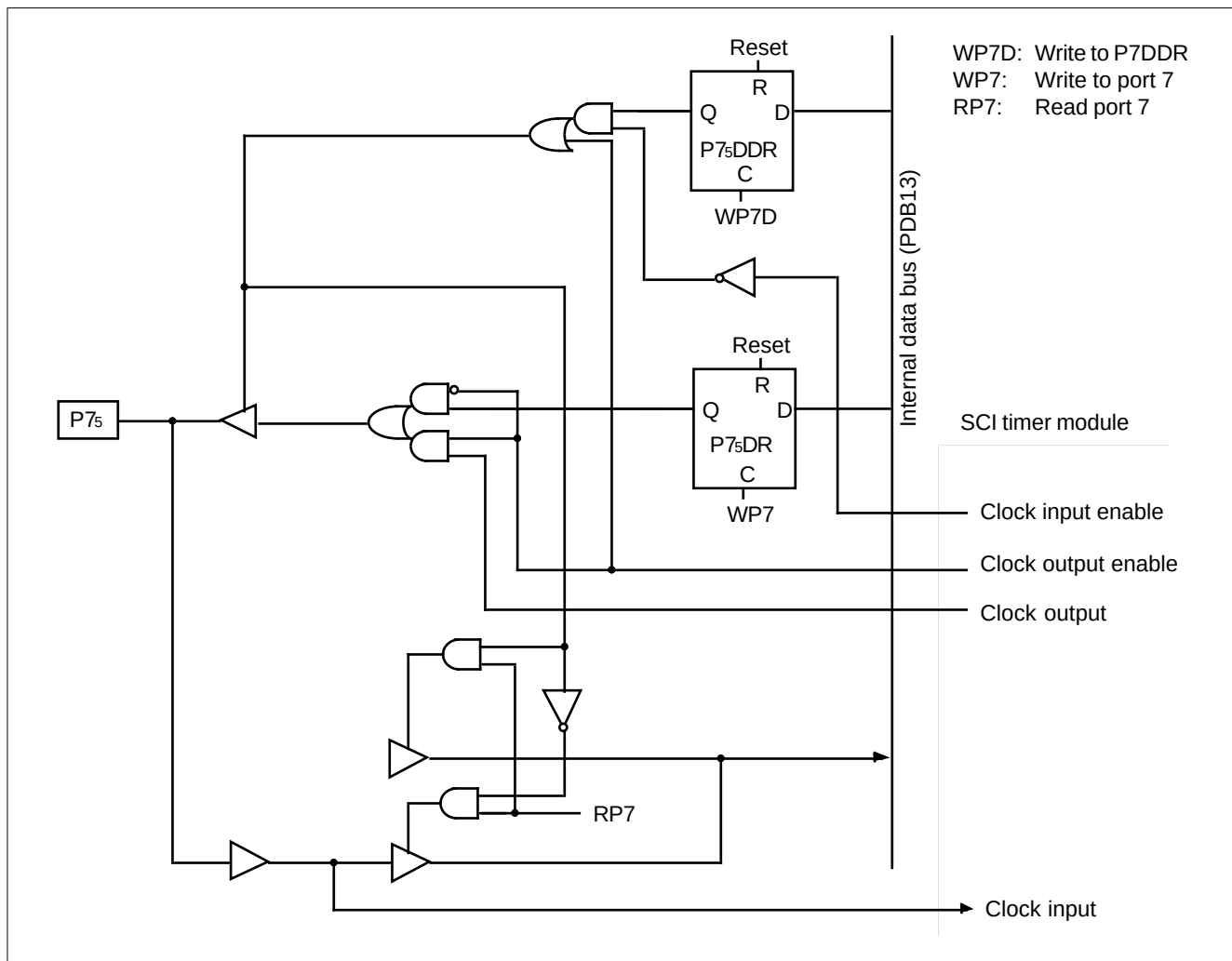
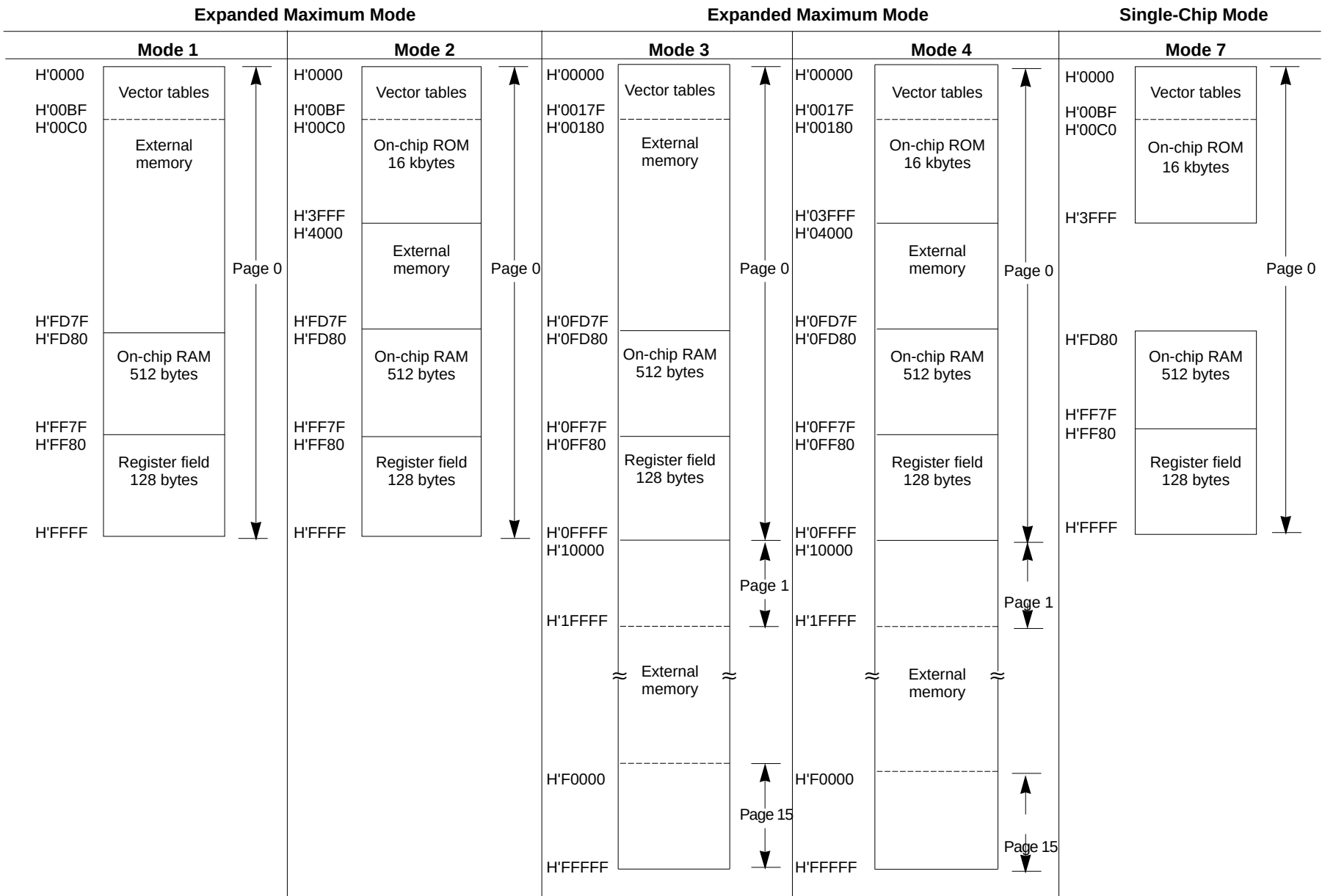


Figure C-7 (d) Schematic Diagram of Port 7, Pin P75

Table C-7 (d) Data Read from Port 7 (Pin P75)

Setting		Port Read Data
Serial clock input enabled		Input clock value
Serial clock output enabled		Output clock value
Serial clock input and output disabled	DDR = 0	Pin value
	DDR = 1	Value written in DR

Appendix D Memory Map



Appendix E Pin States

E.1 Port State of Each Pin State

Table E-1 Port Pin States in Each Mode

Port Pin Name	Mode	Reset	Hardware Standby Mode	Software Standby Mode	Sleep Mode	Program Execution State (Normal Operation)	Notes
P10 / $\overline{\text{WAIT}}$ P10 / $\overline{\text{IRQ}}_0$	1	T	T	keep	keep	Control signal input or input/output port	
	2						
	3						
	4						
	7						
P14 to P12 $\overline{\text{IRQ}}_3$ / $\overline{\text{IRQ}}_1$ (A16 to A18) (ADTRG)	1	T	T	keep	keep	Control signal input or input/output port	In the program execution state, P12 also receives the A/D trigger $\overline{\text{ADTRG}}$
	2						
	3	L		T	L	A16 to A18	
	4	T			*2	Address bus, control signal input, or input port	
	7			keep	keep	Control signal input or input/output port	
P17 to P15 $\overline{\text{AS}}$, $\overline{\text{RD}}$, $\overline{\text{WR}}$,	1	H	T	T	H	$\overline{\text{AS}}$, $\overline{\text{RD}}$, $\overline{\text{WR}}$,	
	2						
	3						
	4						
	7	T		keep	keep	input/output port	
P27 to P20 D7 to D0	1	T	T	T	T	D7 to D0	
	2						
	3						
	4						
	7			keep	keep	Input/output port	
P37 to P30 A7 to A0	1	L	T	T	L	A7 to A0	Programmable MOS input pull-ups
	2						
	3						
	4						
	7	T		keep	keep	Input/output port	

Table E-1 Port Pin States in Each Mode (cont)

Port Pin Name	Mode	Reset	Hardware Standby Mode	Software Standby Mode	Sleep Mode	Program Execution State (Normal Operation)	Notes
P4 ₃ to P4 ₀ A ₁₁ to A ₈	1	L	T	T	L	A ₁₁ to A ₈	Programmable
	2	T			*2	Address bus or input port	MOS input pull-ups
	3	L			L	A ₁₁ to A ₈	
	4	T			*2	Address bus or input port	
	7			keep	keep	Input/output port	
P4 ₇ to P4 ₄ A ₁₅ to A ₁₂ ($\overline{\text{IRQ}}_7$ to $\overline{\text{IRQ}}_4$)	1	L	T	T	L	A ₁₅ to A ₁₂	Programmable
	2	T			*2	Address bus, $\overline{\text{IRQ}}$ or input port	MOS input pull-ups
	3	L			L	A ₁₅ to A ₁₂	
	4	T			*2	Address bus, $\overline{\text{IRQ}}$ or input port	
	7			keep	keep	$\overline{\text{IRQ}}$ or input/ output port	
P5 ₇ / FTOA ₈ /∅	1	∅ output	∅ output	keep* ¹ (including ∅ output	keep	Input/output port	
	2						
	3						
	4						
	7	T	T				
P5 ₆ to P5 ₀ Timer input pins	1	T	T	keep* ¹	keep	Input/Output port	Schmitt trigger input
	2						
	3						
	4						
	7						
P6 ₃ to P6 ₀ [P6 ₇ to P6 ₀] AN ₃ to AN ₀ [AN ₇ to AN ₀]	1	T	T	T	T	Input port	Brackets [] apply to CP-68 package
	2						
	3						
	4						
	7						
P7 ₁ to P7 ₀ P7 ₅ to P7 ₃ TXD ₂ , RXD ₂ TXD ₁ , RXD ₁ SCK ₁	1	T	T	keep* ¹	keep	Input/output port	
	2						
	3						
	4						
	7						

Table E-1 Port Pin States in Each Mode (cont)

Port Pin Name	Mode	Reset	Hardware Standby Mode	Software Standby Mode	Sleep Mode	Program Execution State (Normal Operation)	Notes
P7 ₂ / SCK ₂ / A ₁₉	1	T	T	keep* ¹	keep	Input/output port	
	2						
	3	L		T	L	A ₁₉	
	4	T			* ²	Address bus or input/output port	
	7			keep* ¹	keep	Input/output port	

H: High

L: Low

T: Three-state (high impedance)

keep: Output pins retain their previous states. Input pins go to the high-impedance state. In ports 3 and 4, if DDR = 0 and DR = 1, input MOS pull-ups remain on.

- Notes: 1. On-chip supporting modules are reset, so these pins become input or output ports controlled by DDR and DR.
2. Input ports go to the high-impedance state. Address outputs go to low.

Table E-2 MOS Pull-Up States

Port	Mode	Reset	Hardware Standby Mode	Other Operating States*
P3 ₇ to P3 ₀ A ₇ to A ₀	1	OFF	OFF	OFF
	2			
	3			
	4			
	7			ON/OFF
P4 ₇ to P4 ₀ A ₁₅ to A ₈	1	OFF	OFF	OFF
	2			ON/OFF
	3			OFF
	4			ON/OFF
	7			

OFF: Pull-up MOS pull-up is always off.

ON/OFF: MOS pull-up is on when DDR = 0 and DR = 1 off at other times.

Note: * Including software standby mode

E.2 Pin Status in the Reset State

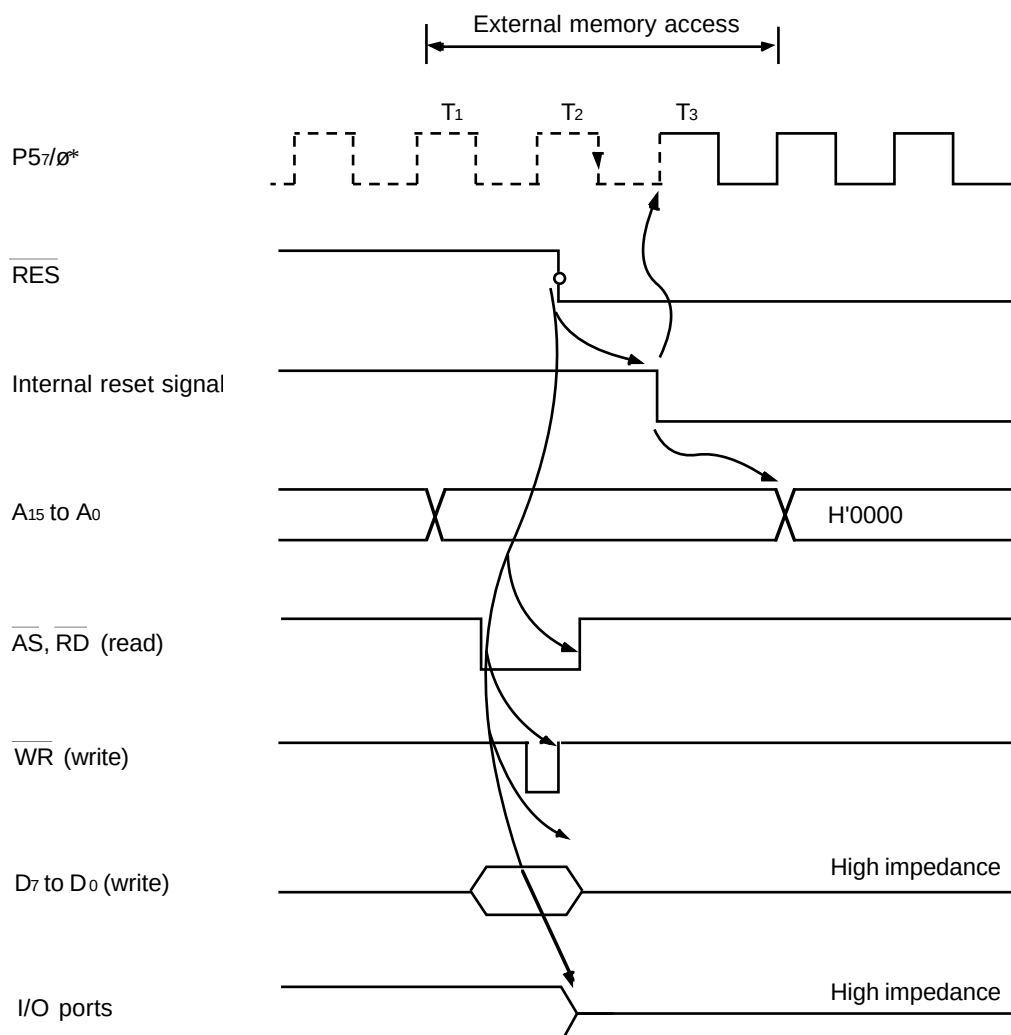
1. Mode 1

Figure E-1 shows how the pin states change when the $\overline{\text{RES}}$ pin goes low during external memory access in mode 1.

As soon as $\overline{\text{RES}}$ goes low, all ports are initialized to the input state. The $\overline{\text{AS}}$, $\overline{\text{RD}}$, and $\overline{\text{WR}}$ signals all go high. The data bus (D7 to D0) is placed in the high-impedance state.

The address bus is initialized 1.5 ϕ clock periods after the low state of the $\overline{\text{RES}}$ pin is sampled. All address bus signals are driven low.

The clock output pin P57/ ϕ is initialized to the clock output state and begins clock output 0.5 ϕ clock periods after the low state of the $\overline{\text{RES}}$ pin is sampled.



Note: * The dotted line indicates that P57/Ø is an input port or timer output pin if the ØOE bit is 0, but a clock output pin if the ØOE bit is 1.

Figure E-1 Reset During Memory Access (Mode 1)

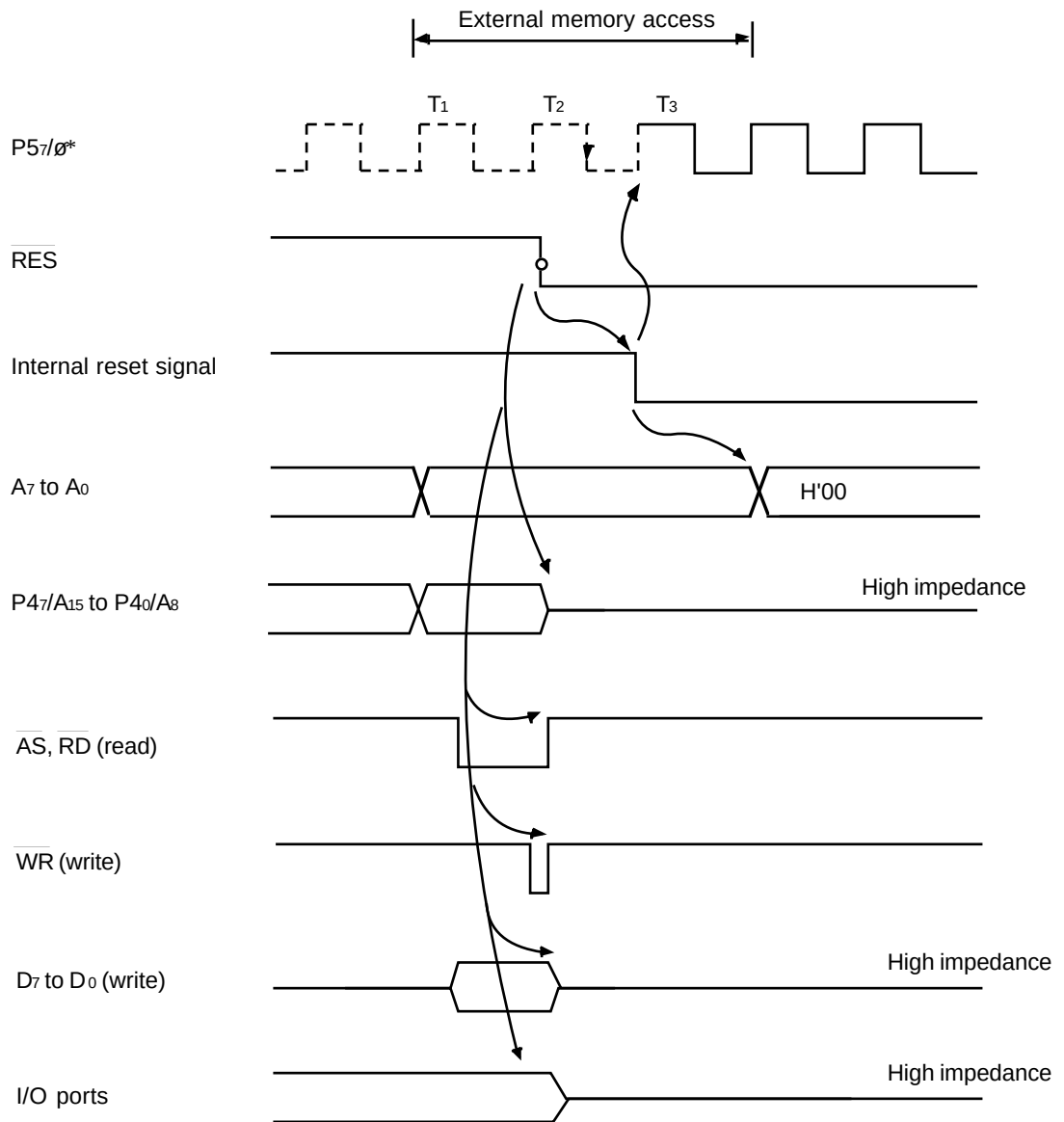
2. Mode 2

Figure E-2 shows how the pin states change when the RES pin goes low during external memory access in mode 2.

As soon as RES goes low, all ports are initialized to the input state. The AS, RD, and WR signals all go high. The data bus (D7 to D0) is placed in the high-impedance state. Pins P47/A15 to P40/A8 of the address bus are initialized as input ports.

Pins A7 to A0 of the address bus are initialized to the low state 1.5 ϕ clock periods after the low state of the $\overline{\text{RES}}$ pin is sampled.

The clock output pin P57/ ϕ is initialized to the clock output state and begins clock output 0.5 ϕ clock periods after the low state of the $\overline{\text{RES}}$ pin is sampled.



Note: * The dotted line indicates that P57/ ϕ is an input port or timer output pin if the ϕOE bit is 0, but a clock output pin if the ϕOE bit is 1.

Figure E-2 Reset During Memory Access (Mode 2)

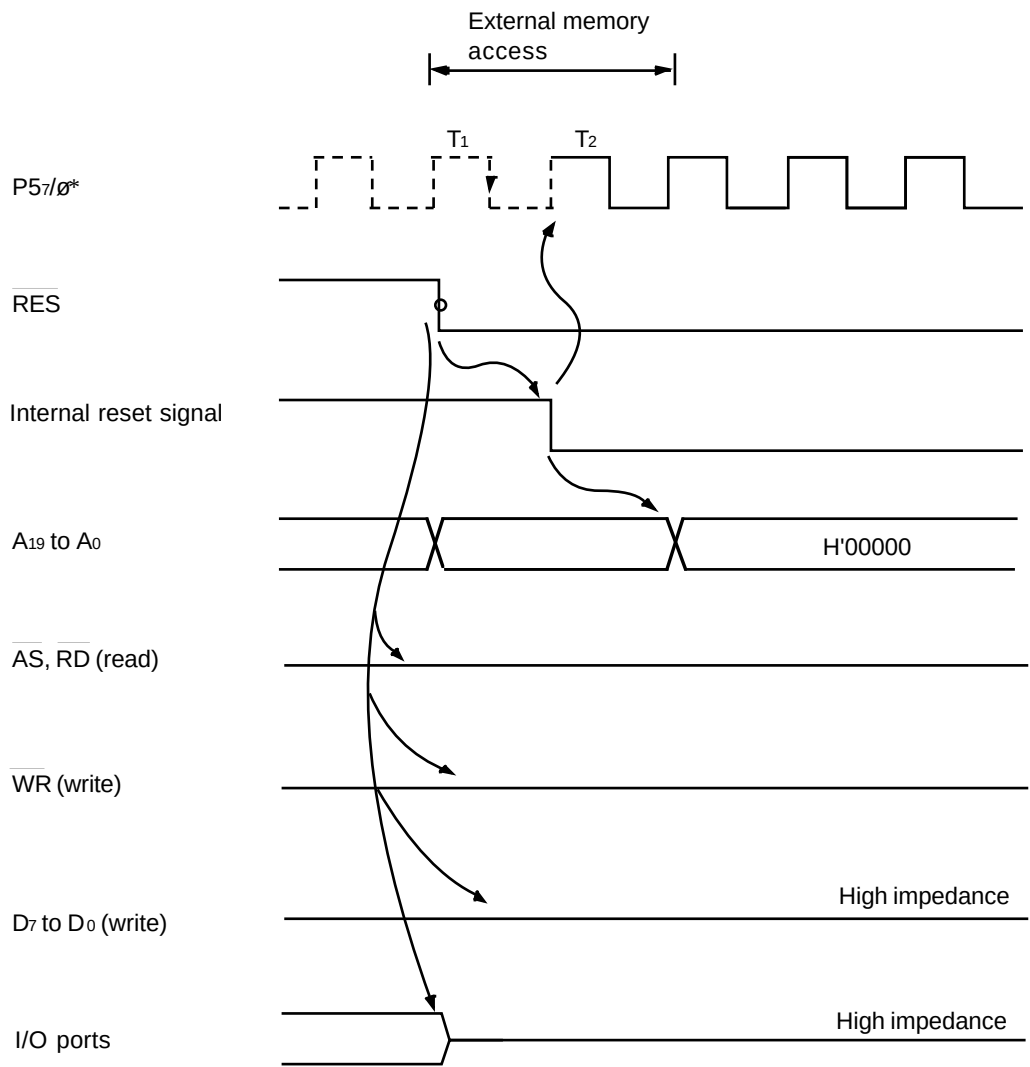
3. Mode 3

Figure E-3 shows how the pin states change when the $\overline{\text{RES}}$ pin goes low during external memory access in mode 3.

As soon as $\overline{\text{RES}}$ goes low, all ports are initialized to the input state. The $\overline{\text{AS}}$, $\overline{\text{RD}}$, and $\overline{\text{WR}}$ signals all go high. The data bus (D7 to D0) is placed in the high-impedance state.

The address bus is initialized 1.5 ϕ clock periods after the low state of the $\overline{\text{RES}}$ pin is sampled. All address bus signals are driven low.

The clock output pin P57/ ϕ is initialized to the clock output state and begins clock output 0.5 ϕ clock periods after the low state of the $\overline{\text{RES}}$ pin is sampled.



Note: * The dotted line indicates that P57/Ø is an input port or timer output port if the ØOE bit is 0, but a clock output pin if the ØOE bit is 1.

Figure E-3 Reset During Memory Access (Mode 3)

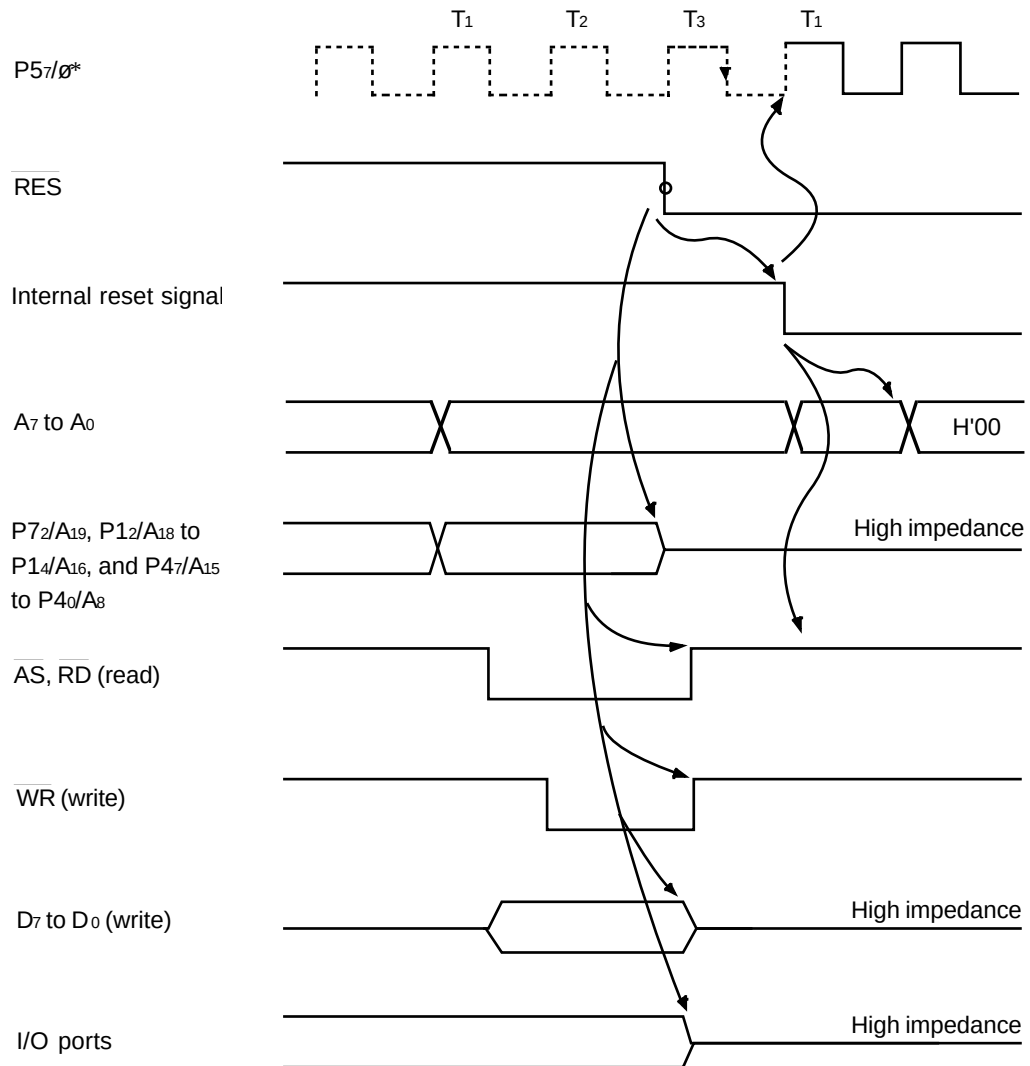
4. Mode 4

Figure E-4 shows how the pin states change when the RES pin goes low during external memory access in mode 4.

As soon as RES goes low, all ports are initialized to the input state. The AS, RD, and WR signals all go high. The data bus (D7 to D0) is placed in the high-impedance state. Pins P47/A15 to P40/A8 of the address bus and pins P72/A19 and P12/A18 to P14/A16 of the page address bus are initialized as input ports.

Pins A7 to A0 of the address bus are initialized to the low state 1.5 ϕ clock periods after the low state of the $\overline{\text{RES}}$ pin is sampled.

The clock output pin P57/ ϕ is initialized to the clock output state and begins clock output 0.5 ϕ clock periods after the low state of the $\overline{\text{RES}}$ pin is sampled.



Note: * The dotted line indicates that P57/ ϕ is an input port or timer output port if the ϕOE bit is 0, but a clock output pin if the ϕOE bit is 1.

Figure E-4 Reset During Memory Access (Mode 4)

5. Mode 7

Figure E-5 shows how the pin states change when the $\overline{\text{RES}}$ pin goes low in mode 7.

As soon as $\overline{\text{RES}}$ goes low, all ports are initialized to the input state.

The clock output pin P57/0 is also initialized to the input state.

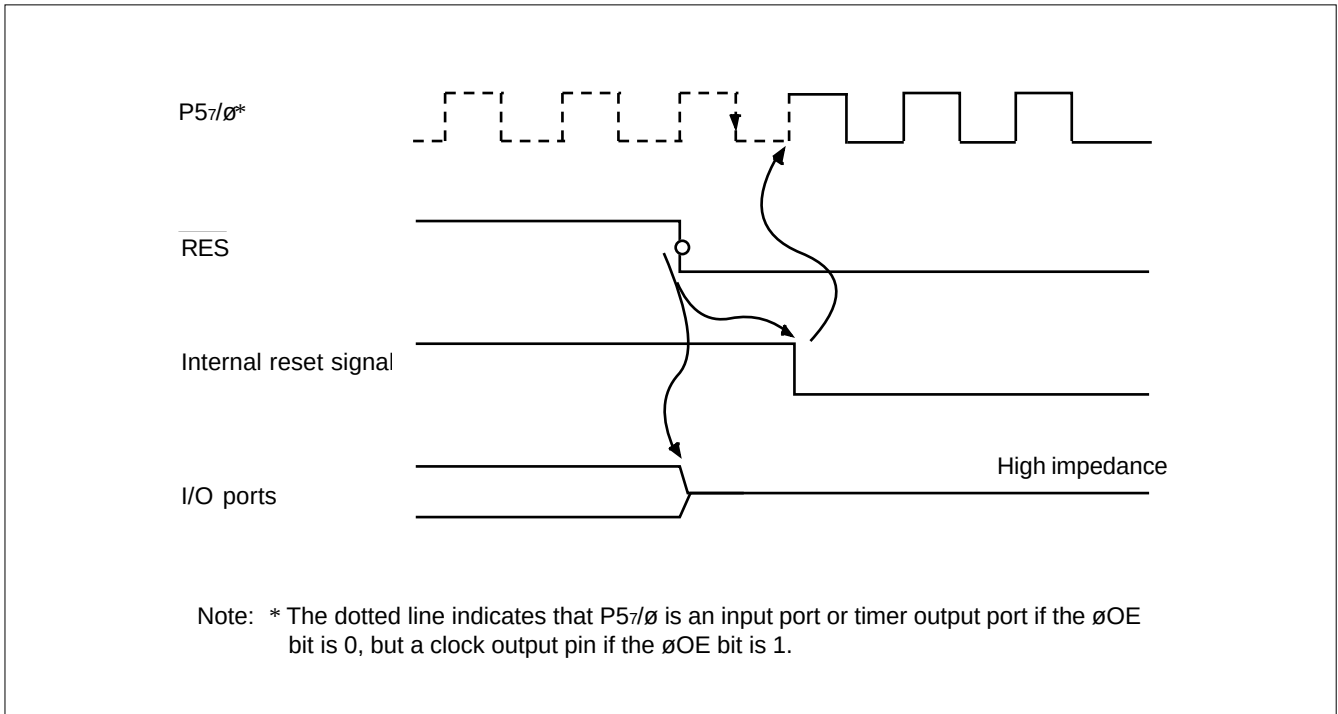


Figure E-5 Reset During Memory Access (Mode 7)

Appendix F Package Dimensions

Figure F-1 shows the dimensions of the DC-64S package. Figure F-2 shows the dimensions of the DP-64S package. Figure F-3 shows the dimensions of the FP-64A package. Figure F-4 shows the dimensions of the CP-68 package.

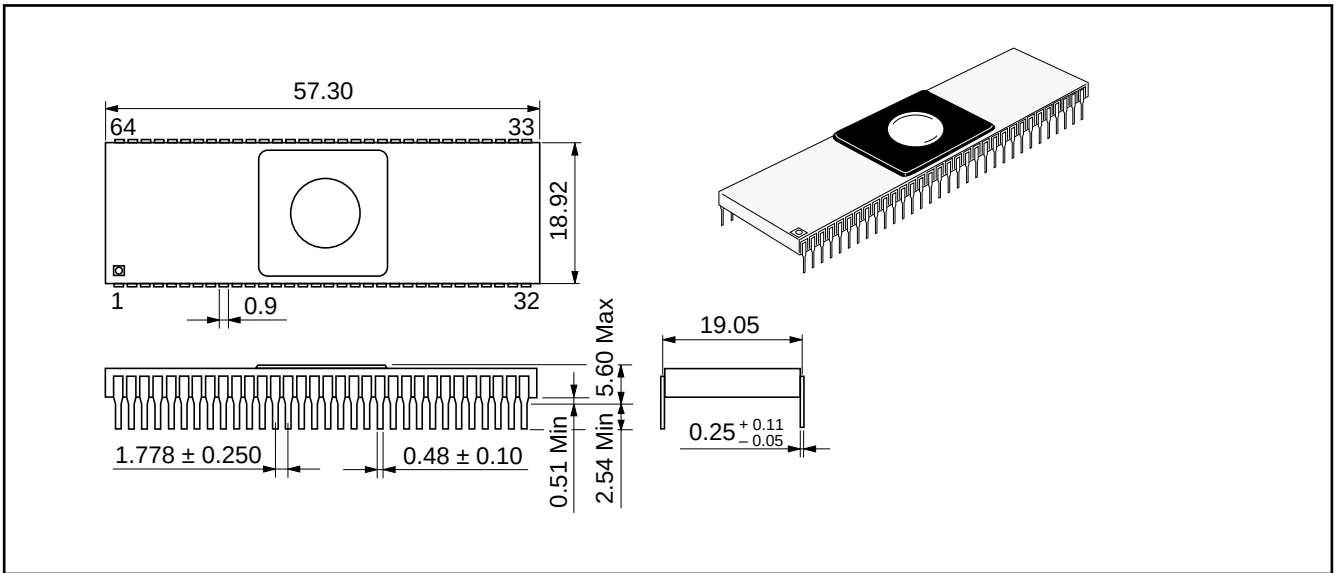


Figure F-1 Package Dimensions (DC-64S)

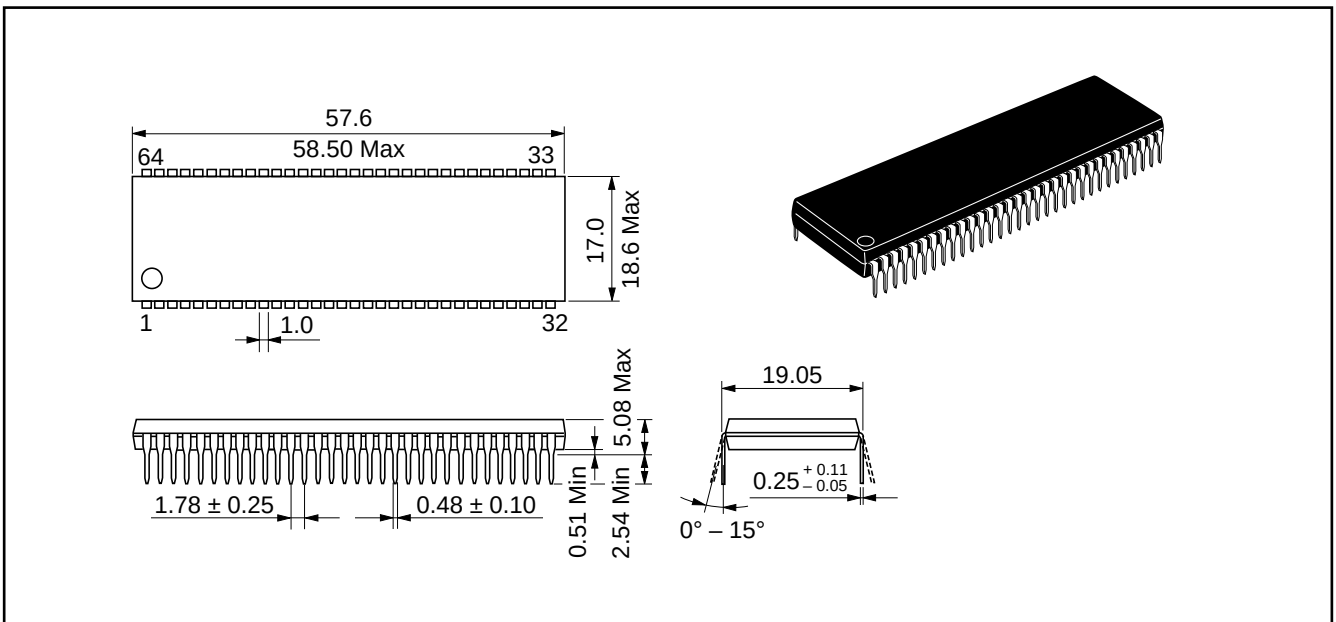


Figure F-2 Package Dimensions (DP-64S)

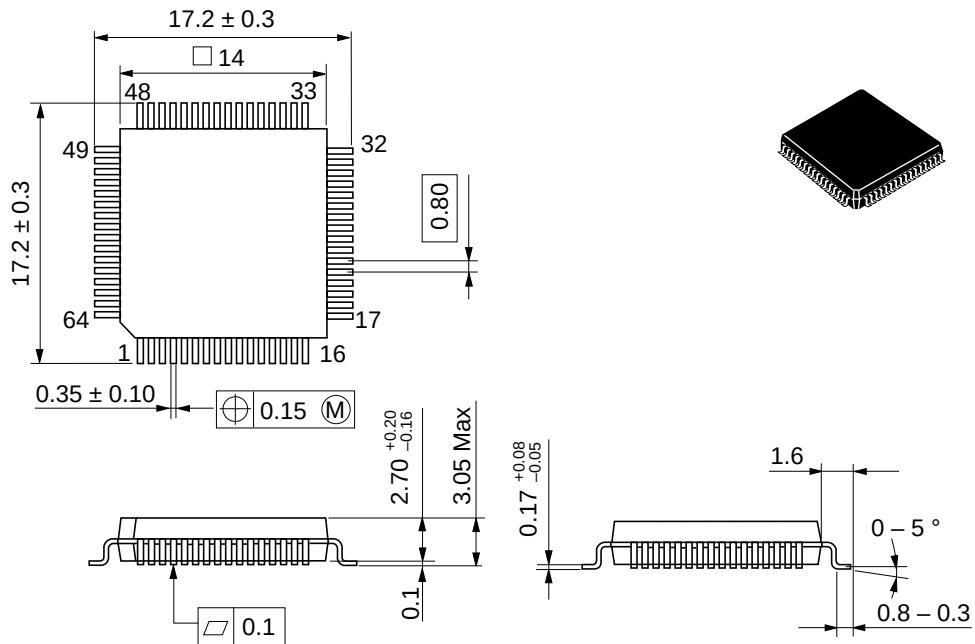


Figure F-3 Package Dimensions (FP-64A)

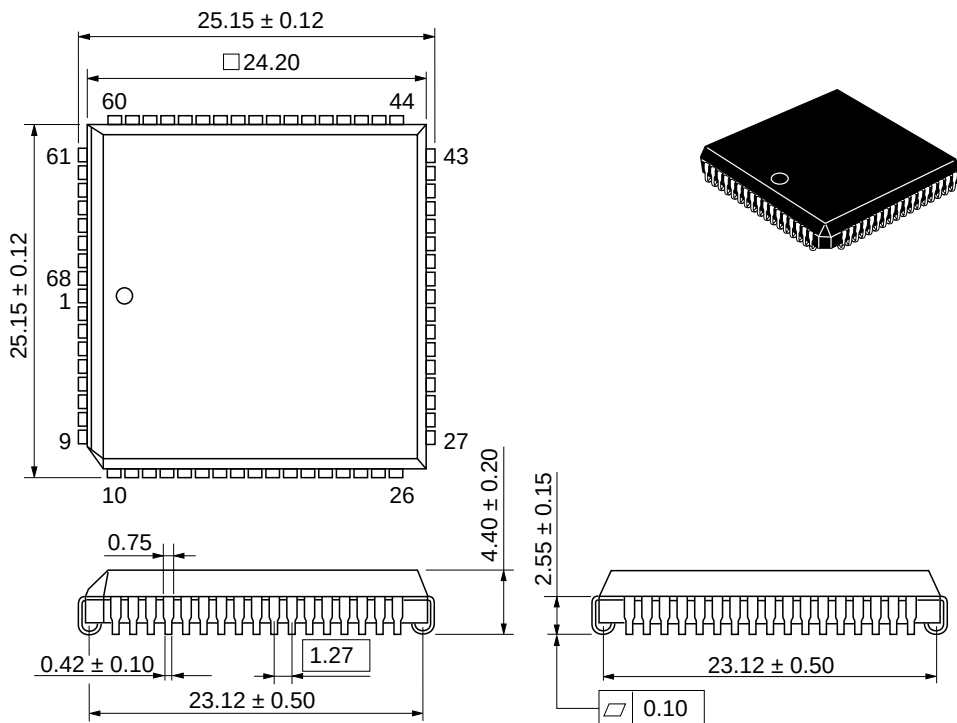


Figure F-4 Package Dimensions (CP-68)